

ARTIFICIAL INTELLIGENCE FOR AUTONOMOUS DRIVING AND AUTONOMOUS VEHICLES



KENNEDY CHENGETA, PhD

A RESEARCH MONOGRAPH

VOLUME I

Artificial Intelligence for Autonomous Driving and Autonomous Vehicles

*Foundations and the Core Stack —
Perception, Localization, Prediction, Planning, Control, Safety*

Kennedy Chengeta, PhD

First Edition

April 25, 2026

Artificial Intelligence for Autonomous Driving and Autonomous Vehicles

First Edition

© 2026 by Kennedy Chengeta.

All rights reserved.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without the prior written permission of the author, except for brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

The information in this book is presented in good faith for educational and research purposes. The author and publisher assume no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein. Trademarks referenced in the text are the property of their respective owners.

ISBN (print): 978-1-83294-617-2

ISBN (ebook): 978-1-83294-618-9

*To all researchers, engineers, and students who work to make mobility
safer, more inclusive, and more intelligent.*

And to those whose lives the technology must, above all, protect.

*“The test of intelligence is not the ability to avoid problems,
but the ability to solve them when avoidance fails.”*

—Kennedy Chengeta, PhD

Preface

The past decade has transformed autonomous driving from an academic curiosity into a measurable engineering discipline. What were, in 2010, demonstrations on closed courses have become commercial robotaxi services operating on public roads, handling millions of trips across multiple cities. The engine of this progress has been artificial intelligence—specifically, the convergence of deep learning, probabilistic robotics, and optimal control—deployed on sensor suites and compute platforms that would have been unimaginable a generation ago.

This book is a technical, integrative treatment of the field. It is aimed at graduate students, research engineers, and advanced practitioners who need a principled account of the full autonomy stack: how a self-driving system senses the world, estimates its own state, predicts what will happen next, decides what to do, and safely executes its decisions. The perspective is unapologetically mathematical. Concepts are introduced with the equations, algorithms, and probabilistic assumptions that make them verifiable and reproducible. Where empirical results drive design decisions—as in much of deep learning—we present the architectures and training objectives with enough precision to implement them.

Volume I is organized into eight parts. Part I establishes foundations: definitions, system architectures, and the mathematical and machine-learning primitives used throughout both volumes. Part II covers perception, from sensor physics through deep learning for images, detection, tracking, and segmentation. Part III addresses localization and mapping, including filter- and graph-based SLAM, high-definition maps, wheel odometry, and visual odometry. Part IV treats prediction, planning, and reinforcement learning—the decision layer. Part V covers low-level vehicle control, model predictive control in depth, and end-to-end learned policies. Part VI surveys specialized perception and deployment domains including pedestrian detection, lane detection, and rural and dust-dominated driving. Part VII develops advanced topics: digital twins, advanced prediction, uncertainty and explainability, advanced scene representations, frontier technologies, and ontologies. Part VIII concludes the technical core with safety and verification.

Each chapter includes self-contained derivations, reference algorithms, worked examples, and exercises. Appendices review notation, the major public datasets and benchmarks, and the linear algebra used throughout the text.

No book can substitute for the hands-on experience of debugging a stack at three in the morning as a rain-slicked cone disappears from the segmentation mask. But the goal here is to give you the

conceptual scaffolding to diagnose, improve, and—most importantly—trust or appropriately distrust the systems you build.

A note on the two-volume edition. This book is published in two volumes. **Volume I** (this volume) covers the technical core of building an autonomous vehicle: foundations, perception, localization and mapping, prediction and planning, vehicle control and end-to-end learning, specialized perception domains, advanced topics, and safety. **Volume II** covers the surrounding ecosystem: connectivity and infrastructure, cybersecurity and law, ethics and responsible AI, specialized vehicle classes, engineering at scale, evaluation and testing, economics, environment, and society. Some chapters in this volume reference chapters in Volume II by their Volume II chapter numbers; readers consulting only Volume I may treat such references as forward pointers to the companion volume.

Kennedy Chengeta

April 25, 2026

Acknowledgments

This book is, before anything else, the late fruit of an upbringing for which I owe more than I can repay. I thank my parents, who raised me with the conviction that education was the one inheritance no one could take away, and who worked hard so that their children could read books they themselves had not had the chance to read; this book is one of those. I thank my family, who made the long path through schooling feel like a shared project rather than a solitary one.

I thank my early teachers, who gave generously of their time to students whose futures were not yet visible, and I am one of those futures. Any errors that remain in this book are mine; the persistence to write through to the end is not.

Kennedy Chengeta

Johannesburg

About the Author

Kennedy Chengeta holds a PhD in Computer Science from the University of KwaZulu-Natal, South Africa, with a dissertation on computer vision for human characterization from facial images—covering feature extraction algorithms, facial behaviour analysis, and the recognition of micro-expressions in 2D, 3D, and 4D video for automatic pain, depression, and deception detection. He earned an MSc in Advanced Software Engineering *cum laude* from the University of Leicester in the United Kingdom, with research on Probability Latent Semantic Analysis, Singular Value Decomposition, and collaborative-filtering recommender algorithms; an MSc in Finance and Investments from the National University of Science and Technology in Zimbabwe; and a BBS with Computing Science from the University of Zimbabwe. He is currently completing an MSc in Self-Driving Cars Engineering at the University of Sligo and an MEng in Telecommunications Engineering at the University of Cape Town. His published work spans more than twenty peer-reviewed papers in computer vision, deep learning, and applied machine learning—including surveys of local binary and local directional patterns for facial recognition, deep-learning approaches to facial expression recognition, genetic-algorithm-based feature selection, image preprocessing with Canny and Kirsch edge detectors, convolutional neural networks for peer-to-peer social lending default prediction, and probability latent-semantic-analysis on unsupervised web-mined data. This breadth of academic work—from biometric computer vision to credit-risk deep learning to unsupervised semantic analysis—provides the technical foundation for the book’s integrative treatment of autonomous-vehicle perception, planning, and control.

Contents

Preface	iv
Acknowledgments	vi
About the Author	vii
List of Figures	xlvi
List of Tables	xlvii
List of Algorithms	xlviii
Notation	xlix
I Foundations	1
1 Introduction to Autonomous Vehicles	2
1.1 What is an Autonomous Vehicle?	2
1.1.1 Terminology and Related Concepts	3
1.1.2 Why Autonomous Driving is Hard	4
1.2 Levels of Driving Automation	4
1.2.1 The Levels in Detail	4
1.2.2 Operational Design Domain	5
1.2.3 Minimum Risk Conditions and Graceful Degradation	6
1.3 The Autonomy Stack	6
1.3.1 Rate Hierarchy and Latency Budget	7
1.3.2 Interfaces and Abstractions	7
1.4 Sensor and Compute Platform	8
1.5 Modular Versus End-to-End Paradigms	8
1.6 Use Cases and Deployment Contexts	9
1.6.1 Robotaxi	9
1.6.2 Consumer Highway Pilot (Level 2/3)	9
1.6.3 Long-Haul Trucking	9

1.6.4	Last-Mile Delivery	10
1.6.5	Shuttle and Fixed-Route Services	10
1.6.6	Automated Parking	10
1.6.7	Off-Road and Industrial Autonomy	10
1.7	Actors and the AV Ecosystem	10
1.8	A Brief History	11
1.8.1	Pre-history: Automation on Rails and Roads	11
1.8.2	1980s: Vision-Based Driving	11
1.8.3	1990s–2000s: DARPA and the Grand Challenges	11
1.8.4	2010s: Deep Learning and Commercialization	12
1.8.5	2020s: Scale, Scrutiny, and Foundations	12
1.9	Technical Challenges	13
1.9.1	The Long Tail of Rare Events	13
1.9.2	Observability	13
1.9.3	Real-Time Compute	13
1.9.4	Weather and Degraded Sensing	13
1.9.5	Multi-Agent Interaction	13
1.9.6	Data, Labels, and Lifecycle	14
1.9.7	Regulatory and Standards Landscape	14
1.9.8	Human Factors and Trust	14
1.10	Safety Metrics and the Role of Benchmarks	14
1.11	Scope and Organization of This Book	15
1.11.1	Intended Audience	15
1.11.2	Prerequisites and Notation	15
1.11.3	Structure	15
1.11.4	Reading Paths	15
1.11.5	What This Book Does Not Cover	15
1.12	Worked Examples	15
1.13	Industrial Deployment	16
1.14	Chapter Summary	17
1.15	Exercises	17
2	Mathematical and Machine Learning Foundations	19
2.1	Introduction	19
2.2	Rigid-Body Geometry	20
2.2.1	Rotations in Three Dimensions	21
2.2.2	Quaternions	23
2.2.3	Rigid Transforms	24
2.2.4	Perturbations and Jacobians on Lie Groups	24
2.2.5	The Baker–Campbell–Hausdorff Relation	25
2.2.6	Ground-Vehicle Simplifications	25
2.2.7	Frame Conventions	26

2.3	Probability and Estimation	26
2.3.1	Gaussian Distributions	26
2.3.2	Gaussian Mixture Models	29
2.3.3	The EM Algorithm	30
2.3.4	Bayesian State Estimation	30
2.3.5	Robust Estimation	33
2.3.6	Calibration and Reliability	35
2.3.7	Conformal Prediction	35
2.4	Optimization	36
2.4.1	Gradient Descent and its Variants	36
2.4.2	Newton’s Method, Line Search, and Trust Regions	37
2.4.3	Gauss–Newton and Levenberg–Marquardt	37
2.4.4	Constrained Optimization and KKT Conditions	39
2.4.5	Convex Relaxations and Quadratic Programming	39
2.4.6	Proximal and Splitting Methods	40
2.5	Deep Learning Essentials	40
2.5.1	Feedforward Networks and Backpropagation	41
2.5.2	Weight Initialization	41
2.5.3	Normalization Layers	42
2.5.4	Convolutional Networks	43
2.5.5	Residual Connections	44
2.5.6	Attention and Transformers	44
2.5.7	Training Stability	49
2.5.8	Regularization	49
2.6	Graphs and Graph Neural Networks	49
2.6.1	Message Passing	50
2.6.2	Attention on Graphs	51
2.7	Uncertainty Quantification	51
2.7.1	Bayesian Neural Networks and MC Dropout	51
2.7.2	Deep Ensembles	51
2.7.3	Evidential Deep Learning	52
2.7.4	Calibration in Practice	52
2.8	Information-Theoretic Quantities	52
2.8.1	Evidence Lower Bound (ELBO)	52
2.8.2	Fisher Information and Natural Gradient	53
2.9	The Learning Setups We Use	53
2.10	Computational Considerations	53
2.10.1	Mixed Precision	54
2.10.2	Quantization	54
2.10.3	Pruning and Distillation	54
2.10.4	Compilation and Graph Optimization	54
2.11	Tooling and Reproducibility	54

2.12	Worked Examples	55
2.13	Industrial Deployment	57
2.14	Chapter Summary	58
2.15	Exercises	58
II	Perception	60
3	Autonomous Perception	61
3.1	Introduction	61
3.2	The Role of Perception in the Autonomy Stack	62
3.2.1	What Perception Is Responsible For	62
3.2.2	What Perception Is Not Responsible For	63
3.3	The Canonical Perception Stack	63
3.4	Levels of Autonomy	64
3.5	Key Challenges in Real-World Perception	65
3.5.1	Long-Tail and Rare Events	65
3.5.2	Distribution Shift	66
3.5.3	Sensor Failures and Adversarial Conditions	66
3.5.4	Uncertainty and Calibration	67
3.5.5	Latency and Compute	67
3.6	How This Book Organizes Perception	67
3.7	Worked Examples	68
3.8	Industrial Deployment	69
3.9	Chapter Summary	69
3.10	Exercises	69
4	Deep Learning for Computer Vision	71
4.1	Introduction	71
4.2	Convolutional Backbones	72
4.2.1	From LeNet to ResNet	72
4.2.2	Batch Normalization and Alternatives	73
4.2.3	Efficient Backbones for On-Vehicle Deployment	74
4.3	Vision Transformers	76
4.4	Monocular Depth Estimation	77
4.4.1	Supervised Depth	77
4.4.2	Self-Supervised Depth from Stereo and Video	78
4.5	Optical Flow	78
4.6	Semantic Representations and Pretraining	79
4.6.1	Supervised Pretraining	79
4.6.2	Self-Supervised Pretraining	80
4.6.3	Vision-Language Pretraining	80
4.7	Task Heads for Driving	80

4.8	Bird's-Eye-View Representations	81
4.8.1	Lift-Splat-Shoot	82
4.8.2	Temporal BEV	82
4.9	Data Augmentation for Driving	83
4.10	Evaluation Metrics	83
4.11	Interpretability and Debugging	83
4.12	Worked Examples	84
4.13	Industrial Deployment	85
4.14	Chapter Summary	85
4.15	Exercises	85
5	LiDAR Perception	87
5.1	Introduction	87
5.2	LiDAR Hardware	88
5.2.1	Pulsed Time-of-Flight LiDAR	88
5.2.2	FMCW LiDAR	89
5.2.3	Mechanical Spinning LiDAR	89
5.2.4	MEMS-Mirror LiDAR	90
5.2.5	Optical Phased Array (True Solid State)	90
5.2.6	Flash LiDAR	90
5.3	From Photons to Points: Waveform Processing	90
5.3.1	Peak Detection and Pulse Shaping	90
5.3.2	Multi-Return Processing	91
5.3.3	Intensity Calibration	91
5.4	Point Cloud Representations	91
5.4.1	Raw Point Set	92
5.4.2	Range Image	92
5.4.3	Voxel Grid	92
5.4.4	BEV Pillars	93
5.4.5	Tradeoffs	93
5.5	Calibration and Motion Compensation	93
5.5.1	Extrinsic Calibration to the Body Frame	93
5.5.2	Motion Undistortion	93
5.6	Deep Learning on Point Clouds	94
5.6.1	PointNet and PointNet++	94
5.6.2	Voxel-Based Methods	95
5.6.3	Range-Image Methods	95
5.6.4	BEV and Pillar Methods	95
5.6.5	Transformer-Based Approaches	95
5.7	3D Object Detection	95
5.7.1	The Output Parameterization	96
5.7.2	Loss Functions	96

5.7.3	Handling Imbalance	96
5.7.4	Non-Maximum Suppression in 3D	97
5.7.5	Evaluation Metrics	97
5.8	Semantic Segmentation	97
5.9	LiDAR Odometry and Scan Matching	98
5.9.1	Iterative Closest Point	98
5.9.2	Feature-Based Methods	98
5.10	Weather and Adverse Conditions	98
5.11	Simulation and Sim-to-Real	99
5.12	Datasets and Benchmarks	99
5.13	Worked Examples	99
5.14	Industrial Deployment	100
5.15	Chapter Summary	101
5.16	Exercises	101
6	Radar and Sonar Perception	103
6.1	Introduction	103
6.2	FMCW Radar Fundamentals	104
6.2.1	The Chirp Waveform	104
6.2.2	Range Resolution and Maximum Range	105
6.2.3	Velocity and Doppler	105
6.2.4	Range-Velocity Coupling	106
6.3	Angle Estimation and MIMO Arrays	106
6.3.1	Phased-Array Basics	106
6.3.2	MIMO Virtual Arrays	107
6.3.3	Super-resolution Methods	108
6.4	Target Detection	108
6.4.1	Constant False Alarm Rate (CFAR)	108
6.4.2	Peak Extraction and Clustering	109
6.5	Radar-Specific Tracking	109
6.5.1	Doppler-Gated Association	109
6.5.2	Angle Uncertainty Handling	109
6.5.3	Micro-Doppler for Classification	110
6.6	4D Imaging Radar	110
6.7	Automotive Ultrasonic Sensors	111
6.8	Radar and Ultrasonic Fusion Architectures	111
6.8.1	Radar-Camera Fusion	111
6.8.2	Radar-LiDAR Fusion	112
6.9	Deep Learning on Radar	112
6.9.1	Deep Learning on the Range-Doppler Map	113
6.9.2	Deep Learning on Range-Doppler-Angle Tensors	113
6.9.3	Deep Learning for Imaging-Radar Point Clouds	113

6.9.4	Micro-Doppler Classifiers	113
6.10	Datasets and Benchmarks	113
6.11	Worked Examples	113
6.12	Industrial Deployment	114
6.13	Chapter Summary	115
6.14	Exercises	115
7	GNSS, RTK, and Inertial Sensing	117
7.1	Introduction	117
7.2	GNSS Fundamentals	118
7.2.1	Trilateration and the Pseudorange Equation	119
7.2.2	Satellite Constellations	120
7.2.3	Error Sources	120
7.2.4	Dilution of Precision	121
7.2.5	Velocity and Time from GNSS	121
7.3	RTK and Differential GPS	121
7.3.1	The Key Insight: Shared Errors	121
7.3.2	Differential GPS (DGPS)	122
7.3.3	Real-Time Kinematic (RTK)	122
7.3.4	Precise Point Positioning	123
7.3.5	AV Use Cases for Centimeter Positioning	123
7.4	Inertial Measurement Units	123
7.4.1	Components	123
7.4.2	IMU Grades	124
7.4.3	Error Model	124
7.4.4	Allan Variance Characterization	124
7.4.5	Strapdown Mechanization	125
7.4.6	Dead Reckoning and Drift	126
7.5	GNSS/IMU Fusion	126
7.5.1	Loose vs Tight Coupling	126
7.5.2	State Representation	126
7.5.3	Initialization	127
7.5.4	Outlier Rejection	127
7.6	Worked Examples	127
7.7	Industrial Deployment	128
7.8	Chapter Summary	128
7.9	Exercises	129
8	Thermal and Infrared Imaging	130
8.1	Introduction	130
8.2	Physics of Thermal Radiation	131
8.2.1	The Infrared Bands	131

8.3	Thermal Sensor Hardware	132
8.3.1	Uncooled Microbolometers	132
8.3.2	Cooled Detectors	132
8.3.3	Pixel Array Geometry and Optics	133
8.4	Image Calibration	133
8.4.1	Non-Uniformity Correction (NUC)	133
8.4.2	Dead-Pixel Replacement	133
8.4.3	Vignetting and Lens Thermal Artifacts	133
8.5	Pedestrian and Animal Detection	133
8.5.1	Thermal-Specific Training Data	134
8.5.2	Class-Specific Features	134
8.5.3	Animal Detection	134
8.6	Fusion with Visible-Spectrum Cameras	134
8.7	Production Deployments	135
8.8	Worked Examples	135
8.9	Industrial Deployment	136
8.10	Chapter Summary	136
8.11	Exercises	136
9	Environmental Sensors	138
9.1	Introduction	138
9.2	Rain Sensors	139
9.2.1	Optical Rain Sensors	139
9.2.2	Capacitive Rain Sensors	140
9.2.3	Role in the AV Stack	140
9.3	Light Sensors	140
9.3.1	Integration with AV Perception	140
9.4	Temperature Sensors	141
9.4.1	Ambient Air Temperature	141
9.4.2	Road-Surface Temperature	141
9.4.3	Self-Heating and Placement	141
9.5	Humidity Sensors	141
9.6	Integration: The Environmental Sensor Bus	142
9.7	Worked Examples	142
9.8	Industrial Deployment	143
9.9	Chapter Summary	143
9.10	Exercises	144
10	Sensor Technologies and Sensor Fusion	145
10.1	Introduction	145
10.2	Cameras	146
10.2.1	Pinhole Model and Calibration	146

10.2.2	Global Versus Rolling Shutter	147
10.2.3	Photometric Noise and Dynamic Range	148
10.3	LiDAR	149
10.3.1	Principle of Operation	149
10.3.2	Point Cloud Structure	150
10.3.3	Range Image Representation	150
10.3.4	Weather and Reflectivity	150
10.4	Radar	151
10.5	GNSS and Inertial Sensing	152
10.5.1	GNSS Basics	152
10.5.2	Inertial Measurement Units	153
10.6	Sensor Extrinsic Calibration	154
10.7	Time Synchronization	154
10.8	Sensor Fusion: The Kalman Filter	155
10.8.1	Derivation	155
10.8.2	Extended and Unscented Variants	156
10.8.3	Information Filter and Factor Graphs	156
10.9	Multi-Sensor Fusion Architectures	156
10.10	Sensor Comparison and Selection	158
10.11	Worked Examples	158
10.12	Industrial Deployment	160
10.13	Chapter Summary	160
10.14	Exercises	160
11	Object Detection and Multi-Object Tracking	162
11.1	Introduction	162
11.2	Two-Dimensional Object Detection	163
11.2.1	Problem Statement	163
11.2.2	Two-Stage Detectors	164
11.2.3	Single-Stage Detectors	165
11.2.4	Anchor-Free Detectors	165
11.2.5	Transformer-Based Detection: DETR	165
11.3	Non-Maximum Suppression	166
11.4	Three-Dimensional Object Detection	166
11.4.1	LiDAR-Based 3D Detection	167
11.4.2	Camera-Based 3D Detection	167
11.4.3	LiDAR–Camera Fusion	167
11.4.4	3D Evaluation Metrics	168
11.5	Multi-Object Tracking	168
11.5.1	Tracking by Detection	168
11.5.2	State Model for Driving	169
11.5.3	Joint Detection and Tracking	170

11.5.4 Tracking Evaluation	170
11.6 Hard Cases: Long-Range, Small-Object, and Adverse-Weather Detection	171
11.6.1 Small and Distant Objects	171
11.6.2 Adverse Weather	171
11.7 Open-World and Continual Learning	172
11.8 Worked Examples	172
11.9 Industrial Deployment	173
11.10 Chapter Summary	173
11.11 Exercises	173
12 Semantic, Instance, and Panoptic Segmentation	175
12.1 Introduction	175
12.2 Problem Taxonomy	176
12.3 Image Semantic Segmentation	176
12.3.1 Fully Convolutional Networks	177
12.3.2 Dilated Convolutions and DeepLab	177
12.3.3 Transformer-Based Segmentation	178
12.3.4 Losses	178
12.3.5 Lane and Drivable-Area Segmentation	179
12.4 Instance and Panoptic Segmentation	179
12.4.1 Instance: Mask R-CNN	179
12.4.2 Panoptic Segmentation	180
12.5 LiDAR Point-Cloud Segmentation	180
12.5.1 PointNet and PointNet++	180
12.5.2 Voxel and Range-Image Methods	181
12.5.3 Scene-Level Transformer Methods	181
12.5.4 Panoptic LiDAR Segmentation	181
12.6 Segmentation in BEV Space	181
12.7 Scaling: Labels, Annotation, and Self-Training	182
12.7.1 Self-Training and Pseudo-Labels	182
12.7.2 Weakly-Supervised Segmentation	182
12.7.3 Synthetic and Simulated Data	182
12.7.4 Foundation-Model Annotation	183
12.8 Uncertainty in Segmentation	183
12.9 Case Study: A Production Panoptic Segmenter	183
12.10 Worked Examples	184
12.11 Industrial Deployment	185
12.12 Chapter Summary	185
12.13 Exercises	185

III	Localization and Mapping	187
13	Simultaneous Localization and Mapping	188
13.1	Introduction	188
13.2	Problem Formulation	189
13.3	Factor-Graph SLAM	190
13.3.1	Factor Graphs	191
13.3.2	Solving by Gauss–Newton	191
13.3.3	Incremental Smoothing	191
13.4	Visual SLAM and Visual-Inertial Odometry	192
13.4.1	Feature-Based Methods	192
13.4.2	Direct Methods	193
13.4.3	Visual-Inertial Fusion	193
13.5	LiDAR SLAM	194
13.5.1	Scan Registration: ICP and Variants	194
13.5.2	LOAM and its Descendants	194
13.5.3	Loop Closure	195
13.6	Learned SLAM Components	195
13.7	Uncertainty and Consistency	195
13.8	What’s Different About Driving SLAM	196
13.9	Worked Examples	196
13.10	Industrial Deployment	197
13.11	Chapter Summary	197
13.12	Exercises	197
14	High-Definition Maps and Map-Based Localization	199
14.1	Introduction	199
14.2	Contents of an HD Map	200
14.3	Map Construction Pipelines	201
14.3.1	Survey Acquisition	202
14.3.2	Global Alignment	202
14.3.3	Automated Semantic Extraction	202
14.3.4	Human Validation	202
14.3.5	Crowd-Sourced Updates	202
14.4	Map-Based Localization	203
14.4.1	LiDAR Point-Cloud Matching	203
14.4.2	Normal Distributions Transform	204
14.4.3	Semantic-Landmark Matching	204
14.4.4	Visual Localization	204
14.4.5	Fusion with GNSS/IMU	204
14.5	Mapless and Map-Light Driving	205
14.6	Map Versioning and Freshness	205

14.7	Privacy and Regulation	206
14.8	Worked Examples	206
14.9	Industrial Deployment	207
14.10	Chapter Summary	207
14.11	Exercises	207
15	Wheel Odometry	209
15.1	Introduction	209
15.2	The Physics of Wheel Rotation	210
15.2.1	Rolling Without Slipping	210
15.2.2	Effective Rolling Radius	210
15.3	Encoder Hardware	211
15.3.1	Optical Encoders	211
15.3.2	Magnetic Hall-Effect Encoders	211
15.3.3	Resolvers	211
15.3.4	ABS Wheel-Speed Sensors	211
15.4	Kinematic Models	212
15.4.1	Differential-Drive Model	212
15.4.2	Ackermann Steering Model	212
15.4.3	Four-Wheel Independent	213
15.5	Error Sources	213
15.5.1	Systematic Errors	213
15.5.2	Stochastic Errors	213
15.5.3	Drift Growth Model	214
15.6	Calibration	214
15.6.1	Static Calibration	214
15.6.2	Trajectory-Based Calibration	214
15.6.3	Online Calibration via GNSS Fusion	215
15.7	Dead Reckoning	215
15.7.1	Position Integration	215
15.7.2	Uncertainty Propagation	215
15.8	Fusion with Other Sensors	216
15.8.1	Wheel-IMU Fusion	216
15.8.2	Wheel-GNSS Fusion	216
15.8.3	Wheel-SLAM Fusion	216
15.9	Specific Production Usages	216
15.9.1	Low-Level Control	217
15.9.2	State Estimator	217
15.9.3	SLAM Backend	217
15.10	Failure Modes in Production	217
15.10.1	Stuck Wheel Due to Ice or Mud	217
15.10.2	Wheel Lockup Under Hard Braking	217

15.10.3 Encoder Failure	217
15.10.4 Sustained Slip on Loose Surfaces	217
15.11 Industrial Deployment	218
15.12 Chapter Summary	218
15.13 Exercises	218
16 Visual Odometry	220
16.1 Introduction	220
16.2 Visual Odometry: Problem Formulation	221
16.2.1 The Core Geometric Relationship	221
16.2.2 Why Visual Odometry Works at All	221
16.2.3 Scale Ambiguity in Monocular VO	222
16.3 Feature-Based Visual Odometry	222
16.3.1 Feature Detection	222
16.3.2 Matching	223
16.3.3 Initial Pose Estimation	223
16.3.4 Local Bundle Adjustment	224
16.4 Direct Visual Odometry	224
16.4.1 Photometric Error	224
16.4.2 DSO and LSD-SLAM	224
16.5 Stereo and RGB-D Visual Odometry	225
16.5.1 Stereo VO	225
16.5.2 RGB-D VO	225
16.6 Visual-Inertial Odometry	225
16.6.1 IMU Preintegration	225
16.6.2 Joint Optimization	226
16.6.3 Initialization	226
16.7 Weather Robustness	226
16.7.1 Rain and Wet Surfaces	226
16.7.2 Snow	227
16.7.3 Fog	227
16.7.4 Low Light and Night	227
16.8 GPS-Denied Environments	227
16.8.1 Tunnels	227
16.8.2 Parking Garages	228
16.8.3 Urban Canyons	228
16.8.4 Mine and Construction Sites	228
16.9 Learned Visual Odometry	228
16.9.1 End-to-End Approaches	228
16.9.2 Hybrid Architectures	228
16.9.3 Self-Supervised Depth and Motion	229
16.10 Production VO in the AV Stack	229

16.10.1 As SLAM's Front End	229
16.10.2 As a GPS Backup	229
16.10.3 For Redundancy and Cross-Check	229
16.11 Industrial Deployment	229
16.12 Chapter Summary	230
16.13 Exercises	230
IV Prediction and Planning	232
17 Motion Prediction	233
17.1 Introduction	233
17.2 Problem Statement	234
17.3 Physics-Based Baselines	234
17.3.1 Constant Velocity and Constant Turn Rate	235
17.3.2 Kinematic Bicycle	235
17.4 Map-Aware Prediction	235
17.4.1 Lane Anchors	236
17.4.2 Vectorized Map Encoding	236
17.4.3 Production Architectures	236
17.5 Interaction Modeling	238
17.6 Joint vs. Marginal Prediction	238
17.7 Output Parameterizations	239
17.7.1 Mixture of Gaussians	239
17.7.2 Trajectory Sets	239
17.7.3 Diffusion-Based	239
17.8 Pedestrian and Cyclist Prediction	239
17.9 Training Objectives	240
17.9.1 Negative Log-Likelihood	240
17.9.2 Winner-Takes-All	240
17.9.3 Behavior-Cloning Regularizers	240
17.10 Evaluation Metrics	240
17.11 Latency and Compute Budgets	241
17.12 Worked Examples	242
17.13 Industrial Deployment	242
17.14 Chapter Summary	243
17.15 Exercises	243
18 Motion Planning	244
18.1 Introduction	244
18.2 The Planning Hierarchy	245
18.3 Route Planning	245
18.4 Behavior Planning	246

18.4.1	Finite-State Machines	246
18.4.2	MDPs and POMDPs	246
18.5	Motion Planning: Problem Formulation	247
18.6	Sampling-Based Planning	247
18.6.1	RRT and RRT*	247
18.6.2	Kinodynamic Planners	247
18.7	Lattice and Graph-Search Planners	247
18.8	Optimization-Based Planning	248
18.8.1	Frenet Frame Planning	248
18.8.2	QP-Based Trajectory Optimization	248
18.8.3	Model Predictive Control (MPC)	248
18.9	Collision Checking	249
18.9.1	Dynamic Obstacles	249
18.10	Interaction-Aware Planning	250
18.11	Comfort	250
18.12	Planner Output and Handoff to Control	250
18.13	Worked Examples	250
18.14	Industrial Deployment	251
18.15	Chapter Summary	251
18.16	Exercises	252
19	Reinforcement Learning for Driving	253
19.1	Introduction	253
19.2	Markov Decision Processes	254
19.3	Algorithmic Families	255
19.3.1	Value-Based: DQN	255
19.3.2	Policy Gradient: REINFORCE to PPO	255
19.3.3	Actor-Critic: SAC	256
19.3.4	Model-Based RL	256
19.4	Designing State and Action Spaces for Driving	256
19.4.1	State Representations	256
19.4.2	Action Representations	257
19.5	Reward Design	257
19.6	Simulators for Driving RL	257
19.6.1	Sim-to-Real	258
19.7	Offline Reinforcement Learning	258
19.8	Safe RL	258
19.9	Applications in the AV Stack	259
19.10	Worked Examples	259
19.11	Industrial Deployment	260
19.12	Chapter Summary	260
19.13	Exercises	260

V	Vehicle Control and End-to-End Learning	262
20	Vehicle Control	263
20.1	Introduction	263
20.2	Vehicle Dynamics	264
20.2.1	Kinematic Bicycle Model	264
20.2.2	Dynamic Bicycle Model	265
20.2.3	Tire Models	265
20.3	Longitudinal Control	266
20.3.1	PID with Feedforward	266
20.3.2	Adaptive Cruise Control	266
20.4	Lateral Control	266
20.4.1	Pure Pursuit	266
20.4.2	Stanley Controller	267
20.4.3	LQR for Path Tracking	267
20.5	Model Predictive Control	267
20.5.1	Linear MPC as QP	268
20.5.2	Nonlinear MPC	268
20.5.3	Terminal Cost and Stability	268
20.6	Delay, Disturbance, and Robustness	268
20.6.1	Actuator Delay	268
20.6.2	Model Mismatch	269
20.6.3	Road Banking and Grade	269
20.7	Handover with the Planner	269
20.8	Validation Beyond Tracking Error	269
20.9	Worked Examples	270
20.10	Industrial Deployment	270
20.11	Chapter Summary	271
20.12	Exercises	271
21	Model Predictive Control in Depth	272
21.1	Introduction	272
21.2	The MPC Formulation	273
21.2.1	The General Problem	273
21.2.2	Stage Cost Choices	274
21.3	Linear vs Nonlinear MPC	274
21.3.1	Linear MPC (LMPC) and QP Form	274
21.3.2	Nonlinear MPC (NMPC)	274
21.4	Solving MPC in Real Time	275
21.4.1	Code Generation	275
21.4.2	Warm Starting	275
21.4.3	Real-Time Iteration (RTI)	275

21.5	Open-Source MPC Toolchains	276
21.5.1	do-MPC (TU Dortmund)	276
21.5.2	Acados (University of Freiburg)	276
21.5.3	Other Production-Relevant Toolchains	277
21.6	MPC for Autonomous Driving	277
21.6.1	Trajectory Tracking	277
21.6.2	Joint Planning + Control	277
21.6.3	Distributed MPC for Platoons	278
21.7	Stochastic and Robust MPC	278
21.7.1	Robust MPC	278
21.7.2	Stochastic MPC	278
21.7.3	Learning-Based MPC	278
21.8	Industrial Deployment	278
21.9	Chapter Summary	279
21.10	Exercises	279
22	End-to-End Learning for Autonomous Driving	280
22.1	Introduction	280
22.2	The End-to-End Spectrum	281
22.3	Behavior Cloning: The Core	282
22.3.1	The Covariate Shift Problem	283
22.3.2	DAgger and Its Descendants	283
22.3.3	Augmentation for Off-Distribution States	283
22.4	Conditional Imitation Learning	283
22.5	Foundation Models for Driving	283
22.6	Causal Confusion and Related Pathologies	284
22.6.1	Causal Confusion	284
22.6.2	Shortcut Learning	285
22.7	Hybrid Architectures	285
22.8	Training Data at Scale	285
22.9	Evaluation Methodology	286
22.10	Worked Examples	286
22.11	Foundational and Open-Source Implementations	287
22.11.1	DAVE-2 (NVIDIA, 2016)	287
22.11.2	Comma.ai's openpilot	290
22.12	Industrial Deployment	292
22.13	Chapter Summary	293
22.14	Exercises	293
VI	Specialized Perception and Deployment Domains	295
23	Pedestrian Perception, Intent, and Interaction	296

23.1	Introduction	296
23.2	Vulnerable Road Users: Taxonomy	297
23.2.1	Class Structure	297
23.3	Pedestrian-Specific Detection Challenges	298
23.3.1	Small Apparent Size at Range	298
23.3.2	Occlusion and Partial Visibility	298
23.3.3	Unusual Poses and Attire	299
23.3.4	Motion Blur and Viewpoint	299
23.4	Sensor Modality Choices	299
23.4.1	Camera	299
23.4.2	LiDAR	299
23.4.3	Radar	300
23.4.4	Thermal	300
23.5	Pedestrian Detection Architectures	300
23.5.1	Dedicated Pedestrian Heads	300
23.5.2	Small-Object-Optimized Architectures	300
23.5.3	Multi-Modal Fusion	300
23.6	Pose and Action Recognition	301
23.6.1	Skeletal Keypoints	301
23.6.2	Action Classification	301
23.6.3	Gaze Direction	301
23.7	Intent Prediction	302
23.7.1	Features for Intent	302
23.7.2	Temporal Modeling	302
23.7.3	Calibration of Probabilities	303
23.8	Scenario-Specific Challenges	303
23.8.1	Controlled Crosswalks	303
23.8.2	Uncontrolled Crossings (Jaywalking)	304
23.8.3	School Zones	304
23.8.4	Dense Crowds	304
23.9	Demographic Equity	304
23.9.1	Measurement Protocol	305
23.9.2	Closing the Gaps	305
23.10	Integration with the Planner	305
23.10.1	Data Passed	305
23.10.2	Planning-Layer Usage	305
23.11	Industrial Deployment	306
23.12	Chapter Summary	307
23.13	Exercises	307
24	Lane Detection	308
24.1	Introduction	308

24.2	Lane Taxonomy and Geometry	309
24.2.1	Lane Line Types	309
24.2.2	Coordinate Systems	309
24.3	Classical Lane Detection	310
24.3.1	Edge Detection + Hough Transform	311
24.3.2	Polynomial Fit + RANSAC	311
24.3.3	Limitations of Classical Methods	312
24.4	Learning-Based Lane Detection	312
24.4.1	Lane Segmentation Approaches	313
24.4.2	Lane Proposal Methods	313
24.4.3	3D and BEV Methods	313
24.4.4	Temporal Fusion	313
24.5	Evaluation Metrics	314
24.5.1	Classification-Style Metrics	314
24.5.2	Geometric Metrics	315
24.5.3	Benchmarks	315
24.6	Production Considerations	316
24.6.1	Integration with HD Maps	316
24.6.2	Handling Absent Lanes	316
24.6.3	Weather-Robustness	316
24.7	Industrial Deployment	317
24.8	Chapter Summary	317
24.9	Exercises	318
25	Rural and Dust-Dominated Autonomous Travel	319
25.1	Introduction	319
25.2	The Rural Operating Domain	320
25.2.1	Road Surface Types	320
25.2.2	Terrain Characteristics	321
25.2.3	Climate Extremes	321
25.3	Dust as a Perception Adversary	321
25.3.1	Physical Basis	321
25.3.2	Dust Classification	323
25.4	Mixed Traffic and Road Users	323
25.4.1	Non-Motorized Traffic	323
25.4.2	Livestock and Wildlife	324
25.4.3	Informal Road Users	324
25.5	Sparse Infrastructure	324
25.5.1	HD Maps	325
25.5.2	GNSS Coverage	325
25.5.3	Cellular Connectivity	325
25.6	Sensor-Suite Adaptations	326

25.6.1	Radar Prioritization	326
25.6.2	Thermal Imaging Inclusion	326
25.6.3	LiDAR as Short-Range Supplement	326
25.6.4	Robust IMU and Wheel Odometry	326
25.6.5	Sensor Cleaning	326
25.7	Control Adaptations	327
25.7.1	Traction Estimation	327
25.7.2	Suspension-Aware Planning	328
25.7.3	Low-Speed Robustness	328
25.8	Production Deployments	328
25.8.1	Mining	328
25.8.2	Agricultural Autonomy	328
25.8.3	Rural Delivery	328
25.8.4	Defense and Specialized Autonomy	328
25.9	Industrial Deployment	329
25.10	Chapter Summary	329
25.11	Exercises	330

VII Advanced Topics and Frontier Directions 331

26	Digital Twins and Scene Intelligence	332
26.1	Introduction	332
26.2	3D Scene Reconstruction	333
26.2.1	Classical Multi-View Stereo	333
26.2.2	LiDAR-Based Reconstruction	333
26.2.3	Learned Depth from Single or Stereo Views	333
26.2.4	Neural Radiance Fields (NeRF)	334
26.3	Occupancy Networks	335
26.3.1	Formulation	335
26.3.2	BEV Occupancy	335
26.3.3	3D Voxel Occupancy	336
26.3.4	Occupancy Prediction	336
26.3.5	Evaluation Metrics	336
26.4	Panoptic Segmentation	337
26.4.1	Stuff and Things	337
26.4.2	Architectures	337
26.4.3	Panoptic Quality Metric	337
26.5	Diffusion Models for Perception	338
26.5.1	Sensor Completion	338
26.5.2	Data Augmentation and Scenario Generation	338
26.5.3	Perception as Denoising	338
26.6	Digital Twin Ecosystems	339

26.6.1	City-Scale Twins	339
26.6.2	Value Proposition	339
26.6.3	Twin Fidelity and Validation	339
26.7	Integration into the Production Stack	340
26.7.1	Runtime Scene Intelligence	340
26.7.2	Offline Scene Intelligence	340
26.7.3	Digital Twin Feedback	340
26.8	Industrial Deployment	340
26.9	Chapter Summary	341
26.10	Exercises	341
27	Advanced Prediction and Multi-Agent Behavior	343
27.1	Introduction	343
27.2	Advanced Trajectory Prediction	344
27.2.1	Transformer-Based Predictors	344
27.2.2	Diffusion-Based Prediction	344
27.3	Behavior Modeling and Intent	345
27.3.1	Intent Representations	345
27.3.2	Intent Prediction Architectures	345
27.4	Multi-Agent Interaction via Graph Neural Networks	346
27.4.1	Interaction Graph Construction	346
27.4.2	Graph Attention and Message Passing	347
27.4.3	Scene-Level Reasoning	347
27.4.4	Notable Architectures	347
27.5	Risk-Aware Perception	348
27.5.1	Risk Quantification	348
27.5.2	Attention Cascade	348
27.6	Social Interaction Modeling	349
27.6.1	Social Forces Model	349
27.6.2	Implicit Social Behavior in Data	349
27.6.3	Cultural and Regional Variation	349
27.7	Integration with Planning	350
27.7.1	Occupancy-Time Projections	350
27.7.2	Interaction-Aware Planning	350
27.7.3	Risk-Bounded Planning	350
27.8	Industrial Deployment	350
27.9	Chapter Summary	350
27.10	Exercises	351
28	Uncertainty, OOD Detection, Explainability, and Adversarial Robustness	352
28.1	Introduction	352
28.2	Uncertainty Estimation	353

28.2.1	Aleatoric vs Epistemic Uncertainty	353
28.2.2	Uncertainty Estimation Methods	353
28.2.3	Calibration	354
28.3	Out-of-Distribution Detection	356
28.3.1	Why OOD Matters for AVs	356
28.3.2	OOD Detection Methods	356
28.4	Explainable AI (XAI) for AV Perception	357
28.4.1	Attribution Methods	357
28.4.2	Attention Visualization	358
28.4.3	Counterfactual Explanations	358
28.5	Adversarial Attacks on Vision	358
28.5.1	Digital Adversarial Examples	358
28.5.2	Physical Adversarial Attacks	358
28.5.3	Adversarial Training	359
28.5.4	Certified Defenses	359
28.6	Sensor Spoofing and Jamming	359
28.6.1	GPS/GNSS Spoofing	360
28.6.2	LiDAR Attacks	360
28.6.3	Camera Attacks	360
28.6.4	Radar Attacks	360
28.7	Sensor Degradation Modeling	360
28.7.1	Degradation Modes	360
28.7.2	Detection and Estimation	360
28.7.3	Modeling Degradation in Training	360
28.8	Redundant Perception Pipelines	361
28.8.1	Diverse Redundancy	361
28.8.2	Voting and Arbitration	361
28.8.3	Cost-Benefit of Redundancy	361
28.9	Industrial Deployment	362
28.10	Chapter Summary	362
28.11	Exercises	363
29	Advanced Scene Representations	364
29.1	Introduction	364
29.2	Layered Scene Decomposition	364
29.2.1	Benefits	365
29.3	Probabilistic Occupancy Fields	365
29.3.1	Formulation	366
29.3.2	Advantages over Discrete Grids	366
29.3.3	Implementations	366
29.4	Semantic Flow Fields	367
29.4.1	Representation	367

29.4.2	Why Combined?	367
29.5	Implicit Scene Representations	367
29.5.1	Coordinate-Based Networks	367
29.5.2	Relation to NeRF and Occupancy Networks	367
29.6	Industrial Deployment	368
29.7	Chapter Summary	368
29.8	Exercises	368
30	Advanced Validation and Frontier Technologies	370
30.1	Introduction	370
30.2	The Test Pyramid for AV Validation	371
30.2.1	Unit Tests	371
30.2.2	Software-in-the-Loop (SIL)	371
30.2.3	Closed-Loop Simulation	371
30.2.4	Hardware-in-the-Loop (HIL)	372
30.2.5	Real-World Testing	372
30.3	Scenario-Based Validation	372
30.3.1	Scenario Taxonomy	372
30.3.2	Scenario Libraries	373
30.3.3	Scenario Coverage	373
30.4	Corner-Case Mining	373
30.4.1	Fleet-Data Mining	373
30.4.2	Failure Injection	373
30.4.3	Model-Targeted Mining	373
30.4.4	Generative Corner-Case Mining	374
30.5	AI-Driven Scenario Generation	374
30.5.1	Parametric Generation	374
30.5.2	Adversarial Generation	374
30.5.3	Large Language Models for Scenarios	375
30.6	Sim-to-Real Transfer	375
30.6.1	Domain Randomization	375
30.6.2	Domain Adaptation	375
30.6.3	Closing the Gap with Real-to-Sim	375
30.6.4	Validation of Sim-to-Real	375
30.7	Neuromorphic Computing	376
30.7.1	Event Cameras	376
30.7.2	Spiking Neural Networks	376
30.7.3	Chips and Platforms	376
30.8	Quantum Sensing	377
30.8.1	Quantum Inertial Navigation	377
30.8.2	Quantum Gravity Gradiometers	377
30.8.3	Timeline and Practicality	377

30.9 Multi-Modal Foundation Models	377
30.9.1 Vision-Language Models	377
30.9.2 Multi-Sensor Fusion at Foundation Scale	378
30.9.3 Challenges	378
30.10 Zero-Shot and Few-Shot Learning	378
30.10.1 Prompt-Based Classification	378
30.10.2 Few-Shot Learning	379
30.10.3 Production Role	379
30.11 Industrial Deployment	379
30.12 Chapter Summary	380
30.13 Exercises	380
31 Ontologies and Semantic Representations for AVs	381
31.1 Introduction	381
31.2 Ontology Fundamentals	382
31.2.1 Description Logics and OWL	382
31.3 ASAM OpenXOntology	383
31.3.1 Structure	383
31.4 Scene Graphs	383
31.4.1 Relations of Interest	383
31.5 Neuro-Symbolic Integration	384
31.6 Maintenance at Fleet Scale	384
31.7 Industrial Deployment	385
31.8 Chapter Summary	385
31.9 Exercises	385
VIII Safety and Verification	387
32 Safety and Verification	388
32.1 Introduction	388
32.2 Hazard Analysis and Safety Cases	389
32.2.1 Hazard Analysis	389
32.2.2 Safety Case Structure	390
32.3 Operational Design Domain	390
32.3.1 ODD Specification	391
32.3.2 ODD Exit and Minimum-Risk Condition	391
32.4 Formal Verification	391
32.5 Runtime Assurance and Safety Shields	392
32.6 Redundancy and Graceful Degradation	392
32.7 Fault Detection and Response	393
32.8 Validation and Testing	393
32.8.1 Scenario-Based Testing	393

32.8.2 Simulation at Scale	394
32.8.3 Real-World Testing	394
32.8.4 Disengagement Reports	394
32.9 Safety Metrics	394
32.10 Worked Examples	395
32.11 Industrial Deployment	395
32.12 Chapter Summary	396
32.13 Exercises	396
A Notation Reference	397
B Datasets and Benchmarks	400
C Linear Algebra and Numerical Methods Refresher	405
C.1 Vector Spaces and Norms	405
C.2 Eigen- and Singular-Value Decompositions	405
C.3 Matrix Identities	405
C.4 Positive (Semi)definite Matrices	406
C.5 Solving Linear Systems	406
C.6 Numerical Stability	406
C.7 Differentiation on Lie Groups	406
C.8 Random Matrix and Covariance Propagation	407
C.9 Graph Laplacians and Spectral Tools	407
D Solutions to Selected Exercises	408
Bibliography	421

List of Figures

1.1	Chapter roadmap. The chapter establishes shared vocabulary (SAE Levels, ODD, MRC) before introducing the canonical autonomy stack and the long-tail challenge that defines the engineering problem.	2
1.2	The canonical modular autonomy stack. Data flows from sensors through perception and prediction into planning and control. Localization and mapping provide the geometric reference frame used by every downstream module.	6
2.1	Chapter roadmap. Four pillars feed every subsequent chapter: rigid-body geometry for poses, probability for sensor fusion, optimisation for SLAM and control, and deep learning for perception, prediction, and policy.	20
2.2	Scaled dot-product attention. Queries and keys compute a similarity matrix $\mathbf{QK}^\top$; the scaling factor $1/\sqrt{d_k}$ prevents the pre-softmax values from saturating. An optional mask (causal for decoders, padding for variable-length sequences) zeros out disallowed positions. The softmax normalizes to a probability distribution over keys for each query; values are mixed by these attention weights to produce the output.	45
2.3	Multi-head attention. The input is linearly projected into h separate (Q, K, V) triplets—one per attention “head.” Each head runs its own scaled dot-product attention (Figure 2.2) in parallel, producing h output sequences of dimension d_v . These are concatenated along the feature axis and linearly projected back to the original dimension. Different heads learn to attend to different structural relationships; production AV transformers typically use $h = 8$ or $h = 16$	46
2.4	The transformer encoder block (pre-norm variant). Two residual sub-blocks: (1) LayerNorm $\rightarrow$ multi-head self-attention (expanded in Figure 2.3) $\rightarrow$ residual add; (2) LayerNorm $\rightarrow$ position-wise feed-forward $\rightarrow$ residual add. Stacking L such blocks ($L = 6\text{--}24$ typical) produces the encoder used in BEVFormer, scene transformers, and language models. The pre-norm placement (LayerNorm before each sub-block) trains more stably at depth than the post-norm variant from the original paper.	46

- 2.5 A driving-specific transformer stack (schematic). Heterogeneous tokens—image patches from multi-view cameras, agent history sequences, and lane-graph vertices—are embedded with type-specific positional encodings and processed through a shared encoder stack of L transformer blocks (Figure 2.4). Multiple task heads (3D detection, trajectory prediction, occupancy) decode the shared representation. This pattern underlies production architectures such as BEVFormer, Wayformer, and unified perception- prediction models at leading AV programs. 47
- 2.6 Attention compute scaling with sequence length (log–log). Full self-attention scales as N^2d ; at large token counts this is prohibitive. Window attention (Swin-style, window size $W = 7$) scales linearly. Linear attention variants (Performer, Linformer) scale with constant d^2 overhead. Dashed verticals mark relevant automotive token counts: ~ 256 for ViT-base classification, $\sim 16k$ for a dense BEV grid. At BEV resolution, full attention is $64\times$ more expensive than window attention and is rarely used in production. 47
- 2.7 GNN message passing on a four-node intersection graph. (a) Nodes are the ego, two nearby cars c_1, c_2 , and a pedestrian p ; edges connect agents whose trajectories interact. (b) In one GNN layer, each node receives messages $\phi(\mathbf{h}_u, \mathbf{h}_v, \mathbf{e}_{uv})$ from its graph neighbors (Eq. 2.48), aggregates them, and updates its own state (Eq. 2.49). Stacking L such layers propagates information across L -hop neighborhoods—exactly the mechanism by which VectorNet, SceneTransformer, and related prediction models reason about multi-agent interactions. 50
- 3.1 Canonical autonomy stack. Sensors (left) feed perception modules that produce detections, tracks, segmentations, and localization. These combine into a unified *scene representation* that prediction and planning consume. The planner’s output is a reference trajectory; the control module turns it into actuator commands. Perception is the interface between the physical world and everything downstream: the quality of its output bounds the quality of the whole system’s behavior. 62
- 3.2 The three canonical sensor-fusion hierarchies. *Early* fusion (a) concatenates raw sensor data and runs a joint network – maximum information preserved, but joint calibration must be exact and sensor failures propagate. *Mid* fusion (b) extracts features per modality, then fuses in a shared latent space (typically BEV) – the dominant production pattern: BEVFormer-style architectures live here. *Late* fusion (c) runs separate per-modality detectors and combines at the track level via Kalman-filter or graph methods – modular, robust to sensor failure, but loses cross-modal correlations. 64
- 3.3 SAE J3016 levels and their perception implications. Perception requirements grow non-linearly with level: L0-L2 perception can operate with a driver as backup; L3 must self-monitor and hand back with warning; L4/L5 must self-recover from any perception failure without human intervention. Most commercial deployments in 2024-2026 operate at L2 (consumer) or L4 within constrained ODDs (robotaxi). L5 remains a research target. 65

- 4.1 Chapter roadmap. Camera input flows through a shared backbone into multiple task heads. Augmentation and multi-task loss weighting (lower) regularise the shared representation across the heads. 72
- 4.2 Accuracy-vs-compute Pareto frontier for major backbone families. EfficientNet dominates the mid-compute on-vehicle zone; MobileNets dominate the ultra-low-compute left edge; Swin transformers dominate the right edge where offline auto-labeling runs. ResNet remains competitive at mid-compute as a battle-tested baseline with excellent tooling support. The dashed box roughly delineates the deployment envelope for a typical automotive-grade accelerator running multi-camera inference at 30 Hz. 76
- 4.3 Two-stage pretraining paradigm for AV perception. Stage 1: a backbone learns a task-agnostic representation from large-scale unlabeled or weakly-labeled driving data using a self-supervised objective (MAE reconstruction, CLIP image-text alignment, or contrastive learning). Stage 2: the pretrained backbone is copied, extended with task heads (detection, segmentation, BEV perception), and fine-tuned on comparatively small labeled datasets. This scheme is how modern AV stacks exploit the petabytes of unlabeled video their fleets collect. 79
- 4.4 Lift-Splat-Shoot BEV architecture. Each camera’s image is passed through a 2D backbone (stage 1). The network then “lifts” each 2D feature into a 3D feature frustum by predicting a per-pixel depth distribution and taking the outer product with the feature (stage 2, Eq. 4.7). All lifted frustums are “splatted” into a common BEV grid in the ego frame (stage 3). BEV task heads (3D detection, BEV semantic segmentation, motion forecasting) consume the grid. The architecture is differentiable end-to-end and shares the BEV grid across all tasks—the key structural simplification that unifies modern multi-camera AV perception. 82
- 5.1 Four LiDAR beam-steering architectures. (a) *Mechanical* spinning: the whole optical head rotates, sweeping typically 64 or 128 laser channels through a full 360°. (b) *MEMS*: a tiny mirror deflects a single laser over a $\pm 20^\circ$ field of view; no bulk rotation, but still a mechanical moving part. (c) *Solid-state* (optical phased array): electronic beam steering with no moving parts; promising for automotive reliability but range-limited in current products. (d) *Flash*: VCSEL floods the scene; a SPAD array detects returns in parallel. Short-range (tens of metres), high frame rate. 88
- 5.2 Four common LiDAR point-cloud representations. (a) *Raw point set*: unordered list of 3D coordinates plus per-point features; consumed by PointNet++ and KPConv. (b) *Range image*: the ring-azimuth structure of a mechanical LiDAR reshaped into a dense 2D image, consumed by RangeNet++ and SqueezeSeg. (c) *Voxel grid*: 3D occupancy in a fixed grid, extremely sparse (typically $< 1\%$ occupied), consumed by VoxelNet and SparseConv-based networks. (d) *BEV pillars*: a top-down grid where each cell aggregates a vertical column of points, consumed by PointPillars and CenterPoint. Each representation trades fidelity against compute differently. 92

5.3	PointPillars architecture. Raw LiDAR points are assigned to BEV pillars (vertical columns over a (x, y) grid). A per-pillar MLP aggregates the points inside each pillar into a fixed-dimensional feature vector, producing a 2D BEV tensor. A standard 2D CNN backbone then extracts multi-scale features, which feed detection heads for 3D bounding-box regression, classification, and velocity estimation. The architecture trades the fidelity of full voxel processing for a $5\times$ speedup that keeps it tractable at 10 Hz on production automotive hardware.	94
6.1	(a) An FMCW chirp train: a burst of N_c linear chirps of duration T_c spans the full frame $T_f = N_c T_c$. The fast-time (within-chirp) FFT of the beat signal resolves range (Eq. 6.4); the slow-time (across-chirp) FFT resolves Doppler frequency (Eq. 6.7). (b) Together they form the range-Doppler map, a 2D intensity image in which each target appears as a peak at its (r, v) location. Multiple simultaneously tracked targets separate cleanly when they differ in range, velocity, or both.	106
6.2	MIMO virtual-array construction. Two transmit antennas spaced $4d$ apart (top, red) and four receive antennas spaced d (middle, blue) combine to form an 8-element virtual array (bottom, green, dashed) with aperture $7d$. Angular resolution improves proportionally to aperture: with $d = \lambda/2$, this virtual array achieves $\sim 15^\circ$ resolution compared to $\sim 25^\circ$ for the physical 4-element Rx alone. Production 4D imaging radars scale this to $N_T = 12, N_R = 16$ with clever antenna placement, producing 192-element virtual arrays at 1° resolution.	107
6.3	Three common radar-camera fusion architectures. (a) <i>Late</i> fusion runs independent radar and camera detectors and fuses at the track level. (b) <i>Proposal-refine</i> uses radar detections as region proposals that a camera classifier refines with fine-grained class labels—a natural complement between radar’s localization and camera’s semantics. (c) <i>BEV mid-fusion</i> projects both modalities into a common BEV tensor and processes them jointly, the pattern used by most 2023+ detection leaderboards.	112
7.1	GNSS trilateration. Each satellite S_i broadcasts a timestamped signal; the receiver measures the apparent range $r_i = c(t_{\text{rx}} - t_{\text{tx}})$ to four or more satellites. Each range places the receiver on a sphere of radius r_i centered on the satellite’s known position. The intersection of four spheres determines the receiver’s 3D position (x, y, z) and the receiver clock bias Δt_{rx} —four unknowns, four measurements.	119
7.2	RTK differential architecture. A surveyed base station tracks the same satellites as a mobile rover. Because the two receivers share atmospheric and satellite-ephemeris errors to first order, the base computes a correction (the difference between its measured pseudorange and its known geometric range) and broadcasts this correction to the rover over NTRIP, radio, or cellular links in the RTCM protocol. The rover subtracts the correction from its own pseudoranges, canceling common-mode errors and achieving centimeter-level accuracy.	122

- 7.3 Strapdown IMU mechanization (simplified). Gyro measurements are integrated to propagate attitude $\mathbf{R}_{WB}$ (the rotation from body to navigation frame). Accelerometer measurements are rotated into the nav frame, gravity-compensated, and double-integrated to produce velocity and position. Any bias in the gyro causes attitude drift, which in turn corrupts gravity compensation and produces accelerating position error—the classical unbounded-drift problem that GNSS corrections bound. 125
- 8.1 The infrared spectrum relevant to imaging. Visible light peaks in the 0.4 μm to 0.7 μm range. SWIR (1 μm to 3 μm) is useful for some industrial applications (see-through haze, reflective imaging). MWIR (3 μm to 5 μm) is a high-performance thermal band but requires cooled detectors. LWIR (8 μm to 14 μm) is the automotive band: human body emission peaks here, and the atmosphere transmits well. Uncooled microbolometers operate natively in LWIR, making the band cost-effective at automotive scale. 132
- 8.2 Three canonical thermal/visible fusion architectures. *Early* fusion stacks both images as channels and runs a single joint backbone—efficient but requires tight registration. *Mid* fusion runs separate backbones and concatenates features at a mid-layer, tolerating modest misalignment. *Late* fusion produces independent detection lists from each modality and combines them at the box level—robust to one sensor failing, but misses shared-feature gains. 134
- 9.1 Optical rain-sensor principle. A near-infrared LED launches light into the windshield at an angle above the critical angle for total internal reflection (TIR); on a dry surface, nearly all light reflects along the glass and reaches the photodiode (PD). A water droplet on the outer surface changes the local refractive index, breaking TIR—light escapes upward out of the glass and the photodiode’s received signal drops. The drop magnitude is roughly proportional to the covered droplet area. 139
- 9.2 Environmental-sensor integration in an AV. Rain, light, temperature, and humidity sensors feed a weather classifier that produces discrete weather states (clear, rain-light, rain-heavy, fog, snow, ice-risk). The classifier output modulates three downstream consumers: perception (confidence scaling, modality prioritization), the planner (speed/follow-distance envelope), and HVAC (defogging/defrosting). The loop from environmental sensors to perception/planning behavior is tight: updates at 1 Hz are typical, and the state is smoothed to avoid oscillation. 142
- 10.1 Representative Level 4 AV sensor coverage (top view). A 360° LiDAR provides baseline 3D geometry, overlapped by forward-looking and corner radars for velocity and weather robustness, surround cameras for semantic perception, and short-range ultrasonics for close-in parking. The deliberate overlap supports cross-modal consistency checks. 146

10.2	Time synchronization in a multi-sensor AV stack. Different sensors sample at different rates and phases; a fusion module reconciles them by either buffering and aligning to a common timestamp t^* or by propagating asynchronous observations through the filter as they arrive with their per-sample timestamps.	155
10.3	Three canonical fusion architectures. <i>Late fusion</i> runs independent detectors per sensor and fuses at the track level (robust, interpretable, throws away cross-modal evidence). <i>Mid fusion</i> combines learned features from each sensor before a fused task head (state of the art in 3D detection, depends on tight spatial-temporal registration). <i>Early fusion</i> concatenates raw or near-raw modalities into a single encoder (maximum joint information, maximum compute and calibration requirements).	157
11.1	Three canonical 2D-detection pipelines. <i>Two-stage</i> (top): a Region Proposal Network proposes candidate boxes; a second stage crops features via ROI pooling and refines + classifies each. High accuracy, higher latency. <i>Single-stage</i> (middle): a dense prediction head emits box+class at every feature-map cell; NMS prunes duplicates. Lower latency, slightly lower accuracy on small objects. <i>Transformer-based</i> (bottom): a fixed set of learned object queries attends to encoder features; outputs are a direct set, so no NMS is needed. Latency comparable to single-stage; training requires more data.	164
11.2	Three architectures for 3D detection. (a) <i>LiDAR-only</i> voxelizes or pillarizes the point cloud, runs a sparse-conv 3D CNN, and emits 3D boxes directly (PointPillars, CenterPoint, VoxelNet). (b) <i>Camera-only</i> extracts 2D features, lifts them to a BEV grid (Lift-Splat-Shoot, BEVFormer), and runs a BEV head. (c) <i>LiDAR+camera fusion</i> projects both modalities into BEV and runs a joint head (BEVFusion, TransFusion, DeepFusion). Fusion recovers most of the gap to LiDAR while adding camera semantics.	166
11.3	Tracking-by-detection data flow. Each frame's detections are matched against propagated track predictions via a gated Hungarian assignment. Matched pairs update their Kalman-filtered track state; unmatched detections spawn new tracks; unmatched tracks decay after a fixed time-out. The resulting track list is fed to downstream modules and propagated forward to the next frame.	169
12.1	U-Net-style encoder-decoder. The encoder (left, blue) compresses the image through successive downsampling; the decoder (right, orange) upsamples back to full resolution. Skip connections (red, dashed) carry high-resolution features from the encoder directly to corresponding decoder stages, preserving fine detail that would otherwise be lost through downsampling. This pattern is the backbone of nearly every modern segmenter, from U-Net itself to DeepLab, Mask2Former, and beyond.	177
12.2	Classical panoptic-segmentation pipeline. A shared backbone feeds two heads: a stuff-class semantic segmenter and a thing-class instance segmenter. A fusion step combines them into a single per-pixel output where each pixel has a class and (for thing classes) an instance ID. Unified architectures (Mask2Former, OneFormer) collapse this into a single query-based head with one forward pass.	180

- 13.1 Chapter roadmap. SLAM is a two-stage pipeline: a fast front-end aligns successive observations; a slower back-end optimises a global graph of constraints. Loop closure (red) provides drift correction by detecting revisited places. 189
- 13.2 The Bayes-filter recursion underlies every probabilistic SLAM and state-estimation algorithm. At each step the prior belief is propagated through the motion model (**predict**), then corrected against the latest measurement (**update**), giving the posterior belief. The posterior becomes the prior at the next step (dashed feedback). Different filters (KF, EKF, UKF, particle filter) realise this recursion under different parametric or sample-based representations of the belief. 190
- 13.3 A small SLAM factor graph with 4 poses $\mathbf{x}_i$ (blue circles), 3 landmarks $\mathbf{l}_j$ (green circles), and factors (orange squares). A prior anchors $\mathbf{x}_0$ (red square). Consecutive-pose factors $f_{i,i+1}$ encode odometry constraints. Pose-landmark factors $f_{i,l}$ encode observations (e.g., camera reprojection or LiDAR measurement). The dashed purple factor is a *loop closure*: a constraint linking $\mathbf{x}_0$ and $\mathbf{x}_3$ because the vehicle revisits the starting region. Loop closures are what turn accumulating odometry drift into a globally consistent trajectory. 190
- 13.4 Visual-Inertial Odometry (VIO) pipeline. Camera features are extracted and tracked across frames; IMU samples are preintegrated into relative motion constraints between keyframes. Both feed a local bundle-adjustment optimizer that estimates the current pose, velocity, and IMU biases. The output is fed back into feature tracking (to reject outliers via predicted positions) and out to the planner / localization layer. VIO is the production standard for smooth, high-rate ego-motion in visual-primary AV stacks. . 192
- 14.1 Layered HD-map contents. The dense geometric base (LiDAR point cloud or surfel surface) is acquired from survey vehicles and underlies everything. Landmarks (traffic signs, poles) are annotated on top; road-level topology captures intersection structure; lane-level geometry captures the precise path of each lane; the tactical layer encodes rules (speed limits, yields); the dynamic layer carries live updates (closures, traffic, weather). Layers are built bottom-up and consumed top-down by the planning stack. 200
- 14.2 HD-map construction pipeline. Dedicated survey vehicles capture LiDAR, camera, and RTK-GNSS data over the target area. Global alignment (SLAM, pose-graph optimization) produces a coherent large-scale map. Automated semantic extraction identifies lanes, signs, and landmarks; human validation corrects errors. Versioning and tiling prepares the map for on-vehicle use. Fleet observations (purple, dashed) feed a continuous update loop that keeps the map fresh. 201
- 14.3 Map-based localization pipeline. GNSS/IMU produces a prior pose estimate with coarse (1–2 m) accuracy. The LiDAR scan (or camera-derived features) is aligned to the map tile containing the estimated region via ICP or NDT, refining the pose to centimeter-level accuracy. The result updates the EKF, closing the loop. The dashed feedback makes the pipeline a running estimator rather than a one-shot matcher. . . . 203

15.1	Geometric parameters for wheel odometry on an Ackermann- steered vehicle. Wheel angular rates $\omega_{RL}, \omega_{RR}, \omega_{FL}, \omega_{FR}$ are measured at each wheel. Track width w separates the left and right wheels; wheelbase L separates front and rear axles. The front wheels steer at angle δ . Vehicle linear velocity v and yaw rate ω follow from the kinematic model.	215
15.2	Wheel-odometry fusion architecture. Wheel encoders report angular rates; IMU reports specific force and angular velocity; GNSS reports absolute position (when available). An EKF or factor-graph estimator fuses all three and produces a smoothed pose estimate. Feedback from the estimator updates the wheel-radius calibration online, correcting systematic bias that would otherwise accumulate as drift.	216
16.1	Feature-based visual odometry pipeline. Each image is processed to detect sparse features (ORB, SIFT, or learned SuperPoint-style). Descriptors from consecutive frames are matched; RANSAC rejects outlier matches. A minimal-case solver (5-point or 8-point algorithm) computes the initial relative pose; local bundle adjustment refines pose and triangulated 3D points jointly. Output feeds the trajectory chain and downstream SLAM or localization layers.	222
16.2	Visual-inertial odometry architecture. Visual features from consecutive frames and preintegrated IMU measurements feed a joint optimizer that solves for camera pose, linear velocity, and IMU biases simultaneously. The IMU provides metric scale (which resolves monocular ambiguity), short-timescale smoothness, and graceful degradation through brief visual dropouts. The camera provides long-timescale anchoring and absolute orientation reference.	225
16.3	Weather-adaptive VO confidence cascade. Environmental sensors (rain, light) plus intrinsic VO quality signals (feature count, RANSAC inlier ratio, residual magnitude) feed a confidence estimator that outputs a weight $\alpha \in [0, 1]$. The weight modulates VO's contribution in the downstream sensor-fusion estimator. Under good conditions α is near 1; under poor conditions it approaches 0 and other sensors dominate.	227
17.1	Chapter roadmap. Past trajectories and map context feed a shared scene encoder; a multi-modal decoder produces K candidate futures with associated probabilities. Evaluated by minADE, minFDE, and miss rate.	234
17.2	Modern map-aware, interaction-aware predictor. Each agent's recent trajectory and the local lane graph are encoded separately. A scene-level transformer or graph neural network then jointly attends to all agent and map tokens, producing K trajectory hypotheses per agent with mode probabilities. Representative implementations: VectorNet, SceneTransformer, Wayformer, MTR.	236
17.3	VectorNet architecture (schematic). Heterogeneous polyline inputs (agent trajectories, lane segments, crosswalks, etc.) are each processed by a per-polyline <i>subgraph</i> that aggregates intra- polyline structure. Polyline embeddings then enter a <i>global interaction graph</i> with self-attention across all polylines. The target agent's embedding decodes a predicted trajectory (or trajectory distribution).	237

17.4	SceneTransformer architecture (schematic). The full agent $\times$ time tensor of trajectory states, plus map polyline tokens, are processed by a stack of L factorized attention layers: attention over agents (capturing inter-agent interaction at each time step), attention over time (capturing dynamics for each agent), and cross-attention to map context. The decoder produces a <i>joint</i> K -mode prediction: each mode is a coherent assignment of futures to all agents simultaneously, rather than independent per-agent distributions.	237
17.5	MultiPath++ architecture (schematic). Three input streams (agent history, map polylines, agent-agent interactions) are encoded separately and fused by a <i>multi-context gating</i> layer that learns dynamic weights over the contexts. Two task heads then predict K trajectory candidates (anchor regression) and their probabilities (mode classifier). The joint output is a Gaussian-mixture-style distribution over the K modes, calibrated for downstream planner consumption.	238
17.6	Prediction error vs horizon for four classes of predictor on typical urban benchmark data. Error grows roughly linearly for short horizons and super-linearly beyond 5 s as uncertainty compounds. Constant-velocity (red) is the floor a usable predictor must beat. Physics + map (orange) captures lane-following behavior. Learned multi-modal models (green) win significantly on minADE by covering diverse possible futures. Joint scene transformers (blue) add further gains by modelling inter-agent consistency. Shaded region marks the planner-critical horizon (≤ 3 s) where prediction quality most strongly affects ego safety.	241
18.1	Chapter roadmap. Hierarchical planning: route gives the goal, predictions feed the behaviour planner (lane choice, gap selection), the trajectory optimiser produces the executable path under safety constraints (RSS-style guarantees enforced above). . . .	245
18.2	The planning hierarchy. At the top, a route planner picks a lane-level path from origin to destination. A behavior planner selects tactical decisions (which lane, when to yield, whether to pass). A motion planner produces the specific trajectory over a 1 s to 5 s horizon. A controller executes the trajectory at control-loop rate. Each layer's output is the next layer's context; timescales shrink by orders of magnitude as we descend.	245
18.3	The Frenet frame. The curved blue reference is the lane centerline. A point on the ego trajectory (green dashed) is expressed as arc-length s along the reference plus perpendicular offset d . This decomposition decouples longitudinal and lateral dynamics and produces much simpler trajectory optimizations than working in Cartesian coordinates directly.	248
18.4	Model-predictive control loop. At each control tick, MPC builds a quadratic program over a short (1 s to 3 s) horizon, solves it, applies the first control action, observes the new state, and repeats. Horizons are short because the problem grows with horizon length; control rate (50 Hz to 100 Hz) is high because we re-solve frequently to close the loop.	249

19.1	The agent–environment loop. At each step, the agent observes state s_t , chooses action a_t from its policy, and the environment returns a new state s_{t+1} and reward r_t . RL algorithms update the policy to maximize expected discounted reward over this loop. In driving RL, the environment is a simulator; the agent is a neural network policy; state and action are engineered from the driving context (perception outputs, ego plan, etc.).	254
19.2	Soft Actor-Critic architecture. The actor network produces stochastic actions; the environment returns rewards to a replay buffer. Two critic networks (Q_1, Q_2) estimate action values (double-Q reduces over-estimation bias). A slowly-updating target network $\bar{Q}$ stabilizes training. The actor’s gradient flows from a soft value combining expected Q with an entropy bonus. SAC has become the default choice for continuous-action RL.	256
20.1	Chapter roadmap. The controller realises the planner’s trajectory under vehicle dynamics. Reference and current state feed the controller; commands flow to actuators; closed-loop feedback (dashed) updates the state model.	264
20.2	Kinematic bicycle model. The two front and two rear wheels are collapsed onto centreline pseudo-wheels at front and rear axles separated by wheelbase L . Steering angle δ is applied to the front pseudo-wheel. The body’s velocity v points along the heading; the yaw rate $\dot{\theta} = v \tan \delta / L$ follows from the geometry of the instantaneous turn centre. This model is accurate at parking-lot speeds where tire slip is negligible.	264
20.3	Friction-circle (or "g-g") constraint. Combined longitudinal (a_x , brake/throttle) and lateral (a_y , cornering) accelerations must satisfy $\sqrt{a_x^2 + a_y^2} \leq \mu g$, where μ is the tire-road friction coefficient. The dashed inner ellipse shows the linear-Pacejka regime where $F_y \approx C_\alpha \alpha$ holds; beyond it tire forces saturate. Production NMPC formulations enforce the friction circle as a hard constraint (Chapter 21); LMPC linearises around an operating point and silently violates it at the edge of grip.	265
20.4	The control closed loop. Reference trajectory (from the planner) feeds a controller (here, an MPC) that produces the actuator commands (a, δ) . The vehicle responds; a state estimator (fed from the perception stack) recovers the new state for the controller’s next tick. Closing this loop at 50 Hz to 200 Hz is what rejects unmodeled disturbances and actuator errors that would otherwise accumulate.	268
21.1	Chapter roadmap. MPC is studied at three depths: the general receding-horizon formulation, the linear-QP and nonlinear-SQP / RTI solvers, and the production toolchains (do-MPC, Acados, HPIPM, OSQP). Stochastic and robust extensions handle uncertainty.	273

21.2	Receding-horizon MPC. At each control step t the solver predicts the optimal control sequence over a horizon of N future steps (blue rectangles). Only the first control u_0^* is applied (red); the rest is discarded. At step $t + 1$ the entire plan is re-solved, shifted by one step. The horizon “recedes” as time advances. The closed-loop applied path (bottom row) is the concatenation of all the first-control segments. This rolling-window re-planning is what makes MPC robust to model error and disturbances: mistakes from the previous plan are corrected on the next solve.	273
22.1	Chapter roadmap. Behavior cloning is the core paradigm; conditional and foundation-model variants extend it. Failure modes (covariate shift, causal confusion) and hybrid architectures (DAVE-2, openpilot) are studied as separate threads.	281
22.2	The end-to-end spectrum. Classical modular architectures (left) have interpretable boundaries and well-understood failure modes, but require substantial hand-engineering. Fully end-to-end systems (right) learn everything from data, trading interpretability for data efficiency and flexibility. Most production AV programs operate in the middle: learned perception and behavior modules within a structured hierarchy, often with rule-based safety envelopes.	281
22.3	Detailed view of a classical modular AV architecture. Sensors (left, blue) feed perception modules (yellow); perception outputs feed reasoning modules (motion prediction, localization, mapping); reasoning feeds planning (red) which finally produces control commands. Solid black arrows show forward data flow; dashed red feedback paths show control-state and pose information returned to upstream modules; dashed blue paths show localization-priors fed back into perception (the position helps disambiguate which lane, which detected object); dashed green paths show map information flowing into route- and behaviour-planning. Each module is the subject of one or more chapters in this book.	282
22.4	Foundation-model-style end-to-end driving. Multi-modal sensor inputs (cameras, radar, navigation, optional natural-language context) feed a large transformer backbone. The backbone produces action predictions (steering + throttle) along with auxiliary outputs—BEV semantics, occupancy maps, or natural-language explanations of decisions. Auxiliary heads both regularize training and provide interpretability when the system is deployed.	284

22.5	The DAVE-2 architecture (Bojarski et al., 2016) flowing in a single end-to-end pipeline from forward-camera input to steering output. Tensor shapes are annotated below each layer in <i>italic</i> . The <i>entire</i> stack—perception, prediction, planning, control—is collapsed into 5 convolutional and 3 fully-connected layers totalling $\approx 250\,000$ parameters. Compare with the classical modular architecture in Figure 1.2 (sensors $\rightarrow$ perception $\rightarrow$ prediction $\rightarrow$ planning $\rightarrow$ control), where each stage is a separately-designed and separately-trained module. DAVE-2’s contribution was to demonstrate that this modular decomposition—universally assumed in 2016—was <i>a choice</i> , not a requirement. The trained network’s salience maps subsequently revealed that it had developed its own internal lane-line and road-edge features purely from gradient descent on a steering-loss objective, without those features being labelled or designed for.	289
23.1	Pedestrian perception pipeline. Multiple sensor modalities feed a dedicated pedestrian detector that outputs 2D and 3D bounding boxes with classification. Detected pedestrians flow into a pose/action-recognition stage and then an intent-prediction stage producing crossing probability and trajectory distribution. The output feeds the planner’s pedestrian-interaction logic.	301
23.2	Crosswalk decision cascade. Once a pedestrian is detected near a crosswalk, a sequence of decisions determines the ego’s behavior: is the pedestrian in the ego’s path? Does intent prediction indicate crossing? Has crossing already begun? Each answer produces a specific ego action, from maintain-plan to full-brake. The cascade is deliberately conservative: thresholds err toward yielding.	304
24.1	Classical lane-detection pipeline. Input image is converted to grayscale and cropped to a region of interest (road area below the horizon). Canny edge detection produces edge pixels. Hough transform finds dominant straight-line segments; these are refined into polynomial curves that model lane geometry. Each stage has parameters tuned to the vehicle’s camera and operating conditions.	311
24.2	Modern multi-task lane-detection architecture. Camera features are transformed to BEV representation via learned or geometric means. Parallel heads regress lane geometry (polynomial coefficients or point sequences) and lane semantics (line type, color, crossing-permission). Outputs combine into a lane graph that downstream planning consumes directly.	313
25.1	Relative sensor degradation in moderate-dust conditions. Camera performance drops almost completely (95 %+ loss beyond a few meters). LiDAR degrades severely (70 % range loss). Radar loses only a small fraction. Thermal imaging is partially resilient, depending on particle heating. The sensor mix for dust-tolerant operation therefore differs substantially from that of urban operation.	323

25.2	Degraded-perception fallback cascade for rural operation. Sensor-health monitoring continuously assesses each sensor's reliability. Under degradation, the system checks for fallback modalities; if available, fused operation continues with reduced weights. If not, the system checks whether the current trajectory can complete safely; if yes, reduce speed and plan; if not, execute a safe stop.	327
26.1	NeRF training loop. Rays sampled from posed training images are passed through the neural field f_θ to produce density and color at each sample point. Volume rendering integrates these into a predicted pixel color; photometric loss against the ground-truth pixel drives gradient updates (dashed red) to the network weights. Training proceeds until reconstructions match observations.	334
26.2	Digital twin ecosystem. Data sources—real fleet logs, high-definition LiDAR surveys, government map and infrastructure data, live weather and traffic feeds—combine in a persistent digital twin. The twin serves multiple purposes: training AV models (synthetic data), validating releases (simulated regression suites), generating adversarial scenarios, and continuously monitoring a real fleet against simulated expectations. . .	339
27.1	Intent-then-trajectory hierarchical prediction. Observations feed an intent classifier producing a distribution over maneuver or goal classes. Trajectory predictor, conditioned on both observations and the intent distribution, produces multi-modal trajectories with the intent conditioning providing strong structural priors. Skip connection preserves fine-grained features the intent bottleneck would discard. . . .	346
27.2	Risk-aware perception cascade. Standard detection, tracking, and prediction produce a risk score for each object. An attention-allocation module feeds back to tracker and detection resource budgets (dashed feedback). High-risk objects get more tracker precision and higher-resolution detection processing. Overall compute budget is preserved; its allocation changes based on scene risk.	349
28.1	Reliability diagram showing the three calibration regimes. The dashed diagonal is the locus of perfect calibration (predicted confidence equals empirical accuracy). Typical un-calibrated deep networks (red) lie below, signalling overconfidence—especially in the high-confidence region. Under-confident models (blue) lie above. The example model from the ECE calculation (green) is mostly on the diagonal but diverges in the highest bucket. Temperature scaling pulls the curve back toward the diagonal. . . .	355
28.2	OOD detection pipeline. The scene's feature representation is scored for out-of-distributionness (via energy, feature-space distance, or ensemble disagreement). If the score exceeds a threshold, the AV enters a fallback mode: conservative behavior, operator alert, and extensive logging for later analysis. Normal operation continues for in-distribution scenes.	356
28.3	Redundant perception architecture. Sensors feed two independent perception pipelines built with different architectures by different teams. A safety-monitor + voting module compares outputs: on agreement, high-confidence output; on disagreement, safer interpretation. Independence of failure modes is the core assumption.	362

30.1	The AV test pyramid. Lower layers (unit tests, SIL) are fast, cheap, and run continuously at high volume. Upper layers (HIL, real-world) are slow, expensive, and run selectively. Each layer catches specific classes of issues; no layer alone is sufficient. Production programs calibrate the ratio of investment across layers based on where issues are found.	371
30.2	AI-driven scenario generation loop. The AV stack is evaluated against scenarios; failing scenarios (and rare successes) feed a generator that creates related variants. New scenarios enter the library for subsequent rounds of testing. The loop progressively discovers weaknesses and constructs regression protection.	375
30.3	Foundation-model-based AV perception. A single large model encodes all input modalities into a shared representation. Lightweight task-specific heads produce detections, segmentation, prediction, and natural-language query answers. Compared to per-task specialized models, a foundation-model approach promises better transfer, easier adaptation, and unified reasoning across tasks.	378
31.1	Scene graph for an intersection. Nodes (entities) carry typed class information; edges (relations) carry the semantic structure that supports reasoning. Compared to an object list, the scene graph makes interaction-relevant relationships explicit: “ego is yielding to pedestrian”, “lane is controlled by stop sign”, etc.	383
32.1	Chapter roadmap. Safety progresses through four stages: hazard analysis identifies what can go wrong; safety envelopes define provable guarantees; validation gathers evidence; standards certification turns the evidence into a deployable safety case. . .	389
32.2	Operational Design Domain (ODD) envelope. Inside the ODD (green), the AV operates autonomously with its normal safety case. Outside (red circles), the AV either degrades or hands over. The ODD boundary is the most important line in the safety case: what happens at and near the boundary defines how the system fails gracefully. Real ODDs are multi-dimensional (weather $\times$ time $\times$ road-class $\times$ speed $\times$ map-availability) and may be disjoint (e.g., highway-only L3 plus mapped-urban L4 but not the connecting roads).	390
32.3	Simplex (runtime assurance) architecture. A complex primary controller proposes actions; a simple, formally-verified backup produces safe-but-possibly-suboptimal actions; a safety monitor decides which to forward to the vehicle based on real-time safety checks. The architecture accepts complex learning-based decisions when they are safe and falls back to simple guarantees when they are not. Widely used for learned-component deployment in safety-critical domains.	392

List of Tables

1.1	SAE levels of driving automation (summary). ODD = Operational Design Domain; DDT = Dynamic Driving Task.	4
2.1	Rotation representations and their AV use cases.	21
5.1	Point-cloud representation tradeoffs.	93
7.1	The four major GNSS constellations.	120
7.2	Rough IMU grades and automotive suitability.	124
10.1	Relative sensor capabilities for AV perception.	158
25.1	Characteristics of unsealed road surfaces and the operational concerns each imposes on autonomous-vehicle operation.	320
B.1	Major perception benchmarks.	400
B.2	Prediction and planning benchmarks.	401

List of Algorithms

Notation

Throughout the book we adopt the following conventions unless otherwise stated.

Symbol	Meaning
$a, b, c, \dots$	scalars
$\mathbf{x}, \mathbf{y}, \mathbf{u}$	column vectors (bold lowercase)
$\mathbf{A}, \mathbf{B}, \mathbf{K}$	matrices (bold uppercase)
$\mathbf{A}^\top$	transpose of $\mathbf{A}$
$\mathbf{I}_n$	$n \times n$ identity matrix
$\ \mathbf{x}\ _p$	ℓ_p norm of $\mathbf{x}$
$\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$	Gaussian with mean $\boldsymbol{\mu}$, covariance $\boldsymbol{\Sigma}$
$\mathbb{E}[\cdot], \text{Var}[\cdot]$	expectation, variance
$D_{\text{KL}}(p \ q)$	Kullback–Leibler divergence
$\text{SE}(3), \text{SO}(3)$	special Euclidean / orthogonal groups
$\mathcal{X}, \mathcal{Y}, \mathcal{U}$	state, observation, action spaces
$\pi(a s)$	policy probability of action a given state s
$V^\pi(s), Q^\pi(s, a)$	state / action-value under policy π
$\nabla_{\boldsymbol{\theta}} f$	gradient of f w.r.t. $\boldsymbol{\theta}$

Mathematical objects use standard serifs; algorithmic variables in code listings use typewriter font.

Part I

Foundations

Chapter 1

Introduction to Autonomous Vehicles

“The automobile has practically reached the limit of its development, is suggested by the fact that during the past year no improvements of a radical nature have been introduced.”

Scientific American, 1909

This chapter introduces the subject that the rest of the book develops in depth: the engineering of road vehicles that perceive, decide, and act without a human driver in the loop. We begin by defining what an autonomous vehicle is (and is not), survey the standardized levels of driving automation, describe the canonical autonomy stack, tour the principal use cases and the ecosystem of actors that build and deploy them, recount the field’s short but eventful history, and enumerate the technical, regulatory, and societal challenges that motivate the rest of the book.

1.1 What is an Autonomous Vehicle?

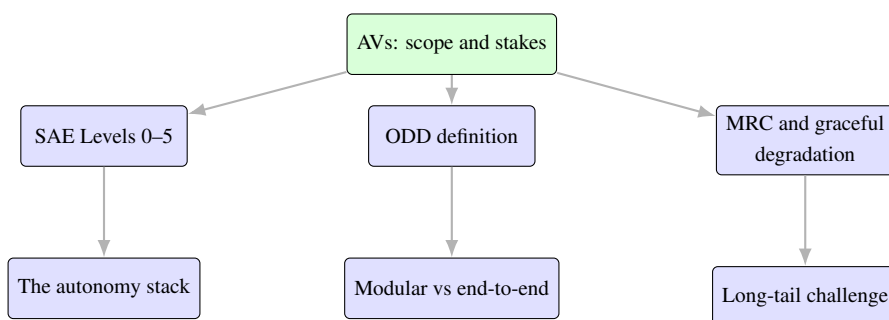


Figure 1.1: Chapter roadmap. The chapter establishes shared vocabulary (SAE Levels, ODD, MRC) before introducing the canonical autonomy stack and the long-tail challenge that defines the engineering problem.

An *autonomous vehicle* (AV) is a mobile robot whose task is safe, efficient, and socially acceptable navigation of public or private environments without human driving input. Formally, we can model an

AV as a discrete-time controlled stochastic system

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w}_t), \quad \mathbf{y}_t = h(\mathbf{x}_t, \mathbf{v}_t), \quad (1.1)$$

where $\mathbf{x}_t \in \mathbb{R}^n$ is the ego and world state, $\mathbf{u}_t \in \mathbb{R}^m$ is the commanded actuation (steering, throttle, brake), $\mathbf{y}_t \in \mathbb{R}^p$ is the sensor observation, and $\mathbf{w}_t, \mathbf{v}_t$ are process and measurement noise. The AV's policy $\pi : \mathcal{H}_t \rightarrow \mathcal{U}$ maps the history of observations $\mathcal{H}_t = \{\mathbf{y}_{0:t}\}$ to an action. Engineering an AV is, in this framing, the design of a policy that is safe under realistic noise, disturbance, and adversarial conditions.

The compactness of Equation (1.1) is deceptive. The state $\mathbf{x}_t$ must include not only the kinematic state of the ego vehicle but also the states, intentions, and attention of every other road user relevant to safe operation. The dynamics f couple physics (tire and powertrain models), psychology (how pedestrians behave at a crosswalk), and sociology (what drivers typically do at a four-way stop). The observation map h is high-dimensional, nonlinear, and severely degraded by weather, lighting, and occlusion. A sizable fraction of the engineering effort in an AV program goes into *inferring a usable approximation of $\mathbf{x}_t$ from the sensor stream* before any planning or control reasoning can begin.

1.1.1 Terminology and Related Concepts

Several overlapping terms are used in the literature and popular press. It is useful to be precise.

Automated driving system (ADS).

The preferred regulatory term. An ADS is the hardware and software that performs the dynamic driving task within a specified operational design domain.

Advanced driver-assistance system (ADAS).

A feature or set of features that assists a human driver without assuming responsibility for the driving task (adaptive cruise control, lane-keeping assist, automatic emergency braking).

Self-driving car / autonomous vehicle / driverless vehicle.

Informal synonyms for a vehicle equipped with a high-level ADS.

Robotaxi.

An on-demand passenger transport service operated by ADS-equipped vehicles without a safety driver; the clearest commercial manifestation of Level 4 autonomy at the time of writing.

Teleoperation / remote assistance.

A human operator provides high-level guidance or fallback decisions for an ADS without driving it directly. Most commercial Level 4 deployments rely on remote assistance for edge cases, though they do not depend on it for immediate safety.

A useful distinction: *automation* is the mechanization of a previously-manual task (cruise control automates speed holding); *autonomy* is automation plus the authority to make decisions under

uncertainty (when to overtake, when to yield, when to stop). A modern ADS is an autonomy system, not merely an automation system.

1.1.2 Why Autonomous Driving is Hard

Three properties make driving a stringent robotics problem.

1. **It is safety-critical.** A wrong action can kill. The acceptable failure rate is therefore extremely low—human drivers in high-income countries cause fatal accidents at roughly 10^{-8} per mile, and an ADS that is judged acceptable must be at least as good and preferably better.
2. **It is open-world.** The environment contains arbitrary agents (humans, animals, other vehicles), arbitrary objects, and arbitrary weather. The system cannot assume a closed set of possible inputs.
3. **It is shared with humans.** Other road users are not opponents, collaborators, or disinterested physical processes; they are humans with their own goals and moral standing. Behavior that is technically legal but socially unacceptable is a failure mode.

These properties rule out a purely empirical approach (“try things and see”), a purely theoretical approach (“prove the system is safe in a closed model”), and a purely data-driven approach (“train a big enough network”). The working methodology combines all three: formal safety envelopes, large-scale data, and extensive structured testing.

1.2 Levels of Driving Automation

The Society of Automotive Engineers (SAE) J3016 standard [57] defines six levels of driving automation, summarized in Table 1.1. These levels are operational, not technological: the defining difference between Level 2 and Level 3 is not which sensors are present, but who is legally and functionally responsible for safe driving when the system is engaged.

Table 1.1: SAE levels of driving automation (summary). ODD = Operational Design Domain; DDT = Dynamic Driving Task.

Level	Name	Human role under engagement	Fallback
0	No automation	Performs entire DDT	Human
1	Driver assistance	Performs lat. or long. DDT	Human
2	Partial automation	Monitors env., takes over on request	Human
3	Conditional automation	Fallback-ready, can disengage	Human
4	High automation	No human fallback within ODD	System
5	Full automation	Unconstrained ODD	System

1.2.1 The Levels in Detail

Level 0: No Automation. The human performs the entire dynamic driving task. Warning systems (lane-departure warnings, blind-spot warnings) are permitted—they inform but do not act.

Level 1: Driver Assistance. A single longitudinal *or* lateral function (adaptive cruise control, or lane-keeping assist). The driver remains responsible for the orthogonal axis and for monitoring.

Level 2: Partial Automation. Longitudinal *and* lateral automation simultaneously (“hands-free” highway systems). The driver must monitor the environment and intervene on demand. Most consumer systems marketed as “Autopilot”, “BlueCruise”, “Super Cruise”, “Pilot Assist”, and similar fall in Level 2. Driver monitoring (cameras tracking gaze, torque sensors on the wheel) is increasingly mandated.

Level 3: Conditional Automation. The system performs the entire driving task within a specified ODD, with the human standing by as a fallback. On receipt of a takeover request the human must resume control within a bounded transition window (tens of seconds at highway speeds). Level 3 introduces a subtle but consequential question: can a distracted, disengaged human re-acquire situation awareness quickly enough to safely take over? Regulators have historically been skeptical; Mercedes-Benz’s DRIVE PILOT and Honda’s Traffic Jam Pilot are among the first certified Level 3 systems, restricted to low-speed freeway traffic jams.

Level 4: High Automation. Within the ODD, no human fallback is required. If the system cannot continue, it must execute a *minimum risk condition* (MRC)—pull over, stop in lane, or otherwise bring itself to a safe state on its own. Robotaxi services (Waymo in Phoenix and San Francisco, WeRide in several Chinese cities, Apollo Go, Motional, Zoox, and others) are the clearest commercial Level 4 deployments at the time of writing.

Level 5: Full Automation. The ODD is unconstrained: the vehicle drives anywhere a competent human driver could. No production system has credibly claimed Level 5 operation; it remains a long-term aspiration.

1.2.2 Operational Design Domain

A crucial concept is the *operational design domain* (ODD): the set of road types, weather, lighting, speeds, and geographic regions within which the automated driving system is designed to function. An ODD might be “geofenced urban daylight below 72 km h^{-1} , no heavy rain, no construction zones, no emergency vehicles with lights active”. The distinction between Level 4 (self-driving *within* an ODD) and Level 5 (self-driving *everywhere*) is in practice a question of how expansive and verifiable the ODD is.

ODD specification is technical, commercial, and legal all at once. A good ODD specification enumerates:

Geographic scope.

Mapped area, allowed roads, excluded zones.

Environmental envelope.

Weather, precipitation rates, temperature, visibility distance.

Temporal envelope.

Daylight, twilight, night; specific hours.

Traffic envelope.

Speed limits, traffic density, types of other road users present.

Infrastructure dependencies.

Map freshness, communication coverage, charging and refueling availability.

Vehicle dependencies.

Specific hardware, calibration state, component health thresholds.

An engineering organization will maintain multiple ODDs in parallel: a large permissive ODD for ADAS, a tightly scoped ODD for autonomous deployment, an expanded ODD under active testing, and historical ODDs for which incident data is compared.

1.2.3 Minimum Risk Conditions and Graceful Degradation

A Level 3+ system must be able to reach a safe state from any point inside its ODD when it can no longer continue. Common MRCs:

- Continue to the end of the trip (preferred when possible).
- Pull to the shoulder and stop.
- Stop in lane with hazards.
- Hand off to a remote operator for guidance.

Graceful degradation of sensors and compute is a sub-problem of its own. Loss of a single LiDAR must not force an emergency stop at highway speed; loss of GPS must not leave the planner blind. Systems use redundant sensing, cross-checks between modalities, and pre-validated degraded operating modes.

1.3 The Autonomy Stack

Modern production AV stacks are organized into a pipeline of interdependent modules. Figure 1.2 depicts the canonical arrangement.

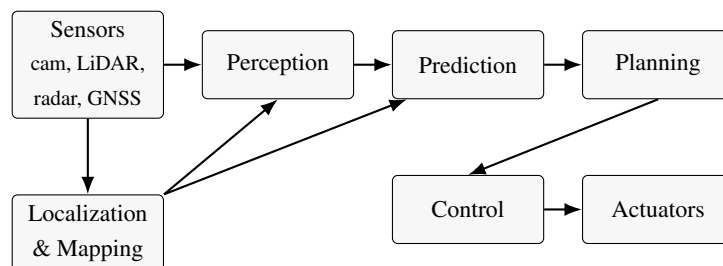


Figure 1.2: The canonical modular autonomy stack. Data flows from sensors through perception and prediction into planning and control. Localization and mapping provide the geometric reference frame used by every downstream module.

Sensing. Cameras, LiDARs, radars, ultrasonic sensors, GNSS, and inertial measurement units (IMUs) together produce the raw multimodal stream from which everything else is derived. A representative urban AV produces sensor data in the low GB/s range—several cameras at 10 Hz to 30 Hz, one or more LiDARs at 10 Hz, a few radars at 10 Hz to 20 Hz, and IMU/wheel data at 100 Hz to 400 Hz. Sensing is treated in depth in Chapter 10.

Perception. Converts raw sensor data into structured semantic output: classified and localized objects, lane geometry, traffic signals, free-space estimates. Deep neural networks dominate this layer (Chapters 4–12). Typical output rates: 10 Hz to 20 Hz for detection and tracking, 5 Hz or lower for dense semantic segmentation.

Localization and Mapping. Estimates the pose $\mathbf{T}_{WB} \in \text{SE}(3)$ of the ego vehicle with respect to a map. In urban domains centimeter-level accuracy is typically required; SLAM (Chapter 13) and HD-map-based localization (Chapter 14) fulfill this role. Localization typically runs at 50 Hz to 200 Hz (dominated by IMU integration) with slower updates from visual or LiDAR matching.

Prediction. Forecasts the future motion of other road users conditioned on scene context. Multimodal probabilistic prediction (Chapter 17) is essential because intent is unobservable. Prediction runs at 5 Hz to 20 Hz and must produce scene-consistent futures over horizons of 3–8 seconds.

Planning. Produces a reference trajectory. It is commonly hierarchical: a route planner selects a lane-level path to the destination over minutes of travel; a behavior layer picks a discrete maneuver (lane change, yield, proceed) over seconds; and a motion planner produces a continuous reference compatible with vehicle dynamics over fractions of a second (Chapter 18).

Control. Tracks the reference trajectory under vehicle dynamics using feedback controllers—PID, LQR, and model predictive control (Chapter 20). Control runs an order of magnitude faster than planning, at 50 Hz to 200 Hz, and is typically the most timing-critical module in the stack.

1.3.1 Rate Hierarchy and Latency Budget

The modules of Figure 1.2 run at very different rates because their output time-constants differ. Control must be fast because the vehicle dynamics demand it; planning is medium-rate because trajectory-level decisions change slowly; route planning is slow-rate because routes change only when the ego deviates or traffic shifts. A working latency budget for a Level 4 urban stack, from camera capture to actuator command, is typically below 150 ms. Of that budget, perception consumes the largest share (40 ms to 80 ms), followed by prediction, planning, and control in descending order.

1.3.2 Interfaces and Abstractions

The *interfaces* between modules are as important as the modules themselves. The interface between perception and prediction/planning is a list of tracked objects—class, pose, extent, velocity, acceleration, tracking confidence, and a unique ID that persists across frames—plus lane-geometry primitives

and traffic-light states. The interface between planning and control is a time-parameterized trajectory of poses and curvatures (or, in some stacks, a control set-point sequence).

These interfaces are deliberately narrow. A narrow interface allows independent development and testing of each module; permits swapping one implementation (a new detector) without re-validating the entire stack; and makes failure modes locally diagnosable. A recurring theme of the book is how learned systems can be grafted into the stack without compromising these properties—typically by producing compatible interface outputs even when the internals are end-to-end trained.

1.4 Sensor and Compute Platform

The stack of Figure 1.2 runs on a physical platform. A representative Level 4 AV carries several surround cameras of a few megapixels, one or more LiDARs (mechanical spinning or solid-state flash/MEMS), a perimeter of radars (short, mid, and long range), ultrasonic sensors for parking, a high-grade IMU, multi-band GNSS with RTK or PPP corrections, wheel encoders, and steering-angle sensors.

Onboard compute is typically one or two automotive-qualified accelerators (NVIDIA Drive, Qualcomm Ride, Tesla’s in-house HW4, Mobileye EyeQ-family) capable of several hundred TOPS, paired with a safety microcontroller cluster for functional safety. Power consumption is a persistent engineering constraint: a 500 W autonomy stack consumes a noticeable fraction of an electric vehicle’s energy budget, with range and thermal implications.

Consumer Level 2 systems run on dramatically less compute—often a single low-power accelerator—by scoping the feature set and relying on camera-dominant sensing.

1.5 Modular Versus End-to-End Paradigms

Two design paradigms dominate the literature.

1. **Modular.** Each module is trained or designed against a well-specified sub-objective and communicates with the next module via human-interpretable interfaces (bounding boxes, occupancy grids, lane polylines). Modular systems are the standard for commercial deployment because failure modes are locally diagnosable.
2. **End-to-end.** A single learned function maps raw sensor input directly to actuation commands. End-to-end models benefit from joint optimization across tasks, but their failure modes are less interpretable and verification is harder (Chapter 22).

In practice, modern production stacks are *hybrid*: e.g., a modular perception and localization frontend feeds a learned planner, or a modular planner is supervised with imitation learning from human drivers. A contemporary trend, reflected in work such as UniAD, is a *jointly trained* stack with explicit intermediate representations (detections, predicted trajectories, planned paths) that are both supervised and available as interfaces for legacy downstream consumers. This preserves much of

the interpretability of modular systems while capturing some of the information-flow benefits of end-to-end training.

Choosing between paradigms involves trade-offs along at least the following axes:

- **Data efficiency.** Modular systems can be trained per-module with per-module labels, which is often more data-efficient than end-to-end training with sparse driving-quality signals.
- **Interpretability and debugging.** Modular systems localize errors to specific modules; end-to-end systems require whole-system analysis.
- **Joint optimization.** End-to-end systems can avoid cascaded error amplification and can trade off features across modules in ways modular interfaces do not expose.
- **Verification and certification.** Regulators currently have more tools and more precedent for modular systems.
- **Scalability with data.** As fleets and datasets grow, the relative advantage of end-to-end training plausibly increases.

1.6 Use Cases and Deployment Contexts

AV technology is not deployed uniformly across all driving. Several distinct use cases have different ODDs, economics, and technical focus.

1.6.1 Robotaxi

Driverless passenger transport in geofenced urban areas. Representative: Waymo, Cruise (paused after incident in 2023), Zoox, Motional, WeRide, Apollo Go, Pony.ai. Technical emphasis: urban perception, pedestrian and cyclist prediction, unprotected left turns, interaction with emergency vehicles. Economic thesis: eventual service cost below human-driven taxi, at lower variable cost per mile. Current fleets are in the low thousands across the world.

1.6.2 Consumer Highway Pilot (Level 2/3)

ADAS systems sold on consumer vehicles that can perform extended highway driving with hands-off (Level 2) or eyes-off (Level 3 in limited cases) operation. Representative: Tesla Autopilot/FSD (Level 2 at the time of writing), Ford BlueCruise, GM Super Cruise, Mercedes DRIVE PILOT (Level 3, limited), Honda Sensing Elite. Technical emphasis: highway lane keeping, adaptive cruise, lane changes, driver monitoring, failure-to-takeover handling. Commercial thesis: feature differentiation and subscription revenue.

1.6.3 Long-Haul Trucking

Highway-only autonomous trucks aiming for hub-to-hub transportation. Representative (varying status): Aurora, Kodiak, Plus, TuSimple. Technical emphasis: highway perception at long range (often 300 m+ detection), heavy-vehicle dynamics (long braking distances, trailer articulation), geofenced freeway ODDs. Commercial thesis: labor cost reduction and driver-shortage mitigation.

1.6.4 Last-Mile Delivery

Autonomous sidewalk, bike-lane, or low-speed road robots for parcel and food delivery. Representative: Starship, Nuro, Amazon Scout (paused), Kiwibot. Technical emphasis: pedestrian-safe low-speed operation, dense urban mapping, package security. Commercial thesis: unit-cost reduction in last-mile logistics.

1.6.5 Shuttle and Fixed-Route Services

Low-speed autonomous shuttles along fixed routes in airports, campuses, business parks, and some transit supplements. Technical emphasis: fixed ODD simplifies mapping and prediction; accessibility requirements drive vehicle design. Commercial thesis: public-transit and first/last-mile augmentation.

1.6.6 Automated Parking

Valet parking within structured lots, sometimes infrastructure-assisted. Technical emphasis: low-speed precise maneuvering, handling of parking geometry, often without GNSS. Commercial thesis: user convenience and vehicle storage density.

1.6.7 Off-Road and Industrial Autonomy

Mining, agriculture, ports, and warehouse environments are among the most mature deployments of autonomous driving. Private, structured environments remove much of the uncertainty that afflicts public-road driving. Technical and algorithmic content is similar; legal and certification obstacles are lower.

The ADS technology described in this book is most directly applicable to robotaxi and highway use cases. Many algorithms transfer cleanly to delivery and industrial domains, but operational and safety-case considerations differ.

1.7 Actors and the AV Ecosystem

AV systems are built by an ecosystem of organizations with distinct roles.

- **OEMs (original equipment manufacturers).** Ford, GM, Volkswagen, Toyota, BYD, and others. Design and manufacture the vehicles; increasingly develop in-house ADAS and ADS software.
- **Tier-1 suppliers.** Bosch, Continental, Denso, Aptiv, ZF, Magna. Provide integrated sensor and ECU subsystems; many develop their own ADS stacks.
- **ADS developers.** Waymo, Cruise, Zoox, Aurora, Mobileye, WeRide, Pony.ai, Baidu, and others. Full-stack software and, in some cases, integrated vehicles.
- **Consumer ADAS/ADS.** Tesla, and increasingly the OEMs internally. Ship features directly to consumers.

- **Data and labeling.** Scale AI, Labelbox, and internal annotation operations; cloud-scale data pipelines are a substantial part of the capital expense.
- **Mapping providers.** HERE, TomTom, DeepMap (now NVIDIA), Civil Maps, in-house mapping by Waymo, Mobileye, and others.
- **Simulation providers.** NVIDIA Drive Sim, Applied Intuition, Cognata, Parallel Domain, and in-house simulators from every major developer.
- **Regulators and safety bodies.** NHTSA, state DMVs in the US; UNECE, national regulators, and safety-testing bodies (EuroNCAP) internationally.
- **Insurance, fleet operators, and infrastructure.** Fleet deployment partners (rideshare integrators, transit agencies), insurers pricing ADS-related risk, cities managing curb access and traffic rules.

No single organization spans the full stack; even the most vertically integrated developers depend on chip vendors, sensor OEMs, and external data providers. This has engineering consequences: interface standards (ADAS/AD sensor protocols, map formats such as OpenDRIVE and Lanelet2, simulation exchange formats such as OpenSCENARIO) matter enormously.

1.8 A Brief History

1.8.1 Pre-history: Automation on Rails and Roads

Automated driving is older than the modern computer. Electrical railway signaling in the late 19th century automated train safety. Cruise control appeared on consumer vehicles in the late 1950s. Anti-lock braking entered mass production in the 1970s. Stability control followed in the 1990s. Each of these was a bounded automation step that reshaped driver expectations and laid groundwork for higher-level systems.

1.8.2 1980s: Vision-Based Driving

- Ernst Dickmanns's VaMP and VITA vehicles at Bundeswehr University Munich achieved autonomous freeway driving at over 100 kmh^{-1} using cameras and Kalman-filter-based dynamic vision [17].
- Pomerleau's ALVINN at Carnegie Mellon—a small neural network driving from camera input—arrived in 1989, an end-to-end approach decades ahead of its compute era.
- Parallel efforts in the European EUREKA Prometheus project (1987–1995) produced hundreds of kilometers of autonomous driving across Europe.

1.8.3 1990s–2000s: DARPA and the Grand Challenges

- DARPA Grand Challenges (2004, 2005) and Urban Challenge (2007) catalyze sensor-fusion and planning research. Stanford's *Stanley* wins the 2005 desert event [65]; CMU's *Boss* wins the 2007 Urban Challenge [66].

- The Urban Challenge ships, for the first time, a full autonomy stack capable of driving among other autonomous vehicles in a rule-governed urban setting. Many of the engineers involved go on to found or lead today's industry—Thrun, Urmson, Levandowski, Dolgov, Whittaker, and others.
- Google's Self-Driving Car Project (later Waymo) begins in 2009.

1.8.4 2010s: Deep Learning and Commercialization

- AlexNet [40] ignites the deep learning revolution. Within years, convolutional networks dominate AV perception.
- Mobileye scales monocular-camera ADAS to millions of vehicles, creating the dominant supplier for consumer lane-keeping and automatic emergency braking.
- Tesla ships Autopilot (2014+) and scales consumer ADAS capability rapidly, accumulating substantial real-world data.
- NVIDIA demonstrates end-to-end steering from a single camera [7].
- Waymo launches commercial driverless rides in suburban Phoenix in 2018. The same year, a pedestrian fatality involving an Uber test vehicle in Tempe, Arizona prompts industry-wide reassessment of safety practices.
- A wave of venture funding creates dozens of AV developers; many do not survive the subsequent downturn.

1.8.5 2020s: Scale, Scrutiny, and Foundations

- Robotaxi services scale in Phoenix, San Francisco, Beijing, and Shanghai. Commercial Level 3 systems are certified in Japan and Germany in restricted domains.
- Transformer architectures and bird's-eye-view perception become standard in AV research and are increasingly standard in deployed systems.
- Large vision-language and video foundation models begin to influence AV pipelines, both as pretrained perception backbones and as reasoning modules for corner cases.
- High-profile incidents—notably the 2023 Cruise pedestrian-drag incident in San Francisco—trigger regulatory action and industry restructuring. The pattern of accelerated deployment followed by regulatory response is now a mature dynamic.
- Consolidation: several mid-size developers are acquired, shut down, or reorient toward trucking or ADAS. A small set of well-capitalized players dominate Level 4 passenger deployment.

The history is short by the standards of aviation or rail—the modern field is not yet two decades old—but already rich enough that its lessons (data matters, interfaces matter, safety matters, incentives matter) shape today's practice.

1.9 Technical Challenges

1.9.1 The Long Tail of Rare Events

The distribution of driving scenarios is heavy-tailed: most road miles are easy, but safety is determined by the tail. If the ego vehicle encounters a truly novel scenario (e.g., a mattress falling off a truck) with probability 10^{-8} per mile, then achieving a target rate of 10^{-9} fatal events per mile requires the system to handle each such scenario correctly *at least* one time in ten. Bulk road testing is therefore insufficient for safety validation [35]: even a fleet driving 10 million miles per year accumulates real-world experience too slowly to statistically certify new software versions between releases. Synthetic data, scenario-based testing, counterfactual simulation, and formal methods (Chapter 32) are needed.

1.9.2 Observability

Other drivers' *intentions* and *attentiveness* are unobservable. A pedestrian on a curb looking at their phone is statistically more likely to step into traffic than one looking at oncoming cars. Encoding and exploiting these soft, sociological signals is the central problem of behavior prediction. More broadly, the world state $\mathbf{x}_t$ in Eq. (1.1) is only partially observable through the sensor stream, and many of its most safety-relevant components (driver intent, pedestrian purpose) are never directly observable at all.

1.9.3 Real-Time Compute

An AV must close the full perception-planning-control loop at 10 Hz to 30 Hz on automotive-grade, power-constrained compute. End-to-end latency budgets on the order of 100 ms leave little room for the largest models; extensive neural-network quantization, pruning, and dedicated accelerators are required. Unlike in a data center, the latency distribution matters more than the average: a 99.99 % tail latency of 500 ms can be lethal at highway speeds, regardless of the mean.

1.9.4 Weather and Degraded Sensing

Rain, snow, fog, direct sunlight, and road spray each degrade specific sensors in specific ways. LiDAR returns degrade in heavy precipitation; camera dynamic range is overwhelmed by low-sun or bright-headlight scenes; radar suffers from multi-path in metropolitan canyons. Cross-modal fusion (Chapter 10) mitigates these, but every weather mode extends the validation surface and stretches the ODD envelope further from conditions in which perception models are confidently trained.

1.9.5 Multi-Agent Interaction

Planning interacts with other road users' behavior—the ego's yield signals that it is safe for a pedestrian to cross, which in turn frees the ego to proceed. These feedback loops are hard to capture in purely supervised imitation learning; they are captured partially by game-theoretic and interaction-aware prediction models (Chapter 17) and partially by planning modules that explicitly score ego plans against reactive predictions of other agents' responses.

1.9.6 Data, Labels, and Lifecycle

A production AV program consumes petabytes of log data per month. Storing, cataloging, curating, and labeling this data is the principal non-engineering expense of most programs. The *data flywheel*—deployed vehicles generate data, data improves models, improved models enable new deployment scope—is the operational thesis of fleet-based development. Making the flywheel turn fast is itself a systems problem: active learning to prioritize informative examples, automatic labeling to reduce human effort, and tight coupling between model regressions and data ingestion.

1.9.7 Regulatory and Standards Landscape

Regulation is non-uniform across jurisdictions and evolves at its own pace. An engineering program designed for a single regulatory regime (“US state-by-state”) may be unsuitable for another (“EU type-approval with documented safety case”). International standards (ISO 26262, ISO 21448, ISO/SAE 21434; UNECE regulations R155, R156, R157) codify expectations around functional safety, SOTIF, cybersecurity, and specific feature classes. The details occupy Chapter 32 and Chapter 5; here we note only that these constraints shape architecture choices from the earliest design phase.

1.9.8 Human Factors and Trust

Level 2 and Level 3 systems depend on humans to intervene appropriately. Humans are bad at monitoring automation: attention degrades, complacency develops, and takeover performance suffers after prolonged automated driving. Level 4 systems avoid this problem by not requiring human fallback inside the ODD, but must now earn trust from passengers and from non-participating road users. How an AV signals intent (turn signals, but also nuanced behavior such as slight nudging forward to indicate intent to cross an intersection) is as much a human-factors problem as a control one.

1.10 Safety Metrics and the Role of Benchmarks

Two classes of safety metric coexist in the literature: *proxy metrics* (minimum time-to-collision, minimum distance headway, lateral jerk) and *outcome metrics* (collisions, injuries, fatalities per mile). Outcome metrics are sparse and slow to accumulate; proxy metrics are dense but incomplete. A central engineering task is to design proxies that are strongly predictive of outcomes; the responsibility-sensitive safety (RSS) framework [60] is one such attempt, discussed in Chapter 32.

Two additional metric classes have emerged more recently. *Counterfactual metrics* ask what would have happened had the ego acted differently, via simulation replay of logged events. *Human-baseline metrics* compare ADS performance to local human-driver benchmarks on matched conditions, mitigating the problem that global fatality statistics are dominated by scenarios an ODD-restricted ADS never enters.

Benchmarks—KITTI [24], nuScenes [9], Waymo Open [61], Argoverse 2 [70], Cityscapes [16], BDD100K [74]—provide standardized tasks and leaderboards. They are necessary but not sufficient: state-of-the-art on KITTI is not a guarantee of safe deployment. A benchmark leaderboard captures a

fraction of what a production safety case requires, and benchmark progress has outpaced deployment progress by a large margin.

1.11 Scope and Organization of This Book

1.11.1 Intended Audience

This book is written for graduate students, researchers, and practitioners working on autonomous driving or adjacent robotics problems. The target reader is comfortable with linear algebra, probability at the level of a first graduate course, some exposure to control theory, and some exposure to modern deep learning. No prior automotive or robotics engineering is assumed.

1.11.2 Prerequisites and Notation

A condensed refresher on linear algebra and numerical methods is in Appendix C. Notation is summarized in Appendix A; the most important conventions are that vectors are bold lowercase, matrices bold uppercase, poses in $SE(3)$, and rotations in $SO(3)$. The custom macro *autonomousvehicle* abbreviates “autonomous vehicle” and is used sparingly.

1.11.3 Structure

We treat the full stack. The emphasis is on techniques that are both *principled* (admitting mathematical analysis) and *practical* (deployed or near-deployment). We do not presume a specific sensor configuration but do assume modern compute.

The first half (Parts I–III) develops the foundations, sensors, and perception. The second half (Parts IV–VI) develops prediction, planning, control, learning, safety, and ethics. The chapters can be read sequentially, but each is designed to stand on its own given the notation of the frontmatter.

1.11.4 Reading Paths

A reader short on time can follow one of the paths below:

- **Perception-focused.** Chapters 1, 2, 10, 4, 11, 12.
- **Planning-and-control-focused.** Chapters 1, 2, 17, 18, 20.
- **Learning-focused.** Chapters 1, 2, 4, 19, 22.
- **Safety-and-policy-focused.** Chapters 1, 32, 5, with selective reference to earlier chapters as needed.

1.11.5 What This Book Does Not Cover

We do not cover vehicle hardware design (powertrains, chassis, battery systems), detailed functional-safety certification processes beyond the architectural implications, or the business and regulatory history of specific AV programs. Each of these is a book of its own.

1.12 Worked Examples

Example 1.1 (Classifying a consumer system under SAE J3016). A manufacturer markets a highway system that maintains lane position and adjusts speed to surrounding traffic while the driver is present, with a wheel-mounted torque sensor that disengages the system if the driver's hands leave the wheel for more than 15 s. The system geofences itself to divided highways. Which SAE level is this?

Answer. The system performs sustained lateral *and* longitudinal control (ruling out Level 1), relies on the driver as the fallback (the 15 s torque check confirms this, ruling out Level 3 where the vehicle itself must be fallback-ready), and does not attempt a full driving task without driver oversight. It is therefore Level 2. The geofencing specifies its ODD but does not promote the level; level is determined by who is responsible when engaged.

Example 1.2 (Drafting a minimal ODD). An operator plans a campus shuttle service on a single private road loop. Draft a minimal ODD.

Answer. A defensible ODD names the specific road loop by a map reference, restricts speed to 25 km h^{-1} , constrains operating hours to daylight, excludes precipitation above a threshold (e.g., 2.5 mm h^{-1} rain), excludes snow and icy surfaces, requires HD map version $\geq v$, requires all three LiDARs and both front cameras to report healthy at the mission start, and specifies that the MRC is pull-to-shoulder and stop. This is a tightly scoped Level 4 ODD suitable for early deployment.

Example 1.3 (Miles required to demonstrate safety). Assume human drivers in a region cause fatal accidents at rate $\lambda_h = 10^{-8}$ per mile. An ADS developer wants to demonstrate the ADS fatality rate is no higher than $\lambda_s = 0.5\lambda_h$ at the 95 % confidence level. Using a Poisson event model, how many miles of failure-free operation are needed?

Answer. Under a Poisson model with rate λ , observing zero fatalities over m miles gives a one-sided upper confidence bound $\lambda \leq -\ln(1 - \alpha)/m$. Setting this equal to λ_s with $\alpha = 0.95$ gives $m = -\ln(0.05)/\lambda_s = 3.00/(0.5 \times 10^{-8}) \approx 6 \times 10^8$ miles. A fleet driving 10^7 miles per year would take roughly 60 years to accumulate this evidence—which is why simulation and scenario-based testing, not mileage alone, underpin safety cases [35].

1.13 Industrial Deployment

Waymo (a subsidiary of Alphabet Inc.) operates a fully driverless commercial robotaxi service called Waymo One in several U.S. metropolitan areas including Phoenix, San Francisco, Los Angeles, and Austin. The company has publicly reported tens of millions of fully autonomous miles driven on public roads. Cruise (a subsidiary of General Motors) deployed driverless robotaxis in San Francisco starting in 2022, suspended its operations in late 2023 following an October 2023 incident, and General Motors subsequently announced in December 2024 that it would refocus Cruise's resources on personal autonomous vehicles rather than continuing the robotaxi business. Tesla has shipped its Autopilot

driver-assistance system in millions of vehicles worldwide and markets a more advanced “Full Self-Driving (Supervised)” option, both of which remain SAE Level 2 systems requiring continuous driver supervision.

In China, Baidu’s Apollo Go robotaxi service operates in multiple cities including Wuhan and Beijing; Pony.ai and WeRide have likewise commercialised robotaxi pilots and conducted IPOs on U.S. exchanges in 2024. Chinese vehicle manufacturers including BYD, NIO, XPeng, and Li Auto have integrated increasingly capable Level 2+ assisted-driving systems into mass-market electric vehicles, with XPeng’s XNGP and NIO’s NOP+ representing production-deployed urban-driving stacks.

1.14 Chapter Summary

Autonomous driving is the closed-loop control of a stochastic system under severe observability, safety, and compute constraints. SAE levels, the ODD concept, the modular stack, and the tension between modular and end-to-end paradigms frame the rest of the book. Use cases range from robotaxi to trucking to delivery to industrial autonomy, with distinct ODDs and economics. The ecosystem is fragmented across OEMs, suppliers, ADS developers, and regulators; interfaces and standards matter. The long-tail nature of the event distribution makes safety validation a first-class research problem, not something to be bolted on after the fact.

1.15 Exercises

1. Give two examples of requirements that an ODD specification should include beyond geographic area.
2. Using Equation (1.1), write the state and observation for a two-wheeled bicycle model with GPS and compass sensing.
3. If fatal accidents occur in human driving at about 10^{-8} per mile, how many miles of failure-free autonomous driving are needed to statistically demonstrate an order-of-magnitude safety improvement at 95% confidence? Use a one-sided binomial test.
4. Argue for or against: “A Level 4 system is just a Level 5 system with a more restrictive ODD.” Identify at least two distinctions.
5. Sketch a latency budget for an urban Level 4 stack with a 150 ms total budget, allocating time to sensor capture, perception, prediction, planning, and control. Justify each allocation with one sentence.
6. Consider a Level 3 system whose ODD is “divided highway, clear weather, below 60 km h^{-1} , in daylight”. Propose a takeover-request protocol, including a timeout and the behavior on timeout. What assumptions about the human driver are you implicitly making?
7. For each of the use cases in the deployment-contexts section (robotaxi, highway pilot, long-haul trucking, last-mile delivery, shuttle, parking, industrial), identify the single technical challenge

most responsible for limiting expansion of the use case, and justify your choice briefly.

8. An ADS developer reports a 30 % reduction in collisions-per-mile after a software update, based on 500,000 miles of fleet driving before and after. Estimate whether this claim is statistically credible and state the assumptions in your estimate.
9. Propose a metric that captures “social acceptability” of AV behavior that is not captured by collision-rate, distance-to- obstacle, or passenger-comfort metrics. Give an operational definition and one way to measure it.
10. The interface between perception and prediction in the stack of Figure 1.2 is a list of tracked objects. What information, if any, is lost at this interface that would be available inside perception? Design an extended interface that preserves one piece of this information while remaining independent of the perception implementation.

Chapter 2

Mathematical and Machine Learning Foundations

2.1 Introduction

The algorithms that drive autonomous vehicles are implementations of a handful of mathematical ideas applied in concert. When a perception network hands a list of bounding boxes to a tracker, the tracker is running a Kalman filter—linear-Gaussian Bayesian inference wrapped around a motion model. When a LiDAR SLAM module registers a new scan against a map, it is solving a nonlinear least-squares problem on the Lie group $SE(3)$, with Jacobians computed through Rodrigues-formula perturbation. When a planner outputs a reference trajectory for a lane change, the trajectory is the solution of a small quadratic program whose cost and constraints encode comfort, safety, and the kinematic limits of a car. When the control layer tracks that trajectory, it is doing model predictive control—again a QP, with updated Jacobians every few milliseconds. Every layer of the stack, perception through control, is a specialization of the material this chapter develops.

The mathematical toolkit for autonomous driving is distinctive in three ways. First, it is dominated by *geometry* in a way that most machine-learning curricula do not emphasize. Pose, rigid transform, tangent space, and Jacobian-on-a-manifold are not exotic topics but daily calculations. A bug that confuses two frames silently corrupts every downstream decision; a bug that applies a Euler-angle formula near gimbal lock silently explodes the covariance. Second, the toolkit is fundamentally *probabilistic*. Every sensor is noisy, every predicted future is uncertain, every plan must be safe not only in expectation but with high probability. Kalman filters, particle filters, Gaussian mixture models, and chance constraints are not optional background—they are the bread and butter of the working engineer. Third, the toolkit must be *computationally efficient*. A beautiful algorithm that takes a second per frame at desk-scale is worthless on a car that must close its perception-planning-control loop in a hundred milliseconds. Sparse linear algebra, quantization, mixed precision, and careful architecture choices are part of the mathematics of the field, not afterthoughts.

This chapter is organized by the order in which the ideas enter a typical AV pipeline. We begin

with rigid-body geometry, because every pose of the ego vehicle, every object detection, every sensor extrinsic, and every map element is represented by elements of $SO(3)$ or $SE(3)$ (or their 2D counterparts). Then we cover probability and Bayesian estimation, because every perception and localization module estimates hidden state from noisy measurements. Next comes optimization—gradient-based for learning, least-squares for estimation, quadratic programming for planning and control. We then develop the deep-learning substrate: initialization, normalization, convolution, attention, residual connections, the architectural patterns that show up in every perception paper. After that we treat graph-structured models (because agent-interaction graphs, lane graphs, and factor graphs are all special cases), uncertainty quantification (because safety requires more than a point estimate), information theory (because losses and objectives are information-theoretic in disguise), and the computational realities of running all of this on automotive silicon. The chapter closes with a consolidated summary and an expanded exercise set.

Throughout, each formula is accompanied by either a concrete worked example or a pointer to where it is worked out downstream, and each subsection explicitly ties its material to the AV pipeline. Readers comfortable with this machinery can skim for notation; readers new to robotics or deep learning should read linearly, because later chapters assume fluency with everything introduced here.

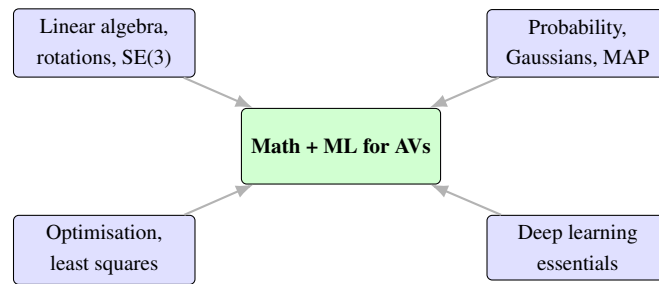


Figure 2.1: Chapter roadmap. Four pillars feed every subsequent chapter: rigid-body geometry for poses, probability for sensor fusion, optimisation for SLAM and control, and deep learning for perception, prediction, and policy.

2.2 Rigid-Body Geometry

An autonomous vehicle spends its life computing *where things are*. A detected car is at a pose in the world frame. A LiDAR is at a pose in the body frame. A lane segment lives in the map frame. A pedestrian’s predicted trajectory is a sequence of poses indexed by future time. Every one of these is an element of a Lie group— $SO(3)$ for rotations, $SE(3)$ for rigid transforms, or the 2D analogues $SO(2)$ and $SE(2)$ for planar driving—and the calculus we do on those groups is not optional bookkeeping. Errors in geometry propagate silently through the stack: a one-degree yaw error in sensor calibration produces a meter of lateral displacement at 60 m range. The algorithms of subsequent chapters, from PnP and sensor calibration through pose-graph SLAM, BEV perception, and MPC, all rest on this foundation.

2.2.1 Rotations in Three Dimensions

The group of rotations of $\mathbb{R}^3$ is the special orthogonal group

$$\text{SO}(3) = \{ \mathbf{R} \in \mathbb{R}^{3 \times 3} : \mathbf{R}^\top \mathbf{R} = \mathbf{I}, \det \mathbf{R} = +1 \}. \quad (2.1)$$

Elements of $\text{SO}(3)$ have three degrees of freedom; over-parameterization is inevitable. Common representations and their trade-offs are summarized in Table 2.1.

Table 2.1: Rotation representations and their AV use cases.

Representation	Parameters	Notes and typical AV use
Rotation matrix	9 (6 constraints)	Closed under multiplication; no singularities; memory-heavy. Stored for sensor extrinsics and used in BEV projection.
Euler angles	3	Intuitive (roll, pitch, yaw); gimbal lock at $\theta_2 = \pm\pi/2$. Used in IMU reports and vehicle telemetry where pitch stays small.
Axis-angle ($\boldsymbol{\omega}$)	3	Tangent at identity; minimum-DoF perturbation for optimization. Underlies Rodrigues formula and Lie-algebra Jacobians.
Unit quaternion (q_w, q_x, q_y, q_z)	4 (1 constraint)	Double cover of $\text{SO}(3)$; no singularities; composable. Standard for attitude filters (EKF/UKF) in VINS-style VIO.

Why three representations. An AV stack needs all of them. Rotation matrices are the workhorse for actually applying a rotation to a point cloud or a bounding box. Euler angles are how humans specify calibrations and how many sensor telemetry streams report attitude. Quaternions are the preferred internal state for attitude filters because they avoid gimbal lock and compose cleanly without accumulating numerical drift as fast. Axis-angle vectors are the preferred *perturbation* coordinates: when we do nonlinear optimization over rotations (bundle adjustment, pose-graph SLAM), we take steps in the three-dimensional tangent space and apply them via the exponential map below.

The Rodrigues formula. The *Rodrigues formula* gives the exponential map $\exp : \mathfrak{so}(3) \rightarrow \text{SO}(3)$:

$$\mathbf{R}(\boldsymbol{\omega}) = \mathbf{I} + \frac{\sin \theta}{\theta} [\boldsymbol{\omega}]_{\times} + \frac{1 - \cos \theta}{\theta^2} [\boldsymbol{\omega}]_{\times}^2, \quad (2.2)$$

where $\theta = \|\boldsymbol{\omega}\|$ and $[\boldsymbol{\omega}]_{\times}$ is the skew-symmetric matrix

$$[\boldsymbol{\omega}]_{\times} = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}. \quad (2.3)$$

Example 2.1 (Rodrigues: 10° yaw correction applied to LiDAR points). A LiDAR-IMU calibration step reveals that the LiDAR's frame is rotated 10° about the body's z -axis (yaw).

Step 1: Rotation parameters.

$$\begin{aligned}\theta &= 10^\circ = 0.1745 \text{ rad}, \\ \sin \theta &\approx 0.1736, \\ \cos \theta &\approx 0.9848.\end{aligned}$$

Step 2: Assemble rotation matrix from Eq. (2.2).

$$\mathbf{R} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \approx \begin{bmatrix} 0.985 & -0.174 & 0 \\ 0.174 & 0.985 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Step 3: Apply to a LiDAR return at $\mathbf{p}^L = (30, 0, 0.5) \text{ m}$ (a point 30 m directly ahead).

$$\begin{aligned}\mathbf{p}^B &= \mathbf{R}\mathbf{p}^L \\ &\approx (30 \cdot 0.985, 30 \cdot 0.174, 0.5) \\ &\approx (29.54, 5.21, 0.5) \text{ m}.\end{aligned}$$

Interpretation. An uncorrected point cloud would place this target 5.2 m off-lane at 30 m range—a gross failure for any downstream tracker. Per 1° of yaw miscalibration, the lateral error at 30 m is approximately 0.52 m. Extrinsic calibration must be done to arc-minute accuracy for long-range sensors.

Example 2.2 (Rodrigues degenerating at $\theta = 0$). For very small rotations both coefficients in Eq. (2.2) become ill-defined ($0/0$). Taylor expansions give $\sin \theta / \theta \rightarrow 1 - \theta^2/6$ and $(1 - \cos \theta) / \theta^2 \rightarrow 1/2 - \theta^2/24$. For IMU preintegration at 1 kHz, a single-sample rotation is on the order of $\theta \sim 10^{-4}$ rad, squarely in the small-angle regime. A naive implementation that computes $\sin \theta / \theta$ directly in single precision produces catastrophic cancellation; production code branches to the Taylor series for $\theta < 10^{-7}$. Failure to do so has been documented as the proximate cause of at least one open-source VIO divergence bug.

The inverse operation—recovering $\boldsymbol{\omega}$ from $\mathbf{R}$ —uses the matrix logarithm and is called the *log map*:

$$\theta = \arccos\left(\frac{\text{tr}(\mathbf{R}) - 1}{2}\right), \quad [\boldsymbol{\omega}]_{\times} = \frac{\theta}{2 \sin \theta} (\mathbf{R} - \mathbf{R}^{\top}). \quad (2.4)$$

Both Rodrigues and its inverse are singular near $\theta = 0$ and $\theta = \pi$; stable implementations switch to Taylor expansions in those regimes.

Example 2.3 (Log map of a pose increment). Two consecutive IMU keyframe attitudes differ by rotation $\mathbf{R}_{k,k+1}$ with $\text{tr}(\mathbf{R}_{k,k+1}) = 2.98$. Then $\theta = \arccos((2.98 - 1)/2) = \arccos(0.99) \approx 0.1415 \text{ rad} \approx 8.1^\circ$. The axis is extracted from the skew-symmetric part $(\mathbf{R} - \mathbf{R}^\top)/(2 \sin \theta)$. This is the canonical operation for turning a rotation matrix into a tangent-space vector that the optimizer can perturb.

2.2.2 Quaternions

Unit quaternions avoid the singularities of Euler angles and use one fewer parameter than a rotation matrix. A quaternion $q = (q_w, q_x, q_y, q_z)$ represents a rotation by angle θ about axis $\hat{\mathbf{n}}$ as

$$q = (\cos(\theta/2), \hat{\mathbf{n}} \sin(\theta/2)). \quad (2.5)$$

Example 2.4 (Quaternion for a 90° yaw). Represent a 90° rotation about the z -axis as a unit quaternion and verify by applying it to a unit vector.

Step 1: Build the quaternion.

$$\begin{aligned} \hat{\mathbf{n}} &= (0, 0, 1), \quad \theta = \pi/2, \\ q &= (\cos(\theta/2), \hat{\mathbf{n}} \sin(\theta/2)) \\ &= (\cos(\pi/4), 0, 0, \sin(\pi/4)) \\ &= (0.7071, 0, 0, 0.7071). \end{aligned}$$

Step 2: Apply to the point $\mathbf{p} = (1, 0, 0)$.

$$\begin{aligned} \mathbf{p}' &= q \otimes (0, 1, 0, 0) \otimes q^{-1} \\ &= (0, 1, 0). \end{aligned}$$

The expected 90° yaw result.

Storage comparison.

A quaternion attitude is 4 floats; a rotation matrix is 9 floats. On a VIO filter with a few hundred keyframes this is not the memory saving that matters. The saving that matters is *not having to renormalize nine entries with six constraints*.

Composition is quaternion multiplication $q_3 = q_1 \otimes q_2$; the action of q on a point $\mathbf{p}$ is $q \otimes (0, \mathbf{p}) \otimes q^{-1}$. Numerical implementations must periodically renormalize $q \leftarrow q/\|q\|$ to compensate for floating-point drift from unit-sphere constraint.

A subtlety: q and $-q$ represent the same rotation (the double-cover property). When interpolating or learning in quaternion space, this ambiguity must be resolved—typically by enforcing $q_w \geq 0$, or by using the quaternion logarithm as a three-parameter tangent representation that has no sign

ambiguity near identity. For an AV, the most common place this bites is in learned heading regression: a pedestrian facing slightly left of forward is near $q_w = 1$ in one chart and near $q_w = -1$ in the other, producing a huge apparent loss gradient for a tiny physical mismatch. Standard fix: regress the first two columns of the rotation matrix and Gram–Schmidt orthogonalize.

2.2.3 Rigid Transforms

The special Euclidean group $SE(3)$ combines rotation and translation:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3), \quad \mathbf{R} \in SO(3), \mathbf{t} \in \mathbb{R}^3. \quad (2.6)$$

Composition is matrix multiplication: $\mathbf{T}_{AC} = \mathbf{T}_{AB}\mathbf{T}_{BC}$. The subscripts encode frames: $\mathbf{T}_{WB}$ denotes the pose of frame B expressed in frame W , and transforms a point $\mathbf{p}^B$ as $\mathbf{p}^W = \mathbf{R}_{WB}\mathbf{p}^B + \mathbf{t}_{WB}$. The inverse is

$$\mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}. \quad (2.7)$$

Example 2.5 (Transforming a LiDAR point to the world frame). LiDAR-to-body extrinsic: $\mathbf{R}_{BL} = \mathbf{I}$ (aligned axes), $\mathbf{t}_{BL} = (1.5, 0, 1.8)$ m (LiDAR mounted 1.5 m ahead of body origin, 1.8 m above ground). Body-to-world pose from localization: $\mathbf{R}_{WB} = \text{yaw}(30^\circ)$, $\mathbf{t}_{WB} = (100, 50, 0)$ m. A LiDAR return at $\mathbf{p}^L = (20, 3, -1)$ m is first mapped to the body frame as $\mathbf{p}^B = \mathbf{p}^L + (1.5, 0, 1.8) = (21.5, 3, 0.8)$ m, then to the world frame as $\mathbf{p}^W = \mathbf{R}_{WB}\mathbf{p}^B + \mathbf{t}_{WB}$. With $\mathbf{R}_{WB}$ from Example 2.1’s pattern scaled to 30° ($\cos \approx 0.866$, $\sin \approx 0.5$), $\mathbf{p}^W \approx (100 + 0.866 \cdot 21.5 - 0.5 \cdot 3, 50 + 0.5 \cdot 21.5 + 0.866 \cdot 3, 0.8) = (117.1, 63.4, 0.8)$ m. This chain $\mathbf{p}^L \rightarrow \mathbf{p}^B \rightarrow \mathbf{p}^W$ is executed for every one of several hundred thousand LiDAR points per frame; it is the most frequently-evaluated geometric operation in any AV stack.

Definition 2.6 (Twist and adjoint). A twist $\xi = (\omega, \mathbf{v}) \in \mathbb{R}^6$ parameterizes $SE(3)$ near identity via the exponential map $\exp : \mathfrak{se}(3) \rightarrow SE(3)$. The adjoint $\text{Ad}_{\mathbf{T}} \in \mathbb{R}^{6 \times 6}$ maps twists between frames: if $\mathbf{T}_{AB}$ is the pose of B in A , then a twist ξ^B expressed in B becomes $\text{Ad}_{\mathbf{T}_{AB}} \xi^B$ in A . The adjoint will reappear in SLAM Jacobians (Chapter 13).

2.2.4 Perturbations and Jacobians on Lie Groups

A standard pattern throughout the book: parameterize small perturbations of a pose $\mathbf{T}$ by a six-vector ξ via $\mathbf{T} \leftarrow \exp(\xi^\wedge) \mathbf{T}$ (left perturbation) or $\mathbf{T} \leftarrow \mathbf{T} \exp(\xi^\wedge)$ (right perturbation), where $\xi^\wedge$ is the skew-symmetric form of the twist. Jacobians of residuals with respect to ξ replace classical multivariate Jacobians; their structure underpins the sparse SLAM Jacobians of Chapter 13.

For a point $\mathbf{p}^W = \mathbf{R}_{WB}\mathbf{p}^B + \mathbf{t}_{WB}$ under a right perturbation $\xi = (\phi, \rho)$ of $\mathbf{T}_{WB}$, to first order,

$$\mathbf{p}^W \mapsto \mathbf{p}^W + \mathbf{R}_{WB}\rho - \mathbf{R}_{WB}[\mathbf{p}^B]_\times \phi, \quad (2.8)$$

giving Jacobian

$$\frac{\partial \mathbf{p}^W}{\partial \boldsymbol{\xi}} = \begin{bmatrix} \mathbf{R}_{WB} & -\mathbf{R}_{WB}[\mathbf{p}^B]_{\times} \end{bmatrix}. \quad (2.9)$$

Example 2.7 (Jacobian of a LiDAR residual). A map point $\mathbf{q}^W = (117, 63, 0.8)$ m is matched to a live LiDAR point at body-frame position $\mathbf{p}^B = (21.5, 3, 0.8)$ m. The residual $\mathbf{r}(\mathbf{T}_{WB}) = \mathbf{T}_{WB}\mathbf{p}^B - \mathbf{q}^W$ has Jacobian given by Eq. (2.9) with $\mathbf{R}_{WB}$ the current estimate. Taking $\mathbf{R}_{WB} = \mathbf{I}$ for simplicity, $[\mathbf{p}^B]_{\times} = \begin{pmatrix} 0 & -0.8 & 3 \\ 0.8 & 0 & -21.5 \\ -3 & 21.5 & 0 \end{pmatrix}$, and

$$\frac{\partial \mathbf{r}}{\partial \boldsymbol{\xi}} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0.8 & -3 \\ 0 & 1 & 0 & -0.8 & 0 & 21.5 \\ 0 & 0 & 1 & 3 & -21.5 & 0 \end{bmatrix}.$$

The 21.5 entries reveal why yaw miscalibration amplifies with range: a one-milliradian yaw perturbation produces a 21.5 mm residual change. Stacking many such Jacobians yields the sparse matrix SLAM actually solves.

2.2.5 The Baker–Campbell–Hausdorff Relation

When composing two small rotations $\exp(\boldsymbol{\omega}_1^{\times})\exp(\boldsymbol{\omega}_2^{\times}) = \exp(\boldsymbol{\omega}_3^{\times})$, the composed angle is *not* simply $\boldsymbol{\omega}_1 + \boldsymbol{\omega}_2$. The BCH formula gives the leading corrections:

$$\boldsymbol{\omega}_3 = \boldsymbol{\omega}_1 + \boldsymbol{\omega}_2 + \frac{1}{2}\boldsymbol{\omega}_1 \times \boldsymbol{\omega}_2 + \frac{1}{12}\boldsymbol{\omega}_1 \times (\boldsymbol{\omega}_1 \times \boldsymbol{\omega}_2) + \mathcal{O}(\|\boldsymbol{\omega}\|^4). \quad (2.10)$$

Example 2.8 (BCH quadratic correction for IMU integration). Two successive 1 ms IMU updates report $\boldsymbol{\omega}_1 = (0, 0.01, 0)$, $\boldsymbol{\omega}_2 = (0.01, 0, 0)$ rad (order-of-magnitude 10°s^{-1} rates). The naive sum is $(0.01, 0.01, 0)$. The BCH correction is $\frac{1}{2}\boldsymbol{\omega}_1 \times \boldsymbol{\omega}_2 = \frac{1}{2}(0.01 \cdot 0 - 0 \cdot 0.01, \dots, 0.01 \cdot 0.01 - 0 \cdot 0) = (0, 0, 0.00005)$. Small in one step, but accumulated over a kilometer of driving at 30 m s^{-1} (≈ 33 seconds, 3.3×10^4 samples), the term grows to ~ 1.65 rad of uncorrected yaw drift if ignored in a naïve accumulator. IMU preintegration [22] retains the quadratic terms exactly.

2.2.6 Ground-Vehicle Simplifications

Most road-vehicle calculations operate on $\text{SE}(2)$ —rotation about the vertical axis plus 2D translation—because pitch and roll are small. An $\text{SE}(2)$ pose has three degrees of freedom (x, y, θ) , composes as

$$\begin{bmatrix} x_3 \\ y_3 \\ \theta_3 \end{bmatrix} = \begin{bmatrix} x_1 + \cos \theta_1 x_2 - \sin \theta_1 y_2 \\ y_1 + \sin \theta_1 x_2 + \cos \theta_1 y_2 \\ \theta_1 + \theta_2 \end{bmatrix}, \quad (2.11)$$

and suffices for planning, many control applications, and BEV perception. Vehicle pitch and roll become important for camera-to-LiDAR calibration, aggressive dynamics, and banked roads.

Example 2.9 (SE(2) composition for a parking maneuver). An ego pose $(x_1, y_1, \theta_1) = (10, 5, \pi/4)$ m, m, rad. A planned relative motion $(x_2, y_2, \theta_2) = (2, 0, -\pi/4)$ m, m, rad (2 m forward, straighten heading). Using Eq. (2.11), $\cos(\pi/4) = \sin(\pi/4) = 0.7071$: $x_3 = 10 + 0.7071 \cdot 2 - 0.7071 \cdot 0 = 11.41$, $y_3 = 5 + 0.7071 \cdot 2 + 0.7071 \cdot 0 = 6.41$, $\theta_3 = 0$. The car ends at (11.4, 6.4) with heading zero. A controller tracking a sequence of such increments is doing SE(2) arithmetic every control tick.

2.2.7 Frame Conventions

Consistent frame conventions prevent entire classes of silent errors.

- **Vehicle/body frame (ISO 8855).** x forward (toward driver's sight line), y left, z up. Origin typically at the center of the rear axle.
- **Camera frame.** z forward (out of the lens), x right, y down—the opposite y -convention from the body frame.
- **LiDAR frame.** Sensor-specific; typically x forward, y left, z up like the vehicle.
- **World frame.** Local tangent-plane ENU (east-north-up) or NED (north-east-down); choice depends on GNSS and map conventions.
- **UTM / WGS84.** Global frames used by maps; local calculations are performed in a tangent-plane approximation for numerical conditioning.

Every coordinate transform between these frames is a $\mathbf{T} \in \text{SE}(3)$ estimated by calibration (Chapter 10). The bookkeeping of who-is-in-what-frame is a nontrivial source of bugs; most production stacks maintain an explicit transform tree with typed frame IDs.

2.3 Probability and Estimation

The geometric machinery just developed tells us how to *represent* positions, orientations, and the relationships between frames. It does not tell us how to *infer* them from noisy sensor data, which is what every perception and localization module must actually do. Probability theory is the language of that inference.

Every AV module estimates something hidden—agent position from pixels, ego pose from IMU, a neighbor's intent from trajectory history—from observations that are noisy, partial, or both. Probability theory is how we manage this hiddenness rigorously. The Kalman filter tracks objects; the particle filter bootstraps LiDAR localization; Gaussian mixture models parameterize multi-modal predictions; χ^2 tests gate spurious associations in trackers. The subsections that follow develop each building block and connect it to the module that uses it.

2.3.1 Gaussian Distributions

The multivariate Gaussian $\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$ with $\boldsymbol{\Sigma} \succ 0$ is defined by

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^n \det \boldsymbol{\Sigma}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right). \quad (2.12)$$

Example 2.10 (Gaussian density at a track's predicted location). A tracker believes a car to be at $\boldsymbol{\mu} = (50, -2)$ m with covariance $\boldsymbol{\Sigma} = \text{diag}(4, 1)$ m² (larger uncertainty along-range than across).

Candidate detection at $\mathbf{x} = (52, -2.5)$.

$$\begin{aligned} d_M^2 &= \frac{(52 - 50)^2}{4} + \frac{(-2.5 + 2)^2}{1} \\ &= \frac{4}{4} + \frac{0.25}{1} \\ &= 1.25, \\ p(\mathbf{x}) &\propto e^{-0.625} \approx 0.535. \end{aligned}$$

Candidate detection at $\mathbf{x} = (52, -3.0)$.

$$\begin{aligned} d_M^2 &= \frac{4}{4} + \frac{1}{1} = 2, \\ p(\mathbf{x}) &\propto e^{-1} \approx 0.368. \end{aligned}$$

Gaussian densities decay exponentially in the squared Mahalanobis distance; a tracker's gating test rejects anything beyond a fixed chi-squared threshold.

Two key closure properties are used repeatedly.

Lemma 2.11 (Linear Gaussian closure). *If $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ and $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b} + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$, independent of $\mathbf{x}$, then $\mathbf{y} \sim \mathcal{N}(\mathbf{A}\boldsymbol{\mu} + \mathbf{b}, \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top + \mathbf{Q})$.*

Example 2.12 (Propagating position-velocity covariance forward). State $\mathbf{x} = (p, v)$, $\boldsymbol{\mu} = (0, 10)$ m, m/s, $\boldsymbol{\Sigma} = \text{diag}(0.25, 1)$. Constant-velocity dynamics over $\Delta t = 0.1$ s: $\mathbf{A} = \begin{pmatrix} 1 & 0.1 \\ 0 & 1 \end{pmatrix}$, $\mathbf{Q} = \text{diag}(0.001, 0.01)$ (process noise). By Lemma 2.11,

$$\mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top = \begin{pmatrix} 0.25 + 0.01 & 0.1 \\ 0.1 & 1 \end{pmatrix} = \begin{pmatrix} 0.26 & 0.1 \\ 0.1 & 1 \end{pmatrix},$$

so the predicted covariance is $\boldsymbol{\Sigma}' = \begin{pmatrix} 0.261 & 0.1 \\ 0.1 & 1.01 \end{pmatrix}$. Position variance grew from 0.25 m² to 0.261 m², and a position–velocity correlation of $0.1 / \sqrt{0.261 \cdot 1.01} \approx 0.19$ has appeared. This correlation is what lets a position measurement tighten the velocity estimate—the Kalman filter's essential magic.

Lemma 2.13 (Conditioning). *Partition $\mathbf{x} = (\mathbf{x}_a, \mathbf{x}_b)$ with mean $\boldsymbol{\mu} = (\boldsymbol{\mu}_a, \boldsymbol{\mu}_b)$ and block covariance $\boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{aa} & \boldsymbol{\Sigma}_{ab} \\ \boldsymbol{\Sigma}_{ba} & \boldsymbol{\Sigma}_{bb} \end{pmatrix}$. Then $p(\mathbf{x}_a | \mathbf{x}_b)$ is Gaussian with*

$$\boldsymbol{\mu}_{a|b} = \boldsymbol{\mu}_a + \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}(\mathbf{x}_b - \boldsymbol{\mu}_b), \quad (2.13)$$

$$\boldsymbol{\Sigma}_{a|b} = \boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba}. \quad (2.14)$$

Example 2.14 (Conditioning on a noisy position measurement). Pre-measurement state from Example 2.12: $\boldsymbol{\mu} = (1, 10)$, $\boldsymbol{\Sigma} = \begin{pmatrix} 0.261 & 0.1 \\ 0.1 & 1.01 \end{pmatrix}$. A scalar position measurement $y = 1.05$ with noise variance $R = 0.04$ arrives.

Step 1: Innovation covariance (scalar case).

$$\begin{aligned} S &= \boldsymbol{\Sigma}_{pp} + R \\ &= 0.261 + 0.04 = 0.301. \end{aligned}$$

Step 2: Kalman gains.

$$\begin{aligned} K_p &= \boldsymbol{\Sigma}_{pp}/S = 0.261/0.301 \approx 0.867, \\ K_v &= \boldsymbol{\Sigma}_{vp}/S = 0.1/0.301 \approx 0.332. \end{aligned}$$

Step 3: Mean update.

$$\begin{aligned} \hat{p} &= \mu_p + K_p(y - \mu_p) \\ &= 1 + 0.867 \cdot 0.05 \\ &= 1.043, \\ \hat{v} &= \mu_v + K_v(y - \mu_p) \\ &= 10 + 0.332 \cdot 0.05 \\ &= 10.017. \end{aligned}$$

Step 4: Covariance update.

$$\begin{aligned} \hat{\boldsymbol{\Sigma}}_{pp} &= \boldsymbol{\Sigma}_{pp} - \boldsymbol{\Sigma}_{pp}^2/S \\ &= 0.261 - 0.261^2/0.301 \\ &\approx 0.0347, \\ \hat{\boldsymbol{\Sigma}}_{vv} &= \boldsymbol{\Sigma}_{vv} - \boldsymbol{\Sigma}_{vp}^2/S \\ &= 1.01 - 0.1^2/0.301 \\ &\approx 0.977. \end{aligned}$$

Interpretation. Position variance collapsed by roughly $7\times$; velocity variance barely moved—as expected from a single position observation.

The *Mahalanobis distance* between a sample $\mathbf{x}$ and a Gaussian $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ is

$$d_M^2(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}), \quad (2.15)$$

which under the Gaussian follows a χ_n^2 distribution with n degrees of freedom. Thresholding d_M^2

against a χ^2 quantile is the standard *gating* test in tracking and SLAM, rejecting unlikely associations before they corrupt downstream estimates.

Example 2.15 (χ^2 gating a detection to a track). A track predicts a detection at $\boldsymbol{\mu} = (52, 0)$ with covariance $\boldsymbol{\Sigma} = \text{diag}(4, 1)$ (2D bounding-box center in pixels).

Candidate A: Detection arrives at $\mathbf{x} = (55, 2)$.

$$\begin{aligned} d_M^2 &= (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \\ &= \frac{(55 - 52)^2}{4} + \frac{(2 - 0)^2}{1} \\ &= \frac{9}{4} + \frac{4}{1} \\ &= 6.25. \end{aligned}$$

The χ^2 threshold at the 99 % confidence level is 9.21. Since $6.25 < 9.21$, the gate passes and the match is accepted.

Candidate B: Detection arrives at $\mathbf{x} = (58, 2)$.

$$\begin{aligned} d_M^2 &= \frac{(58 - 52)^2}{4} + \frac{(2 - 0)^2}{1} \\ &= \frac{36}{4} + 4 \\ &= 13. \end{aligned}$$

Now $13 > 9.21$; the gate rejects.

Gating thresholds are how SORT-family trackers avoid catastrophically mis-associating a pedestrian onto a car track; the threshold is tuned against false-rejection rate on the tracker's validation set.

2.3.2 Gaussian Mixture Models

A GMM represents a distribution as a weighted sum of Gaussians:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad \sum_k \pi_k = 1. \quad (2.16)$$

GMMs are the standard device for representing multi-modal predictions (Chapter 17) and for modeling map-point distributions (Chapter 14). Parameters are typically fit via expectation-maximization or learned as outputs of a neural network with a mixture density head.

Example 2.16 (GMM mode probabilities at an intersection). A prediction head outputs three modes for a vehicle approaching a T-intersection: left-turn, right-turn, proceed-straight. Mode weights $\pi = (0.4, 0.3, 0.3)$; mode means (at horizon τ) $\boldsymbol{\mu}_L = (30, 10)$, $\boldsymbol{\mu}_R = (30, -10)$, $\boldsymbol{\mu}_S =$

$(60, 0)$; covariances $0.5\mathbf{I}_2$ each. Evaluating Eq. (2.16) at an observation $\mathbf{x} = (31, 9)$ (close to left-turn): $\mathcal{N}(\boldsymbol{\mu}_L) \propto e^{-(1+1)/1} \approx 0.135$, the other two contribute e^{-200} -ish. So $p(\mathbf{x}) \approx 0.4 \cdot 0.135 / (2\pi \cdot 0.5) \approx 0.0172$, dominated by the left-turn component. A downstream planner reasons with the whole mixture, not just the argmax, which is why the mode weights themselves matter.

2.3.3 The EM Algorithm

Expectation-maximization fits latent-variable models by alternating a posterior step and a parameter step. For a GMM:

1. **E-step.** Compute responsibilities

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}. \quad (2.17)$$

2. **M-step.** Update $\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$ by the weighted moments $\sum_i \gamma_{ik} \mathbf{x}_i$ etc.

EM monotonically increases a lower bound on the log-likelihood. Its general form underlies variational inference and many semi-supervised methods. In AV pipelines EM is the workhorse for offline fitting of GMM-based lane-change behavior models and for clustering LiDAR returns during ground-plane segmentation.

Example 2.17 (One EM E-step). Two-component 1D GMM, $\pi = (0.5, 0.5)$, $\mu = (0, 10)$, $\sigma = (1, 2)$. A sample $x = 3$. $\mathcal{N}(3; 0, 1) \propto e^{-4.5} \approx 0.0111$; $\mathcal{N}(3; 10, 2) \propto e^{-49/8} \approx 0.00216$. Responsibilities: $\gamma_1 = 0.5 \cdot 0.0111 / (0.5 \cdot 0.0111 + 0.5 \cdot 0.00216) \approx 0.837$, $\gamma_2 \approx 0.163$. The sample is much more likely to belong to component 1 than 2, reflected in its responsibility.

2.3.4 Bayesian State Estimation

Given the system (1.1) and a belief $b_t(\mathbf{x}_t) = p(\mathbf{x}_t | \mathbf{y}_{0:t})$, Bayes updates are

$$\bar{b}_{t+1}(\mathbf{x}_{t+1}) = \int p(\mathbf{x}_{t+1} | \mathbf{x}_t, \mathbf{u}_t) b_t(\mathbf{x}_t) d\mathbf{x}_t, \quad (\text{predict}) \quad (2.18)$$

$$b_{t+1}(\mathbf{x}_{t+1}) \propto p(\mathbf{y}_{t+1} | \mathbf{x}_{t+1}) \bar{b}_{t+1}(\mathbf{x}_{t+1}). \quad (\text{update}) \quad (2.19)$$

Different assumptions reduce these to tractable algorithms: linear plus Gaussian yields the Kalman filter; nonlinear but Gaussian-approximable yields the extended or unscented Kalman filter; a sample-based approximation yields the particle filter. Every Bayesian estimator in subsequent chapters is a realization of one of these.

The Extended Kalman Filter

The EKF is the single most widely deployed Bayesian estimator in autonomous vehicles. It powers the inertial-navigation stack that fuses IMU, wheel odometry, and GNSS into the 200 Hz ego-pose estimate every other module depends on; it drives the per-track Kalman filters inside multi-object trackers (Chapter 11); and it localizes the vehicle against HD maps by fusing map-match residuals with

dead- reckoned priors (Chapter 14). The EKF's appeal is that it handles mildly nonlinear dynamics and measurements at the cost of one Jacobian evaluation per step, and its arithmetic is simple enough to verify by hand.

Linearize the dynamics and measurement models: $\mathbf{F}_t = \partial f / \partial \mathbf{x} |_{\hat{\mathbf{x}}_t}$ and $\mathbf{H}_t = \partial h / \partial \mathbf{x} |_{\hat{\mathbf{x}}_t}$. EKF equations:

$$\hat{\mathbf{x}}_{t+1}^- = f(\hat{\mathbf{x}}_t, \mathbf{u}_t, \mathbf{0}), \quad \mathbf{P}_{t+1}^- = \mathbf{F}_t \mathbf{P}_t \mathbf{F}_t^\top + \mathbf{Q}_t, \quad (2.20)$$

$$\mathbf{K}_{t+1} = \mathbf{P}_{t+1}^- \mathbf{H}_{t+1}^\top (\mathbf{H}_{t+1} \mathbf{P}_{t+1}^- \mathbf{H}_{t+1}^\top + \mathbf{R}_{t+1})^{-1}, \quad (2.21)$$

$$\hat{\mathbf{x}}_{t+1} = \hat{\mathbf{x}}_{t+1}^- + \mathbf{K}_{t+1} (\mathbf{y}_{t+1} - h(\hat{\mathbf{x}}_{t+1}^-)), \quad \mathbf{P}_{t+1} = (\mathbf{I} - \mathbf{K}_{t+1} \mathbf{H}_{t+1}) \mathbf{P}_{t+1}^-. \quad (2.22)$$

EKF is the workhorse of AV state estimation for IMU-GNSS-wheel fusion, despite well-known inconsistency issues (Chapter 13).

Example 2.18 (EKF step for a bicycle-model vehicle tracker). State $\mathbf{x} = (x, y, \theta)$, input $\mathbf{u} = (v, \delta)$ (speed and steering). Kinematic bicycle model:

$$f(\mathbf{x}, \mathbf{u}) = \begin{pmatrix} x + v\Delta t \cos \theta \\ y + v\Delta t \sin \theta \\ \theta + v\Delta t \tan \delta / L \end{pmatrix}.$$

Scenario: $\hat{\mathbf{x}} = (0, 0, 0)$, $\mathbf{u} = (10, 0.1)$, $\Delta t = 0.1$ s, wheelbase $L = 2.7$ m.

Step 1: State Jacobian.

$$\begin{aligned} \mathbf{F} &= \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}} \\ &= \begin{pmatrix} 1 & 0 & -v\Delta t \sin \theta \\ 0 & 1 & v\Delta t \cos \theta \\ 0 & 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}. \end{aligned}$$

Step 2: Covariance prediction.

With $\mathbf{P}_t = \text{diag}(0.1, 0.1, 0.01)$ and process noise $\mathbf{Q} = \text{diag}(0.01, 0.01, 0.001)$:

$$\mathbf{P}_{t+1}^- = \mathbf{F} \mathbf{P}_t \mathbf{F}^\top + \mathbf{Q}.$$

The (y, y) element inflates because heading uncertainty couples through the $v\Delta t \cos \theta$ Jacobian term:

$$[\mathbf{P}_{t+1}^-]_{yy} = 0.1 + 0.01 + 0.01 = 0.12.$$

Step 3: GPS measurement update.

A GPS measurement of (x, y) with noise $\mathbf{R} = \text{diag}(0.5, 0.5)$ feeds Eqs. (2.21)–(2.22), shrinking $\mathbf{P}$ on the (x, y) block.

This is the whole arithmetic of localization: linearize, predict, measure, correct.

The Unscented Kalman Filter

When the EKF's linearization becomes the dominant source of error—typically in VIO during aggressive cornering, in sharp braking under load transfer, or when the state includes attitude as quaternions—the unscented Kalman filter often pays for its extra cost. UKF is a staple of research-grade VINS-style visual-inertial systems and is increasingly found in production attitude filters where a handful of extra function evaluations per step buy noticeable consistency.

UKF replaces linearization with deterministic sigma-point sampling. It captures the posterior mean and covariance correctly to second order for arbitrary nonlinear f and h , and avoids analytical Jacobians. Cost is $2n + 1$ evaluations of the nonlinear function per step, which for moderate n is competitive with EKF. In AV practice, UKF is preferred over EKF for IMU-preintegration backends where the bicycle-model nonlinearities cause EKF to become inconsistent in aggressive maneuvers.

Particle Filters

Some AV problems break the Gaussian assumption that the EKF and UKF rely on. The canonical example is map-based localization on a highway where multiple positions along a straight road are nearly indistinguishable to LiDAR—the posterior is genuinely multi-modal until a distinguishing feature appears. Particle filters handle this cleanly by representing the posterior as a sample set rather than a parametric density. Modern LiDAR-map localization stacks use particle filters for initial bootstrap and then hand off to an EKF once the posterior has collapsed to a single mode.

When the posterior is multi-modal or highly non-Gaussian, particle filters represent the belief by a weighted sample set $\{(\mathbf{x}_t^{(i)}, w_t^{(i)})\}_{i=1}^N$. The recursion is:

1. **Predict.** Draw $\mathbf{x}_{t+1}^{(i)} \sim p(\mathbf{x}_{t+1} | \mathbf{x}_t^{(i)}, \mathbf{u}_t)$.
2. **Weight.** $w_{t+1}^{(i)} \propto w_t^{(i)} p(\mathbf{y}_{t+1} | \mathbf{x}_{t+1}^{(i)})$.
3. **Resample** when effective sample size

$$N_{\text{eff}} = \frac{1}{\sum_i (w^{(i)})^2} \quad (2.23)$$

falls below a threshold.

Particle filters scale poorly with state dimension (the curse of dimensionality—the number of particles required grows exponentially with $\dim(\mathbf{x})$) but shine for low-dimensional, highly non-Gaussian problems: map-based localization with strong priors (Chapter 14), multi-hypothesis association.

Example 2.19 (Effective sample size for a skewed particle set). $N = 100$ particles. Ninety have weight $w = 0.001$ (total 0.09) and ten have weight $w = 0.091$ (total 0.91). $\sum w_i^2 = 90 \cdot 10^{-6} + 10 \cdot 0.091^2 \approx 9 \times 10^{-5} + 0.0828 \approx 0.0829$. $N_{\text{eff}} = 1/0.0829 \approx 12$. With typical threshold $N/2 = 50$, the filter resamples, redrawing particles in proportion to weights. If we skipped the check the filter would drift toward representing only the top-10 hypotheses, a degeneracy pathology.

Information Filter Form

Pose-graph SLAM (Chapter 13) and visual-inertial bundle adjustment produce estimation problems whose natural representation is not a covariance but a sparse Hessian—the information matrix. The information filter is the dual of the Kalman filter in this representation: each new observation is an additive contribution to the Hessian, making landmark-addition and observation-folding operations structurally simple. GTSAM, g2o, and Ceres all operate on this representation internally, even when the user-facing API speaks in covariance terms.

Parameterizing a Gaussian by its *information matrix* $\mathbf{\Lambda} = \mathbf{\Sigma}^{-1}$ and *information vector* $\boldsymbol{\eta} = \mathbf{\Sigma}^{-1}\boldsymbol{\mu}$ makes measurement updates cheap

$$\mathbf{\Lambda} \leftarrow \mathbf{\Lambda} + \mathbf{H}^\top \mathbf{R}^{-1} \mathbf{H}, \quad \boldsymbol{\eta} \leftarrow \boldsymbol{\eta} + \mathbf{H}^\top \mathbf{R}^{-1} \mathbf{y}, \quad (2.24)$$

while making prediction expensive. Information form is natural for sparse factor-graph problems in SLAM and sensor fusion; GTSAM and Ceres operate effectively in information form on the Hessian $\mathbf{J}^\top \mathbf{J}$.

2.3.5 Robust Estimation

The Gaussian assumption is convenient but brittle: a single outlier can catastrophically bias a least-squares estimate. Robust M-estimators replace the quadratic loss with a loss $\rho(r)$ that grows sub-quadratically for large residuals. Common choices:

Huber.

$$\rho_{\text{Huber}}(r) = \begin{cases} r^2/2 & |r| \leq k, \\ k|r| - k^2/2 & |r| > k. \end{cases} \quad (2.25)$$

Cauchy.

$$\rho_{\text{Cauchy}}(r) = \frac{c^2}{2} \log(1 + (r/c)^2). \quad (2.26)$$

Tukey biweight. Zero influence beyond a cutoff; purely local convex optimization otherwise.

Example 2.20 (Huber vs quadratic loss at a PnP outlier). A PnP solver sees three reprojection residuals: $(0.2, 0.5, 15)$ px. The last is an outlier.

Quadratic loss:

$$\begin{aligned} L_{\text{quad}} &= 0.2^2 + 0.5^2 + 15^2 \\ &= 0.04 + 0.25 + 225 \\ &= 225.29. \end{aligned}$$

Dominated by the outlier.

Huber loss with $k = 2$: quadratic for $|r| \leq k$, linear beyond.

$$\begin{aligned} \rho_H(0.2) &= 0.5 \cdot 0.04 = 0.02, \\ \rho_H(0.5) &= 0.5 \cdot 0.25 = 0.125, \\ \rho_H(15) &= k|15| - 0.5k^2 = 30 - 2 = 28, \\ L_{\text{huber}} &= 0.02 + 0.125 + 28 \approx 28.1. \end{aligned}$$

Outlier contribution becomes linear, bounded influence.

Interpretation. The Huber-weighted Gauss–Newton step is insensitive to the outlier; a plain least-squares step would rotate the camera to partially minimize the 15 px residual, displacing the solution by several degrees.

An alternative paradigm is *voting*: RANSAC fits a hypothesis to a minimal sample and scores it by inlier count, repeating to find the hypothesis with most support. RANSAC is the default for feature-based geometry problems (fundamental matrix estimation, PnP) where the outlier rate can be 50 % or higher.

Example 2.21 (RANSAC iteration count). Fundamental-matrix estimation requires $s = 8$ correspondences per hypothesis. Inlier rate $w = 0.5$. Target success probability $p = 0.99$.

Probability of a clean minimal sample:

$$\begin{aligned} P(\text{all inliers}) &= w^s \\ &= 0.5^8 \\ &= 0.00391. \end{aligned}$$

Iterations for 99 % confidence:

$$\begin{aligned} N &= \frac{\log(1 - p)}{\log(1 - w^s)} \\ &= \frac{\log 0.01}{\log(1 - 0.00391)} \\ &\approx 1177. \end{aligned}$$

Wall-clock cost.

At $\sim 10\mu\text{s}$ per 8-point computation, RANSAC finishes in $\sim 1177 \times 10\mu\text{s} = 12\text{ms}$.

Sensitivity to inlier rate.

If we double effective inliers to $w = 0.7$:

$$\begin{aligned} N &\approx \log(0.01)/\log(1 - 0.7^8) \\ &\approx 70. \end{aligned}$$

Data-quality improvements cascade into runtime wins.

2.3.6 Calibration and Reliability

A classifier is *calibrated* if its predicted probability matches its empirical accuracy: among examples assigned confidence 0.9, about 90 % should be correct. Deep networks are typically under-calibrated (over-confident) in production. Post-hoc calibration methods—temperature scaling, isotonic regression, Platt scaling—fit a simple transform on a held-out set to correct the miscalibration without retraining.

Example 2.22 (Temperature scaling fix). A pedestrian classifier is 95 % confident on average, but empirical accuracy is only 82 %—overconfident. Temperature scaling searches for T minimizing NLL on the validation set. Typical result: $T^* \approx 1.4$. Applying softmax with $\hat{p}_c = e^{z_c/T^*} / \sum e^{z_{c'}/T^*}$ flattens the distribution; a former 95 % confidence becomes $\sim 86\%$, now aligned with true accuracy. Downstream prediction and planning consume these probabilities directly—an overconfident detector leads the prediction module to ignore plausible alternatives.

2.3.7 Conformal Prediction

Conformal prediction produces distribution-free, finite-sample coverage guarantees: given a nominal error level α and a held-out calibration set, conformal methods return prediction sets with coverage $\geq 1 - \alpha$ regardless of the underlying model or data distribution. In AV perception, conformal prediction over bounding-box regression or trajectory forecasts gives calibrated uncertainty bounds for downstream safety logic.

Example 2.23 (Conformal bounding-box inflation). An object detector outputs bounding boxes; the calibration set has $n = 1000$ samples with normalized residuals $s_i = (\text{IoU shortfall})$. For coverage $1 - \alpha = 0.9$, use the $\lceil (n+1)(1 - \alpha) \rceil = 901$ th ordered residual as $\hat{q}$. If $\hat{q} = 0.12$, every deployed box is inflated by 12 % in area; the resulting inflated box contains the true object with probability ≥ 0.9 . Downstream path planning consumes the inflated box and reserves margin, providing a distribution-free safety guarantee unavailable from raw softmax scores.

2.4 Optimization

Representation (geometry) and inference (probability) each reduce, when we take pen to paper, to optimization problems. The Kalman filter’s gain minimizes posterior covariance; its MAP interpretation is nonlinear least squares. A planner minimizes a cost over trajectories subject to vehicle-dynamics and collision constraints. A neural network trains by gradient descent on a loss. Optimization is therefore the third pillar of the chapter, and this section collects the workhorses every downstream chapter leans on.

Every learning system trains by optimization. Every SLAM backend factor-graph-optimizes. Every planner solves a QP or NLP every few tens of milliseconds. The sophistication level needed for AV work is modest—Gauss-Newton, Levenberg-Marquardt, PPO-style surrogate objectives, small convex QPs—but the volume is relentless.

2.4.1 Gradient Descent and its Variants

For a differentiable loss $L(\boldsymbol{\theta})$, gradient descent iterates $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \nabla L(\boldsymbol{\theta}_k)$. Stochastic gradient descent (SGD) uses a noisy estimate of the gradient from a minibatch. Adaptive methods—Adam [37] and its relatives—track first and second gradient moments:

$$\mathbf{m}_k = \beta_1 \mathbf{m}_{k-1} + (1 - \beta_1) \nabla L(\boldsymbol{\theta}_k), \quad (2.27)$$

$$\mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) \nabla L(\boldsymbol{\theta}_k)^{\odot 2}, \quad (2.28)$$

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \frac{\hat{\mathbf{m}}_k}{\sqrt{\hat{\mathbf{v}}_k + \varepsilon}}, \quad (2.29)$$

where $\hat{\cdot}$ denotes bias-corrected values and $\odot$ is elementwise square. AdamW decouples weight decay from the adaptive gradient, and is the de facto choice for transformer training.

Example 2.24 (One Adam step). A single scalar parameter $\theta_k = 0.5$ with gradient $\nabla L = 2$. Hyperparameters:

- Learning rate $\eta = 0.001$
- Momentum decay $\beta_1 = 0.9$
- Variance decay $\beta_2 = 0.999$
- Numerical floor $\varepsilon = 10^{-8}$

Initial moments $\mathbf{m}_0 = \mathbf{v}_0 = 0$; evaluate at $k = 1$.

Step 1: Update moments.

$$m_1 = \beta_1 m_0 + (1 - \beta_1) \nabla L = 0.1 \cdot 2 = 0.2,$$

$$v_1 = \beta_2 v_0 + (1 - \beta_2) (\nabla L)^2 = 0.001 \cdot 4 = 0.004.$$

Step 2: Bias-correct.

$$\begin{aligned}\hat{m}_1 &= \frac{m_1}{1 - \beta_1^k} = \frac{0.2}{0.1} = 2, \\ \hat{v}_1 &= \frac{v_1}{1 - \beta_2^k} = \frac{0.004}{0.001} = 4.\end{aligned}$$

Step 3: Parameter update.

$$\begin{aligned}\theta_1 &= \theta_0 - \eta \frac{\hat{m}_1}{\sqrt{\hat{v}_1} + \epsilon} \\ &= 0.5 - 0.001 \cdot \frac{2}{\sqrt{4}} \\ &= 0.5 - 0.001 \\ &= 0.499.\end{aligned}$$

The bias correction is why Adam takes a normal-sized step on the very first iteration rather than a microscopic one.

Learning-rate schedules are as important as the optimizer. Common patterns: linear warmup followed by cosine decay; step decay at fixed milestones; reduce-on-plateau. For supervised perception training warmup of 500–2000 iterations is standard practice.

2.4.2 Newton’s Method, Line Search, and Trust Regions

Newton’s method uses the Hessian $\mathbf{H} = \nabla^2 L$:

$$\Delta \theta = -\mathbf{H}^{-1} \nabla L. \quad (2.30)$$

Near a minimum this converges quadratically. Far from a minimum the Hessian may not be positive-definite, motivating line search, trust regions, or Hessian modifications.

2.4.3 Gauss–Newton and Levenberg–Marquardt

Many robotics problems reduce to nonlinear least squares $\min_{\theta} \frac{1}{2} \|\mathbf{r}(\theta)\|_2^2$. Let $\mathbf{J} = \partial \mathbf{r} / \partial \theta$. Gauss–Newton solves

$$(\mathbf{J}^\top \mathbf{J}) \Delta \theta = -\mathbf{J}^\top \mathbf{r}, \quad (2.31)$$

and updates $\theta \leftarrow \theta + \Delta \theta$. Levenberg–Marquardt damps ill-conditioning:

$$(\mathbf{J}^\top \mathbf{J} + \lambda \text{diag}(\mathbf{J}^\top \mathbf{J})) \Delta \theta = -\mathbf{J}^\top \mathbf{r}. \quad (2.32)$$

These methods underlie bundle adjustment, pose-graph optimization, and factor-graph SLAM. Scale: a modern bundle-adjustment problem may have 10^5 – 10^6 variables; exploiting the sparsity of $\mathbf{J}$ is the difference between minutes and infeasibility.

Example 2.25 (Gauss-Newton step on a scalar exponential fit). Fit $y = \theta_0 e^{\theta_1 x}$ to the three data points $(x_i, y_i) = \{(0, 1), (1, 2.8), (2, 7.0)\}$, starting from $\boldsymbol{\theta}^{(0)} = (1, 1)$.

Step 1: Compute predictions and residuals.

At $\boldsymbol{\theta}^{(0)} = (1, 1)$ the model gives $y_i = e^{x_i}$:

$$\hat{y} = (e^0, e^1, e^2) = (1, 2.72, 7.39).$$

Residuals $r_i = y_i - \hat{y}_i$:

$$\mathbf{r} = (0, 0.08, -0.39).$$

Step 2: Assemble the Jacobian.

Row-wise, $\partial r_i / \partial \boldsymbol{\theta} = (-e^{x_i}, -\theta_0 x_i e^{x_i})$:

$$\mathbf{J} = \begin{pmatrix} -1 & 0 \\ -2.72 & -2.72 \\ -7.39 & -14.78 \end{pmatrix}.$$

Step 3: Form the normal equations.

Compute $\mathbf{J}^\top \mathbf{J}$:

$$\begin{aligned} \mathbf{J}^\top \mathbf{J} &= \begin{pmatrix} 1 + 7.39 + 54.6 & 0 + 7.39 + 109.2 \\ 0 + 7.39 + 109.2 & 0 + 7.39 + 218.4 \end{pmatrix} \\ &= \begin{pmatrix} 63 & 117 \\ 117 & 226 \end{pmatrix}. \end{aligned}$$

Compute $\mathbf{J}^\top \mathbf{r}$:

$$\mathbf{J}^\top \mathbf{r} \approx (2.66, -6.0).$$

Step 4: Solve $(\mathbf{J}^\top \mathbf{J})\Delta = -\mathbf{J}^\top \mathbf{r}$.

The 2×2 determinant:

$$\begin{aligned} \det &= 63 \cdot 226 - 117^2 \\ &= 14238 - 13689 \\ &= 549. \end{aligned}$$

The Gauss–Newton update:

$$\begin{aligned}\Delta &\approx \frac{1}{549} \begin{pmatrix} (-2.66) \cdot 226 - 6 \cdot 117 \\ -117 \cdot (-2.66) + 63 \cdot 6 \end{pmatrix} \\ &\approx (-2.38, 1.25).\end{aligned}$$

Interpretation. One plain Gauss–Newton step would overshoot—the step magnitude is far larger than the current residual scale. Levenberg–Marquardt with $\lambda = 1$ would dampen this into a small, safe step; this is precisely the situation LM is designed for.

2.4.4 Constrained Optimization and KKT Conditions

For equality and inequality constrained problems $\min f(\mathbf{x})$ s.t. $g_i(\mathbf{x}) = 0, h_j(\mathbf{x}) \leq 0$, the Karush–Kuhn–Tucker conditions are necessary at a local optimum:

$$\nabla f + \sum_i \lambda_i \nabla g_i + \sum_j \mu_j \nabla h_j = \mathbf{0}, \quad (2.33)$$

$$g_i(\mathbf{x}) = 0, h_j(\mathbf{x}) \leq 0, \quad (2.34)$$

$$\mu_j \geq 0, \mu_j h_j(\mathbf{x}) = 0. \quad (2.35)$$

They are sufficient under convexity (and certain constraint qualifications). The KKT conditions motivate interior-point and active-set methods for constrained optimization; they also underpin the Lagrangian formulations used in constrained RL (Chapter 19).

2.4.5 Convex Relaxations and Quadratic Programming

Motion planning (Chapter 18) and model predictive control (Chapter 20) reduce to quadratic programs (QPs) of the form

$$\min_{\mathbf{x}} \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{c}^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{A} \mathbf{x} \leq \mathbf{b}, \mathbf{E} \mathbf{x} = \mathbf{f}. \quad (2.36)$$

When $\mathbf{Q} \succeq 0$, the QP is convex and solvable in polynomial time. Two algorithmic families dominate: *active-set* methods (track which inequalities are tight) and *interior-point* methods (maintain strict feasibility by barrier terms). For typical MPC problems with dozens of variables and constraints, both are fast enough for 100+Hz control loops; automotive-grade libraries (OSQP, qpOASES, HPIPM) solve these in microseconds.

Example 2.26 (Tiny 2-D QP by hand). Solve:

$$\begin{aligned}\min_{\mathbf{x}} \quad & \frac{1}{2}(x_1^2 + x_2^2) \\ \text{s.t.} \quad & x_1 + x_2 \geq 1, \\ & x_1 \geq 0, x_2 \geq 0.\end{aligned}$$

Step 1: Lagrangian gradient.

$$\nabla_{\mathbf{x}}L = (x_1 - \mu_1, x_2 - \mu_2) + \lambda(-1, -1).$$

Stationarity gives:

$$x_1 = \lambda - \mu_1,$$

$$x_2 = \lambda - \mu_2.$$

Step 2: Guess active set.

Try both non-negativity constraints inactive ($\mu_1 = \mu_2 = 0$) and the sum equality active ($x_1 + x_2 = 1$):

$$x_1 = x_2 = \lambda,$$

$$2\lambda = 1,$$

$$\lambda = 0.5.$$

Step 3: Verify guess is consistent.

$$\mathbf{x}^* = (0.5, 0.5),$$

$$\text{objective} = \frac{1}{2}(0.25 + 0.25) = 0.25.$$

Both $x_1, x_2 \geq 0$ are satisfied strictly, so our guess was consistent.

Every MPC step solves a bigger version of this exact problem.

2.4.6 Proximal and Splitting Methods

When the objective is a sum of simple terms $L(\boldsymbol{\theta}) = f(\boldsymbol{\theta}) + g(\boldsymbol{\theta})$ where g is non-smooth but has a cheap *proximal operator*

$$\text{prox}_{\eta g}(\mathbf{v}) = \arg \min_{\mathbf{x}} g(\mathbf{x}) + \frac{1}{2\eta} \|\mathbf{x} - \mathbf{v}\|^2, \quad (2.37)$$

proximal gradient methods alternate a gradient step on f with a proximal step on g . ADMM splits constrained problems into alternating subproblems and is the computational backbone of several distributed and large-scale estimation pipelines. These methods are less common in real-time AV pipelines but appear in offline training, batch SLAM solvers, and certain learned-planning objectives.

2.5 Deep Learning Essentials

Optimization by itself is enough to fit the parametric models the previous sections described—bicycle models, GMMs, linear regressors. What makes modern AV perception possible is that very-high-

capacity parametric models, optimized by stochastic gradient descent on very large datasets, learn representations of images and point clouds that no one could engineer by hand. That body of techniques is what we call deep learning.

The perception, prediction, and increasingly the planning layers of a modern AV are neural networks. This section develops the architectural primitives those networks are built from, with enough detail to read Chapters 4–22 without further detour.

2.5.1 Feedforward Networks and Backpropagation

A feedforward network of depth L computes

$$\mathbf{h}^{(\ell)} = \sigma(\mathbf{W}^{(\ell)} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \ell = 1, \dots, L, \quad (2.38)$$

with $\mathbf{h}^{(0)} = \mathbf{x}$. Gradients are computed by the chain rule (*backpropagation*):

$$\frac{\partial L}{\partial \mathbf{W}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \mathbf{h}^{(\ell-1)\top}, \quad \boldsymbol{\delta}^{(\ell)} = \left(\mathbf{W}^{(\ell+1)\top} \boldsymbol{\delta}^{(\ell+1)} \right) \odot \sigma'(\mathbf{z}^{(\ell)}). \quad (2.39)$$

Common activations: ReLU $\sigma(z) = \max(0, z)$ (cheap, sparse, dies at zero), GELU (smooth, used in transformers), Swish/SiLU, Softplus.

Example 2.27 (Tiny backpropagation). Single-neuron net: $h = \sigma(wx + b)$ with $\sigma = \text{ReLU}$, $x = 2$, $w = 0.5$, $b = 0.1$. Forward: $z = 1.1$, $h = 1.1$, loss $L = \frac{1}{2}(h - 1)^2 = 0.005$. Backward: $\partial L / \partial h = h - 1 = 0.1$; $\sigma'(z) = 1$ (positive); $\delta = 0.1$; $\partial L / \partial w = \delta \cdot x = 0.2$; $\partial L / \partial b = 0.1$. SGD update at $\eta = 0.1$: $w \leftarrow 0.48$, $b \leftarrow 0.09$. Deep networks chain this operation through millions of parameters; the principle is identical.

2.5.2 Weight Initialization

Depth amplifies variance. Good initialization keeps activation and gradient norms stable across layers at initialization. The two standard schemes:

Xavier/Glorot. Weights drawn with variance

$$\text{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}. \quad (2.40)$$

Designed for tanh and linear activations.

Kaiming/He. Variance

$$\text{Var}(W) = \frac{2}{n_{\text{in}}}, \quad (2.41)$$

designed for ReLU, accounting for the activation's half-rectifying behavior.

Example 2.28 (Init variance for a convolutional filter). A 3×3 convolution with $C_{\text{in}} = 64$ input channels and $C_{\text{out}} = 128$ output channels.

Step 1: Fan-in.

$$\begin{aligned} n_{\text{in}} &= k \cdot k \cdot C_{\text{in}} \\ &= 3 \cdot 3 \cdot 64 \\ &= 576. \end{aligned}$$

Step 2: Kaiming standard deviation.

$$\begin{aligned} \sigma &= \sqrt{\frac{2}{n_{\text{in}}}} \\ &= \sqrt{\frac{2}{576}} \\ &\approx 0.0589. \end{aligned}$$

Miscalibration cost.

A factor-of-four miscalibration (e.g., using Xavier when ReLU is used) causes early-layer activations to shrink by roughly $\sqrt{2}$ per layer. Through a 50-layer network, signal reaches $\sim 2^{-25}$ —a vanished gradient that will take many epochs to recover.

2.5.3 Normalization Layers

Normalization layers stabilize training by reducing internal covariate shift and smoothing the loss landscape. The main variants:

Batch normalization [33]. Normalizes each channel’s activations over the minibatch:

$$\hat{x}_{b,c,h,w} = \frac{x_{b,c,h,w} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}, \quad y_{b,c,h,w} = \gamma_c \hat{x}_{b,c,h,w} + \beta_c. \quad (2.42)$$

Requires sufficient batch size; couples samples within a batch.

Layer normalization. Normalizes each sample’s activations over the feature dimension. Batch-independent; the default in transformers.

Group normalization [72]. Splits channels into groups and normalizes within each group. Useful for small batches (detection, segmentation) where BN is unreliable.

Instance normalization, RMSNorm. Variants tuned to style transfer and large language models respectively.

Example 2.29 (BatchNorm on a tiny batch). Four activations in a single channel: $x = (1, 2, 3, 4)$. Affine parameters $\gamma = 2$, $\beta = 1$.

Step 1: Batch statistics.

$$\begin{aligned}\mu &= \frac{1+2+3+4}{4} = 2.5, \\ \sigma^2 &= \frac{(1-2.5)^2 + (2-2.5)^2 + (3-2.5)^2 + (4-2.5)^2}{4} \\ &= \frac{2.25 + 0.25 + 0.25 + 2.25}{4} = 1.25, \\ \sigma &\approx 1.118.\end{aligned}$$

Step 2: Normalize.

$$\begin{aligned}\hat{x} &= \frac{x - \mu}{\sigma} \\ &\approx (-1.342, -0.447, 0.447, 1.342).\end{aligned}$$

Step 3: Apply affine.

$$\begin{aligned}y &= \gamma\hat{x} + \beta \\ &\approx (-1.683, 0.106, 1.894, 3.683).\end{aligned}$$

The normalization recenters and rescales; γ and β let the network recover any shift or scale it wants. At inference, running statistics replace the minibatch mean and variance—which is why frozen BN stats and batch size 1 are consistent at inference but can be inconsistent if the deployment batch distribution differs from training.

2.5.4 Convolutional Networks

Convolution exploits translational equivariance. A 2D convolution with kernel $\mathbf{K} \in \mathbb{R}^{k \times k \times C_{\text{in}} \times C_{\text{out}}}$ produces

$$\mathbf{y}_{i,j,c_o} = \sum_{u,v,c_i} \mathbf{K}_{u,v,c_i,c_o} \mathbf{x}_{i+u,j+v,c_i} + b_{c_o}. \quad (2.43)$$

With stride s and padding p , the output spatial size of a $H \times W$ input is

$$H_{\text{out}} = \lfloor (H - k + 2p)/s \rfloor + 1. \quad (2.44)$$

Example 2.30 (Output size and FLOPs for a perception stage). Input feature map $512 \times 896 \times 64$ (typical AV resolution). Apply a 3×3 conv with 128 filters, stride 1, padding 1.

Step 1: Output spatial size.

$$\begin{aligned} H_{\text{out}} &= \lfloor (H - k + 2p) / s \rfloor + 1 \\ &= \lfloor (512 - 3 + 2) / 1 \rfloor + 1 \\ &= 512. \end{aligned}$$

By the same formula, $W_{\text{out}} = 896$. Output shape: $512 \times 896 \times 128$.

Step 2: FLOPs per forward pass.

$$\begin{aligned} \text{FLOPs} &= H_{\text{out}} \cdot W_{\text{out}} \cdot C_{\text{out}} \cdot C_{\text{in}} \cdot k^2 \\ &= 512 \cdot 896 \cdot 128 \cdot 64 \cdot 9 \\ &\approx 3.4 \times 10^{10}. \end{aligned}$$

Stack-level implication.

An AV stack with ten such stages spends $\sim 3 \times 10^{11}$ FLOPs per frame on convolutions alone—roughly 10 ms on a 30 TFLOP accelerator at best-case utilization. This is why FLOP count matters from day one.

Dilated convolutions expand receptive field without extra parameters or reducing resolution (important for semantic segmentation, Chapter 12).

2.5.5 Residual Connections

Very deep networks suffer from vanishing gradients. The residual connection

$$\mathbf{h}^{(\ell+1)} = \mathbf{h}^{(\ell)} + F(\mathbf{h}^{(\ell)}) \quad (2.45)$$

from ResNet [29] makes the network easy to optimize at depth 100+ by providing an identity-preserving gradient path.

Example 2.31 (Gradient through a residual block). A vanilla 50-layer network with per-layer gradient multiplier 0.8 contracts the gradient by $0.8^{50} \approx 1.4 \times 10^{-5}$, a vanishing signal. A residual network's gradient through each block is $\mathbf{I} + \partial F / \partial \mathbf{h}$; the identity path ensures gradient norm does not collapse regardless of what F does. ResNet-50 trains to 76 % ImageNet accuracy; a plain CNN of the same depth struggles to reach 70 %.

2.5.6 Attention and Transformers

Scaled dot-product attention [68] maps queries $\mathbf{Q}$, keys $\mathbf{K}$, values $\mathbf{V}$ to outputs via

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (2.46)$$

Multi-head attention runs h projected attentions in parallel and concatenates. A transformer block stacks:

- LayerNorm $\rightarrow$ multi-head self-attention $\rightarrow$ residual add.
- LayerNorm $\rightarrow$ feed-forward (two-layer MLP) $\rightarrow$ residual add.

The full architecture admits three useful levels of resolution: the scaled dot-product attention unit (Figure 2.2), the multi-head attention module (Figure 2.3), the encoder block (Figure 2.4), and finally a driving-specific transformer stack (Figure 2.5). Each zooms out one level from the previous.

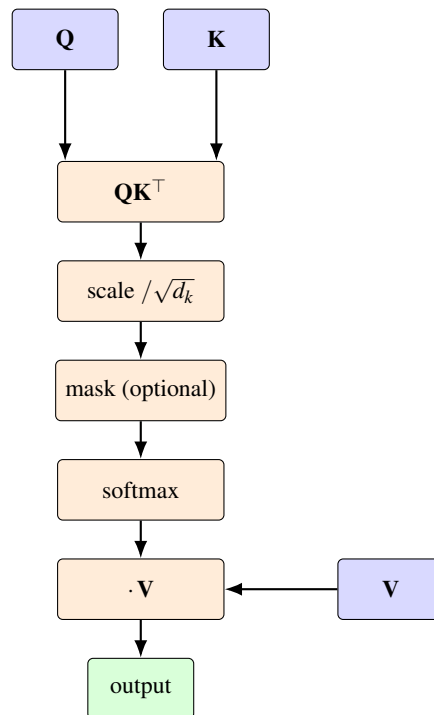


Figure 2.2: Scaled dot-product attention. Queries and keys compute a similarity matrix $\mathbf{QK}^\top$; the scaling factor $1/\sqrt{d_k}$ prevents the pre-softmax values from saturating. An optional mask (causal for decoders, padding for variable-length sequences) zeros out disallowed positions. The softmax normalizes to a probability distribution over keys for each query; values are mixed by these attention weights to produce the output.

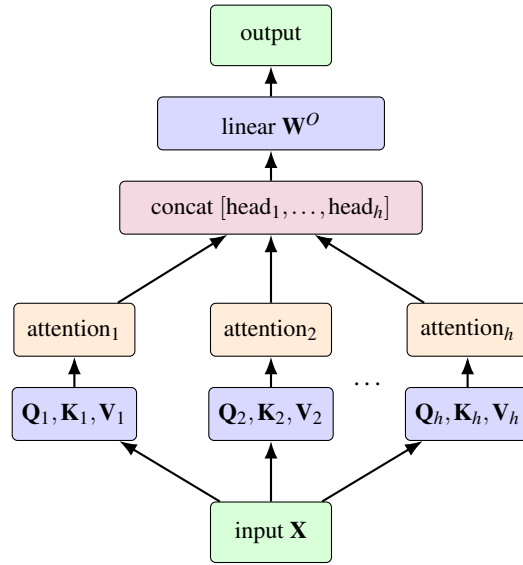


Figure 2.3: Multi-head attention. The input is linearly projected into h separate (Q, K, V) triplets—one per attention “head.” Each head runs its own scaled dot-product attention (Figure 2.2) in parallel, producing h output sequences of dimension d_v . These are concatenated along the feature axis and linearly projected back to the original dimension. Different heads learn to attend to different structural relationships; production AV transformers typically use $h = 8$ or $h = 16$.

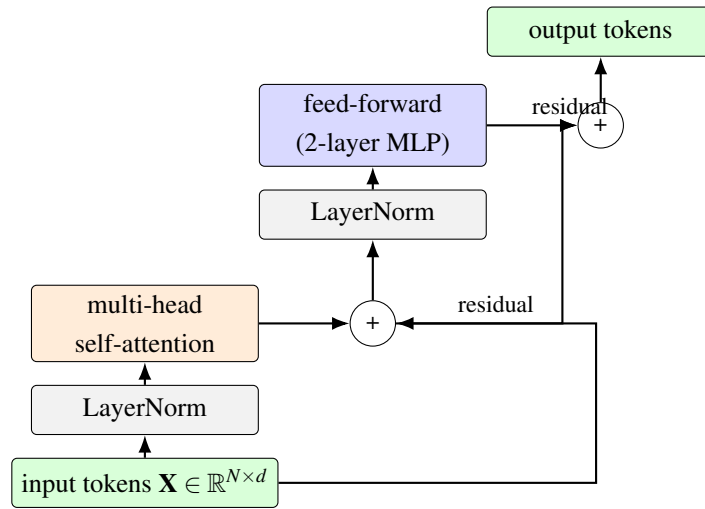


Figure 2.4: The transformer encoder block (pre-norm variant). Two residual sub-blocks: (1) LayerNorm $\rightarrow$ multi-head self-attention (expanded in Figure 2.3) $\rightarrow$ residual add; (2) LayerNorm $\rightarrow$ position-wise feed-forward $\rightarrow$ residual add. Stacking L such blocks ($L = 6$ – 24 typical) produces the encoder used in BEVFormer, scene transformers, and language models. The pre-norm placement (LayerNorm before each sub-block) trains more stably at depth than the post-norm variant from the original paper.

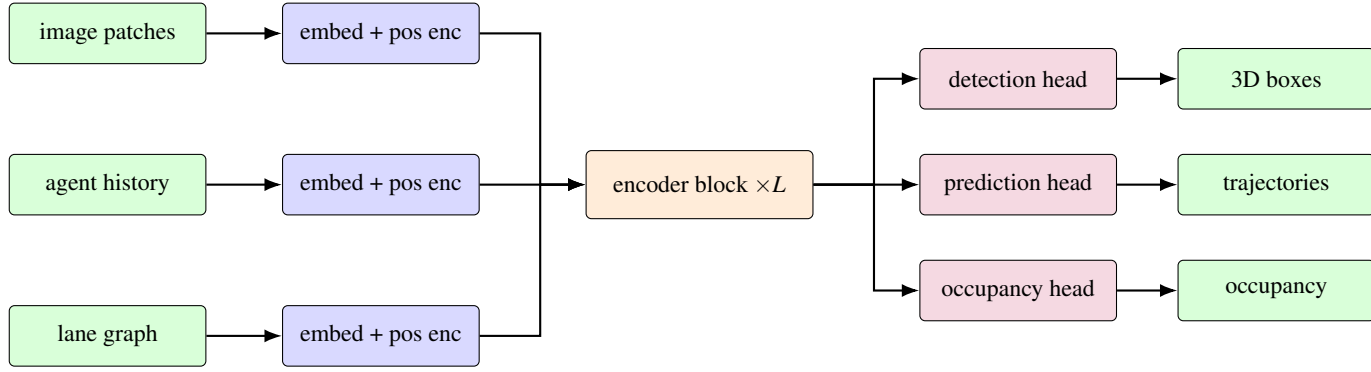


Figure 2.5: A driving-specific transformer stack (schematic). Heterogeneous tokens—image patches from multi-view cameras, agent history sequences, and lane-graph vertices—are embedded with type-specific positional encodings and processed through a shared encoder stack of L transformer blocks (Figure 2.4). Multiple task heads (3D detection, trajectory prediction, occupancy) decode the shared representation. This pattern underlies production architectures such as BEVFormer, Wayformer, and unified perception- prediction models at leading AV programs.

Self-attention’s quadratic cost is the dominant concern for production perception, where token counts are much larger than in language. Figure 2.6 plots the compute growth of full attention against two efficient alternatives.

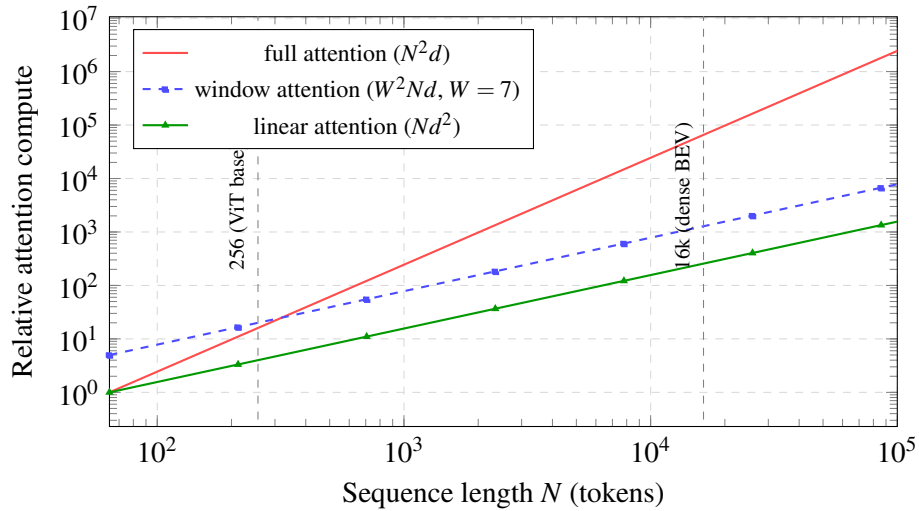


Figure 2.6: Attention compute scaling with sequence length (log-log). Full self-attention scales as N^2d ; at large token counts this is prohibitive. Window attention (Swin-style, window size $W = 7$) scales linearly. Linear attention variants (Performer, Linformer) scale with constant d^2 overhead. Dashed verticals mark relevant automotive token counts: ~ 256 for ViT-base classification, $\sim 16k$ for a dense BEV grid. At BEV resolution, full attention is $64\times$ more expensive than window attention and is rarely used in production.

Example 2.32 (Attention over agents at an intersection). A prediction model has three agents represented as tokens with feature dimension $d_k = 4$: ego, a left-turning car, and a pedestrian.

After the query/key projections:

$$\begin{aligned}\mathbf{Q}_{\text{ego}} &= (1, 0, 0, 0.5), \\ \mathbf{K}_{\text{car}} &= (0.9, 0, 0.1, 0.4), \\ \mathbf{K}_{\text{ped}} &= (0, 1, 0, 0.3).\end{aligned}$$

Step 1: Dot products.

$$\begin{aligned}\mathbf{Q}_{\text{ego}} \cdot \mathbf{K}_{\text{car}} &= 0.9 + 0 + 0 + 0.2 = 1.1, \\ \mathbf{Q}_{\text{ego}} \cdot \mathbf{K}_{\text{ped}} &= 0 + 0 + 0 + 0.15 = 0.15.\end{aligned}$$

Step 2: Scale by $\sqrt{d_k} = 2$.

$$\begin{aligned}s_{\text{car}} &= 1.1/2 = 0.55, \\ s_{\text{ped}} &= 0.15/2 = 0.075.\end{aligned}$$

Step 3: Softmax.

$$\begin{aligned}a_{\text{car}} &= \frac{e^{0.55}}{e^{0.55} + e^{0.075}} \approx 0.617, \\ a_{\text{ped}} &= \frac{e^{0.075}}{e^{0.55} + e^{0.075}} \approx 0.383.\end{aligned}$$

Interpretation. The ego attends most to the car, less to the pedestrian—correct if the car is the immediate threat. Attention learns these weights from data, without hand-written rules.

Self-attention’s quadratic cost $\mathcal{O}(N^2d)$ in sequence length N is a real bottleneck for dense video or high-resolution imagery. Mitigations: windowed/local attention (Swin), linearized attention, Flash-Attention kernels, hierarchical/coarse-to-fine attention.

Positional Encoding

In driving transformers, position matters in a richer sense than in language models. A BEVFormer token encodes “which (row, col) on the ground plane,” a scene-transformer agent token encodes “which (x, y) in the ego frame and which timestep,” and a lane-graph token encodes “where on which polyline.” All of these require the network to know positional information that permutation-equivariant attention does not by itself convey. The choice of positional encoding therefore directly shapes what a prediction or BEV model can learn about geometric relationships—which is why new AV architectures often debut a new positional-encoding scheme.

Attention is permutation-equivariant, so positional information must be added explicitly. Sinusoidal:

$$\text{PE}(t, 2i) = \sin(t/10000^{2i/d}), \quad \text{PE}(t, 2i+1) = \cos(t/10000^{2i/d}). \quad (2.47)$$

Learned, relative, and rotary (RoPE) alternatives are all in use.

2.5.7 Training Stability

Large networks require several techniques to train stably: gradient clipping ($\|\nabla L\| \leq c$), mixed-precision training (FP16/BF16 with FP32 master weights), warmup for the first few thousand steps, and exponential moving average of weights. Each is cheap and routinely turned on in production perception training.

2.5.8 Regularization

Weight decay, dropout, stochastic depth, label smoothing, mixup, CutMix, and data augmentation combat overfitting. For AV perception two specific issues dominate: *class imbalance* (pedestrians are rare; focal loss helps, Chapter 11) and *long-tail distribution shift*. Domain adaptation and self-supervised pretraining (Chapter 4) mitigate but do not solve these.

2.6 Graphs and Graph Neural Networks

Several of the most important objects in an AV stack are graphs, even when we do not call them that. The *lane graph* is a directed graph whose nodes are lane segments and whose edges encode successor, predecessor, left-neighbor, and right-neighbor relationships (Chapter 14). The *road-scene interaction graph* connects the ego and every detected agent by edges that model who yields to whom, who is in whose path, and which pairs need to resolve their intents before either can proceed (Chapter 17). The *factor graph* of a SLAM back-end has one node per pose or landmark, with edges representing odometry, reprojection, and loop-closure constraints (Chapter 13). Even the *compute graph* of a neural network itself—layers as nodes, tensors as edges—is a graph, even though most deep-learning libraries hide the underlying traversal.

Graphs are the natural substrate here for a reason. Regular grids are the right abstraction when data has uniform spatial structure (images), sequences are the right abstraction when data is one-dimensional and ordered (language, timeseries), but neither captures the topology of roads or the variable-cardinality nature of a traffic scene. A four-way intersection and a roundabout have completely different connectivity that no fixed grid can represent, and the set of nearby vehicles changes from frame to frame. A graph neural network (GNN) is the natural generalization of a CNN or transformer to these variable-topology domains: it learns features by aggregating information along edges, and it scales naturally to graphs of different sizes at inference without retraining.

This section develops the mathematical substrate of GNNs—message passing and graph attention—because four of the downstream chapters rely on it:

- **Perception** (Chapter 4). Point-cloud backbones such as PointNet++ and dynamic-graph CNNs are GNNs on k -nearest-neighbor graphs over LiDAR points.
- **Prediction** (Chapter 17). VectorNet and most modern multi-agent forecasters are GNNs (or GNN-transformer hybrids) over mixed agent-and-map-element graphs.

- **SLAM** (Chapter 13). Factor graphs are exactly the graphs on which nonlinear least-squares solvers operate; their algorithmic structure parallels GNN message passing.
- **HD Maps** (Chapter 14). Map-based priors feed prediction and planning through explicit lane-graph encodings.

A graph is $G = (\mathcal{V}, \mathcal{E})$ with node features $\mathbf{x}_v$ and optional edge features $\mathbf{e}_{uv}$. For an AV interaction graph at a single timestep, $\mathcal{V}$ might be the ego plus N nearby agents and M nearby lane centerlines, with $\mathbf{x}_v$ encoding class, pose, and velocity; edges then capture spatial proximity, lane affiliation, and coarse interaction priors.

2.6.1 Message Passing

A graph neural network (GNN) alternates message and update steps:

$$\mathbf{m}_v^{(\ell)} = \text{Agg}\{\phi(\mathbf{h}_u^{(\ell-1)}, \mathbf{h}_v^{(\ell-1)}, \mathbf{e}_{uv}) : u \in \mathcal{N}(v)\}, \quad (2.48)$$

$$\mathbf{h}_v^{(\ell)} = \psi(\mathbf{h}_v^{(\ell-1)}, \mathbf{m}_v^{(\ell)}). \quad (2.49)$$

The aggregator Agg is typically sum, mean, or max. Special cases: Graph Convolutional Networks (GCN) use normalized sum; Graph Attention Networks (GAT) use attention-weighted sum; Message-Passing Neural Networks generalize both.

Figure 2.7 visualizes one layer of message passing on a small interaction graph at a four-way intersection.

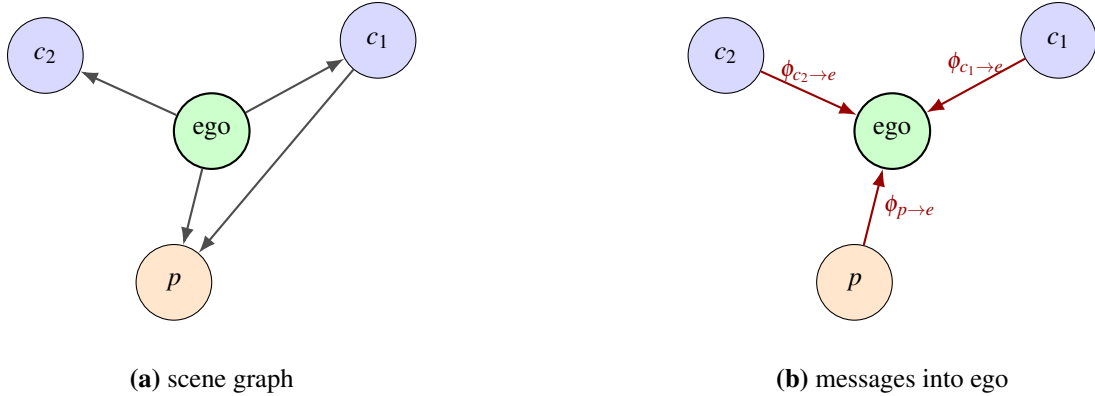


Figure 2.7: GNN message passing on a four-node intersection graph. (a) Nodes are the ego, two nearby cars c_1, c_2 , and a pedestrian p ; edges connect agents whose trajectories interact. (b) In one GNN layer, each node receives messages $\phi(\mathbf{h}_u, \mathbf{h}_v, \mathbf{e}_{uv})$ from its graph neighbors (Eq. 2.48), aggregates them, and updates its own state (Eq. 2.49). Stacking L such layers propagates information across L -hop neighborhoods—exactly the mechanism by which VectorNet, SceneTransformer, and related prediction models reason about multi-agent interactions.

Example 2.33 (Single GNN layer on a three-agent scene). Three agents with features $\mathbf{h} = ((1, 0), (0, 1), (1, 1))$. Edges: 1-2, 2-3 (line graph). Message: $\phi(u, v) = \mathbf{h}_u$; aggregation: mean. For node 2, neighbors are 1 and 3: $\mathbf{m}_2 = ((1, 0) + (1, 1))/2 = (1, 0.5)$. Update: $\psi(\mathbf{h}_v, \mathbf{m}_v) = \mathbf{h}_v + \mathbf{m}_v$: $\mathbf{h}'_2 = (1, 1.5)$. This single-layer operation propagates information across one hop; stacking layers propagates further.

2.6.2 Attention on Graphs

Transformers are a special case of GNNs where the graph is fully connected and attention scores replace edge weights. Many AV prediction models (VectorNet, SceneTransformer; Chapter 17) are GNN-transformer hybrids: agents and map elements are tokens, with attention restricted to a local neighborhood in the lane graph.

2.7 Uncertainty Quantification

The Gaussian machinery earlier in the chapter gives us *parametric* uncertainty for models we can write down—Kalman filters know their own covariance. Neural networks do not have this property for free: a CNN with a softmax output will hand you a confidence number, but that number is not a principled posterior and is routinely overconfident by 10–20 %. An AV that brakes only when the perception network is “confident” is at the mercy of that miscalibration. Uncertainty quantification for neural networks is the set of techniques that recover trustworthy confidence from otherwise black-box models.

Safety-critical systems need to distinguish confident predictions from guesses. Two kinds of uncertainty arise:

- **Aleatoric** uncertainty reflects irreducible randomness in the data (sensor noise, inherent multimodality). It remains even with infinite training data.
- **Epistemic** uncertainty reflects the model’s lack of knowledge; it shrinks with more data.

Aleatoric uncertainty is typically captured by probabilistic output heads (e.g., a network predicting both a mean and a log-variance). Epistemic uncertainty requires more machinery.

2.7.1 Bayesian Neural Networks and MC Dropout

Exact Bayesian inference over neural-network weights is intractable. Monte Carlo dropout approximates it by keeping dropout active at inference and averaging predictions over several stochastic forward passes.

Example 2.34 (Epistemic uncertainty via MC dropout). A detector outputs detection probabilities for a pedestrian at $T = 20$ MC-dropout passes: $\{0.91, 0.89, 0.93, 0.30, 0.88, 0.85, \dots\}$. The 0.30 outlier comes from a stochastic pass where a key neuron was dropped. Mean ≈ 0.85 , sample std ≈ 0.15 . The std is the epistemic-uncertainty estimate—high std means “this prediction is a guess.” A safety layer may refuse to commit to braking decisions based on predictions with std > 0.1 .

2.7.2 Deep Ensembles

Training M independent networks with different random seeds and averaging predictions gives strong epistemic-uncertainty estimates, better calibrated than MC dropout in most benchmarks. Cost scales linearly with M ; $M = 5$ is typical.

2.7.3 Evidential Deep Learning

A single-pass alternative parameterizes the output distribution with a conjugate prior (Dirichlet for classification, Normal-Inverse-Gamma for regression) and trains the network to output hyperparameters. The resulting model returns both point predictions and uncertainty without inference-time sampling.

2.7.4 Calibration in Practice

Post-hoc temperature scaling is the cheapest and most reliable calibration technique: fit a single scalar T on held-out data so that $\text{softmax}(\ell/T)$ produces probabilities matching empirical accuracy. More elaborate methods (isotonic, Platt, conformal) may be warranted when the downstream consumer depends on calibrated probabilities.

2.8 Information-Theoretic Quantities

Many of the loss functions introduced in the previous two sections are information-theoretic at heart. The cross-entropy loss that trains every perception classifier, the KL term in the ELBO that regularizes a prediction VAE, the KL penalty in PPO that stabilizes RL training on a driving task—all are instances of a small handful of information-theoretic quantities that this section collects.

Entropy, cross-entropy, KL divergence, and mutual information recur in perception losses, prediction objectives, and RL exploration bonuses. Useful identities:

$$H(p) = -\mathbb{E}_{\mathbf{x} \sim p}[\log p(\mathbf{x})], \quad (2.50)$$

$$H(p, q) = -\mathbb{E}_{\mathbf{x} \sim p}[\log q(\mathbf{x})] = H(p) + D_{\text{KL}}(p \parallel q), \quad (2.51)$$

$$I(X; Y) = D_{\text{KL}}(p_{XY} \parallel p_X p_Y) = H(X) - H(X \mid Y). \quad (2.52)$$

Cross-entropy loss is the dominant classification objective ($\mathcal{L} = H(p_{\text{true}}, p_{\text{pred}})$); KL-regularized objectives underlie variational autoencoders and PPO (Chapter 19).

Example 2.35 (Cross-entropy for a three-class detection). Three classes (car, truck, ped), true class car: $p = (1, 0, 0)$. Predicted probabilities from softmax: $q = (0.7, 0.2, 0.1)$. $H(p, q) = -1 \cdot \log 0.7 = 0.357$. If prediction $q = (0.3, 0.5, 0.2)$ (wrong argmax), $H(p, q) = -\log 0.3 = 1.204$ —more than $3\times$ larger. This is why cross-entropy penalizes wrong-class confidence sharply.

2.8.1 Evidence Lower Bound (ELBO)

Variational inference bounds the log marginal likelihood by

$$\log p(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z})) \equiv \text{ELBO}(q). \quad (2.53)$$

2.8.2 Fisher Information and Natural Gradient

The Fisher information matrix

$$\mathbf{F} = \mathbb{E}[\nabla \log p_{\boldsymbol{\theta}} \nabla \log p_{\boldsymbol{\theta}}^{\top}] \quad (2.54)$$

gives the natural-gradient preconditioner $\mathbf{F}^{-1} \nabla L$, which makes gradient descent invariant to reparameterization.

2.9 The Learning Setups We Use

We have developed the architectures and the loss functions; the missing ingredient is *where the supervision comes from*. The AV literature uses nearly every learning paradigm the machine-learning community has invented, because different subproblems have different supervision budgets. Object detection has dense labels; self-supervised pretraining has none; RL generates its own signal via simulation; imitation learning reuses human driving logs. The list below taxonomizes these setups and flags the chapters where each dominates.

- **Supervised learning.** Labels are available: object classes, lane segmentations, steering angles. Risk minimization $\min_{\boldsymbol{\theta}} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [\ell(f_{\boldsymbol{\theta}}(\mathbf{x}), y)]$.
- **Self-supervised learning.** Labels are derived from the data itself (masked image modeling, contrastive pairs [13]). Crucial because AV sensor data is plentiful but human labeling is expensive.
- **Imitation learning.** Learn a policy from demonstrations $(s, a) \sim \mathcal{D}$. Addressed in Chapter 22.
- **Reinforcement learning.** Learn to maximize return $J(\pi) = \mathbb{E}_{\pi} \sum_t \gamma^t r_t$. Addressed in Chapter 19.
- **Semi-supervised and weakly-supervised learning.** Small labeled set plus large unlabeled set, often combined with pseudo-labeling or consistency regularization.
- **Active learning.** Model selects informative examples for human labeling. Key to the data flywheel of fleet-based development.
- **Continual / lifelong learning.** Adapt to new data over time without catastrophic forgetting of old capabilities.
- **Transfer learning.** Reuse pretrained representations (foundation models, large-scale supervised backbones) for downstream tasks with smaller datasets.

2.10 Computational Considerations

The preceding sections developed the mathematical content of the AV stack as if compute were unlimited. It is not. A Level 4 AV has perhaps 500 TOPS of usable automotive accelerator capacity and a 100 ms end-to-end latency budget shared across perception, prediction, planning, and control. Every architectural decision (depth, width, attention span, sensor fusion strategy) is ultimately constrained by these numbers, and a technique that gains 1 % accuracy while doubling FLOP cost

will often lose to the baseline on deployment. This section gives the toolkit for fitting research-grade models onto production silicon.

AV models must run on power-constrained hardware at real-time rates. Two quantities matter at design time: **parameter count** (memory) and **FLOPs per forward pass** (roughly proportional to latency on a given accelerator).

2.10.1 Mixed Precision

Automotive accelerators strongly favor FP16, BF16, or INT8. FP16 training requires loss scaling; BF16 has larger dynamic range and is easier. INT8 inference requires quantization-aware training or post-training calibration to preserve accuracy.

2.10.2 Quantization

Weights (and often activations) are quantized from FP32 to INT8 or lower for deployment. Post-training quantization calibrates scales from a held-out dataset; quantization-aware training simulates quantization during training for better final accuracy.

Example 2.36 (INT8 quantization scale). A weight tensor has range $[-2.3, 1.8]$. Symmetric INT8 scale: $s = \max(|-2.3|, |1.8|)/127 = 2.3/127 \approx 0.0181$. Quantized values $q = \text{round}(w/s) \in [-127, 127]$. A weight of 1.25 becomes $\text{round}(69.1) = 69$; de-quantized back to $69 \cdot 0.0181 = 1.249$ —negligible error. Over millions of weights the errors accumulate but are typically below 0.5 % accuracy loss for perception backbones.

2.10.3 Pruning and Distillation

Structured pruning reduces both parameter count and FLOPs. Knowledge distillation trains a smaller student network to mimic a larger teacher, routinely yielding $2\text{--}4\times$ speedups without large accuracy loss.

2.10.4 Compilation and Graph Optimization

Framework-level optimizations (operator fusion, constant folding, layout conversion) are typically handled by the compiler (TensorRT, ONNX Runtime, TVM, automotive vendor toolchains).

2.11 Tooling and Reproducibility

The practical fabric of this book is built on open tools: Python; PyTorch and JAX for deep learning; ROS 2 for middleware; Open3D, PCL, and Ceres for geometry and optimization; GTSAM and g2o for factor-graph SLAM; CARLA, LGSVL, MetaDrive, and Waymax for closed-loop evaluation; Hydra and Weights & Biases for experiment management.

```
1 import torch, torch.nn as nn
2 from torch.utils.data import DataLoader
3 from torch.cuda.amp import autocast, GradScaler
```

```

4
5 def train(model: nn.Module,
6           loader: DataLoader,
7           epochs: int = 10,
8           lr: float = 1e-3,
9           wd: float = 1e-4,
10          clip_norm: float = 1.0):
11     model.cuda()
12     opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=wd
13                             )
14     sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=
15               epochs)
16     loss_fn = nn.CrossEntropyLoss()
17     scaler = GradScaler()
18
19     for epoch in range(epochs):
20         model.train()
21         for step, (x, y) in enumerate(loader):
22             x = x.cuda(non_blocking=True)
23             y = y.cuda(non_blocking=True)
24             opt.zero_grad(set_to_none=True)
25             with autocast(dtype=torch.float16):
26                 logits = model(x)
27                 loss = loss_fn(logits, y)
28                 scaler.scale(loss).backward()
29                 scaler.unscale_(opt)
30                 torch.nn.utils.clip_grad_norm_(model.parameters(),
31                 clip_norm)
32                 scaler.step(opt)
33                 scaler.update()
34             sched.step()
35         torch.save({"epoch": epoch,
36                   "model": model.state_dict(),
37                   "opt": opt.state_dict()},
38                   f"ckpt_{epoch}.pt")

```

Listing 2.1: A minimal SGD training loop skeleton in PyTorch, including mixed precision, gradient clipping, and checkpointing.

2.12 Worked Examples

The preceding sections interleave worked examples with each formula. This short consolidation illustrates the chapter's techniques on a single end-to-end mini-problem representative of an AV estimation step.

Example 2.37 (End-to-end: projecting a 3D object, gating a detection, updating a Kalman tracker). A 3D object is detected in the world frame at $\mathbf{p}^W = (120, 60, 1.2)$ m. Body pose: $\mathbf{R}_{WB}$ is yaw 30° and $\mathbf{t}_{WB} = (100, 50, 0)$. Camera extrinsic: $\mathbf{R}_{BC} = \mathbf{I}$, $\mathbf{t}_{BC} = (1.5, 0, 1.8)$. Camera intrinsics: $f = 1000$ pixels, principal point $c = (960, 540)$.

Step 1: transform to body frame.

Relative position in world coordinates:

$$\mathbf{p}^W - \mathbf{t}_{WB} = (20, 10, 1.2).$$

Apply inverse 30° yaw ($\cos \theta = 0.866$, $\sin \theta = 0.5$):

$$\begin{aligned} \mathbf{p}^B &= \mathbf{R}_{WB}^\top (\mathbf{p}^W - \mathbf{t}_{WB}) \\ &\approx (0.866 \cdot 20 + 0.5 \cdot 10, -0.5 \cdot 20 + 0.866 \cdot 10, 1.2) \\ &\approx (22.32, -1.34, 1.2). \end{aligned}$$

Step 2: transform to camera frame.

Subtract camera translation, then swap axes to camera convention (z forward, y down):

$$\begin{aligned} \mathbf{p}^B - \mathbf{t}_{BC} &= (20.82, -1.34, -0.6), \\ \mathbf{p}^C &= (-1.34, 0.6, 20.82). \end{aligned}$$

Step 3: pinhole projection.

$$\begin{aligned} u &= f \cdot \frac{x}{z} + c_x \\ &= 1000 \cdot \frac{-1.34}{20.82} + 960 \\ &\approx 896, \\ v &= f \cdot \frac{y}{z} + c_y \\ &= 1000 \cdot \frac{0.6}{20.82} + 540 \\ &\approx 569. \end{aligned}$$

Step 4: χ^2 gate.

Predicted bounding-box center (896, 569) with covariance $\text{diag}(16, 9)$. Observed detection at

(900, 572):

$$\begin{aligned} d_M^2 &= \frac{(900 - 896)^2}{16} + \frac{(572 - 569)^2}{9} \\ &= \frac{16}{16} + \frac{9}{9} \\ &= 2. \end{aligned}$$

With $2 < 9.21$ (χ^2_2 at 99 %), the gate passes.

Step 5: Kalman update.

Track state $(u, v, \dot{u}, \dot{v})$; measurement innovation (4, 3) pixels. With gain matrix tuned so the position block contributes ~ 0.6 of the innovation to position and ~ 0.05 to velocity, the track mean updates:

position : $(896, 569) \rightarrow (898.4, 570.8)$,
velocity : updates by $\approx (0.2, 0.15)$.

Covariance shrinks on the position block.

This single trajectory exercises SE(3) geometry, pinhole projection, Mahalanobis gating, and Kalman update—every estimator and optimizer in this chapter contributes, in some form, to producing the output a downstream tracker consumes.

2.13 Industrial Deployment

The mathematical and machine-learning foundations developed in this chapter underpin essentially every production AV stack. Deep learning frameworks PyTorch (developed at Meta AI) and TensorFlow (Google) are both used widely across the industry; NVIDIA’s CUDA and TensorRT toolchains, together with their DRIVE platform, are the dominant production-acceleration target for on-vehicle inference. The ResNet architecture [29], originally published in 2015 by Microsoft Research, remains a baseline image backbone in production AV perception stacks. The transformer architecture introduced in “Attention Is All You Need” [68] forms the backbone of newer production-perception models including BEVFormer-style architectures and the visual-language models used for offline auto-labeling at multiple AV programs.

For probabilistic robotics, the open-source Ceres Solver (originally developed at Google) and GTSAM (developed at Georgia Tech) are widely used non-linear-least-squares libraries used in production SLAM back-ends; the Kalman-filter formulations described in this chapter are standard across automotive sensor-fusion implementations.

2.14 Chapter Summary

We have reviewed: rigid-body geometry on $\text{SO}(3)$, $\text{SE}(3)$, and their ground-vehicle reductions; the Gaussian closure properties underpinning Kalman filtering; particle filters for non-Gaussian posteriors; robust estimation for outlier-rich data; Gauss–Newton, Levenberg–Marquardt, and quadratic programming as optimization workhorses; the deep learning ingredients (initialization, normalization, convolution, attention, residual connections, regularization) that dominate modern AV perception and policy learning; graph neural networks; uncertainty quantification; information-theoretic objectives; and the computational considerations (mixed precision, quantization, distillation) that determine what actually runs on automotive compute. Every formula introduced here recurs in the chapters that follow, usually specialized but rarely modified; a reader fluent in this chapter is equipped to read the rest.

2.15 Exercises

1. Derive the Kalman filter from Lemmas 2.11 and 2.13.
2. Show that the sum of two independent Gaussians is Gaussian and give the mean and covariance.
3. For a 224×224 image fed through a stack of three 3×3 convolutions with stride 1, no padding, what is the receptive field size of a unit in the final feature map?
4. Write out a single step of Levenberg–Marquardt applied to the least-squares fit $\mathbf{r}_i(\boldsymbol{\theta}) = y_i - \theta_0 e^{\theta_1 x_i}$.
5. Using Rodrigues’ formula, compute $\mathbf{R}$ for $\boldsymbol{\omega} = (\pi/2, 0, 0)$ and verify it corresponds to a 90° rotation about the x -axis.
6. Derive the UKF sigma-point weights for a state of dimension n and scaling parameter α . What happens as $\alpha \rightarrow 0$?
7. A particle filter has $N = 1000$ particles with weights $w^{(i)}$. Ten particles have $w^{(i)} = 0.09$ each; the rest have $w^{(i)} = 0.1/990$. Compute the effective sample size N_{eff} and decide whether to resample.
8. Show that the Huber loss is the MAP estimate under a Laplace-tail Gaussian-core noise model. Specify the model explicitly.
9. Give a one-line argument for why temperature scaling cannot change the ranking of two predictions but can change their probabilities.
10. Explain, in terms of the Fisher information matrix, why natural gradient is invariant to reparameterization while vanilla gradient is not.
11. For a transformer block with sequence length N and hidden size d , compute the FLOP count of a single forward pass (attention + feed-forward). At which N does attention dominate over the feed-forward?

12. A quantization-aware training pipeline introduces fake-quantize ops with per-tensor scales. Explain why per-channel scales are typically preferred for weights of convolutions.
13. Given a BatchNorm layer, a LayerNorm layer, and a GroupNorm layer with identical computational cost at train time, argue which you would deploy to a Level 4 AV that freezes the BN statistics (inference only) and must run at batch size 1.
14. Derive the ELBO for a Gaussian latent variable model $p(\mathbf{x} | \mathbf{z}) = \mathcal{N}(\mathbf{x}; g_{\boldsymbol{\theta}}(\mathbf{z}), \sigma^2 \mathbf{I})$, $p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$, $q(\mathbf{z} | \mathbf{x}) = \mathcal{N}(\mathbf{z}; \mu_{\phi}(\mathbf{x}), \sigma_{\phi}^2(\mathbf{x}) \mathbf{I})$.
15. Set up a conformal prediction wrapper for a neural regression head, using a held-out calibration set of size n_{cal} . State the coverage guarantee and the assumption that makes it hold.

Part II

Perception

Chapter 3

Autonomous Perception

3.1 Introduction

Autonomous driving is the coordinated output of three loops running continuously in the vehicle: *perception*, which answers the question “what is around me?”; *prediction and planning*, which answers “what will happen, and what should I do?”; and *control*, which answers “how do I execute my plan through the throttle, brake, and steering rack?” This chapter sits at the opening of Part II because perception is the interface through which everything about the physical world enters the stack, and because the later chapters on specific sensors and algorithms make more sense once the reader has a clear map of where each fits.

A useful framing: downstream safety is upper-bounded by perception. If the perception layer silently misses a pedestrian at 60 m, no amount of sophistication in the planner can produce a safe maneuver—you cannot avoid an obstacle you do not know about. Conversely, a high-quality perception output lets even a modest planner achieve good behavior, because the decision problem is well-posed. The asymmetry means that perception quality, measured the right way, is the single strongest predictor of end-to-end safety metrics in benchmarked AV programs.

This chapter develops three ideas. First, we place perception in context: how it relates to prediction, planning, and control; which problems perception solves and which it explicitly does not; and how information flows across module boundaries. Second, we walk through the canonical perception stack architecture—sensors, drivers, signal processing, feature extraction, detection, tracking, fusion, and scene-level interpretation—with pointers to the chapters that develop each stage in detail. Third, we survey the SAE levels of driving automation (introduced in Chapter 1 and expanded here with their implications for what perception must do) and the long-tail, distribution-shift, and compute challenges that make real-world perception hard.

The chapter is deliberately light on formulas: the mathematical content arrives in the chapters it introduces. The architecture diagrams, however, are the spine: Figure 3.1 visualizes the canonical perception stack, and Figure 3.3 summarizes the SAE-level hierarchy as constraints on what the

system must handle. Worked examples illustrate how a single scene flows through the stack and how apparently minor perception failures cascade.

3.2 The Role of Perception in the Autonomy Stack

Figure 3.1 shows where perception sits. Raw sensor signals come in from the left; structured scene descriptions flow out to the right, feeding prediction and planning. Control closes the loop back to the physical vehicle. The boundaries between modules are not arbitrary—they are engineering conveniences that let large teams work in parallel and let the system degrade gracefully when parts fail—but they are also not sacred. Modern “end-to-end” architectures (Chapter 22) blur or erase these boundaries in exchange for data-efficiency gains, at the cost of interpretability.

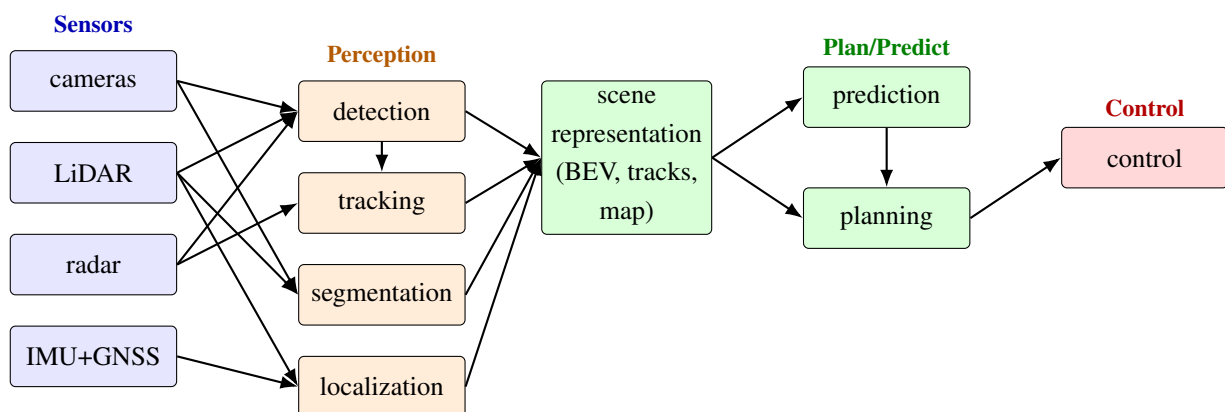


Figure 3.1: Canonical autonomy stack. Sensors (left) feed perception modules that produce detections, tracks, segmentations, and localization. These combine into a unified *scene representation* that prediction and planning consume. The planner’s output is a reference trajectory; the control module turns it into actuator commands. Perception is the interface between the physical world and everything downstream: the quality of its output bounds the quality of the whole system’s behavior.

3.2.1 What Perception Is Responsible For

Concretely, the perception stack must produce at each timestep:

- **Dynamic agents:** all vehicles, pedestrians, cyclists, and other moving things in the environment, with position, velocity, orientation, extent, and class.
- **Static infrastructure:** lane geometry, traffic signs, traffic lights, intersections, stop lines, crosswalks.
- **Free space:** which regions the ego could safely occupy, often encoded as BEV occupancy or drivable-area masks.
- **Ego localization:** the ego’s 6-DoF pose in a global frame (GNSS / map-relative) with calibrated uncertainty.
- **Scene semantics:** road weather, visibility, time of day, unusual conditions (construction zones, emergency vehicles).

Each item corresponds to one or more chapters downstream. Detection and tracking is Chapter 11; segmentation Chapter 12; localization and HD maps Chapters 13 and 14; scene-level perception flows through prediction in Chapter 17.

3.2.2 What Perception Is Not Responsible For

The clean division of labor between perception and the downstream layers is:

- Perception is *descriptive*, not *prescriptive*. It states what the world is; deciding what to do about it is the planner's job.
- Perception is *short-horizon*. It reports the current state. Predicting the future is the prediction module's job, though increasingly the boundary is blurry (occupancy flow networks predict one or two seconds of motion as a perception output).
- Perception is *probabilistic*. Its outputs should carry calibrated uncertainty. The planner then reasons with the uncertainty, not around it.

Architectural disputes in the AV community are largely about where to cut this boundary. End-to-end architectures (Chapter 22) question whether an intermediate, structured perception output is even the right interface; instead, a single network maps sensor data to plans.

3.3 The Canonical Perception Stack

Within the perception box of Figure 3.1, a consistent set of stages does the work.

Stage 1: Sensor drivers and synchronization. Each sensor produces timestamped data; a synchronization layer aligns streams to a common clock so that data products can be meaningfully combined. Chapter 10 develops this in detail.

Stage 2: Signal processing. Raw sensor data is preprocessed: camera demosaicking and auto-exposure, LiDAR pulse detection and motion undistortion, radar FFT and CFAR, IMU integration. Chapters 5 and 6 develop the LiDAR and radar specifics.

Stage 3: Feature extraction. Deep backbones (Chapter 4, Chapter 5) transform the preprocessed data into learned features. These may remain in the native sensor domain or be projected to BEV.

Stage 4: Per-task heads. Detection, segmentation, pose regression, etc., each a differentiable head attached to the shared backbone. Chapters 11 and 12 develop the per-task methods.

Stage 5: Tracking and fusion. Per-frame detections become temporally-consistent tracks with identity, velocity, and history. Multi-sensor fusion (Chapter 10) combines evidence across modalities.

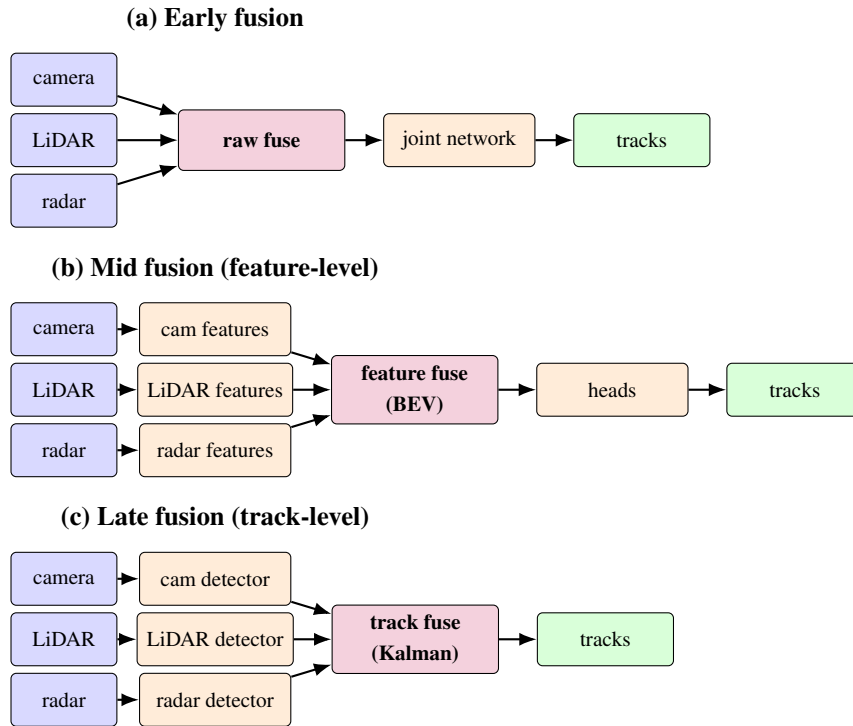


Figure 3.2: The three canonical sensor-fusion hierarchies. *Early* fusion (a) concatenates raw sensor data and runs a joint network – maximum information preserved, but joint calibration must be exact and sensor failures propagate. *Mid* fusion (b) extracts features per modality, then fuses in a shared latent space (typically BEV) – the dominant production pattern: BEVFormer-style architectures live here. *Late* fusion (c) runs separate per-modality detectors and combines at the track level via Kalman-filter or graph methods – modular, robust to sensor failure, but loses cross-modal correlations.

Stage 6: Scene representation. The collected outputs are assembled into a structured scene representation that downstream modules consume. This is often a BEV grid with several overlays (occupancy, semantics, agents, map), together with a tracked agent list and the ego’s localized pose.

An important property of this architecture: *redundancy is baked in*. Each stage can, with degraded quality, operate from a subset of its inputs. A camera-only perception stack can produce usable outputs when LiDAR is disabled; a LiDAR-only stack degrades more gracefully in fog than camera-primary. A production system tracks the per-stage health continuously and reports perceived capability to the planner so that plans become conservative when perception quality is below nominal.

3.4 Levels of Autonomy

The SAE J3016 standard defines six levels of driving automation (Chapter 1 develops these fully). Figure 3.3 summarizes them with an emphasis on what each level requires of perception.

L0 no auto	Warnings only. Perception detects hazards, alerts driver. No actuation.
L1 driver assist	One axis (ACC <i>or</i> LKA). Perception narrower: follow-distance, lane markings.
L2 partial	Longitudinal <i>and</i> lateral. Perception adds object detection, free-space. Driver monitors.
L3 conditional	Eyes-off within ODD. Perception must flag its own decline and hand back to the driver with seconds of margin.
L4 high	No human fallback within the ODD. Perception must support full minimum-risk condition (pull over, stop safely) autonomously.
L5 full	Unconstrained ODD. Perception must handle every scene a human could. Not yet achieved at scale.

Figure 3.3: SAE J3016 levels and their perception implications. Perception requirements grow non-linearly with level: L0-L2 perception can operate with a driver as backup; L3 must self-monitor and hand back with warning; L4/L5 must self-recover from any perception failure without human intervention. Most commercial deployments in 2024-2026 operate at L2 (consumer) or L4 within constrained ODDs (robotaxi). L5 remains a research target.

The jump from L2 to L3 is architecturally the hardest: L2 has a human driver whose continuous gaze is the ultimate safety net, so perception can miss things and be corrected. L3 promises “eyes off” within a defined operational design domain (ODD), which means perception must itself detect when it is operating out of distribution and pass control back to the human with enough margin (regulatory minimum typically 10 seconds) to resume safely. The self-monitoring requirement is radically different from what L2 asks for.

The jump from L3 to L4 is that there is no driver to return to. If perception cannot trust its own output, the vehicle must bring itself to a safe stop (a “minimum risk condition”) using *degraded* perception. This is why production L4 systems carry substantial sensor redundancy (overlapping LiDAR + camera + radar coverage of every safety-critical region) and why so much work goes into graceful degradation as a feature, not just a failure mode.

3.5 Key Challenges in Real-World Perception

The state of the art measured on benchmarks has been solid for years. The state of the art *deployed in the real world*, with the reliability required to remove a safety driver, remains hard. Five classes of challenge account for most of the gap.

3.5.1 Long-Tail and Rare Events

A driving scene distribution has a heavy tail: ordinary vehicles and pedestrians are common; unusual configurations (a mattress in the road, a child on a scooter behind a delivery truck, a wheelchair user crossing at an angle) are rare but disproportionately represented in the failure set. A detector trained

to 99 % average precision on a balanced benchmark may still fail dangerously on the last 1 %, because that 1 % is where the novel-and-hazardous live.

Example 3.1 (Long-tail driving the fleet data strategy). A production AV program logs a scenario of interest once every 10 000 miles: an unusual construction-zone configuration that its perception stack currently handles poorly. To gather enough examples for meaningful retraining (say, 500 scenarios), the fleet must drive 5 000 000 miles—a year of continuous operation for a 100-vehicle fleet. This is why “data flywheel” pipelines, in which fleet operation continuously mines scenarios for retraining, are seen as the core competitive advantage of the mature AV programs. Benchmarks cannot substitute for fleet data at this scale.

3.5.2 Distribution Shift

Perception networks trained on one distribution fail to different degrees on another. The shifts most commonly encountered:

- **Geographic.** Phoenix vs Pittsburgh vs Mumbai—road surfaces, driving norms, sign conventions, and even typical cars differ.
- **Temporal.** A new vehicle model enters the fleet distribution; a new construction pattern becomes common; a seasonal change in vegetation shifts the LiDAR background.
- **Sensor.** An upgrade from 64-beam to 128-beam LiDAR doubles point density and shifts every downstream statistic.
- **Environmental.** Fog, rain, snow, and low sun angles each produce data distributions absent from clear-weather training sets.

Domain adaptation, self-supervised pretraining, and continual learning (Chapter 2) are the tools; none of them is a complete solution.

3.5.3 Sensor Failures and Adversarial Conditions

Every sensor has failure modes. Cameras saturate, LiDARs ice over, radars suffer interference from neighboring vehicles’ radars. Production systems must (i) detect these failures, (ii) continue operating with the remaining sensors, and (iii) communicate degraded capability to the planner. Adversarial conditions (deliberate laser-dazzle attacks on cameras, radar jamming, spoofed GNSS signals) are an active research area and an increasing concern for deployed systems.

Example 3.2 (Graceful degradation under a camera failure). A forward camera fails mid-drive (lens obstruction by debris). The perception stack detects the failure via an image-quality monitor (mean intensity collapse plus feature-density drop). It (i) removes the camera from downstream fusion, (ii) raises the uncertainty of any object that relied primarily on that camera, (iii) broadcasts degraded state to the planner. The planner reduces maximum speed and increases follow-distance until the issue is resolved. The vehicle continues to operate, just more conservatively. The same event with a single-camera stack would have required an immediate pull-over.

3.5.4 Uncertainty and Calibration

A detector that reports 80 % confidence on outputs that are actually right 60 % of the time is overconfident; the planner treats 80 % as more reliable than it deserves. Perception outputs must carry *calibrated* uncertainty—either directly from the model (ensembles, dropout, evidential learning) or via post-hoc calibration (temperature scaling, conformal prediction, isotonic regression). Chapter 2 develops the machinery; Chapter 32 develops the downstream safety implications.

3.5.5 Latency and Compute

Real-time means real-time. A model that produces perfect outputs 50 ms too late is worse than a slightly noisier model that hits its latency budget. Automotive accelerators are power-constrained (< 100W for the entire perception compute in most production systems) and operate in temperature ranges camera-lab benchmarks never see. The compute-efficient variants of this book’s architectures—MobileNet, EfficientNet, PointPillars, Swin-T—are not optional; they are the practical minimum.

Example 3.3 (Latency budget cascade). Total perception budget: 100 ms from sensor capture to usable scene output. Budget is subdivided across stages:

Stage	Budget (ms)	Share
Sensor capture + transfer	10	10 %
Preprocessing	20	20 %
Backbone forward pass	40	40 %
Task heads + NMS	15	15 %
Tracking + fusion	15	15 %
Total	100	100 %

Going over budget in any single stage means the whole pipeline blows its loop rate. Because of the tight interdependence, any architecture change is validated against a full-stack latency regression test, not just its own standalone benchmark.

3.6 How This Book Organizes Perception

With the above framework in place, the remainder of Part II develops the perception stack in the order dictated by the dataflow:

- **Chapter 4:** camera-based perception, starting from backbone design through to BEV-camera representations.
- **Chapter 5:** LiDAR-based perception, including 3D detection, segmentation, and point-cloud processing architectures.

- **Chapter 6:** radar signal processing, object detection and velocity estimation, and ultrasonic sensing for close-in safety.
- **Chapter 10:** sensor technologies and multi-sensor fusion, bringing the modality-specific chapters together through calibration, synchronization, and cross-modal fusion architectures.
- **Chapter 11:** detection and multi-object tracking, which assembles per-sensor outputs into scene-level agents.
- **Chapter 12:** semantic, instance, and panoptic segmentation for scene-level understanding.

The reader can read linearly or jump by modality. Cross-references throughout the text keep the threads connected.

3.7 Worked Examples

Example 3.4 (Perception output for a single timestep). At $t = 0$, the perception stack produces:

- Ego pose: $(x, y, \theta) = (123.45, 67.89, 0.34)$ m, m, rad with covariance diag(0.02, 0.02, 0.001).
- Dynamic agents: 12 tracks with state $(x, y, \theta, v, w, h, l, \text{class}, \text{conf})$.
- Free-space grid: 200×200 BEV cells flagged drivable/not.
- Lane graph: 4 nearby lane centerlines with successor/neighbor relationships.
- Traffic lights: 2 detected, one green with confidence 0.92, one red with confidence 0.88.

The planner consumes this entire structure and produces a reference trajectory. If the perception stack had missed a pedestrian in one of the free-space cells, the planner would plan through that cell—a direct safety failure. Every element of the output matters; none is optional.

Example 3.5 (Effect of a 100 ms perception delay). The perception stack nominally completes at 100 ms latency. During a software update, it slips to 150 ms—a 50 ms regression.

Additional ego travel before reaction.

$$\begin{aligned}\Delta d &= v_{\text{ego}} \cdot \Delta t_{\text{lag}} \\ &= 25 \text{ m/s} \times 0.05 \text{ s} \\ &= 1.25 \text{ m.}\end{aligned}$$

Consequence.

1.25 m of added travel before any reaction can begin is enough to convert a safe stop into a collision at a marginal follow-distance. At a lead-vehicle braking profile of 6 ms^{-2} , this distance is roughly 20 % of the minimum safe following gap at highway speeds.

This is why latency regressions are tracked as first-class safety signals in production AV programs, with CI pipelines that block merges when any component exceeds its budget.

3.8 Industrial Deployment

Production perception stacks in the AV industry are typically organised as modular pipelines (used by Waymo, Cruise, Pony.ai, Baidu Apollo) or as end-to-end learned systems (popularized by Tesla's announcement at AI Day 2021 and 2022, and adopted in research at Wayve and elsewhere). Mobileye, an Intel subsidiary headquartered in Israel and listed on NASDAQ, supplies vision-based ADAS perception systems integrated into vehicles from BMW, Volkswagen, Ford, Honda, Nissan, and many other manufacturers; the company has publicly reported tens of millions of EyeQ chips deployed in production vehicles. The SAE J3016 taxonomy that defines automation levels referenced throughout this book is maintained by SAE International, an industry-standards organisation; it is used as the reference framework by regulators including the U.S. National Highway Traffic Safety Administration (NHTSA) and the United Nations Economic Commission for Europe (UNECE).

3.9 Chapter Summary

Perception is the interface through which the physical world enters the autonomy stack; its quality upper-bounds the quality of everything downstream. The canonical stack runs sensor drivers, preprocessing, deep backbones, task heads, tracking, and fusion, producing a structured scene representation for prediction and planning. The SAE levels define what perception must handle; production L4 deployment requires handling of long-tail scenarios, distribution shift, sensor failures, and adversarial conditions that are not covered by benchmarks alone. The remainder of Part II develops each stage of the stack in turn, beginning with camera-based perception and proceeding through LiDAR, radar, sensor fusion, and finally detection and segmentation.

3.10 Exercises

1. For the SAE levels of Figure 3.3, identify the minimal perception capability (list specific sub-tasks) required at each level.
2. Draw a version of Figure 3.1 that omits an intermediate scene representation, connecting perception directly to planning. What are the implications for interpretability and for partial-failure handling?
3. A perception module reports 90 % confidence on detections that are correct only 70 % of the time. Describe two separate mechanisms for detecting this miscalibration in production.
4. Estimate how many fleet miles are required to collect 200 examples of a rare scenario that appears once per 30000 miles. How does this scale with fleet size?
5. For a Level 3 system with a 10-second handoff requirement, what is the minimum warning latency that the perception stack must provide when it detects out-of-distribution inputs? Assume ego speed 30 m s^{-1} .
6. A new camera model increases resolution from 2 MP to 8 MP. Estimate the impact on perception-stack latency at each stage, and propose two mitigations.

7. Describe three distinct forms of distribution shift that a perception system might encounter after deployment in a new city, and the mitigation most likely to help for each.
8. The planner receives a detection with 0.5 m position std and a detection with 0.05 m position std. How should the planner's reaction differ between the two? Discuss the role of calibrated uncertainty in this decision.
9. Sketch a monitor that detects camera failures, LiDAR failures, and radar failures, and the signals each monitor would use.
10. Compare the perception requirements of a robotaxi fleet (Level 4, urban) and a highway-Level 2 consumer system. List at least five ways in which the required perception differs.

Chapter 4

Deep Learning for Computer Vision

4.1 Introduction

Cameras produce more raw data than every other sensor on an AV combined. A single six-camera surround rig at 20 Hz streams roughly 1 GB s^{-1} of RGB imagery; across a billion-mile deployment, this amounts to petabytes of video a year. Extracting semantic understanding from that torrent—what is a pedestrian, what is a traffic light, which pixels are drivable road—is the central job of the perception stack, and deep neural networks are the only technology that has been shown to do this at the scale and robustness that driving requires. This chapter develops the architectures, training regimes, and task heads that turn camera pixels into the structured outputs subsequent chapters consume.

Three properties make AV vision different from benchmark computer vision, and they shape every design choice in what follows. First, the data distribution is *domain-specific and long-tailed*: roads and vehicles dominate, but safety is determined by the rare and underrepresented (a child chasing a ball, a truck with an odd cargo overhang, a reflective construction cone at dusk). Second, downstream consumers need *multi-resolution, multi-task outputs*: a detector needs a feature pyramid, a segmenter needs dense output, a BEV perception head needs metric-space features, a lane module needs high-resolution geometry. Third, the entire pipeline must run at 10 Hz to 30 Hz on automotive silicon, so architecture choices are inseparable from their FLOP, memory, and latency budgets—a technique that gains 1 % mAP while doubling compute routinely loses to the simpler baseline on deployment.

The chapter is structured to follow the flow of a modern vision pipeline. We begin with convolutional backbones—AlexNet to ResNet to EfficientNet—because they remain the first operation on every camera frame. We then treat vision transformers (ViT, Swin) which have become co-equal to CNNs for high-capacity feature extraction. The auxiliary tasks follow: monocular and self-supervised depth, optical flow, and representation-learning paradigms (contrastive, masked autoencoder, CLIP-style). These produce the feature priors that subsequent detection, segmentation, and prediction chapters consume. The chapter’s centerpiece is the bird’s-eye-view (BEV) pipeline: lift-splat-shoot [52] and BEVFormer [44] explicitly convert multi-camera features into a unified ground-plane representation that is now the de facto spatial substrate for downstream detection, prediction, and planning. We

close with training and evaluation concerns: data augmentation tuned for driving, metrics that actually matter for safety, and interpretability tools for debugging. Each formula is accompanied by a worked numerical example, and key architectures are visualized with diagrams.

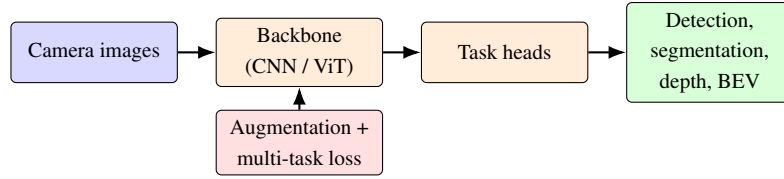


Figure 4.1: Chapter roadmap. Camera input flows through a shared backbone into multiple task heads. Augmentation and multi-task loss weighting (lower) regularise the shared representation across the heads.

4.2 Convolutional Backbones

Convolutional neural networks are the first operation on every camera frame in an AV stack. They extract hierarchical features—edges and textures at early layers, object parts in the middle, semantically rich representations at the top—that detectors, segmenters, and trackers consume. A modern backbone is a pipeline of conv blocks, normalizations, and residual connections, tuned to balance accuracy against the latency budget available on an automotive accelerator.

4.2.1 From LeNet to ResNet

The family of CNNs relevant to AV perception is the one descending from AlexNet [40], VGG, and most importantly ResNet [29]. ResNet introduced residual connections that solved the vanishing-gradient problem and unlocked networks of 50, 101, and 152 layers that routinely beat shallower alternatives on ImageNet and on driving-domain transfer:

$$\mathbf{h}^{(\ell+1)} = \mathbf{h}^{(\ell)} + F(\mathbf{h}^{(\ell)}; \boldsymbol{\theta}^{(\ell)}), \quad (4.1)$$

where F is typically a two- or three-layer bottleneck (conv, BatchNorm, ReLU, conv, BatchNorm) applied to $\mathbf{h}^{(\ell)}$ and added back.

Example 4.1 (Residual connection preserves gradient norm). Compare a 50-layer plain CNN and a residual network through the lens of backward-pass gradient attenuation.

Plain network with per-layer gradient-multiplier 0.8:

$$\begin{aligned} \text{attenuation} &= 0.8^{50} \\ &\approx 1.4 \times 10^{-5}. \end{aligned}$$

A vanished gradient; training stalls.

Residual network:

$$\frac{\partial \mathbf{h}^{(\ell+1)}}{\partial \mathbf{h}^{(\ell)}} = \mathbf{I} + \frac{\partial F}{\partial \mathbf{h}^{(\ell)}}.$$

The identity term guarantees the gradient norm does not collapse through the stack even if F produces small gradients.

Empirical consequence.

ResNet-50 trains to 76 % ImageNet accuracy while a plain 50-layer CNN struggles to reach 70 %. This is why every production perception backbone includes residual connections.

The stages of a modern backbone halve spatial resolution and double channel count as depth increases: input $H \times W \times 3 \rightarrow H/4 \times W/4 \times 64 \rightarrow H/8 \times W/8 \times 128 \rightarrow \dots \rightarrow H/32 \times W/32 \times 512$. This feature pyramid is what detectors consume.

4.2.2 Batch Normalization and Alternatives

Batch normalization [33] normalizes each channel's activations over the minibatch:

$$\hat{x}_{b,c} = \frac{x_{b,c} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}, \quad y_{b,c} = \gamma_c \hat{x}_{b,c} + \beta_c, \quad (4.2)$$

stabilizing training and typically accelerating convergence by 2–3 $\times$.

Example 4.2 (BatchNorm on a mini-batch of activations). A channel receives four activations $x = (1.2, 0.8, 1.0, 2.0)$ in a minibatch. Learned affine parameters $\gamma = 1.2$, $\beta = 0.3$.

Step 1: Batch statistics.

$$\begin{aligned} \mu &= \frac{1.2 + 0.8 + 1.0 + 2.0}{4} = 1.25, \\ \sigma^2 &= \frac{(1.2 - 1.25)^2 + (0.8 - 1.25)^2}{4} \\ &\quad + \frac{(1.0 - 1.25)^2 + (2.0 - 1.25)^2}{4} \\ &= 0.2075, \\ \sigma &\approx 0.456. \end{aligned}$$

Step 2: Normalize.

$$\begin{aligned} \hat{x} &= \frac{x - \mu}{\sigma} \\ &\approx (-0.110, -0.988, -0.549, 1.647). \end{aligned}$$

Step 3: Apply affine.

$$\begin{aligned} y &= \gamma \hat{x} + \beta \\ &\approx (0.168, -0.886, -0.359, 2.276). \end{aligned}$$

Deployment pitfall.

At inference, running-mean/running-var estimates replace the batch statistics. If the inference distribution differs from the training distribution (e.g., the car drives in rain the model rarely saw), the frozen BN stats become miscalibrated. This is one reason large driving labs gate every deployment behind a regression test on a diverse validation set.

BatchNorm degrades at small batch size, because σ_c becomes a noisy estimate. Detection and segmentation pipelines, which are memory-bound and hence use small batches, prefer GroupNorm [72] or LayerNorm. A modern AV stack often deploys BatchNorm for backbones (fused at inference) and GroupNorm or LayerNorm in heads.

4.2.3 Efficient Backbones for On-Vehicle Deployment

ResNet is the workhorse, but its FLOP cost is too high for some deployment targets. Two families of efficient backbones address this.

MobileNets [32] replace standard convolutions with depthwise-separable convolutions: first a depthwise filter per channel, then a 1×1 pointwise convolution mixing channels. This reduces the FLOPs of a 3×3 convolution on C_{in} input and C_{out} output channels from $9 \cdot C_{\text{in}} \cdot C_{\text{out}}$ per pixel to $9 \cdot C_{\text{in}} + C_{\text{in}} \cdot C_{\text{out}}$ per pixel: roughly an order of magnitude cheaper for typical channel counts.

Example 4.3 (FLOP savings from depthwise-separable). Compare standard vs depthwise-separable convolution at $C_{\text{in}} = C_{\text{out}} = 256$.

Standard 3×3 convolution, FLOPs per pixel:

$$\begin{aligned}\text{std} &= k^2 \cdot C_{\text{in}} \cdot C_{\text{out}} \\ &= 9 \cdot 256 \cdot 256 \\ &= 589,824.\end{aligned}$$

Depthwise-separable (3×3 depthwise + 1×1 pointwise), FLOPs per pixel:

$$\begin{aligned}\text{ds} &= k^2 \cdot C_{\text{in}} + C_{\text{in}} \cdot C_{\text{out}} \\ &= 9 \cdot 256 + 256 \cdot 256 \\ &= 2,304 + 65,536 \\ &= 67,840.\end{aligned}$$

FLOP ratio:

$$\begin{aligned}\frac{\text{std}}{\text{ds}} &= \frac{589,824}{67,840} \\ &\approx 8.7 \times .\end{aligned}$$

Full-map scaling.

Over a 256×512 feature map, the savings are dramatic: 77 GFLOPs (standard) vs 8.9 GFLOPs (depthwise- separable) for one layer. MobileNets exploit this pattern to hit on-device latency budgets that plain ResNet could not.

EfficientNet [62] co-scales depth, width, and input resolution using a compound coefficient ϕ :

$$\text{depth } d = \alpha^\phi, \quad \text{width } w = \beta^\phi, \quad \text{resolution } r = \gamma^\phi, \quad \alpha\beta^2\gamma^2 \approx 2. \quad (4.3)$$

Constraint $\alpha\beta^2\gamma^2 \approx 2$ ensures total FLOP count doubles for every unit increase in ϕ ; for EfficientNet-B0 through B7, $\phi = 0, \dots, 7$. The framework formalizes the intuition that accuracy benefits from balanced scaling, not from scaling any single axis in isolation.

Example 4.4 (FLOP scaling across EfficientNet-B0 to B3). EfficientNet is designed so each step up in ϕ doubles the total FLOP count.

Variant	ϕ	GFLOPs	Top-1 ImageNet
B0	0	0.39	77.1 %
B1	1	0.78	79.1 %
B2	2	1.55	80.1 %
B3	3	3.10	81.6 %

Accuracy gain per doubling of FLOPs.

$$\Delta(\text{B0} \rightarrow \text{B1}) = +2.0\text{pp},$$

$$\Delta(\text{B1} \rightarrow \text{B2}) = +1.0\text{pp},$$

$$\Delta(\text{B2} \rightarrow \text{B3}) = +1.5\text{pp}.$$

The accuracy gain per doubling shrinks—a recurring pattern in deep learning—so AV deployment picks the B-level just before the FLOP-for-accuracy curve inflects, typically B0–B2 for on-vehicle and B4–B5 for offline auto-labeling.

Figure 4.2 places the main backbone families on a compute-versus-accuracy plane. The shape of the Pareto frontier—and where each family sits on it—drives most production architecture decisions.

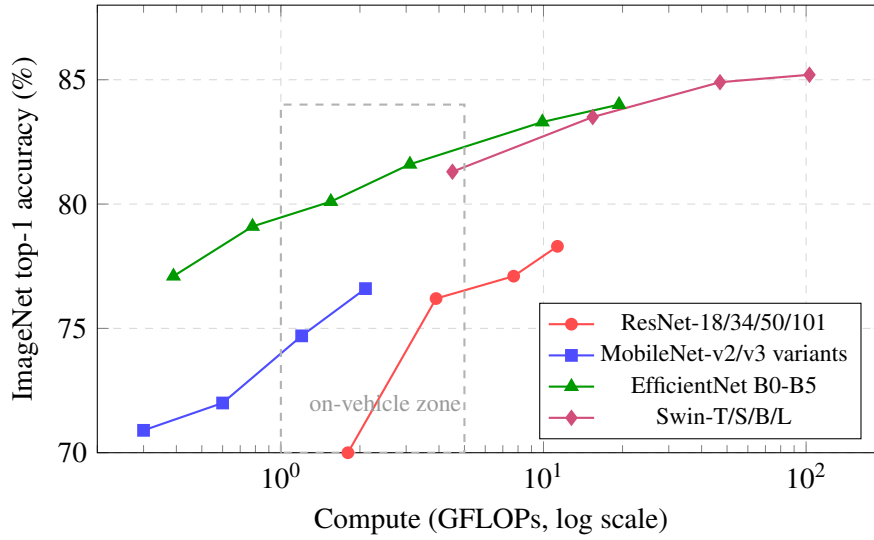


Figure 4.2: Accuracy-vs-compute Pareto frontier for major backbone families. EfficientNet dominates the mid-compute on-vehicle zone; MobileNets dominate the ultra-low-compute left edge; Swin transformers dominate the right edge where offline auto-labeling runs. ResNet remains competitive at mid-compute as a battle-tested baseline with excellent tooling support. The dashed box roughly delineates the deployment envelope for a typical automotive-grade accelerator running multi-camera inference at 30 Hz.

4.3 Vision Transformers

Transformers (Chapter 2) were first applied to vision via the Vision Transformer [18], which tokenizes an image into 16×16 patches and applies standard transformer blocks over the resulting sequence of $\approx N = 200$ tokens for a 224×224 input. ViTs rival CNNs at scale but require more data to train from scratch; pretrained on ImageNet-21k or larger they match or exceed ResNets on transfer.

Swin Transformer [46] made transformers tractable for dense-prediction tasks (detection, segmentation) by introducing *shifted window attention*: self-attention is computed within non-overlapping windows, with the windows shifted between consecutive layers to allow cross-window information flow. Cost drops from $\mathcal{O}(N^2d)$ to $\mathcal{O}(W^2Nd)$ with $W \ll N$ the window size.

Example 4.5 (Window attention vs full attention cost). Image feature map of $128 \times 128 = 16,384$ tokens at embedding dim $d = 128$.

Full self-attention cost.

$$\begin{aligned}
 \text{FLOPs}_{\text{full}} &= N^2d \\
 &= 16,384^2 \times 128 \\
 &\approx 3.4 \times 10^{10}.
 \end{aligned}$$

Swin window attention with $W = 7$.

Tokens per window: $W^2 = 49$. Number of windows: $(128/7)^2 \approx 334$.

$$\begin{aligned}\text{FLOPs}_{\text{swin}} &= N_{\text{win}} \cdot W^4 \cdot d \\ &= 334 \times 49^2 \times 128 \\ &\approx 10^8.\end{aligned}$$

Cost ratio.

$$\begin{aligned}\frac{\text{full}}{\text{swin}} &\approx \frac{3.4 \times 10^{10}}{10^8} \\ &\approx 340 \times .\end{aligned}$$

Two orders of magnitude cheaper. This is why Swin-style windowed transformers dominate dense-prediction driving perception benchmarks while flat ViTs remain research-grade for such tasks.

Driving-specific transformers include **BEVFormer** [44], which uses spatial cross-attention over a learned BEV-grid query set to combine multi-camera features into a unified BEV tensor, and **DETR3D**, which decodes 3D queries directly from multi-view image features. Both are now production architectures at several robotaxi developers.

4.4 Monocular Depth Estimation

A single image has no depth. But most of the time, there is enough prior information in the scene—perspective, shading, occlusion, object priors—for a neural network to *estimate* depth, and for many AV purposes an estimate within 5 % to 10 % is useful. Monocular depth complements LiDAR (which is sparse at range) and enables camera-only stacks.

4.4.1 Supervised Depth

Given paired images and ground-truth depth (from LiDAR re-projection), supervised networks regress per-pixel depth. The Eigen loss [19] accounts for the scale ambiguity of depth:

$$\mathcal{L}(\hat{d}, d^*) = \frac{1}{n} \sum_i (\log \hat{d}_i - \log d_i^*)^2 - \lambda \left(\frac{1}{n} \sum_i (\log \hat{d}_i - \log d_i^*) \right)^2, \quad (4.4)$$

with $\lambda \in [0, 1]$. The second term removes the optimum over global scale, making the loss scale-invariant at $\lambda = 1$.

Example 4.6 (Scale-invariance property). Predictions $\hat{\mathbf{d}} = (10, 20, 30)$, truth $\mathbf{d}^* = (5, 10, 15)$ (uniform $\times 2$ scale error). Log-MSE (first term): mean of $(\log 2)^2$ over three entries = 0.480. Mean log-error: $\log 2 = 0.693$; squared: 0.480. At $\lambda = 1$, $\mathcal{L} = 0.480 - 0.480 = 0$ —the uniform scale error contributes no loss. The network is free to get *relative* depth right even when absolute scale is ambiguous, a property that makes monocular networks workable despite the ill-posed

problem.

4.4.2 Self-Supervised Depth from Stereo and Video

Self-supervised methods learn depth without ground truth by warping one image into another using a predicted depth and relative pose, and minimizing photometric error. For stereo with known baseline, Godard et al. [25] and variants minimize

$$\mathcal{L}_{\text{photo}} = \sum_{\mathbf{u}} \rho(I_t(\mathbf{u}) - I_{t'}(\pi(\mathbf{u}, \hat{d}(\mathbf{u}), \mathbf{T}_{t \rightarrow t'}))), \quad (4.5)$$

where π warps pixel $\mathbf{u}$ via depth $\hat{d}$ and relative pose $\mathbf{T}_{t \rightarrow t'}$, and ρ is a robust photometric kernel (typically SSIM + ℓ_1). A second network predicts the relative pose between frames, so the full system trains end-to-end without any external ground-truth depth.

Example 4.7 (Photometric loss for a single pixel). Source pixel $I_t(\mathbf{u}) = 125$ (DN). Predicted depth $\hat{d} = 20$ m; predicted relative pose forward motion $(0.5, 0, 0)$ m. Target frame pixel at warped location $I_{t'}(\mathbf{u}') = 123$, giving absolute difference 2. If the network mispredicts depth as $\hat{d} = 15$ m, the warped target pixel is at a different location where $I_{t'}(\mathbf{u}'') = 180$: difference 55. The photometric loss therefore has a clear gradient toward the correct depth. The technique breaks in homogeneous regions (sky, blank walls) where any depth produces low photometric error—this is why self-supervised depth networks predict noisy estimates on sky pixels, which downstream modules must discount.

Monocular depth has an intrinsic scale ambiguity that self-supervised training does not resolve. Production systems either recover scale from stereo cues, from vehicle velocity via visual-inertial odometry, or from sparse LiDAR anchors. Without a scale reference, monocular depth is useful for *relative* reasoning (this object is closer than that one) but cannot directly produce metric distances.

4.5 Optical Flow

Optical flow estimates per-pixel 2D velocity between consecutive frames, providing motion cues that support dynamic-object segmentation, feature tracking, and self-supervised depth. Modern flow networks (RAFT [63]) iteratively refine a flow field using a learned all-pairs correlation volume, achieving sub-pixel accuracy on benchmark datasets at 20 Hz on an RTX-class GPU.

Example 4.8 (Time-to-collision from flow divergence). An ego-mounted camera observes a car directly ahead. The apparent size of the target grows with range; the divergence of the optical-flow field around a looming target projects to $\nabla \cdot \mathbf{f} \approx 2/\tau$ with τ the time-to-collision.

Given. The flow field expands by 2 % per frame at 30 Hz, so divergence per second is:

$$\begin{aligned} \nabla \cdot \mathbf{f} &= 0.02 \cdot 30 \\ &= 0.6 \text{ s}^{-1}. \end{aligned}$$

Time-to-collision.

$$\begin{aligned}\tau &= \frac{2}{\mathbf{V} \cdot \mathbf{f}} \\ &= \frac{2}{0.6} \\ &\approx 3.3 \text{ s.}\end{aligned}$$

Interpretation.

A 3.3-second warning window at highway speed leaves enough margin for a graceful brake but demands the alert be produced promptly. Flow-based TTC does not require an explicit object detector and is useful as a fallback safety check complementary to LiDAR/radar-based tracking.

For AV use, flow is most valuable as an *intermediate* signal: LiDAR and radar provide more reliable direct motion measurements, so camera flow is typically consumed by downstream modules for cross-checks or for scene-motion context rather than as a primary output.

4.6 Semantic Representations and Pretraining

Backbones trained on a specific task (say, vehicle detection) do not generalize as well as backbones that first learn a task-agnostic representation and then fine-tune. Pretraining is therefore the dominant paradigm. Three varieties matter for AV perception: supervised, self-supervised, and vision-language.

1. Pretrain

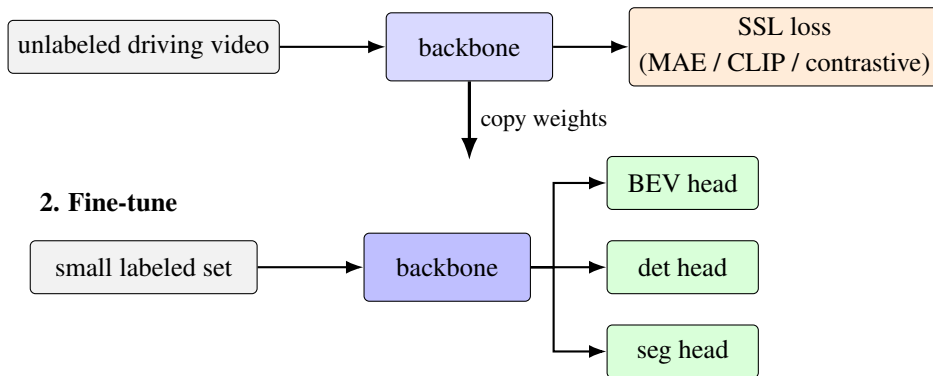


Figure 4.3: Two-stage pretraining paradigm for AV perception. Stage 1: a backbone learns a task-agnostic representation from large-scale unlabeled or weakly-labeled driving data using a self-supervised objective (MAE reconstruction, CLIP image-text alignment, or contrastive learning). Stage 2: the pretrained backbone is copied, extended with task heads (detection, segmentation, BEV perception), and fine-tuned on comparatively small labeled datasets. This scheme is how modern AV stacks exploit the petabytes of unlabeled video their fleets collect.

4.6.1 Supervised Pretraining

The original pretraining paradigm: train a backbone on ImageNet classification, transfer to downstream tasks. ImageNet bias (object-centric, web-scraped) is partly compensated by large scale. Nearly every

production detector starts from ImageNet-pretrained weights.

4.6.2 Self-Supervised Pretraining

When labels are scarce but images are abundant (AV case par excellence), self-supervised learning trains the backbone without external labels. Three families dominate.

Contrastive learning (SimCLR [13], MoCo, BYOL) pulls augmented views of the same image together and pushes different images apart in an embedding space, via the InfoNCE loss:

$$\mathcal{L}_{\text{NCE}} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_i^+)/\tau)}{\sum_j \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}, \quad (4.6)$$

with temperature τ typically in $[0.05, 0.2]$.

Example 4.9 (InfoNCE gradient pulls toward positive). Two normalized embeddings of the same scene $\mathbf{z}, \mathbf{z}^+$ have cosine similarity $s^+ = 0.80$; a negative from a different scene has $s^- = 0.20$. With $\tau = 0.1$: numerator $e^8 = 2981$; denominator $e^8 + e^2 \approx 2988$. $\mathcal{L} \approx -\log(2981/2988) \approx 0.0023$ —small, so the network is already near-optimal for this pair. If instead $s^+ = 0.30, s^- = 0.80$: $\mathcal{L} = -\log(e^3/(e^3 + e^8)) \approx 5.0$ —large. The gradient pulls $\mathbf{z}$ toward $\mathbf{z}^+$ and away from negatives.

Masked autoencoding (MAE [28]) randomly masks a fraction of image patches (typically 75 %) and trains an encoder-decoder to reconstruct them. The encoder ends up learning high-level semantic features that transfer well.

CLIP [54] aligns an image encoder and a text encoder such that paired image-text embeddings are close and mismatched pairs are far. For AV applications, CLIP-style pretraining enables open-vocabulary detection and text-conditioned scene search (“find all frames with a construction worker holding a sign”).

4.6.3 Vision-Language Pretraining

Beyond CLIP, larger vision-language models (BLIP, LLaVA, Flamingo) combine a vision encoder with a large language decoder, enabling detailed scene captioning, visual question answering, and instruction following. Applications in driving include automated data labeling, scenario mining, and (increasingly) high-level reasoning in end-to-end systems (Chapter 22).

4.7 Task Heads for Driving

A backbone produces a feature pyramid; *task heads* convert features into the outputs downstream modules need. Heads are smaller than the backbone but disproportionately shape the failure modes of the perception system.

Task head	Output type	Supervision	Typical loss
Classification	class label per box / pixel	class labels	cross-entropy
Bounding-box regression	2D or 3D box parameters	box annotations	smooth-L1 / IoU
Dense regression	depth, normals, optical flow	dense per-pixel	L1 / L2 / Huber
Semantic segmentation	per-pixel class	dense class masks	cross-entropy + Dice
Instance segmentation	per-pixel class + instance ID	instance masks	per-instance loss
Keypoint regression	specific points on objects	point annotations	MSE / heatmap
BEV perception	top-down occupancy, lanes, objects	BEV-projected GT	multi-task fusion

A production stack typically uses a shared backbone and multiple task heads, each with its own loss; the total loss is a weighted sum $\mathcal{L} = \sum_k w_k \mathcal{L}_k$ with w_k tuned by validation performance or learned adaptively. Multi-task training gives regularization benefits (each task acts as a prior on the shared features) but raises the risk of *negative transfer* if tasks conflict.

Example 4.10 (Multi-task loss weighting). Detection loss $\mathcal{L}_{\text{det}} = 1.2$, segmentation loss $\mathcal{L}_{\text{seg}} = 0.8$, depth loss $\mathcal{L}_{\text{depth}} = 2.5$. Equal weighting $\mathcal{L} = 1.2 + 0.8 + 2.5 = 4.5$: gradients are dominated by depth. Weighted to equalize scales ($w_{\text{det}} = 2, w_{\text{seg}} = 3, w_{\text{depth}} = 1$): $\mathcal{L} = 2.4 + 2.4 + 2.5 = 7.3$; each task contributes roughly one-third of the total. Uncertainty-based auto-weighting [36] replaces hand-tuned w_k with learned per-task noise variances; it is the production-standard approach.

4.8 Bird’s-Eye-View Representations

The most significant architectural shift in AV vision of the past five years is the move from per-camera image-plane reasoning to unified BEV (bird’s-eye-view) reasoning. A BEV representation projects every sensor’s features into a shared top-down grid (typically 200×200 cells over 100×100 m centered on the ego vehicle). Planners, predictors, and trackers consume this representation directly.

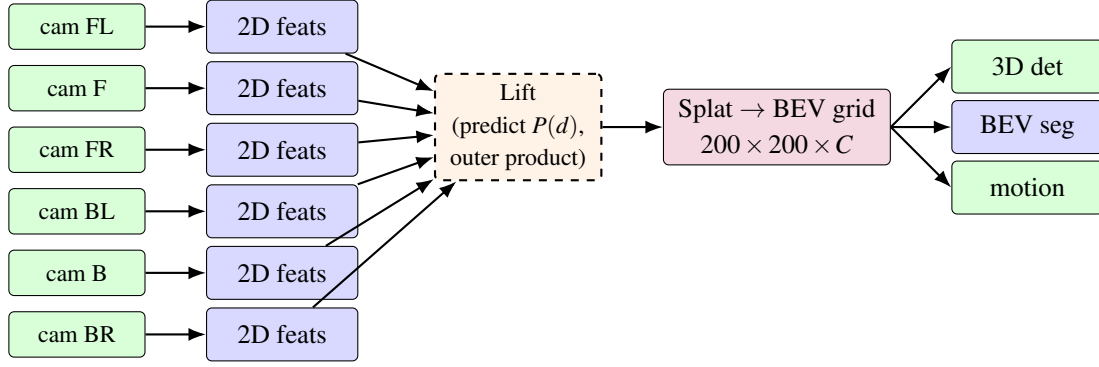


Figure 4.4: Lift-Splat-Shoot BEV architecture. Each camera’s image is passed through a 2D backbone (stage 1). The network then “lifts” each 2D feature into a 3D feature frustum by predicting a per-pixel depth distribution and taking the outer product with the feature (stage 2, Eq. 4.7). All lifted frustums are “splatted” into a common BEV grid in the ego frame (stage 3). BEV task heads (3D detection, BEV semantic segmentation, motion forecasting) consume the grid. The architecture is differentiable end-to-end and shares the BEV grid across all tasks—the key structural simplification that unifies modern multi-camera AV perception.

4.8.1 Lift-Splat-Shoot

Philion and Fidler [52] proposed the architecture depicted in Figure 4.4, which lifts each pixel into a 3D frustum via a per-pixel discrete depth distribution:

$$\mathbf{F}_{3D}(\mathbf{u}, d) = \mathbf{F}_{2D}(\mathbf{u}) \cdot P(d | \mathbf{u}), \quad (4.7)$$

where $P(d | \mathbf{u})$ is a learned softmax over D discrete depth bins (typically 40–80 bins from 2 m to 80 m). The lifted frustums from all cameras are splatted into a common BEV grid by weighted pooling; downstream task heads operate on the BEV tensor.

Example 4.11 (Lift arithmetic at a single pixel). A camera pixel has 2D feature $\mathbf{F}_{2D}(\mathbf{u}) \in \mathbb{R}^{64}$ and predicted depth distribution over $D = 41$ bins (2 m to 80 m in 2 m steps). Probabilities are peaked at $d = 20$ m ($P = 0.35$) and $d = 40$ m ($P = 0.25$), sum to 1. The lifted 3D feature for that pixel is a sparse volume where bin $d = 20$ carries $0.35 \cdot \mathbf{F}_{2D}$ and bin $d = 40$ carries $0.25 \cdot \mathbf{F}_{2D}$. The splat step accumulates these contributions into the BEV cell corresponding to the pixel’s 3D position at each depth. A bimodal depth distribution therefore correctly contributes to two BEV cells, preserving the network’s own uncertainty about the 3D location.

4.8.2 Temporal BEV

Single-frame BEV is a static snapshot; BEVFormer [44] extends this with temporal self-attention that fuses BEV features from previous timesteps, giving the representation implicit motion information. Temporal fusion also improves robustness to momentary occlusions.

4.9 Data Augmentation for Driving

Data augmentation is regularization via data diversity. For AV perception, augmentations must be domain-sensitive: a 10-degree color shift is fine, but a vertical flip of a driving image is nonsense (flipping up and down produces a scene impossible in reality). Standard augmentations used in production:

Augmentation	What it does	Caveat
Horizontal flip	doubles effective dataset size	flip directional labels (turn signals)
Random crop and resize	varies framing while preserving aspect ratio	matched aspect required
Color jitter	varies brightness, contrast, saturation, hue	avoid altering safety-relevant colours
Weather simulation	synthesizes rain streaks, raindrops, fog attenuation	quality varies; may not match real
MixUp and CutMix	linear-interpolated or region-swapped samples	can confuse small-object detection
Domain randomization	randomizes textures in simulator data	improves sim-to-real transfer
Sensor-aware augmentation	simulates LiDAR dropouts, bad pixels, shutter skew	needs sensor-model fidelity

4.10 Evaluation Metrics

Benchmark leaderboards measure perception quality with a handful of standard metrics, each an imperfect proxy for what matters to downstream safety.

Metric	Measures	Primary task
mAP	area under precision-recall (averaged over IoU thresholds)	2D / 3D detection
mIoU	per-class intersection-over-union, class-averaged	semantic segmentation
PQ	combined segmentation + instance-ID quality	panoptic segmentation
NDS	composite: class, localization, velocity	nuScenes 3D detection
HOTA	higher-order tracking: detection + association	multi-object tracking

None of these is the same as the downstream safety metric; a detector that improves nuScenes NDS by 2 points may not improve the planner’s collision rate by an equivalent amount. Closed-loop evaluation (Chapter 32) on realistic scenarios is the final adjudicator.

4.11 Interpretability and Debugging

A production perception network is too large to understand purely top-down, so practitioners lean on a handful of interpretability tools. Grad-CAM [59] visualizes which input regions contribute most to a given class score:

$$L_{i,j}^c = \text{ReLU}\left(\sum_k \alpha_k^c A_{i,j}^k\right), \quad \alpha_k^c = \frac{1}{Z} \sum_{i,j} \frac{\partial y^c}{\partial A_{i,j}^k}, \quad (4.8)$$

with A^k the activations of a chosen conv layer and y^c the class score.

Example 4.12 (Grad-CAM heat map of a false positive). A perception network mis-classifies a billboard as a pedestrian with confidence 0.82. Grad-CAM reveals that the network’s activations concentrate on a person-shaped print on the billboard, not on a real pedestrian. The debugging fix is to augment the training set with billboard images labelled as background, retraining until the activation pattern shifts. Without the heat map the engineer has only a confidence number and no signal as to *why* the network was wrong—a critical lack for a safety-critical system.

4.12 Worked Examples

This section consolidates a few end-to-end numerical calculations spanning the chapter’s concepts.

Example 4.13 (Full forward pass of a detection backbone). A ResNet-50 backbone takes a 768×1408 image as input and produces a 4-level feature pyramid. Strides and channel counts per level:

Level	Stride	Spatial	Channels	Activations
C2	1/4	192×352	64	4.33×10^6
C3	1/8	96×176	128	2.16×10^6
C4	1/16	48×88	256	1.08×10^6
C5	1/32	24×44	512	0.54×10^6
Combined				8.11×10^6

Activation memory per forward pass (FP16).

$$M = 8.1 \times 10^6 \times 2 \text{ B}$$

$$\approx 16 \text{ MB.}$$

Multi-camera scaling.

With four cameras processed concurrently, total activation memory is $\sim 64 \text{ MB}$ —comparable to the SRAM budget of a typical automotive accelerator. This is a binding constraint that drives architecture choices downward in resolution or channel count.

Example 4.14 (Comparing CNN vs ViT compute). ResNet-50: 4.1 GFLOPs per 224×224 image. ViT-Base: 16×16 patches give 196 tokens; hidden dim 768; 12 transformer blocks. FLOPs per block dominated by attention (N^2d) and FFN ($4Nd^2$ where 4 is the FFN expansion ratio): $196^2 \cdot 768 + 4 \cdot 196 \cdot 768^2 \approx 2.95 \times 10^7 + 4.6 \times 10^8 \approx 5 \times 10^8$ per block; 12 blocks give $\sim 6 \text{ GFLOPs}$. Swin-T is explicitly engineered to sit near 4.5 GFLOPs (matching ResNet-50). In practice, CNNs retain a latency edge on specialized accelerators; ViTs catch up as hardware

FLA-flops (Flash-attention-like optimizations) become standard.

Example 4.15 (Full BEV fusion step count). Six cameras at $H \times W = 200 \times 352$ effective feature resolution, $C = 64$ channels after a backbone, $D = 41$ depth bins. Per-camera lift operation: $200 \times 352 \times 64 \times 41 = 1.85 \times 10^8$ multiply-adds per camera, 1.1×10^9 across six cameras. Splat into BEV grid ($200 \times 200 \times 64$, 2.56×10^6 cells): bilinear scatter dominated by memory-bandwidth; sparse fraction of cells is written per camera. Total per-frame cost is on the order of 10 GFLOPs—tractable at 10 Hz on modern automotive accelerators. This is why lift-splat-shoot became the reference architecture: it packs multi-camera fusion into a single-pass differentiable pipeline.

4.13 Industrial Deployment

Deep convolutional and transformer architectures form the backbone of production AV perception. Tesla’s Autopilot perception stack, as described in their public “AI Day” presentations in 2021 and 2022, uses a multi-camera HydraNet architecture with shared backbones and task-specific heads. The BEVFormer architecture [43] from researchers at Shanghai AI Laboratory has influenced multiple production stacks for multi-camera bird’s-eye-view perception. NVIDIA’s DriveOS and the open-source Apollo project (Baidu) ship reference implementations of camera-based 3D detection. EfficientNet (Google Research) and Swin Transformer (Microsoft Research Asia) are widely used backbone families in both production and research AV systems. The ImageNet dataset, originated at Stanford and Princeton, remains the standard pretraining target whose feature representations transfer to automotive perception tasks.

4.14 Chapter Summary

Deep vision for AVs is a stack of learned components organized around three principles: strong pretrained backbones (CNNs or transformers, often on self-supervised objectives), multi-task heads sharing those backbones, and a unified BEV representation for downstream consumption. Efficient variants (MobileNet, EfficientNet, Swin) trade accuracy against compute to hit automotive latency budgets. Pretraining on massive unlabeled fleet data—contrastive, MAE, or CLIP-style—is the lever that overcomes the long-tail labeling cost. Evaluation mixes benchmark metrics (mAP, mIoU, NDS, HOTA) with closed-loop safety evaluation; the two do not always agree. The vision outputs described here feed the detection, tracking, and prediction stacks of the next three chapters.

4.15 Exercises

1. Show that the residual connection (Eq. 4.1) produces a gradient norm that does not collapse with depth, even when $\|\partial F / \partial \mathbf{h}\|$ is small at every layer.
2. Derive the FLOP savings of a depthwise-separable convolution over a standard convolution as a function of kernel size k and channel counts $C_{\text{in}}, C_{\text{out}}$. At what channel count does the saving

exceed $5\times$?

3. A Swin Transformer with window size $W = 7$ is applied to a feature map of $N = 96 \times 96$ tokens. Compute the attention FLOPs for windowed vs full attention.
4. Compute the scale-invariant term of the Eigen depth loss (Eq. 4.4) for predictions $(8, 16, 24)$ and truth $(10, 20, 30)$.
5. A contrastive learner has positive similarity $s^+ = 0.7$ and three negatives $(0.3, 0.2, 0.5)$, temperature $\tau = 0.1$. Compute the InfoNCE loss.
6. For the Lift-Splat-Shoot pipeline (Fig. 4.4), compute the total number of BEV cells receiving nonzero contribution from a single camera with 200×352 feature resolution and $D = 40$ depth bins, assuming each pixel lifts into all depth bins.
7. Propose three augmentations appropriate for AV camera training and three that should *not* be used. Justify each.
8. Grad-CAM highlights a road-sign area for a detector that misclassifies a pedestrian. Describe two training interventions that would fix the issue.
9. Explain why mIoU averaged over classes can hide poor pedestrian-class performance, and propose a stratified metric that reveals the problem.
10. An EfficientNet-B3 runs at 90 ms/frame on target compute, but the latency budget is 50 ms. Suggest three concrete actions to close the gap, quantifying the expected improvement of each.

Chapter 5

LiDAR Perception

5.1 Introduction

If camera-based perception is the AV’s semantic sense, LiDAR is its geometric sense. A modern automotive LiDAR produces hundreds of thousands of 3D points per frame with range accuracy of a few centimeters at 100 m to 250 m—precision that monocular depth networks cannot approach, and that stereo cameras give up at a fraction of that range. The result is a sensor that provides, for every frame, a direct metric measurement of the shape of the world within the vehicle’s envelope. Detection, tracking, segmentation, SLAM, and HD-map localization all lean on LiDAR when it is present. This chapter develops the physics, signal processing, point-cloud representations, and deep-learning architectures that turn raw laser returns into the structured outputs the rest of the stack consumes.

Chapter 10 later treats LiDAR in the broader context of sensor fusion: calibration against cameras and radar, time synchronization, and the Kalman-filter primitive that combines modalities. The present chapter is deeper in three directions. *First*, it treats LiDAR *hardware taxonomy* carefully: pulsed vs FMCW, mechanical vs MEMS vs solid-state vs flash. Each architecture has a characteristic signature in range, resolution, cost, and reliability, and the choice made by an AV program shapes every downstream decision. *Second*, it develops the *representations* used to process point clouds—raw point sets, voxel grids, range images, and BEV pillars—because representation choice determines which deep-learning architecture is applicable and what the compute profile looks like. *Third*, it covers the *learned architectures* that have come to dominate LiDAR perception: PointNet++, VoxelNet, PointPillars, CenterPoint, KPConv, MinkowskiNet, and transformer-based variants. The goal is to give a working engineer enough to read the modern literature and make architecture choices with numbers.

Three recurring themes shape LiDAR processing and worth stating at the start. *Sparsity*: a LiDAR point cloud occupies roughly 0.1–1 % of the cells in a dense voxel grid over its measurement volume; every efficient LiDAR architecture exploits this. *Anisotropy*: angular resolution is much better than range-wise resolution for a given sensor (beam divergence is tight, range is a function of distance),

producing elongated error ellipsoids that confuse off-the-shelf 3D methods. *Scanning geometry*: mechanical LiDAR returns arrive at different wall-clock times across the sweep, so a single “frame” is actually a temporal snapshot that must be motion-compensated against ego motion. Each is developed with worked examples below.

The chapter’s flow follows a natural pipeline: we begin with hardware and waveform processing, then move to point-cloud representations, then to the learned architectures that consume them, then to segmentation and odometry, and close with weather robustness, simulation, and datasets. Every formula is accompanied by a worked numerical example, and three architecture diagrams illustrate the key concepts.

5.2 LiDAR Hardware

The term “LiDAR” encompasses a surprisingly diverse set of architectures. The common thread is ranging via laser light, but the mechanism (pulsed vs continuous-wave) and the beam-steering technology (mechanical spinning, MEMS mirrors, optical phased array, flash illumination) each make a fundamentally different tradeoff in cost, range, resolution, and automotive reliability. Figure 5.1 visualizes the four main architectures.

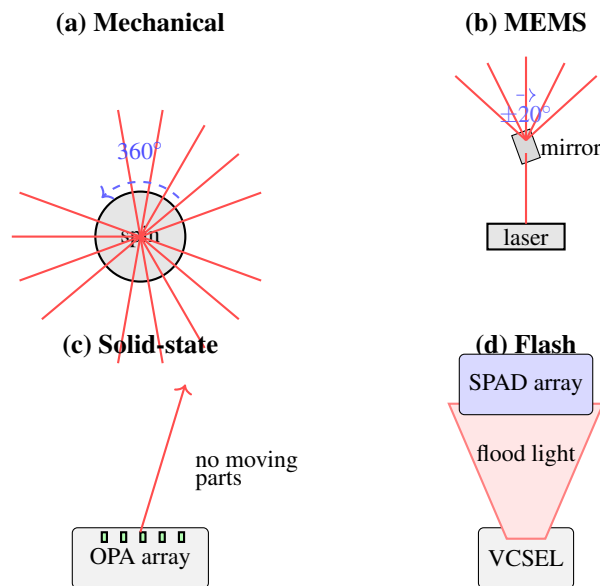


Figure 5.1: Four LiDAR beam-steering architectures. (a) *Mechanical* spinning: the whole optical head rotates, sweeping typically 64 or 128 laser channels through a full 360° . (b) *MEMS*: a tiny mirror deflects a single laser over a $\pm 20^\circ$ field of view; no bulk rotation, but still a mechanical moving part. (c) *Solid-state* (optical phased array): electronic beam steering with no moving parts; promising for automotive reliability but range-limited in current products. (d) *Flash*: VCSEL floods the scene; a SPAD array detects returns in parallel. Short-range (tens of metres), high frame rate.

5.2.1 Pulsed Time-of-Flight LiDAR

The dominant waveform in production automotive LiDAR is a short laser pulse ($\sim 1\text{--}10\text{ ns}$ wide). The receiver times the return edge with precision typically on the order of hundreds of picoseconds, giving

centimeter-scale ranging:

$$r = \frac{c\tau}{2}, \quad \sigma_r = \frac{c\sigma_\tau}{2}, \quad (5.1)$$

with c the speed of light and σ_τ the timing precision.

Example 5.1 (Photon budget at range). Laser peak power 100 W, pulse duration 5 ns, giving per-pulse energy $E_p = 5 \times 10^{-7}$ J. At 100 m range reflecting off a target with reflectance $\rho = 0.1$ (asphalt) and receiver aperture $A_{rx} = 25 \text{ cm}^2$, the returned energy is approximately $E_r = E_p \rho A_{rx} / (2\pi r^2) \approx 5 \times 10^{-7} \cdot 0.1 \cdot 2.5 \times 10^{-3} / (2\pi \cdot 10^4) \approx 2 \times 10^{-15}$ J. At photon energy $h\nu \approx 2 \times 10^{-19}$ J for 905 nm lasers, this is $\sim 10,000$ photons per return—well above a SPAD’s single-photon detection threshold. At 250 m with $r^2 = 6.25 \times 10^4$, returned photons drop to ~ 1600 , still detectable. Beyond 500 m, quantum noise begins to dominate. This quadratic falloff with range is why automotive LiDAR rarely advertises useful data beyond 200 m to 300 m.

Timing precision is bounded by the shape of the returned pulse and the electronics that digitize it. A 5 ns pulse with 100 ps leading-edge timing resolution gives $\sigma_r = c \cdot 100 \times 10^{-12} / 2 = 1.5$ cm, matching published automotive specifications. Tightening σ_τ below 50 ps requires time-to-digital converters with specialized design; it is rarely worth the cost since atmospheric variations and beam footprint dominate at shorter distances.

5.2.2 FMCW LiDAR

FMCW LiDAR (sometimes called “coherent LiDAR”) adapts the radar FMCW waveform to optical wavelengths:

$$r = \frac{cf_b}{2S}, \quad v = \frac{\lambda f_d}{2}, \quad (5.2)$$

where f_b is the beat frequency of the returned signal against a local oscillator, S is the chirp slope, and f_d is the Doppler shift. Unlike pulsed ToF, FMCW produces per-return velocity in a single measurement, and because the coherent detection rejects non-Doppler-matching noise, it is robust to ambient light and to solar jamming.

Example 5.2 (FMCW LiDAR per-point velocity). $\lambda = 1550$ nm (eye-safe telecom wavelength). A target moving at $v = 10 \text{ m s}^{-1}$ toward the sensor produces Doppler shift $f_d = 2v/\lambda = 20/1.55 \times 10^{-6} \approx 12.9$ MHz. Such a shift is clearly separable from the beat frequency (typically in the sub-GHz range) at the detector. Every LiDAR point carries $(x, y, z, v_{\text{radial}})$ rather than just (x, y, z) ; this lets a tracker distinguish moving from static objects without motion across frames. The technology is emerging in 2024+ production (Aeva, SiLC, Mobileye Chauffeur).

5.2.3 Mechanical Spinning LiDAR

The iconic rotating rooftop unit. A bank of 16, 32, 64, or 128 laser diodes is mounted on a spinning platform that completes one revolution per 100 ms to 50 ms (10–20 Hz). Each channel is fixed in elevation; the platform rotation provides azimuth coverage. Pros: full 360° FOV in a single frame, technologically mature, high angular resolution. Cons: a moving mechanical assembly (automotive

lifetime concerns), large form factor, cost (historically \$100k+ for high-end units, now \$1000-class for mass market).

5.2.4 MEMS-Mirror LiDAR

A micro-electromechanical mirror oscillates to steer a single laser across a limited FOV (typically $\pm 20^\circ$ horizontal, $\pm 10^\circ$ vertical). Four-to-six MEMS units cover the full forward hemisphere. The moving element is microscopic and sealed, eliminating the mechanical wear of a rooftop unit.

5.2.5 Optical Phased Array (True Solid State)

An optical phased array steers the beam electronically, with no moving parts, by adjusting the phase of emitters in a chip-scale array. The analog of MIMO in radar, it is the most automotive-reliable architecture in principle. Current products have modest range and FOV limitations but are closing the gap with each generation.

5.2.6 Flash LiDAR

A single laser pulse floods the entire FOV; a 2D SPAD array detects returns per pixel simultaneously, producing a dense depth image without scanning. Frame rates can exceed 30 Hz with no motion artifacts. The tradeoff: flood illumination divides the energy budget across many pixels, severely limiting useful range. Flash LiDAR is therefore most commonly found in short-range applications (driver-monitoring, in-cabin sensing, 10 m to 30 m automotive safety).

5.3 From Photons to Points: Waveform Processing

A raw LiDAR measurement is a time series of photon counts per channel per firing. Extracting clean 3D points from this time series is a signal-processing problem that the hardware vendor typically handles, but whose principles the perception engineer must understand because its failure modes determine the quality of the points that downstream code receives.

5.3.1 Peak Detection and Pulse Shaping

For pulsed ToF LiDAR, each firing produces a waveform at the receiver: ambient-light background plus, if a target is present, a return pulse whose shape roughly matches the transmitted pulse. Classical peak detection identifies the leading edge of the return pulse and stores a timestamp; more sophisticated processors fit a Gaussian to the digitized waveform and report the center of the fit.

Example 5.3 (Leading-edge vs Gaussian-fit ranging). A 5 ns pulse sampled at 1 GS/s produces 5 digital samples across the return. The leading-edge method reports the timestamp of the first sample above a 3-sigma threshold: precision limited by the sample period, 1 ns $\Rightarrow$ 15 cm. Gaussian fitting across all five samples yields sub-sample precision of 100 ps-ish via interpolation, giving 1.5 cm. Production automotive LiDAR overwhelmingly uses waveform fitting for this order-of-magnitude gain.

5.3.2 Multi-Return Processing

A single laser pulse can produce multiple returns: a leaf partially obstructs the beam, and the rest of the pulse energy hits the trunk behind. Trees, wet windshields, rain droplets, and translucent barriers all produce multi-return signatures. Modern LiDARs report up to 3 returns per firing (“first,” “strongest,” “last”), labeled by their order along the return path.

Example 5.4 (Multi-return in rain). A rain droplet at 4 m reflects a small fraction of the pulse; the rest continues to the road at 40 m. First return: droplet; last return: road. A perception system using only the “first return” (the default on many single-return sensors) would reject the scene as foggy; using the last return recovers the road geometry. Production AV LiDAR stacks explicitly process both.

5.3.3 Intensity Calibration

Each LiDAR return carries an “intensity” value—the amplitude of the detected pulse. In principle this encodes the target’s reflectivity at the laser’s wavelength; in practice it is contaminated by range (inverse-square falloff), angle of incidence (cosine falloff), and sensor-internal gain. A raw intensity image shows obvious radial brightness gradients; calibrated intensity approximates surface reflectivity.

$$I_{\text{cal}} = I_{\text{raw}} \cdot r^2 / \cos \theta_i, \quad (5.3)$$

with θ_i the angle of incidence to the surface normal. Calibrated intensity is useful for lane-line extraction (paint is $10\times$ more reflective than asphalt at 905 nm) and material classification.

Example 5.5 (Intensity calibration on asphalt and paint). A lane-line paint marking at $r = 20$ m and normal incidence ($\cos \theta_i = 1$) returns raw intensity $I_{\text{raw}} = 200$ (on a 0–255 scale). A patch of asphalt at $r = 60$ m at $\theta_i = 60^\circ$ returns $I_{\text{raw}} = 30$. Calibrated: paint $200 \cdot 400/1 = 80,000$; asphalt $30 \cdot 3600/0.5 = 216,000$, oddly larger? The raw numbers are per-vendor scaled; after proper normalization the ratio is paint/asphalt $\approx 10\times$ at comparable range. Production pipelines maintain lookup tables indexed by $(r, \theta_i, \text{temperature})$ because intensity is not a first-class metric across sensors or conditions.

5.4 Point Cloud Representations

A LiDAR frame is a set of $N \in [10^5, 2 \times 10^6]$ unordered 3D points, each with coordinates and auxiliary channels (intensity, ring index, timestamp). Turning this set into a tensor that a neural network can consume requires a choice of representation. The choice determines compute cost, memory layout, and which architectures are usable.

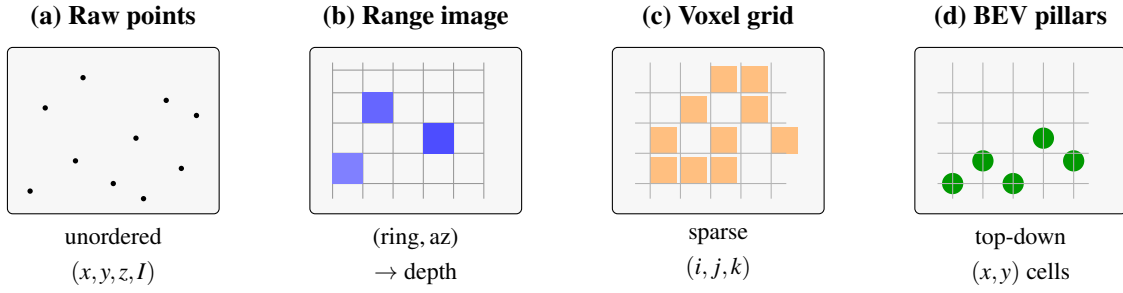


Figure 5.2: Four common LiDAR point-cloud representations. (a) *Raw point set*: unordered list of 3D coordinates plus per-point features; consumed by PointNet++ and KPConv. (b) *Range image*: the ring-azimuth structure of a mechanical LiDAR reshaped into a dense 2D image, consumed by RangeNet++ and SqueezeSeg. (c) *Voxel grid*: 3D occupancy in a fixed grid, extremely sparse (typically $<1\%$ occupied), consumed by VoxelNet and SparseConv-based networks. (d) *BEV pillars*: a top-down grid where each cell aggregates a vertical column of points, consumed by PointPillars and CenterPoint. Each representation trades fidelity against compute differently.

5.4.1 Raw Point Set

Store each point as a row $(x, y, z, I, \dots)$ in an $N \times d$ tensor. No quantization loss; memory scales with point count (a few MB per frame). The challenge: neural networks want ordered, permutation-dependent inputs; point sets are permutation-invariant. PointNet and PointNet++ solve this by applying per-point MLPs followed by a symmetric aggregation (max, mean).

5.4.2 Range Image

For mechanical LiDAR, the native acquisition structure is $(\text{ring}, \text{azimuth}) \rightarrow (\text{range}, \text{intensity}, \text{normal})$. Reshape into a $H \times W$ image with H rings and W azimuth bins. A 64×2048 image covers a 64-beam sensor at 2048 azimuth samples per revolution; this is small compared to camera images and runs fast on standard 2D CNN hardware. Downside: range images are irregular (missing returns appear as holes) and the ring-azimuth dimension is not Euclidean.

5.4.3 Voxel Grid

Discretize the space into 3D voxels of edge length Δ (typically 5 cm to 20 cm). Each voxel aggregates the points falling inside it, either by average, by max, or by a per-voxel feature network (VoxelNet). The grid is huge— $200 \times 200 \times 16 \approx 6.4 \times 10^5$ voxels in a standard automotive volume—but sparse: typically 1–5% of voxels contain any points. Sparse-conv libraries (MinkowskiEngine, spconv) process only occupied voxels, giving order-of-magnitude speedups over dense 3D CNNs.

Example 5.6 (Voxel-grid sparsity). Point cloud of $N = 180000$ points over a $120 \times 120 \times 4\text{m}$ volume. At $\Delta = 0.2\text{m}$, the grid is $600 \times 600 \times 20 = 7.2 \times 10^6$ voxels. Occupied voxel fraction: $\leq N / (\text{grid cells}) = 1.8 \times 10^5 / 7.2 \times 10^6 = 2.5\%$ (fewer in practice due to clustering). A dense 3D conv would touch all 7.2M cells; a sparse conv touches only ~ 180000 , a $40\times$ speedup. At $\Delta = 0.4\text{m}$ the speedup is less (coarser grid) but the information loss rises. The choice is application-specific.

5.4.4 BEV Pillars

Pillars [41] coarsen the voxel idea by collapsing the vertical dimension into a single learned feature vector per (x, y) cell: the “pillar.” The output is a dense 2D BEV feature map that downstream 2D CNN heads can process directly. This is the dominant representation for 3D object detection because it combines the geometric fidelity of voxels with the speed of 2D CNNs.

5.4.5 Tradeoffs

Table 5.1 summarizes. No representation dominates; production stacks often process the same frame in multiple representations in parallel, with each feeding a different task.

Table 5.1: Point-cloud representation tradeoffs.

Representation	Memory	Compute	Best for
Raw points	low	high	semantic seg, SLAM features
Range image	low	low	fast semantic seg (RangeNet++), segmentation
Voxel grid	high (sparse)	medium (sparse-conv)	3D detection, shape completion
BEV pillars	medium	low	3D detection (PointPillars, CenterPoint)

5.5 Calibration and Motion Compensation

A raw point cloud is expressed in the LiDAR’s own frame and corresponds to points acquired at different wall-clock times (within one sweep). Both properties must be corrected before the cloud can be used by any downstream algorithm.

5.5.1 Extrinsic Calibration to the Body Frame

The LiDAR-to-body extrinsic $\mathbf{T}_{BL} \in \text{SE}(3)$ is typically estimated offline with known targets and verified online against redundant sensors (camera-LiDAR edge correlation, IMU-LiDAR motion correlation). Chapter 10 covers extrinsic calibration in detail with a worked example showing that a 0.5° yaw miscalibration produces 0.44 m of lateral error at 50 m range.

5.5.2 Motion Undistortion

A mechanical LiDAR takes 50 ms to 100 ms to complete one sweep. During that time the ego vehicle moves at $10\text{--}30\text{ ms}^{-1}$, so points acquired at different azimuth angles belong to different ego poses. Reconstructing a single-time snapshot requires transforming each point from the pose at its individual timestamp to the pose at a chosen reference timestamp (typically the middle of the sweep):

$$\mathbf{p}_i^{\text{corr}} = \mathbf{T}_{t_{\text{ref}}, t_i} \mathbf{p}_i^{\text{raw}}, \quad (5.4)$$

with $\mathbf{T}_{t_{\text{ref}}, t_i}$ the relative ego pose between the per-point timestamp t_i and the reference timestamp. The relative pose is interpolated from the IMU-integrated ego trajectory, which must therefore run at a rate $\gtrsim$ the LiDAR’s sweep rate.

Example 5.7 (Motion undistortion at highway speed). Vehicle at 30 m s^{-1} , mechanical LiDAR sweeping 360° in 100 ms. A point acquired at the very start of the sweep ($t_i = -50 \text{ ms}$ from reference) was captured when the ego was 1.5 m behind its reference pose. Without correction, a static wall appears shifted by 1.5 m along the ego direction of motion. Azimuthally symmetric static objects appear as a $\sim 3 \text{ m}$ -long arc rather than points—a signature visible in any uncorrected sweep. Correction using the IMU-integrated trajectory reduces the arc to residuals below 5 cm.

Motion undistortion is especially important for subsequent ICP-style registration: ICP assumes each cloud is a single-time snapshot, so undistortion is a prerequisite to clean scan-matching.

5.6 Deep Learning on Point Clouds

Early LiDAR perception was dominated by hand-crafted geometric features (curvature, planarity, eigenvalues of the local covariance). Modern LiDAR perception is dominated by learned features, with four architectural families corresponding to the four representations above. Figure 5.3 illustrates the PointPillars pipeline as a representative example of the BEV-pillar family.

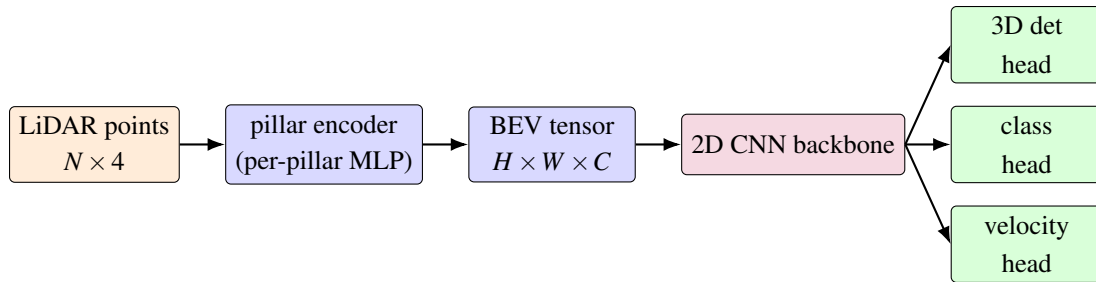


Figure 5.3: PointPillars architecture. Raw LiDAR points are assigned to BEV pillars (vertical columns over a (x, y) grid). A per-pillar MLP aggregates the points inside each pillar into a fixed-dimensional feature vector, producing a 2D BEV tensor. A standard 2D CNN backbone then extracts multi-scale features, which feed detection heads for 3D bounding-box regression, classification, and velocity estimation. The architecture trades the fidelity of full voxel processing for a $5\times$ speedup that keeps it tractable at 10 Hz on production automotive hardware.

5.6.1 PointNet and PointNet++

PointNet [53] operates directly on unordered point sets by applying the same MLP to every point independently, then aggregating with a symmetric operator (max):

$$f(\{\mathbf{p}_i\}) = \gamma \left(\max_{i=1, \dots, N} h(\mathbf{p}_i) \right), \quad (5.5)$$

where $h: \mathbb{R}^d \rightarrow \mathbb{R}^k$ is a per-point MLP and γ is a post-aggregation MLP. The symmetric max makes f permutation-invariant.

Example 5.8 (PointNet max-pool on a small cloud). Three points with features $h(\mathbf{p}_1) = (0.5, 0.2, 0.7)$, $h(\mathbf{p}_2) = (0.1, 0.8, 0.3)$, $h(\mathbf{p}_3) = (0.4, 0.3, 0.5)$. Per-feature max: $(0.5, 0.8, 0.7)$.

Each feature dimension “learns” a global aggregator: feature 1 is maximal at $\mathbf{p}_1$ (which might represent “farthest in x”), feature 2 at $\mathbf{p}_2$, feature 3 at $\mathbf{p}_1$. Different features can specialize to different geometric summaries. This is why PointNet works despite its simplicity: the network learns to carve up the feature space such that max captures useful statistics.

PointNet++ adds hierarchical grouping: nearby points are clustered via farthest-point sampling, each cluster is processed by a local PointNet, and the operation is repeated at coarser scales. The result is a multi-scale feature hierarchy analogous to a CNN’s feature pyramid.

5.6.2 Voxel-Based Methods

VoxelNet [78] was the first end-to-end 3D detector operating on voxelized point clouds. It applies a per-voxel PointNet to produce per-voxel features, then runs a 3D CNN over the voxel grid. SECOND, PV-RCNN, and Voxel-RCNN extend the idea with sparse convolution. The practical bottleneck is dense 3D conv cost; sparse-conv frameworks (spconv, Minkowski) reduce this by an order of magnitude at no accuracy cost.

5.6.3 Range-Image Methods

RangeNet++ and SqueezeSegV3 apply 2D CNNs directly to the range-image representation, exploiting the speed of 2D convolutions. The twist is that the range-image coordinates are not Euclidean: adjacent pixels differ by a fixed angular increment, which at close range corresponds to a few centimeters of geometric distance and at long range to several meters. Networks that ignore this produce bleeding across object boundaries; successful architectures use angle-aware convolutions or explicitly re-project features back to the point set for refinement.

5.6.4 BEV and Pillar Methods

PointPillars [41], CenterPoint, and BEVFormer-LiDAR flatten the point cloud into a BEV pillar representation (Figure 5.3) and apply a 2D CNN backbone with detection heads similar to those used on camera BEV tensors (Chapter 4). The shared BEV representation is the key enabler of multi-modal fusion: camera BEV features from Lift-Splat-Shoot and LiDAR BEV features from PointPillars can be concatenated or cross-attended in the same space.

5.6.5 Transformer-Based Approaches

Recent architectures (SST, VoxelTransformer, SWFormer) replace sparse-conv backbones with attention-based ones, often with 3D windowed attention analogous to Swin. Accuracy gains are incremental on standard benchmarks but transformer backbones integrate more naturally with mid-fusion camera-LiDAR stacks.

5.7 3D Object Detection

Three-dimensional object detection is LiDAR’s flagship perception task: outputting per-object position, extent, orientation, and class in the vehicle frame. The outputs feed the tracker (Chapter 11) and the

planner (Chapter 18) directly, and the accuracy of the whole downstream stack is bounded by how well 3D detection localizes things.

5.7.1 The Output Parameterization

A 3D bounding box is seven numbers: $\mathbf{b} = (x, y, z, l, w, h, \theta)$ —center coordinates, extents, and yaw. Some parameterizations add pitch and roll (for surfaces on slopes) or a velocity pair (v_x, v_y) that the network regresses jointly. Production detectors typically output ≥ 10 attributes per box: the seven geometric plus class, confidence, and velocity.

Example 5.9 (Anchor-vs-anchor-free box regression). An anchor-based detector (SECOND) predefines a grid of anchor boxes—say, car-shaped, pedestrian-shaped, cyclist-shaped—at each BEV location and regresses offsets. An anchor-free detector (CenterPoint) predicts a heatmap over BEV cells with peaks at object centers and regresses a single box per peak. For typical automotive scenes, CenterPoint matches or exceeds SECOND while using $\sim 30\%$ fewer output channels, because no class-specific anchor geometry is needed. This is why anchor-free is the dominant paradigm in 2024+ LiDAR detection.

5.7.2 Loss Functions

The total loss combines classification (focal loss on heatmap), localization (smooth-L1 on box parameters), and orientation (often a bin-plus-residual or a direction classifier):

$$\mathcal{L} = \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} + \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{dir}} \mathcal{L}_{\text{dir}}. \quad (5.6)$$

Typical weights: $\lambda_{\text{cls}} = 1$, $\lambda_{\text{box}} = 2$, $\lambda_{\text{dir}} = 0.2$.

Example 5.10 (Localization loss scale at different distances). A predicted box at range 30 m has absolute position error 0.4 m; at range 80 m, the same relative accuracy would permit 1.1 m. Smooth-L1 applied directly to absolute errors favors near targets and ignores far ones. Range-normalized loss (r_i -dependent weighting) or IoU-aware loss (3D IoU directly) corrects this; production detectors use both, typically with a 0.5-weight toward IoU-aware terms at training time.

5.7.3 Handling Imbalance

Benchmark datasets are heavily dominated by vehicles (often 80 %+ of labels), with pedestrians, cyclists, and other classes in the long tail. Detectors trained with uniform sampling under-perform on rare classes at deployment. Common mitigations:

- **Class-balanced sampling:** oversample rare-class frames.
- **Focal loss:** down-weight easy examples, emphasize hard ones. Typical $\gamma = 2$.
- **Copy-paste augmentation:** duplicate rare-class annotated instances into other training frames with proper geometric consistency.

Example 5.11 (Copy-paste augmentation pedestrian gain). A training set has 100k vehicle annotations and 3k pedestrian annotations. Straight training gives pedestrian AP 25–35 %. Applying copy-paste augmentation to synthesize 10000 additional pedestrian instances by pasting existing labeled pedestrians into random other frames (with lighting, shadow, and LiDAR-occlusion consistency) routinely lifts pedestrian AP by 10–15 points without affecting vehicle AP. It is one of the highest-leverage tricks in 3D detection.

5.7.4 Non-Maximum Suppression in 3D

A single object produces many overlapping candidate boxes; NMS keeps the highest-scoring candidate and suppresses others with high IoU. Classical axis-aligned NMS fails on rotated BEV boxes; the standard is *rotated BEV IoU* NMS, which correctly accounts for box orientation in the BEV plane. Typical thresholds: 0.5 for vehicles, 0.3 for pedestrians (smaller boxes tolerate less overlap). Mis-tuned thresholds are a common silent source of missed or duplicated detections in the production log.

5.7.5 Evaluation Metrics

3D detection is scored by per-class average precision (AP) at various IoU thresholds. The Waymo and nuScenes benchmarks specify:

- **BEV AP**: evaluating matches on the top-down box projection, ignoring height.
- **3D AP**: full 3D IoU with height.
- **LEVEL 1 / LEVEL 2** difficulty (Waymo): L2 includes distant and heavily-occluded targets excluded from L1.
- **NDS** (nuScenes): composite of mAP plus translation, scale, orientation, velocity, and attribute errors.

A detector improving BEV-AP by 2 points while regressing on NDS is a common pattern—easy to overfit the IoU metric without improving the factors that actually matter downstream. Closed-loop evaluation (Chapter 32) is the ultimate adjudicator.

5.8 Semantic Segmentation

Per-point semantic labels (road, sidewalk, building, vegetation, car, pedestrian, cyclist) are the LiDAR analog of camera semantic segmentation, consumed by SLAM (dynamic-object rejection), HD-map building, and free-space estimation. Production architectures are typically RangeNet++-style (range-image CNN) for speed, or KPConv / Cylinder3D / MinkNet for accuracy. The SemanticKITTI and nuScenes-LidarSeg datasets are the public benchmarks.

Example 5.12 (Per-class IoU tradeoffs). On SemanticKITTI, RangeNet++ achieves mIoU 52 % at 25 ms per frame; Cylinder3D reaches mIoU 64 % at 100 ms per frame. Per-class analysis shows RangeNet++ struggles with small objects (traffic signs, poles) because the range-image

aliases them; Cylinder3D’s cylindrical coordinates preserve such features but at $4\times$ the compute. Production systems typically run range-image segmentation online and cylindrical models in offline auto-labeling.

5.9 LiDAR Odometry and Scan Matching

A cleaned point cloud can be used to estimate ego motion by registering the current scan against a previous scan or against a map. Two operations do most of the work: ICP (Iterative Closest Point) and its variants, and feature-based methods such as LOAM.

5.9.1 Iterative Closest Point

Point-to-point ICP [2] alternates between *association* (for each point in the source cloud, find the nearest point in the target) and *alignment* (find the rigid transform that minimizes the sum of squared distances between associated pairs). The alignment step solves a closed-form SVD problem; iteration typically converges in 10–30 steps. The point-to-plane variant, which minimizes the distance from each source point to the tangent plane of its associated target point, converges faster and more reliably on planar-dominant scenes:

$$\mathbf{T}^* = \arg \min_{\mathbf{T}} \sum_i [(\mathbf{T}\mathbf{p}_i - \mathbf{q}_i) \cdot \mathbf{n}_i]^2, \quad (5.7)$$

with $\mathbf{n}_i$ the surface normal at target point $\mathbf{q}_i$.

Example 5.13 (ICP step on a small set of correspondences). Suppose 4 point-plane pairs with residuals $r_i = (\mathbf{p}_i - \mathbf{q}_i) \cdot \mathbf{n}_i = (-0.12, 0.08, 0.11, -0.09)$ m. Sum of squared residuals = 0.046. The ICP step linearizes the rigid transform around identity and solves a 6×6 normal- equations system; one iteration typically reduces the residual sum by 80–95 % for well-conditioned scenes. After 10 iterations, residuals drop to centimeter levels. Outlier pairs (from moving objects or wrong associations) are downweighted via Huber-robust variants.

5.9.2 Feature-Based Methods

LOAM [76] and its derivatives (A-LOAM, LeGO-LOAM, LIO-SAM) extract geometric features—edge points (high curvature) and planar points (low curvature)—from each scan and align them against the map rather than doing full dense ICP. The feature extraction reduces the matching problem by an order of magnitude and makes the approach real-time at the LiDAR’s native rate. Chapter 13 develops these algorithms fully.

5.10 Weather and Adverse Conditions

LiDAR returns attenuate in rain, fog, and snow. The physics is Mie scattering: droplets or particles on the order of the laser wavelength scatter light, reducing the return amplitude at the receiver.

$$I(r) = I_0 \exp(-\alpha r), \quad (5.8)$$

with α the attenuation coefficient (m^{-1}) and r the path length. For light rain $\alpha \sim 0.01 \text{ m}^{-1}$; for heavy fog $\alpha \sim 0.1 \text{ m}^{-1}$.

Example 5.14 (Attenuation in light vs heavy fog). Light rain ($\alpha = 0.01$): at $r = 100 \text{ m}$, signal is $e^{-1} \approx 37\%$ of clear-weather intensity—still usable. Heavy fog ($\alpha = 0.1$): at $r = 50 \text{ m}$, signal is $e^{-5} \approx 0.7\%$ —below the SPAD detection floor. This is why automotive LiDAR range drops precipitously in heavy precipitation and is widely regarded as the Achilles’ heel of LiDAR-first AV programs. Radar’s longer wavelength is Mie-scatter-exempt at 77 GHz, motivating the radar/LiDAR pairing.

Mitigation strategies include multi-return processing (report the last strongest return rather than the first return, which may be a droplet), per-point weather classification (learned filters that flag suspected precipitation returns), and fusion with weather-robust radar.

5.11 Simulation and Sim-to-Real

LiDAR simulation is simpler than camera simulation because the physics is more geometric and less texture-dependent. Ray-casting engines (CARLA, LGSVL, NVIDIA DRIVE Sim) produce synthetic point clouds with essentially perfect ground truth. The remaining challenges are in matching the noise model of real sensors, incorporating beam divergence, handling translucent and reflective materials realistically, and simulating weather. Sim-to-real transfer is correspondingly better for LiDAR than for camera, with sim-trained models often achieving 80–95 % of their real-data performance without fine-tuning.

5.12 Datasets and Benchmarks

Public LiDAR datasets have been the backbone of AV research. A short survey:

- **KITTI**: the first large-scale driving dataset; Velodyne HDL-64E data with 3D bounding-box annotations.
- **Waymo Open Dataset**: five LiDARs synced with five cameras; 3D detection, tracking, motion, and domain-shift challenges.
- **nuScenes**: Velodyne HDL-32 plus radars and cameras; 3D detection, tracking, and prediction benchmarks.
- **SemanticKITTI**: KITTI point clouds with per-point semantic labels; the standard benchmark for LiDAR semantic segmentation.
- **ONCE**: large-scale LiDAR dataset with emphasis on self-supervised pretraining benchmarks.
- **Waymo Motion Open Dataset**: scenario-level data including LiDAR for motion forecasting.

5.13 Worked Examples

Example 5.15 (End-to-end: acquire, undistort, voxelize, detect). A LiDAR frame has $N = 120\,000$ points acquired over 100 ms at ego speed 25 m s^{-1} .

Step 1: undistort. Per-point timestamps span $[-50, +50]$ ms relative to the frame center. Ego displacement over 50 ms: 1.25 m. Apply Eq. 5.4 with IMU-integrated $\mathbf{T}_{t_{\text{ref}}, t_i}$ per point. A static wall at 30 m ahead, initially showing a 2.5 m-long arc, collapses to points with 5 cm spread.

Step 2: voxelize. Grid $200 \times 200 \times 8$ at $\Delta = 0.2\text{ m}$ over a $40 \times 40 \times 1.6\text{ m}$ slab. Assign each point to its voxel; typical occupancy 3 % (9600 occupied voxels). A PointPillars-style pillar encoder processes each occupied (x, y) column.

Step 3: detect. Backbone runs 2D conv over 200×200 BEV feature map; head produces 3D bounding-box anchors at each cell. After NMS, typical output: 15–60 boxes per frame at recall $> 90\%$ on mid-size vehicles out to 60 m.

Total pipeline budget: $\sim 40\text{ ms}$ on an automotive accelerator. The dominant cost is the 2D backbone, typically 20–25 ms; voxelization and pillar encoding are each under 5 ms.

Example 5.16 (Intensity-based lane-line detection). Raw intensity image from range-image representation reveals lane-line markings as consistently high-intensity pixels at grazing angles. Calibrating by Eq. 5.3 removes the r^2 falloff; thresholding at 80th percentile gives a binary mask. Morphological closing along ring dimension joins segments. RANSAC line fit on the ring with maximum white-pixel density recovers the line geometry. The pipeline runs in 1 ms and works at night, in dense fog (where cameras fail), and with arbitrary lane colors.

Example 5.17 (ICP-based pose tracking). Two consecutive scans with 30 % overlap. Initial guess: zero motion. After 12 ICP point-to-plane iterations, converged residual sum drops from 4.2 m^2 to 0.003 m^2 . Recovered transform: $\mathbf{t} = (2.45, -0.08, 0.02)\text{ m}$, $\mathbf{R} = \text{yaw } 0.3^\circ$. Over 100 ms, ego moved 2.45 m forward at 24.5 m s^{-1} with a slight right drift. This is one primitive of the LiDAR odometry pipelines developed fully in Chapter 13.

5.14 Industrial Deployment

The LiDAR market has seen dramatic consolidation and cost reduction since 2018. Hesai, headquartered in Shanghai and listed on NASDAQ, became the world’s largest automotive LiDAR supplier by shipment volume by 2023, supplying multiple Chinese OEMs including Li Auto, NIO, and others. Innoviz Technologies (Israel) supplies LiDAR to BMW; Luminar Technologies (U.S.) has supplied LiDAR to Volvo’s EX90 and Polestar 3. Velodyne (now merged with Ouster as Ouster, Inc.) pioneered the spinning mechanical LiDAR architecture used in early Waymo and Cruise vehicles. Solid-state and MEMS-based LiDARs have moved into production around 2022–2024, with Hesai’s AT128 and ATX, Innoviz One, and RoboSense’s M-series among production-deployed examples. Cost has declined from tens of thousands of dollars per unit (early Velodyne HDL-64 around 2007–2015) to high hundreds of dollars per unit for some recent automotive solid-state designs.

5.15 Chapter Summary

LiDAR is the AV's geometric sense. Hardware diversity—mechanical, MEMS, solid-state, flash—is substantial, but the resulting point clouds can be processed through a common set of representations (raw, range image, voxel, BEV pillar) each with characteristic strengths. Modern LiDAR perception is dominated by learned architectures in one of four families: PointNet on raw points, CNNs on range images, sparse-conv networks on voxels, and PointPillars-style 2D CNNs on BEV. Semantic segmentation, detection, and odometry all use variants of these. Motion undistortion, weather-aware processing, and calibrated intensity are the standard pre-processing steps. The radar chapter that follows complements LiDAR with velocity-native, weather-robust measurement that fills the failure modes this chapter has identified.

5.16 Exercises

1. A LiDAR has $\sigma_\tau = 150\text{ps}$ timing precision. Compute σ_r and plot the range error as a function of SNR, assuming linear degradation with the square root of SNR.
2. A 128-beam LiDAR has 40° vertical FoV and 0.1° horizontal resolution at 10 Hz. Compute total points per frame, and the ground-spot spacing between neighboring vertical beams at 60 m range on a flat road.
3. For the voxel-grid sparsity example (5.6), compute the sparse-conv memory savings at $\Delta = 0.1, 0.2, 0.4\text{ m}$. Discuss the tradeoff with detection recall for small objects.
4. Derive Eq. 5.3 from the LiDAR range equation (incident power, target reflectivity, geometric spread). State your assumptions.
5. A mechanical LiDAR at 20 Hz sweeping 360° captures a scene with ego speed 20 ms^{-1} . Without undistortion, how long does a stationary pole 10 m away appear?
6. A PointNet++ model operates on 4096 points per sample. How many per-point MLP evaluations are required in one forward pass through its four set-abstraction levels if the grouping hops use 256, 64, 16, 4 points respectively?
7. At what attenuation coefficient α does a LiDAR operating at 200 m range drop to the 1 % return-signal floor?
8. For the PointPillars pipeline in Fig. 5.3, sketch the equivalent 3D voxel architecture and compute the relative memory and compute cost at a $40 \times 40 \times 1.6\text{ m}$ volume with $\Delta = 0.2\text{ m}$.
9. A scene has 60 % ground, 25 % vehicles, 10 % vegetation, 5 % pedestrians by point count. What is the expected per-class IoU trend for a naive uniform-sample training vs a class-balanced training, and why?
10. Propose a LiDAR-specific data augmentation that simulates rain (first-return droplet reflections)

from clear-weather data, and explain how it would be validated.

Chapter 6

Radar and Sonar Perception

6.1 Introduction

Cameras and LiDAR get the attention, but radar is what keeps production autonomous vehicles driving when the weather turns. A camera's image washes out in direct sun, degrades in rain, and fails entirely at night without headlights. A LiDAR's laser returns attenuate in fog and scatter in heavy snow. Automotive radar, operating at 24 GHz or 77 GHz, penetrates all three with little loss of range. It measures Doppler velocity directly from phase—no differencing of positions across frames, no cascading tracker latency—and does so out to 200 m to 300 m at a hardware cost of tens of dollars per unit. Every serious AV program carries at least a front-facing radar and several corner radars; a growing number carry imaging radars with LiDAR-like point-cloud density. The chapter introduces the physics, signal processing, and deep-learning techniques that turn raw radar returns into the object tracks and occupancy maps the planner consumes.

Sonar plays a quieter but essential role: ultrasonic sensors in the bumpers handle close-in parking and low-speed maneuvering, the one regime in which radar is poorly suited (bumper clutter, short-range ambiguity) and cameras struggle (tight geometry, poor depth). The same physical principle that governs sonar—time-of-flight of a pressure wave—also governs automotive ultrasonics, and they share a small body of signal-processing technique worth collecting.

What makes this chapter different from the radar material covered alongside other sensors in Chapter 10 is depth. That later chapter treats FMCW range and Doppler at the level needed for sensor-fusion arithmetic. Here we develop waveform design, ambiguity analysis, angle-of-arrival estimation via multiple antennas, MIMO virtual-array construction, radar-specific target tracking, micro-Doppler for target classification, imaging-radar point clouds, and the deep-learning architectures that operate directly on radar tensors. The point is to treat radar as a first-class perception modality rather than as a secondary cue to LiDAR and camera.

Three structural properties of radar shape everything that follows. *First*, radar measurements are ambiguous: range and velocity fold around a maximum unambiguous value determined by the

waveform. *Second*, radar angular resolution is poor relative to the other sensors unless explicit multi-antenna architectures are used; even a high-end imaging radar is coarser than LiDAR. *Third*, radar directly measures Doppler velocity, which for moving targets is often a sharper discriminator than geometric position. Algorithms that ignore these properties produce poor radar perception; algorithms that exploit them (velocity-gated association, micro-Doppler classification, radar-specific data association) often outperform camera-only methods at long range or in bad weather. We develop each of these properties with worked examples below.

The chapter is organized as follows. We first revisit FMCW fundamentals in greater depth and derive range-Doppler processing. We then develop angle estimation via multi-antenna architectures and MIMO virtual arrays. Target detection (CFAR and its variants), clustering, and tracking are next. We devote a section to *micro-Doppler*, the modulation of a target's radar signature by its internal motion (pedestrian limb swing, cyclist wheel rotation), because it enables classification that radar could not otherwise do. Imaging ("4D") radar follows. We then cover automotive ultrasonics and their use in close-in maneuvers. The chapter closes with radar-camera and radar-LiDAR fusion patterns, the deep-learning architectures that process radar as a first-class modality, and a survey of available datasets. Worked examples appear immediately after every formula.

6.2 FMCW Radar Fundamentals

Frequency-modulated continuous-wave radar is the dominant automotive waveform. The transmitter emits a linear chirp—a sinusoid whose frequency sweeps linearly from f_c to $f_c + B$ over duration T_c —and the receiver mixes the reflected signal with the transmitted signal to produce a *beat* tone whose frequency encodes range.

6.2.1 The Chirp Waveform

The transmitted chirp is

$$s_{\text{tx}}(t) = A \cos(2\pi f_c t + \pi S t^2), \quad 0 \leq t \leq T_c, \quad (6.1)$$

where $S = B/T_c$ is the chirp slope (Hz/s). The instantaneous frequency $f_c + St$ sweeps linearly from f_c at $t = 0$ to $f_c + B$ at $t = T_c$.

A reflection from a target at range r returns after delay $\tau = 2r/c$, so the received signal is a time-shifted and attenuated copy:

$$s_{\text{rx}}(t) = A' \cos(2\pi f_c(t - \tau) + \pi S(t - \tau)^2). \quad (6.2)$$

Mixing transmitted and received signals produces a beat signal whose leading-order frequency component is

$$f_b = S\tau = \frac{2Sr}{c}. \quad (6.3)$$

Solving for range:

$$r = \frac{cf_b}{2S}. \quad (6.4)$$

Example 6.1 (Beat frequency from a target at 50 m). $B = 1$ GHz, $T_c = 50 \mu\text{s}$, so $S = 2 \times 10^{13} \text{ Hz s}^{-1}$. Target at $r = 50$ m gives $\tau = 2 \cdot 50 / (3 \times 10^8) = 333 \text{ ns}$, and $f_b = S\tau = 6.67 \text{ MHz}$. A typical ADC samples at 20 MS/s (Nyquist rate for the max f_b of interest), so 6.67 MHz is well within the sampling band. An FFT over the chirp produces a peak in the bin corresponding to this frequency—the target’s range.

6.2.2 Range Resolution and Maximum Range

The range resolution of an FMCW radar is set by the bandwidth:

$$\Delta r = \frac{c}{2B}. \quad (6.5)$$

The maximum unambiguous range is set by the sampling rate f_s of the ADC (assuming we want to keep all beat frequencies within the first Nyquist zone):

$$r_{\max} = \frac{cf_s}{2S}. \quad (6.6)$$

Example 6.2 (Range resolution and max range). With $B = 4$ GHz: $\Delta r = 3 \times 10^8 / (2 \cdot 4 \times 10^9) = 3.75 \text{ cm}$ —comparable to high-end LiDAR. At $B = 0.5$ GHz: $\Delta r = 30 \text{ cm}$, sufficient for distinguishing two vehicles a lane width apart but too coarse for per-object pose.

With $S = 2 \times 10^{13} \text{ Hz s}^{-1}$ and $f_s = 20 \text{ MHz}$, $r_{\max} = 3 \times 10^8 \cdot 2 \times 10^7 / (2 \cdot 2 \times 10^{13}) = 150 \text{ m}$. Production long-range radars push $r_{\max}$ beyond 250 m by lowering S (slower chirps) at the cost of velocity ambiguity, discussed below.

6.2.3 Velocity and Doppler

A single chirp measures range. Velocity comes from the phase change between successive chirps: if the target moves by Δr between chirp n and chirp $n + 1$, the beat-signal phase advances by $\Delta\phi = 4\pi\Delta r/\lambda$, where $\lambda = c/f_c$ is the carrier wavelength. Accumulating N_c chirps into a frame of duration $T_f = N_c T_c$, an FFT over the chirp index (*slow-time*) resolves the Doppler frequency f_d :

$$v = \frac{\lambda f_d}{2}. \quad (6.7)$$

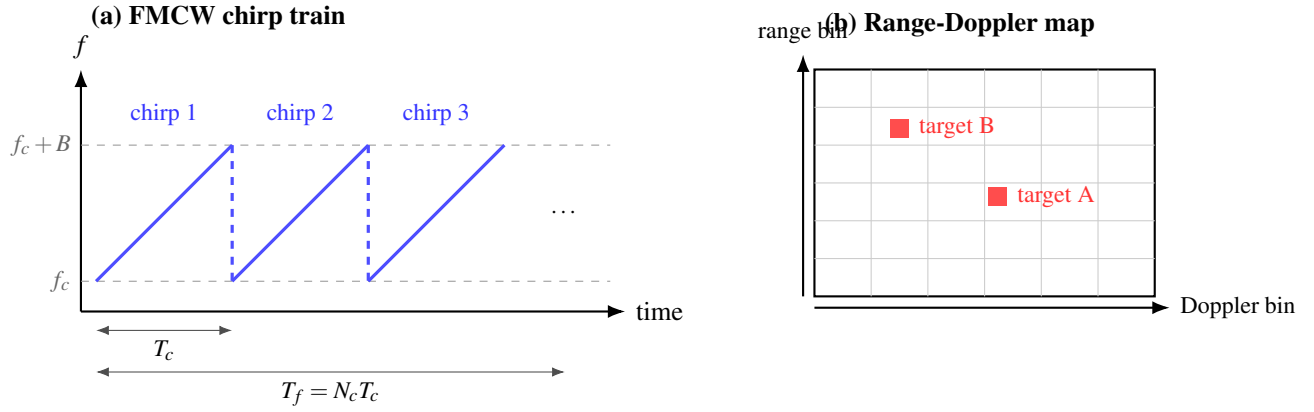


Figure 6.1: (a) An FMCW chirp train: a burst of N_c linear chirps of duration T_c spans the full frame $T_f = N_c T_c$. The fast-time (within-chirp) FFT of the beat signal resolves range (Eq. 6.4); the slow-time (across-chirp) FFT resolves Doppler frequency (Eq. 6.7). (b) Together they form the range-Doppler map, a 2D intensity image in which each target appears as a peak at its (r, v) location. Multiple simultaneously tracked targets separate cleanly when they differ in range, velocity, or both.

Example 6.3 (Doppler resolution and max velocity). $f_c = 77$ GHz, $\lambda = 3.9$ mm. Frame $N_c = 128$ chirps at $T_c = 50 \mu\text{s}$ gives $T_f = 6.4$ ms. Doppler resolution $\Delta f_d = 1/T_f = 156.25$ Hz translates to velocity resolution $\Delta v = \lambda \Delta f_d / 2 = 0.305 \text{ m s}^{-1}$. Max unambiguous Doppler $\pm 1/(2T_c) = \pm 10 \text{ kHz}$ gives max unambiguous radial velocity $\pm \lambda / (4T_c) \approx \pm 19.5 \text{ m s}^{-1}$. Two cars closing head-on at 30 m s^{-1} each (relative velocity 60 m s^{-1}) would alias; production radars either increase chirp rate or use waveform-diversity tricks to disambiguate.

6.2.4 Range-Velocity Coupling

The inseparability of range and Doppler in a single chirp produces *range-Doppler coupling*: a target moving during the chirp imprints an extra phase that biases the estimated range by $\Delta r = v T_c / 2$. For a closing target at 30 m s^{-1} with $T_c = 50 \mu\text{s}$, this is 0.75 mm —negligible. For a Doppler chirp waveform at $T_c = 1 \text{ ms}$, the bias is 1.5 cm , borderline. Long-chirp waveforms trade Doppler resolution for range-Doppler coupling and must correct it.

6.3 Angle Estimation and MIMO Arrays

A single antenna has no angular resolution: it receives equally from everywhere in its beam pattern. Angular resolution comes from multiple antennas, via differences in received phase across the array.

6.3.1 Phased-Array Basics

A linear array of N antennas spaced d apart receives a plane wave from angle θ with a progressive phase shift between elements of

$$\Delta\phi = \frac{2\pi d \sin \theta}{\lambda}. \quad (6.8)$$

Beamforming weights $w_n = e^{-jn\Delta\phi_0}$ steer the array to point in direction θ_0 (where $\Delta\phi_0 = 2\pi d \sin \theta_0 / \lambda$); the combined output

$$y(\theta_0) = \sum_{n=0}^{N-1} w_n^* x_n \quad (6.9)$$

constructively sums plane waves arriving from θ_0 and attenuates others.

Example 6.4 (Beamwidth of a 12-element array). $N = 12$, $d = \lambda/2 = 1.95$ mm at 77 GHz. Array aperture $L = (N - 1)d = 11 \cdot 1.95 = 21.5$ mm. Rayleigh angular resolution $\theta_{\text{res}} \approx \lambda/L = 3.9/21.5 = 0.181$ rad $\approx 10.4^\circ$. A 12-element array therefore separates two targets in azimuth that differ by at least 10° —one lane width away at 20 m range. Finer resolution requires either a larger aperture (longer antenna) or more elements (via MIMO, below).

6.3.2 MIMO Virtual Arrays

Adding physical receive antennas is expensive and takes bumper space. The MIMO trick uses N_T transmit antennas firing orthogonal waveforms (time-division, code-division, or frequency-division multiplexed) together with N_R receive antennas, producing a *virtual* array of $N_T N_R$ effective elements. Each transmit-receive pair contributes a distinct spatial signature, and the joint system resolves angles as if it were a single physical array of $N_T N_R$ elements.

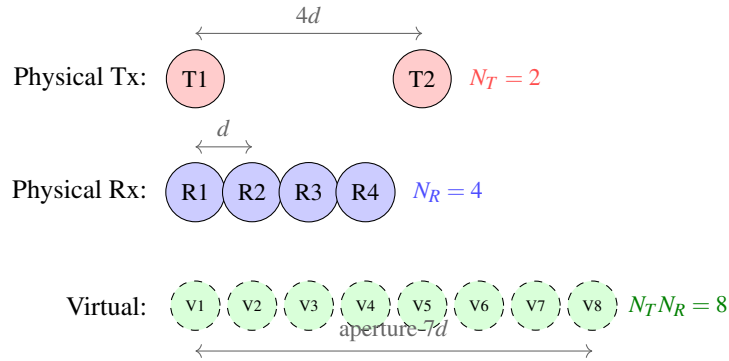


Figure 6.2: MIMO virtual-array construction. Two transmit antennas spaced $4d$ apart (top, red) and four receive antennas spaced d (middle, blue) combine to form an 8-element virtual array (bottom, green, dashed) with aperture $7d$. Angular resolution improves proportionally to aperture: with $d = \lambda/2$, this virtual array achieves $\sim 15^\circ$ resolution compared to $\sim 25^\circ$ for the physical 4-element Rx alone. Production 4D imaging radars scale this to $N_T = 12, N_R = 16$ with clever antenna placement, producing 192-element virtual arrays at 1° resolution.

Example 6.5 (Resolution gain from MIMO). Physical $N_R = 4$ receive antennas spaced $\lambda/2$: aperture $3\lambda/2 = 5.85$ mm; Rayleigh resolution $\lambda/\text{aperture} \approx 38^\circ$ —useless. Adding $N_T = 2$ transmit antennas spaced $4d = 2\lambda$ apart produces 8 virtual elements with aperture $7\lambda/2 = 13.7$ mm; resolution improves to $\sim 16^\circ$. A MIMO radar with $N_T = 3, N_R = 4$ gets 12 virtual elements and $\sim 11^\circ$. The resolution scales linearly in the total virtual element count for fixed carrier—which is why imaging radars aggressively grow both N_T and N_R .

6.3.3 Super-resolution Methods

Classical beamforming is Rayleigh-limited. Super-resolution algorithms (MUSIC, ESPRIT, compressed sensing, deep unfolding) exploit sparsity—the assumption that only a few targets exist in each range-Doppler cell—to achieve sub-Rayleigh angular accuracy. MUSIC, for instance, forms a spatial covariance matrix from array snapshots, separates it into signal and noise subspaces, and searches for directions whose steering vectors lie in the signal subspace:

$$P_{\text{MUSIC}}(\theta) = \frac{1}{\mathbf{a}(\theta)^H \mathbf{U}_N \mathbf{U}_N^H \mathbf{a}(\theta)}, \quad (6.10)$$

with $\mathbf{a}(\theta)$ the steering vector and $\mathbf{U}_N$ the noise-subspace basis. Peaks of P_{MUSIC} correspond to target angles.

Example 6.6 (MUSIC angular gain). A 12-element virtual array with 15° Rayleigh resolution observes two targets at $\theta = -3^\circ$ and $\theta = 3^\circ$ (separation 6°). Classical beamforming produces a single blurred peak—unresolvable. MUSIC, at SNR 20 dB with 64 snapshots, routinely separates targets at fractions of a degree when the subspace dimension matches the target count. The catch: MUSIC assumes the correct target count and is sensitive to model mismatch (closely-spaced correlated targets collapse the covariance matrix). Production imaging radars combine classical DBF (robust but coarse) with super-resolution refinement (sharp but brittle) to balance both.

6.4 Target Detection

A range-Doppler map is a 2D intensity image; turning its peaks into target detections requires thresholding against the local noise level. Because clutter and noise vary with range and with the scene, a fixed threshold fails.

6.4.1 Constant False Alarm Rate (CFAR)

CFAR estimates the local noise power from cells near the test cell (excluding *guard* cells that might leak target energy) and sets a threshold that yields a constant false-alarm probability. For cell-averaging (CA-)CFAR:

$$T = \alpha \bar{P}_R, \quad \alpha = N_R \left(P_{fa}^{-1/N_R} - 1 \right), \quad (6.11)$$

where N_R is the total reference-cell count and $\bar{P}_R$ their average power.

Example 6.7 (CFAR threshold for $P_{fa} = 10^{-5}$). $N_R = 16$ reference cells, $P_{fa} = 10^{-5}$. $P_{fa}^{-1/N_R} = (10^{-5})^{-1/16} = 10^{5/16} \approx 2.05$. $\alpha = 16(2.05 - 1) = 16.8$. In dB: threshold is 12.2 dB above mean reference power. Tightening P_{fa} to 10^{-7} raises α to 28 (14.5 dB). The tradeoff is classical: a stricter false-alarm rate makes the detector miss fainter targets. Automotive radar tunes P_{fa} against a per-class recall target on a labeled test set.

Variants—OS-CFAR (order-statistic), GO/SO-CFAR (greatest/smallest of), and learned CFAR—

handle clutter edges, multi-target scenarios, and correlated noise better than CA-CFAR at incremental computational cost.

6.4.2 Peak Extraction and Clustering

CFAR detections typically come in clusters (one target generates several adjacent bin detections). DBSCAN [21] clusters detections in (r, v, θ) space:

- each detection is a point with range, velocity, and angle coordinates;
- points within distance ε of a core point (a point with at least $N_{\min}$ neighbors) join the same cluster;
- each final cluster is summarized by its centroid plus velocity and size.

Example 6.8 (DBSCAN cluster from 20 detections). CFAR returns 20 raw detections near a vehicle. In a normalized (r, v, θ) space with $\varepsilon = 1$ and $N_{\min} = 3$, 18 detections cluster into a single core with 15-point density, and 2 isolated detections are flagged as noise. The cluster centroid is $(r, v, \theta) = (42.1 \text{ m}, -8.3 \text{ m/s}, 3.2^\circ)$; spatial extent spans $\sim 1.8 \text{ m}$ in range—consistent with a vehicle length. Downstream trackers consume these cluster centroids rather than raw detections.

6.5 Radar-Specific Tracking

Multi-target tracking (Chapter 11) is modality-generic, but radar’s high-rate Doppler and low angular resolution call for specializations.

6.5.1 Doppler-Gated Association

Standard bounding-box association uses position gating alone. Radar trackers also gate on relative velocity, rejecting candidate detection-track pairs whose Doppler shifts are inconsistent with the track’s state. Since Doppler is measured per-detection to centimeters per second, this is a much tighter gate than position and rejects many spurious associations that position-only trackers accept.

Example 6.9 (Velocity gate rejection). A track predicts $(r, v) = (50, -8) \text{ m, m/s}$ with $(\sigma_r, \sigma_v) = (0.5, 0.3)$. A detection arrives at $(50.1, 2)$: position residual is in-gate ($d_M^2 = 0.04/0.25 = 0.16$), but the velocity residual $(2 - (-8))^2/0.3^2 = 1111$ is far outside the χ^2 gate. The detection is rejected; without velocity gating, a position-only tracker would have accepted it and corrupted the state. This is why radar tracks converge faster than camera tracks on benchmark multi-object tracking metrics.

6.5.2 Angle Uncertainty Handling

Radar angular uncertainty is typically much larger than position uncertainty derived from range alone. Most radar trackers represent per-detection covariance with a dominant cross-range component: $\Sigma_{\text{rad}} = \text{diag}(\sigma_r^2, \sigma_\theta^2 r^2)$ (range $\sigma_r = \text{cm}$, cross-range $\sigma_\theta r = 0.5 \text{ m}$ to 3 m depending on range and angular resolution). The elongated error ellipse means the tracker must represent uncertainty anisotropically or it will falsely reject matches along the bearing direction.

6.5.3 Micro-Doppler for Classification

The Doppler spectrum of a target is not a single line: internal moving parts (a pedestrian’s swinging limbs, a cyclist’s rotating wheel spokes, a truck’s rotating tires) produce side lobes modulated at the frequency of their motion. The collection of these sidebands, as a function of time, is the *micro-Doppler signature*. A short-time Fourier transform of the slow-time signal produces a 2D spectrogram—time vs Doppler—whose structure encodes the target’s gait or mechanical dynamics.

Example 6.10 (Pedestrian gait vs vehicle: micro-Doppler contrast). A pedestrian walking at $v_{\text{body}} = 1.5 \text{ ms}^{-1}$ produces a main Doppler line at $2v_{\text{body}}/\lambda = 770 \text{ Hz}$ (at 77 GHz). Leg swings at 1.0 Hz cadence peak at $v_{\text{foot}} \sim 3.5 \text{ ms}^{-1}$, contributing Doppler sidebands at $\sim 1800 \text{ Hz}$. The spectrogram shows oscillating side lobes synchronized with the gait period. A vehicle at 1.5 ms^{-1} with no internal periodic motion shows a clean main line and no oscillation. A CNN classifier trained on these 2D spectrogram images distinguishes pedestrian from vehicle from cyclist at radar-appropriate ranges (out to $\sim 80 \text{ m}$) with accuracies of 90+ %.

Micro-Doppler classification is one of the few perception tasks radar does *better* than camera or LiDAR. It does not require lighting, weather, or a clean line of sight, and it exploits a physical cue invisible to geometric sensors.

6.6 4D Imaging Radar

Until the late 2010s automotive radar was essentially a 2D sensor: range and velocity, with coarse azimuth. Imaging radars (also called “4D” radars, counting range, Doppler, azimuth, and elevation) push angular resolution to 1° to 2° and add elevation, producing point clouds with LiDAR-like density at 100 m to 300 m. Representative products: Arbe, Uhnder, Oculii, Continental ARS540.

The enabling architectural advances are large MIMO virtual arrays (100+ elements), digital beamforming in both azimuth and elevation, and dense-chirp waveforms that preserve Doppler resolution across the expanded beam count. A single imaging-radar frame may produce 10 000–100 000 detections, comparable to a mid-density LiDAR.

Example 6.11 (Imaging-radar point density). A 4D radar with 1.5° azimuth and 2° elevation resolution over a $\pm 60^\circ \times \pm 15^\circ$ FOV: $80 \cdot 15 = 1200$ angular cells per range bin. With 256 range bins and 128 Doppler bins producing dense data on roughly 10 % of cells (scene-dependent): $1200 \cdot 256 \cdot 0.1 \approx 30\,000$ detections per frame at 20 Hz. Compare LiDAR: a 64-beam mechanical sensor gives $\sim 130\,000$ points per frame at 10 Hz. Imaging radar approaches LiDAR in raw count while preserving Doppler, at a fraction of the cost and with weather robustness.

Imaging radar changes the algorithmic landscape: instead of treating radar as a sparse velocity sensor consumed by camera-driven perception, it becomes plausible to run detection and segmentation on radar point clouds directly, using architectures borrowed from LiDAR (PointNet++, VoxelNet; Chapter 11).

6.7 Automotive Ultrasonic Sensors

Ultrasonic sensors—automotive sonar—operate at 40 kHz or 48 kHz and range via pulse time-of-flight:

$$r = \frac{c_{\text{air}} \tau}{2}, \quad (6.12)$$

with $c_{\text{air}} \approx 343 \text{ ms}^{-1}$ and τ the round-trip time. Because the sound speed depends on temperature and humidity, production systems calibrate c_{air} at startup.

Example 6.12 (Ultrasonic time-of-flight at 1 m). At $r = 1 \text{ m}$, $\tau = 2 \cdot 1/343 = 5.83 \text{ ms}$. At the speed of sound, a 1-cm range error corresponds to $\Delta\tau = 2 \cdot 0.01/343 = 58 \mu\text{s}$ —comfortably resolvable by cheap timing electronics. Temperature change of 10 K shifts c_{air} by about 1.5 %, producing 1.5 cm range error at 1 m if uncorrected. Production systems calibrate against a known reference, typically a slightly pitched mount that reflects off a flat surface at a known distance.

Eight to twelve ultrasonic sensors are typically distributed across the front and rear bumpers. Because they have wide beamwidths (60° to 90°), angular resolution comes from sensor spacing and timing differences, not within-sensor beamforming. Ultrasonics serve three functions:

- **Parking assistance.** Close-range obstacle detection (0–5 m) for autoparking and self-parking features.
- **Curb detection.** Height profile of the curb near the vehicle, informing tight maneuvers.
- **Redundant short-range safety.** A cross-check against camera and radar in the near field, where both suffer blind spots.

Ultrasonic sensing is inherently short-range: atmospheric attenuation at 40 kHz limits practical range to roughly 5 m. They are the sensor of the last meter.

6.8 Radar and Ultrasonic Fusion Architectures

Radar plugs into the fusion architectures covered in Chapter 10; this section deepens the treatment of radar-specific fusion patterns.

6.8.1 Radar-Camera Fusion

Three fusion strategies dominate (Figure 6.3).

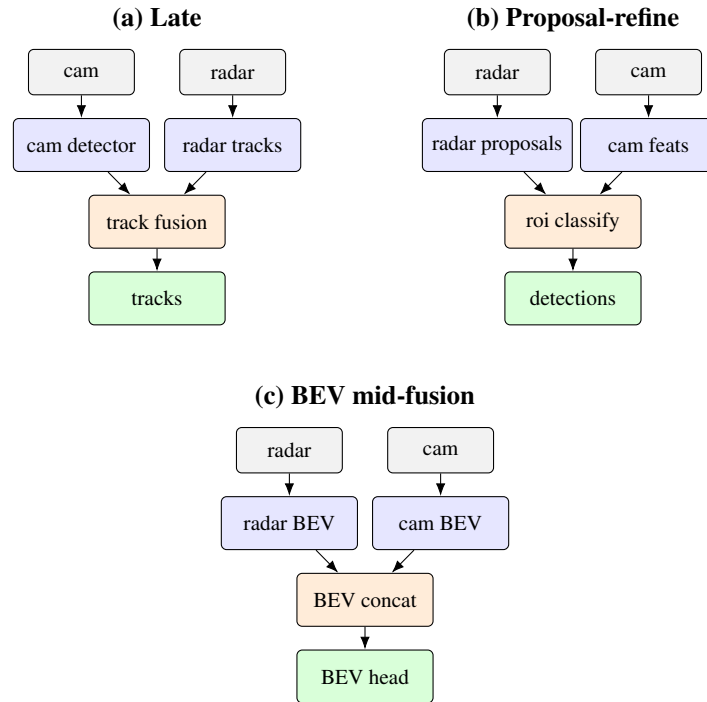


Figure 6.3: Three common radar-camera fusion architectures. (a) *Late* fusion runs independent radar and camera detectors and fuses at the track level. (b) *Proposal-refine* uses radar detections as region proposals that a camera classifier refines with fine-grained class labels—a natural complement between radar’s localization and camera’s semantics. (c) *BEV mid-fusion* projects both modalities into a common BEV tensor and processes them jointly, the pattern used by most 2023+ detection leaderboards.

Example 6.13 (Proposal-refine gain on a distant vehicle). A radar detects a vehicle at $r = 120\text{m}$ with 2 m cross-range uncertainty. In the camera image, this region projects to roughly 40×80 pixels. The camera’s detector alone produces 0.4-confidence per-frame (far, small) and drops out half the frames. Using the radar detection as a proposal, a camera classifier scoring only the projected ROI produces 0.85-confidence and holds across frames. The combined track is firm; either modality alone was unreliable. This pattern (radar as a proposer, camera as a classifier) is robust and interpretable, which is why it persists in production despite end-to-end mid-fusion results on benchmarks.

6.8.2 Radar-LiDAR Fusion

Radar-LiDAR fusion is less common because LiDAR already provides strong geometric localization; the main value of adding radar is Doppler velocity and weather robustness. When present, fusion occurs at the tracker level: LiDAR contributes 3D extent and geometric certainty, radar contributes radial velocity.

6.9 Deep Learning on Radar

Classical signal processing—FFT, CFAR, beamforming, DBSCAN—still dominates the raw-to-detection pipeline. But from detection onward, deep-learning methods have progressively replaced

hand-crafted heuristics.

6.9.1 Deep Learning on the Range-Doppler Map

A range-Doppler map is a 2D image; any CNN architecture applies. Networks in this space perform CFAR-free detection, noise classification, and clutter suppression. The gain is typically 1–3 dB of effective SNR, which translates to detection-range extensions of 5 % to 15 %.

6.9.2 Deep Learning on Range-Doppler-Angle Tensors

With MIMO arrays, the full tensor is $N_{\text{range}} \times N_{\text{Doppler}} \times N_{\text{angle}}$, typically a few million elements. 3D CNNs or point-based networks process these tensors for end-to-end detection and classification. RadarNet, TransRad, RODNet, and RAMP-CNN are representative architectures.

6.9.3 Deep Learning for Imaging-Radar Point Clouds

Because imaging radar produces LiDAR-like point densities, architectures designed for LiDAR (PointNet++, VoxelNet, CenterPoint) apply with minor modifications. The essential difference is that radar point clouds carry Doppler as an additional per-point channel, which radar-adapted architectures exploit explicitly as a feature rather than treating it as auxiliary.

6.9.4 Micro-Doppler Classifiers

A 2D spectrogram (time vs Doppler) per target is fed to a CNN or a transformer, trained to classify target type (pedestrian, cyclist, vehicle, motorcycle, stationary). Published accuracies on well-separated targets at $r \leq 80\text{m}$ exceed 90 %; performance drops sharply at the edge of the detection envelope.

6.10 Datasets and Benchmarks

Radar datasets lag camera and LiDAR datasets in scale but are growing. A non-exhaustive survey:

- **nuScenes** [9] includes 5 radars synchronized with 6 cameras and a LiDAR; the largest open multimodal dataset with radar.
- **Oxford Radar RobotCar**: rotating high-resolution radar sweeps over a year of repeat driving in Oxford.
- **RADIATE**: bad-weather radar dataset (fog, rain, snow, night) with synchronized camera and LiDAR.
- **Astyx HiRes 2019**: high-resolution 4D radar with 3D object annotations.
- **CARRADA**: range-angle-Doppler tensors with ground truth labels for learning on raw radar data.
- **K-Radar**: 4D radar dataset covering adverse weather.

6.11 Worked Examples

Example 6.14 (Full FMCW processing pipeline on a two-target scene). $f_c = 77\text{ GHz}$, $B = 2\text{ GHz}$, $T_c = 25\text{ }\mu\text{s}$, $N_c = 128$, $N_s = 256$ samples per chirp. Targets: T_1 at $r = 20\text{ m}$, $v = -10\text{ m s}^{-1}$; T_2 at $r = 40\text{ m}$, $v = 5\text{ m s}^{-1}$. $\lambda = 3.9\text{ mm}$, $S = 8 \times 10^{13}\text{ Hz/s}$.

Beat frequencies. $f_{b,1} = 2Sr_1/c = 10.67\text{ MHz}$, $f_{b,2} = 21.33\text{ MHz}$. An ADC at 50 MS/s per chirp captures both.

Range bins. 256 samples per chirp give 256 range bins of width $\Delta r = c/(2B) = 7.5\text{ cm}$. T_1 falls in bin $20/0.075 = 267$; T_2 in bin 533.

Doppler frequencies. $f_{d,1} = 2v_1/\lambda = -5.13\text{ kHz}$, $f_{d,2} = +2.56\text{ kHz}$.

Doppler bins. 128 chirps at $T_c = 25\text{ }\mu\text{s}$ give frame $T_f = 3.2\text{ ms}$ and Doppler bin width $\Delta f_d = 1/T_f = 312.5\text{ Hz}$; T_1 falls in bin -16 (centered spectrum), T_2 in bin $+8$.

After 2D FFT, the range-Doppler map has two clean peaks, well-separated in both dimensions. CFAR with $P_{fa} = 10^{-5}$ detects both. Velocity and range estimates are at sub-bin precision by peak interpolation. The entire pipeline—FFT + CFAR + peak extraction—runs in 1 ms to 3 ms on a modern automotive radar MCU.

Example 6.15 (MIMO angular estimation from beat phase). A MIMO radar has 2 transmit, 4 receive antennas with the geometry of Figure 6.2, $d = \lambda/2$. Target at $\theta = 10^\circ$. The virtual array has 8 elements at spacings $(0, 1, 2, 3, 4, 5, 6, 7)d$. Phase progression $\Delta\phi = 2\pi d \sin\theta/\lambda = \pi \sin(10^\circ) = 0.546\text{ rad}$ per element. After range-Doppler processing, the 8-element complex-valued vector at the target's (range, Doppler) bin is $e^{jn \cdot 0.546}$, $n = 0, \dots, 7$. A 8-point angular FFT produces a peak at bin index $\Delta\phi/(2\pi/N) = 0.546 \cdot 8/(2\pi) \approx 0.695$, i.e. between bins 0 and 1 (linear interpolation yields the exact angle). Replacing the FFT with MUSIC increases angular accuracy to fractions of a degree at good SNR.

Example 6.16 (Close-in ultrasonic gate closure). A self-parking system monitors an ultrasonic sensor firing at 20 Hz . Current reading: $r = 0.80\text{ m}$; ego speed 0.3 m s^{-1} . At this rate, r shrinks by 1.5 cm per tick. Threshold for emergency stop: 0.30 m . Frames remaining: $(0.80 - 0.30)/0.015 = 33\text{ frames} \approx 1.65\text{ seconds}$. Control computes braking profile accordingly. The ultrasonic ring is the redundant last line of defense: camera and radar have already flagged obstacle at this range, but the short-range dedicated sensor is the authoritative voice for the final meter before contact.

6.12 Industrial Deployment

Automotive radar is the most widely deployed AV-relevant sensor by volume, shipped in tens of millions of vehicles annually for adaptive cruise control, automatic emergency braking, and blind-spot monitoring. Bosch, Continental, Aptiv, Denso, Valeo, ZF, and Veoneer are among the major Tier-1 suppliers. The 76–81 GHz band is the standard frequency for automotive long-range radar, with imaging-radar variants (using larger antenna arrays) emerging from Arbe Robotics (Israel),

Continental, and others around 2020–2024. Texas Instruments’ AWR series and NXP’s TEF810x family are widely used radar MMIC platforms. Mobileye and others have published research on radar-camera fusion architectures similar to those described in this chapter; production deployments commonly use proposal-refinement or late fusion patterns.

6.13 Chapter Summary

Radar and sonar are the AV sensors that complete the perception picture painted by camera and LiDAR: weather-robust, velocity-native, cheap, and short-range respectively. FMCW waveforms give range and Doppler simultaneously via 2D FFT processing; MIMO virtual arrays give angular resolution at modest physical cost; CFAR and DBSCAN turn the range-Doppler-angle tensor into detections; specialized trackers exploit Doppler to gate associations more aggressively than position-only trackers allow. Micro-Doppler is radar’s unique discriminator for pedestrians and cyclists. Imaging (“4D”) radars are closing the density gap with LiDAR. Ultrasonic sensors cover the last meter where radar and cameras are weakest. Deep learning on radar tensors and point clouds is active research and already enters production pipelines. The outputs of this chapter—radar tracks, radar point clouds, ultrasonic proximity signals—feed the detection, tracking, and fusion pipelines of the chapters that follow.

6.14 Exercises

1. An FMCW radar has $B = 1.5$ GHz, $T_c = 40\mu\text{s}$, $N_c = 192$, carrier 77 GHz. Compute range resolution, maximum unambiguous range at ADC rate 25 MS/s, velocity resolution, and maximum unambiguous velocity.
2. A target at $r = 75$ m, $v = -12\text{ m s}^{-1}$ is observed by the radar in Exercise 1. Compute the beat frequency, Doppler frequency, and the range-Doppler bin indices.
3. A MIMO radar has $N_T = 4$ transmit and $N_R = 4$ receive antennas. Sketch an antenna placement that produces a 16-element virtual array and compute the angular resolution at 77 GHz.
4. Derive CA-CFAR threshold multiplier α for $N_R = 24$, $P_{fa} = 10^{-4}$, and compute the threshold in dB.
5. Explain why Doppler-gated association (Example 6.9) is more effective than position-only gating when two vehicles are at nearly the same range.
6. Describe the micro-Doppler signature that differentiates a pedestrian from a cyclist at the same body-center velocity. Why is the distinction robust across lighting and weather?
7. An imaging radar produces 40000 detections per frame at 20 Hz. Compare this to a 128-beam LiDAR at 10 Hz in terms of total detections per second. What does the comparison suggest about future sensor suite composition?

8. For the radar-camera fusion architectures of Figure 6.3, identify the one that is most robust to radar calibration drift and justify.
9. A production AV uses 8 ultrasonic sensors, each with 60° beamwidth, mounted on the front and rear bumpers. Estimate the total angular coverage overlap, and explain why overlap is required for redundant safety.
10. Propose a deep-learning architecture that processes a radar range-Doppler-angle tensor and outputs 3D bounding boxes. Compare it to the LiDAR 3D-detection architectures of Chapter 11.

Chapter 7

GNSS, RTK, and Inertial Sensing

7.1 Introduction

The chapters preceding this one have developed the modalities that tell an autonomous vehicle *what is around it*: cameras for semantic richness, LiDAR for geometric precision, radar for weather-robust velocity. None of them, by themselves, tells the vehicle *where it is*. A pedestrian detected 20 m ahead is only actionable if we know the ego’s own pose in a global or map-relative frame, because the planner’s decisions (which lane to take, which route to follow, which stop-line to yield at) are all expressed in coordinates defined by the map. Localization is the layer that closes this gap, and its workhorse technologies are the Global Navigation Satellite Systems (GNSS) and the inertial measurement unit (IMU) this chapter develops.

Four observations motivate the chapter’s structure.

First, GNSS and IMU are complementary in a very specific way: GNSS provides globally-referenced absolute position at modest rate ($\leq 10\text{Hz}$) with errors that are bounded but not always small; IMU provides relative motion at high rate ($\geq 100\text{Hz}$) with errors that grow unbounded as time integrates. Fused, they produce a pose estimate that is both accurate over long horizons (GNSS anchoring) and smooth over short horizons (IMU propagation). Neither alone is sufficient for automotive deployment.

Second, there is a large gap between *consumer-grade GNSS* (3–10 m standalone accuracy, adequate for phone navigation) and what an AV actually needs (sub-lane, ideally centimeter-level, certainly under a meter). Bridging this gap requires real-time kinematic (RTK) corrections or precise point positioning (PPP); both are now commercially available at scale, but each brings its own failure modes (correction outages, long convergence times, urban multipath).

Third, the IMU’s failure mode is subtle. An IMU does not “fail” by returning wrong data; it returns data with slow biases and scale errors that, when double-integrated to position, produce unbounded drift. A vehicle in a long tunnel can accumulate tens of meters of position error over a minute if the IMU is allowed to dead-reckon without bound. Understanding the Allan-variance characterization of

IMU noise, and the filters that bound the drift, is essential.

Fourth, the localization pipeline is best understood as a set of modular components that the SLAM pipeline (Chapter 13) composes. We develop the GNSS and IMU components here; SLAM will combine them with LiDAR / camera features and map constraints to produce the full localization solution.

The chapter is organized as follows. We first cover GNSS fundamentals: satellite constellations, trilateration, and the error sources that limit standalone accuracy. We then cover RTK and differential GPS, which exploit a nearby reference station to reduce errors by one to two orders of magnitude. IMU technology follows: the accelerometer, gyroscope, and magnetometer components, their error models, and the strap-down mechanization that turns raw measurements into position and attitude. We close with GNSS/IMU fusion (loose and tight coupling), dead-reckoning in GNSS-denied environments, and the role of the magnetometer. Every formula is accompanied by a worked numerical example, and three architecture diagrams visualize key pipelines.

7.2 GNSS Fundamentals

A Global Navigation Satellite System is a constellation of satellites in medium earth orbit (20 000 km to 24 000 km altitude), each broadcasting a precisely-timed radio signal encoding its own identity and ephemeris (orbital parameters). A ground receiver measures the propagation delay from multiple satellites simultaneously and *trilaterates* its own 3D position. Figure 7.1 illustrates the geometry.

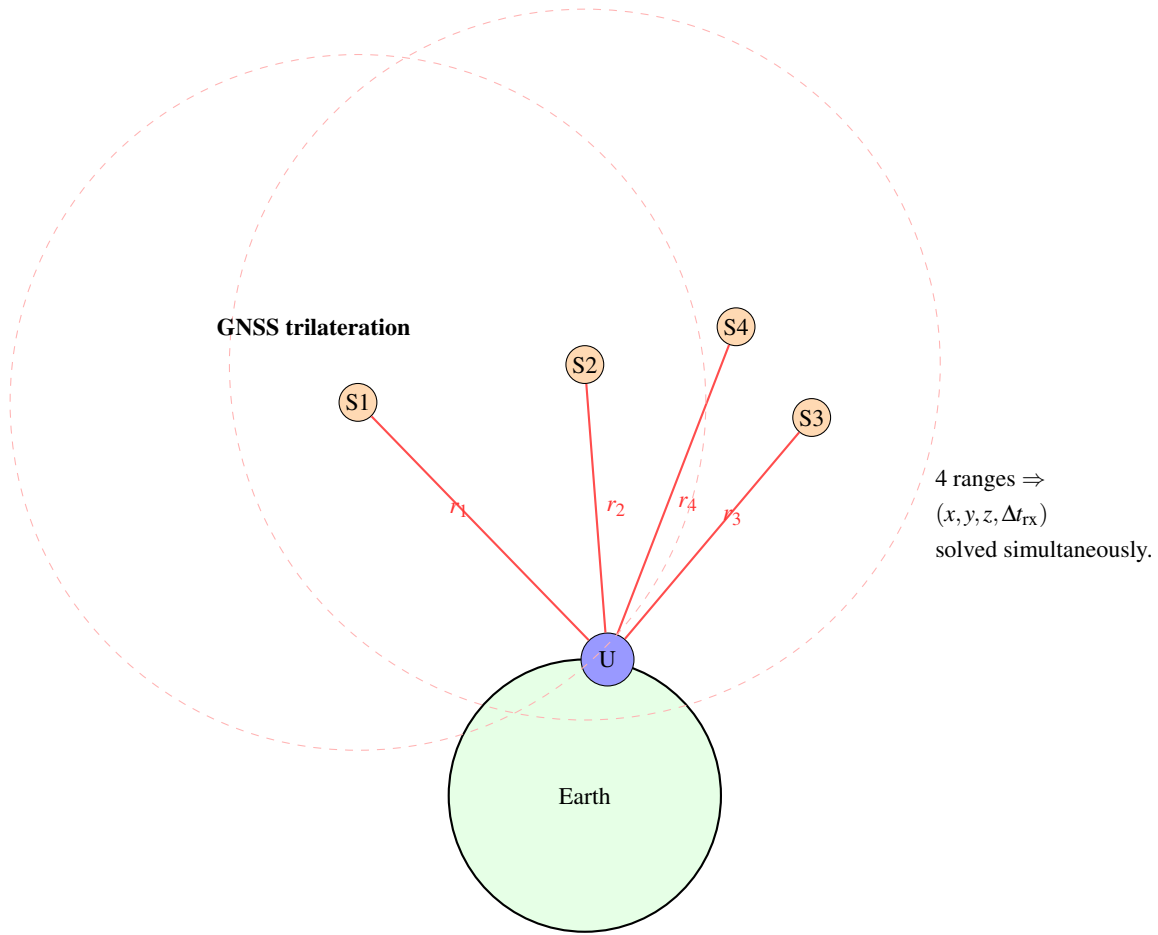


Figure 7.1: GNSS trilateration. Each satellite S_i broadcasts a timestamped signal; the receiver measures the apparent range $r_i = c(t_{rx} - t_{tx})$ to four or more satellites. Each range places the receiver on a sphere of radius r_i centered on the satellite’s known position. The intersection of four spheres determines the receiver’s 3D position (x, y, z) and the receiver clock bias Δt_{rx} —four unknowns, four measurements.

7.2.1 Trilateration and the Pseudorange Equation

Let $\mathbf{s}_i$ be the known 3D position of satellite i , and $\mathbf{u}$ the unknown receiver position. The *true* range is $\|\mathbf{s}_i - \mathbf{u}\|$. The receiver’s measured range (called the *pseudorange*) differs from the true range because the receiver clock is not perfectly synchronized with the satellite clocks:

$$\rho_i = \|\mathbf{s}_i - \mathbf{u}\| + c \Delta t_{rx} + \varepsilon_i, \quad (7.1)$$

where c is the speed of light, Δt_{rx} is the receiver clock bias, and ε_i aggregates all other error sources. Four pseudoranges give four equations in four unknowns ($u_x, u_y, u_z, \Delta t_{rx}$); a nonlinear least-squares solve recovers all four.

Example 7.1 (Pseudorange solve for a stationary receiver). A receiver sees four satellites at positions (in ECEF, km): $\mathbf{s}_1 = (15600, 7540, 20140)$, $\mathbf{s}_2 = (18760, 2750, 18610)$, $\mathbf{s}_3 = (17610, 14630, 13480)$, $\mathbf{s}_4 = (19170, 610, 18390)$. Measured pseudoranges (km): (21075, 21785, 24910, 21310). Start with guess $\mathbf{u}^{(0)} = \mathbf{0}$, $\Delta t^{(0)} = 0$. One Newton step on the linearized system converges within 3–4 iterations to $\mathbf{u} \approx (-41, -16, 6370)$ km — the receiver is on the Earth’s surface at roughly 0°

latitude — and clock bias $\Delta t_{rx} \approx 0.015$ ms (4.5 km of equivalent range). Production receivers solve this tens of times per second.

7.2.2 Satellite Constellations

Four major GNSS constellations are operational today; a modern receiver typically tracks satellites from all four simultaneously, which improves geometry and availability.

Table 7.1: The four major GNSS constellations.

Name	Operator	Satellites	Freq. bands	Notes
GPS	USA	31	L1 (1.57 GHz), L2, L5	Original; global coverage since 1995
GLONASS	Russia	24	L1 (1.6 GHz), L2, L3	FDMA, not CDMA; global
Galileo	EU	28	E1 (1.57 GHz), E5, E6	Civil-first design; high precision
BeiDou	China	35	B1, B2, B3	Global since 2020; extensive in Asia

A “multi-constellation” receiver that tracks 20+ satellites across constellations achieves meaningfully better accuracy than a single-constellation receiver (fewer outages, better geometric dilution), and is now standard in automotive-grade modules.

Example 7.2 (Availability gain from multi-constellation). In an urban canyon with half the sky view blocked by buildings, a GPS-only receiver might see 5 satellites on average, sometimes dropping to 3 (below the minimum 4 for a solution). A multi-constellation receiver seeing GPS + GLONASS + Galileo + BeiDou averages ~ 20 in view and rarely drops below 10, giving a continuous solution through the urban drive. The downside: tracking more satellites takes more compute and power; automotive receivers are designed to balance the tradeoff.

7.2.3 Error Sources

Even a clean four-satellite solution has residual error from several physical sources:

- **Ionospheric delay.** Free electrons in the ionosphere slow the carrier phase and advance the code, producing a frequency-dependent range error of 5 m to 30 m at L1. Dual-frequency receivers estimate it directly from the L1/L2 difference; single-frequency receivers use a model.
- **Tropospheric delay.** Water vapor and dry atmosphere slow the signal by 2.3 m to 30 m (zenith to horizon). Largely predictable from meteorological models.
- **Satellite clock and ephemeris errors.** Satellites do not know their own positions or clocks perfectly; broadcast corrections reduce this to 0.5–2 m.
- **Multipath.** Signals reflect off buildings and arrive via non-direct paths. Hard to correct without extra antennas or knowledge of the environment.
- **Receiver noise and thermal effects.** Small, typically well-characterized.

Standalone single-frequency GPS achieves 3 m to 10 m accuracy in open sky, degrading in cities.

7.2.4 Dilution of Precision

Even with perfect pseudoranges, the accuracy of the position solution depends on the *geometry* of the satellites. When satellites are clustered near the horizon, a given pseudorange error translates to a large position error; when they are well distributed across the sky, the same pseudorange error produces a small position error. This geometric amplification is quantified by the *Dilution of Precision* (DOP). For a pseudorange noise standard deviation σ_p :

$$\sigma_{\text{pos}} = \text{PDOP} \cdot \sigma_p, \quad (7.2)$$

with PDOP the 3D Position DOP (a dimensionless number, typically 1–5 in open sky). HDOP (horizontal) and VDOP (vertical) further separate horizontal and vertical accuracy.

Example 7.3 (DOP in open sky vs urban canyon). Open-sky PDOP ≈ 2 . With $\sigma_p = 3$ m (single-frequency), position std ≈ 6 m. Urban canyon with sky view restricted to 30° strip: PDOP ≈ 10 . Same pseudorange std gives position std 30 m. The same receiver, same satellites, just different geometry— $10\times$ worse localization. AV receivers report PDOP along with the solution; the localization stack downgrades the weight of GNSS updates when PDOP is large.

7.2.5 Velocity and Time from GNSS

Beyond position, GNSS also provides velocity (from Doppler measurements on the carrier) and precise time (from the satellite clocks). Velocity accuracy: 0.05 ms^{-1} typical, much better than differentiating position. GNSS time is the master clock that automotive compute fabric (PTP) typically syncs to.

7.3 RTK and Differential GPS

Consumer-grade accuracy (3 m to 10 m) is not adequate for autonomous driving: at 10 meters of lateral uncertainty, the vehicle does not know which lane it is in. Bridging to sub-meter and ideally sub-decimeter accuracy requires *differential* techniques that exploit a nearby reference station.

7.3.1 The Key Insight: Shared Errors

Two receivers separated by tens of kilometers see nearly identical atmospheric and satellite errors. The key insight: subtract the errors measured by a fixed, surveyed reference station from the pseudoranges seen at a moving rover, and most of the common-mode error cancels. What remains is dominated by receiver noise and multipath.

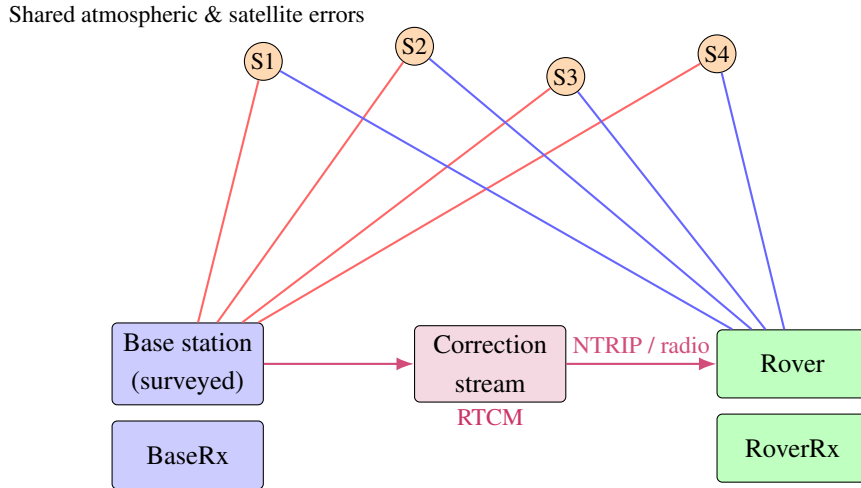


Figure 7.2: RTK differential architecture. A surveyed base station tracks the same satellites as a mobile rover. Because the two receivers share atmospheric and satellite-ephemeris errors to first order, the base computes a correction (the difference between its measured pseudorange and its known geometric range) and broadcasts this correction to the rover over NTRIP, radio, or cellular links in the RTCM protocol. The rover subtracts the correction from its own pseudoranges, canceling common-mode errors and achieving centimeter-level accuracy.

7.3.2 Differential GPS (DGPS)

Classical DGPS broadcasts code-phase corrections from the base to the rover, reducing standalone position error from 3 m to 10 m to roughly 0.5 m to 1 m. This is sufficient for some AV functions (lane detection) but still not lane-accurate.

7.3.3 Real-Time Kinematic (RTK)

RTK uses the *carrier phase* of the satellite signals, not just the code. The carrier is a sinusoid with a wavelength of 19 cm (L1); by tracking phase, receivers can measure pseudo-range to millimeter precision, modulo an unknown integer ambiguity (the number of whole cycles between satellite and receiver).

Carrier-phase pseudorange:

$$\phi_i = \frac{1}{\lambda} \|\mathbf{s}_i - \mathbf{u}\| + N_i + \frac{c}{\lambda} \Delta t_{\text{rx}} + \epsilon_i, \quad (7.3)$$

where λ is the carrier wavelength, N_i is the integer ambiguity, and the other terms are as before. The key algorithmic challenge in RTK is *integer ambiguity resolution*: determining the N_i values correctly. Once fixed (via the LAMBDA algorithm or similar), position accuracy is 1–3 cm—the “RTK-fix” state.

Example 7.4 (From RTK-float to RTK-fix). A rover starts a cold start. Initially, the integer ambiguities are unknown; the receiver uses real-valued estimates, yielding a “float” solution with 20 cm to 50 cm accuracy. Over 30–60 seconds, enough measurements accumulate for LAMBDA to resolve the integers with high confidence, and the receiver transitions to “fix” state with 1–3 cm accuracy. A cycle slip (the receiver momentarily loses phase lock due to a signal obstruction) drops back to float and restarts the 30–60s convergence. In heavily urban environments, cycle

slips are frequent enough that RTK-fix is occasionally unavailable.

7.3.4 Precise Point Positioning

PPP achieves RTK-comparable accuracy without a nearby base station, using a stream of global corrections (from providers like NovAtel TerraStar or Hexagon) and taking a longer convergence time (typically 30 minutes to an hour). It is useful where local base stations are not available or when the rover is far from any base.

7.3.5 AV Use Cases for Centimeter Positioning

Sub-decimeter positioning is critical for:

- **HD map alignment.** Production HD maps are accurate to 5 cm to 10 cm; the ego's position must match this precision to exploit the map.
- **Lane-level localization.** A lane is 3.0 m to 3.7 m wide; lane-keeping requires sub-meter lateral accuracy.
- **Autonomous parking.** Precision maneuvers in confined spaces benefit from decimeter-level global positioning.
- **Fleet coordination.** Multi-vehicle tasks (platooning, intersection coordination) rely on a shared absolute frame.

7.4 Inertial Measurement Units

GNSS updates at 10 Hz at best; AV control loops run at 100 Hz or faster. Between GNSS updates, the vehicle's pose is propagated by the IMU—a self-contained sensor that measures angular velocity and specific force at hundreds or thousands of Hz without any external reference.

7.4.1 Components

A modern automotive IMU contains three orthogonal triads:

Accelerometer. Measures the specific force $\tilde{\mathbf{a}} = \mathbf{a} - \mathbf{g}$ in the sensor frame, where $\mathbf{a}$ is true acceleration and $\mathbf{g}$ is gravity. Used for attitude (via gravity direction), velocity, and position.

Gyroscope. Measures angular velocity $\boldsymbol{\omega}$ in the sensor frame. Used for attitude propagation and for decoupling rotational from translational acceleration.

Magnetometer. Measures the local Earth magnetic field vector $\mathbf{m}$ in the sensor frame. Used primarily for heading (yaw) reference in the absence of GNSS or straight-ahead motion. Subject to iron-and-electromagnetic distortion, so calibrated online during motion.

Example 7.5 (Gravity reading at rest). A stationary IMU, leveled in the body frame with z up, reads specific force $\tilde{\mathbf{a}} = (0, 0, +g)$ where $g \approx 9.81 \text{ ms}^{-2}$. This constant reading on the z -axis

at rest confirms the sensor is working and provides the pitch and roll reference. If the vehicle tilts 10° nose-up, the x -axis reads $g \sin(10^\circ) = 1.70 \text{ ms}^{-2}$ and the z -axis reads $g \cos(10^\circ) = 9.66 \text{ ms}^{-2}$. Pitch is recoverable as $\arctan(\tilde{a}_x/\tilde{a}_z)$ as long as the vehicle is accelerating slowly; during hard braking or acceleration, the gravity signal is mixed with kinematic acceleration and pitch estimation becomes ambiguous without a gyro.

7.4.2 IMU Grades

Table 7.2: Rough IMU grades and automotive suitability.

Grade	Gyro bias	Accel bias	Typical use
Consumer MEMS	$> 10^\circ/\text{h}$	10 mg	phones, toys
Industrial MEMS	$\sim 1^\circ/\text{h}$	1 mg	drones, AV L2
Tactical	$< 0.1^\circ/\text{h}$	$< 100 \mu\text{g}$	AV L4, robotics
Navigation	$< 0.01^\circ/\text{h}$	$< 10 \mu\text{g}$	aerospace

An industrial-grade MEMS IMU costs $\sim \$100$ and is adequate for most AV applications when tightly fused with GNSS and visual odometry. Tactical-grade units cost more and are used in robotaxi programs where GNSS outages are long enough that unaided IMU drift matters for safety.

7.4.3 Error Model

Real IMU measurements deviate from the true values by three kinds of error, for each axis:

$$\tilde{\boldsymbol{\omega}} = \mathbf{S}_\omega \boldsymbol{\omega} + \mathbf{b}_\omega + \mathbf{n}_\omega, \quad (7.4)$$

$$\tilde{\mathbf{a}} = \mathbf{S}_a (\mathbf{a} - \mathbf{g}) + \mathbf{b}_a + \mathbf{n}_a. \quad (7.5)$$

Here $\mathbf{S}$ is the scale-factor matrix (including cross-axis coupling), $\mathbf{b}$ is the bias (slowly varying), and $\mathbf{n}$ is white noise. Biases are modeled as random walks: $\dot{\mathbf{b}} = \mathbf{n}_b$.

7.4.4 Allan Variance Characterization

IMU noise has multiple regimes: white noise at short time scales, bias instability at medium scales, rate random walk at long. The *Allan variance* $\sigma^2(\tau)$ (or its square root, the Allan deviation) plotted against averaging time τ reveals these regimes:

$$\sigma^2(\tau) = \frac{1}{2(N-1)} \sum_k (\bar{\omega}_{k+1}(\tau) - \bar{\omega}_k(\tau))^2, \quad (7.6)$$

where $\bar{\omega}_k(\tau)$ is the average of ω over the k -th window of length τ .

Example 7.6 (Reading an Allan-variance curve). A MEMS gyro's Allan-deviation curve shows:

- Slope $-1/2$ at small τ : white noise. Intercept at $\tau = 1$ gives the *angle random walk* (ARW) parameter, typically $0.5^\circ/\sqrt{\text{h}}$.
- Flat minimum around $\tau \sim 100\text{s}$: bias instability. Value: 5°h^{-1} —the irreducible long-term drift.
- Slope $+1/2$ at large τ : rate random walk. Shows how bias wanders over hours.

Knowing these parameters lets us tune a Kalman filter’s process noise and predict attitude drift between GNSS fixes. An ARW of $0.5^\circ/\sqrt{\text{h}}$ gives $0.5 \cdot \sqrt{10/3600} \approx 0.026^\circ$ attitude error over 10 seconds, well within lane-level tolerance.

7.4.5 Strapdown Mechanization

The raw IMU measurements are integrated to produce attitude, velocity, and position in a chosen navigation frame (typically local-level East-North-Up, ENU). Figure 7.3 diagrams the integrator.

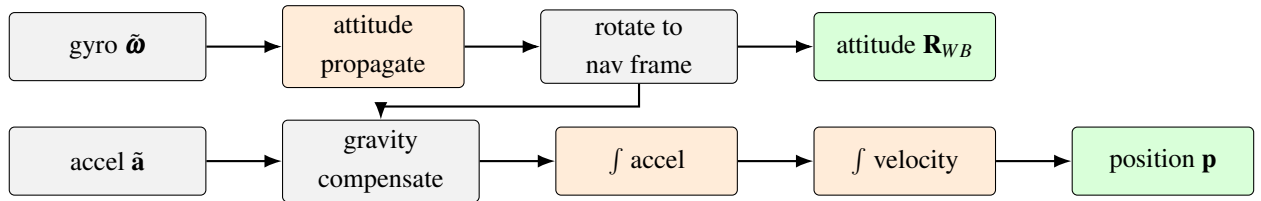


Figure 7.3: Strapdown IMU mechanization (simplified). Gyro measurements are integrated to propagate attitude $\mathbf{R}_{WB}$ (the rotation from body to navigation frame). Accelerometer measurements are rotated into the nav frame, gravity-compensated, and double-integrated to produce velocity and position. Any bias in the gyro causes attitude drift, which in turn corrupts gravity compensation and produces accelerating position error—the classical unbounded-drift problem that GNSS corrections bound.

The discrete update for attitude over interval Δt is:

$$\mathbf{R}_{WB}^{t+\Delta t} = \mathbf{R}_{WB}^t \exp((\tilde{\omega} - \hat{\mathbf{b}}_\omega) \Delta t)^\wedge, \quad (7.7)$$

where $\exp$ is the $\text{SO}(3)$ exponential map (Rodrigues formula, Chapter 2). The velocity and position updates follow:

$$\mathbf{v}^{t+\Delta t} = \mathbf{v}^t + (\mathbf{R}_{WB}^t (\tilde{\mathbf{a}} - \hat{\mathbf{b}}_a) - \mathbf{g}) \Delta t, \quad (7.8)$$

$$\mathbf{p}^{t+\Delta t} = \mathbf{p}^t + \mathbf{v}^t \Delta t + \frac{1}{2} (\mathbf{R}_{WB}^t (\tilde{\mathbf{a}} - \hat{\mathbf{b}}_a) - \mathbf{g}) (\Delta t)^2. \quad (7.9)$$

Example 7.7 (Strapdown step arithmetic). $\Delta t = 10\text{ms}$, current attitude identity, gyro reading $\tilde{\omega} = (0, 0, 0.1)\text{rad/s}$ (5.7°s^{-1} yaw rate), bias $\hat{\mathbf{b}}_\omega = 0$. Attitude increment $\Delta\theta = 0.1 \cdot 0.01 = 0.001\text{rad}$, applied about z : $\mathbf{R}_{WB}^{t+\Delta t}$ is a tiny yaw rotation.

Accel reading $\tilde{\mathbf{a}} = (1, 0, 9.81)\text{m/s}^2$ (forward acceleration plus gravity). Subtract gravity: body-frame kinematic $\mathbf{a}_{\text{kin}} = (1, 0, 0)$. Rotate to nav (approximately identity): nav acc = $(1, 0, 0)$. Velocity update: $\mathbf{v}^{t+\Delta t} = \mathbf{v}^t + (0.01, 0, 0)$. Position update: $\mathbf{p}^{t+\Delta t} = \mathbf{p}^t + \mathbf{v}^t \cdot 0.01 + (0.5 \times 10^{-4}, 0, 0)$.

This is the entire arithmetic of IMU propagation, repeated at 100–1000 Hz. Unmodeled errors accumulate quadratically in position; the corrective Kalman filter loop below bounds them.

7.4.6 Dead Reckoning and Drift

Without external corrections, IMU-only position drifts *quadratically* in time due to bias integration. An industrial MEMS with 0.01 m s^{-2} accel bias produces 5 cm position drift after 1 second, 5 m after 10 seconds, 50 m after 30 seconds. In a long tunnel, an AV relying purely on IMU propagation between GNSS returns could be 10+ meters off position, well outside lane-level tolerance.

Example 7.8 (Drift budget for tunnel traversal). Tunnel: 2 km long, ego speed 20 m s^{-1} (100 s duration). Gyro bias $0.05^\circ \text{ s}^{-1}$; integrated over 100 s gives 5° heading drift. Accel bias 0.005 m s^{-2} gives position error $0.5 \cdot 0.005 \cdot 100^2 = 25 \text{ m}$ —four lane widths. Unacceptable. Mitigations: wheel odometry (constrains longitudinal velocity from ABS ticks), LiDAR odometry (constrains motion from scan-matching), and sparse map features (stop signs, lane markings re-acquired visually). The best production stacks combine all four.

7.5 GNSS/IMU Fusion

The two sensors are made for each other: IMU provides high-rate motion with bounded short-term accuracy; GNSS provides low-rate absolute position with bounded long-term accuracy. Fusion, via an Extended Kalman Filter (Chapter 2) or a factor-graph backend (Chapter 13), yields a pose estimate superior to either alone.

7.5.1 Loose vs Tight Coupling

Loosely coupled. GNSS is processed to a position/velocity/time solution by the receiver; this solution is injected as an EKF update at the GNSS rate. Simple and modular; does not gain from the GNSS raw measurements.

Tightly coupled. Raw GNSS pseudoranges are used as EKF updates directly, alongside IMU propagation. The filter maintains a consistent error model across sensors and can benefit from even single-satellite updates when full four-satellite geometry is unavailable.

Deeply coupled. The receiver's tracking loops are aided by IMU-predicted motion, so that the receiver can maintain lock through brief signal outages. Used in military and high-end aerospace systems; gaining ground in automotive.

7.5.2 State Representation

A typical GNSS/IMU EKF state is 15-dimensional:

$$\mathbf{x} = (\mathbf{p}, \mathbf{v}, \mathbf{R}_{WB}^\vee, \mathbf{b}_\omega, \mathbf{b}_a) \in \mathbb{R}^{15}, \quad (7.10)$$

with attitude parameterized by tangent-space perturbations of a nominal rotation. Gravity is a known constant; the reference frame is local ENU tangent to the Earth's ellipsoid at a datum point.

Example 7.9 (Tight coupling through a single satellite). In an urban canyon, the rover temporarily sees only one satellite (the others are blocked by buildings). Loose coupling rejects this update—four satellites are needed for a position fix. Tight coupling uses it: the single-satellite pseudorange constrains one component of the state, tightening the posterior along that direction. Over a few seconds, as different single satellites come in and out of view, a tightly-coupled filter maintains a usable—if degraded—fix. This is why tight coupling is the production standard in L4 robotaxi stacks.

7.5.3 Initialization

Cold-start localization: the EKF needs a reasonable initial estimate of attitude, velocity, and position. Position and velocity come from the first GNSS fix. Attitude comes from (i) an “alignment” procedure at rest using the gravity vector, which gives roll and pitch but not yaw, and (ii) motion-based yaw recovery once the vehicle starts driving (the direction of velocity agrees with the heading for a non-holonomic vehicle).

7.5.4 Outlier Rejection

Both GNSS and IMU deliver occasional outliers: GNSS multipath produces pseudoranges tens of meters off; IMU occasional glitches (vibration, mechanical knock) produce spurious spikes. The EKF must detect and reject these or suffer corrupted state. Standard mechanism: reject updates whose Mahalanobis distance from the prior exceeds a χ^2 threshold.

7.6 Worked Examples

Example 7.10 (RTK accuracy near a building). A vehicle in open sky receives RTK-fix solution with 2 cm horizontal std. Approaching a downtown building, one satellite signal is lost, sky view drops to 70 %. Receiver drops to RTK-float; reported accuracy becomes 40 cm std. The localization stack must detect this transition (via the reported fix-type field in the NMEA/proprietary messages) and adjust the weight given to GNSS in its fusion. Failure to adjust weights has been documented as the cause of several localization drift incidents.

Example 7.11 (IMU drift in a tunnel with wheel odometry aiding). Tunnel 1.5 km long at 25 m s^{-1} ; 60 s duration. Pure IMU drift: 9 m from accel bias 0.005 m s^{-2} double-integrated. Adding wheel odometry (constraining longitudinal velocity to within 1 %), the along-track error shrinks to 0.25 m, a $\sim 36\times$ improvement. Cross-track error remains dominated by gyro drift; LiDAR scan matching against tunnel walls can further bound this. With all four sensors (IMU + wheel + LiDAR + occasional GNSS at portal), localization stays within lane-level tolerance throughout.

Example 7.12 (Full GNSS/IMU EKF single-step update). At time t : EKF state $\hat{\mathbf{x}} = (\mathbf{p}, \mathbf{v}, \mathbf{R}_{WB}^V, \mathbf{b}_\omega, \mathbf{b}_a)$ with $\mathbf{p} = (100, 50, 0)$ m, $\mathbf{v} = (20, 0, 0)$ m/s, $\mathbf{R}_{WB} = \mathbf{I}$. Covariance $\mathbf{P}$ with position std 0.5 m, velocity std 0.1 m s^{-1} , attitude std 0.5° .

IMU propagate 10 ms: accel $(0.3, 0, 9.81)$, gyro $(0, 0, 0.02)$. Apply Eqs. 7.7–7.9: $\mathbf{p} \rightarrow (100.2, 50.0, 0)$, $\mathbf{v} \rightarrow (20.003, 0, 0)$, yaw increment 0.011° . Covariance inflated via $\mathbf{P} \rightarrow \mathbf{F}\mathbf{P}\mathbf{F}^\top + \mathbf{Q}$; position std barely changes, attitude std grows slightly.

GNSS update at 100 ms: receiver reports position $(100.05, 50.1, 0)$ with covariance $\text{diag}(0.04, 0.04, 0.09)$. Kalman gain computed; state correction applied. Position std tightens from propagated 0.52 m to 0.19 m.

7.7 Industrial Deployment

Production AVs typically combine GNSS receivers from suppliers such as u-blox, Trimble, NovAtel (now part of Hexagon), and Septentrio with MEMS IMUs from suppliers including Bosch, STMicroelectronics, Analog Devices, Honeywell, and Murata. Real-Time Kinematic (RTK) and Precise Point Positioning (PPP) services provide centimeter-level positioning when correction signals are available; commercial correction-network providers include Trimble VRS, Leica SmartNet, Hexagon HxGN SmartNet, and Swift Navigation’s Skylark. The L1/L2/L5 multi-frequency multi-constellation receivers used in production AV systems incorporate signals from GPS (United States), GLONASS (Russia), Galileo (European Union), and BeiDou (China). The IEEE Standard for Inertial Sensor Terminology (IEEE 528) defines noise-density and bias-stability terms used to specify automotive IMUs.

7.8 Chapter Summary

GNSS and IMU together provide the absolute and incremental motion information every AV pose estimate depends on. GNSS anchors the stack in a globally-referenced frame with bounded but modest accuracy (3–10 m standalone, 1–3 cm with RTK); IMU propagates motion at high rate with unbounded drift that external corrections must bound. The four operational GNSS constellations together provide better availability and geometry than any single one. RTK carrier-phase corrections achieve centimeter-level accuracy in open sky; urban canyons degrade this through multipath and signal blockage. IMU error models (bias, scale, noise, random walk) are characterized by Allan variance; strapdown mechanization turns raw measurements into pose, velocity, and attitude. Fusion via tightly-coupled EKF or factor graph produces the best results, especially in GNSS-denied environments where single-satellite updates and wheel/visual odometry become critical. The pose output of this chapter feeds directly into the SLAM and HD-map chapters of Part III.

7.9 Exercises

1. Derive the linearized Newton step for the pseudorange solve (Eq. 7.1) given an initial guess. What quantities define the Jacobian?
2. For the DOP example (7.3) with PDOP = 6 and pseudorange noise $\sigma_p = 2$ m, compute the expected position std and compare to open-sky PDOP = 2.
3. Explain why dual-frequency receivers can estimate ionospheric delay from the L1/L2 signal difference. Write the equations.
4. What is the minimum number of satellites needed for a 3D position fix, and why?
5. For an RTK receiver with $\lambda = 19$ cm and phase precision of 1 % of λ , compute the ranging precision and the position accuracy assuming PDOP = 2.
6. A gyro has $\text{ARW} = 0.3^\circ/\sqrt{\text{h}}$. Compute the expected heading drift after 30 seconds of unaided integration.
7. For the strapdown example (7.7), what is the effect of a 0.1 mg accel bias on position after 10 seconds of flat driving?
8. Describe the *tight coupling* advantage in an urban canyon where only 2 of 4 satellites are reliable.
9. Why is magnetometer-based heading unreliable near large structures (bridges, overpasses, rail tracks)? Propose a detection mechanism for magnetometer-unreliable regions.
10. Propose an end-to-end localization pipeline for an L4 robotaxi that must handle (a) open sky, (b) urban canyon with multipath, (c) tunnel with no GNSS, (d) emergence from tunnel back to open sky. Identify the sensor and algorithm responsible for localization in each regime.

Chapter 8

Thermal and Infrared Imaging

8.1 Introduction

Every object warmer than absolute zero radiates electromagnetic energy; the warmer it is, the more it radiates and the shorter the peak wavelength. A human body at 310 K radiates strongly in the long-wave infrared (LWIR) band around $8\text{ }\mu\text{m}$ to $14\text{ }\mu\text{m}$, a wavelength the human eye cannot see and a visible-spectrum camera cannot detect. A thermal infrared camera can. The result is a perception modality that sees in absolute darkness, that shows living bodies as bright features against cool backgrounds regardless of lighting, that penetrates light haze and smoke more effectively than visible light, and that produces a signature for pedestrians and animals so distinctive that detection rates exceed visible cameras in every one of these conditions. This chapter develops the physics, hardware, calibration, and perception algorithms that make thermal imaging a first-class AV sensor.

Three observations motivate the chapter's structure.

First, thermal imaging is a geometry-free semantic cue. Unlike LiDAR or radar, it does not measure range or velocity; it measures apparent surface temperature. This means thermal output cannot stand alone for localization or 3D reasoning. Its value is in complementing geometric sensors with a detection cue that is *robust to lighting* and *specific to living bodies*. Every production deployment uses thermal as a secondary modality paired with camera or radar.

Second, thermal-specific failure modes are different from visible-spectrum failure modes. Glass is opaque in LWIR; a pedestrian behind a glass door is invisible. Rain-wet surfaces equilibrate to ambient temperature and lose contrast. Direct sun on a dark road heats it to body-temperature-like values, producing false-positive warm regions. Understanding these failure modes is essential to fusing thermal with other modalities sensibly.

Third, thermal cameras have historically been expensive. Military-grade cooled detectors cost tens of thousands of dollars; automotive-grade uncooled microbolometers now cost a few hundred dollars and ship on vehicles from BMW, Cadillac, Volkswagen, and others. The cost trajectory is favorable for AV integration; the engineering work required to make thermal useful is now the bottleneck, not

the hardware cost.

The chapter is organized as follows: we first review the physics of thermal radiation, then cover sensor hardware (uncooled vs cooled, detector arrays), then image calibration, then pedestrian and animal detection algorithms, then fusion with visible cameras, and close with production deployments. Three architecture diagrams visualize the IR spectrum and the fusion pipelines; worked examples cover the radiometric arithmetic throughout.

8.2 Physics of Thermal Radiation

A blackbody (ideal absorber and emitter) at absolute temperature T emits spectral radiance described by Planck's law:

$$B_\lambda(T) = \frac{2hc^2}{\lambda^5} \cdot \frac{1}{\exp(hc/\lambda kT) - 1}, \quad (8.1)$$

with h Planck's constant, c the speed of light, k the Boltzmann constant, and λ the wavelength. The wavelength of peak emission depends only on T (Wien's displacement law):

$$\lambda_{\text{peak}} = \frac{2898}{T} \mu\text{m}, \quad (8.2)$$

and the total emitted power per unit area follows the Stefan–Boltzmann law:

$$P = \sigma T^4, \quad \sigma = 5.67 \times 10^{-8} \text{ W/m}^2/\text{K}^4. \quad (8.3)$$

Real objects emit a fraction $\varepsilon \leq 1$ of the blackbody power, where ε is the material's emissivity.

Example 8.1 (Peak wavelength for three scene elements). Human body at $T = 310\text{ K}$: $\lambda_{\text{peak}} = 2898/310 = 9.35 \mu\text{m}$ —squarely in LWIR. Hot asphalt at $T = 330\text{ K}$: $\lambda_{\text{peak}} = 8.78 \mu\text{m}$. Sunlit windshield at $T = 280\text{ K}$: $\lambda_{\text{peak}} = 10.35 \mu\text{m}$. All three peak within the LWIR $8 \mu\text{m}$ to $14 \mu\text{m}$ band, which is why LWIR sensors dominate automotive thermal. The body- versus-background temperature difference ($\sim 15\text{ K}$ on a cool night) produces substantial radiometric contrast even though peak wavelengths differ by only a few μm .

Example 8.2 (Stefan-Boltzmann power from human body). Person with skin area $A = 1.8\text{ m}^2$, temperature $T = 307\text{ K}$, emissivity $\varepsilon \approx 0.98$ (skin). Total radiative power: $P = \varepsilon \sigma A T^4 = 0.98 \cdot 5.67 \times 10^{-8} \cdot 1.8 \cdot 307^4 \approx 890\text{ W}$. A thermal camera detects a fraction of this (the solid-angle subtended by its lens). Even at 100 m range, enough photons reach the detector for 10 mK temperature resolution—extremely fine, enough to distinguish a walking person from a cooler object at the same position.

8.2.1 The Infrared Bands

Not all IR is useful for thermal imaging. Figure 8.1 shows the three bands.

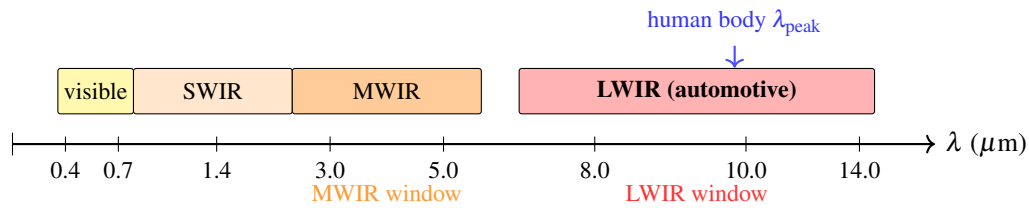


Figure 8.1: The infrared spectrum relevant to imaging. Visible light peaks in the $0.4\text{ }\mu\text{m}$ to $0.7\text{ }\mu\text{m}$ range. SWIR ($1\text{ }\mu\text{m}$ to $3\text{ }\mu\text{m}$) is useful for some industrial applications (see-through haze, reflective imaging). MWIR ($3\text{ }\mu\text{m}$ to $5\text{ }\mu\text{m}$) is a high-performance thermal band but requires cooled detectors. LWIR ($8\text{ }\mu\text{m}$ to $14\text{ }\mu\text{m}$) is the automotive band: human body emission peaks here, and the atmosphere transmits well. Uncooled microbolometers operate natively in LWIR, making the band cost-effective at automotive scale.

- **SWIR** (Short-Wave Infrared, $1\text{ }\mu\text{m}$ to $3\text{ }\mu\text{m}$). Dominated by reflected sunlight, not thermal emission. Penetrates light haze; sees through some visible-opaque materials. Specialized applications only.
- **MWIR** (Mid-Wave Infrared, $3\text{ }\mu\text{m}$ to $5\text{ }\mu\text{m}$). High radiometric contrast between warm objects and cool backgrounds. Requires cooled detectors ($T_{\text{cold}} \approx 80\text{ K}$) for best performance; used in military/aerospace, rarely in automotive.
- **LWIR** (Long-Wave Infrared, $8\text{ }\mu\text{m}$ to $14\text{ }\mu\text{m}$). The automotive thermal band. Peak human-body emission; atmospheric transparency; uncooled detectors (microbolometers) work natively. Cost-performance sweet spot for vehicles.

8.3 Thermal Sensor Hardware

Thermal imagers fall into two categories based on detector technology.

8.3.1 Uncooled Microbolometers

A microbolometer is an array of tiny resistors whose resistance changes with temperature. Each pixel absorbs incident LWIR radiation, heats up by a small amount ($< 1\text{ K}$), and the resistance change is read out. Typical array sizes: 320×240 to 1024×768 pixels, pixel pitch $12\text{ }\mu\text{m}$ to $25\text{ }\mu\text{m}$. Response time (thermal time constant): 8 ms to 15 ms , suitable for vehicle-rate imaging at 30 Hz to 60 Hz .

Example 8.3 (Noise-equivalent temperature difference (NETD)). A typical automotive-grade microbolometer has $\text{NETD} = 50\text{ mK}$ at 300 K scene temperature. A pedestrian ($T_{\text{skin}} \approx 307$) on a 280 K -road background: contrast $\Delta T = 27\text{ K}$, which is $540\times$ above NETD. At 100 m range, atmospheric attenuation reduces apparent ΔT to perhaps 20 K , still far above noise. Detection is limited by pixel count (at 100 m , a pedestrian fills only a few pixels laterally) rather than radiometric SNR.

8.3.2 Cooled Detectors

MWIR and high-performance LWIR detectors use cryogenic cooling (typically 77 K via Stirling cooler) to reduce thermal noise in the detector itself. NETD drops to 10 mK to 20 mK , pixel pitch

can be smaller, and frame rates higher. Cost: $\geq \$10k$ per camera, plus maintenance. Not automotive-scale.

8.3.3 Pixel Array Geometry and Optics

Thermal optics require germanium or zinc-selenide lenses (silica glass is opaque in LWIR). Germanium lenses are expensive but compact; molded chalcogenide glass is increasingly used to lower cost. Fields of view for automotive thermal cameras: 24° to 36° horizontal, roughly matching the forward driver-view. Narrow-field tele thermal cameras for long-range pedestrian detection are emerging.

8.4 Image Calibration

Raw thermal output is non-uniform, temperature-dependent, and requires substantial calibration to be usable.

8.4.1 Non-Uniformity Correction (NUC)

Each pixel has a slightly different response curve. A two-point calibration against known-temperature references produces per-pixel gain and offset corrections:

$$I_{\text{corr}}(x, y) = g(x, y) \cdot I_{\text{raw}}(x, y) + o(x, y), \quad (8.4)$$

where g and o are per-pixel calibration maps. Factory calibration is supplemented by in-vehicle recalibration against internal shutter references every few minutes.

Example 8.4 (Before and after NUC). Raw thermal frame of a uniform-temperature wall: pixel intensity std across the frame 500 counts (severe non-uniformity). After NUC: std drops to 15 counts—a $33\times$ improvement. NUC residuals are the limit of radiometric accuracy; with careful calibration, absolute temperatures are accurate to ± 2 K.

8.4.2 Dead-Pixel Replacement

Manufacturing yields produce cameras with a few dead or erratic pixels. These are mapped in factory calibration and replaced at runtime with interpolation from neighbors.

8.4.3 Vignetting and Lens Thermal Artifacts

The lens itself emits LWIR. A hot lens barrel produces a halo of false signal. Production calibration models this lens-bore emission and subtracts it.

8.5 Pedestrian and Animal Detection

The primary AV application of thermal is pedestrian/animal detection in low-light or total-darkness conditions. Standard deep-learning detectors (Chapter 11) apply directly; the main engineering question is how to train.

8.5.1 Thermal-Specific Training Data

Public datasets: FLIR-PV (paired RGB-thermal), KAIST multispectral, BU-TIV, and automotive-vendor internal sets. Most datasets pair thermal with visible so that detectors can learn cross-modal correspondences.

8.5.2 Class-Specific Features

A thermal pedestrian signature has a characteristic appearance: exposed skin (face, hands) is hotter than clothed body; head is often the brightest feature. Detectors trained on thermal learn these regularities and can work at ranges and lighting conditions where visible detectors fail.

Example 8.5 (Detection range in darkness). Visible-camera pedestrian detection on a dark rural road (headlights only): max reliable range 30 m. Thermal detection on the same scene: reliable to 150 m ($5\times$ improvement). The ego has ~ 7 s additional reaction time at highway speeds—the difference between “avoid” and “collide”. This is the value proposition behind Cadillac’s Night Vision system and BMW’s Night Vision 3 system.

8.5.3 Animal Detection

Large animals (deer, moose, cattle, horses) produce distinctive thermal signatures on cool backgrounds. Collision with large animals is a major safety problem on rural roads; thermal detection reduces it significantly.

8.6 Fusion with Visible-Spectrum Cameras

Thermal and visible cameras produce complementary information; fusion produces better detection than either alone. Figure 8.2 shows the three dominant patterns.

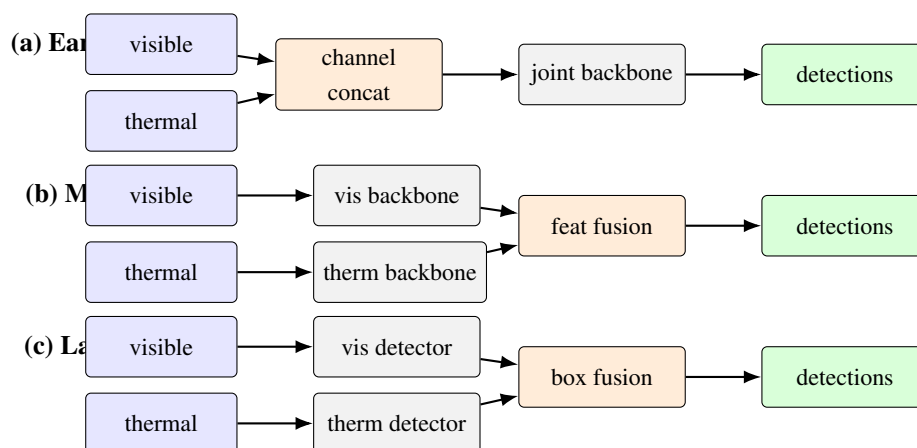


Figure 8.2: Three canonical thermal/visible fusion architectures. *Early* fusion stacks both images as channels and runs a single joint backbone—efficient but requires tight registration. *Mid* fusion runs separate backbones and concatenates features at a mid-layer, tolerating modest misalignment. *Late* fusion produces independent detection lists from each modality and combines them at the box level—robust to one sensor failing, but misses shared-feature gains.

Example 8.6 (Fusion gain on pedestrian detection). Visible-only AP on KAIST multispectral: 0.72. Thermal-only AP: 0.79 (better than visible on this dataset, which is skewed to night scenes). Early-fusion AP: 0.85; mid-fusion: 0.87; late-fusion: 0.81. Mid-fusion wins because it captures shared features without being hurt by registration errors; early fusion is close when the calibration is tight. Late fusion is worst because box-level combination cannot exploit feature-level complementarity.

8.7 Production Deployments

Notable automotive thermal deployments:

- **BMW Night Vision:** LWIR camera + HUD overlay with pedestrian alerts; deployed since 2005, continuously iterated.
- **Cadillac Night Vision:** FLIR-based, pedestrian highlighting in the center cluster display.
- **Volkswagen Night Vision:** similar to Cadillac; commercialization of AI pedestrian classification.
- **Teledyne FLIR ADK:** automotive development kit explicitly targeting AV programs; pre-integrated with perception stacks.
- **Mobileye AEB-Dark:** software-only dark-conditions pedestrian detection paired with forward visible camera; approaches thermal performance via learned night-time inference.

Production deployments remain secondary-modality: thermal augments visible and radar rather than replacing them. The forward-camera suite plus thermal covers the pedestrian-dark use case more reliably than any single modality.

8.8 Worked Examples

Example 8.7 (End-to-end pedestrian detection at night). Dark rural road, ego at 25 m s^{-1} . Pedestrian 120 m ahead with dark clothing ($T_{\text{clothing}} \approx 295 \text{ K}$) but visible skin ($T_{\text{skin}} \approx 307 \text{ K}$) on a 285 K road surface. Thermal contrast (skin vs road): 22 K; clothing vs road: 10 K. Both exceed NETD by large margins. Pedestrian occupies $\sim 5 \times 20$ pixels at 120 m in a 640×480 thermal camera (typical automotive config). Deep detector trained on FLIR-PV produces detection with confidence 0.92. The visible camera sees nothing useful at that range (headlight attenuation + pedestrian dark clothing). Thermal is the difference between safe and unsafe here; ego has $\sim 4.8 \text{ s}$ to react at 25 m s^{-1} , easily sufficient.

Example 8.8 (NUC drift over operation). A thermal camera calibrated at factory has NETD 50 mK. After 30 minutes of continuous operation with internal lens heating, camera responsivity drifts; residual non-uniformity climbs to effective NETD 120 mK—degrading detection

marginally. Internal shutter recalibration (every 5 minutes, takes 200 ms) restores NETD to 55 mK. Without periodic recalibration, thermal detection range would drop by 10 % to 20 % over a long drive.

Example 8.9 (Animal detection on highway). Deer on the side of a rural 2-lane at night, $T_{\text{animal}} \approx 310\text{ K}$. Thermal contrast against grass ($T_{\text{grass}} \approx 280\text{ K}$): 30 K. Detection range of a 320×240 LWIR camera with 50 mK NETD: $\sim 140\text{ m}$, limited by angular resolution (at 140 m, a 1 m-tall deer spans 2° -of-view, about 2 pixels). Combined with radar velocity/motion cues, the perception stack issues an alert and prepares brake profile—enough time at 25 m s^{-1} ego speed to avoid collision.

8.9 Industrial Deployment

Thermal-imaging cameras have entered automotive production primarily for night-time pedestrian detection. FLIR Systems (now part of Teledyne Technologies) supplies thermal cameras used in research and select production night-vision systems; their Boson and ADK product lines are positioned for automotive integration. BMW and Mercedes-Benz have offered thermal-night-vision options on premium models for over a decade. Production volume of thermal sensors in automotive remains far below visible-light cameras, with cost per unit historically much higher, though microbolometer prices have declined as smartphone-thermal and consumer applications have grown the supplier base.

8.10 Chapter Summary

Thermal imaging complements visible-spectrum cameras by providing robust detection in darkness, fog, and low contrast. The physics is well-understood (Planck, Wien, Stefan-Boltzmann); the automotive sweet spot is LWIR with uncooled microbolometers, at cost points now attractive for mass-market deployment. Pedestrian and animal detection are the primary use cases, where thermal extends detection range by $3\text{--}5\times$ over visible-only. Fusion with visible cameras at either early, mid, or late stages produces detection performance exceeding either alone. Production deployments (BMW, Cadillac, VW) validate the technology; integration into L2+ and L3 ADAS stacks is increasingly standard.

8.11 Exercises

1. Using Wien's law, compute the peak emission wavelength for a tire at $T = 330\text{ K}$ and a block of ice at $T = 270\text{ K}$.
2. Compute the Stefan-Boltzmann radiative power of a 2.3 m^2 person at $T = 306\text{ K}$, $\varepsilon = 0.97$.
3. Explain why glass is opaque in LWIR but transparent in visible. Cite one AV implication.
4. For the NETD example (8.3), compute the required pedestrian contrast in Kelvin for a detector

with NETD 150 mK at 3-sigma detection threshold.

5. A thermal camera's NUC gain $g(x, y)$ and offset $o(x, y)$ are calibrated at two reference temperatures. Derive the formulas for g and o given two readings at known temperatures.
6. Compare the cost, performance, and use cases of MWIR vs LWIR in the automotive context.
7. A thermal camera has 640×512 pixels, 50° horizontal FoV. At what range does a 1.7 m-tall person span 4 pixels vertically?
8. For the fusion-gain example (8.6), explain why mid-fusion outperforms early fusion on the KAIST dataset.
9. Propose a thermal-specific augmentation strategy for training a detector on a dataset collected mostly in one season.
10. Design a fusion architecture that gracefully degrades when the thermal camera becomes misaligned with the visible camera due to a minor impact.

Chapter 9

Environmental Sensors

9.1 Introduction

The previous chapters developed the *primary perception* modalities: cameras, LiDAR, radar, thermal imaging, GNSS/IMU. Each produces high-bandwidth structured data that the perception stack consumes. This chapter is about a different, often overlooked category of sensors that measure the *environmental context* in which driving happens: is it raining? how bright is the ambient light? what is the temperature? These measurements are small scalars or low-resolution streams, not megapixel tensors, but they play an outsized role in vehicle behavior. They drive automatic wipers, headlights, defoggers, and increasingly they feed the perception stack itself as priors that modify how raw sensor data is interpreted.

Three observations motivate the chapter’s structure.

First, environmental sensors are *cheap and ubiquitous* —nearly every modern car carries them for ADAS and comfort functions. But their integration into higher-level AV perception is newer. A perception stack that knows it is raining can downgrade camera and LiDAR confidence, elevate radar priority, and reduce the planner’s aggressiveness. A stack that doesn’t know will face the same environmental challenge unaware.

Second, each environmental sensor has its own physics and its own failure modes. An optical rain sensor sees rain by total internal reflection loss; it will not detect rain on a non-windshield surface. A photodiode light sensor measures ambient illuminance with a spectral-response curve matched to human photopic vision; a different response would give different readings. Understanding these details is what separates a well-tuned stack from one that oscillates based on noisy sensor outputs.

Third, environmental sensors *cascade*. The rain sensor triggers the wipers; the wipers alter the forward-camera view; the camera’s perception output changes; the planner’s behavior changes. This cascade is a feature of AVs that goes beyond any single sensor. Understanding the cascade is essential to designing a coherent end-to-end stack.

The chapter is organized as follows: rain sensors, light sensors, temperature sensors (ambient and road-surface), and humidity sensors. We then develop how these environmental readings feed the perception and planning stack, with particular attention to the weather-adaptive policy cascade. Two architecture diagrams visualize the sensor principles and the integration patterns; worked examples cover the physical arithmetic throughout.

9.2 Rain Sensors

Automotive rain sensors detect the presence and intensity of precipitation on the windshield so that the wipers can respond automatically. Two dominant designs.

9.2.1 Optical Rain Sensors

An optical rain sensor uses total internal reflection (TIR) at the windshield glass-air interface. A near-infrared LED injects light into the windshield at an angle exceeding the critical angle for TIR; on a dry surface, nearly all the light reflects and reaches a photodiode. A water droplet on the outer surface disrupts the glass-air boundary—the critical angle condition fails, light escapes, and the photodiode signal drops.

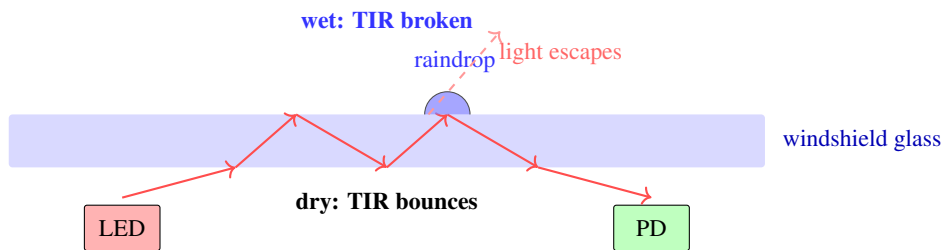


Figure 9.1: Optical rain-sensor principle. A near-infrared LED launches light into the windshield at an angle above the critical angle for total internal reflection (TIR); on a dry surface, nearly all light reflects along the glass and reaches the photodiode (PD). A water droplet on the outer surface changes the local refractive index, breaking TIR—light escapes upward out of the glass and the photodiode’s received signal drops. The drop magnitude is roughly proportional to the covered droplet area.

The critical angle for glass-air TIR:

$$\theta_c = \arcsin(n_2/n_1), \quad (9.1)$$

with $n_1 \approx 1.52$ (glass) and $n_2 \approx 1.0$ (air), giving $\theta_c \approx 41^\circ$. When water ($n_2 \approx 1.33$) replaces air at a droplet location: $\theta_c \approx 61^\circ$ —if the light was traveling at the dry-critical angle of 41° , it no longer meets the wet-critical angle and escapes the glass.

Example 9.1 (Droplet coverage vs signal drop). Sensor area 4 cm^2 . Light rain: 2 droplets per cm^2 on the sensor area, each 2 mm diameter covering $\sim 3 \text{ mm}^2$. Covered fraction: $8 \cdot 3/400 = 6\%$. Photodiode signal drops by roughly this fraction—small but repeatable. Heavy rain: 20 droplets/ cm^2 , 3 mm diameter: covered fraction $\sim 40\%$; signal drops substantially. Wiper

controller maps the signal to a wipe rate. Production sensors do fractional-level detection in under 50 ms.

9.2.2 Capacitive Rain Sensors

Some rain sensors use capacitive detection between embedded electrodes in the windshield; water changes the dielectric. Capacitive sensors detect sheet water (films) better than optical sensors but are more expensive to integrate.

9.2.3 Role in the AV Stack

Beyond driving the wipers, rain-sensor data enters the perception stack as a weather prior. A “heavy rain” signal triggers:

- Camera confidence downgrade (typically -20 to -40% on small-object detection).
- LiDAR confidence downgrade (multi-return artifacts).
- Radar confidence maintained (radar sees through rain).
- Planner speed reduction (typically -10 to -20% from the nominal speed limit).
- Increased follow-distance (typically $1.5\times$ longer headway).

9.3 Light Sensors

A light sensor (photodiode or photoresistor) measures the ambient illuminance, typically in lux. The output drives automatic headlight on/off, auto-dimming rear-view mirrors, and display brightness adaptation. The production sensor is typically a silicon photodiode with a filter matching the human photopic luminous efficiency function:

$$E_v = 683 \text{ lm/W} \int V(\lambda) E_e(\lambda) d\lambda, \quad (9.2)$$

with $V(\lambda)$ the photopic curve (peaked at 555 nm) and E_e the incident spectral irradiance.

Example 9.2 (Illuminance at three times of day). Noon direct sun: $E_v \approx 100000 \text{ lx}$. Overcast midday: $E_v \approx 10000 \text{ lx}$. Twilight: $E_v \approx 400 \text{ lx}$. Moonless starlight: $E_v \approx 0.001 \text{ lx}$. Auto-headlight threshold: typically 100 lx to 500 lx . Auto-headlights-on is triggered at the twilight transition; the specific threshold is tuned to avoid nuisance activation on cloudy days. A hysteresis loop of $\sim 50\%$ prevents oscillation around the threshold.

9.3.1 Integration with AV Perception

For AVs, the light sensor provides a dark/day signal that modulates:

- Camera exposure priors (camera firmware uses the sensor signal to set exposure rapidly rather than via image-based AE loops).
- Thermal-camera priority (at low light, elevate thermal confidence in fusion).
- Driving-policy envelope (reduce speed at dusk/dawn transitions).

9.4 Temperature Sensors

Automotive temperature measurements serve multiple purposes.

9.4.1 Ambient Air Temperature

A thermistor (negative-temperature-coefficient resistor) behind the front grille measures the ambient air temperature. The Steinhart-Hart equation maps thermistor resistance to temperature:

$$\frac{1}{T} = A + B \ln R + C (\ln R)^3, \quad (9.3)$$

where A, B, C are calibration constants and R is the measured resistance. Precision: ± 1 K at room temperature, with calibration.

9.4.2 Road-Surface Temperature

A contactless IR pyrometer (effectively a single-pixel LWIR thermometer) aimed at the road measures the surface temperature directly. This is useful for ice detection: a road-surface temperature at or below freezing, in the presence of humidity or precipitation, triggers an ice-risk warning.

Example 9.3 (Ice-risk heuristic). Ambient air: -2°C ; road surface (pyrometer): -4°C ; humidity 85 %; recently rain-detected. Ice-risk classifier: all four conditions satisfied ($T_{\text{road}} \leq 0^\circ\text{C}$, high humidity, recent moisture, cold air); output: “high ice risk.” Planner responds: reduce speed by 25 %, increase follow-distance 50 %, avoid hard brakes. Compare to ambient $+5^\circ\text{C}$, dry road: no alert. This simple multi-sensor rule underlies the winter-driving capability of production L2 systems.

9.4.3 Self-Heating and Placement

Thermistors dissipate heat when measuring; the self-heating error in a sensor mounted inside a warm engine bay can be several degrees. Production sensors are shielded and ventilated, mounted in external trim, away from engine heat. The ambient temperature reading drifts upward during long idling as accumulated heat around the sensor rises; a well-designed placement minimizes this.

9.5 Humidity Sensors

Relative humidity sensors (typically polymer capacitive) provide readings used for:

- **Defogging control:** the HVAC system pre-empts windshield fogging before the driver notices.
- **Ice-risk classification:** high humidity at near-freezing temperatures is a strong ice precursor.
- **Camera fogging mitigation:** high cabin humidity with cold exterior glass pre-emptively activates demisting.

9.6 Integration: The Environmental Sensor Bus

Figure 9.2 shows how these small sensors feed into the AV stack.

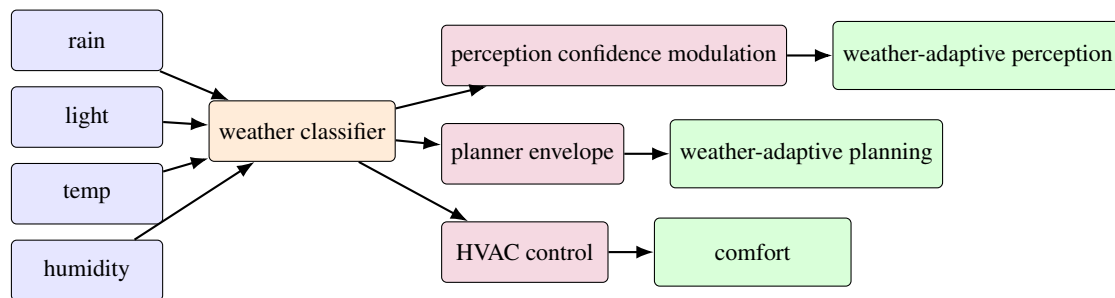


Figure 9.2: Environmental-sensor integration in an AV. Rain, light, temperature, and humidity sensors feed a weather classifier that produces discrete weather states (clear, rain-light, rain-heavy, fog, snow, ice-risk). The classifier output modulates three downstream consumers: perception (confidence scaling, modality prioritization), the planner (speed/follow-distance envelope), and HVAC (defogging/defrosting). The loop from environmental sensors to perception/planning behavior is tight: updates at 1 Hz are typical, and the state is smoothed to avoid oscillation.

Example 9.4 (Weather-state transition cascade). $t = 0$: sunny. Rain signal rises over 20 s. Weather classifier transitions through “drizzle” to “light rain”. Cascade:

- Wipers: off $\rightarrow$ intermittent $\rightarrow$ low.
- Headlights: on (auto-low-light triggered by cloud).
- Camera-perception confidence: -15% on <50 m classes.
- Radar: unchanged.
- Planner envelope: max speed -10% below limit, follow-distance $+25\%$.

All changes take effect smoothly over several seconds, not abruptly. Hysteresis on the weather classifier prevents oscillation between “drizzle” and “light rain.”

9.7 Worked Examples

Example 9.5 (Ice-risk response over a 30 min drive). Drive starts sunny, air $+5^\circ\text{C}$. Vehicle enters a shaded valley; road surface pyrometer drops to -1°C while air reads $+2^\circ\text{C}$. Humidity 80 %. Weather classifier elevates ice-risk from “none” to “moderate.” Planner reduces speed 15 % below limit, eliminates braking below -3 m/s^2 , increases follow-distance. After the valley: road surface returns to $+4^\circ\text{C}$; ice-risk drops to “low” after a 30-second hysteresis delay. Speed

and margins return to nominal. A single-sensor heuristic (only using air temperature) would have missed the shaded-road ice risk entirely.

Example 9.6 (Night/dusk transition for camera-primary stack). Sunset; light sensor reading drops from 2000 lx at 18:00 to 100 lx at 19:00. Camera auto-exposure increases, but noise grows; on a camera-primary stack, pedestrian detection range drops from 80 m (day) to 40 m (twilight). Light-sensor cascade: (a) enables headlights, (b) increases thermal camera’s weight in fusion from 0.3 to 0.6, (c) adjusts planner max speed by -10% . Net effect: detection range via thermal restores to ~ 100 m and planner envelope compensates for the reduced margin. Without the light-sensor cue, the camera stack would be operating at day-tuned thresholds into the night, producing either missed detections (threshold too strict) or false positives (threshold too loose).

Example 9.7 (Rain sensor driving wiper + perception cascade). Optical rain-sensor signal drops by 5% (light drizzle). Wiper controller: slow intermittent. Perception stack weather-state: “drizzle.” Camera confidence reduction: marginal (-5%). Signal drops further to 30% : heavy rain. Wipers continuous. Weather state: “rain.” Camera penalty: -30% . LiDAR penalty: -15% (multi-return artifacts). Radar: unchanged. Fusion reweights toward radar. Planner speed reduced to 80% of nominal; follow-distance $\times 1.5$. The whole chain executes within a couple seconds of the first rain-sensor signal change—well before any driver would manually adjust behavior.

9.8 Industrial Deployment

Environmental sensing in production AVs uses a combination of dedicated rain sensors (commonly photoelectric units behind the windshield, supplied by Bosch, Hella, and others), ambient-light sensors, and inferences from camera and wiper-system signals. Tesla’s vehicles famously use their forward-facing cameras to detect raindrops and trigger automatic wipers, moving away from a dedicated photoelectric rain sensor as a parts-reduction measure. Bosch and Continental supply combined rain-light-humidity sensor modules used by many European OEMs. Air-quality sensing inside the cabin is increasingly common for HVAC management and uses sensors from suppliers such as Sensirion (Switzerland).

9.9 Chapter Summary

Environmental sensors measure the weather-and-light context in which the AV operates: rain, ambient light, air and road temperature, humidity. Individually, each sensor serves a comfort or ADAS function (auto-wipers, auto-headlights, ice warning). Integrated into the AV stack, they feed a weather classifier whose output modulates perception confidence, modality weighting, planner envelope (speed, follow-distance, acceleration limits), and HVAC behavior. The cascade is what makes an AV behave appropriately in conditions it was not specifically trained on: it does not need to be a rain-expert if it knows that it is raining and adjusts accordingly. Production L2+ and L3 systems all include environmental sensors as standard equipment; L4 programs increasingly add road-surface pyrometers

and dedicated weather- classifier modules to handle edge conditions.

9.10 Exercises

1. Using Eq. 9.1, compute the critical angle for TIR at a glass-air interface with $n_{\text{glass}} = 1.50$ and at a glass-water interface with $n_{\text{water}} = 1.33$.
2. For the rain-coverage example (9.1), compute the expected signal drop for a uniform 20 % covered-fraction.
3. A light sensor's photodiode has peak responsivity 0.5 A W^{-1} at 555 nm. Estimate the photocurrent at 500 lx incident illuminance.
4. Derive the Steinhart-Hart equation's A, B, C coefficients from three calibration points (R_i, T_i) .
5. Propose an ice-risk classifier using ambient air, road surface, humidity, and rain signals as inputs. Specify the decision thresholds.
6. Explain why a capacitive rain sensor detects sheet water better than an optical rain sensor.
7. For the night-transition example (9.6), quantify the effective reliable-detection range with and without the light-sensor cascade.
8. A road-surface pyrometer reads -3°C while air reads $+1^\circ\text{C}$. What is the physical explanation and what should the planner do?
9. Design a humidity-driven defogging controller. What additional sensor information would it need?
10. Propose a fusion of the environmental-sensor bus into a learned perception network. Should the weather classifier be explicit or learned end-to-end?

Chapter 10

Sensor Technologies and Sensor Fusion

10.1 Introduction

Autonomous driving begins with measurement. Every decision the system makes—whether to brake, steer, yield, or proceed—derives, directly or through many layers of processing, from the outputs of the sensor suite bolted to the vehicle. A perception network cannot detect a pedestrian that was not imaged; a tracker cannot refine a state it was never told about; a planner cannot reason about an obstacle whose existence was missed at the sensor. The quality of the downstream stack is upper-bounded by the quality of the sensing.

A recurring theme of the chapter, and one worth stating at the outset: *no single sensor is good at everything*. Cameras are dense, cheap, and semantically rich, but their single-frame depth is ambiguous and they degrade in low light. LiDAR gives accurate 3D geometry out to hundreds of meters but is sparse, expensive, and attenuated by precipitation. Radar is weather-robust and measures Doppler velocity directly, but has coarse angular resolution that makes per-object extent hard to recover. GNSS gives globally-referenced position when corrections are available, but fails under urban canyons, in tunnels, and during multipath. IMUs are high-rate and environmentally indifferent, but drift without external reference. Production Level 4 vehicles therefore carry all of these in combination, and the algorithmic content of the chapter is largely concerned with how to fuse their outputs into a coherent estimate of the world.

Figure 10.1 depicts a representative sensor suite. The coverage overlaps are deliberate: every region around the vehicle is sensed by at least two modalities, so that a silent failure of a single sensor does not produce a silent failure of the whole system.

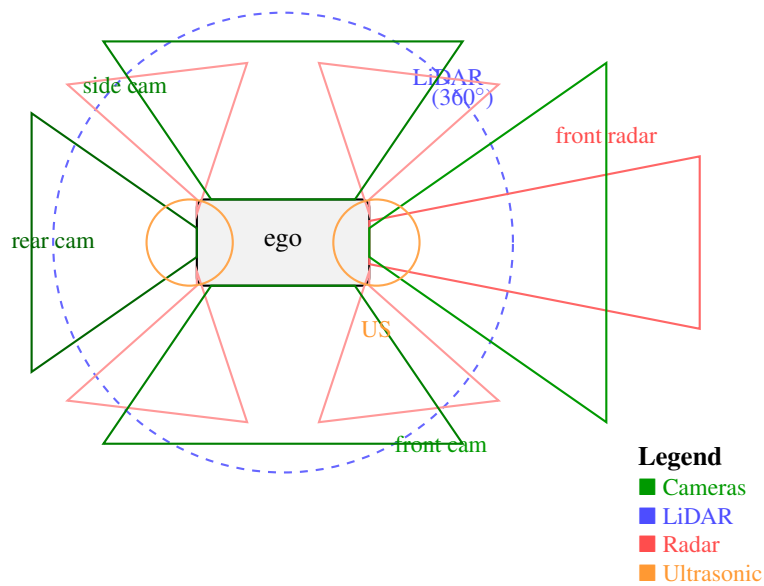


Figure 10.1: Representative Level 4 AV sensor coverage (top view). A 360° LiDAR provides baseline 3D geometry, overlapped by forward-looking and corner radars for velocity and weather robustness, surround cameras for semantic perception, and short-range ultrasonics for close-in parking. The deliberate overlap supports cross-modal consistency checks.

The chapter develops each modality in turn, with enough physics and noise modeling to understand its failure modes and enough worked examples to make the formulas concrete. We then treat the two cross-cutting problems every multi-sensor stack must solve: *calibration* (relating each sensor’s frame to the body) and *time synchronization* (making measurements from sensors at different rates and clocks comparable). We develop the Kalman filter from first principles as the canonical fusion algorithm, survey the architectural patterns (early, mid, late fusion) that combine heterogeneous modalities in production, and close with a worked-examples section covering camera projection, Kalman tracking, and FMCW radar range-Doppler.

10.2 Cameras

Cameras are the highest-bandwidth, lowest-cost sensor on an AV. A modest surround camera suite produces several gigabits per second of imagery at 10 Hz to 30 Hz—more raw data than every other modality combined—and makes possible everything semantic the stack does: reading a stop sign, distinguishing a pedestrian from a bollard, detecting that a traffic light has just turned red. The caveat is that a single image contains no reliable absolute depth, only a projection onto the image plane. Without LiDAR or stereo, depth must be recovered from learned priors, motion, or multi-camera geometry—each with characteristic failure modes that subsequent chapters address.

10.2.1 Pinhole Model and Calibration

The pinhole model is the foundation on which essentially every camera-based computation in the book rests. Object detections project from image pixels into a 3D ray via the pinhole model; BEV perception lifts image features into 3D by inverting the projection along a depth estimate; PnP solvers

recover a camera's pose by minimizing the reprojection of known 3D points. A bug in the pinhole arithmetic silently corrupts every one of these.

A perspective camera projects a 3D point $\mathbf{p}^C = (X, Y, Z)^\top$ in the camera frame to the pixel $\mathbf{u} = (u, v)^\top$ via

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} X/Z \\ Y/Z \\ 1 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \quad (10.1)$$

where f_x, f_y are focal lengths in pixels and (c_x, c_y) is the principal point (nominally the image center).

Example 10.1 (Projecting a vehicle and a pedestrian). Camera: $f_x = f_y = 1200$ px, $c_x = 960$ px, $c_y = 540$ px (HD 1920×1080). A car center at $\mathbf{p}^C = (2.0, 0.0, 50.0)$ m projects to $u = 1200 \cdot 2/50 + 960 = 1008$ px, $v = 1200 \cdot 0/50 + 540 = 540$ px. A pedestrian at $\mathbf{p}^C = (-1.0, 0.8, 20.0)$ m projects to $u = 1200 \cdot (-1.0)/20 + 960 = 900$ px, $v = 1200 \cdot 0.8/20 + 540 = 588$ px. At 20 m the pedestrian's 1.7 m height spans $1200 \cdot 1.7/20 = 102$ px (easy detection); at 80 m, the same pedestrian spans only 25 px—near the limit of reliable detection. The signal shrinks linearly with range, which is why long-range pedestrian detection is fundamentally harder than near-range.

Real lenses introduce distortion. The Brown-Conrady model captures radial (k_1, k_2, k_3) and tangential (p_1, p_2) distortion:

$$x_d = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2), \quad (10.2)$$

$$y_d = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy, \quad (10.3)$$

with $r^2 = x^2 + y^2$ and (x, y) the normalized image-plane coordinates. Calibration (Zhang's method, checkerboard-based [77]) estimates $\mathbf{K}$ and the distortion parameters from images of a known target; intrinsics are typically stable over the vehicle's lifetime but must be verified after mechanical shocks.

Example 10.2 (Radial distortion at image corners). A moderate wide-angle lens has $k_1 = -0.12$, $k_2 = 0.015$. At image corner (normalized $(x, y) = (0.5, 0.3)$, so $r^2 = 0.34$), the radial scaling factor is $1 - 0.12 \cdot 0.34 + 0.015 \cdot 0.34^2 = 1 - 0.0408 + 0.00173 \approx 0.961$. A feature ideally at $(0.5, 0.3)$ lands at $(0.481, 0.288)$ —a 4 % shift toward the center. Uncorrected, this puts stereo-matched features off by several pixels at image corners, ruining depth triangulation. Undistortion is therefore the first operation applied to any camera stream before downstream geometric reasoning.

10.2.2 Global Versus Rolling Shutter

CMOS sensors are cheap and ubiquitous, but most use a rolling shutter: each row of pixels is read out at a slightly different time, producing *skew* in scenes with relative motion. A pedestrian walking at 1.5 m s^{-1} past a camera with 30 ms total readout produces a 4.5 cm shear across the image. Naive

detection and triangulation treat the pedestrian as if captured instantaneously, producing biased poses. Global-shutter cameras (more expensive) or per-row timestamp handling in the vision stack eliminate the artifact; modern driving datasets are increasingly shipping with global-shutter or well-modeled rolling-shutter data.

Example 10.3 (Rolling-shutter skew of a lane marking). Camera readout spans 25 ms top-to-bottom over 1080 rows. Ego speed 20 ms^{-1} . A perfectly vertical lane marking in the world appears in the image as a line with slope $(25 \text{ ms}) \cdot (20 \text{ m/s}) / (1080 \text{ rows}) = 0.46 \text{ mm}$ of horizontal drift per row. Over the full image height the marking tilts by 50 cm—a significant lateral error for lane-geometry extraction if uncorrected. The fix is either global-shutter hardware or per-row unrolling using the ego velocity.

10.2.3 Photometric Noise and Dynamic Range

Image sensors add approximately Gaussian read noise plus Poisson shot noise:

$$\sigma_{\text{pix}}^2(I) = \sigma_r^2 + \alpha I, \quad (10.4)$$

where I is pixel intensity, α quantifies conversion gain, and σ_r is the pedestal read noise. Driving scenes routinely span 100 dB of scene luminance (deep shadow to direct sunlight), exceeding the 60–80 dB dynamic range of a typical sensor; HDR fusion from multiple exposures, or logarithmic sensors, are used to recover detail at both extremes.

Example 10.4 (Noise-limited detection contrast). A sensor has $\sigma_r = 2 \text{ DN}$, $\alpha = 0.05 \text{ DN}$. At pixel level $I = 100 \text{ DN}$ (midtone), $\sigma = \sqrt{4 + 5} = 3 \text{ DN}$. The minimum detectable intensity difference at 3σ is 9 DN, so a target must differ from its background by at least 9 DN to be separable per-pixel. At $I = 10$ (deep shadow), $\sigma = \sqrt{4 + 0.5} \approx 2.1$: detection threshold drops to 6.3 DN but the dynamic range available is much smaller. This is why perception in low light is disproportionately hard—not only do signals shrink, but the signal-to-noise floor determines whether the shrunken signal is distinguishable.

Example 10.5 (Stereo depth from disparity). A stereo rig with baseline $b = 0.55 \text{ m}$ and focal length $f = 1200 \text{ px}$. A feature matched across the two images has disparity $d = 40 \text{ px}$. Depth:

$$Z = \frac{fb}{d} = \frac{1200 \cdot 0.55}{40} = 16.5 \text{ m}.$$

A single-pixel disparity error at that depth changes the estimate to $Z' = 1200 \cdot 0.55 / 39 \approx 16.9 \text{ m}$: a 2.5 % relative error. At $d = 5$ (a target 132 m away), the same one-pixel error shifts depth to $1200 \cdot 0.55 / 4 = 165 \text{ m}$ —a 25 % error. Stereo depth error grows quadratically with range, which is why stereo supplements but does not replace LiDAR for long-range perception.

Example 10.6 (PnP pose of a detected vehicle). Four known 3D corner points of a vehicle's back bumper (in its own body frame) are $\mathbf{P}_i \in \{(\pm 0.9, 0.4, 0), (\pm 0.9, -0.4, 0)\} \text{ m}$ (a 1.8 m-

wide bumper). A detector finds their projections in the ego camera at pixel coordinates $\mathbf{u}_i = \{(1125, 580), (1208, 579), (1124, 605), (1210, 604)\}$. Camera intrinsics as in Example 10.1.

Given $\mathbf{K}$, the minimal PnP problem (4 points, one camera) solves for $\mathbf{T} \in \text{SE}(3)$ by minimizing reprojection error. The centered mean of the observed pixels is roughly (1167, 592); with $c = (960, 540)$ and $f = 1200$, the vehicle is at angle $(207/1200, 52/1200) \approx (0.17, 0.043)$ rad off the optical axis. The observed bumper width in pixels (~ 84 px) corresponds to $Z \approx f \cdot 1.8/84 \approx 26$ m range. A full iterative PnP (EPnP, then Levenberg–Marquardt refinement) returns $\mathbf{t} \approx (4.5, 1.1, 25.7)$ m and a small rotation consistent with the observed bumper top/bottom parallax. Monocular PnP is the workhorse of object 6-DoF pose from bounding-box keypoints.

10.3 LiDAR

If cameras are the AV’s semantic sense, LiDAR is its geometric sense. A LiDAR produces a point cloud—tens to hundreds of thousands of 3D points per frame—with range accuracy of a few centimeters out to 100 m to 200 m. That geometry is the basis of most 3D detection, SLAM, and HD-map matching. LiDARs come in mechanical spinning, MEMS, solid-state flash, and FMCW flavors, each with its own cost, range, and reliability profile.

10.3.1 Principle of Operation

Time-of-flight LiDAR ranges by measuring round-trip time of a laser pulse:

$$r = \frac{c\tau}{2}, \quad (10.5)$$

with c the speed of light and τ the round-trip time. Range accuracy is fundamentally bounded by the timing precision of the receiver’s photodetector.

Example 10.7 (Range resolution from timing resolution). A receiver times arrival to within $\sigma_\tau = 100$ ps. Range uncertainty: $\sigma_r = c\sigma_\tau/2 = 3 \times 10^8 \cdot 10^{-10}/2 = 1.5$ cm. To reach 3 cm accuracy with cheaper timing (200 ps) requires averaging 4 pulses (factor $\sqrt{4}$). High-end automotive LiDARs quote $\sigma_r \leq 3$ cm at 100 m under typical reflectivity.

FMCW LiDAR, in contrast, ranges by frequency modulation rather than pulse timing, giving both range and radial velocity in a single measurement (like FMCW radar):

$$r = \frac{cf_b}{2S}, \quad v = \frac{\lambda f_d}{2}, \quad (10.6)$$

with f_b the beat frequency, S the chirp slope, f_d the Doppler offset, λ the laser wavelength. FMCW LiDAR is rare in current production but gaining ground.

10.3.2 Point Cloud Structure

A typical mechanical LiDAR (e.g., Velodyne VLP-32, Ouster OS-1-64) produces a ring-organized point cloud at 10 Hz: 32 or 64 laser channels at fixed elevation angles, each sweeping 360° in azimuth. Each point carries $(x, y, z, \text{intensity}, \text{ring}, \text{time})$ at minimum. Total points per frame: 100 000 to 2 000 000 depending on sensor.

Example 10.8 (Angular resolution at range). A 64-beam LiDAR with 26.9° vertical field of view has vertical resolution 0.43° ($26.9/63$). Azimuthal resolution at 10 Hz with 18 000 samples per revolution: 0.02° . At 50 m range, the ground projection of two neighboring vertical beams separates by $50 \tan(0.43^\circ) \approx 0.37$ m, comparable to a child's height—which is why small obstacles are easily missed between beams at moderate range. This sparsity is the fundamental limitation of mechanical LiDAR.

10.3.3 Range Image Representation

Because of its ring structure, a LiDAR frame can be reshaped into a dense 2D image of size $(\text{rings}) \times (\text{azimuth samples})$, with range, intensity, and Cartesian coordinates as channels. Range images unlock efficient CNN-based processing of what would otherwise be an unordered point set. Architectures such as RangeNet++ and SqueezeSegV3 use this representation directly for semantic segmentation of LiDAR.

10.3.4 Weather and Reflectivity

LiDAR returns attenuate with fog, rain, and snow. Production stacks detect these conditions (via return statistics, intensity drops, or dedicated weather classifiers) and either fall back to radar-heavy processing or signal degraded operation. Reflectivity variation across materials (lane-marking paint is a factor of $5\text{--}10\times$ more reflective than asphalt) is exploited for lane-line detection directly from LiDAR intensity, independently of camera imagery.

Example 10.9 (Ground plane RANSAC from LiDAR). A LiDAR returns 80 000 points per frame. Ground-plane extraction assumes the ground is locally a plane $ax + by + cz + d = 0$; RANSAC samples three points, fits a plane through them, and counts inliers (points within 5 cm of the plane). With an estimated inlier ratio $w = 0.6$ (ground typically makes up 50 % to 70 % of a sweep over a flat road), the probability that a three-point sample is all-inlier is $w^3 = 0.216$. For 99 % confidence, $N = \log(0.01)/\log(1 - 0.216) \approx 19$ iterations suffice. On modern hardware each iteration takes microseconds; the full ground-plane fit completes in under 1 ms. The fitted plane seeds free-space reasoning and also provides a pitch reference for adaptive banking of the BEV projection.

Example 10.10 (LiDAR motion undistortion). A mechanical LiDAR spinning at 10 Hz produces its 360° sweep over 100 ms. The ego vehicle moves $v \cdot \Delta t = 30 \cdot 0.1 = 3$ m during one sweep. Points acquired at t_0 and $t_0 + 100$ ms are separated by 3 m of ego motion that naive processing

treats as stationary. Motion undistortion transforms each point to the pose it would have had at the frame's central timestamp: $\mathbf{p}^{\text{corrected}} = \mathbf{T}_{t_c, t_i} \mathbf{p}_i^{\text{raw}}$. Without this correction, a static wall appears as a 3 m-long arc. Every production LiDAR pipeline does this; VIO ego pose (Chapter 13) provides the per-point $\mathbf{T}$.

10.4 Radar

Radar is the AV sensor that keeps working when the others stop. It penetrates rain and fog at reasonable rates, measures Doppler velocity directly, and is robust to lighting. Its cost is angular resolution: a typical automotive radar has few-degree beam widths where a LiDAR has sub-degree resolution, so object extent is imprecise. Radar is therefore most valuable as a velocity sensor and as a weather backup, not as a primary geometric sense.

Modern automotive radars are frequency-modulated continuous-wave (FMCW). A linear chirp of bandwidth B and duration T_c transmits; the received signal beats against the transmitted signal to produce a difference frequency proportional to range:

$$r = \frac{c f_b}{2S}, \quad S = B/T_c. \quad (10.7)$$

Radial velocity comes from the chirp-to-chirp phase progression via the Doppler shift:

$$v = \frac{\lambda f_d}{2}, \quad (10.8)$$

with $\lambda = c/f_c$ the carrier wavelength. A 2D FFT over fast-time (range) and slow-time (Doppler) resolves multiple targets simultaneously across both dimensions.

Example 10.11 (Range and Doppler resolution). A 77-GHz radar with $B = 1$ GHz, $T_c = 40 \mu\text{s}$, $N_c = 128$ chirps per frame. Range resolution: $\Delta r = c/(2B) = 0.15$ m. Max unambiguous range (from ADC sample rate assumed matching $B/2$): $r_{\text{max}} \sim 150$ m. Doppler resolution with frame duration $T_f = N_c T_c = 5.12$ ms: $\Delta v = \lambda/(2T_f) = (3 \times 10^8 / 7.7 \times 10^{10}) / (2 \cdot 5.12 \times 10^{-3}) \approx 0.38 \text{ m s}^{-1}$. Max unambiguous velocity: $\pm \lambda/(4T_c) \approx \pm 24 \text{ m s}^{-1}$. A typical automotive radar therefore distinguishes vehicle closing rates to within 0.4 m s^{-1} , which is sufficient for ACC and AEB; but its 0.15 m range resolution and several-degree angular resolution make it coarse for per-object extent.

Angular resolution comes from multiple transmit/receive antennas and digital beamforming. Modern MIMO radars with virtual arrays of 12 or more elements achieve $\sim 1^\circ$ angular resolution; four-dimensional (“4D”) radars additionally resolve elevation, approaching LiDAR-like point-cloud density at a fraction of the cost.

Example 10.12 (CFAR detection threshold). Cell-averaging CFAR compares the power in a test cell to the average power in N_R reference cells on either side (skipping guard cells to avoid bleed-over from the target). Threshold: $T = \alpha \bar{P}_R$ where α sets the false-alarm rate: $\alpha = N_R(P_{fa}^{-1/N_R} - 1)$.

For $N_R = 16$ and $P_{fa} = 10^{-4}$, $\alpha = 16 \cdot (10^{4/16} - 1) = 16 \cdot (3.162 - 1) \cdot 10^{-1} \approx 2.16$. Wait: rewriting, $P_{fa}^{-1/N_R} = (10^{-4})^{-1/16} = 10^{0.25} \approx 1.778$, so $\alpha = 16(1.778 - 1) \approx 12.45$ (linear scale). A target at 3.0 m range yielding range-bin power P_t is declared a detection iff $P_t > 12.45 \bar{P}_R$ (roughly 11 dB above the local noise floor). Heavy clutter (tunnel walls, highway guardrails) inflates $\bar{P}_R$ and raises the threshold, protecting against false alarms but also suppressing distant faint targets—a central radar tradeoff.

Example 10.13 (MIMO virtual array and angular resolution). A MIMO radar has $N_T = 2$ transmit and $N_R = 4$ receive antennas, giving $N_T N_R = 8$ virtual-array elements. Element spacing $d = \lambda/2 = 1.95$ mm at 77 GHz, so array aperture $L = 7d = 13.7$ mm. Angular (Rayleigh) resolution: $\Delta\theta \approx \lambda/L = 0.29$ rad $\approx 16^\circ$. Increasing to $N_T = 3, N_R = 4$ gives 12 virtual elements, $L = 11d$, $\Delta\theta \approx 10^\circ$: doubling element count roughly halves resolution. Production 4D imaging radars reach 192 or more virtual elements and approach 1° resolution—still coarser than LiDAR but fine enough for many detection tasks.

10.5 GNSS and Inertial Sensing

GNSS and IMU together provide the absolute and incremental position information that every ego-pose estimate leans on. GNSS anchors the stack in a global reference frame when it works; the IMU keeps the estimate rolling at hundreds of Hz when GNSS is poor or absent.

10.5.1 GNSS Basics

A GNSS receiver computes its 3D position by trilateration from the time-of-flight of signals from ≥ 4 satellites. Standalone accuracy is 3 m to 10 m in open sky, driven by ionospheric delays, ephemeris error, and receiver noise. Real-time kinematic (RTK) corrections from a fixed reference station reduce this to 1 cm to 3 cm; precise point positioning (PPP) achieves similar accuracy over longer convergence times using satellite-broadcast corrections.

Example 10.14 (RTK accuracy near a building). An RTK receiver in open sky reports 2 cm *horizontal* accuracy. As the vehicle enters a street with a 10-story building on one side, the sky view drops to 70 % of the hemisphere, multipath appears in several signals, and the RTK solution downgrades to “RTK-float” with reported 40 cm accuracy. The localization stack must detect this degradation and weight the GNSS measurement accordingly; feeding a 40 cm fix into an EKF with covariance tuned for 2 cm causes the filter to become overconfident and reject correct LiDAR updates as outliers.

Urban environments are the graveyard of GNSS. Multipath—signals arriving via reflected paths—creates position errors uncorrelated with nominal noise models; tunnels drop the fix entirely; even overpasses cause brief outages. A production AV therefore treats GNSS as one sensor among many, never as ground truth.

10.5.2 Inertial Measurement Units

An IMU reports specific force $\tilde{\mathbf{a}}_k$ (accelerometer) and angular velocity $\tilde{\boldsymbol{\omega}}_k$ (gyroscope), typically at 100 Hz to 1000 Hz. The measurement model is

$$\tilde{\boldsymbol{\omega}}_k = \boldsymbol{\omega}_k + \mathbf{b}_\omega + \mathbf{n}_\omega, \quad (10.9)$$

$$\tilde{\mathbf{a}}_k = \mathbf{R}_{BW}(\mathbf{a}_k - \mathbf{g}) + \mathbf{b}_a + \mathbf{n}_a, \quad (10.10)$$

with biases $\mathbf{b}_\omega, \mathbf{b}_a$ drifting as slow random walks, $\mathbf{n}_\omega, \mathbf{n}_a$ zero-mean white noise, and $\mathbf{g}$ gravity in the world frame. Without correction, biases drive integrator drift; a raw integration of IMU over 1 s accumulates position error at the 10 cm-per-second scale for an automotive-grade IMU.

Example 10.15 (Bias-induced drift). An automotive-grade accelerometer has bias $b_a = 0.01 \text{ m s}^{-2}$ (unknown). Integrated once, this produces velocity error $t \cdot b_a$; integrated twice, position error $t^2 b_a / 2$. Over $t = 1 \text{ s}$: position drift 0.005 m—negligible. Over $t = 10 \text{ s}$: 0.5 m—significant. Over $t = 60 \text{ s}$: 18 m—catastrophic without external correction. This is why IMU preintegration must be paired with GNSS, LiDAR, or visual fixes; and why a vehicle in a long tunnel (GNSS-denied, few visual features) is a stress case for the entire localization stack.

Wheel odometry (tick counts from ABS encoders) complements the IMU with a scale-true but slip-sensitive constraint on longitudinal velocity. The tri-fusion of IMU, wheel, and GNSS (plus LiDAR/camera when available) is the standard ego-localization architecture.

Example 10.16 (Wheel-odometry velocity from encoder ticks). An ABS encoder produces 48 pulses per wheel revolution. Tire effective radius $r_e = 0.34 \text{ m}$, so distance per tick is $2\pi r_e / 48 = 0.0445 \text{ m}$. Over a 10 ms window at highway speed, a wheel sees $\sim 30 \text{ m/s} \cdot 0.01 / 0.0445 \approx 6.7$ ticks; the integer-tick quantization implies velocity resolution $\pm 4.5 \text{ m s}^{-1}$ at that window—unusable. Extending the window to 100 ms reduces quantization error to $\pm 0.45 \text{ m s}^{-1}$, tolerable for an EKF. Tire radius r_e changes with wear, pressure, and load; online identification against GNSS-derived speed is standard.

Example 10.17 (Multipath GNSS error). A GNSS signal reflects off a building with excess path length $\Delta L = 15 \text{ m}$ before arriving at the receiver. The direct path range is r_d ; the reflected path contributes an apparent range $r_d + \Delta L$. If the receiver's correlator locks on the combined (direct + reflected) peak, it can bias the pseudorange estimate by 1–5 m depending on relative powers. A fix affected by multipath from several satellites may deviate by 5 m to 30 m from truth, an order of magnitude worse than open-sky error. Runtime multipath detection combines carrier-to-noise dips, satellite-geometry (DOP) degradation, and filter-innovation statistics to flag affected measurements.

10.6 Sensor Extrinsic Calibration

Each sensor on the vehicle lives in its own frame; downstream perception and fusion require each sensor's frame to be expressed in a common frame, usually the body (ISO 8855). A sensor's *extrinsic* calibration is the rigid transform $\mathbf{T}_{BS}$ from sensor frame S to body B . If this transform is off by even small amounts, the point-clouds, images, and radar returns are spatially misregistered and every downstream module degrades.

$$\mathbf{p}^B = \mathbf{T}_{BS}\mathbf{p}^S = \mathbf{R}_{BS}\mathbf{p}^S + \mathbf{t}_{BS}. \quad (10.11)$$

Example 10.18 (Lateral error from extrinsic yaw miscalibration). A LiDAR is mounted with true yaw $\mathbf{T}_{BS}$ having $\theta_{BS} = 0^\circ$, but calibration has estimated $\hat{\theta}_{BS} = 0.5^\circ$. A point at $\mathbf{p}^S = (50, 0, 0)$ (50 m ahead) should map to $(50, 0, 0)$ in the body but instead maps to $(50 \cos 0.5^\circ, 50 \sin 0.5^\circ, 0) \approx (50, 0.44, 0)$. The 0.44 m lateral offset at 50 m range is half a lane width—enough to mis-associate a leading car to the wrong lane. Extrinsic calibration is therefore specified to arc-minute accuracy for long-range sensors. Online calibration monitors detect drift and trigger recalibration when thresholds are exceeded.

Calibration is either done offline with targets (checkerboards for camera-to-camera, retroreflective patterns for camera-to-LiDAR, corner-cube targets for radar) or online via motion correlation (hand-eye calibration between an IMU and a camera from trajectory-consistency constraints). Online methods are increasingly preferred because mechanical shifts in the vehicle shift extrinsics faster than offline calibration campaigns can keep up.

Example 10.19 (Hand-eye calibration from motion). Two sensors A and B are rigidly mounted on the vehicle; we do not know $\mathbf{T}_{AB}$ but each sensor independently estimates its own trajectory. At two instants, sensor A reports motion $\mathbf{T}_A = \text{yaw}(5^\circ), \mathbf{t} = (1.0, 0, 0)$ and sensor B reports the same motion but in its own frame: $\mathbf{T}_B = \text{yaw}(5^\circ), \mathbf{t} = (0.98, 0.17, 0)$ (same rotation magnitude, but translation tilted because B sits to the side). The hand-eye relation $\mathbf{T}_A\mathbf{T}_{AB} = \mathbf{T}_{AB}\mathbf{T}_B$ constrains $\mathbf{T}_{AB}$: the rotation matches are automatic, and the translation equation yields $(\mathbf{R}_A - \mathbf{I})\mathbf{t}_{AB} = \mathbf{R}_{AB}\mathbf{t}_B - \mathbf{t}_A$. Stacking several such pairs over a few seconds of varied motion gives an overdetermined linear system whose least-squares solution recovers $\mathbf{t}_{AB}$. In production, this is how camera-IMU and LiDAR-IMU extrinsics are kept calibrated online without stopping the vehicle for a target.

10.7 Time Synchronization

A fused estimate at time t requires knowing what each sensor measured *at time* t . In an AV this is nontrivial: sensors have independent clocks, different latencies, and different sampling rates. A 10 ms timing error between a camera and a LiDAR at 30 m s^{-1} produces 30 cm of spatial mismatch, enough to destroy feature-level fusion.

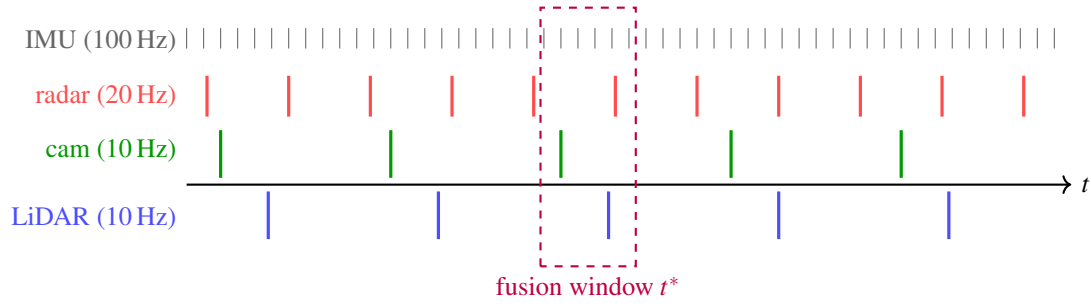


Figure 10.2: Time synchronization in a multi-sensor AV stack. Different sensors sample at different rates and phases; a fusion module reconciles them by either buffering and aligning to a common timestamp t^* or by propagating asynchronous observations through the filter as they arrive with their per-sample timestamps.

Two patterns solve synchronization. **Hardware triggers** generate a common pulse that starts each sensor's integration, and each sensor's latency to data-ready is characterized and compensated. **Software alignment** timestamps every measurement at the receiving compute node using a precision protocol (PTP, IEEE 1588) and buffers streams for a short interval before fusing. Most production stacks combine both: PTP across the network backbone, plus hardware triggers for critical sensors (camera and LiDAR) that must be phase-locked.

Example 10.20 (Motion compensation between camera and LiDAR). Camera frame is captured at t_c ; LiDAR frame at $t_l = t_c + 20$ ms. Ego speed is 30 ms^{-1} . If we wish to project LiDAR points into the camera image at t_c , we must transform them backwards by the ego motion over 20 ms: $\Delta t \cdot v = 0.6 \text{ m}$. Ignoring this offset yields a 0.6 m lateral mismatch at every projected LiDAR point (for perpendicular motion) or longitudinal mismatch of the same scale. Motion compensation using the IMU-integrated ego trajectory over the interval is standard.

10.8 Sensor Fusion: The Kalman Filter

With calibration and synchronization handled, we now have comparable measurements from multiple sensors and must combine them into a single estimate. The Kalman filter is the workhorse fusion algorithm, and essentially every mid-level tracker, every IMU-GNSS fusion, and every map-matching localizer in an AV stack is a Kalman filter (or one of its nonlinear variants).

10.8.1 Derivation

Given a linear-Gaussian system $\mathbf{x}_{k+1} = \mathbf{F}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$ with $\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k)$ and measurements $\mathbf{y}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$ with $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_k)$, the Kalman filter maintains posterior mean $\hat{\mathbf{x}}_k$ and covariance $\mathbf{P}_k$. Predict:

$$\hat{\mathbf{x}}_{k+1}^- = \mathbf{F}_k \hat{\mathbf{x}}_k + \mathbf{B}_k \mathbf{u}_k, \quad (10.12)$$

$$\mathbf{P}_{k+1}^- = \mathbf{F}_k \mathbf{P}_k \mathbf{F}_k^\top + \mathbf{Q}_k. \quad (10.13)$$

Update:

$$\mathbf{S}_{k+1} = \mathbf{H}_{k+1} \mathbf{P}_{k+1}^- \mathbf{H}_{k+1}^\top + \mathbf{R}_{k+1}, \quad (10.14)$$

$$\mathbf{K}_{k+1} = \mathbf{P}_{k+1}^- \mathbf{H}_{k+1}^\top \mathbf{S}_{k+1}^{-1}, \quad (10.15)$$

$$\hat{\mathbf{x}}_{k+1} = \hat{\mathbf{x}}_{k+1}^- + \mathbf{K}_{k+1} (\mathbf{y}_{k+1} - \mathbf{H}_{k+1} \hat{\mathbf{x}}_{k+1}^-), \quad (10.16)$$

$$\mathbf{P}_{k+1} = (\mathbf{I} - \mathbf{K}_{k+1} \mathbf{H}_{k+1}) \mathbf{P}_{k+1}^-. \quad (10.17)$$

Example 10.21 (Tracking across camera and radar fusion). Track a vehicle with state $\mathbf{x} = (x, y, v_x, v_y)$. CV dynamics, $\Delta t = 0.05$ s: $\mathbf{F} = \begin{pmatrix} \mathbf{I}_2 & \Delta t \mathbf{I}_2 \\ \mathbf{0} & \mathbf{I}_2 \end{pmatrix}$. Prior $\hat{\mathbf{x}}_0 = (20, 0, 15, 0)$, $\mathbf{P}_0 = \text{diag}(4, 1, 4, 1)$.

A camera detection gives position only: $\mathbf{H}_c = [\mathbf{I}_2 \ \mathbf{0}]$, $\mathbf{R}_c = \text{diag}(0.25, 0.25)$ (sharp lateral from bounding box). A radar return arrives 5 ms later giving range-rate: $\mathbf{H}_r = [\mathbf{0} \ \hat{\mathbf{n}}^\top \ \mathbf{0}]$ (direction of relative motion), $R_r = 0.09$.

Camera update shrinks position covariance to about $\text{diag}(0.24, 0.20)$ (the 0.25-std camera beats the 2-std prior). Radar update then collapses the radial-velocity variance from 4 to about 0.1 via Eq. (10.14)—a $40\times$ improvement driven by one measurement. Fused ego-relative velocity is known to 0.3 ms^{-1} , good enough for adaptive cruise control. A camera-only tracker would have taken 5–10 frames to reach the same velocity precision via numerical differentiation of positions.

10.8.2 Extended and Unscented Variants

Real AV dynamics and measurement models are nonlinear. The bicycle model is nonlinear in heading; camera projection is nonlinear in depth; radar's bearing-range measurement is nonlinear in position. The EKF linearizes f and h about the current estimate; the UKF propagates deterministic sigma points through f and h without requiring Jacobians (see Chapter 2). Both have well-characterized failure modes: EKF is inconsistent in strong nonlinearities, UKF is expensive for high state dimension. Production AV stacks overwhelmingly use EKF for IMU-GNSS fusion and per-track object tracking because bicycle-model nonlinearities are mild and state dimension is low.

10.8.3 Information Filter and Factor Graphs

The information-form dual of the Kalman filter (see Chapter 2) becomes the natural representation when we move from a single-track filter to a pose-graph SLAM backend or a bundle adjustment. Each observation contributes additively to the Hessian; sparse solvers exploit the factor-graph topology to scale to millions of variables (Chapter 13).

10.9 Multi-Sensor Fusion Architectures

Having fixed the Kalman filter as the local fusion primitive, the question at system scale is *where in the pipeline* different modalities are combined. Three canonical architectures compete (Figure 10.3).

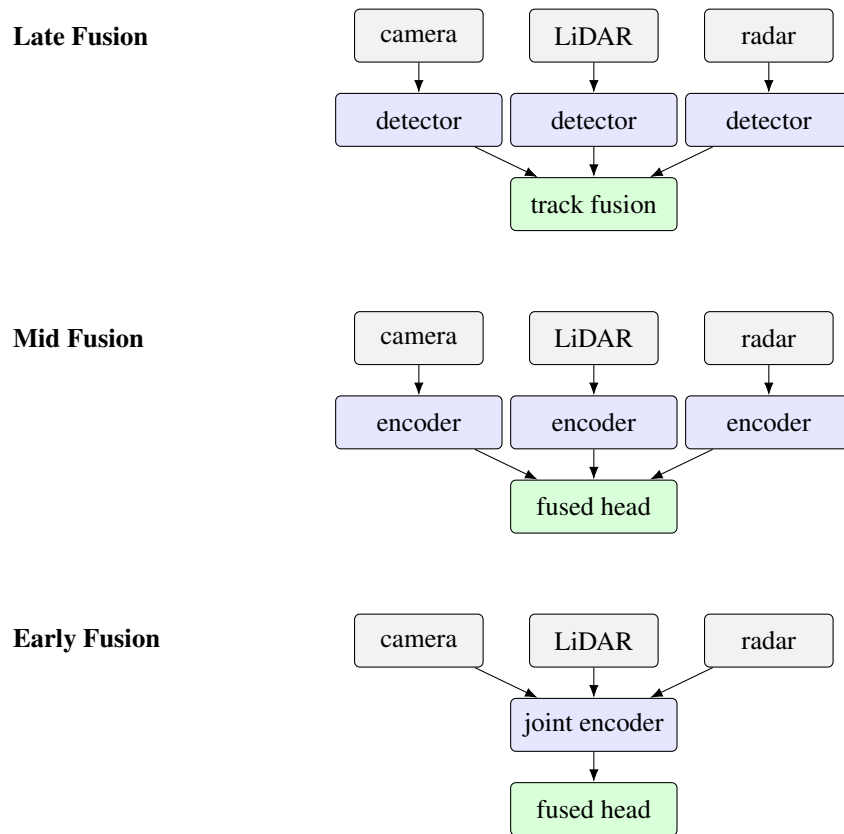


Figure 10.3: Three canonical fusion architectures. *Late fusion* runs independent detectors per sensor and fuses at the track level (robust, interpretable, throws away cross-modal evidence). *Mid fusion* combines learned features from each sensor before a fused task head (state of the art in 3D detection, depends on tight spatial-temporal registration). *Early fusion* concatenates raw or near-raw modalities into a single encoder (maximum joint information, maximum compute and calibration requirements).

Late fusion. Each sensor runs an independent perception pipeline; outputs (tracks, detections) are fused at a high level via a Kalman filter or graph-matching procedure. Pros: modular, testable, degrades gracefully when one sensor fails. Cons: discards cross-modal cues that the individual detectors could not recover by themselves (e.g., camera fine-grained class attached to LiDAR geometric extent).

Mid fusion. Each sensor produces learned features which are combined in a shared representation space (BEV grid, voxel grid, or transformer tokens) before a shared task head. Pros: leverages cross-modal cues; state of the art on published 3D-detection benchmarks. Cons: requires tight extrinsic and temporal registration; harder to reason about per-sensor failure.

Early fusion. Raw or near-raw sensor data is projected into a common frame and processed by a single joint encoder (e.g., painting LiDAR points with camera colors [69]). Pros: the learned representation can exploit any pattern present in the joint data. Cons: largest compute and registration requirements; monolithic in a way that makes fallback modes difficult.

Production stacks typically use *hybrid* architectures: late fusion for detections (so a vision-only detector still works when LiDAR fails), combined with mid-level fusion for 3D extent and velocity.

Example 10.22 (When late fusion throws away evidence). A pedestrian is partially occluded by a parked car. The camera detector gives class “pedestrian” with confidence 0.9 but cannot recover 3D extent due to occlusion. The LiDAR detector segments a point cluster but, lacking the color context, classifies it as “unknown” with confidence 0.55. In late fusion, the camera-track and LiDAR-track are associated at the tracker level; the pedestrian class survives and the LiDAR extent is attached. In pure single-modality pipelines, the LiDAR-only track would have 0.55 –confidence and possibly be filtered; the camera-only track would lack 3D extent. Mid fusion does even better: a joint feature encoder can upgrade the LiDAR cluster’s classification to “pedestrian” given camera context, producing a single high-confidence output.

10.10 Sensor Comparison and Selection

Table 10.1 summarizes the relative strengths of each sensor. The choice for a specific AV program depends on the target ODD, unit economics, and regulatory constraints:

Table 10.1: Relative sensor capabilities for AV perception.

	Camera	LiDAR	Radar	GNSS+IMU
Range	limited	good	good	N/A
Angular resolution	good	good	poor	N/A
Depth accuracy	poor (monocular) / fair (stereo)	excellent	good	N/A
Velocity	from differencing	from differencing	direct (Doppler)	fair
Semantics	excellent	limited	very limited	none
Weather	poor	fair	excellent	fair
Low-light	poor	unaffected	unaffected	N/A
Cost	low	high	moderate	low–moderate

A camera-only stack (Tesla) bets on scale and learned depth to overcome the geometric disadvantage, gaining unit-cost savings; a LiDAR-first stack (Waymo, Cruise, Zoox) pays for accurate 3D geometry at the cost of per-vehicle bill-of-materials. Camera-radar stacks (many L2 systems) skip LiDAR entirely and lean on radar for geometry. No choice is universally correct; each expresses a different bet on how the deep-learning and hardware cost curves evolve.

10.11 Worked Examples

This section consolidates a handful of multi-step numerical calculations that tie together the formulas above.

Example 10.23 (End-to-end: projecting a pedestrian from LiDAR into the camera). A LiDAR return at $\mathbf{p}^L = (22.0, 1.5, -0.5)$ m (LiDAR frame). LiDAR-to-body extrinsic: $\mathbf{R}_{BL} = \mathbf{I}$, $\mathbf{t}_{BL} = (1.5, 0, 1.8)$ m. Body-to-camera extrinsic: $\mathbf{R}_{CB}$ is a 90° roll swapping body axes to cam-

era convention ($x^C = -y^B$, $y^C = -z^B$, $z^C = x^B$), $\mathbf{t}_{CB} = (0, -0.3, 0)$. Camera intrinsics as in Example 10.1.

Step 1: body frame. $\mathbf{p}^B = \mathbf{p}^L + (1.5, 0, 1.8) = (23.5, 1.5, 1.3)$.

Step 2: camera frame. $\mathbf{p}^C = \mathbf{R}_{CB}\mathbf{p}^B + \mathbf{t}_{CB}$. $\mathbf{R}_{CB}\mathbf{p}^B = (-1.5, -1.3, 23.5)$. Plus translation: $(-1.5, -1.6, 23.5)$.

Step 3: pinhole projection. $u = 1200 \cdot (-1.5)/23.5 + 960 = -76.6 + 960 = 883$ px. $v = 1200 \cdot (-1.6)/23.5 + 540 = -81.7 + 540 = 458$ px.

The pedestrian LiDAR return lands at pixel (883, 458)—left of center, slightly above the horizon line. A camera classifier evaluated at that pixel can “paint” the LiDAR point with a class label, enabling mid-fusion. This arithmetic—frame composition plus pinhole projection—runs for hundreds of thousands of LiDAR points per frame and is the hottest loop in many perception stacks.

Example 10.24 (Full Kalman-filter step for a 1D CV tracker). Track a leading vehicle with state (p, v) , $\Delta t = 0.1$ s. Dynamics $\mathbf{F} = \begin{pmatrix} 1 & 0.1 \\ 0 & 1 \end{pmatrix}$, $\mathbf{Q} = \text{diag}(0.01, 0.1)$. Prior: $\hat{\mathbf{x}}_0 = (50.0, 20.0)$, $\mathbf{P}_0 = \text{diag}(1, 4)$. Measurement: $y = 50.2$, $\mathbf{H} = (1 \ 0)$, $R = 0.09$.

Predict. $\hat{\mathbf{x}}_1^- = (52.0, 20.0)$, $\mathbf{P}_1^- = \mathbf{F}\mathbf{P}_0\mathbf{F}^\top + \mathbf{Q}$. $\mathbf{F}\mathbf{P}_0\mathbf{F}^\top$ element (1, 1) = $1 + 0.01 \cdot 4 = 1.04$; (1, 2) = (2, 1) = $0.1 \cdot 4 = 0.4$; (2, 2) = 4. Plus $\mathbf{Q}$: $\mathbf{P}_1^- = \begin{pmatrix} 1.05 & 0.4 \\ 0.4 & 4.1 \end{pmatrix}$.

Update. Innovation $v = 50.2 - 52.0 = -1.8$; innovation variance $S = 1.05 + 0.09 = 1.14$; gain $\mathbf{K} = (1.05, 0.4)^\top / 1.14 \approx (0.921, 0.351)^\top$. Posterior mean: $\hat{\mathbf{x}}_1 = (52.0 + 0.921 \cdot (-1.8), 20 + 0.351 \cdot (-1.8)) = (50.34, 19.37)$. Covariance: $\mathbf{P}_1 = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}_1^-$; (1, 1) entry: $(1 - 0.921) \cdot 1.05 \approx 0.083$; (2, 2): $4.1 - 0.351 \cdot 0.4 \approx 3.96$. Position variance collapsed $12 \times$ (prior was 1.04); velocity only slightly tightened from its prior. This asymmetric tightening is Kalman’s core magic: one position measurement improved our velocity estimate too, via the correlation induced by the dynamics.

Example 10.25 (FMCW radar range-Doppler for a closing target). 77-GHz radar, $B = 1$ GHz, $T_c = 40 \mu\text{s}$, $N_c = 128$. Detected: $f_b = 500$ kHz, $f_d = -1.5$ kHz (negative = closing).

Range. $S = B/T_c = 2.5 \times 10^{13} \text{ Hz s}^{-1}$. $r = c f_b / (2S) = (3 \times 10^8 \cdot 5 \times 10^5) / (5 \times 10^{13}) = 3.0$ m.

Velocity. $\lambda = c/f_c = 3.9 \times 10^{-3}$ m. $v = \lambda f_d / 2 = 3.9 \times 10^{-3} \cdot (-1500) / 2 = -2.9 \text{ m s}^{-1}$.

So the target is 3.0 m ahead, closing at 2.9 m s^{-1} —a tailgater. Radar’s simultaneous resolution of range and velocity in a single fast-scan makes it an order-of-magnitude faster source of closing-rate information than camera- or LiDAR-only tracking (which must difference positions over multiple frames).

10.12 Industrial Deployment

Production AV sensor suites combine cameras, radar, and (often) LiDAR with GNSS and IMU. Waymo’s fifth-generation sensor suite includes 29 cameras and multiple LiDARs; Cruise’s Origin platform carried a roof-mounted sensor stack of LiDAR, radar, and cameras; Tesla famously moved away from radar in 2021 and from ultrasonic sensors in 2022, pursuing a vision-only architecture under the “Tesla Vision” branding. Mobileye’s EyeQ platform supports camera-primary and camera-radar configurations; the company’s higher-end “Mobileye Chauffeur” offering integrates LiDAR. Continental, Bosch, Valeo, and Aptiv (now Motional in joint venture with Hyundai for AV systems) are among the major Tier-1 suppliers integrating these sensors into production ADAS and AD vehicles; their automotive-radar product lines are widely used in adaptive cruise control and forward collision warning across the global automotive industry.

10.13 Chapter Summary

The AV sensor suite combines cameras (semantic richness), LiDAR (geometric precision), radar (weather robustness and Doppler velocity), GNSS (global reference), IMU (high-rate motion), and ultrasonics (close-in). Each has characteristic noise models and failure modes; combining them requires accurate extrinsic calibration and tight time synchronization. The Kalman filter is the local fusion primitive; at system scale, late, mid, and early fusion architectures trade modularity against joint-evidence exploitation. Sensor selection is a program-level bet on cost, ODD, and the shape of future hardware curves. The sensor suite described here is the upstream boundary of every algorithm in the remaining chapters.

10.14 Exercises

1. Given camera intrinsics $f_x = f_y = 1000$, $c_x = 640$, $c_y = 360$ and a 3D point at $(-3, 2, 15)$ m in the camera frame, compute the pixel projection.
2. A rolling-shutter camera has 30 ms readout and 1080 rows. At ego speed 15 ms^{-1} , compute the horizontal skew of a perfectly vertical telephone pole at 15 m range.
3. A 32-beam LiDAR covers 25° vertical FoV. What is its vertical angular resolution, and what ground gap appears between beams at 75 m range on a flat road?
4. An FMCW radar has $B = 750 \text{ MHz}$, $T_c = 30 \mu\text{s}$, $N_c = 256$. Compute range resolution, maximum unambiguous range (assuming ADC limits), velocity resolution, and maximum unambiguous velocity. Carrier is 77 GHz.
5. A GNSS receiver loses RTK fix (drops to “float” with 30 cm reported accuracy). The EKF is tuned for 3 cm GNSS noise. Describe what happens if the tuning is not updated, and propose two fixes.
6. Compute IMU preintegration position drift for $b_a = 0.02 \text{ ms}^{-2}$ over 30 seconds of tunnel with no external correction.

7. Work through one EKF predict+update step for a 2D constant-velocity tracker with state (x, y, v_x, v_y) , $\Delta t = 0.1$ s, prior $\mathbf{0}$ -mean with covariance $\text{diag}(1, 1, 4, 4)$, process noise $\mathbf{Q} = \text{diag}(0.01, 0.01, 0.1, 0.1)$, and a position measurement $(1.2, 0.8)$ with $\mathbf{R} = \text{diag}(0.04, 0.04)$.
8. Explain why late fusion is more robust to single-sensor failures than early fusion, using the sensor suite of Figure 10.1 as a reference.
9. Sketch a time-synchronization plan for a stack with 4 cameras, 1 LiDAR, 3 radars, and 1 IMU. Identify which sensors need hardware triggers and which can rely on PTP alone.
10. For the fusion architectures of Figure 10.3, identify which requires the tightest extrinsic calibration and explain why.

Chapter 11

Object Detection and Multi-Object Tracking

11.1 Introduction

The previous chapters gave us raw sensor data (Chapters 4 through 6) and the fused representation that combines modalities (Chapter 10). What we do not yet have is a structured list of the *things* in the scene the ego needs to reason about: a red sedan three lanes to the left merging toward us at 28 ms^{-1} , a pedestrian at 18 m ahead currently stationary, a cyclist approaching from the right at the intersection. That list—with identity, position, extent, orientation, velocity, and class per agent—is what detection and tracking produce. It is the representation the planner actually consumes; everything upstream is in service of producing it cleanly.

Three observations motivate the chapter’s content.

First, detection and tracking are distinct problems with coupled solutions. Detection answers *what is in the scene at this instant*; tracking answers *which detection at time t corresponds to which at $t + 1$* . A good tracker tolerates a noisy detector because temporal consistency averages out errors; a great detector reduces the work tracking has to do. The best modern systems blur the boundary, jointly optimizing both.

Second, the work is fundamentally *3D*. A detector that outputs accurate 2D bounding boxes in image space is worth far less to the planner than one that outputs 3D oriented boxes in the ego frame, because the planner reasons about collisions in metric space. Accurate 3D understanding from 2D images alone is hard; LiDAR-based 3D detection is the most reliable path, with multi-modal fusion closing the gap where LiDAR is unavailable.

Third, the long tail dominates safety. A detector that hits 99 % AP on benchmark cars may still fail on the rare-and-hazardous—a mattress in the road, a person exiting a parked car into traffic, a scooter behind a truck. The production engineering task is as much about *recall on the tail* as about *accuracy on the mean*, and the loss functions, architectural choices, and data pipelines developed below reflect

that.

This chapter is organized by the pipeline order. We begin with 2D detection as the foundation, because the architectural patterns (two-stage, single-stage, anchor-free, transformer-based) set up everything that follows. We then move to 3D detection from LiDAR, from cameras, and from their fusion. Multi-object tracking is next, first in the classical “tracking by detection” form and then in joint detection-tracking architectures. We close with specific hard cases—long-range, small-object, and adverse-weather detection—and with the open-world and continual-learning challenges that real deployments force. Every formula is accompanied by a worked numerical example; three architecture diagrams visualize the key pipelines.

11.2 Two-Dimensional Object Detection

2D detection is where we start because it is the most heavily studied case, its architectural patterns transfer directly to 3D, and many perception sub-tasks (traffic-sign recognition, vehicle identification, pedestrian detection) still benefit from a strong 2D front-end before 3D lifting. The goal is to output, for each input image, a list of $(b_1, b_2, b_3, b_4, c, p)$ tuples: the box corners, class index, and class probability. The variations in how networks produce this list differ primarily in how region proposals are generated and how the final classification head is structured.

11.2.1 Problem Statement

Given an image, output a set of bounding boxes each with a class label and confidence. The canonical loss for box regression is smooth- ℓ_1 :

$$\text{smooth}_{\ell_1}(x) = \begin{cases} 0.5x^2 & |x| < 1, \\ |x| - 0.5 & \text{otherwise,} \end{cases} \quad (11.1)$$

applied per-coordinate on the difference between predicted and target box. Smooth- ℓ_1 is quadratic near zero (for fine-tuning) and linear far from zero (robust to outlier targets).

Example 11.1 (Smooth- ℓ_1 vs ℓ_2 on an outlier). Four per-coordinate residuals: $(0.1, 0.2, -0.3, 5.0)$. ℓ_2 sum: $0.01 + 0.04 + 0.09 + 25 = 25.14$ —dominated by the outlier. Smooth- ℓ_1 sum: $0.005 + 0.02 + 0.045 + 4.5 = 4.57$ —outlier contributes linearly, not quadratically. The gradient from the outlier in smooth- ℓ_1 is constant (magnitude 1), not linearly amplified. Detection training typically has many such outliers (poorly-matched anchors, labeling noise), so smooth- ℓ_1 prevents a few bad pairs from distorting the whole gradient step.

For classification, cross-entropy is standard; for heavily imbalanced classes, focal loss [45]:

$$\mathcal{L}_{\text{focal}} = -\alpha(1 - p_t)^\gamma \log p_t, \quad (11.2)$$

where p_t is the model’s probability for the true class, α is a class-weight scalar, and γ (typically 2)

down-weights easy examples.

Example 11.2 (Focal-loss down-weighting). Easy example: $p_t = 0.99$. Cross-entropy: $-\log 0.99 = 0.01$. Focal ($\gamma = 2$): $-(0.01)^2 \cdot 0.01 = 1 \times 10^{-6}$ —down-weighted 10,000 $\times$. Hard example: $p_t = 0.3$. Cross-entropy: 1.2. Focal: $(0.7)^2 \cdot 1.2 = 0.588$ —down-weighted 2 $\times$. The effect: easy correct examples contribute almost nothing to the gradient; hard cases drive learning. This is why focal loss is standard for pedestrian detection on long-tailed data.

11.2.2 Two-Stage Detectors

Faster R-CNN [56] and its successors (Mask R-CNN, Cascade R-CNN) proceed in two stages: a *Region Proposal Network* (RPN) emits class-agnostic candidate boxes, and a second network refines the boxes and classifies them. Figure 11.1 contrasts this with single-stage pipelines. The strength of two-stage is accuracy: the second stage has a hard set of candidates and can afford per-candidate classification. The weakness is latency: the second stage scales with the number of proposals.

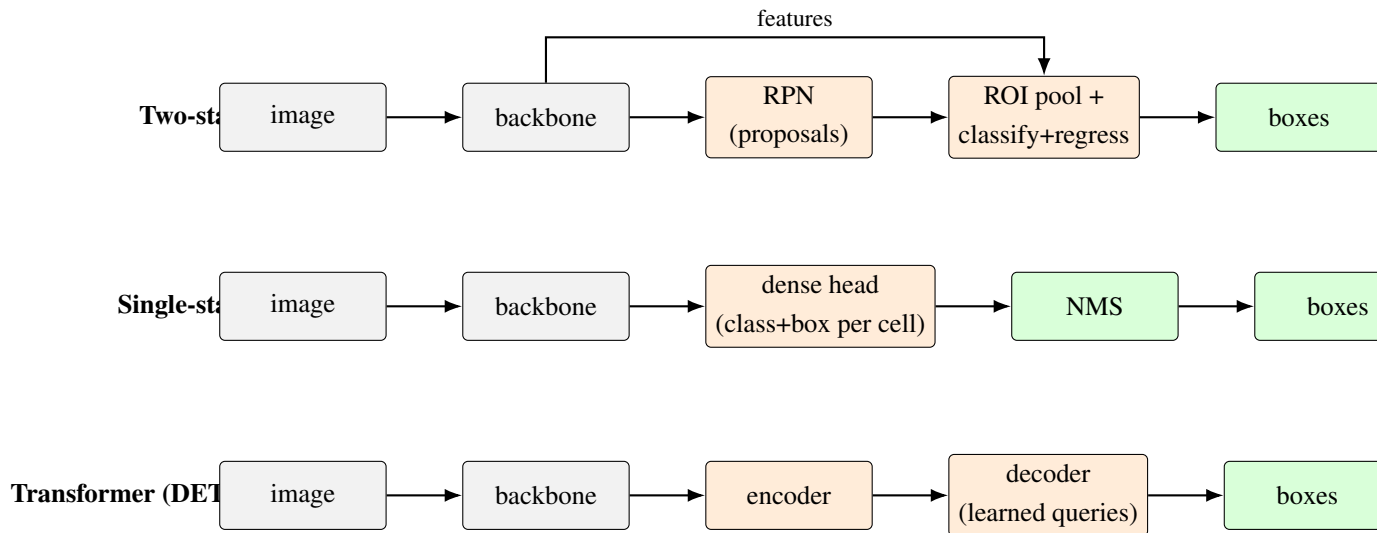


Figure 11.1: Three canonical 2D-detection pipelines. *Two-stage* (top): a Region Proposal Network proposes candidate boxes; a second stage crops features via ROI pooling and refines + classifies each. High accuracy, higher latency. *Single-stage* (middle): a dense prediction head emits box+class at every feature-map cell; NMS prunes duplicates. Lower latency, slightly lower accuracy on small objects. *Transformer-based* (bottom): a fixed set of learned object queries attends to encoder features; outputs are a direct set, so no NMS is needed. Latency comparable to single-stage; training requires more data.

Example 11.3 (Throughput of a two-stage detector). Backbone forward pass: 18 ms. RPN: 3 ms. Second stage at 1000 proposals per image: 12 ms. Total: 33 ms, i.e., ~ 30 Hz. Cutting proposals to 300 brings second stage to 4 ms, total 25 ms, ~ 40 Hz—at modest recall cost on small/distant objects. Production systems sweep this tradeoff per deployment target.

11.2.3 Single-Stage Detectors

YOLO [55] and its descendants (YOLOv3, YOLOv5, YOLOX, YOLOv7/8/10), SSD, and RetinaNet emit class and box predictions densely across the feature map, with no separate proposal stage. They are the workhorse of real-time detection: at 30 Hz to 60 Hz they comfortably outpace two-stage, with a small accuracy gap that modern variants have largely closed. RetinaNet’s focal-loss innovation (Eq. 11.2) was motivated precisely by the class imbalance that single-stage detectors face when they classify many background cells per foreground.

11.2.4 Anchor-Free Detectors

CenterNet [73], FCOS, and their descendants eliminate anchor boxes altogether. They predict a center-heatmap with peaks at object centers and regress per-pixel the box extent and offset. Pros: no anchor tuning, cleaner training, elegant generalization to 3D (CenterPoint). Cons: for very elongated objects (long trucks) the regression range is wide and single-cell prediction can be noisy; keypoint-based variants compensate.

Example 11.4 (Center-heatmap loss). For an image with one object of center (c_x, c_y) , the target heatmap Y is a 2D Gaussian bump of amplitude 1 at (c_x, c_y) with standard deviation proportional to object size. The loss is focal on the heatmap:

$$\mathcal{L}_{\text{ctr}} = -\frac{1}{N} \sum_{x,y} \begin{cases} (1 - \hat{Y})^\alpha \log \hat{Y} & Y = 1, \\ (1 - Y)^\beta \hat{Y}^\alpha \log(1 - \hat{Y}) & \text{otherwise.} \end{cases} \quad (11.3)$$

With $\alpha = 2, \beta = 4$, a predicted peak of $\hat{Y} = 0.8$ at the true center contributes $-(0.2)^2 \log 0.8 \approx 0.009$; a background cell with $\hat{Y} = 0.1$ contributes ≈ 0.001 . The loss focuses on near-center cells where the model is still uncertain.

11.2.5 Transformer-Based Detection: DETR

DETR [10] replaces the anchor-and-NMS machinery with a transformer decoder applied to a set of N learned object queries. The decoder outputs N box-and-class predictions; Hungarian matching pairs predictions to ground truth during training. Pros: no hand-designed priors (anchors, NMS), cleaner set-prediction formulation, natural extension to set-based objectives. Cons: slow to train (300+ epochs for original DETR; Deformable-DETR and DINO reduce this), and the set-size N is fixed at inference.

Example 11.5 (Hungarian matching between predictions and targets). Five predictions and three ground-truth boxes. The 5×3 cost matrix has entries $C_{ij} = \lambda_{\text{box}} \mathcal{L}_{\text{box}}(i, j) + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}(i, j)$. Solve the assignment as a bipartite matching (Hungarian algorithm, < 1 ms for these sizes): the three ground truth boxes each match the single lowest-cost prediction; unmatched predictions are supervised as “no object.” Contrast with anchor matching, which assigns every anchor to a

ground-truth box by IoU threshold—a one-to-many matching that forces the network to predict multiple redundant outputs per object that NMS must later suppress.

11.3 Non-Maximum Suppression

Most detectors emit multiple candidate boxes per object. NMS prunes duplicates by keeping the highest-confidence box and suppressing any other box whose IoU with it exceeds a threshold. Soft-NMS [5] decays the confidence of overlapping boxes rather than deleting them, preserving recall on adjacent objects.

Example 11.6 (NMS and soft-NMS on overlapping pedestrians). Two closely-spaced pedestrians generate predictions $A : p = 0.92, \text{IoU}(A, B) = 0.55$ and $B : p = 0.85$. Classical NMS with threshold 0.5 keeps A and deletes B —a *miss* on the second pedestrian. Soft-NMS decays B 's confidence by factor $(1 - \text{IoU})^\gamma$; with $\gamma = 1$, B 's confidence becomes $0.85 \cdot 0.45 = 0.38$, below a 0.5 threshold but above a 0.3 threshold. In dense-pedestrian scenes (crosswalks, loading zones), soft-NMS typically improves recall by 5–10 points without harming precision.

Rotated-BEV NMS is the 3D-detection generalization (Chapter 5 briefly discussed it): IoU is computed between oriented BEV rectangles, using either analytic or Monte-Carlo intersection area.

11.4 Three-Dimensional Object Detection

The planner needs 3D boxes in the ego frame. Three architectural families dominate: LiDAR-only (precise, expensive sensor), camera-only (cheap, less precise), and fusion (the best of both at some compute cost). Figure 11.2 visualizes these three.

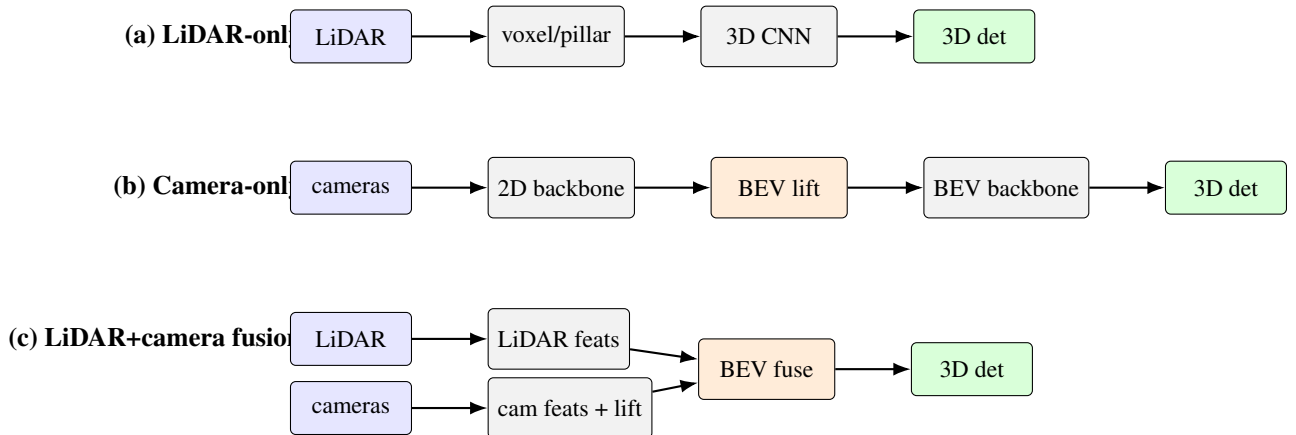


Figure 11.2: Three architectures for 3D detection. (a) *LiDAR-only* voxelizes or pillarizes the point cloud, runs a sparse-conv 3D CNN, and emits 3D boxes directly (PointPillars, CenterPoint, VoxelNet). (b) *Camera-only* extracts 2D features, lifts them to a BEV grid (Lift-Splat-Shoot, BEVFormer), and runs a BEV head. (c) *LiDAR+camera fusion* projects both modalities into BEV and runs a joint head (BEVFusion, TransFusion, DeepFusion). Fusion recovers most of the gap to LiDAR while adding camera semantics.

11.4.1 LiDAR-Based 3D Detection

Architectures developed in Chapter 5—PointPillars, CenterPoint, VoxelNet—produce 3D bounding boxes $(x, y, z, l, w, h, \theta)$ in the ego frame. The standard benchmark pattern is: pretrain on a large-scale set (Waymo Open or nuScenes), fine-tune to target domain, evaluate at several IoU thresholds.

Example 11.7 (3D IoU for two oriented boxes). Box A: $(x, y, z, l, w, h, \theta) = (0, 0, 0, 4.5, 1.8, 1.6, 0)$. Box B: same geometry, shifted to $(0.5, 0.1, 0, 4.5, 1.8, 1.6, 0.05)$. BEV intersection area by analytic polygon clipping: $\approx 7.6\text{m}^2$ (BEV area of each box: 8.1m^2). Height overlap: full (1.6 m). 3D intersection: $7.6 \cdot 1.6 = 12.16\text{m}^3$. 3D union: $2 \cdot (8.1 \cdot 1.6) - 12.16 = 13.76\text{m}^3$. 3D IoU: $12.16/13.76 \approx 0.88$. Above threshold 0.7: counts as a match. Drop box B's center by 1.5 m: IoU drops below 0.5, counted as a miss at the 0.7 threshold.

11.4.2 Camera-Based 3D Detection

Camera-only 3D detection relies on learned depth cues. Key architectures:

- **FCOS3D, PGD**: directly regress 3D box attributes from single-image features.
- **BEVFormer, BEVDet, Lift-Splat**: transform multi-camera features to a unified BEV grid, then use a BEV detector head (Chapter 4).
- **PETR, StreamPETR**: 3D position encoding that lets a transformer decoder consume multi-view images directly, without an explicit BEV grid.

Example 11.8 (Depth uncertainty dominates camera 3D detection). A vehicle detected with 2D pixel precision 1 px at focal length 1200 px and range 40 m. Lateral precision: $40 \cdot 1/1200 \approx 3.3\text{cm}$ —excellent. Depth precision from stereo baseline 0.6 m: $\sigma_z \approx Z^2/(f \cdot b) \cdot \sigma_d = 1600/(1200 \cdot 0.6) \approx 2.2\text{m}$ per pixel of disparity error. Monocular is worse by the ratio of the depth-network's effective baseline, typically $5\times$ – $10\times$. This is why camera-only 3D detectors trail LiDAR benchmarks by 10–20 mAP points at long range, and why production camera-only stacks lean heavily on temporal fusion and motion parallax.

11.4.3 LiDAR–Camera Fusion

Fusion architectures combine the geometric precision of LiDAR with the semantic richness of cameras. Three patterns dominate:

- **Feature-level fusion** (BEVFusion, DeepFusion): project camera features into a BEV aligned with LiDAR BEV; concatenate and run a joint head.
- **Point-level painting** (PointPainting [69]): paint each LiDAR point with camera semantic features; run a standard LiDAR detector on the painted cloud.

- **Query-based fusion** (TransFusion): queries attend first to LiDAR BEV, then to camera features, producing final 3D boxes.

Production-scale fusion typically gains 3–8 nuScenes NDS points over LiDAR-only at a 20–40 % compute cost increase.

11.4.4 3D Evaluation Metrics

Waymo Open and nuScenes dominate the public benchmarks. Waymo uses 3D IoU at thresholds 0.5/0.7 for mAP; nuScenes uses a center-distance metric (more forgiving for orientation errors) plus per-class errors in translation, scale, orientation, velocity, and attributes, combined into the Nuscenes Detection Score (NDS).

Example 11.9 (NDS composite calculation). A detector’s per-class mAP: 0.60. Per-axis mean errors: $T = 0.3$ m (translation), $S = 0.2$ (1-IoU on size), $O = 0.1$ rad (orientation), $V = 0.4$ m/s, $A = 0.1$ (attribute). Each error is scaled to a 0-1 score (smaller = better) via vendor formulas; NDS combines as a weighted average. A typical modern submission: NDS 0.70. Breaking a single sub-metric (e.g., velocity estimation off by 2x) can cost 5 NDS points—sometimes the difference between 1st and 10th on the leaderboard.

11.5 Multi-Object Tracking

Tracking converts a stream of per-frame detections into temporally-consistent tracks with identity, velocity, and history. The planner depends on this consistency: without stable IDs across frames, a “pedestrian” at frame t and a “pedestrian” at frame $t + 1$ would be treated as separate entities, and the prediction module could not reason about their trajectories.

11.5.1 Tracking by Detection

The dominant paradigm. Per-frame detections are associated to existing tracks via an assignment algorithm (Hungarian is standard), gated by predicted position and motion consistency. Successfully associated detections update their track’s Kalman state; unassociated detections spawn new tracks; unmatched tracks decay. Figure 11.3 shows the dataflow.

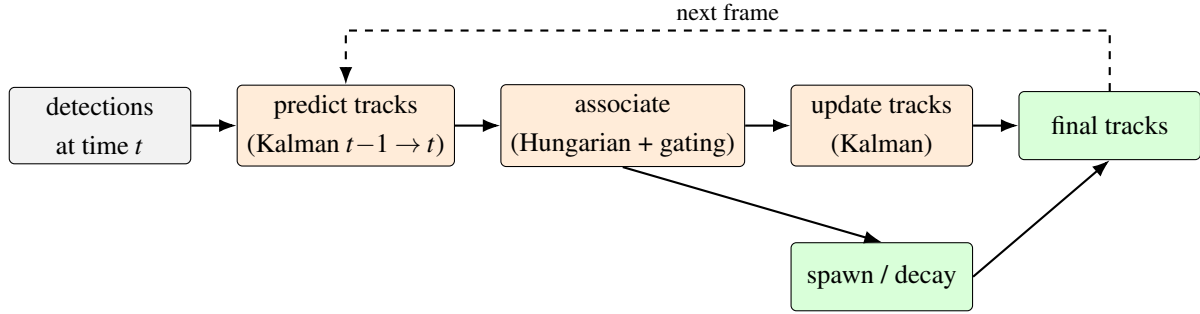


Figure 11.3: Tracking-by-detection data flow. Each frame’s detections are matched against propagated track predictions via a gated Hungarian assignment. Matched pairs update their Kalman-filtered track state; unmatched detections spawn new tracks; unmatched tracks decay after a fixed time-out. The resulting track list is fed to downstream modules and propagated forward to the next frame.

SORT [3] and DeepSORT [71] are representative tracking-by-detection algorithms. SORT uses a constant-velocity Kalman filter and IoU gating for association; DeepSORT adds a learned appearance embedding that provides cross-frame identity even under long occlusions. ByteTrack and OC-SORT build on the idea with higher-accuracy association.

11.5.2 State Model for Driving

A typical track state is 9-dimensional: 3D center, 3D velocity, 3D acceleration (sometimes suppressed to 6-D velocity-only). For wheeled vehicles, the bicycle model is often used instead, parameterizing state by heading, speed, yaw-rate, and acceleration with a kinematically consistent update.

Example 11.10 (Kalman predict for a constant-velocity vehicle track). State $\mathbf{x} = (x, y, v_x, v_y)$, covariance $\mathbf{P}$. Dynamics matrix with $\Delta t = 0.05$ s: $\mathbf{F} = \begin{pmatrix} \mathbf{I}_2 & 0.05\mathbf{I}_2 \\ 0 & \mathbf{I}_2 \end{pmatrix}$. Previous state: $\mathbf{x} = (50, 0, 25, 0)$, $\mathbf{P} = \text{diag}(0.04, 0.04, 0.09, 0.09)$. Predict: $\hat{\mathbf{x}}^- = (50 + 1.25, 0, 25, 0) = (51.25, 0, 25, 0)$. $\mathbf{P}^- = \mathbf{F}\mathbf{P}\mathbf{F}^\top + \mathbf{Q}$ with $\mathbf{Q} = \text{diag}(0.001, 0.001, 0.01, 0.01)$. Element (1,1): $0.04 + 0.05^2 \cdot 0.09 + 0.001 = 0.0413$. Element (1,3): $0.05 \cdot 0.09 = 0.0045$. Element (3,3): $0.09 + 0.01 = 0.1$. The position-velocity cross-covariance the predict induces is what lets a position measurement tighten the velocity estimate at the update step.

Example 11.11 (Association cost matrix and Hungarian solve). Three live tracks (T_1, T_2, T_3) predict centers $(51, 0), (20, -4), (80, 2)$. Three new detections D_A, D_B, D_C arrive at $(51.3, 0.1), (19.8, -3.9), (82, 1.5)$. Cost matrix entries $C_{ij} = \|T_i^{\text{pred}} - D_j\|^2$

$$C = \begin{pmatrix} 0.10 & 978 & 845 \\ 992 & 0.05 & 3872 \\ 842 & 3638 & 4.25 \end{pmatrix}.$$

Hungarian solution: $T_1 \rightarrow D_A, T_2 \rightarrow D_B, T_3 \rightarrow D_C$ with total cost $0.10 + 0.05 + 4.25 = 4.4$. Gating via a χ^2 threshold at the covariance-weighted distance rejects all off-diagonal matches as implausible before the solve, producing a near-diagonal assignment. Production trackers often add an appearance cost term (learned embedding distance) to disambiguate physically close but

visually distinct objects.

11.5.3 Joint Detection and Tracking

Recent architectures (CenterTrack, MOTR, TrackFormer) jointly perform per-frame detection and cross-frame association by propagating features temporally inside the network. This avoids the hand-designed association stage and its tuning, and typically improves tracking metrics by 5–15 HOTA points over tracking-by-detection on benchmarks. The tradeoff: the combined architecture is less modular and harder to ablate when a specific failure mode needs fixing.

11.5.4 Tracking Evaluation

The standard tracking metrics combine detection quality with identity consistency:

MOTA (multi-object tracking accuracy).

Defined as $1 - (\text{FN} + \text{FP} + \text{IDSW})/\text{GT}$. Combines false negatives, false positives, and identity switches into a single score.

IDF1.

Identity-level F1, weighted by how often the same physical object keeps the same predicted ID across a trajectory.

HOTA [48].

Geometric mean of detection, association, and localization accuracy. Has become the preferred benchmark metric for being less easily gamed than MOTA.

Example 11.12 (MOTA breakdown for a short sequence). Evaluation over a short tracking sequence.

Ground truth and errors.

- Ground-truth objects across all frames: 100
- False positives (FP): 5
- False negatives (FN): 8
- Identity switches (IDSW): 3

MOTA computation.

$$\begin{aligned}
 \text{MOTA} &= 1 - \frac{\text{FN} + \text{FP} + \text{IDSW}}{\text{GT}} \\
 &= 1 - \frac{8 + 5 + 3}{100} \\
 &= 1 - 0.16 \\
 &= 0.84.
 \end{aligned}$$

Engineering implications.

A 10-point MOTA improvement could come from fewer false negatives (better detector) or fewer ID switches (better association / appearance embedding). Which fix is right depends on which error dominates; careful MOTA component analysis guides the engineering attention.

11.6 Hard Cases: Long-Range, Small-Object, and Adverse-Weather Detection

Production AV requires uniform reliability across operating conditions. A detector strong on the benchmark mean but weak on edge cases produces brittle behavior. This section collects the three hardest cases and the techniques that address each.

11.6.1 Small and Distant Objects

A pedestrian at 80 m spans a handful of pixels in the image and a handful of points in the LiDAR cloud. Standard detectors underperform; specialized techniques include:

- **Multi-scale feature pyramids (FPN):** explicit high-resolution features for small objects.
- **Adaptive anchor / center loss:** scale-aware loss weighting so small-object errors count proportionally.
- **Two-pass detection:** first pass at low resolution, crops around suspicious regions refined at high resolution.

Example 11.13 (Small-object recall gain from two-pass detection). Full-resolution detector on 80 m pedestrians: 45 % recall. Two-pass system: first pass at quarter resolution generates “maybe pedestrian” heatmap peaks; full-resolution refinement on 128×256 crops around each peak. Recall rises to 72 % on the same targets at 10 % added compute. This is the standard recipe for long-range pedestrian detection in camera-primary stacks.

11.6.2 Adverse Weather

Rain, snow, and fog degrade camera and LiDAR signal but leave radar largely unaffected. Detection in adverse weather relies on either (i) training on weather-diverse data—typically augmented with synthetic rain/fog—or (ii) fusion stacks that fall back on radar when other modalities degrade.

Example 11.14 (Synthetic rain augmentation). A clear-weather dataset is augmented with rendered rain streaks at training time: random streak orientations, lengths, and intensities. On a held-out rainy-weather test set, a detector trained without augmentation achieves AP 0.55; with synthetic rain augmentation, AP rises to 0.68. The augmentation cannot close the full gap to a detector fine-tuned on real rainy data (AP 0.76), but is a meaningful improvement at zero labeling cost.

11.7 Open-World and Continual Learning

Closed-set detection assumes the taxonomy is known at training time. Deployed AVs encounter objects they were not trained on: a fallen tree, a lost mattress, an exotic scooter. Open-world detection labels these “unknown” rather than forcing them into a known class or missing them entirely. Continual learning adds new classes over time without forgetting existing ones; the challenge is catastrophic forgetting when the new-class data dominates training.

Production programs handle this through a *data flywheel*: fleet vehicles flag low-confidence detections and upload the relevant clips; human labelers add them to the training set; retrained models roll back out to the fleet. The end-to-end loop time is typically days to weeks for major releases. A robust open-world detector, combined with an efficient mining pipeline, is the main mechanism by which long-tail coverage improves over a program’s lifetime.

11.8 Worked Examples

Example 11.15 (End-to-end 3D detection + tracking of a scene). Input: LiDAR cloud with 140 000 points; 4 cameras at 1920×1080 ; radar with 1200 detections.

Step 1: detection. BEVFusion-style pipeline produces 38 raw 3D-box candidates in the ego frame with associated confidences.

Step 2: NMS. Rotated-BEV NMS at threshold 0.5 reduces to 24 final detections.

Step 3: prediction against tracks. 21 existing tracks propagate their Kalman states forward by $\Delta t = 0.1$ s.

Step 4: association. 24×21 cost matrix built from position + velocity + class + appearance features. Hungarian solve matches 19 detections to existing tracks, leaves 5 unmatched detections and 2 unmatched tracks.

Step 5: update. 19 matched detections run Kalman-update against their track; 5 unmatched spawn new tentative tracks; 2 unmatched tracks increment their miss counter (track is terminated after 5 consecutive misses).

Final output: 21 confirmed tracks + 5 tentative, each with position, velocity, class, confidence, and history. Total latency from sensor-frame-timestamp to track-output: 85 ms.

Example 11.16 (Failure cascade from a missed detection). At $t = 0$, a pedestrian in a crosswalk is missed (occluded behind a parked truck). Tracker has no prior track. At $t = 0.1$ s, pedestrian re-emerges; detector produces a new detection. Tracker spawns a new *tentative* track. Many trackers require ≥ 3 consecutive frames before promoting a tentative track to “confirmed”—the pedestrian does not enter the planner’s agent list until $t = 0.3$ s. Ego at 15 ms^{-1} traveled 4.5 m before even registering the pedestrian. This is why minimum-confirmation thresholds are tuned aggressively for safety-critical classes: pedestrians and cyclists typically promote after 1–2

frames, not 3, accepting a higher false-track rate to reduce latency.

Example 11.17 (Class imbalance in training mixed classes). A dataset has 80000 car-class annotations and 4000 pedestrian-class annotations (20:1 imbalance). Uniform sampling trains the detector with $\sim 95\%$ car exposure; per-class AP at convergence: cars 0.82, pedestrians 0.44. Class-balanced sampling (equal pedestrian and car batches) gives pedestrian AP 0.62 at minor cost to car AP (0.80). Focal loss without balanced sampling gives pedestrian AP 0.56. Both together: pedestrian AP 0.66. Production training combines both, and further adds copy-paste augmentation for the rarest classes.

11.9 Industrial Deployment

The detection and tracking architectures developed in this chapter underpin production AV systems and have substantial open-source representation. The COCO dataset (Microsoft Research) and KITTI (Karlsruhe Institute of Technology, with Toyota Technological Institute at Chicago) are widely used benchmarks. The CenterPoint architecture [73] from researchers at UT Austin has been widely adopted for LiDAR-based 3D detection; 3D detection challenges hosted by nuScenes, Waymo Open Dataset, and Argoverse have driven public progress. Tracking-by-detection architectures along the lines of SORT and DeepSORT [71] are widely adopted in production stacks, with Waymo, Cruise, and others publishing variations and extensions in the academic literature. NMS-free detectors such as DETR (Facebook AI Research) and its derivatives are increasingly used in production where end-to-end set prediction is preferred over heuristic post-processing.

11.10 Chapter Summary

Detection turns sensor data into per-frame structured outputs; tracking turns per-frame detections into temporally coherent agent tracks. 2D detection evolved through two-stage, single-stage, anchor-free, and transformer-based designs, each addressing specific speed-accuracy tradeoffs. 3D detection draws from the same toolbox applied to LiDAR point clouds, camera BEV features, or their fusion. Tracking is dominated by tracking-by-detection with Kalman filtering and Hungarian association, with joint-learning variants gaining ground. Hard cases—small objects, adverse weather, open-world—drive production engineering effort; the fleet data flywheel is the main mechanism by which long-tail performance improves over a program’s lifetime. The outputs of this chapter (tracked 3D agents with velocity, class, and identity) feed the prediction, planning, and control layers that follow.

11.11 Exercises

1. Compute the smooth- ℓ_1 loss for residuals $(0.8, 1.2, -0.5, 2.3)$ and compare to ℓ_2 .
2. A focal-loss detector has $\gamma = 3$, $\alpha = 0.25$. Compute the loss for $p_t = 0.95, 0.5, 0.1$ and discuss the gradient contributions.

3. Sketch why NMS fails on two pedestrians 0.5 m apart captured by a forward-facing camera at 10 m range. Propose a fix.
4. Derive the 3D-IoU formula for two axis-aligned 3D boxes, then generalize to oriented BEV boxes.
5. For the tracking pipeline of Fig. 11.3, describe two distinct failure modes and the robustness techniques that address each.
6. A Kalman-filtered track has position covariance $\text{diag}(0.1, 0.1)$ and a new detection arrives with residual $(0.6, 0.3)$. Compute the Mahalanobis distance and decide whether to associate at χ^2_2 threshold 9.21.
7. For the Hungarian solve in Example 11.11, write the cost matrix as if two of the predictions had non-trivial overlap (e.g., two vehicles at a merge). How does gating change the solution?
8. A camera-only 3D detector has range-dependent depth error $\sigma_Z = 0.02Z^2$. At what range does σ_Z exceed a typical lane width (3.5 m)?
9. A data flywheel mines 20 low-confidence clips per vehicle per day. A fleet of 500 vehicles retrains weekly. Estimate the rate at which the training set grows and the fraction that is actually useful (i.e., actually improves the model).
10. For the ch8 pipeline end-to-end example (11.15), the latency budget allows 100 ms. The pipeline currently uses 85 ms. Propose three specific changes that could each save 5 ms and quantify the accuracy impact of each.

Chapter 12

Semantic, Instance, and Panoptic Segmentation

12.1 Introduction

Detection (Chapter 11) gives us a list of objects; segmentation gives us a per-pixel or per-point label for *everything*—which pixels are road, which are sidewalk, which are sky, which are the specific pedestrian the planner is about to yield to. The outputs are denser and in several respects more informative, because the planner cannot make safe decisions from bounding boxes alone: it needs the *precise shape* of the drivable surface, the geometry of each obstacle, and the extent of free space in the scene. Segmentation answers these questions directly.

Three observations motivate the chapter’s structure.

First, there are three kinds of segmentation and a practical AV needs all three. Semantic segmentation assigns a class to every pixel but does not distinguish two instances of the same class (“car” vs “car 1 and car 2”). Instance segmentation distinguishes instances but only for *thing* classes (countable objects like cars and pedestrians). Panoptic segmentation unifies both—every pixel gets a class, and thing-class pixels additionally get an instance ID. The planner consumes all three: drivable-area segmentation for free space, instance masks for per-agent reasoning, panoptic output for scene-level understanding.

Second, segmentation works on every modality. Camera segmentation is the richest—semantic cues in color are abundant—but camera segmentation without calibrated 3D is hard to use. LiDAR segmentation directly produces 3D labels that the planner can consume. BEV segmentation takes multi-modal feature maps and produces top-down labels that match the planner’s natural representation. Each modality has a characteristic architectural family; we cover all three.

Third, segmentation labeling is expensive. A single camera image with 10 object classes may take 30 minutes of careful human annotation; a LiDAR frame with per-point labels takes several hours. Scaling to the data volumes a production AV program needs requires aggressive use of self-training,

weak supervision, and synthetic data. The final section of the chapter covers these techniques.

The chapter is organized by the dataflow: we start with a problem taxonomy, then develop image segmentation (FCN, U-Net, DeepLab, Mask2Former), then instance and panoptic methods, then LiDAR segmentation (PointNet++, Cylinder3D, range-image networks), then BEV segmentation (the AV-native case), then the scaling techniques that make production labeling budgets work, and we close with uncertainty quantification and a case study of a deployed panoptic segmenter. Every formula has a worked example; three architecture diagrams visualize key pipelines.

12.2 Problem Taxonomy

A clean vocabulary prevents confusion across the chapter:

- **Stuff** classes: uncountable regions. Road, sidewalk, sky, vegetation, building.
- **Thing** classes: countable instances. Car, pedestrian, cyclist, traffic sign.
- **Semantic segmentation**: per-pixel class label. Does *not* distinguish instances.
- **Instance segmentation**: per-pixel mask, *only for thing classes*, with instance IDs.
- **Panoptic segmentation**: unified output. Every pixel has a class; thing-class pixels additionally have an instance ID.
- **Point-cloud / BEV segmentation**: the same taxonomy lifted to 3D point sets or top-down grids.

For an AV, the planner typically consumes a BEV panoptic map plus a per-class free-space mask. Camera and LiDAR segmentation are intermediate products that feed this fused output; instance segmentation is sometimes used directly (to refine detection-based masks) and sometimes used only for training data labeling.

12.3 Image Semantic Segmentation

We begin with the camera-image case because its architectures seed every other modality's approach. Figure 12.1 shows the encoder-decoder pattern that dominates: a backbone compresses the image into low-resolution, feature-rich maps (encoder); a decoder upsamples these back to full resolution using skip connections from the encoder to preserve fine detail.

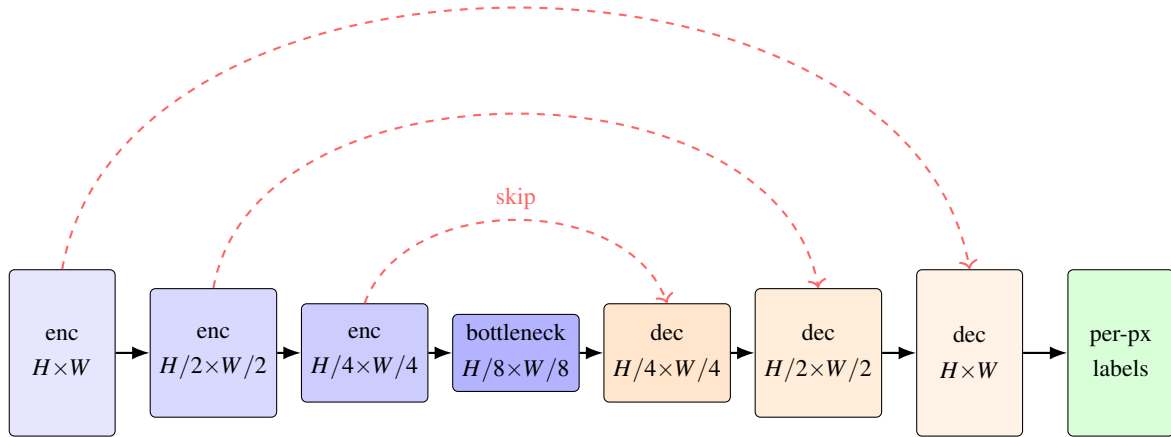


Figure 12.1: U-Net-style encoder-decoder. The encoder (left, blue) compresses the image through successive downsampling; the decoder (right, orange) upsamples back to full resolution. Skip connections (red, dashed) carry high-resolution features from the encoder directly to corresponding decoder stages, preserving fine detail that would otherwise be lost through downsampling. This pattern is the backbone of nearly every modern segmenter, from U-Net itself to DeepLab, Mask2Former, and beyond.

12.3.1 Fully Convolutional Networks

The seminal FCN [47] showed that classification backbones could be turned into per-pixel predictors simply by replacing the final fully-connected layers with 1×1 convolutions and upsampling the coarse output. The idea is simple; its practical effect was enormous: essentially every modern segmenter descends from FCN.

Example 12.1 (Receptive field vs output size). ResNet-50 backbone with three downsampling stages produces a $H/32 \times W/32$ feature map. For a 1024×2048 input, output is 32×64 —a $2000\times$ compression. Upsampling back to the input size loses fine detail; this is why atrous (dilated) convolutions (below) and encoder-decoder networks with skip connections replaced the original FCN for AV-grade applications.

12.3.2 Dilated Convolutions and DeepLab

The DeepLab family [12] replaces downsampling with *dilated* convolutions, which enlarge the receptive field without reducing resolution. A dilated 3×3 convolution with rate r samples inputs spaced r pixels apart:

$$\mathbf{y}_{i,j,c_o} = \sum_{u,v,c_i} \mathbf{K}_{u,v,c_i,c_o} \mathbf{x}_{i+ru,j+rv,c_i} + b_{c_o}. \quad (12.1)$$

Stacking dilated convolutions with rates $(1, 2, 4, 8)$ produces a large effective receptive field at full resolution. DeepLab augments this with *atrous spatial pyramid pooling* (ASPP), parallel dilated branches at multiple rates concatenated before the final classifier.

Example 12.2 (Receptive field with dilated stack). Four 3×3 convolutions with rates $(1, 2, 4, 8)$ cascaded: receptive-field growth per layer is $(1 \cdot 2 + 1) = 3$, then $3 + 2 \cdot 2 = 7$, then $7 + 2 \cdot 4 = 15$, then $15 + 2 \cdot 8 = 31$. A final output pixel at full resolution sees a 31×31 input context without ever downsampling. For 1024×2048 driving imagery, this is enough receptive field to capture typical vehicles and most pedestrians within a single forward pass.

12.3.3 Transformer-Based Segmentation

Segment-level transformers (SegFormer, Mask2Former [14]) treat pixels as tokens and learn mask predictions via cross-attention between learned queries and image features. Mask2Former unifies semantic, instance, and panoptic segmentation under a single architecture: N learned queries each produce one mask prediction, with Hungarian matching to ground-truth masks during training. The same network handles all three tasks by changing only the training target.

12.3.4 Losses

Two dominant pixel-level losses:

Cross-entropy (per-pixel over classes):

$$\mathcal{L}_{\text{CE}} = -\frac{1}{HW} \sum_{i,j} \sum_c y_{i,j,c} \log \hat{p}_{i,j,c}. \quad (12.2)$$

Dice loss (region overlap):

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_{i,j} y_{i,j} \hat{p}_{i,j}}{\sum_{i,j} y_{i,j}^2 + \sum_{i,j} \hat{p}_{i,j}^2 + \epsilon}. \quad (12.3)$$

Dice loss handles class imbalance better than cross-entropy because it weighs the intersection and union directly, so tiny classes contribute in proportion to their spatial extent rather than in proportion to pixel count.

Example 12.3 (Dice vs cross-entropy on a small class). An image is 99 % background, 1 % a road-sign class. A detector that labels everything as background has cross-entropy loss $\approx 0.01 \cdot (-\log 0) = \infty$ for the road-sign pixels theoretically, but in practice the model predicts ~ 0.01 probability for road-sign, giving finite but small per-image loss $\approx 0.01 \cdot 4.6 = 0.046$ contribution from the rare class—easily dominated by the 0.99-weighted cross-entropy on background. Dice loss on the same prediction: numerator is nearly zero (almost no intersection), denominator is about 0.01; $\mathcal{L}_{\text{Dice}} \approx 1 - 0 = 1$ —maximum loss. The gradient forces the network to predict the road-sign pixels correctly. This is why Dice (or its Tversky/Focal variants) is standard for lane-line and traffic-sign segmentation.

12.3.5 Lane and Drivable-Area Segmentation

Two special segmentation tasks dominate AV deployment because their outputs feed the planner directly.

Drivable-area segmentation produces a single binary mask over the road surface the ego could safely occupy. Failure modes are asymmetric: a false positive (mislabeling a sidewalk as drivable) is dangerous, a false negative (missing a legitimate lane) is merely conservative. Production segmenters are therefore tuned for precision on drivable-area at the cost of some recall.

Lane-line segmentation produces thin, elongated masks along lane boundaries. The extreme aspect ratio of lane lines (many pixels long, 2–5 pixels wide) is a known failure mode for standard segmenters; specialized architectures (LaneNet, LSTR, CurveFormer) regress lane polynomial or keypoint parameters rather than raw masks, improving robustness at the expense of requiring lane-graph post-processing.

Example 12.4 (Aspect-ratio pathology for lane lines). A typical lane-line mask is 800 pixels long, 3 pixels wide—an aspect ratio of 267:1. Cross-entropy over a 2048-pixel-wide image: line pixels are ~ 2400 out of $1024 \cdot 2048 = 2.1 \times 10^6$, or 0.11 % of the image. A network that labels everything as background achieves cross-entropy loss $\approx 0.0002 \cdot 7 = 0.0014$ —close to optimal by per-pixel count, but with zero lane detection. Specialized lane-line losses (Dice, focal, or parametric regression) are not optional for this task.

12.4 Instance and Panoptic Segmentation

Semantic segmentation treats all cars as one undifferentiated “car” region. When two cars touch in the image, the resulting single blob is the wrong abstraction for a planner that needs per-agent reasoning. Instance segmentation fixes this for thing classes; panoptic segmentation extends the fix to full-image understanding.

12.4.1 Instance: Mask R-CNN

Mask R-CNN [30] extends the Faster R-CNN detector with a mask head: each detected object gets not just a class and box, but a per-pixel mask within the box. The mask branch is a small FCN applied to ROI-aligned features. Training adds a binary cross-entropy on the mask in addition to the classification and box-regression losses.

Example 12.5 (Mask loss balance). Three loss terms: classification $\mathcal{L}_{\text{cls}} = 0.7$, box $\mathcal{L}_{\text{box}} = 1.3$, mask $\mathcal{L}_{\text{mask}} = 0.9$. Total with default weights (1, 1, 1): 2.9. Mask-heavy weighting (1, 1, 5): $0.7 + 1.3 + 4.5 = 6.5$, gradient dominated by mask term—improves mask IoU but hurts box accuracy. Production tuning typically keeps the default weights and instead increases the mask head’s capacity if mask quality is the bottleneck.

12.4.2 Panoptic Segmentation

Panoptic segmentation [39] requires that every pixel gets either (a) a stuff-class label or (b) a thing-class label plus instance ID. The standard metric is *Panoptic Quality* (PQ):

$$\text{PQ} = \underbrace{\frac{\sum_{(p,g) \in \text{TP}} \text{IoU}(p,g)}{|\text{TP}|}}_{\text{SQ: segmentation quality}} \cdot \underbrace{\frac{|\text{TP}|}{|\text{TP}| + \frac{1}{2}|\text{FP}| + \frac{1}{2}|\text{FN}|}}_{\text{RQ: recognition quality}}. \quad (12.4)$$

SQ measures the IoU quality of matched predictions; RQ measures detection-style recall/precision. $\text{PQ} = \text{SQ} \cdot \text{RQ}$ combines both into a single number comparable across semantic and instance components.

Example 12.6 (PQ numerical example). Scene has 5 ground-truth pedestrian instances. Predictions: 6 candidate masks. Hungarian matching at 0.5-IoU threshold yields 4 true positives with mean IoU 0.82; 1 false positive (unmatched prediction); 1 false negative (unmatched ground truth). $\text{SQ} = 0.82$. $\text{RQ} = 4 / (4 + 0.5 \cdot 1 + 0.5 \cdot 1) = 4/5 = 0.80$. $\text{PQ} = 0.82 \cdot 0.80 = 0.656$. A good pedestrian PQ on nuScenes is in the 0.60–0.75 range; our example would be typical of a solid baseline, below the state of the art.

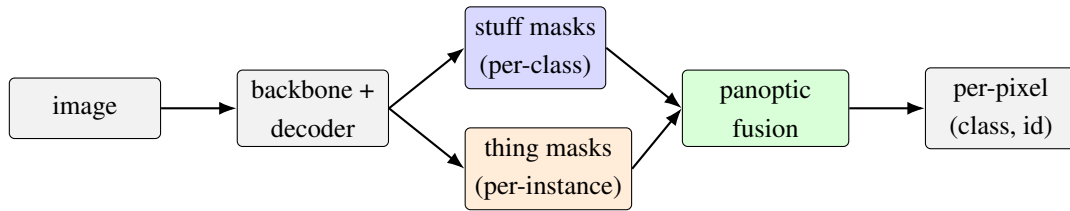


Figure 12.2: Classical panoptic-segmentation pipeline. A shared backbone feeds two heads: a stuff-class semantic segmenter and a thing-class instance segmenter. A fusion step combines them into a single per-pixel output where each pixel has a class and (for thing classes) an instance ID. Unified architectures (Mask2Former, OneFormer) collapse this into a single query-based head with one forward pass.

12.5 LiDAR Point-Cloud Segmentation

Camera segmentation gives us 2D per-pixel labels; to use them for free-space reasoning we must lift them back to 3D via the camera projection, which reintroduces depth ambiguity. LiDAR segmentation sidesteps this entirely by producing per-point labels in the already-3D point cloud. The dominant architectures follow the same representation families as LiDAR detection (Chapter 5).

12.5.1 PointNet and PointNet++

PointNet and PointNet++ (Chapter 5) extend naturally to segmentation by outputting a class per point rather than a single classification. Set-abstraction modules extract multi-scale features; feature-propagation modules upsample them back to per-point resolution. The architecture closely mirrors U-Net for point clouds.

Example 12.7 (Per-point vs per-frame classification cost). A frame has $N = 130000$ points. Per-point MLP with 3 layers of 256 units each: roughly $N \cdot 3 \cdot 256^2 \approx 2.5 \times 10^{10}$ MACs for the per-point branch. Per-frame classification with the same MLP applied to a pooled 1024-d feature: $256^2 \cdot 3 \approx 2 \times 10^5$ MACs. Per-point segmentation is therefore $\sim 10^5 \times$ more expensive than per-frame classification—which is why efficient architectures (Cylinder3D, range-image methods) are preferred for production.

12.5.2 Voxel and Range-Image Methods

Cylinder3D [79] voxelizes in cylindrical coordinates (radius, azimuth, height), which matches the natural acquisition geometry of a spinning LiDAR and preserves angular resolution at range. RangeNet++ and SqueezeSegV3 apply 2D CNNs to the range-image representation, exploiting the speed of 2D convolutions. Both approaches are $\geq 4\times$ faster than PointNet++ at similar accuracy on SemanticKITTI.

12.5.3 Scene-Level Transformer Methods

SST, SphereFormer, and related transformer approaches tokenize the point cloud (raw points, voxels, or range-image cells) and apply sparse self-attention. Results on public benchmarks are gradually overtaking sparse-conv baselines.

12.5.4 Panoptic LiDAR Segmentation

The SemanticKITTI-Panoptic and nuScenes-LidarSeg benchmarks extend per-point semantic segmentation with instance IDs for thing classes. Dominant architectures couple a semantic segmenter (for stuff) with a clustering head or learned instance centers (for things). DS-Net and Panoptic-PolarNet are representative.

12.6 Segmentation in BEV Space

The planner’s native representation is a top-down BEV grid. It is therefore natural to produce segmentation directly in BEV—a unified representation that spans all sensors and consumers.

BEV segmentation outputs typically include:

- Drivable area / road mask.
- Lane polygons and connectivity.
- Per-class occupancy.
- Pedestrian-zone mask.
- Crosswalk and intersection masks.

Architectures combine Lift-Splat-Shoot (Chapter 4) or LiDAR BEV-pillar encoders (Chapter 5) with a BEV-tensor segmentation head. Because inputs from both modalities live in the same BEV coordinate system, fusion is straightforward (concatenation or cross-attention).

Example 12.8 (BEV grid resolution vs compute). 100×100 m ego-centered grid. At $\Delta = 0.5$ m: $200 \times 200 = 40\,000$ cells. At $\Delta = 0.25$ m: $400 \times 400 = 160\,000$ cells ($4\times$). At $\Delta = 0.1$ m: $1000 \times 1000 = 10^6$ cells ($25\times$). Convolutions scale with cell count; finer resolution is better for fine features (lane lines, sidewalk edges) but is more costly. Production stacks typically run 0.25 m for the near field (20 m radius) and 0.5 m beyond, combined via a multi-resolution head.

12.7 Scaling: Labels, Annotation, and Self-Training

Human per-pixel labeling is expensive. A camera frame with 20 classes takes 15 min to 45 min to annotate; a LiDAR frame with per-point panoptic labels can take several hours. Production AV programs cannot afford to label their entire fleet data by hand. Several scaling techniques address this.

12.7.1 Self-Training and Pseudo-Labels

Train a segmenter on a modest labeled set; run it over a vast unlabeled set; treat high-confidence predictions as pseudo-labels and retrain on the union. Iterate. Modern pseudo-labeling adds confidence thresholding, uncertainty estimation, and consistency regularization (Noisy-Student, FixMatch) to avoid confirmation bias.

Example 12.9 (Self-training effective labeling-gain). Labeled set: 1000 frames. Unlabeled set: 100000 frames. Initial model mIoU 0.62. Generate pseudo-labels on unlabeled data, keep frames where predicted confidence > 0.9 : 24 % of the unlabeled set, 24000 frames. Retrain on 25k-frame union: mIoU 0.71. Two iterations improve to mIoU 0.74. Essentially a $25\times$ effective labeling gain from the same human budget.

12.7.2 Weakly-Supervised Segmentation

Weak labels (image-level class tags, bounding boxes, point-annotations) are cheaper than pixel-level masks and can still train a segmenter when combined with appropriate supervision techniques (class-activation mapping, boundary regularization, region-based loss).

12.7.3 Synthetic and Simulated Data

Driving simulators (CARLA, NVIDIA DRIVE Sim) produce synthetic data with perfect segmentation ground truth. The sim-to-real gap is real but smaller for geometric tasks (drivable-area) than semantic tasks (color-dependent classes like traffic signs). Domain-randomization and sim-to-real adaptation narrow the gap; production AV programs typically use synthetic data as a supplement, not a replacement.

12.7.4 Foundation-Model Annotation

Segment Anything (SAM [38]) and similar foundation models produce class-agnostic masks from a prompt (a point, a box, or text). Labeling a frame becomes a point-and-click affair that is $5\times$ – $10\times$ faster than drawing polygons manually. Production labeling pipelines increasingly use foundation models as pre-labelers that humans then correct.

12.8 Uncertainty in Segmentation

A segmenter that outputs 95 % confidence on pixels that are right only 70 % of the time feeds the planner overconfident beliefs. Calibration matters as much for segmentation as for detection (Chapter 2).

Mechanisms:

- **MC dropout:** stochastic forward passes give per-pixel standard deviations as epistemic uncertainty.
- **Deep ensembles:** average predictions from M independently trained models. Gold standard but $M\times$ compute.
- **Evidential segmentation:** single-forward-pass methods that output Dirichlet distributions, recovering uncertainty without ensembles.
- **Temperature scaling on class logits:** cheapest post-hoc calibration, routinely used.

Example 12.10 (Temperature-scaling calibration). A segmenter’s per-pixel confidences average 0.88, but empirical accuracy is only 0.73 (overconfident by 15 pp). Fit temperature T on held-out set to minimize NLL; typical result $T \approx 1.6$. Post-calibration logits z/T produce confidences averaging 0.75, closely matching the empirical accuracy. The planner can now trust segmentation confidence; previously it could not.

12.9 Case Study: A Production Panoptic Segmenter

Consider a production panoptic segmenter processing multi-camera input at 10 Hz. Architecture:

- **Backbone:** shared Swin-T across 6 cameras, 1024×768 each.
- **Feature lift:** Lift-Splat-Shoot to BEV (Chapter 4).
- **BEV backbone:** 2D CNN at 200×200 BEV resolution.
- **Heads:** semantic (per-cell stuff/thing class) and instance (per-cell offset to nearest instance center).
- **Post-processing:** cluster thing-class pixels by their regressed instance centers; merge with semantic output.

Latency budget: 70 ms out of the 100 ms perception budget. Per-stage:

- Backbone (6×): 35 ms.
- Lift-Splat: 10 ms.
- BEV backbone: 15 ms.
- Heads + postproc: 10 ms.

Key failure modes actually seen in fleet data: (i) snow cover on the ground degrading drivable-area masks, (ii) tall truck trailers produce oversized BEV masks because the lift misestimates their vertical extent, (iii) shadows at dawn/dusk produce spurious drivable-area pixels. Each was resolved through a mix of augmentation, label correction, and architectural tweaks—the standard cycle of production segmentation.

12.10 Worked Examples

Example 12.11 (End-to-end BEV panoptic output). Scene: urban intersection with 3 cars, 2 pedestrians, 1 cyclist, lane markings visible, stop line ahead.

Segmenter produces a 200×200 BEV tensor with:

- Per-cell stuff class: 18000 road cells, 14000 sidewalk cells, 2000 crosswalk cells, 6000 other.
- Per-cell instance ID for thing classes: 6 distinct agent masks.

Total 40000 cells, processed at 10 Hz with 70 ms latency. Downstream planner receives drivable-area (road + crosswalk = 20000 cells), per-instance agent geometries (6), and lane-line polygons (5 detected). Safety-critical planning decisions depend on every element of this output being correct.

Example 12.12 (Dice + cross-entropy joint loss). Production lane-line segmenter uses $\mathcal{L} = 0.5\mathcal{L}_{\text{CE}} + 0.5\mathcal{L}_{\text{Dice}}$. On a single batch: $\mathcal{L}_{\text{CE}} = 0.08$ (background dominated), $\mathcal{L}_{\text{Dice}} = 0.42$. Joint: $0.04 + 0.21 = 0.25$. Gradient from Dice term is $5 \times$ larger than from cross-entropy—exactly right for lane-line training, where the rare lane-pixels matter disproportionately.

Example 12.13 (Panoptic fusion resolution of conflicting outputs). A pixel is scored 0.7 “car” (thing) and 0.6 “road” (stuff) by the two heads. Panoptic fusion rule: thing classes take precedence if confidence > 0.5 , otherwise fall back to the highest-confidence stuff class. The pixel is labeled “car, instance 7.” A harder case: 0.3 “pedestrian” (thing), 0.55 “road” (stuff). Fusion rule rejects the low-confidence thing prediction and labels the pixel “road.” Mis-tuned thresholds cause silent thing-class recall loss at boundaries; production pipelines sweep these carefully.

12.11 Industrial Deployment

Production segmentation in AVs draws on architectures including DeepLabv3+ [12] (Google), Mask R-CNN (Facebook AI Research), and Cylinder3D [79] (Chinese University of Hong Kong) for LiDAR semantic segmentation. The Cityscapes dataset (Daimler AG, Max Planck Institute, TU Darmstadt) remains a standard benchmark for urban scene segmentation; SemanticKITTI extends KITTI for LiDAR semantic segmentation. The Segment Anything Model (SAM) [38] released by Meta AI in 2023 has reshaped the segmentation landscape and is used in many AV labeling pipelines. Tesla, Mobileye, and Chinese ADAS suppliers have all publicly described production segmentation pipelines for free-space, drivable-area, and instance-level scene understanding.

12.12 Chapter Summary

Segmentation produces dense per-pixel or per-point outputs that detection cannot. Semantic segmentation labels class per pixel; instance segmentation adds per-instance masks for thing classes; panoptic segmentation unifies both. Encoder-decoder and dilated-conv CNNs dominated the field through the late 2010s; transformer-based approaches (Mask2Former, SegFormer) are now state of the art across benchmarks. LiDAR and BEV segmentation use the same architectural families adapted to 3D representations. The hard practical problem is label cost; self-training, weak supervision, synthetic data, and foundation-model pre-labeling each scale the labeling budget at some accuracy cost. Production segmenters output a BEV panoptic map that the planner consumes directly for free-space reasoning and per-agent geometry.

12.13 Exercises

1. Compute the cross-entropy and Dice losses for a single image with 100 pixels, 5 of which are a rare class. Predicted probabilities: 0.1 for every pixel to be rare-class, ground truth: the 5 rare pixels are rare-class, rest background.
2. A dilated-convolution stack has rates (1, 2, 4, 8, 16). Compute the receptive-field size of the final output pixel relative to the input.
3. A Mask R-CNN training pipeline has loss terms with magnitudes $\mathcal{L}_{\text{cls}} = 0.5$, $\mathcal{L}_{\text{box}} = 1.2$, $\mathcal{L}_{\text{mask}} = 3.5$. Propose loss weights that balance their gradient contributions.
4. Compute PQ for a panoptic output with 4 TP (mean IoU 0.78), 2 FP, 1 FN.
5. A production BEV segmenter runs at 200x200 resolution. Estimate the memory required for its BEV feature tensor assuming 128 channels and FP16 precision.
6. Describe three failure modes of image-plane semantic segmentation for AV deployment. Propose a BEV-based architecture that addresses each.
7. A point-cloud segmenter achieves mIoU 0.64 on SemanticKITTI benchmarks but only 0.48 in

- rainy deployment. List three likely causes and mitigations.
8. Explain why Dice loss is preferred over cross-entropy for lane-line segmentation, using the aspect-ratio arithmetic from Example 12.4.
 9. A self-training loop produces pseudo-labels at 0.9 confidence threshold. After one iteration, the confidence distribution shifts: many labels are now at 0.85. Propose a schedule for the confidence threshold.
 10. For the case-study production panoptic segmenter, propose three specific architectural or training changes that would each improve a stated failure mode without breaking the others.

Part III

Localization and Mapping

Chapter 13

Simultaneous Localization and Mapping

13.1 Introduction

Part II developed sensors and the per-frame perception outputs that describe *what is around the vehicle*. Part III shifts the question to *where the vehicle is*. These are not the same problem: a perception stack can report a pedestrian 20 m ahead and still leave the planner helpless if the ego itself has 5 m of localization error. Safe driving requires both—a precise map of the environment, and a precise pose in it—updated at control-loop rate with bounded uncertainty.

Simultaneous Localization and Mapping, or SLAM, is the classical solution when both are unknown. A robot moves through an environment, observing landmarks through noisy sensors while its own motion is corrupted by noise; SLAM estimates the robot’s trajectory and the landmark map jointly. For autonomous driving the problem has specific flavors: *visual SLAM* uses cameras, *LiDAR SLAM* uses point clouds, and *visual-inertial odometry* (VIO) fuses camera and IMU. Each produces an estimated pose stream that the downstream localization layer can either use directly (mapless driving) or align with a prior HD map (Chapter 14).

Three observations motivate the chapter’s content.

First, SLAM is fundamentally a *graph optimization*. The unknowns—poses and landmarks—are nodes in a factor graph. The measurements—odometry, reprojection, scan matching, loop closure—are edges that impose soft constraints. A nonlinear least-squares backend finds the configuration that minimizes the total constraint error. Understanding this one abstraction unifies the whole field: bundle adjustment, pose-graph SLAM, VIO, LiDAR SLAM are all variations on it.

Second, SLAM for driving is *large-scale and sparse*. A city-scale map has millions of poses and landmarks, but most pairs of them are unrelated—they are not simultaneously observable. The resulting Hessian is very sparse. Exploiting sparsity makes the difference between a solver that finishes in seconds and one that does not finish at all; modern libraries (g2o, GTSAM, Ceres) are built around this fact.

Third, SLAM is most useful in a driving stack as the *ego-motion and local-consistency* layer, not as a standalone global localization system. A typical production stack uses SLAM (specifically VIO or LiDAR odometry) to produce smooth, high-rate relative motion; it then uses a separate map-matching layer (Chapter 14) to anchor the smooth trajectory to a global HD map. Coherent combination of the two is the production pattern.

The chapter is organized as follows. We first formulate the SLAM problem and develop the factor-graph framework that unifies its variants. We then specialize to visual SLAM, LiDAR SLAM, and their inertial-aided forms, each with worked examples of the key operations. Learned SLAM components are next. We close with uncertainty, consistency, and the specifics of what makes driving SLAM different from textbook SLAM. Every formula is accompanied by a worked numerical example; three architecture diagrams visualize the key pipelines.

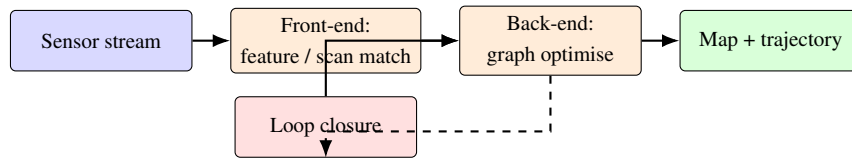


Figure 13.1: Chapter roadmap. SLAM is a two-stage pipeline: a fast front-end aligns successive observations; a slower back-end optimises a global graph of constraints. Loop closure (red) provides drift correction by detecting revisited places.

13.2 Problem Formulation

Let $\mathcal{X} = \{\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T\}$ be the sequence of ego poses $\mathbf{x}_t \in \text{SE}(3)$ and $\mathcal{L} = \{\mathbf{l}_1, \dots, \mathbf{l}_N\}$ the landmark set (3D points, plane parameters, or higher-level features). Measurements $\mathbf{z}_k$ are generated by sensor models

$$\mathbf{z}_k = h_k(\mathbf{x}_{i_k}, \mathbf{l}_{j_k}) + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_k), \quad (13.1)$$

where h_k is the measurement function (pinhole projection, LiDAR ray, IMU pre-integration), i_k and j_k are the pose and landmark indices, and $\mathbf{v}_k$ is measurement noise. The maximum a posteriori estimate is

$$(\mathcal{X}^*, \mathcal{L}^*) = \arg \max p(\mathcal{X}, \mathcal{L} \mid \{\mathbf{z}_k\}), \quad (13.2)$$

equivalent, under Gaussian noise, to minimizing the sum of squared, covariance-weighted residuals:

$$(\mathcal{X}^*, \mathcal{L}^*) = \arg \min_{\mathcal{X}, \mathcal{L}} \sum_k \|\mathbf{z}_k - h_k(\mathbf{x}_{i_k}, \mathbf{l}_{j_k})\|_{\mathbf{R}_k^{-1}}^2. \quad (13.3)$$

This is a giant nonlinear least-squares problem. The rest of the chapter is about how to set it up for each sensor modality and how to solve it efficiently.

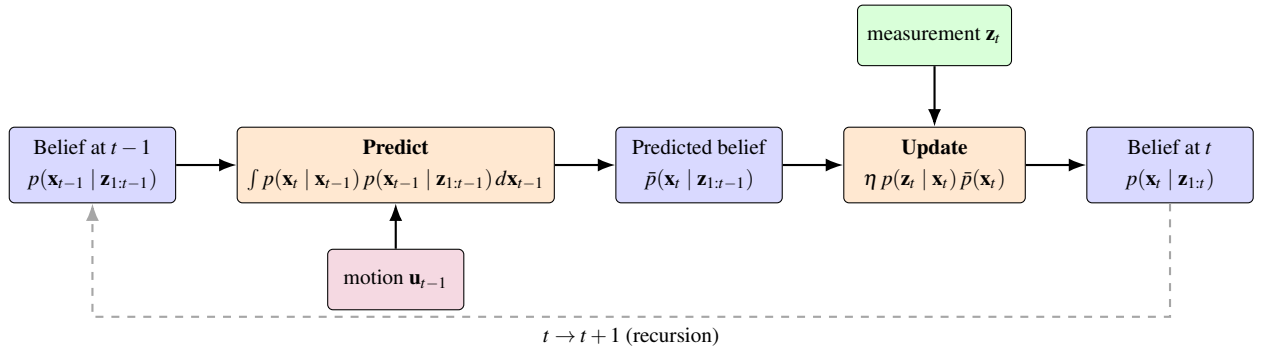


Figure 13.2: The Bayes-filter recursion underlies every probabilistic SLAM and state-estimation algorithm. At each step the prior belief is propagated through the motion model (**predict**), then corrected against the latest measurement (**update**), giving the posterior belief. The posterior becomes the prior at the next step (dashed feedback). Different filters (KF, EKF, UKF, particle filter) realise this recursion under different parametric or sample-based representations of the belief.

Example 13.1 (SLAM state dimension for a short drive). A 2 min drive at 10 Hz keyframe rate produces 1200 keyframes, each with 6 DoF: state dimension 7200. If each keyframe sees 200 landmarks (3D points) on average, and landmarks are 3D: add $200 \cdot 3 = 600$ landmark parameters per keyframe, but most landmarks persist across keyframes. A typical number of unique landmarks for such a sequence: 50000. Total state dimension: $7200 + 50000 \cdot 3 = 157200$. The Hessian is therefore nominally $\sim 1.57 \times 10^5 \times 1.57 \times 10^5$ —too big for a dense solve. Sparsity reduces this to manageable size: each landmark only couples to the ~ 10 poses that see it, so the nonzero fraction of the Hessian is $\ll 0.1\%$.

13.3 Factor-Graph SLAM

A factor graph represents the probabilistic structure of the problem explicitly: variable nodes are the unknowns (poses, landmarks, biases); factor nodes are the measurement-likelihood terms that constrain them. Figure 13.3 shows a small illustrative graph.

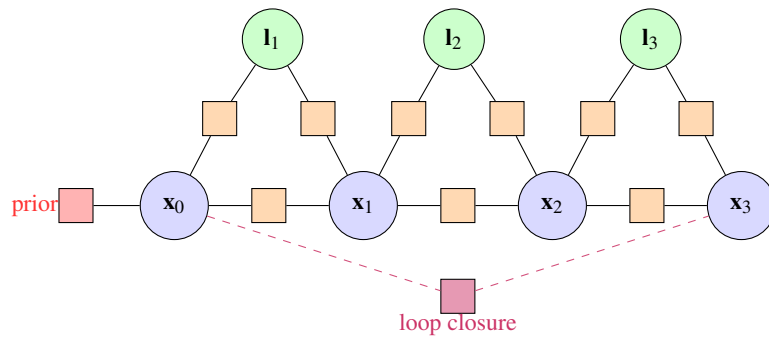


Figure 13.3: A small SLAM factor graph with 4 poses $\mathbf{x}_i$ (blue circles), 3 landmarks $\mathbf{l}_j$ (green circles), and factors (orange squares). A prior anchors $\mathbf{x}_0$ (red square). Consecutive-pose factors $f_{i,i+1}$ encode odometry constraints. Pose-landmark factors $f_{i,l}$ encode observations (e.g., camera reprojection or LiDAR measurement). The dashed purple factor is a *loop closure*: a constraint linking $\mathbf{x}_0$ and $\mathbf{x}_3$ because the vehicle revisits the starting region. Loop closures are what turn accumulating odometry drift into a globally consistent trajectory.

13.3.1 Factor Graphs

Each factor f_k is a probability distribution $p(\mathbf{z}_k | \mathbf{x}_{i_k}, \mathbf{l}_{j_k})$ over its adjacent variables. Under Gaussian noise, the negative log-likelihood of the factor is (up to a constant)

$$\ell_k = \frac{1}{2} \|\mathbf{z}_k - h_k(\mathbf{x}_{i_k}, \mathbf{l}_{j_k})\|_{\mathbf{R}_k}^2. \quad (13.4)$$

The total cost is a sum $\sum_k \ell_k$. Equation 13.3 is just the argmin of this sum.

Example 13.2 (Odometry factor cost). Consecutive poses $\mathbf{x}_i = \mathbf{I}$ and $\mathbf{x}_{i+1}$ with relative transform $\mathbf{T}_{\text{odom}} = (\mathbf{R}, \mathbf{t})$ measured by wheel odometry, $\mathbf{t} = (2.0, 0, 0)$ and $\mathbf{R} = \text{yaw } 1^\circ$. The estimate $\hat{\mathbf{T}}_{i,i+1}$ from the current solution is identity (poses haven't moved yet). Residual: $\log(\mathbf{T}_{\text{odom}}^{-1} \hat{\mathbf{T}}_{i,i+1}) = -\log(\mathbf{T}_{\text{odom}}) \approx (-2.0, 0, 0, 0, 0, -0.0175) \text{ m, rad}$. With $\mathbf{R} = \text{diag}(0.01, 0.01, 0.01, 0.001, 0.001, 0.001)$: $\ell = \frac{1}{2} \cdot (4/0.01 + 0.000306/0.001) = 200.15$. The first iteration of optimization will move $\mathbf{x}_{i+1}$ toward $\mathbf{T}_{\text{odom}}$, reducing this cost rapidly.

13.3.2 Solving by Gauss–Newton

Starting from an initial estimate $\boldsymbol{\theta}^{(0)}$ (stacked poses and landmarks), Gauss–Newton linearizes the residuals around the current estimate, solves a linear normal-equations system, and updates:

$$\mathbf{J}_k = \partial r_k / \partial \boldsymbol{\theta} \Big|_{\boldsymbol{\theta}^{(t)}}, \quad (13.5)$$

$$\mathbf{J}^\top \mathbf{W} \mathbf{J} \Delta \boldsymbol{\theta} = -\mathbf{J}^\top \mathbf{W} \mathbf{r}, \quad (13.6)$$

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} \boxplus \Delta \boldsymbol{\theta}, \quad (13.7)$$

where $\mathbf{J}$ and $\mathbf{r}$ are the stacked measurement Jacobian and residual, and $\mathbf{W}$ is the block-diagonal measurement information matrix. Levenberg–Marquardt damping adds a $\lambda \text{diag}(\mathbf{J}^\top \mathbf{W} \mathbf{J})$ term for robustness.

Example 13.3 (Size of the normal-equations system). The short-drive example above with 1.57×10^5 variables has a Hessian $\mathbf{H} = \mathbf{J}^\top \mathbf{W} \mathbf{J}$ of size $1.57 \times 10^5 \times 1.57 \times 10^5$. Dense storage: $\sim 2.5 \times 10^{10}$ doubles $\approx 200 \text{ GB}$ —not feasible. Sparse storage (only nonzero blocks): each landmark has 3 rows $\times$ 60 nonzero columns (10 observing poses $\times$ 6 DoF); each pose has 6 rows $\times$ (6 odom + $n_{\text{visible}} \cdot 3$ landmark). Nonzero-entry count $\sim 10^7$, storage $\sim 80 \text{ MB}$ —easy. A Cholesky or QR factorization of this sparse matrix completes in seconds with a well-ordered sparsity pattern.

13.3.3 Incremental Smoothing

Classical bundle adjustment is a batch operation. Online SLAM needs incremental solutions as new measurements arrive. Incremental smoothing (iSAM, iSAM2 [34]) reuses previously-computed factorizations, updating only the affected submatrix when a new factor is added. The amortized cost per frame is small enough to run at camera frame rate.

Example 13.4 (Incremental update cost). A new keyframe adds 1 pose (6 DoF) and 50 new landmark observations, touching ~ 100 existing landmarks. Only variables directly linked to the new factors, plus their indirect neighbors, need re-factorization. In a well-ordered graph, this subtree contains ~ 500 – 2000 variables out of the $160\,000$ total. Re-factorizing a 2000×2000 sparse block runs in ~ 20 ms on a modern CPU—comfortably within the keyframe-rate budget.

13.4 Visual SLAM and Visual-Inertial Odometry

Cameras are the cheapest SLAM sensor and, in many scenes, the most information-rich. Visual SLAM has two main flavors: *feature-based* methods (extract and track keypoints) and *direct* methods (use raw pixel intensities). Figure 13.4 shows a VIO pipeline combining cameras with IMU.

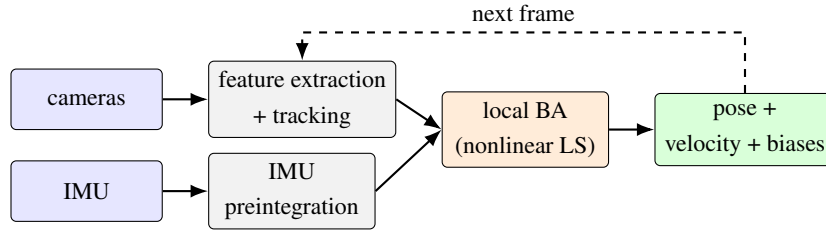


Figure 13.4: Visual-Inertial Odometry (VIO) pipeline. Camera features are extracted and tracked across frames; IMU samples are preintegrated into relative motion constraints between keyframes. Both feed a local bundle-adjustment optimizer that estimates the current pose, velocity, and IMU biases. The output is fed back into feature tracking (to reject outliers via predicted positions) and out to the planner / localization layer. VIO is the production standard for smooth, high-rate ego-motion in visual-primary AV stacks.

13.4.1 Feature-Based Methods

ORB-SLAM [50] and ORB-SLAM2/3 are canonical examples. Per frame:

- Extract ORB keypoints with rotation-invariant descriptors.
- Track across frames by descriptor matching + RANSAC.
- Triangulate matched keypoints for 3D landmarks.
- Add reprojection factors to the factor graph.
- Detect loop closures via bag-of-words similarity search.
- Run local bundle adjustment on the recent sliding window; global optimization on loop closure.

The reprojection residual is

$$\mathbf{r}_{\text{reproj}} = \mathbf{u}_{\text{obs}} - \pi(\mathbf{K} \mathbf{T}_{CW} \mathbf{I}^W), \quad (13.8)$$

where π is the pinhole projection, $\mathbf{K}$ are intrinsics, $\mathbf{T}_{CW} \in \text{SE}(3)$ is the camera pose, and $\mathbf{I}^W$ is the

3D landmark. Jacobians with respect to $\mathbf{T}_{CW}$ are computed via Lie-algebra perturbation (Chapter 2).

Example 13.5 (Reprojection residual of a matched landmark). A landmark at $\mathbf{l}^W = (10, 1, 0.5)$ m, camera pose $\mathbf{T}_{CW} = \mathbf{I}$ (for simplicity), intrinsics $f = 1200$, $c = (960, 540)$. Projection: $\mathbf{l}^C = \mathbf{l}^W = (10, 1, 0.5)$, $u = 1200 \cdot 10/0.5 + 960 = 24960$ —off-image, which indicates a bad pose guess. After Gauss–Newton moves the pose closer to the correct one, say $\mathbf{T}_{CW} = \exp((0, 0, -9.5, 0, 0, 0)^\wedge)$ (9.5 m translation along negative z), projection becomes $\mathbf{l}^C = (10, 1, 10)$, $u = 1200 \cdot 10/10 + 960 = 2160$ —on-image. Observed pixel (2170, 540) gives residual $(-10, 0)$ px, $\ell = 100 \cdot 0.5 \cdot 1/\sigma^2 = 50$ at $\sigma = 1$ px. Large residuals trigger further Gauss–Newton iterations or, at very large, outlier-rejection.

13.4.2 Direct Methods

DSO [20] and LSD-SLAM skip feature extraction and optimize directly over pixel intensities. A photometric residual is

$$\mathbf{r}_{\text{photo}} = I_j(\mathbf{u}_j) - I_i(\mathbf{u}_i), \quad (13.9)$$

where $\mathbf{u}_i$ is a pixel in image i and $\mathbf{u}_j$ is its warped location in image j , computed from depth and relative pose. Advantages: use more pixels (anywhere with intensity gradient), sharper at low texture. Disadvantages: sensitive to illumination change, require careful photometric calibration.

13.4.3 Visual-Inertial Fusion

VINS-Mono, OKVIS, and VI-DSO fuse visual factors with IMU preintegration factors [22]. IMU preintegration summarizes all IMU samples between two keyframes into a single relative-motion constraint while preserving the correct covariance. The result is a local bundle-adjustment problem with both visual and inertial factors.

Example 13.6 (Preintegrated IMU factor over 0.1 s). Between keyframes t_i and $t_{i+1} = t_i + 100$ ms the IMU samples 100 measurements at 1 kHz. Each is used in closed-form analytical preintegration to produce:

- $\Delta \mathbf{R}_{ij} \in \text{SO}(3)$: relative rotation.
- $\Delta \mathbf{v}_{ij} \in \mathbb{R}^3$: velocity change.
- $\Delta \mathbf{p}_{ij} \in \mathbb{R}^3$: position change.

Example values for a straight drive: $\Delta \mathbf{R}_{ij} \approx \mathbf{I}$, $\Delta \mathbf{v}_{ij} = (0.5, 0, 0)$ m/s (mild acceleration), $\Delta \mathbf{p}_{ij} = (2.025, 0, 0)$ m. Associated 15×15 covariance accumulated analytically. Pre-integrated factor connects 6 variables: poses at i and j , velocities at i and j , and biases. One factor replaces 100 raw IMU constraints and remains consistent under bias changes without re-integration.

13.5 LiDAR SLAM

LiDAR SLAM exploits the direct 3D geometry of point-cloud measurements. Unlike visual SLAM, depth is measured rather than triangulated, so scale is known and the problem becomes well-conditioned even without inertial aiding. The central operation is *scan registration*—aligning a live point cloud to a map or previous scan.

13.5.1 Scan Registration: ICP and Variants

Iterative Closest Point (ICP, [2]) alternates association and alignment. At each iteration:

1. For each point $\mathbf{p}_i$ in the source cloud, find the nearest point $\mathbf{q}_i$ in the target cloud.
2. Solve for the rigid transform $\mathbf{T}^* \in \text{SE}(3)$ that minimizes $\sum_i \|\mathbf{T}\mathbf{p}_i - \mathbf{q}_i\|^2$.
3. Apply $\mathbf{T}^*$ to the source cloud; repeat.

The alignment step has a closed-form SVD solution for point-to-point ICP and a small linear solve for point-to-plane ICP. Convergence is typically 10–30 iterations from a reasonable initialization.

Example 13.7 (ICP point-to-plane cost reduction per iteration). Two scans with initial misalignment: rotation 5° , translation 0.3 m. Initial sum of squared point-to-plane residuals: 2.4 m^2 over $N = 10000$ matched points. After iteration 1: 0.35 m^2 ($6.9\times$ reduction). After iteration 3: 0.008 m^2 . Converged residual: 0.0015 m^2 (noise-floor). Typical convergence in 5–10 iterations for moderate initial misalignment; with very poor initialization ($30^\circ+$), ICP can get stuck in local minima—global methods (NDT, feature-based) initialize better.

13.5.2 LOAM and its Descendants

LOAM [76] and its variants (A-LOAM, LeGO-LOAM, LIO-SAM, FAST-LIO) extract geometric features—edge points (high curvature) and planar points (low curvature)—from each scan and match them directly, rather than using raw points. This reduces the matching problem by $10\text{--}100\times$ and makes LiDAR odometry real-time on a modest CPU.

The edge-point residual (line-to-line distance):

$$\mathbf{r}_{\text{edge}} = \frac{|(\mathbf{p}_k - \mathbf{p}_i^m) \times (\mathbf{p}_k - \mathbf{p}_j^m)|}{|\mathbf{p}_i^m - \mathbf{p}_j^m|}, \quad (13.10)$$

where $\mathbf{p}_i^m, \mathbf{p}_j^m$ are two points on the matched map edge. The planar-point residual (point-to-plane distance) is analogous.

Example 13.8 (Edge-point residual for a lamppost). Live edge point $\mathbf{p}_k = (30.01, 5.02, 1.5)$, matched to map edge along the z -axis at $(30, 5, z)$. Two representative map edge points $\mathbf{p}_i^m = (30, 5, 0), \mathbf{p}_j^m = (30, 5, 3)$. The cross product magnitude (scaled by edge length 3) gives the perpendicular distance: $\sqrt{0.01^2 + 0.02^2} = 0.022 \text{ m}$. Residual is small; the match contributes little

to the cost. Misaligned scan at $\mathbf{p}_k = (30.5, 5, 1.5)$ gives residual 0.5 m—large contribution, strong gradient toward correcting the yaw misalignment.

13.5.3 Loop Closure

Loop closures are what turn drift-limited odometry into globally consistent SLAM. The detector recognizes “we’ve been here before” (typically by place recognition: bag-of-words, learned descriptors, or point-cloud descriptors like Scan Context). A loop closure edge is added to the factor graph between the current pose and the recognized earlier pose; re-optimization distributes the drift back across the trajectory.

Example 13.9 (Drift reduction from a loop closure). A 10 km drive accumulates 3 m lateral drift in odometry. The vehicle returns to the starting region; a loop closure factor with covariance std 0.1 m is added. Post-optimization, the drift is redistributed: start and end positions coincide within 0.1 m; intermediate poses are shifted by amounts proportional to their prior uncertainty, giving a trajectory consistent with all measurements. This operation (pose-graph optimization after a loop closure) is one of the most rewarding operations in all of robotics.

13.6 Learned SLAM Components

Classical SLAM pipelines rely on hand-crafted feature detectors, associators, and outlier filters. Learned components replace these with deep networks:

- **Learned feature detectors and descriptors:** SuperPoint, DISK, R2D2 produce repeatable, descriptive keypoints that outperform ORB/SIFT on benchmarks.
- **Learned matchers:** SuperGlue, LightGlue match keypoints using graph attention, dramatically reducing false matches.
- **Learned place recognition:** NetVLAD, MixVPR produce compact global descriptors for loop detection.
- **End-to-end learned odometry:** DeepVO, TartanVO regress camera motion directly from image pairs.

Hybrid pipelines (learned detectors + classical optimization) have largely replaced fully-classical pipelines on public benchmarks; fully-end-to-end learned SLAM is still research-stage for driving.

13.7 Uncertainty and Consistency

SLAM outputs are useful only with calibrated uncertainty: a pose estimate with 10 cm reported std but 1 m true error will be trusted beyond its precision. Two sources of miscalibration are typical: (i)

EKF-style filters are *inconsistent* due to linearization error—the posterior is overconfident; (ii) model mismatch (incorrect noise parameters) produces biased estimates.

Mitigations: factor-graph smoothing (consistently parameterized, better behaved than filtering at cost of additional compute); on-line noise-parameter estimation; posterior tests (chi-squared on residuals). Chapter 2 developed the underlying statistical machinery.

13.8 What's Different About Driving SLAM

Textbook SLAM (a robot exploring an indoor space) and driving SLAM differ in several ways:

- **Scale.** Driving is often pose-graph with millions of nodes over kilometers; indoor SLAM is hundreds over meters.
- **Dynamic scene.** Other vehicles and pedestrians violate the static-landmark assumption. Dynamic filtering (via segmentation output, Chapter 12) is essential.
- **Repetitive structure.** Highway scenes (lane markings, concrete walls) and tunnels lack distinctive features; visual SLAM can lose lock for hundreds of meters.
- **Planarity.** Ground vehicles move mostly in a 2D plane; SE(2) parameterizations are often sufficient and more robust than full SE(3) for certain maneuvers.
- **Prior map.** Most production AVs have HD maps (Chapter 14), so online SLAM often reduces to map-based localization with occasional mapping of previously-unseen regions.

13.9 Worked Examples

Example 13.10 (Full bundle-adjustment step). Problem: 4 keyframes, 10 landmarks, 32 observed correspondences, Gaussian noise $\sigma = 1$ px. Parameters: $4 \cdot 6 + 10 \cdot 3 = 54$. Measurement count: $32 \cdot 2 = 64$ (u, v per observation). Jacobian $\mathbf{J}$: 64×54 , block-sparse. Normal equations: $\mathbf{J}^\top \mathbf{J} \Delta \boldsymbol{\theta} = -\mathbf{J}^\top \mathbf{r}$. Sparse solve via Cholesky: microseconds. Initial cost sum: $\frac{1}{2} \|\mathbf{r}\|^2 = 80$. Post-step cost: 2.4. Converged in 5 iterations to near-zero reprojection error. Final pose accuracy: 1 mm translation, 0.01° rotation. This is bundle-adjustment-by-hand for a tiny problem; all the essential arithmetic is there.

Example 13.11 (VIO tracking a car through a tunnel). Tunnel entry at $t = 0$; exit at $t = 20$ s, 500 m long. Visual features dry up inside the tunnel (uniform walls). VIO switches to IMU-dominated mode: IMU preintegration produces relative motion with accumulated std ~ 0.5 m over 20 s at industrial-grade gyro. At tunnel exit, visual features re-acquire; loop-closure-like re-localization against the map shrinks position error to 0.1 m. Without IMU, visual SLAM would have lost track completely in the tunnel; without visual at exit, IMU drift would grow unbounded. VIO's combination is what makes long-tunnel navigation tractable.

Example 13.12 (LiDAR-only SLAM on a repeated route). Vehicle drives a 5 km closed-loop route, returning to the start. LOAM-style edge+planar feature matching produces odometry with drift 0.5 % of distance; 25 m of cumulative error before loop detection. Scan-Context loop detector recognizes the return, adds a loop closure edge with covariance std 0.05 m. Pose-graph optimization redistributes drift: final trajectory consistent within 5 cm everywhere. The resulting map is now production-deployable.

13.10 Industrial Deployment

SLAM techniques developed in academic robotics over two decades now appear across production autonomous systems. ORB-SLAM (Universidad de Zaragoza), Direct Sparse Odometry (DSO) [20] (TU Munich), and LOAM (Carnegie Mellon University) are landmark public implementations whose patterns are reflected in production stacks. The graph-optimisation back-end GTSAM (Georgia Tech) and the optimisation library Ceres Solver (Google) are widely used in production. Cartographer (Google), open-sourced in 2016, is used in robotics and indoor mapping. Production AVs typically combine SLAM techniques with prior HD maps rather than running pure SLAM at deployment; the combination of map-based localisation with online SLAM-style refinement is the dominant pattern at Waymo, Cruise, Apollo, and other programs.

13.11 Chapter Summary

SLAM jointly estimates a vehicle's trajectory and a map from noisy measurements. The unifying abstraction is the factor graph: nodes are poses and landmarks, factors are measurements, and a large nonlinear least-squares problem is solved for the configuration that minimizes constraint error. Visual SLAM, LiDAR SLAM, and their inertial-aided forms are all specializations of this framework. Incremental smoothing (iSAM2) makes online SLAM feasible at frame rate. Loop closure turns drift-limited odometry into globally consistent maps. Learned components (SuperPoint, SuperGlue) have replaced hand-crafted features on recent benchmarks. In driving, SLAM is typically used as the smooth, high-rate ego-motion layer, with HD-map matching (Chapter 14) providing absolute anchor.

13.12 Exercises

1. For the SLAM state dimension example (13.1), compute the storage requirement for the dense and sparse Hessians, and estimate the factor of sparsity.
2. Derive the Jacobian $\partial \mathbf{r}_{\text{reproj}} / \partial \boldsymbol{\xi}$ of the reprojection residual with respect to a right pose perturbation.
3. A preintegrated IMU factor between keyframes spans 200 ms at 500 Hz IMU rate. How many raw IMU samples does the factor replace, and what are the covariance properties?

4. Point-to-plane ICP converges faster than point-to-point; explain geometrically why.
5. A loop closure factor with std 0.1 m is added to a trajectory with accumulated 5 m drift. Describe qualitatively how the drift is distributed after optimization.
6. Describe three distinct failure modes of visual SLAM in an urban driving scenario and their mitigations.
7. Compute the expected drift after 60 s of LiDAR-only odometry at 0.3 % distance drift, assuming 20 m s^{-1} ego speed.
8. SuperGlue replaces classical ratio-test matching. Identify two driving-specific scenarios where SuperGlue is most beneficial.
9. A VIO system loses visual tracking for 5 s. Compute the expected position drift from industrial-grade IMU alone, and propose what additional sensors would bound it.
10. For a production SLAM system feeding an HD-map matcher, describe the interface between them: what does SLAM send, what does the matcher return, at what rate?

Chapter 14

High-Definition Maps and Map-Based Localization

14.1 Introduction

An HD map is the largest piece of data an autonomous vehicle carries with it. A city’s worth of centimeter-accurate road geometry, lane connectivity, traffic controls, and signage weighs several gigabytes per square kilometer; a robotaxi fleet operating in a metro area may carry tens of gigabytes on every vehicle. The investment pays back because the map converts difficult perception problems into easy lookup problems: *what is the speed limit here?*, *what lane connects to what?*, *where should the vehicle stop at this intersection?* are expensive perception queries that a fresh map answers instantly and reliably. This chapter develops what HD maps contain, how they are built, how the vehicle locates itself within one, and the increasingly relevant question of whether they are required at all.

Three observations motivate the chapter’s structure.

First, HD maps are not just more detailed versions of navigation maps. A navigation map encodes roads as connected polylines for the routing algorithm; an HD map encodes lane-level geometry with centimeter precision, *plus* annotations that the driving policy needs: speed limits, stop lines, yield relationships, lane-change legality, turn restrictions, traffic-light associations. The difference in fidelity is a difference in kind, not a difference in degree.

Second, map-based localization is *the* standard solution for centimeter-level ego-pose estimation in production AVs. Pure SLAM (Chapter 13) handles unknown environments but produces only locally-consistent pose; pure GNSS/RTK (Chapter 7) is globally referenced but urban-fragile. Map matching—aligning live perception output against a prior map—is what ties the two together with the precision and reliability the planner requires.

Third, HD maps are fragile in the same way all data products are fragile: they go out of date. A lane-closure for construction invalidates a map overnight; a new stop sign, a repainted intersection,

a reconfigured merge all require updates. Production map programs therefore treat maps as living data, with fleet-observed changes flowing back through validation into updated tiles. The scale of this “map freshness” operation often exceeds the scale of the original map build, and it is the operation that distinguishes production HD-map programs from research prototypes.

The chapter is organized as follows. We first catalog what HD maps contain, with a layered view that maps directly to how the data is stored and consumed. We then treat map construction pipelines: survey acquisition, automated semantic extraction, and crowd-sourced updates. Map-based localization follows, in detail: LiDAR point-cloud matching via ICP and NDT, semantic-landmark matching, visual localization, and fusion with GNSS/IMU to produce a calibrated pose at control-loop rate. We devote a section to mapless and map-light driving because both are active areas of industrial development. We close with map versioning, freshness operations, and privacy considerations. Every formula is accompanied by a worked example; three architecture diagrams visualize the key pipelines.

14.2 Contents of an HD Map

An HD map is a layered data structure (Figure 14.1). Each layer encodes a different kind of information the AV stack needs; layers are built on top of each other, with each layer depending on the one below.

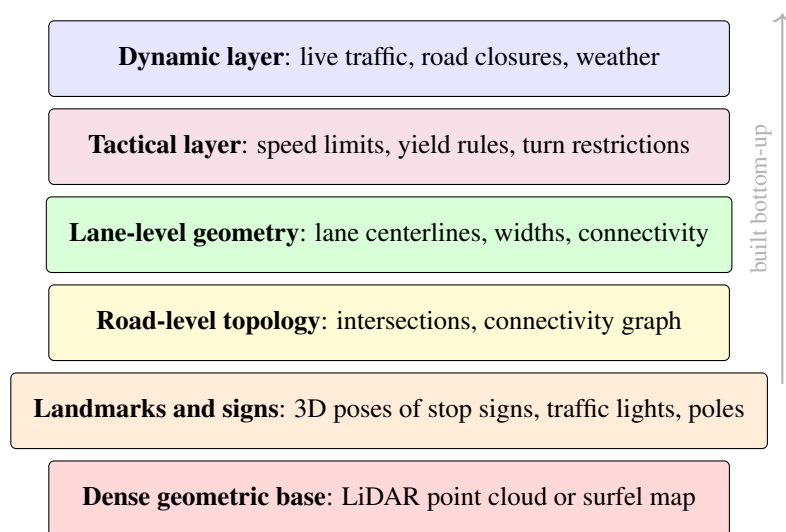


Figure 14.1: Layered HD-map contents. The dense geometric base (LiDAR point cloud or surfel surface) is acquired from survey vehicles and underlies everything. Landmarks (traffic signs, poles) are annotated on top; road-level topology captures intersection structure; lane-level geometry captures the precise path of each lane; the tactical layer encodes rules (speed limits, yields); the dynamic layer carries live updates (closures, traffic, weather). Layers are built bottom-up and consumed top-down by the planning stack.

A production HD map’s contents, layer by layer:

- **Dense geometric base.** A 3D point cloud (LiDAR survey), often stored as a voxel grid or surfel representation. Typical resolution: 10 cm voxels. This is the largest layer by data volume: a city of

500 km² takes a few terabytes at this resolution.

- **Landmarks and signs.** 3D poses of stable features: traffic signs (with class), poles, mailboxes, curbs. Used for semantic localization when LiDAR matching is impractical (short-range cameras only).
- **Road-level topology.** Directed graph of road segments and intersections. Navigation routing uses this layer.
- **Lane-level geometry.** Per-lane centerlines as polylines in 3D, with per-lane attributes (direction, width, type). Topology between lanes: successors, predecessors, left / right neighbors.
- **Tactical layer.** Speed limits, yield rules, turn restrictions, associated traffic lights. The planner consumes this directly.
- **Dynamic layer.** Live information: closures, events, temporary restrictions. Updated continuously through infrastructure feeds and fleet observations.

Example 14.1 (Storage footprint of an HD map). A city-scale HD map for a 100 km² operating area. Dense geometric base at 10 cm voxels: 10⁶ voxels per km², perhaps 16 B per voxel (occupancy + surfel normal + uncertainty) = 1.6 GB per km² ⇒ 160 GB. Compressed to 50 GB. Other layers (landmarks, lanes, tactical): ~ 2 GB combined. Production vehicles typically load map tiles on demand along the planned route rather than the whole city at once, reducing on-vehicle memory to a few hundred megabytes at any moment.

14.3 Map Construction Pipelines

Building an HD map is a multi-stage data pipeline. Figure 14.2 shows the canonical flow.

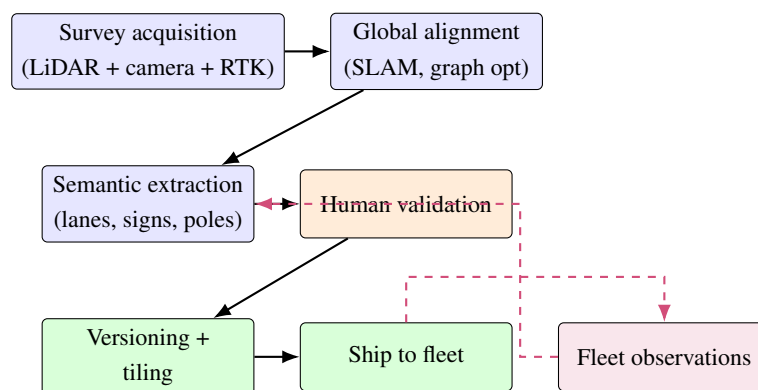


Figure 14.2: HD-map construction pipeline. Dedicated survey vehicles capture LiDAR, camera, and RTK-GNSS data over the target area. Global alignment (SLAM, pose-graph optimization) produces a coherent large-scale map. Automated semantic extraction identifies lanes, signs, and landmarks; human validation corrects errors. Versioning and tiling prepares the map for on-vehicle use. Fleet observations (purple, dashed) feed a continuous update loop that keeps the map fresh.

14.3.1 Survey Acquisition

Dedicated “mapping vehicles” carry high-end sensor suites: multiple 64 to 128-beam LiDARs, surround cameras, RTK GNSS, tactical-grade IMU, wheel encoders. They drive the target area several times to ensure coverage under varying weather/lighting. The raw data per vehicle is on the order of 10 GB min^{-1} ; a survey campaign produces petabytes of raw input.

14.3.2 Global Alignment

Individual drives are fused into a globally consistent map via large-scale SLAM. Loop closures across drives ensure consistency; RTK-GNSS anchors the map to a global geodetic frame. The resulting map is a collection of aligned trajectories, not a single drive.

Example 14.2 (Alignment precision from multi-drive SLAM). Ten survey drives over the same 20 km road section. Per-drive trajectory std (after RTK + VIO): 3 cm. Loop closures across drives (each pair matches at 50+ common landmarks) reduce between-drive std to 2 cm. Per-point map std after multi-drive aggregation: 1 cm—below the 10 cm target for automotive HD maps. Maps go through rigorous QA before release.

14.3.3 Automated Semantic Extraction

Deep-learning models (Chapter 12) run on the aligned data to produce:

- Lane-line polylines (segmentation + polyline fit).
- Traffic-sign class and pose.
- Stop-line and crosswalk polygons.
- Pole and traffic-light 3D poses.
- Drivable-surface and curb-edge extraction.

14.3.4 Human Validation

Even state-of-the-art automated extraction has $\sim 95\%$ accuracy; the remaining 5% (obscured signs, unusual lane configurations) must be corrected by human annotators. Modern annotation tools combine foundation-model pre-labeling (Chapter 12) with skilled human review; typical throughput is 1 km per annotator-hour for lane-level review.

14.3.5 Crowd-Sourced Updates

Once a fleet is deployed, the most efficient way to keep the map fresh is to use the fleet itself. Vehicles flag discrepancies with the current map (a stop sign that appears where none was recorded, a lane painted differently, a closed-lane-line cone configuration) and upload the evidence. Update-candidate

clips are validated by humans or by confirming the discrepancy across multiple vehicles before the map is patched.

Example 14.3 (Fleet-observed change detection). A fleet vehicle sees a new stop sign at an intersection that the current map did not record. The vehicle’s on-board change detector flags this discrepancy, uploads a 10 s clip. The cloud service correlates: 3 other vehicles in the next week see the same sign at the same location. Detection confirmed. A labeler updates the map within 48 hours; updated tiles roll out to the fleet. Without a crowd-sourcing loop, the same update would require a survey vehicle to re-drive the area—orders of magnitude more expensive.

14.4 Map-Based Localization

Given an HD map and current sensor data, the vehicle estimates its 6-DoF pose $\mathbf{T}_{WB} \in \text{SE}(3)$ against the map. Figure 14.3 shows the canonical pipeline.

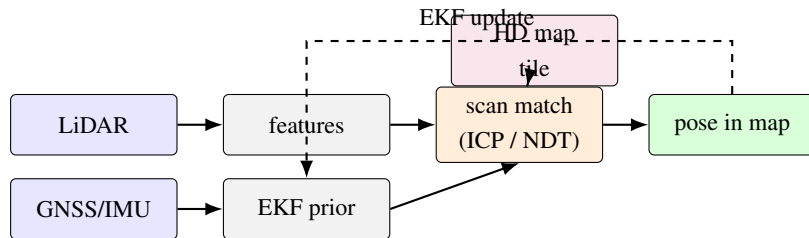


Figure 14.3: Map-based localization pipeline. GNSS/IMU produces a prior pose estimate with coarse (1–2 m) accuracy. The LiDAR scan (or camera-derived features) is aligned to the map tile containing the estimated region via ICP or NDT, refining the pose to centimeter-level accuracy. The result updates the EKF, closing the loop. The dashed feedback makes the pipeline a running estimator rather than a one-shot matcher.

14.4.1 LiDAR Point-Cloud Matching

ICP (Chapter 13) aligns the live LiDAR scan to the map point cloud. Initial pose guess from GNSS/IMU; ICP iterates to convergence. Typical convergence: 5–15 iterations, producing 5 cm-level pose accuracy.

Example 14.4 (ICP convergence against a map). Live scan with 50 000 points after voxel downsampling. Initial pose guess from GNSS: 1 m off true pose. Iteration 1: point-to-plane residual sum 5000 m². Iteration 3: 50 m². Iteration 10: 0.1 m². Converged pose std 3 cm. A typical match runs in 15 ms on production hardware. Without a good initial guess, ICP can land in a local minimum (aligned against a wrong section of map); GNSS/IMU priors prevent this.

14.4.2 Normal Distributions Transform

NDT [4] represents the map as a set of Gaussians over voxels, each with mean and covariance of the enclosed points. A live scan is aligned by maximizing the sum of Gaussian likelihoods:

$$s(\mathbf{T}) = \sum_i \exp\left(-\frac{1}{2}(\mathbf{T}\mathbf{p}_i - \boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}_i^{-1}(\mathbf{T}\mathbf{p}_i - \boldsymbol{\mu}_i)\right). \quad (14.1)$$

NDT is smoother than ICP and tolerant of poor initial alignment, but computationally heavier per iteration. The smoothness makes gradient-based optimization work well.

Example 14.5 (NDT score vs pose offset). A live scan aligned to an NDT map has score $s = 45,000$ at the correct pose. Translating 10 cm: $s = 44,000$. Translating 50 cm: $s = 32,000$. Translating 1.5 m: $s = 8,000$. The score is smooth and has a well-defined maximum; gradient-based search converges from 1 m initial offset in ~ 5 iterations. Relative to ICP's discrete nearest-neighbor association, NDT's Gaussian aggregation is more robust at moderate misalignment but less precise at very tight tolerances.

14.4.3 Semantic-Landmark Matching

When LiDAR is unavailable or when computational budgets are tight, semantic-landmark matching aligns a small number of detected landmarks (traffic signs, poles, lane markings) to their map counterparts. The cost function is a sum of weighted distances between detected landmarks and matched map landmarks.

Example 14.6 (Landmark matching reduces state). Vehicle sees 3 detected traffic signs at relative poses $(20.1, 3.2), (40.5, -2.1), (60.0, 4.5)$ m. Map has signs at world positions $(120.1, 53.2), (140.5, 47.9), (160.0, 54.0)$ m. Matching by class and proximity gives three pose constraints on $\mathbf{T}_{WB}$. Solve: $\mathbf{t}_{WB} \approx (100, 50, 0)$, $\theta = 0$. With only 3 point-pair constraints, the solve is well-posed for the planar case and runs in microseconds. Even this sparse approach achieves sub-meter accuracy when the landmark set is reliable.

14.4.4 Visual Localization

For camera-only stacks, map matching uses either image retrieval (find the most similar map image, estimate relative pose against it) or feature-based matching of camera features against stored 3D landmarks. Visual localization is less precise than LiDAR match (typical 20 cm to 50 cm std vs 5 cm) but adequate when combined with other constraints.

14.4.5 Fusion with GNSS/IMU

The final pose estimate combines the map-matching update (10 Hz) with GNSS/IMU propagation (100 Hz+) in an EKF or factor-graph. The map match provides absolute reference; the IMU propagates smoothly between matches; GNSS provides an outer-loop check against gross matching errors (e.g., matching to the wrong section of map).

Example 14.7 (Pose covariance after multi-rate fusion). IMU at 200 Hz: propagates pose with noise that grows linearly in time; at steady state, position std 2 cm between 10 Hz LiDAR updates. LiDAR map match at 10 Hz: position std 3 cm per match. Fused output std: dominated by the LiDAR match between updates, falling to the IMU-propagated value just after a match. Effective std averaged over the control loop: 2 cm—well under lane-level tolerance.

14.5 Mapless and Map-Light Driving

Driving programs split roughly into two camps: *map-first* (Waymo, Cruise, Zoox, Mobileye REM) build extensive HD maps and rely on them; *map-light* (Tesla FSD, newer Wayve approaches) operate with minimal map priors and derive most information from live perception.

- **Map-first** pros: centimeter-level precision, known constraints, robust to unusual scenes. Cons: build/freshness cost, geographic lock-in to mapped areas.
- **Map-light** pros: generalization to unmapped areas, lower operational cost. Cons: harder perception problem, lower accuracy in corner cases.

No architectural consensus exists yet. Production robotaxi programs in 2025–2026 lean map-first; consumer L2–L3 programs lean map-light. The frontier is a middle ground: *map-light but map-aware*, using coarse priors (satellite imagery, fleet-aggregated hints) without full HD-map construction.

Example 14.8 (Map-light perception compensation). An AV arrives at an intersection the map does not cover. Instead of relying on map-stored speed limits and lane geometry, the vehicle reads traffic signs and lane markings in real time; the stack must produce a plan from zero prior. Detection accuracy at long range drops by 15 % to 25 % relative to the mapped case (no geometric priors, fewer redundant cues). The vehicle compensates by slowing down, reserving extra margin, and refusing aggressive maneuvers until more cues accumulate. This is the operational tax of map-lightness: lower availability within an ODD, but much wider ODD.

14.6 Map Versioning and Freshness

A map is software. It has versions, dependencies, and a CI/CD pipeline. Map tiles are versioned at fine granularity (100 m blocks); each vehicle tracks which version of which tile it is running against; updates are pushed incrementally as new versions are validated. Roll-back capability is essential because a bad map version can cause widespread misbehavior.

Freshness is measured in SLA terms: *how long does it take from a real-world change being observable to it being reflected in the production map?* Typical targets: hours for high-impact changes (new stop signs, closures), days for geometric changes (repaved roads), weeks for semantic updates.

Example 14.9 (A freshness failure and its cost). A lane is re-striped overnight; the map is not updated until 48 hours later. During the interim, 20 fleet vehicles drive through the area; each observes a discrepancy and slows down to 10 km h^{-1} below the posted limit per a safety-margin rule that triggers on map-live mismatches. Passengers experience the slowdown as poor ride quality; operational metrics degrade. Fixing freshness-SLA this one case: map update within 6 hours; cost measured in revenue loss, user satisfaction, and fleet utilization. Mature programs track the SLA distribution continuously.

14.7 Privacy and Regulation

HD maps are derived from real-world data that may include personally-identifying information: faces, license plates, private residences. Production pipelines:

- Blur faces and license plates in stored imagery (automated filters with human QA).
- Aggregate crowd-sourced observations so that no single vehicle's trajectory can be reconstructed.
- Comply with GDPR-style regulations: data minimization, on-request deletion, purpose limitation.

Some jurisdictions restrict detailed mapping of public spaces, and regulatory approval for operating an AV is often conditional on map compliance with these rules.

14.8 Worked Examples

Example 14.10 (End-to-end map-based localization). At $t = 0$: vehicle receives a GNSS fix (123.45, 67.89) m in local coordinates with 0.5 m std. IMU propagates for 100 ms; estimated pose (125.95, 67.89, 0) m with position std grown to 0.52 m. LiDAR scan at $t = 100 \text{ ms}$: ICP match against map tile produces pose estimate (125.93, 67.92, 0) with std 0.04 m. EKF update collapses the pose covariance: final posterior std 0.04 m. Pose delivered to planner with calibrated uncertainty. Pipeline latency: 20 ms ICP + 2 ms EKF = 22 ms within the 100-ms localization budget.

Example 14.11 (NDT robustness to seasonal change). A map built in summer is matched against winter scans; snow on the ground changes the LiDAR intensity pattern but largely preserves geometry. NDT score on a correctly-aligned scan: 90 % of summer baseline (slight degradation from snow-induced noise). ICP on the same scan: 70 % of baseline (more local minima from noisy point matches). NDT's smoothness explains the advantage: the Gaussian voxel representation averages out local noise, while ICP's point-by-point matching is sensitive to it. Production systems typically use NDT as the primary matcher and ICP only for final refinement.

Example 14.12 (Map tile streaming cost). Vehicle is about to drive a 5 km route. Tile size: 100 m squares. Route covers ~ 50 tiles. Average tile size: 80 MB (dense geometric base + semantics).

Total: 4 GB. At 50 Mbits^{-1} cellular, download time: 11 min. Pre-caching the route at trip start is therefore operationally required, not optional. Alternatively, tiles are streamed just-in-time as the vehicle approaches each—this halves peak bandwidth but adds complexity.

14.9 Industrial Deployment

The HD-map industry consolidated in the late 2010s and early 2020s. HERE Technologies (jointly owned by Audi, BMW, Daimler, with subsequent investments) supplies HD map data to multiple OEMs. TomTom (Netherlands) supplies HD-maps to several OEMs and supplies the underlying maps for Microsoft Bing Maps. In China, Baidu, AutoNavi (Alibaba), and Tencent are the dominant HD-map providers, operating under specific Chinese regulatory licenses for surveying and mapping. Mobileye's Roadbook is built using crowd-sourced data from millions of vehicles equipped with EyeQ platforms. Tesla has publicly stated that it does not rely on HD maps for FSD, preferring online perception, while Waymo, Cruise, and Chinese robotaxi operators have historically relied on HD maps as a foundational layer. The Apollo open-source project (Baidu) publishes a reference HD-map format used in research and pilot deployments worldwide.

14.10 Chapter Summary

HD maps are layered representations of the driving environment built by survey vehicles, refined through automated semantic extraction plus human QA, and maintained through fleet-observed updates. They provide centimeter-accurate priors that simplify perception and enable precise localization. Map-based localization combines sensor matching (LiDAR ICP, NDT, or semantic landmarks) with GNSS/IMU fusion to produce pose at control-loop rate. The map-first vs map-light architectural debate remains active in industry. Production map programs must solve three additional problems at scale: versioning, freshness, and privacy/regulation. The pose and map data produced by this chapter feed the prediction and planning layers of Part IV.

14.11 Exercises

1. For the HD map storage example (14.1), estimate the storage for a 500 km^2 metro area at 5 cm voxel resolution.
2. Describe three distinct failure modes of LiDAR ICP matching against an HD map, and the mitigation most likely to help for each.
3. Derive the gradient of the NDT score (Eq. 14.1) with respect to a right pose perturbation.
4. A semantic-landmark localizer sees 2 stop signs, 1 traffic light, and 4 lane markings. What is the minimum number of matches needed for a well-posed $\text{SE}(2)$ pose estimate?
5. For the visual localization case where camera features replace LiDAR points, what is the expected accuracy gap and which techniques reduce it?

6. Describe a crowd-sourced update flow that prevents a single vehicle's hallucinated detection from propagating to the production map.
7. For the freshness-failure example (14.9), compute the cumulative passenger delay over 48 hours, assuming 50 trips through the affected area with 5 min per-trip slowdown.
8. Compare the robustness of ICP and NDT under the three seasonal shifts: snow, rain, leaf-off autumn.
9. Propose a map-light architecture that uses no dense map but does use crowd-sourced lane-graph priors. What perception capabilities does it require?
10. For an L4 robotaxi fleet with 100 vehicles operating in a 100 km^2 area, estimate the storage and bandwidth required to keep each vehicle current with map updates at an average of 50 tile-updates per day.

Chapter 15

Wheel Odometry

15.1 Introduction

Long before satellite navigation or cameras, vehicles tracked their position by counting wheel rotations. A wheel of known radius, rolling without slipping, travels a known distance per revolution; counting revolutions integrates the distance traveled. Modern vehicles still use this principle. Wheel odometry is the *cheapest and most fundamental* motion-sensing modality every wheeled AV has, and its readings feed into nearly every other estimator—the IMU, the SLAM backend, the control loop, the fleet-management system. It is the quiet workhorse that appears in none of the marketing material but underlies most of what the vehicle thinks it is doing.

This chapter develops wheel odometry rigorously: the physics of wheel rotation, the encoder hardware that measures it, the kinematic models that convert rotations into vehicle motion, the error sources that accumulate, the calibration practices that bound them, and the fusion patterns that combine wheel data with IMU, GNSS, and visual sensing.

Three observations motivate the chapter’s structure.

First, wheel odometry is *dead reckoning*—it integrates relative motion from a known starting position. With perfect measurements and a perfect kinematic model, it would reconstruct the trajectory exactly. Real wheels slip, deform, and are measured with finite resolution; errors accumulate over distance. Understanding the error structure (bias, random walk, slip-induced jumps) is the key to using wheel odometry well.

Second, wheel odometry is *cheap and reliable*. A wheel encoder costs tens of dollars, requires no external infrastructure, operates in tunnels and garages where GPS fails, and produces measurements at rates (100 Hz to 1000 Hz) that match inertial sensors. This is the sensor that makes localization work when every other modality is degraded; giving it up for a cleaner architectural story is a costly choice.

Third, wheel odometry is *one of many inputs*, not a stand-alone localization solution. Modern AV

stacks fuse wheel odometry with IMU, GNSS, LiDAR, and vision in a single estimator. The fusion makes each sensor better: wheel odometry bounds IMU drift during short intervals; IMU bounds wheel odometry drift during slip; GNSS bounds both over long distances; visual features resolve absolute position near known landmarks.

The chapter is organized as follows: first, the physics of wheel rotation and encoder hardware; then, kinematic models for different drive architectures; then, error sources and their characterization; calibration techniques follow; dead reckoning and its drift analysis; and finally fusion patterns that combine wheel data with complementary sensors. Two architecture diagrams visualize the geometry and the fusion architecture; worked examples quantify every formula.

15.2 The Physics of Wheel Rotation

Wheel odometry rests on a simple physical relation: a wheel rolling without slipping over distance d rotates by an angle $\phi = d/r$ for wheel radius r . Measure ϕ ; infer d . Every subtlety in practical wheel odometry is about the ways this simple relation can fail.

15.2.1 Rolling Without Slipping

For a rigid wheel on a rigid surface with no lateral motion, the contact point is instantaneously stationary. The wheel's angular velocity ω relates to the vehicle's longitudinal velocity v by

$$v = r\omega, \quad (15.1)$$

where r is the effective rolling radius. Angular displacement over time $[0, t]$:

$$d = \int_0^t r\omega(\tau)d\tau = r\phi. \quad (15.2)$$

Example 15.1 (Distance from encoder count). A wheel has effective radius $r = 0.336$ m and an encoder producing 1024 ticks per revolution. In a measurement interval, the encoder records 342 ticks. Angular displacement: $\phi = 342/1024 \cdot 2\pi = 2.098$ radians. Distance: $d = r\phi = 0.336 \cdot 2.098 = 0.705$ m. At 100 Hz update rate, this corresponds to an instantaneous speed of 70.5 m s^{-1} —far above any automotive speed, so this is an implausible reading. The number of ticks per 10 ms interval at typical highway speed (30 m s^{-1}) is only $30/0.336/2\pi \cdot 1024/100 \approx 145$ ticks. Realistic tick rates drive the selection of encoder resolution.

15.2.2 Effective Rolling Radius

The *effective* rolling radius differs from the nominal tire radius for three reasons. A deflated tire has smaller effective radius. A loaded vehicle compresses the tires more than an empty one. Speed-dependent centrifugal force reduces compression at high speed. Production wheel odometry accounts for all three; high-precision applications measure the effective radius as a slowly-varying state.

Example 15.2 (Radius error and distance bias). Nominal tire radius: 0.336 m. Actual effective radius under current load: 0.332 m (a 1.2 % error). Distance reported after 10 km of travel: $10000 \cdot 0.336 / 0.332 = 10,120\text{m}$ —a 120 m bias. Over a city-scale route, this is a substantial positional offset. Production systems re-estimate effective radius online from GNSS-anchored speed readings.

15.3 Encoder Hardware

A wheel encoder measures rotational position (or velocity, or both) of a wheel or driveshaft. Several technologies dominate automotive use.

15.3.1 Optical Encoders

A slotted or patterned disk rotates between an LED and a photodetector; the alternating light/dark pattern produces pulses as the disk rotates. Optical encoders are precise (thousands of ticks per revolution are achievable) but require sealed enclosures for automotive conditions.

15.3.2 Magnetic Hall-Effect Encoders

A multipole magnet rotates past Hall-effect sensors; the sensors' output alternates as poles pass. Robust to dust and moisture; typically lower resolution than optical (tens to hundreds of ticks per revolution). The dominant automotive choice, including for ABS wheel-speed sensors.

15.3.3 Resolvers

A rotary transformer with primary and secondary windings produces sinusoidal outputs whose ratio encodes angle. High accuracy and robust to harsh environments; used in heavy vehicles, motorsport, and electric vehicle motor-shaft applications.

15.3.4 ABS Wheel-Speed Sensors

Every modern vehicle has ABS wheel-speed sensors on all four wheels. These are Hall-effect or inductive sensors reading a gear-tooth ring at the wheel hub. Modest resolution (typically 40–100 teeth) but ubiquitous and readable from the vehicle CAN bus. AV programs routinely use these for low-cost wheel odometry.

Example 15.3 (Quantization error at low speed). An ABS sensor has 48 teeth per wheel revolution and a nominal radius of 0.336 m. Distance per tick: $2\pi \cdot 0.336 / 48 = 0.044\text{m}$. At 1 m s^{-1} vehicle speed, tick rate is 22.7 Hz. Fast enough for routine control, but at very low speeds (parking, stop-and-go), the quantization error is substantial— $\pm 22\text{mm}$ per tick interval. Systems that need precise low-speed localization supplement wheel odometry with other sensors or higher-resolution encoders.

15.4 Kinematic Models

Given measurements of individual wheel rotations, how does the vehicle infer its motion? The answer depends on the drive geometry.

15.4.1 Differential-Drive Model

Two independently-driven wheels on a common axle. Used in mobile robots and some industrial vehicles, less common in passenger cars but the foundational model.

Forward Kinematics

Let v_L, v_R be the left and right wheel linear velocities, w the wheel separation (track width). Vehicle linear velocity and angular velocity:

$$v = \frac{v_L + v_R}{2}, \quad \omega = \frac{v_R - v_L}{w}. \quad (15.3)$$

Example 15.4 (Turn rate from wheel-speed difference). Differential-drive robot: $w = 0.6\text{ m}$, $v_L = 1.0\text{ ms}^{-1}$, $v_R = 1.2\text{ ms}^{-1}$. Linear velocity: 1.1 ms^{-1} . Angular velocity: $(1.2 - 1.0)/0.6 = 0.333\text{ rad s}^{-1}$ — 19° s^{-1} . Over 3 seconds, heading changes by 1 rad (57°), and the robot traces a circular arc of radius $v/\omega = 3.3\text{ m}$. This closed-form arc geometry is the backbone of differential-drive navigation.

15.4.2 Ackermann Steering Model

Cars use Ackermann steering: the front wheels turn at angles calibrated so all wheels trace concentric arcs around a common instantaneous center. The bicycle model (Chapter 20) is the standard Ackermann abstraction: a single front wheel at steering angle δ represents both front wheels, similarly for the rear.

Bicycle Kinematic Updates

Given wheel rotation rate ω_w at the rear wheels (the vehicle's drive source for rear-wheel drive) and steering angle δ :

$$v = r \omega_w, \quad \dot{\theta} = \frac{v \tan \delta}{L}, \quad (15.4)$$

where L is the wheelbase. Position evolves by

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta. \quad (15.5)$$

Example 15.5 (Ackermann path integration). Ackermann-steered vehicle: $v = 5\text{ ms}^{-1}$, $\delta = 0.1\text{ rad}$, $L = 2.7\text{ m}$. Over 0.1 s : $\dot{\theta} = 5 \tan(0.1)/2.7 \approx 0.186\text{ rad s}^{-1}$, heading change 0.0186 rad (1.07°). Starting heading $\theta_0 = 0$: $\Delta x \approx 5 \cdot 0.1 \cdot \cos(0.0093) \approx 0.50\text{ m}$, $\Delta y \approx 5 \cdot 0.1 \cdot \sin(0.0093) \approx$

0.0046 m. Integrating forward over time builds the trajectory. Error accumulation depends on noise in v , δ , and the integration step size.

15.4.3 Four-Wheel Independent

In four-wheel-drive or electric-vehicle configurations with independently-driven wheels, each wheel's rotation is measured. The vehicle motion can be estimated from any subset; redundant measurements support fault detection and slip identification.

Slip Detection from Wheel Disagreement

Under no-slip assumptions, the four wheel-speeds should be consistent with a single vehicle velocity and yaw rate (with known geometry). Persistent inconsistency signals slip. A classic use: during hard acceleration, wheel-speeds on the driven axle exceed those on the undriven axle; traction-control uses this inconsistency to reduce throttle.

15.5 Error Sources

Real wheel odometry has systematic and stochastic errors that compound over distance.

15.5.1 Systematic Errors

Wheel-Radius Error

The largest single error source. Even with careful calibration, tire-radius drift over temperature, wear, and load produces 0.5 % to 2 % distance errors. Growth: linear in distance.

Wheelbase/Track-Width Error

Geometric constants between wheels affect angular-velocity estimates. Small errors here produce bias in heading rate; heading error grows linearly, and cross-track error grows quadratically.

Steering-Angle Sensor Bias

For Ackermann models, a zero-offset error in the steering measurement biases the estimated yaw rate. Persistent heading drift on straight roads is the symptom.

15.5.2 Stochastic Errors

Encoder Quantization

Finite tick count per revolution produces a position uncertainty equal to the tick-distance divided by 2. For a 48-tooth ABS sensor with 0.336 m wheel radius, this is ± 22 mm per measurement.

Wheel Slip

Tire slip during braking, acceleration, or cornering violates the rolling-without-slipping assumption. Slip can be transient (milliseconds during a hard stop) or sustained (icy road, wheel-spin on loose gravel). Slip is the most safety-critical error source because it is large and unpredictable.

Road-Surface Variations

Rough surfaces produce oscillating effective radius as tires deform. Over smooth asphalt, this is negligible; on cobblestones or washboard dirt, it adds substantial noise.

15.5.3 Drift Growth Model

Under reasonable assumptions, wheel-odometry position error grows with distance traveled. Empirical rule of thumb:

$$\sigma_{\text{pos}}(d) \approx \alpha \cdot d, \quad (15.6)$$

with $\alpha \approx 0.5\text{--}2\%$ depending on sensor quality and surface conditions. Heading error grows similarly. Position drift accumulated over a 10 km trip: tens to hundreds of meters. Unaided wheel odometry is unusable for long-distance navigation without external correction.

Example 15.6 (Drift analysis for a 5 km trip). Drift coefficient $\alpha = 0.01$ (1 %). After 5 km, expected position std: 50 m. Over a freeway drive this is modest; over an urban drive with turns and stops, slip accumulates faster, and real-world drift can be double the rule-of-thumb value. Wheel odometry alone cannot sustain reliable localization over such distances; fusion with GNSS and map-matching is mandatory.

15.6 Calibration

Good wheel odometry requires accurate geometric constants: effective rolling radius of each wheel, wheelbase, track width, steering-angle zero offset. All drift over time; calibration must be periodic or continuous.

15.6.1 Static Calibration

Measure geometric constants directly: tape-measure the track width and wheelbase, measure tire circumference by rolling a known distance. Good starting point; does not account for dynamic variations.

15.6.2 Trajectory-Based Calibration

Drive a known path (figure-eight, circle) and compare measured trajectory to ground truth. Systematic errors (wheel-radius differential, wheelbase error) produce characteristic patterns that a least-squares fit extracts. The UMBmark procedure (Borenstein & Feng, 1996) is the classic reference.

15.6.3 Online Calibration via GNSS Fusion

During GNSS-good operation, the fused estimator observes both GNSS-anchored velocity and wheel-inferred velocity; the difference estimates effective-rolling-radius error. Over time, the estimator converges on a calibrated radius without explicit calibration procedure.

Example 15.7 (Online radius correction). Initial assumed radius: 0.336 m. GNSS-derived vehicle speed: 20.0 m s^{-1} . Wheel-angular-speed measurement at current radius: 59.52 rad s^{-1} (gives implied 20.0 assumed-radius speed). Actual observed wheel-speed: 60.24 rad s^{-1} . Implied effective radius: $20.0/60.24 = 0.332 \text{ m}$ —consistent with an under-inflated tire. Estimator update: moving-average the inferred radius. Future wheel-odometry readings use the corrected value.

15.7 Dead Reckoning

Given calibrated measurements, wheel odometry integrates to produce a running position estimate. This is *dead reckoning*: propagation of relative-motion data from a known starting point.

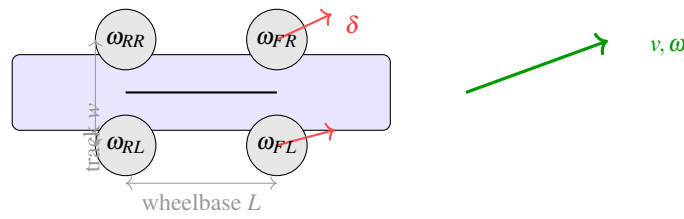


Figure 15.1: Geometric parameters for wheel odometry on an Ackermann- steered vehicle. Wheel angular rates $\omega_{RL}, \omega_{RR}, \omega_{FL}, \omega_{FR}$ are measured at each wheel. Track width w separates the left and right wheels; wheelbase L separates front and rear axes. The front wheels steer at angle δ . Vehicle linear velocity v and yaw rate ω follow from the kinematic model.

15.7.1 Position Integration

At each measurement step:

1. Read wheel rotations $\Delta\phi_i$ for each wheel i .
2. Compute per-wheel distances $\Delta d_i = r_i \Delta\phi_i$.
3. Apply kinematic model to get $\Delta v, \Delta\theta$ for the vehicle.
4. Update pose: $\theta \leftarrow \theta + \Delta\theta, x \leftarrow x + \Delta d \cos \theta, y \leftarrow y + \Delta d \sin \theta$.

15.7.2 Uncertainty Propagation

Each step's pose uncertainty grows by propagating the previous covariance forward plus the measurement noise covariance. The result is an EKF-style update in which the wheel-odometry measurement is a relative-motion constraint between consecutive poses.

15.8 Fusion with Other Sensors

Figure 15.2 shows the canonical fusion architecture used in production AV stacks.

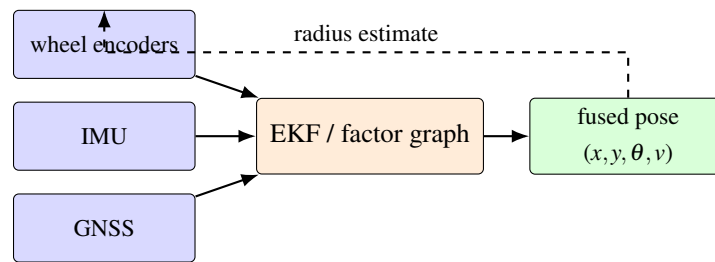


Figure 15.2: Wheel-odometry fusion architecture. Wheel encoders report angular rates; IMU reports specific force and angular velocity; GNSS reports absolute position (when available). An EKF or factor-graph estimator fuses all three and produces a smoothed pose estimate. Feedback from the estimator updates the wheel-radius calibration online, correcting systematic bias that would otherwise accumulate as drift.

15.8.1 Wheel-IMU Fusion

At short timescales, wheel-odometry and IMU are highly complementary. Wheel-odometry reads longitudinal speed directly; IMU integrates acceleration with bias drift. A fused estimator uses wheel-odometry to anchor IMU bias and IMU to interpolate between wheel ticks.

15.8.2 Wheel-GNSS Fusion

GNSS provides absolute position but at 1 Hz to 10 Hz with meter-level noise. Wheel odometry provides smooth, high-rate relative motion. Fusion: GNSS anchors the trajectory long-term; wheel odometry fills in between fixes.

15.8.3 Wheel-SLAM Fusion

In SLAM (Chapter 13), wheel odometry provides a relative-motion prior between keyframes. The prior is tighter than pure IMU integration, reducing optimization work and improving convergence. In LiDAR SLAM, wheel odometry is often the primary relative-motion source; in visual-inertial SLAM, it complements IMU.

Example 15.8 (Fusion gain from wheel-IMU combination). Short-baseline inter-keyframe pose estimate. IMU alone: position std 0.12 m over 0.5 s (random-walk accumulated from accelerometer noise). Wheel odometry alone: 0.05 m std (quantization + slip). Fused: std drops to 0.02 m. The gain is from each sensor covering the other’s weak mode—IMU’s long-term drift and wheel odometry’s slip sensitivity.

15.9 Specific Production Usages

Three places wheel odometry appears in a modern AV stack.

15.9.1 Low-Level Control

The controller (Chapter 20) uses wheel-speed for velocity feedback. ABS/traction control/stability control use per-wheel speeds for slip detection. These are safety-critical paths; wheel sensors must be redundant and monitored.

15.9.2 State Estimator

The EKF / factor-graph state estimator consumes wheel-odometry as a relative-motion measurement; together with IMU and GNSS it produces the pose used by perception and planning.

15.9.3 SLAM Backend

SLAM uses wheel-odometry as the initial odometry prior between keyframes. This is particularly valuable in GNSS-denied regions where wheel-IMU dead-reckoning provides the only continuous motion signal.

15.10 Failure Modes in Production

What can go wrong with wheel odometry, and how is it caught?

15.10.1 Stuck Wheel Due to Ice or Mud

A wheel that is not rotating but the vehicle is moving (because other wheels are driving) produces zero wheel-rate while actual distance advances. The four-wheel consistency check catches this: one wheel reading zero while the others agree is a clear inconsistency signal.

15.10.2 Wheel Lockup Under Hard Braking

All four wheels briefly stop rotating while the vehicle decelerates. Detected by ABS; wheel-odometry during lockup is dropped in favor of IMU propagation.

15.10.3 Encoder Failure

Hardware failure in one wheel sensor. Detection: sustained inconsistency with other wheels plus IMU. Fall-back: use the other three wheels.

15.10.4 Sustained Slip on Loose Surfaces

Gravel, snow, mud: wheels partially slip and the vehicle's reported distance exceeds actual. Visual-odometry or GNSS, where available, flags this by disagreeing with wheel measurements over the slip window.

15.11 Industrial Deployment

Wheel-tick odometry is among the oldest and most reliable production sensing technologies on automobiles. ABS (antilock-braking) and ESC (electronic-stability-control) systems have shipped wheel-speed sensors as standard automotive equipment for decades. Bosch, Continental, ZF, Denso, and Aisin are among the major suppliers. Wheel-tick data is published on production vehicle CAN buses (often through SAE J1939 or proprietary protocols) and is a primary input to ABS, ESC, traction control, and to all production autonomous-driving stacks. The accuracy and latency characteristics described in this chapter reflect production-typical behaviour for vehicles certified under FMVSS 126 (ESC) in the U.S., ECE R13H/R140 in Europe, and equivalent regulations elsewhere.

15.12 Chapter Summary

Wheel odometry is the cheapest, most-reliable, most-universal motion-sensing modality in any wheeled AV. It integrates wheel rotations into distance using the rolling-without-slipping assumption. Encoder hardware spans optical, Hall-effect, resolver, and ABS-style implementations. Kinematic models (differential-drive, Ackermann, four-wheel independent) convert per-wheel measurements to vehicle motion. Systematic errors (radius, wheelbase, track width) dominate at small scale; stochastic errors (slip, quantization, surface variation) dominate at large scale. Unaided drift grows roughly 1 % of distance; fusion with IMU, GNSS, and visual sensors bounds it. Production stacks use wheel odometry as a foundational measurement in control, state estimation, and SLAM; failure modes (stuck wheels, lockup, slip) are handled by cross-sensor consistency checks. The humble wheel encoder is a silent contributor to almost every AV operation.

15.13 Exercises

1. For the distance-from-encoder example, compute the tick rate at 10 m s^{-1} vehicle speed with 0.336 m wheel radius and 512 ticks per revolution.
2. An ABS sensor has 40 teeth, wheel radius 0.29 m. Compute the minimum detectable speed if the tick interval must be at least 20 ms for reliable measurement.
3. Derive the differential-drive kinematic equations from geometric first principles, stating any assumptions.
4. A vehicle has wheel-radius error of 0.8 % in the right rear wheel only. What systematic error appears in the trajectory after driving a straight line for 1 km?
5. For the Ackermann path example, compute the trajectory over 10 seconds with $v = 8 \text{ m s}^{-1}$, $\delta = 0.05 \text{ rad}$.
6. Describe three operational scenarios where wheel odometry outperforms IMU-only dead reckoning.

7. A wheel slips 5 % during a 30 s episode of hard acceleration. Quantify the resulting position error if wheel odometry is unaided during the slip.
8. Design a sanity check that uses four-wheel consistency to detect a single-wheel encoder failure. What is the detection latency?
9. Explain why wheel-odometry radius calibration via GNSS cannot operate during a straight-highway drive but requires driving maneuvers.
10. Compare wheel odometry, IMU-only dead reckoning, and visual odometry for GPS-denied operation in a tunnel. What does each provide?

Chapter 16

Visual Odometry

16.1 Introduction

Cameras see the world. If they see it well, consecutive images encode the motion of the camera that took them: features that moved between frames reveal the rigid transform the camera underwent. *Visual odometry (VO)* is the process of recovering that transform, and by chaining transforms across time, recovering the camera’s trajectory. When the camera is mounted on a vehicle, visual odometry is vehicle odometry: a motion estimate derived entirely from what the vehicle sees, with no GNSS, no map, no external infrastructure. This makes it uniquely valuable in the operational regimes where other localization fails: tunnels, parking garages, urban canyons, off-road terrain, and weather-obscured conditions that defeat other sensors.

This chapter develops visual odometry rigorously. Chapter 13 covered visual SLAM as the larger problem of mapping and localizing simultaneously; this chapter focuses on the narrower and more general task of *tracking motion* from image streams, without necessarily building a map. Visual odometry is a component within SLAM but is also a deployed technology in its own right, running on millions of vehicles in ADAS and robotaxi stacks worldwide.

Three observations motivate the chapter’s structure.

First, visual odometry is *cheap and globally available*. A single camera costs tens of dollars. It operates without external signals, infrastructure, or maps. Wherever the vehicle has light and texture, it has visual odometry. This global availability is what makes VO the fallback modality when GNSS, wheel, and IMU sensors degrade.

Second, visual odometry is *fragile in specific conditions*. Low light, featureless environments (blank walls, snow fields), motion blur from hard acceleration, and rain on the windshield all degrade VO. Production systems treat VO as a primary sensor when conditions are favorable and automatically downgrade its influence when they are not—a classic sensor- fusion pattern.

Third, visual odometry has benefited enormously from *learning*. Classical VO (feature matching +

RANSAC) is still the standard in production; learned VO (DeepVO, TartanVO) is catching up rapidly. Hybrid pipelines (learned features + classical geometry) have largely displaced purely classical methods in research. The production state of the art is a moving target.

The chapter is organized as follows. We first formulate the VO problem in geometric terms. We then develop feature-based VO: feature detection, matching, motion estimation, and bundle-adjustment refinement. Direct VO (intensity-based methods) follows. Monocular vs stereo vs RGB-D setups are compared. Visual-inertial odometry (VIO) combines vision with IMU for the production workhorse configuration. Weather robustness and GPS-denied applications are developed in depth—these are the environments where VO matters most. We close with learned VO and the transition from pipeline to end-to-end. Three architecture diagrams visualize the feature-based pipeline, the VIO fusion, and the weather-adaptive confidence cascade; worked examples apply each method to specific driving scenarios.

16.2 Visual Odometry: Problem Formulation

Given a sequence of images $I_0, I_1, \dots, I_t$ captured by a moving camera, estimate the sequence of camera poses $\mathbf{T}_0, \mathbf{T}_1, \dots, \mathbf{T}_t$ in $\text{SE}(3)$. The relative pose between consecutive frames is the *odometry update*:

$$\mathbf{T}_{t-1,t} = \mathbf{T}_{t-1}^{-1} \mathbf{T}_t. \quad (16.1)$$

Chaining updates reconstructs the trajectory; any error in a single update propagates forward through the chain.

16.2.1 The Core Geometric Relationship

Consider a 3D point $\mathbf{X}$ in the world, observed at consecutive camera poses as pixel coordinates $\mathbf{u}_{t-1}$ and $\mathbf{u}_t$. Under the pinhole model:

$$\mathbf{u}_t = \pi(\mathbf{K} \mathbf{T}_{t-1,t} \mathbf{X}_{t-1}), \quad (16.2)$$

where $\mathbf{X}_{t-1}$ is the point expressed in frame $t-1$, $\mathbf{K}$ is the intrinsic matrix, and π is the perspective division. Given enough such correspondences, the relative pose $\mathbf{T}_{t-1,t}$ is over-constrained and solvable.

16.2.2 Why Visual Odometry Works at All

A single correspondence provides two equations (pixel u, v). The relative pose has six degrees of freedom (rotation, translation). So a minimum of three correspondences could in principle fit the pose, but additional geometry (depth recovery, outlier handling, scale ambiguity) requires more. Practical VO systems use hundreds to thousands of correspondences per frame for robustness.

Example 16.1 (Information content of a VO frame). A typical forward-facing AV camera captures a frame with 500–2000 trackable features at highway speeds. Each feature contributes 2 scalar constraints. With 1000 features, the pose estimation problem has roughly 2000 constraints for 6

unknowns—massively over-determined. Under ideal conditions the resulting pose has sub-degree rotation and few-millimeter translation precision, for the relative pose between consecutive frames at 30 Hz.

16.2.3 Scale Ambiguity in Monocular VO

A monocular (single-camera) system cannot determine absolute translation scale from images alone. Multiply the translation by any positive scalar, and the same images are produced (the whole scene scales with the translation). Resolving scale requires external information: a second camera (stereo), depth sensor, IMU, or known metric features (lane widths, vehicle sizes).

Example 16.2 (Monocular drift due to scale). Monocular VO estimates the translation direction precisely but not its magnitude. Over short baselines (a few meters), scale is typically estimated from continuity with prior frames. Over kilometers, small scale errors accumulate: a 1 % scale error over 10 km produces 100 m of drift. Monocular systems usually augment with IMU (for acceleration-scale) or known features (known-width lane markings) to bound scale drift.

16.3 Feature-Based Visual Odometry

The dominant classical approach. Detect features in each frame, match to previous frame, estimate relative pose from matches, refine.

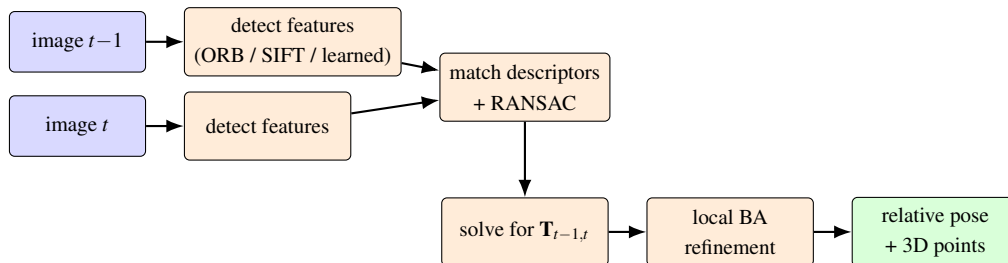


Figure 16.1: Feature-based visual odometry pipeline. Each image is processed to detect sparse features (ORB, SIFT, or learned SuperPoint-style). Descriptors from consecutive frames are matched; RANSAC rejects outlier matches. A minimal-case solver (5-point or 8-point algorithm) computes the initial relative pose; local bundle adjustment refines pose and triangulated 3D points jointly. Output feeds the trajectory chain and downstream SLAM or localization layers.

16.3.1 Feature Detection

ORB, SIFT, and Their Kin

Classical feature detectors identify locally-distinctive image regions and compute a descriptor that characterizes each. ORB (Oriented FAST + Rotated BRIEF) and SIFT (Scale-Invariant Feature Transform) are the standards; SURF, BRISK, AKAZE are alternatives. All produce keypoint coordinates and binary or float descriptors.

Learned Features

SuperPoint, DISK, R2D2 train networks to produce keypoints and descriptors end-to-end. They outperform classical detectors on benchmarks, particularly under challenging illumination or viewpoint changes.

Example 16.3 (Feature count per automotive frame). A forward camera at highway, 1920×1080 , ORB extractor with default settings: produces roughly 1500 keypoints. SuperPoint at the same resolution: 1000 keypoints (with higher repeatability). A typical automotive frame at moderate speed produces 800–2000 usable tracks after matching; fewer than 200 suggests a feature-poor scene (uniform wall, fog, night) and signals VO degradation.

16.3.2 Matching

Descriptor Matching

Compare descriptors between frames; for each keypoint in frame t , find the nearest neighbor in frame $t - 1$. Ratio test (Lowe's criterion): accept a match only if the second-nearest neighbor is sufficiently distant in descriptor space.

RANSAC Outlier Rejection

Matches contain outliers (feature confusion, moving objects, reflections). RANSAC (Random Sample Consensus) fits a geometric model (epipolar or homography) to random minimal samples and retains the model with the most inliers.

Example 16.4 (RANSAC iterations and success probability). Typical 5-point RANSAC: pick 5 matches randomly, fit essential matrix, count inliers. With 60 % inlier rate in the match pool, the probability that a random 5-sample is all-inliers is $0.6^5 = 0.078$. To achieve 99.9 % probability of finding at least one all-inlier sample, need roughly $\log(0.001) / \log(1 - 0.078) \approx 86$ iterations. Each iteration is cheap; production implementations run hundreds.

16.3.3 Initial Pose Estimation

With inlier matches, solve for the relative pose using a minimal- case solver. Two standards:

Five-Point Algorithm

Calibrated cameras (known intrinsics): essential matrix has 5 degrees of freedom. Five matches solve for E up to a few discrete ambiguities; triangulate a test point to pick the correct solution.

Eight-Point Algorithm

Uncalibrated or alternative formulation: fundamental matrix has 7 (or 8 depending on constraints) DoF. Eight matches solve directly; more numerically stable in many situations.

16.3.4 Local Bundle Adjustment

Refine pose and triangulated 3D points jointly by minimizing reprojection error. Chapter 13 developed bundle adjustment in depth; here it serves as a single-frame-pair refinement on the sliding window of recent frames. The Gauss-Newton iteration converges quickly from a good initial guess (from the minimal solver).

Example 16.5 (Bundle-adjustment refinement magnitude). Initial pose from 5-point algorithm: 5-degree error in rotation, 3 % error in translation direction. After one bundle-adjustment pass on a 200-match inlier set: rotation error 0.2° , translation error 0.4 %. Subsequent iterations diminish further. The refinement is the difference between acceptable and production-quality VO.

16.4 Direct Visual Odometry

Feature-based methods throw away most of each image—only the keypoints are used. Direct methods use raw pixel intensities across patches or regions.

16.4.1 Photometric Error

Given relative pose $\mathbf{T}$ and depth z at each pixel, a pixel $\mathbf{u}_{t-1}$ in the previous frame warps to $\mathbf{u}_t$ in the current frame:

$$\mathbf{u}_t = \pi(\mathbf{K}\mathbf{T}\pi^{-1}(\mathbf{u}_{t-1}, z)). \quad (16.3)$$

The photometric residual at pixel $\mathbf{u}$:

$$r(\mathbf{u}; \mathbf{T}) = I_t(\mathbf{u}_t) - I_{t-1}(\mathbf{u}_{t-1}). \quad (16.4)$$

Minimize sum of squared residuals over many pixels to estimate pose.

16.4.2 DSO and LSD-SLAM

Direct Sparse Odometry (DSO, [20]) and LSD-SLAM are the canonical direct methods. They track pose by minimizing photometric error over sparse pixel samples or dense regions, respectively. Advantages: use more image information (anywhere with intensity gradient), don't need distinctive features, smoother cost surfaces. Disadvantages: sensitive to illumination change; require careful photometric calibration.

Example 16.6 (Direct vs feature-based on low-texture scene). Road scene: a uniform asphalt surface with a stripe of lane marking. Feature-based VO: detects only 8 usable features across the frame—insufficient for robust pose estimation. Direct VO: uses intensity gradient along the lane edge, plus subtle road texture variations; produces usable pose estimate. In scenes with poor feature density, direct methods retain capability that feature-based methods lose.

16.5 Stereo and RGB-D Visual Odometry

Monocular scale ambiguity motivates richer camera configurations.

16.5.1 Stereo VO

Two cameras with known baseline produce a depth measurement per pixel (via disparity). Scale is metric (set by the baseline) and depth is direct rather than triangulated. Stereo VO avoids monocular scale drift and has been the production standard for autonomous cars without aggressive monocular or end-to-end strategies.

Example 16.7 (Stereo depth precision at range). Stereo rig: baseline 40 cm, focal length 1200 pixels, disparity resolution 0.1 pixels. Depth at 20 m: $Z = fb/d = 1200 \cdot 0.4/d$, and 20 m corresponds to $d = 24$ pixels. 0.1-pixel disparity noise: $dZ/dd = -Z/d$, so depth uncertainty $\sigma_Z = (Z/d)\sigma_d = (20/24) \cdot 0.1 = 0.083$ m. Depth at 100 m: $d = 4.8$ pixels; $\sigma_Z = (100/4.8) \cdot 0.1 = 2.1$ m. Stereo depth degrades quadratically with range; beyond 50 m, stereo adds little information for motion estimation.

16.5.2 RGB-D VO

Depth-sensing cameras (Kinect-style, Realsense) produce per-pixel depth directly. Common in indoor robots and some low-speed AV applications; less useful at automotive ranges and outdoor conditions due to depth sensor limitations.

16.6 Visual-Inertial Odometry

Cameras and IMUs are complementary. The camera provides rich scene information at low rate (30 Hz to 60 Hz); the IMU provides metric motion at high rate (100 Hz to 1000 Hz) but with drift. Fusing them produces Visual-Inertial Odometry, the production standard for robust monocular motion estimation.

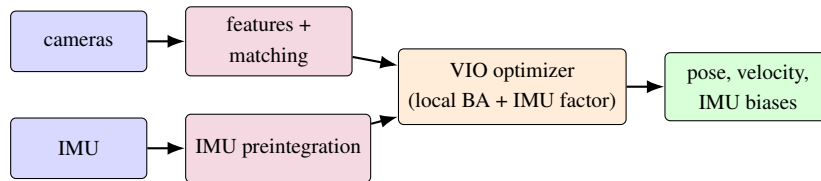


Figure 16.2: Visual-inertial odometry architecture. Visual features from consecutive frames and preintegrated IMU measurements feed a joint optimizer that solves for camera pose, linear velocity, and IMU biases simultaneously. The IMU provides metric scale (which resolves monocular ambiguity), short-timescale smoothness, and graceful degradation through brief visual dropouts. The camera provides long-timescale anchoring and absolute orientation reference.

16.6.1 IMU Preintegration

Integrate IMU samples between camera frames into a single relative-motion constraint (Chapter 13). The preintegrated factor captures all IMU information between two keyframes with bounded computa-

tional cost and correct covariance propagation.

16.6.2 Joint Optimization

A VIO optimizer has variables: camera poses, linear velocities, IMU biases (accelerometer and gyroscope). Factors: reprojection errors (visual), IMU preintegration factors, bias priors. Solved via iterative nonlinear least squares, typically in a sliding window of recent frames.

16.6.3 Initialization

VIO initialization is the hardest phase. The system must recover metric scale (from IMU magnitudes matched to visual geometry), gravity direction, and initial bias estimates, all before the optimizer can run smoothly. Modern VIOs (VINS-Mono, OKVIS, OpenVINS) have robust initialization procedures that converge in 1–5 seconds of vehicle motion.

Example 16.8 (VIO robustness through a brief visual dropout). A tunnel entrance: visual features abruptly dry up as the camera transitions from outdoor lighting to deep shadow for 0.7 s. Monocular VO alone: loses track; recovery requires re-initialization. VIO: IMU preintegration carries the pose estimate through the 0.7 s blackout with accumulated position uncertainty $\sigma \sim 0.3$ m (industrial-grade IMU, short interval). When visual features return at the tunnel interior (illumination adapts), the optimizer re-anchors to the new scene. Continuity is preserved; without VIO, the AV would have had a localization gap during a safety-critical region.

16.7 Weather Robustness

Visual odometry's biggest practical challenge is weather. The section develops the specific failure modes and the production responses.

16.7.1 Rain and Wet Surfaces

Raindrops on the Windshield

Droplets refract nearby regions of the image; feature detectors respond to the droplet edges as false features. Matched across frames, droplet features produce large outliers that RANSAC struggles to reject reliably.

Reflections on Wet Pavement

Wet surfaces reflect headlights, sun, and ambient light. Strong reflections appear as bright features that move inconsistently with the scene. Direct methods are particularly vulnerable.

Production Response

Automatic wipers controlled by rain sensors (Chapter 9) keep droplets cleared during driving. Confidence scaling: rain-sensor signal downgrades VO contribution to the fused estimator. Multi-modal fusion weights radar and LiDAR more heavily; VO is used conservatively.

16.7.2 Snow

Heavy snow fills the scene with feature-poor white regions and creates motion-blur artifacts when snowflakes cross the camera field. Performance degradation is dramatic; production systems often disengage visual-primary modes in heavy snow.

16.7.3 Fog

Contrast reduction degrades feature detection. Features at range effectively disappear; only nearby, high-contrast features remain. VO continues to work but with reduced feature count and correspondingly worse precision.

16.7.4 Low Light and Night

Cameras at night rely on headlights; outside the illuminated cone there is no signal. Thermal cameras (Chapter 8) complement at night; NIR-sensitive cameras extend the effective illumination range.

Example 16.9 (Weather-adaptive VO confidence). Rain-sensor signal drops by 30 % (heavy rain detected). VO's feature-count drops from 1200/frame to 380/frame (droplet and reflection noise). Fused-estimator VO-weight: drops from 0.40 to 0.10. IMU and radar weights increase. Fused pose uncertainty grows by $2\times$ but remains bounded—the system degrades gracefully rather than failing catastrophically.

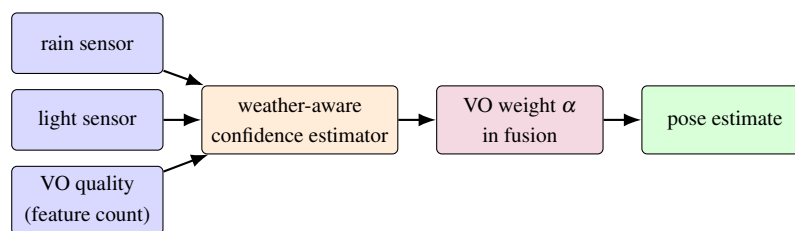


Figure 16.3: Weather-adaptive VO confidence cascade. Environmental sensors (rain, light) plus intrinsic VO quality signals (feature count, RANSAC inlier ratio, residual magnitude) feed a confidence estimator that outputs a weight $\alpha \in [0, 1]$. The weight modulates VO's contribution in the downstream sensor-fusion estimator. Under good conditions α is near 1; under poor conditions it approaches 0 and other sensors dominate.

16.8 GPS-Denied Environments

The operational regime where VO is most valuable. This section surveys the common GPS-denied scenarios AVs encounter.

16.8.1 Tunnels

Complete GNSS loss; length ranges from a few hundred meters (short crossings) to several kilometers (mountain tunnels). VO + IMU carry the vehicle through; wheel odometry provides complementary scale. Well-lit modern tunnels have enough visual features for good VO; older, darker tunnels are harder.

16.8.2 Parking Garages

Multi-level garages: GNSS entirely absent, GPS-like spatial structure repeated across levels. VO provides the only absolute motion reference; map-matching (with a garage map) anchors the trajectory globally.

16.8.3 Urban Canyons

Tall buildings block and reflect GNSS signals, producing multipath errors that look like valid positions but are meters off. VO bridges the multipath intervals, constraining positioning to physically-plausible trajectories.

16.8.4 Mine and Construction Sites

Below-grade operations, dust, metal structures interfering with GNSS. VO combined with dedicated beacons (Chapter 7) supports autonomous mining vehicles. Weather and environmental challenges are severe; VO is augmented rather than replaced.

Example 16.10 (VO-only tracking through a 1.5 km tunnel). Tunnel entry at $t = 0$; exit at $t = 90$ s at 17 m s^{-1} average speed. VO (stereo + IMU) alone accumulates drift during the traversal.

- Drift coefficient: 0.3 % of distance (production stereo VIO).
- Position drift at tunnel exit: $0.003 \times 1500 = 4.5 \text{ m}$.

At exit, GNSS re-acquires and correction collapses the drift. During the tunnel, the 4.5 m accumulated error is acceptable for lane-keeping but would be problematic for precise lane-level maneuvers like lane changes. HD-map matching provides additional correction where maps are available in the tunnel.

16.9 Learned Visual Odometry

A more recent thread: train neural networks to regress camera motion directly from image pairs.

16.9.1 End-to-End Approaches

DeepVO and TartanVO train convolutional + recurrent networks to output 6-DoF pose from consecutive frames. Trained on large driving datasets, they achieve competitive accuracy with classical pipelines in benchmark settings; transfer to new scenes varies.

16.9.2 Hybrid Architectures

More common in practice: learned feature detection (SuperPoint, DISK) combined with classical RANSAC and bundle adjustment. These retain the interpretability and calibration of classical geometry while benefiting from the learned features' robustness to illumination and viewpoint changes.

16.9.3 Self-Supervised Depth and Motion

Given only monocular video, self-supervised methods learn depth and motion jointly by training a model to warp one frame to the next. Monodepth and successors have produced strong monocular results without requiring labeled data; the approach is less developed for production VO but increasingly influential.

Example 16.11 (Learned vs classical on night-driving scenes). KITTI-like daytime benchmarks: classical and learned VO comparable, within a few percent of each other in translation error. Night-driving benchmarks (custom, low-light): classical VO fails on many frames (insufficient features); SuperPoint-based hybrid maintains accuracy within 5 % of daytime performance. The production advantage of learned features is precisely in the challenging conditions where classical methods struggle.

16.10 Production VO in the AV Stack

Three places visual odometry appears in a production stack.

16.10.1 As SLAM's Front End

Visual SLAM (Chapter 13) uses VO for frame-to-frame tracking, with map management and loop closure layered on top. The VO quality directly determines the SLAM trajectory quality.

16.10.2 As a GPS Backup

When GNSS degrades or is lost, VO (often as VIO) carries the pose estimate. The EKF or factor-graph estimator weights VO higher during GNSS outages.

16.10.3 For Redundancy and Cross-Check

Production systems run VO continuously even when it is not the primary localization modality, using it as a cross-check against other sensors. Disagreement between VO and GNSS in GNSS-good regions can signal either GNSS multipath or VO degradation; either is actionable.

16.11 Industrial Deployment

Visual odometry approaches have moved from research to production over the past decade. NASA's Mars rovers (Spirit, Opportunity, Curiosity, Perseverance) have used visual odometry on the Martian surface since 2004; Curiosity and Perseverance use stereo visual odometry as a core navigation capability. Apple's ARKit, Google's ARCore, and Microsoft HoloLens use visual-inertial odometry as the foundation of their tracking. In automotive, Tesla's FSD stack is publicly described as relying heavily on visual odometry given its camera-only sensor architecture. The OpenVSLAM and ORB-SLAM3 open-source projects (from Universidad de Zaragoza and others) provide reference

implementations widely used in research. Skydio’s drones use visual-inertial odometry as part of their autonomous-flight capability.

16.12 Chapter Summary

Visual odometry estimates vehicle motion from consecutive camera frames alone, without external references. Feature-based pipelines (detect, match, solve for pose, refine) remain the production standard; direct methods use raw intensity gradients where features are scarce. Monocular scale ambiguity is resolved by stereo cameras, IMU fusion (VIO), or known metric features. VIO is the production workhorse, combining visual features with preintegrated IMU for robust monocular operation. Weather (rain, snow, fog, low light) is VO’s primary adversary; production systems use weather-adaptive confidence cascades to downgrade VO when conditions degrade. GPS-denied environments (tunnels, parking garages, urban canyons, mines) are where VO is most valuable, carrying the localization through regions other sensors cannot serve. Learned components—especially learned feature detectors—increasingly dominate state-of-the-art research while being adopted in production. VO is the most versatile single motion sensor: cheap, widely available, capable in GPS-denied operation, complementary to every other sensor in the stack.

16.13 Exercises

1. For the VO information-content example, compute the minimum number of features needed to over-determine the 6-DoF pose by a factor of 10.
2. Derive the relationship between stereo baseline, focal length, and depth precision. Compute depth standard deviation at 200 m range with the example stereo rig.
3. For the RANSAC iterations example, compute the required iterations for 40 % inlier rate and 99 % success probability.
4. Describe three specific scenarios where direct VO outperforms feature-based VO. Justify each.
5. A monocular VIO drifts 0.5 % of distance. Compute the expected position error after 20 km without external correction.
6. Design a weather-adaptive confidence estimator that combines rain sensor, VO feature count, and RANSAC inlier ratio. State the weighting scheme.
7. For the tunnel example, compute the drift if IMU quality degrades by $3\times$ (consumer-grade instead of industrial-grade).
8. Compare monocular and stereo VO for automotive deployment. Identify three specific use cases where stereo is mandatory.
9. Propose an initialization strategy for VIO that completes within 1 second of vehicle motion. What

assumptions does it require?

10. Describe how VO in a robotaxi stack can cross-check HD-map localization. What discrepancy signals would trigger an alert?

Part IV

Prediction and Planning

Chapter 17

Motion Prediction

17.1 Introduction

The perception stack tells the planner where every agent is *now*. That is not enough. A planner reasoning about whether to make a left turn across oncoming traffic has to answer: *where will that red sedan be in three seconds?*. Not the instantaneous position, not the best-guess linear extrapolation, but the *distribution* of plausible future trajectories, because the sedan might continue straight, or brake, or swerve, and the planner’s reaction has to be safe under all of them. Motion prediction is the module that produces this distribution, and its quality is one of the strongest determinants of planner performance—downstream of perception quality, upstream of everything else.

Three observations motivate the chapter’s structure.

First, prediction is fundamentally *multi-modal*. At an intersection, a vehicle might turn, go straight, or stop; no single mean trajectory captures that. A network that outputs a unimodal distribution centered between the modes does worse than one that outputs a handful of clearly-distinguished candidates. Modern predictors explicitly produce multiple hypothesis trajectories, each with a probability.

Second, prediction is fundamentally *interactive*. How Agent A will move depends on what Agent B is doing, which in turn depends on what the ego is doing. Linear-in-agents models that predict each agent independently miss this structure and produce implausible predictions (e.g., two vehicles predicted to occupy the same lane). Modern predictors use graph neural networks or transformers to reason about inter-agent dependencies jointly.

Third, prediction is fundamentally *map-conditioned*. A vehicle at an intersection follows the lane graph—it is far more likely to turn onto a viable lane than to cut across three lanes of traffic. Lane-aware predictors outperform map-agnostic predictors by wide margins on every standard benchmark.

The chapter is organized as follows. We first formulate the prediction problem precisely. We then develop physics-based baselines (constant velocity, bicycle model) that anchor the discussion.

Map-aware prediction follows, with emphasis on lane- graph conditioning. Interaction modeling, joint-vs-marginal outputs, and the various output parameterizations (Gaussian mixtures, trajectory sets, diffusion) are next. We close with pedestrian- specific considerations, training objectives, evaluation metrics, and compute-budget realities. Three architecture diagrams visualize the key pipelines; worked examples make each formula concrete.

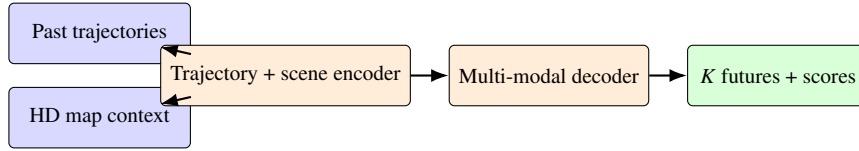


Figure 17.1: Chapter roadmap. Past trajectories and map context feed a shared scene encoder; a multi-modal decoder produces K candidate futures with associated probabilities. Evaluated by minADE, minFDE, and miss rate.

17.2 Problem Statement

Given observation history—the past T_h seconds of each agent’s state, plus the local map—output for each agent a distribution over future trajectories over the next T_f seconds. Typical AV values: $T_h = 1\text{--}2\text{ s}$, $T_f = 3\text{--}8\text{ s}$. Outputs are sampled at discrete timesteps (typically 0.1 s), giving 30–80 waypoints per trajectory.

The conditional distribution to estimate is

$$p(\{\mathbf{y}_{1:T_f}^a\}_a \mid \{\mathbf{x}_{-T_h:0}^a\}_a, \mathcal{M}), \quad (17.1)$$

where $\mathbf{x}^a$ is agent a ’s past state, $\mathbf{y}^a$ is future state, and $\mathcal{M}$ is the map (lane graph, traffic controls). Joint prediction models the whole distribution; marginal prediction factorizes across agents.

Example 17.1 (Dimension of the prediction output). An urban scene has 12 agents of interest. Predict $T_f = 60$ waypoints (6 seconds at 10 Hz) per agent, each with 2D position (ignoring heading for simplicity). 6 modes per agent (left turn, right turn, straight, stop, merge-left, merge-right). Output size per agent: $60 \cdot 2 \cdot 6 + 6 = 726$ scalars (720 waypoint coords + 6 mode probabilities). Total: $12 \cdot 726 = 8712$ scalars per frame. At 10 Hz this is modest compute; at dozens of agents it becomes the dominant prediction-stage cost.

17.3 Physics-Based Baselines

Before learning, we need baselines. Two physics-based models anchor every comparison.

17.3.1 Constant Velocity and Constant Turn Rate

Assume the agent continues at its current velocity:

$$\hat{\mathbf{y}}_t^a = \mathbf{x}_0^a + t \cdot \mathbf{v}_0^a. \quad (17.2)$$

Constant turn rate (CTR) adds yaw rate:

$$\hat{\mathbf{y}}_t^a = \mathbf{x}_0^a + \text{integrate}(\mathbf{v}, \boldsymbol{\omega}) \cdot t, \quad (17.3)$$

following a circular arc.

Example 17.2 (CV prediction error at 6 s). Agent moving at 15 m s^{-1} north. Observer predicts future position at $t = 6 \text{ s}$: $\hat{\mathbf{y}} = \mathbf{x}_0 + 6 \cdot 15 \hat{n}$. If the agent instead brakes at -2 m/s^2 (slows to 3 m/s at $t = 6$), actual displacement is $15 \cdot 6 - 0.5 \cdot 2 \cdot 6^2 = 90 - 36 = 54 \text{ m}$. CV predicts 90 m—36 m overshoot, equivalent to one car length every second of horizon. The CV baseline performs well for tens of meters and seconds but breaks down on any behavior change.

17.3.2 Kinematic Bicycle

The bicycle model (Chapter 20) gives state update

$$\begin{aligned} \dot{x} &= v \cos \theta, \\ \dot{y} &= v \sin \theta, \\ \dot{\theta} &= \frac{v}{L} \tan \delta, \\ \dot{v} &= a, \end{aligned} \quad (17.4)$$

with wheelbase L , steering δ , and acceleration a . As a prediction model: forward-integrate starting from the current state, assuming (a, δ) hold constant. More realistic than CV for turning vehicles, still deterministic.

Example 17.3 (Bicycle prediction for a cornering car). $v_0 = 10 \text{ m/s}$, $\theta_0 = 0$, $L = 2.7$, $\delta = 0.1 \text{ rad}$ (steering into a right turn), $a = 0$. Yaw rate: $\dot{\theta} = 10 \cdot \tan(0.1)/2.7 \approx 0.371 \text{ rad/s}$. Turn radius $R = v/\dot{\theta} = 26.9 \text{ m}$. Over $T_f = 3 \text{ s}$: arc traversed 30 m, heading change 1.11 rad (64°), final position: $R \sin(\dot{\theta} T_f) \approx 24.3 \text{ m}$ forward, $R(1 - \cos(\dot{\theta} T_f)) \approx 14.7 \text{ m}$ right. Captures the curvature a CV model entirely misses.

17.4 Map-Aware Prediction

Vehicles follow lanes. Predictors that know the lane graph outperform predictors that don't by large margins. Figure 17.2 sketches a modern lane-anchored predictor.

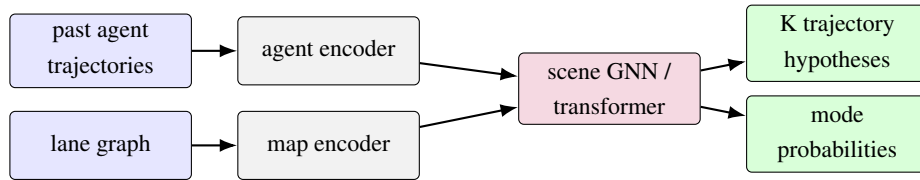


Figure 17.2: Modern map-aware, interaction-aware predictor. Each agent’s recent trajectory and the local lane graph are encoded separately. A scene-level transformer or graph neural network then jointly attends to all agent and map tokens, producing K trajectory hypotheses per agent with mode probabilities. Representative implementations: VectorNet, SceneTransformer, Wayformer, MTR.

17.4.1 Lane Anchors

Lane-anchored predictors (MultiPath, CoverNet, LaneGCN) generate candidate trajectories along the lanes the agent could plausibly follow and score each. A vehicle approaching an intersection with three viable lanes generates three lane-conditioned trajectory hypotheses plus mode probabilities.

Example 17.4 (Lane-anchor probability). Agent 15 m before a T-intersection, aligned with the central lane, approaching at 12 ms^{-1} . Three lane anchors: left-turn (curve), right-turn (curve), straight. Network scores from current state + recent past: $(p_L, p_R, p_S) = (0.55, 0.30, 0.15)$ (agent has been drifting slightly left). Each lane anchor produces a trajectory; planner consumes all three with their probabilities and reacts accordingly.

17.4.2 Vectorized Map Encoding

VectorNet and successors encode the map as a collection of vectors (each lane-line segment, each crosswalk edge, each stop-line) and process them with a graph neural network (Chapter 2). Advantages over rasterized (image-of-map) encodings: efficient, preserves precise geometry, scales to large areas.

17.4.3 Production Architectures

Three architectures published by Waymo Research—VectorNet, SceneTransformer, and MultiPath++—illustrate the architectural progression from per-agent vector encoding to fully joint scene modelling. Each is documented in peer-reviewed publications and the patterns each established are reflected in production prediction stacks across the AV industry.

VectorNet

VectorNet [23] represents map elements and agent trajectories as *polylines*—ordered sequences of vectors. A hierarchical graph network first encodes each polyline locally, then aggregates polylines globally. Figure 17.3 sketches the architecture.

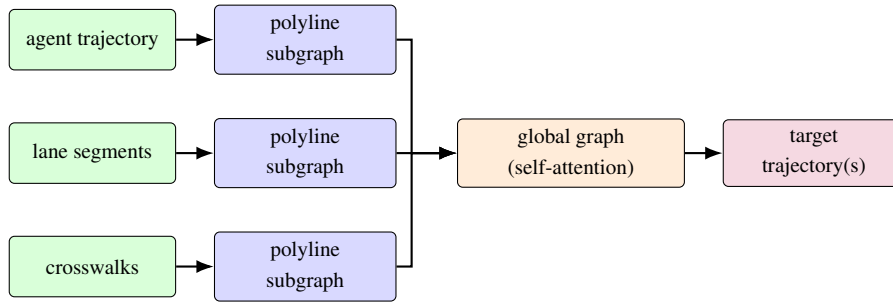


Figure 17.3: VectorNet architecture (schematic). Heterogeneous polyline inputs (agent trajectories, lane segments, crosswalks, etc.) are each processed by a per-polyline *subgraph* that aggregates intra- polyline structure. Polyline embeddings then enter a *global interaction graph* with self-attention across all polylines. The target agent’s embedding decodes a predicted trajectory (or trajectory distribution).

The two-level structure—local polyline encoding then global attention—preserves the sparse vectorised representation while giving each polyline access to the full scene at the global stage. This pattern appears in many subsequent predictors.

SceneTransformer

SceneTransformer [51] pushes interaction modelling further by jointly attending across agents *and* time. Figure 17.4 shows the architecture.

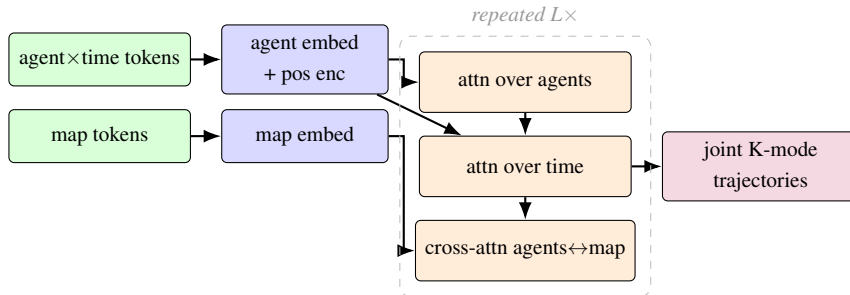


Figure 17.4: SceneTransformer architecture (schematic). The full agent×time tensor of trajectory states, plus map polyline tokens, are processed by a stack of L factorized attention layers: attention over agents (capturing inter-agent interaction at each time step), attention over time (capturing dynamics for each agent), and cross-attention to map context. The decoder produces a *joint K-mode* prediction: each mode is a coherent assignment of futures to all agents simultaneously, rather than independent per-agent distributions.

The factorisation across agent and time axes keeps the attention cost manageable even for scenes with dozens of agents over horizons of several seconds. Joint K -mode output is the architectural signature: SceneTransformer commits to scene-consistent multi-agent futures rather than independent per-agent guesses.

MultiPath++

MultiPath++ [67] is a follow-on to the original MultiPath [11]. The architecture is specifically designed to produce well-calibrated multi-modal trajectory distributions efficiently for production deployment. Figure 17.5 sketches its three stages.

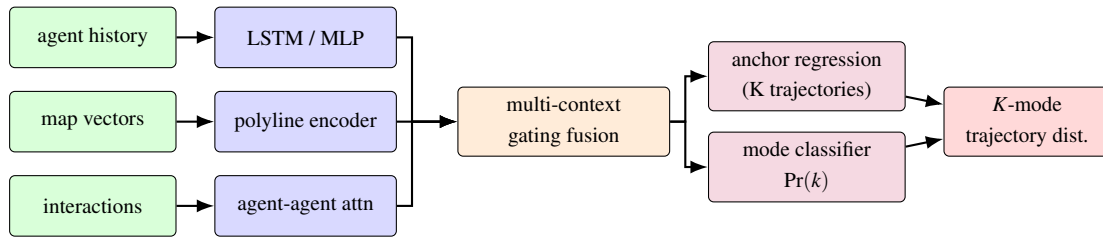


Figure 17.5: MultiPath++ architecture (schematic). Three input streams (agent history, map polylines, agent-agent interactions) are encoded separately and fused by a *multi-context gating* layer that learns dynamic weights over the contexts. Two task heads then predict K trajectory candidates (anchor regression) and their probabilities (mode classifier). The joint output is a Gaussian-mixture-style distribution over the K modes, calibrated for downstream planner consumption.

The defining design choice in MultiPath++ is the decoupling of trajectory *geometry* (regressed from data, no fixed anchors) from mode *probability* (a learned classifier). The earlier MultiPath relied on hand-crafted trajectory anchors; ++’s data-driven regression yields markedly better minADE/minFDE on the Waymo Open Motion Dataset benchmark.

17.5 Interaction Modeling

Two vehicles on a merge cannot both occupy the merge point; one has to yield. Prediction models that reason about this *inter-agent* dependency produce much more realistic outputs. Most modern predictors implement interaction via a scene-level attention mechanism: each agent’s prediction attends to features of every other agent.

Example 17.5 (Interaction-aware yielding behavior). Two vehicles approach a merge from different lanes at similar velocities. Non-interactive prediction: both continue at 20 ms^{-1} , producing predicted trajectories that overlap in space-time—physically impossible. Interaction-aware prediction: the network produces two possible outcomes: (a) Vehicle A yields (decelerates to 15 ms^{-1} , Vehicle B goes first), (b) Vehicle B yields, with probabilities based on subtle cues (relative speed, lane-occupancy precedent). Planner now has non-overlapping hypotheses and can respond to both.

17.6 Joint vs. Marginal Prediction

A predictor can output either:

- **Marginal:** independent distribution per agent. Cheap, simple; misses joint dependencies.
- **Joint:** single distribution over the whole scene. Correct but exponentially large output space.

Most production systems output *marginal* distributions with *interaction-aware encoding*—i.e., the encoder sees all agents, but the decoder outputs per-agent distributions. This captures first-order interaction effects (A’s prediction depends on B being there) while keeping output dimension linear.

Example 17.6 (Marginal vs joint error on a merge). Two vehicles predicted to converge. Marginal mode probabilities: both agents go-straight with $P = 0.8$ each. Joint probability of both-straight collision: if marginals were independent, 0.64. But the true joint probability is ~ 0.05 (they would yield). The marginal factorization overestimates collisions by an order of magnitude. Joint prediction corrects this; at the cost of having to model $O(\text{actions}^N)$ joint possibilities for N agents.

17.7 Output Parameterizations

How to represent the output distribution?

17.7.1 Mixture of Gaussians

The output is a set of K Gaussian modes:

$$p(\mathbf{y}_{1:T_f}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{y}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k). \quad (17.5)$$

Example 17.7 (MoG trajectory). $K = 3$ modes; at $t = 3$ s, means at $(45, 0), (40, 10), (30, -8)$ with covariances $\text{diag}(2, 0.5), \text{diag}(3, 1), \text{diag}(2.5, 0.7)$. Mode weights $(0.55, 0.25, 0.20)$. Probability that the agent is within a 3 m radius of $(44, 0)$: $\approx 0.55 \cdot P_1 + 0.25 \cdot P_2 + 0.20 \cdot P_3$ with P_1 dominant. Planner uses the full mixture to reason about all three possibilities.

17.7.2 Trajectory Sets

Rather than learning the Gaussians, score a large predefined bank of plausible trajectories (CoverNet, TNT). Simpler optimization; requires covering the trajectory space well.

17.7.3 Diffusion-Based

Diffusion models [31] generate trajectories by iteratively denoising a Gaussian sample. Produces high-quality, multi-modal outputs with good coverage. Cost is sample steps at inference (10–100); accelerated samplers (DDIM, DPM-Solver) reduce this.

17.8 Pedestrian and Cyclist Prediction

Pedestrians and cyclists are less constrained than vehicles—they can cross lane boundaries, change direction abruptly, and move into spaces a vehicle cannot. Predictors for these classes rely less on map anchors and more on behavioral modeling. Social-pool models, pedestrian-intent classifiers, and waiting-for-walk-sign detectors augment the general predictor for pedestrian-specific scenarios.

Example 17.8 (Pedestrian-intent classifier cue). Pedestrian at crosswalk edge. Prediction over next 4 seconds: (a) will cross now (probability 0.55), (b) wait for traffic (0.30), (c) not cross at all (0.15). Binary classifiers trained on body pose + gaze direction + traffic context provide these probabilities. The planner treats (a) as an obligation to yield and (b)/(c) as lower-priority; the decision is bounded by the worst case when the pedestrian has already begun committing.

17.9 Training Objectives

17.9.1 Negative Log-Likelihood

For a Gaussian mixture output:

$$\mathcal{L}_{\text{NLL}} = -\log \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{y}^*; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k). \quad (17.6)$$

Example 17.9 (NLL for a single sample). Ground truth trajectory $\mathbf{y}^*$. Mode 1: mean close ($\|\mathbf{y}^* - \boldsymbol{\mu}_1\|^2 = 0.2$), $\pi_1 = 0.5$, $|\boldsymbol{\Sigma}_1| = 1$. Mode 2: mean far ($\|\mathbf{y}^* - \boldsymbol{\mu}_2\|^2 = 20$), $\pi_2 = 0.5$, $|\boldsymbol{\Sigma}_2| = 1$. Gaussian evaluations (dropping constants): $e^{-0.1} \approx 0.905$, $e^{-10} \approx 4.5 \times 10^{-5}$. $\mathcal{L}_{\text{NLL}} \approx -\log(0.5 \cdot 0.905) = 0.79$. Near-exact prediction. If both modes are far ($\|\cdot\|^2 \geq 10$), $\text{NLL} \approx 5$ —much higher. NLL rewards putting mass on the actual future.

17.9.2 Winner-Takes-All

WTA losses supervise only the mode nearest to the ground truth:

$$\mathcal{L}_{\text{WTA}} = \min_k \|\mathbf{y}^* - \boldsymbol{\mu}_k\|^2 - \log \pi_{k^*}, \quad (17.7)$$

with $k^* = \arg \min_k \|\mathbf{y}^* - \boldsymbol{\mu}_k\|$. WTA prevents mode collapse (all modes regressing to the mean), at the cost of harder optimization.

17.9.3 Behavior-Cloning Regularizers

Auxiliary losses on the intermediate representations: kinematic feasibility, lane compliance, smoothness. These constrain the predictor to produce realistic motion even when the NLL loss is ambiguous.

17.10 Evaluation Metrics

Two dominant metrics:

- **ADE (Average Displacement Error)**: mean distance between predicted and ground-truth trajectory over all timesteps.

- **FDE (Final Displacement Error):** distance at the final timestep only.

For K -mode outputs, standard metrics are *minADE* and *minFDE*—the best of K predictions—which measures coverage of the true future, and *miss rate* (MR), the fraction of samples where the best prediction misses by more than a threshold.

Example 17.10 (minADE vs ADE). Single-mode predictor: ADE = 2.1 m, FDE = 5.8 m. 6-mode predictor: ADE of each mode ranges 1.8 m to 4.0 m; mean ADE = 2.8 m; minADE = 1.1 m. The minADE is smaller because the diverse modes include one that approximates the true future well. The mean ADE is larger because five of the six modes are actively wrong—but not in the sense that matters for the downstream planner. Hence, minADE is the primary comparison metric for multi-modal predictors.

Error grows with prediction horizon; Figure 17.6 plots representative error curves for the three main classes of predictor on typical urban-driving benchmark data.

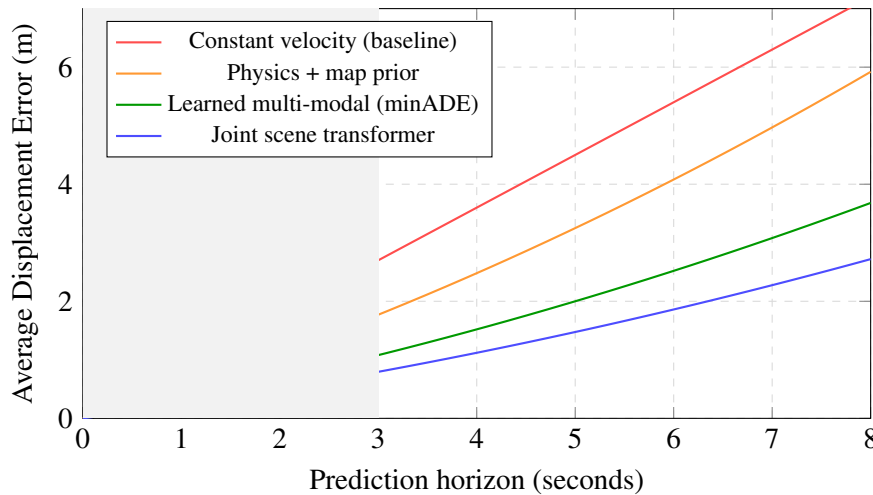


Figure 17.6: Prediction error vs horizon for four classes of predictor on typical urban benchmark data. Error grows roughly linearly for short horizons and super-linearly beyond 5 s as uncertainty compounds. Constant-velocity (red) is the floor a usable predictor must beat. Physics + map (orange) captures lane-following behavior. Learned multi-modal models (green) win significantly on minADE by covering diverse possible futures. Joint scene transformers (blue) add further gains by modelling inter-agent consistency. Shaded region marks the planner-critical horizon (≤ 3 s) where prediction quality most strongly affects ego safety.

17.11 Latency and Compute Budgets

Production predictors run at 5 Hz to 10 Hz within a total prediction-planning latency budget of 50 ms to 200 ms. Typical compute: 5 GFLOP to 50 GFLOP/frame/agent; multi-agent scenes with 30+ agents push this into the 1 to 5 TFLOP range—at the edge of what automotive accelerators can sustain. Shared encoders (one backbone across all agents) and efficient attention (Flash-attention, windowed attention) are not optional for production deployment.

17.12 Worked Examples

Example 17.11 (Full multi-modal prediction for an intersection). Scene: 4 agents near a 4-way intersection. Ego at center, 3 other vehicles approaching from three directions. Encoder: agents (past 2 s trajectories) + map (20 lane segments). Transformer decoder: 6 learned queries per agent, producing 6 trajectory hypotheses over 6 s at 10 Hz (60 waypoints each) + mode probabilities. Output size per agent: $6 \cdot 120 + 6 = 726$ scalars. Total: $3 \text{ agents} \times 726 = 2178$ scalars.

Probabilities per agent:

- Agent 1 (approaching from north): left 0.4, straight 0.45, right 0.15.
- Agent 2 (from east): straight 0.7, stop 0.3.
- Agent 3 (from south): left 0.2, straight 0.5, right 0.3.

Planner reasons over this joint space (6 modes per agent, 3 agents, $6^3 = 216$ combinations), each with a probability computed as the product of marginals (or via a joint scene model). Typical consumed representation: top-10 joint scenarios by probability, each evaluated for collision risk with a candidate ego plan.

Example 17.12 (Training-time gradient through MoG + WTA). Predictor output: 3 modes at $t = T_f$, means $(30, 0)$, $(30, 10)$, $(30, -10)$, probabilities $(0.4, 0.35, 0.25)$. Ground truth: $(30, -9)$ (agent took the right turn). Nearest mode: 3, with distance 1. WTA loss: $1.0 + (-\log 0.25) = 1.0 + 1.386 = 2.386$. Gradient pulls mode 3's mean toward the truth and increases π_3 . Modes 1 and 2 are not pulled (WTA ignores non-winners), avoiding the mode-collapse problem where all means collapse toward the average direction.

Example 17.13 (Prediction latency budget trade). Total prediction budget: 100 ms per frame. Current architecture: encoder (30 ms) + scene transformer (55 ms) + decoder (15 ms) = 100 ms—at budget. Increase to $K = 12$ modes doubles decoder latency to 30 ms—over budget. Two mitigations: (a) reduce transformer layers from 6 to 4 (37 ms, 30 % faster) and keep $K = 12$; (b) use a lighter encoder (MobileNet-based instead of ResNet, saving 8 ms) and keep 6 layers. Both result in 82–85 ms total; pick the option that loses less accuracy on the validation minADE.

17.13 Industrial Deployment

Prediction architectures discussed in this chapter underpin production trajectory-prediction systems across the AV industry. The VectorNet and SceneTransformer architectures (Waymo Research) and MultiPath++ (Waymo) have been published in detail in peer-reviewed venues and influenced many subsequent production designs. The Argoverse Motion Forecasting benchmark (Argo AI, now wound down with assets distributed to Ford and Volkswagen) established standard evaluation metrics including minADE, minFDE, and miss rate that are used industry-wide. The Waymo Open Motion Dataset, released in 2021, provides large-scale prediction-evaluation data to academic researchers. Tesla's FSD prediction stack uses an "occupancy network" that predicts a 3D probabilistic occupancy

field with flow vectors [64], an approach that has influenced academic research on dense prediction representations.

17.14 Chapter Summary

Motion prediction outputs a multi-modal, map-aware, interaction-aware distribution of future trajectories for every agent of interest. Physics-based baselines provide a floor; map-conditioned, graph-neural-network or transformer-based models trained on large-scale driving datasets produce the state of the art. Output is parameterized as a Gaussian mixture, trajectory set, or diffusion model; trained with NLL or WTA losses; evaluated with minADE, minFDE, and miss rate. Production systems process up to 30 to 50 agents per frame at 5 Hz to 10 Hz within automotive compute budgets, feeding the planner with the probabilistic future it needs to make safe decisions.

17.15 Exercises

1. For the prediction-dimension example (17.1), estimate the output size for 20 agents, $K = 10$ modes, and $T_f = 80$ waypoints.
2. Compute the CV prediction position error after 5 s for an agent that actually executes uniform acceleration at 0.5 m/s^2 .
3. Derive the bicycle-model-predicted position of a vehicle at $v_0 = 20 \text{ m/s}$, $\delta = 0.05 \text{ rad}$, $L = 2.8 \text{ m}$, after 2 s.
4. Explain why a marginal predictor is likely to produce an overlapping-trajectory prediction for two adjacent vehicles, and how interaction-aware encoding fixes it.
5. Write out the NLL loss for a 3-mode Gaussian mixture with means $(30, 0)$, $(30, 10)$, $(30, -10)$, identity covariances, and ground truth $(28, -1)$.
6. A WTA predictor has 6 modes; a single training example's ground-truth future is equidistant from modes 1 and 2. Describe what happens and propose a tie-breaking strategy.
7. For the minADE vs ADE example (17.10), compute minADE and FDE if mode probabilities were equal but three modes were further from ground truth.
8. At an intersection, propose three interaction-aware features the encoder should pass to the decoder, and justify each.
9. Compare the output parameterization choices (MoG, trajectory set, diffusion) for a stack that prioritizes interpretability and latency.
10. For the prediction-latency-budget example (17.13), if the encoder alone were tripled in cost (ResNet-101), propose two ways to stay within budget.

Chapter 18

Motion Planning

18.1 Introduction

Perception tells the AV what the world contains; prediction tells it what the world will do next. Planning tells it what *the ego* should do. The output is a reference trajectory: a sequence of poses with timestamps that the control layer turns into throttle, brake, and steering commands. The trajectory must satisfy many constraints simultaneously—physics (the vehicle cannot turn faster than its tire grip allows), safety (it must not collide with predicted futures of other agents), traffic rules (speed limits, yields), and comfort (no jerks, no rapid direction changes). Planning is the layer where these constraints collide.

Three observations motivate the chapter’s structure.

First, planning is multi-scale. A *route planner* picks roads (city to city); a *behavior planner* picks tactical decisions (which lane, merge when); a *motion planner* generates a specific trajectory; a *control* layer tracks it. Each operates at a different timescale with a different abstraction, and the interfaces between them are architecturally important.

Second, planning is fundamentally *constrained*. Unlike many optimization problems in machine learning, a planner cannot simply minimize a cost—it must produce *feasible* plans. Collision avoidance, dynamic limits, and traffic rules are hard constraints that must be satisfied, not merely penalized.

Third, planning is *interactive*. Changing ego behavior changes other agents’ behavior. A predictor that assumes the ego will follow a nominal plan produces futures that are inconsistent with what actually happens if the ego deviates. Production planners either accept this approximation or reason interactively.

The chapter is organized as follows: route planning, behavior planning, motion-planning problem formulation, sampling-based planners, lattice/graph-search planners, optimization-based planners (Frenet frame, QP, MPC), collision checking, interaction-aware planning, comfort, and finally the handoff to control. Worked examples accompany every formula; three architecture diagrams visualize

the hierarchy and key representations.

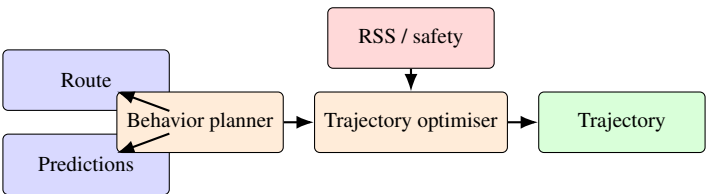


Figure 18.1: Chapter roadmap. Hierarchical planning: route gives the goal, predictions feed the behaviour planner (lane choice, gap selection), the trajectory optimiser produces the executable path under safety constraints (RSS-style guarantees enforced above).

18.2 The Planning Hierarchy

Figure 18.2 shows the canonical planning hierarchy.

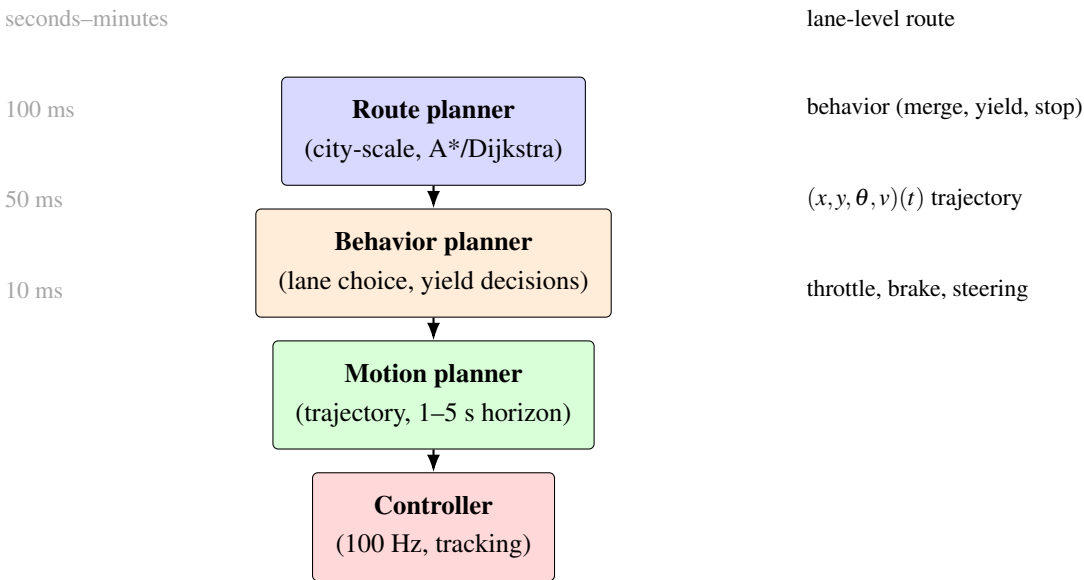


Figure 18.2: The planning hierarchy. At the top, a route planner picks a lane-level path from origin to destination. A behavior planner selects tactical decisions (which lane, when to yield, whether to pass). A motion planner produces the specific trajectory over a 1 s to 5 s horizon. A controller executes the trajectory at control-loop rate. Each layer’s output is the next layer’s context; timescales shrink by orders of magnitude as we descend.

18.3 Route Planning

Route planning answers “which sequence of road segments goes from origin to destination?”. The underlying algorithm is graph search (Dijkstra, A*) on the road graph, weighted by travel time or distance. Modern route planners add real-time traffic, EV range/charging constraints, and vehicle-specific restrictions (height, hazmat).

Example 18.1 (A* on the lane graph). Origin and destination are both intersections in a lane graph of $N = 10000$ nodes. A* with an admissible heuristic (great-circle distance divided by max speed) expands on average $\sqrt{N} = 100$ nodes before reaching the goal, far less than Dijkstra's $\sim N$. Runtime: <1 ms. The route output is a sequence of lane-level waypoints for the behavior planner to turn into tactical decisions.

18.4 Behavior Planning

The behavior planner selects discrete actions: follow-lane, change-left, change-right, yield, pass, stop. Classical approaches: finite-state machines over hand-designed scenarios. Modern approaches: MDP/POMDP formulations with learned or search-based solutions.

18.4.1 Finite-State Machines

A simple FSM: states $\{FollowLane, ChangeLeft, ChangeRight, Yield, Stop\}$ with explicit transition rules (“if a slow vehicle ahead and left lane clear, transition to ChangeLeft”). Pros: interpretable. Cons: hard to cover all scenarios; brittle under the long tail.

18.4.2 MDPs and POMDPs

A Markov decision process formalizes the problem: state s , action a , transition $T(s' | s, a)$, reward $r(s, a)$, discount γ . The value function:

$$V^*(s) = \max_a \{r(s, a) + \gamma \sum_{s'} T(s' | s, a) V^*(s')\}. \quad (18.1)$$

A POMDP extends this to partially observed state (the ego does not know others' intentions); the state is replaced by a belief distribution.

Example 18.2 (POMDP merge). Belief over whether the leading vehicle will yield: $p = 0.7$ yields. Ego actions: (a) aggressive merge (expected reward if yields: +10, if not: -50), (b) slow merge (expected reward: +2 either way). Expected reward of (a): $0.7 \cdot 10 + 0.3 \cdot (-50) = -8$; of (b): 2. Choose (b). As the ego gets more information (the leader's deceleration signal updates p), the expected reward may shift and the decision may flip. POMDP planners solve exactly this kind of belief update + value calculation.

18.5 Motion Planning: Problem Formulation

Given current state $\mathbf{x}_0$ and a desired behavior, produce a trajectory $\mathbf{x}(t)$ over horizon $[0, T]$ minimizing cost and satisfying constraints:

$$\min_{\mathbf{x}(t), \mathbf{u}(t)} \int_0^T \ell(\mathbf{x}, \mathbf{u}, t) dt, \quad (18.2)$$

$$\text{s.t. } \dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}), \quad (18.3)$$

$$\mathbf{g}(\mathbf{x}, t) \leq 0 \quad (\text{collision, rules}), \quad (18.4)$$

$$\mathbf{u} \in \mathcal{U} \quad (\text{dynamic limits}). \quad (18.5)$$

The problem is nonconvex in general (collision constraints, dynamic coupling) and solved by sampling, graph search, or optimization.

18.6 Sampling-Based Planning

18.6.1 RRT and RRT*

Rapidly-exploring Random Trees sample random configurations, connect them to the nearest tree node if collision-free, and grow a tree until reaching the goal. RRT* adds rewiring to produce asymptotically optimal paths.

Example 18.3 (RRT* convergence). Environment: 100 m×50 m open area with 2 obstacles. RRT* with 1000 iterations: path length converges to near-optimal (within 3 % of the geodesic) in under 10 ms. 10000 iterations: within 0.5 %. Production AV planners use sampling-based methods sparingly because comfort constraints (smooth velocity, bounded curvature) are hard to enforce on discrete samples.

18.6.2 Kinodynamic Planners

Extensions that respect dynamics during sampling: each step produces a dynamically feasible motion segment (circular arcs, clothoids, spiral curves). Used in parking planners and tight maneuver planners where RRT's straight-line segments are infeasible.

18.7 Lattice and Graph-Search Planners

A *motion lattice* pre-computes a discrete set of feasible motion primitives at each grid node; planning becomes A* on this lattice. The pre-computation makes each plan query fast (<10 ms) and the lattice structure guarantees kinodynamic feasibility. Lattice planners remain popular for parking and structured-scene driving where the possible motions are limited.

18.8 Optimization-Based Planning

The dominant class of on-road AV planners. Frame the trajectory as a finite set of decision variables (control inputs or polynomial coefficients over time) and solve a constrained nonlinear program.

18.8.1 Frenet Frame Planning

The *Frenet frame* parameterizes position by arc length s along a reference path (typically the lane centerline) and lateral offset d perpendicular to it. Trajectories become functions $s(t), d(t)$ that are much easier to reason about than Cartesian trajectories.

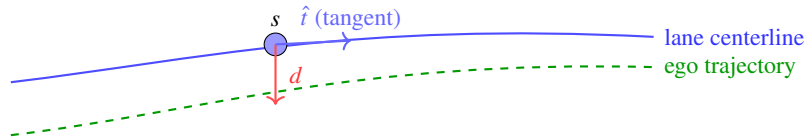


Figure 18.3: The Frenet frame. The curved blue reference is the lane centerline. A point on the ego trajectory (green dashed) is expressed as arc-length s along the reference plus perpendicular offset d . This decomposition decouples longitudinal and lateral dynamics and produces much simpler trajectory optimizations than working in Cartesian coordinates directly.

Example 18.4 (Frenet conversion). Ego Cartesian position $(23.1, -1.5)$. Reference lane path at $s = 23$ has point $(23, 0)$ with tangent $\hat{t} = (1, 0)$, normal $\hat{n} = (0, 1)$. Relative vector: $(0.1, -1.5)$. s projection along $\hat{t}$: $s = 23 + 0.1 = 23.1$. d projection along $\hat{n}$: $d = -1.5$ —ego is 1.5 m right of centerline, well within a lane width. Planning in (s, d) decouples: lateral (stay close to $d = 0$) and longitudinal (match target velocity) objectives are nearly independent.

18.8.2 QP-Based Trajectory Optimization

Parameterize the trajectory as quintic polynomials in s and d . A quintic has 6 coefficients per segment; two segments give a closed-form solution for desired boundary conditions. Solve a quadratic program minimizing jerk subject to state constraints.

Example 18.5 (Quintic lateral trajectory). Start: $d_0 = -1.5, \dot{d}_0 = 0, \ddot{d}_0 = 0$ (centered in lane offset). End: $d_f = 0, \dot{d}_f = 0, \ddot{d}_f = 0$ (fully returned to centerline). Polynomial horizon $T = 2$ s. Quintic coefficients solve a closed-form 6×6 linear system; the resulting $d(t)$ smoothly transitions from -1.5 to 0 with zero jerk at endpoints. Maximum lateral velocity during the transition: 0.94 m/s; max lateral accel: 1.4 m/s²—well within comfort limits. This entire computation takes microseconds and is the backbone of lane-keeping and lane-change planners.

18.8.3 Model Predictive Control (MPC)

MPC solves a constrained optimization over a short horizon, repeatedly re-optimizing at control rate. Figure 18.4 sketches the loop.

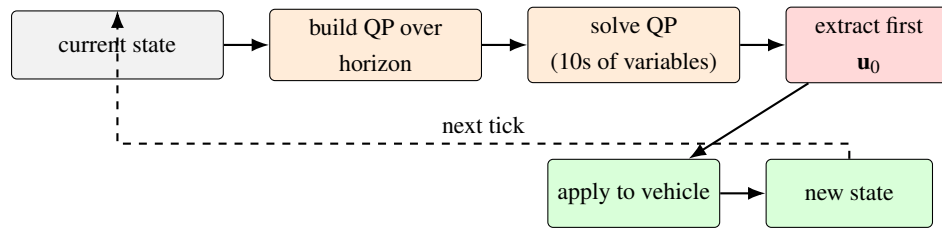


Figure 18.4: Model-predictive control loop. At each control tick, MPC builds a quadratic program over a short (1 s to 3 s) horizon, solves it, applies the first control action, observes the new state, and repeats. Horizons are short because the problem grows with horizon length; control rate (50 Hz to 100 Hz) is high because we re-solve frequently to close the loop.

The MPC cost includes tracking error, control effort, and terminal cost; the constraints include dynamics, limits, and collision avoidance.

Example 18.6 (MPC problem size). Horizon: $N = 20$ steps at 0.1 s. State: 4 variables (x, y, θ, v). Control: 2 (a, δ). Total decision variables: $N \cdot (4 + 2) = 120$. Dynamics constraints: $20 \cdot 4 = 80$. Bound constraints: ~ 80 . QP with 120 vars and 160 constraints solves in 0.1 ms to 1 ms on automotive-grade compute (OSQP, qpOASES, HPIPM). Solving at 50 Hz is comfortable.

18.9 Collision Checking

Every candidate trajectory must be checked for collisions against every predicted other-agent trajectory over the horizon. Naïve: for each candidate, each other agent, each future timestep, compute closest-point distance. Cost grows with (candidates $\times$ agents $\times$ timesteps). Optimizations:

- **Bounding boxes** for cheap initial exclusion.
- **Spatio-temporal sweeps** that check only overlap in time-expanded volumes.
- **Signed-distance fields** precomputed over the BEV grid.

18.9.1 Dynamic Obstacles

For dynamic obstacles, a candidate trajectory is checked against the full multi-modal prediction from Chapter 17. Typically: for each mode, check collision; weight total cost by mode probability; reject the candidate if any high-probability mode produces a collision.

Example 18.7 (Chance-constrained collision avoidance). Agent has two predicted modes: straight ($P=0.7$), brake ($P=0.3$). Ego candidate trajectory collides in mode “brake” but not “straight”. Expected collision prob: 0.3. At chance-constraint threshold 0.05 (5 % collision budget), this trajectory is rejected. A more conservative candidate avoiding both modes is selected—pays the cost in smoothness / progress but meets the safety budget.

18.10 Interaction-Aware Planning

Classical planners assume other agents are fixed obstacles; interaction-aware planners reason about how others respond to the ego. Techniques:

- **Game-theoretic planners:** model as iterated best- response or Nash equilibrium.
- **Joint prediction-planning networks:** learn predict + plan together.
- **Counterfactual prediction:** simulate different ego actions and pick the best response.

18.11 Comfort

A passenger-carrying AV must not jerk. Comfort constraints and objectives in the planner:

Constraint	Bound	Perceived as
Longitudinal jerk $ \dot{a} $	$\leq 1.5 \text{ m s}^{-3}$	smooth acceleration
Lateral acceleration $ a_y $	$\leq 2.5 \text{ m s}^{-2}$	moderate cornering
Yaw rate $ \dot{\theta} $	$\leq 0.3 \text{ rad s}^{-1}$	gentle turning
Decision stability	minimum-duration holds	no “indecisive” feel

Human limits are tighter than physical limits: a car can pull 8 m s^{-2} of braking, but few passengers tolerate more than 3 without alarm. Production planners operate well inside the comfort envelope by default and release the bounds only in emergencies.

Example 18.8 (Comfort-bound braking). Ego at 20 m s^{-1} ; target speed 5 m s^{-1} for obstacle ahead. Jerk-bounded braking profile: ramp jerk up to limit, hold, ramp down. Distance traveled: $\int v dt$. With $|j_{\max}| = 1.5$ and $a_{\max} = -3 \text{ m/s}^2$, braking takes $\sim 7.5 \text{ s}$, using 94 m of distance. Emergency braking (ignoring comfort) at 8 m s^{-2} stops in 1.9 s over 24 m. The 70 m difference is the cost of comfort—and the AV chooses comfort unless the situation escalates to emergency.

18.12 Planner Output and Handoff to Control

The planner emits a trajectory: discrete waypoints $(t_i, x_i, y_i, \theta_i, v_i, a_i)$ typically at 10 Hz to 100 Hz. The controller (Chapter 20) interpolates and tracks. A critical design decision: how far ahead to publish. Too close: controller does not have smoothing margin. Too far: planner’s stale horizon misrepresents the world. Typical: publish 1–2 seconds, refresh at 10 Hz.

18.13 Worked Examples

Example 18.9 (Full planning cycle). *Route*: 5 km trip, 6 intersections, computed in <1 ms. *Behavior*: at current position, follow-lane selected over change-lane based on lower cost in the current scene. *Motion*: MPC with $N = 30$, solved in 1 ms. *Output*: 30 trajectory waypoints at 0.1 s spacing, sent to controller. Total planning-layer latency: <5 ms per cycle, replanning at 10 Hz—comfortable.

Example 18.10 (Chance-constrained quintic lane change). Lane change with chance-constraint $p_{\text{coll}} \leq 0.05$. Predicted other-vehicle trajectories: 3 modes with probabilities (0.5, 0.3, 0.2). Only mode 3 (aggressive lane change) conflicts with an early merge; mode 3 probability $0.2 > 0.05$ chance budget, so early merge rejected. Delayed merge (longer quintic horizon): only mode 3 still conflicts but at lower collision-region overlap, $p = 0.04 < 0.05$: accepted. This is how chance-constraints shape planner behavior in multi-modal scenes.

Example 18.11 (Frenet jerk-minimizing trajectory). Start state: $(s_0, \dot{s}_0, \ddot{s}_0) = (0, 10, 0)$; $(d_0, \dot{d}_0, \ddot{d}_0) = (-1.5, 0, 0)$. Target: $(s_f, \dot{s}_f) = (30, 10)$; $(d_f, \dot{d}_f, \ddot{d}_f) = (0, 0, 0)$. Horizon $T = 3$ s. Compute quintic polynomial for $d(t)$ and quartic for $s(t)$: $6 + 5 = 11$ unknowns total, closed-form matrix solves. Peak lateral jerk 0.56 m s^{-3} ; comfortable. Longitudinal jerk zero (quartic s is constant-velocity). Published to control within $200 \mu\text{s}$.

18.14 Industrial Deployment

Planning stacks at production AV programs range from rule-based to optimisation-based to learned. Apollo (Baidu), open-sourced since 2017, provides a reference EM Planner that uses dynamic programming for path selection followed by quadratic-programming-based smoothing, and has been extensively documented in their public technical reports. Autoware (Tier IV, Japan), the open-source AV stack hosted by the Autoware Foundation, ships a reference planner used in research vehicles globally. Waymo, Cruise, and Chinese robotaxi operators have published peer-reviewed work on optimisation-based and learned planning; Tesla has described their use of Monte Carlo Tree Search and a “vector lanes” representation in their public AI Day presentations. The OpenScenario standard (ASAM, Germany) defines a scenario-description language used industry-wide for planning validation.

18.15 Chapter Summary

Planning is the layer that integrates perception, prediction, and rules into a concrete ego trajectory. Its hierarchy is route $\rightarrow$ behavior $\rightarrow$ motion $\rightarrow$ control. Motion planning is dominated by optimization-based methods (Frenet frame, MPC), with sampling and lattice methods filling special cases (parking, tight maneuvers). Collision checking against multi-modal predictions, interaction-aware planning, and comfort constraints shape the output. Production systems run route planning at <10 ms, motion planning at <50 ms per cycle, and deliver reference trajectories to the controller at 10 Hz.

18.16 Exercises

1. For the A* example (18.1), compute the expected speedup over Dijkstra for a graph of $N = 10^6$ nodes.
2. Formulate a POMDP for an unprotected left turn, specifying state, action, observation, transition, and reward.
3. Convert Cartesian $(42.5, -0.6)$ to Frenet coordinates given lane reference $(42, 0)$ with tangent $(0.98, 0.2)$.
4. A quintic lateral trajectory has coefficients $(-1.5, 0, 0, 1.0, -0.3, 0.02)$ (in terms of t). Compute the maximum jerk over $[0, T]$ with $T = 2$.
5. For the MPC-size example (18.6), compute the number of variables and constraints for $N = 50$ steps and 5-DoF state.
6. A chance-constraint budget is 0.01. Predicted collision mode probability is 0.008. How should the planner respond?
7. Derive the comfort-bounded stopping distance from initial speed v_0 with jerk limit j and accel limit a .
8. Explain why interaction-aware planners can outperform collision-avoidance-only planners in dense-traffic scenarios.
9. For the quintic lane-change example (18.11), modify the horizon to $T = 4$ s; compute the new peak jerk.
10. Design a planner-to-controller handoff schedule for a vehicle operating at 0 ms^{-1} to 30 ms^{-1} . What republish rate and look-ahead horizon make sense at each end of the range?

Chapter 19

Reinforcement Learning for Driving

19.1 Introduction

Supervised learning solves the problems where we have labeled examples: “this pixel is a pedestrian,” “this trajectory is the right one.” Driving has plenty of such problems, and most chapters so far have developed them. But driving also has problems where labels are unavailable or ambiguous: *should the ego merge now, or wait?; should it inch forward in stopped traffic or hold position?; is a gentle nudge against the pedestrian’s path acceptable if the alternative is blocking an emergency vehicle?* These are decision problems, and the right framework for them is reinforcement learning, where an agent learns a policy by interacting with an environment and receiving scalar reward signals.

Three observations motivate the chapter’s structure.

First, RL is powerful but fragile at deployment. A well-trained RL policy can produce impressively natural behavior in the scenarios it was trained on, but tends to fail catastrophically when the scene drifts out of its training distribution. This is especially dangerous in driving, where out-of-distribution failures are often the ones that matter most for safety. Production AV programs therefore rarely deploy pure RL policies for safety-critical decisions; RL sits inside broader architectures that constrain its outputs.

Second, the RL community has converged on a small set of algorithm families (DQN, PPO, SAC, model-based) that are broadly applicable with tuning. The art is less in picking the right algorithm and more in designing the MDP—state space, action space, reward function—so that the RL learning signal has a clear path to good behavior.

Third, the environment for driving RL is necessarily *simulated*. Training in the real world is too expensive and too dangerous; modern simulators (CARLA, Waymax, MetaDrive, NVIDIA DRIVE Sim) produce the interaction data RL requires. The sim-to-real gap is then the main research question: how to train in simulation in a way that generalizes to real-world driving.

The chapter is organized as follows: MDP fundamentals, the four algorithmic families (value-based,

policy-gradient, actor-critic, model-based), state and action space design for driving, reward design, simulators and sim-to-real, offline RL, safe RL, and applications in the production AV stack. Every formula has a worked example; three architecture diagrams visualize the key structures.

19.2 Markov Decision Processes

An MDP is a tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$:

- $\mathcal{S}$: state space.
- $\mathcal{A}$: action space.
- $T(s' | s, a)$: transition dynamics.
- $R(s, a)$: reward function.
- $\gamma \in [0, 1)$: discount factor.

A policy $\pi(a | s)$ maps states to action distributions; its value function is

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right]. \quad (19.1)$$

The optimal policy maximizes the value function: $\pi^* = \arg \max_\pi V^\pi(s)$ for all s . Bellman optimality:

$$V^*(s) = \max_a \left\{ R(s, a) + \gamma \sum_{s'} T(s' | s, a) V^*(s') \right\}. \quad (19.2)$$

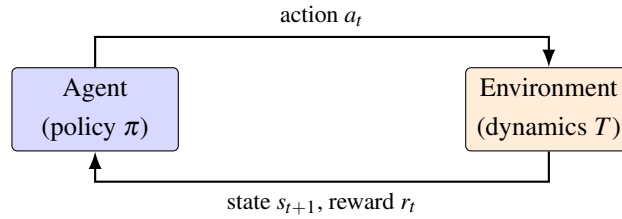


Figure 19.1: The agent–environment loop. At each step, the agent observes state s_t , chooses action a_t from its policy, and the environment returns a new state s_{t+1} and reward r_t . RL algorithms update the policy to maximize expected discounted reward over this loop. In driving RL, the environment is a simulator; the agent is a neural network policy; state and action are engineered from the driving context (perception outputs, ego plan, etc.).

Example 19.1 (Bellman optimality on a small MDP). MDP with 3 states and 2 actions. Rewards: $R(s_1, a_1) = 1$, $R(s_1, a_2) = -1$, $R(s_2, \cdot) = 2$, $R(s_3, \cdot) = 0$. Deterministic transitions with $\gamma = 0.9$. Compute V^* via value iteration: $V^{(0)} = 0$, then $V^{(k+1)}(s) = \max_a [R + \gamma V^{(k)}(s')]$. After convergence: $V^*(s_1) = 1 + 0.9V^*(s_2)$ (choosing a_1), etc. Closed form for this small problem;

$V^*(s_1) \approx 2.8, V^*(s_2) \approx 2.0, V^*(s_3) \approx 0$. Larger MDPs don't admit closed forms; RL approximates V (or related quantities) through interaction.

19.3 Algorithmic Families

19.3.1 Value-Based: DQN

DQN ([49]) learns the action-value function $Q(s, a)$ via temporal-difference update. The target:

$$Q^*(s, a) = R(s, a) + \gamma \mathbb{E}_{s'}[\max_{a'} Q^*(s', a')]. \quad (19.3)$$

Training minimizes Bellman-error: $\mathcal{L} = (Q_\theta(s, a) - y)^2$ with $y = r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$, using a target network $\bar{\theta}$ for stability.

Example 19.2 (Single DQN update). Sampled transition (s, a, r, s') with $r = 0.5$. Current $Q_\theta(s, a) = 1.2$; target $\max_{a'} Q_{\bar{\theta}}(s', a') = 2.1$; $\gamma = 0.99$. Target $y = 0.5 + 0.99 \cdot 2.1 = 2.579$. TD error: $y - Q(s, a) = 1.379$. Squared loss: 1.9. Gradient descent pulls $Q_\theta(s, a)$ toward 2.579. Over many samples, the network approximates the optimal Q^* .

DQN works best when actions are discrete and few. For driving RL, raw continuous control is handled by other methods.

19.3.2 Policy Gradient: REINFORCE to PPO

Policy-gradient methods directly learn a policy network π_θ . REINFORCE computes the policy-gradient estimator:

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi[\nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t], \quad (19.4)$$

with $G_t = \sum_{k \geq t} \gamma^{k-t} r_k$ the return. Variance reduction via baselines (subtract V); further reduction via trust-region updates.

Example 19.3 (REINFORCE step). Trajectory with single step: $\log \pi_\theta(a_t | s_t) = -0.9$, $G_t = 3.0$. Policy gradient contribution: $\nabla_\theta(-0.9) \cdot 3 = -2.7 \cdot \nabla_\theta \log \pi$. Update step (ascent): $\theta \leftarrow \theta + \alpha \cdot 2.7 \cdot \nabla_\theta \log \pi$. Over many trajectories, this increases $\pi(a_t | s_t)$ for high-return outcomes and decreases it for low-return ones.

PPO ([58]) adds a clipped-surrogate objective that bounds policy changes per update:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t)], \quad (19.5)$$

where $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ and $\hat{A}_t$ is the advantage estimate.

Example 19.4 (PPO clip arithmetic). $\varepsilon = 0.2$; $r_t(\theta) = 1.5$; $\hat{A}_t = 0.8$. Unclipped: $r \hat{A} = 1.2$. Clipped: $\text{clip}(1.5, 0.8, 1.2) \cdot 0.8 = 1.2 \cdot 0.8 = 0.96$. PPO uses min, so contributes 0.96. The clip

prevents the update from increasing $\pi(a)$ further than 20 % beyond old; this stabilizes training dramatically vs vanilla policy gradient.

19.3.3 Actor–Critic: SAC

SAC ([27]) combines an actor (policy π) and a critic (value function Q) trained jointly, with an entropy bonus that keeps the policy exploratory:

$$J_{\pi} = \mathbb{E}_s[\mathbb{E}_{a \sim \pi}[\alpha \log \pi(a | s) - Q(s, a)]] . \quad (19.6)$$

SAC is the production-scale algorithm of choice for continuous- action RL due to its stability and sample efficiency.

Example 19.5 (SAC entropy regularization). Policy $\pi(a | s)$ is a Gaussian with mean $\mu = 1.0$, std $\sigma = 0.5$. Entropy: $\frac{1}{2} \log(2\pi e \sigma^2) = \frac{1}{2} \log(2\pi e \cdot 0.25) \approx 0.725$. Entropy bonus with $\alpha = 0.1$: 0.0725 added to reward per step—small but crucial for preventing premature collapse to deterministic policies. As training progresses, α is often annealed (or auto-tuned via the dual gradient method in SAC) so that entropy matches a target exploration level.

Figure 19.2 shows the SAC architecture.

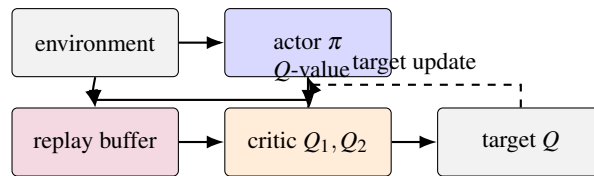


Figure 19.2: Soft Actor-Critic architecture. The actor network produces stochastic actions; the environment returns rewards to a replay buffer. Two critic networks (Q_1 , Q_2) estimate action values (double-Q reduces over-estimation bias). A slowly-updating target network $\bar{Q}$ stabilizes training. The actor’s gradient flows from a soft value combining expected Q with an entropy bonus. SAC has become the default choice for continuous-action RL.

19.3.4 Model-Based RL

Model-based methods learn a dynamics model $\hat{T}(s' | s, a)$ and plan with it (via MPC-style rollouts, or policy improvement against learned model returns). Pros: sample efficiency (10–100× fewer real steps); useful for offline or constrained simulators. Cons: model errors compound over long rollouts.

19.4 Designing State and Action Spaces for Driving

RL success in driving is primarily a function of MDP design.

19.4.1 State Representations

Common choices:

- **Raw sensor:** camera/LiDAR fed to a network that extracts a learned state. End-to-end flavor.
- **Perception output:** agent-list, BEV occupancy, lane-level map. Smaller, more structured, easier for RL.
- **Behavior-level abstraction:** already-planned maneuver with free parameters for RL to tune.

Production programs that use RL typically use the perception-output or behavior-level abstractions, not raw sensors.

19.4.2 Action Representations

- **Raw control:** (a, δ) . High bandwidth, hardest to learn safely.
- **Trajectory parameters:** e.g., target speed v and target lane ℓ ; classical planner executes the chosen parameters.
- **Behavior decisions:** discrete (yield/go/lane-change). Simplest, most interpretable; what production typically uses when RL is deployed.

19.5 Reward Design

The reward function encodes the objective. Driving rewards are composites:

$$R = w_p r_{\text{progress}} + w_c r_{\text{comfort}} + w_s r_{\text{safety}} + w_r r_{\text{rule-compliance}}, \quad (19.7)$$

with weights tuned for the tradeoff the deployment requires.

Example 19.6 (Reward shaping for lane keeping). $r_{\text{progress}} = v/v_{\text{target}}$ (1 when cruising correctly). $r_{\text{comfort}} = -(|\dot{a}| > 1.5) \cdot 1 - (|a_y| > 2.5) \cdot 1$ (penalty for exceeding comfort). $r_{\text{safety}} = -10$ if headway < 2 s, else 0. $r_{\text{rule}} = -100$ if lane boundary crossed, else 0. Weights: $(1, 0.5, 1, 1)$. An agent that drifts across a lane line gets ≈ -100 ; the learning signal is unambiguous. But a rule penalty that is too punitive can cause the policy to refuse to change lanes at all; balancing these pressures is the central art of reward design.

Sparse rewards (only terminal “success”/“failure”) are hard to learn from. Dense shaped rewards (per-step progress + penalties) are the production standard. Reward hacking is a perennial risk: the policy exploits a loophole in the reward function (e.g., maximizing average speed by cutting through parking lots). Validation-in-simulation over diverse scenarios catches most such failures; production systems additionally use runtime constraint satisfiers to prevent behavior from violating hard rules.

19.6 Simulators for Driving RL

- **CARLA:** Unreal-based, photorealistic, community-standard for research.

- **Waymax**: JAX-based, differentiable, specifically for behavior training; large-scale.
- **MetaDrive, SUMMIT, nuPlan**: scenario-focused, benchmark-friendly.
- **NVIDIA DRIVE Sim**: enterprise production simulation.

19.6.1 Sim-to-Real

The sim-to-real gap is narrower for behavior policies than for perception, because the behavior layer works on abstract state (no pixel-level sim-real gap). Domain randomization, adversarial training, and real-data distillation narrow it further.

19.7 Offline Reinforcement Learning

Offline RL [42] learns a policy from a fixed dataset without further environment interaction. Useful when interaction is expensive (driving), dangerous (live AV), or impossible (retrospective analysis of fleet data). Algorithms: CQL (conservative Q-learning), IQL, TD3+BC. The challenge: distributional shift—the learned policy wants to take actions that the dataset doesn’t support, so the value estimates for those actions are unreliable.

Example 19.7 (Offline RL from fleet data). A fleet of 100 vehicles generates 10^9 transitions over a year of operation, all logged. Offline RL trains a policy on this dataset to maximize progress minus comfort violations. The learned policy is then A/B-tested against the baseline planner on another fleet subset; if it wins on defined metrics (time-to-destination, intervention rate), it’s promoted. This loop has become the main way production AVs squeeze additional performance out of their driving logs without shipping untested behavior.

19.8 Safe RL

Safe RL adds hard safety constraints to the objective: $\max \mathbb{E}[R]$ subject to $\Pr[\text{violation}] \leq \epsilon$. Techniques:

- **Shielding**: a safety layer rejects unsafe actions, returning to a safe fallback.
- **Constrained Policy Optimization (CPO)**: Lagrangian constraint satisfaction within the policy-gradient update.
- **Certifiable bounds**: control-barrier functions (CBF) to prove safety mathematically for specific domains.

Example 19.8 (Shield rejection). RL policy proposes action “accelerate to 15 m s^{-1} ”. Safety shield evaluates: headway 1.8 s at current speed; accelerating would drop to 1.1 s—below 1.5 s minimum. Shield rejects, substitutes “maintain current speed”. Planner executes the shielded action. Shield log is fed back for offline training to discourage the RL policy from proposing

unsafe actions in the first place. This is the production pattern: RL proposes, shield decides, both improve over time.

19.9 Applications in the AV Stack

- **Behavior planning** (discrete tactical decisions): RL for lane-change timing, merge decisions, yield negotiation.
- **Comfort tuning**: RL for smoothness within a nominal classical planner's envelope.
- **Driving-policy simulation**: agent models for interactive traffic simulation (CARLA actors powered by RL).
- **Parameter tuning**: RL tunes MPC weights based on outcomes, replacing hand-tuning.
- **Scenario generation**: adversarial RL produces hard training scenarios for supervised systems.

Pure RL for end-to-end driving remains research-stage; hybrid architectures (RL for specific layers within a classical stack) are the production pattern.

19.10 Worked Examples

Example 19.9 (End-to-end RL training episode). Simulator: merging-onto-highway scenario. State: ego pose + 10 nearest-agent poses (220-d vector). Action: target Δv , target lane (continuous + discrete). Reward per step: progress (ego speed normalized) $- 0.1 \cdot (\text{close-agent distance penalty}) - 50$ on collision. Episode horizon: 30 s (~ 300 steps at 10 Hz). Training: 1M episodes across 32 parallel workers, ~ 3 days on a 4-GPU node. Evaluated on held-out scenarios: merge success rate 97%; mean time-to-complete 8.1 s; zero collisions in 10k evaluations. Deployed as a behavior-layer sub-policy invoked only for merge decisions; classical MPC handles everything else.

Example 19.10 (PPO training curve tranche analysis). PPO run on CARLA lane-keeping. Training metrics over 5M steps:

- Mean episode return: $0.3 \rightarrow 3.2$ (monotone).
- Policy entropy: $1.6 \rightarrow 0.9$ (expected; policy becomes more decisive).
- Value function loss: $0.5 \rightarrow 0.04$.

Hyperparameters: $\gamma = 0.99$, $\epsilon = 0.2$, batch size 2048, learning rate 3×10^{-4} . Typical sensitivity: γ below 0.95 induces myopic behavior; above 0.995 produces slow convergence. The learning rate and ϵ choices are the other two dominant tuning knobs.

Example 19.11 (CBF-shielded RL at a pedestrian crossing). Ego at 10 ms^{-1} approaching a crosswalk with a pedestrian detected. Control Barrier Function: $B(x) = d_{\text{pedestrian}} - d_{\text{safety}}$

with $d_{\text{safety}} = v^2 / (2a_{\text{brake}})$. RL proposes: accelerate. Shield computes: post-action $d_{\text{pedestrian}} = 8\text{ m}$, $d_{\text{safety}} = 10^2 / (2 \cdot 6) = 8.3\text{ m}$ —constraint violated. Shield substitutes: brake at $a = 3\text{ m/s}^2$. Verification passes (post-action $B > 0$). Vehicle brakes; pedestrian crosses; ego continues. The CBF approach gives a formally-verifiable safety guarantee (under modeling assumptions), on top of the RL policy’s ordinary objective. Production: RL inside a CBF-shielded envelope is a path being actively explored for safety-critical deployment.

19.11 Industrial Deployment

Production reinforcement-learning deployments in AVs remain limited compared to supervised learning, though several research-to-production examples are notable. Wayve (United Kingdom), founded in 2017, has built its driving stack around end-to-end imitation and reinforcement learning, recently attracting substantial investment including from SoftBank, Microsoft, and NVIDIA. DeepMind has applied reinforcement learning to optimisation problems in Waymo’s stack as part of Alphabet’s broader research portfolio. Offline reinforcement learning [42] as developed in academic research has gained adoption for fine-tuning learned policies from fleet logs without requiring on-vehicle exploration. The CARLA simulator (Intel Labs and Computer Vision Center, Barcelona), open-sourced in 2017, is the dominant academic RL environment for autonomous driving.

19.12 Chapter Summary

Reinforcement learning is the right framework for the decision problems in driving that supervised learning cannot easily solve: tactical behavior, parameter tuning, adversarial scenario generation. The key algorithms (DQN, PPO, SAC, model-based) are mature; the main engineering work is MDP design (state/action abstraction) and reward shaping, with simulator fidelity and sim- to-real the central research problems. Offline RL enables learning from fleet data without further interaction; safe RL adds hard safety constraints via shields or CBFs. Production deployment of RL in AVs is almost always as a sub-module within a classical hierarchy, not as an end-to-end replacement—the safety and interpretability of the classical stack complement the flexibility of the learned policy.

19.13 Exercises

1. For the MDP of Example 19.1, compute V^* with $\gamma = 0.5$ (instead of 0.9) and note how the valuation changes.
2. Derive the DQN gradient step from the squared Bellman-error loss and identify which quantities are treated as constants (target) during the gradient.
3. Write the PPO clipped-surrogate objective for a sample with $r_t = 0.4$ and $\hat{A}_t = -1.2$. Why is clipping one-sided here?
4. For the SAC entropy example (19.5), compute the entropy bonus if σ grows to 1.0 and explain the effect on training.

5. A dense reward function includes a term $r_{\text{comfort}} = -|j|$ (jerk penalty). Explain how this term can inadvertently prevent safety-critical evasive maneuvers.
6. Design a state representation for a lane-change RL policy. Identify at least five features and justify each.
7. For the offline RL example (19.7), estimate the storage required for 10^9 transitions with state dimension 256 and action dimension 4.
8. Describe a safety shield for a cruise-control RL policy that also handles the case where the sensor suddenly loses the lead vehicle.
9. Compare model-based RL to model-free RL for the driving application, considering sample efficiency, deployment cost, and failure modes.
10. Propose a scenario where CBF-shielded RL would outperform pure CBF-based control, and justify.

Part V

Vehicle Control and End-to-End Learning

Chapter 20

Vehicle Control

20.1 Introduction

The planner in Chapter 18 produced a reference trajectory: a sequence of waypoints with times and desired speeds. The controller’s job is to make the vehicle actually follow that trajectory through the throttle, brake, and steering actuators. It is the last bridge between the algorithms of the autonomy stack and the physical vehicle. Done well, the controller is invisible: passengers feel smooth motion, tracking error stays below what perception can even resolve, and the planner’s output is faithfully executed. Done poorly, the controller is everything: oscillating around lane lines, lurching at acceleration starts, missing braking targets by meters.

Three observations motivate the chapter’s structure.

First, control must handle *real physics*. The planner can ignore tire slip, actuator delay, and road banking in its kinematic model; the controller cannot. A successful controller is the one whose model captures enough of the real dynamics that the residual unmodeled effects can be rejected by feedback.

Second, control is a *real-time* problem. The controller runs at 50 Hz to 200 Hz, far faster than the planner (5 Hz to 10 Hz), because closing the loop tightly is what rejects the disturbances (wind, road grade, actuator lag) that accumulate over longer horizons. The loop-rate hierarchy—planner slow, controller fast—is architecturally fundamental.

Third, comfort is as important as tracking. A controller that meets its position target but produces large jerks fails. The specific cost functions used by production controllers include explicit penalties on acceleration and jerk, with hard limits on lateral acceleration and yaw rate that the planner respects reciprocally (Chapter 18).

The chapter is organized as follows: we first develop vehicle dynamics at a level the controller needs; then longitudinal control (speed tracking, adaptive cruise); then lateral control (lane keeping, path tracking); then model-predictive control (MPC) that unifies both; and finally the robustness

engineering that handles delay, disturbance, and model mismatch. Every formula is accompanied by a worked numerical example; three architecture diagrams visualize the key control loops.

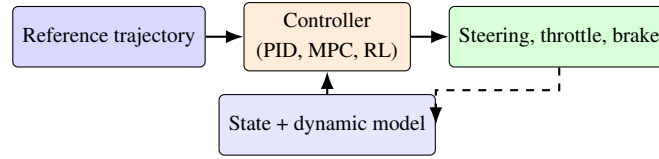


Figure 20.1: Chapter roadmap. The controller realises the planner’s trajectory under vehicle dynamics. Reference and current state feed the controller; commands flow to actuators; closed-loop feedback (dashed) updates the state model.

20.2 Vehicle Dynamics

A controller needs a model of vehicle motion. Three levels of fidelity are common.

20.2.1 Kinematic Bicycle Model

At low speeds, tire slip is small, and a kinematic bicycle model describes the vehicle:

$$\begin{aligned}\dot{x} &= v \cos \theta, \\ \dot{y} &= v \sin \theta, \\ \dot{\theta} &= \frac{v}{L} \tan \delta, \\ \dot{v} &= a,\end{aligned}\tag{20.1}$$

with position (x, y) , heading θ , speed v , wheelbase L , steering angle δ , and longitudinal acceleration a . Inputs: (δ, a) .

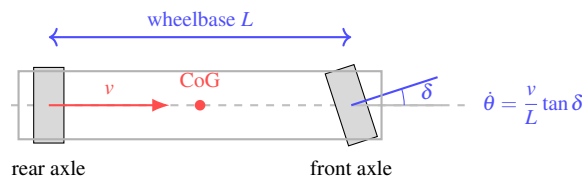


Figure 20.2: Kinematic bicycle model. The two front and two rear wheels are collapsed onto centreline pseudo-wheels at front and rear axles separated by wheelbase L . Steering angle δ is applied to the front pseudo-wheel. The body’s velocity v points along the heading; the yaw rate $\dot{\theta} = v \tan \delta / L$ follows from the geometry of the instantaneous turn centre. This model is accurate at parking-lot speeds where tire slip is negligible.

Example 20.1 (Kinematic bicycle step). $L = 2.7$ m, current state $v = 10$ m/s, $\delta = 0.1$ rad (steering into a right turn). Instantaneous yaw rate: $\dot{\theta} = 10 \tan(0.1) / 2.7 \approx 0.371$ rad/s ($\sim 21^\circ \text{s}^{-1}$). Turn radius: $R = v / \dot{\theta} \approx 26.9$ m. Over 0.1 s: heading changes by 2.1° , vehicle moves forward 1 m

and 1.8 cm laterally. This kinematic picture is accurate at parking-lot speeds but breaks down at highway speeds where tire slip angles become significant.

20.2.2 Dynamic Bicycle Model

At highway speeds, tire forces matter. The dynamic bicycle model adds tire slip:

$$\begin{aligned} m\ddot{y} &= F_{yf} \cos \delta + F_{yr}, \\ I_z \ddot{\theta} &= l_f F_{yf} \cos \delta - l_r F_{yr}, \end{aligned} \quad (20.2)$$

where m is mass, I_z is yaw inertia, l_f, l_r are distances from CoG to front/rear axles, and F_{yf}, F_{yr} are lateral tire forces that depend on slip angles.

20.2.3 Tire Models

Linear tire model: $F_y = C_\alpha \alpha$, where α is the slip angle and C_α is the cornering stiffness ($50\,000 \text{ N rad}^{-1}$ to $200\,000 \text{ N rad}^{-1}$ per axle typical). Valid for small slips ($|\alpha| < 5^\circ$); beyond that, the nonlinear Pacejka “magic formula” captures force saturation.

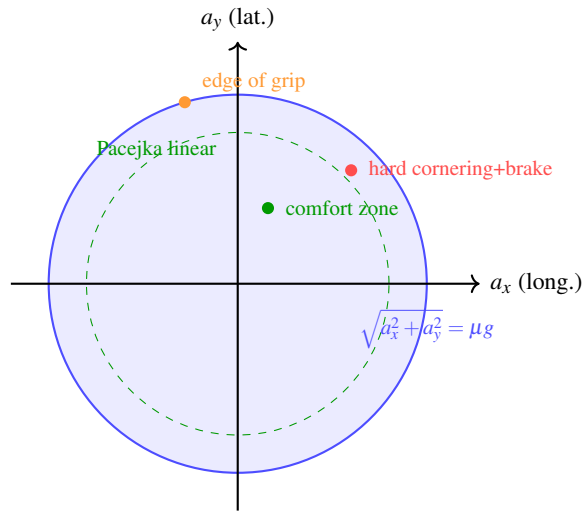


Figure 20.3: Friction-circle (or “g-g”) constraint. Combined longitudinal (a_x , brake/throttle) and lateral (a_y , cornering) accelerations must satisfy $\sqrt{a_x^2 + a_y^2} \leq \mu g$, where μ is the tire-road friction coefficient. The dashed inner ellipse shows the linear-Pacejka regime where $F_y \approx C_\alpha \alpha$ holds; beyond it tire forces saturate. Production NMPC formulations enforce the friction circle as a hard constraint (Chapter 21); LMPC linearises around an operating point and silently violates it at the edge of grip.

Example 20.2 (Linear tire force at 2° slip). $C_\alpha = 100,000 \text{ N/rad}$, $\alpha = 2 \cdot \pi/180 = 0.0349 \text{ rad}$. $F_y = 3490 \text{ N}$. For a 1800 kg vehicle, this produces lateral acceleration $F_y/m \approx 1.94 \text{ m/s}^2$ —half of 0.1 g—well within comfort limits. At $\alpha = 5^\circ$, linear prediction gives $1.94 \times 2.5 \approx 4.85$; the Pacejka model caps it at roughly 6 m/s^2 (friction limit on dry pavement).

20.3 Longitudinal Control

Longitudinal control matches the desired speed profile. Two dominant approaches.

20.3.1 PID with Feedforward

Proportional-Integral-Derivative feedback on speed error:

$$a_{\text{cmd}} = K_P e + K_I \int e \, dt + K_D \dot{e} + a_{\text{ff}}, \quad (20.3)$$

with $e = v_{\text{ref}} - v$ and a_{ff} a feedforward term (reference acceleration + drag/grade compensation). The integral term handles steady-state offset; the derivative damps the response.

Example 20.3 (PID tuning for ACC). $K_P = 0.8$, $K_I = 0.05$, $K_D = 0.1$. Current: $v = 20$ m/s, target $v_{\text{ref}} = 22$. Error: $e = 2$. Integrated-error history: $\int e \, dt = 5$. Derivative: $\dot{e} = 0.3$ (error was larger a moment ago). Command: $a_{\text{cmd}} = 0.8 \cdot 2 + 0.05 \cdot 5 + 0.1 \cdot 0.3 + a_{\text{ff}} = 1.63 + a_{\text{ff}}$. Typical tuning: K_P dominates the transient, K_I kills steady-state error, K_D prevents overshoot. Hand-tuning is tricky but well-documented; Ziegler-Nichols and loop-shaping methods systematize it.

20.3.2 Adaptive Cruise Control

ACC modifies the speed target based on lead-vehicle distance:

$$v_{\text{ref}} = \min(v_{\text{set}}, v_{\text{lead}} + k_d(d - d_{\text{desired}})), \quad (20.4)$$

with v_{set} the cruise setpoint, d the following distance, d_{desired} a time-gap target (e.g., $1.5v$ s). The feedback gain k_d on distance error shapes the following behavior: aggressive (high k_d) responds quickly but oscillates; gentle (low k_d) is smoother but slower to close gaps.

Example 20.4 (ACC following a slowing lead). $v = 25$, $v_{\text{lead}} = 20$, $d = 40$, $d_{\text{desired}} = 37.5$ (1.5 s gap at $v = 25$), $k_d = 0.3$. $v_{\text{ref}} = \min(30, 20 + 0.3 \cdot 2.5) = 20.75$. Ego decelerates toward 20.75 m s^{-1} ; once the gap reaches d_{desired} , v_{ref} converges to v_{lead} . On the lead braking harder, error grows, ACC demands higher deceleration, up to comfort limits; beyond those, AEB engages.

20.4 Lateral Control

Lateral control tracks a path. Three classical methods dominate.

20.4.1 Pure Pursuit

A geometric method that steers toward a target point at a look-ahead distance ℓ along the reference path:

$$\delta = \arctan \frac{2L \sin \alpha}{\ell}, \quad (20.5)$$

where α is the angle between the vehicle heading and the line to the target point. Simple, robust, easy to tune.

Example 20.5 (Pure-pursuit steering). $L = 2.7$, $\ell = 10$ (10 m look-ahead), $\alpha = 0.15$ rad (target slightly to the right). $\delta = \arctan(2 \cdot 2.7 \cdot \sin(0.15)/10) = \arctan(0.0807) \approx 0.0805$ rad. 4.6° steering. Look-ahead ℓ is the main tuning knob: too short oscillates, too long cuts corners. Production look-ahead is often speed-dependent (5 m to 30 m over the operating envelope).

20.4.2 Stanley Controller

Used by Stanley, the autonomous vehicle that won the 2005 DARPA Grand Challenge, and widely adopted since:

$$\delta = \theta_e + \arctan \frac{k_e e_y}{v + v_{\text{soft}}}, \quad (20.6)$$

with θ_e the heading error relative to path tangent, e_y the lateral error to the nearest path point, k_e a gain, and v_{soft} a small positive offset preventing a division-by-zero at low speed. Stanley tracks tightly at highway speeds and handles moderate path curvature gracefully.

20.4.3 LQR for Path Tracking

Linear quadratic regulator: solve a discrete Riccati equation for an optimal state-feedback gain $\mathbf{K}$ that minimizes $\sum (e^\top \mathbf{Q} e + u^\top \mathbf{R} u)$, where e is the tracking-error state (lateral, heading, etc.) and u is the steering command. Result: a linear feedback law $\delta = -\mathbf{K}e$ that is optimal for the linearized model.

Example 20.6 (LQR gain for lateral dynamics). State: $(e_y, \dot{e}_y, e_\theta, \dot{e}_\theta)$. $\mathbf{Q} = \text{diag}(1, 0.1, 1, 0.1)$, $\mathbf{R} = 0.01$. DLQR solve: $\mathbf{K} = (12.3, 4.1, 8.5, 2.7)$. Steering command: $\delta = -\mathbf{K}e$. If $e = (0.1, 0, 0.02, 0)$: $\delta = -(12.3 \cdot 0.1 + 8.5 \cdot 0.02) = -1.4$ rad—that's too aggressive; in practice $\mathbf{Q}$ and $\mathbf{R}$ tuning and saturation limits keep δ in a drivable range. LQR's strength is a principled structure for multivariable tuning.

20.5 Model Predictive Control

MPC (also developed in Chapter 18 as a planner) is equally useful as a combined longitudinal + lateral controller. Figure 20.4 shows the closed-loop structure.

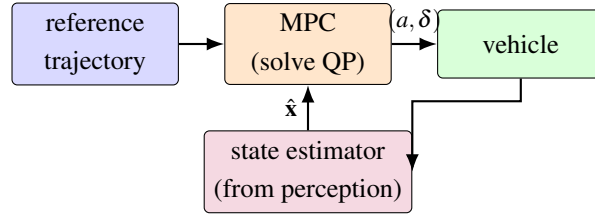


Figure 20.4: The control closed loop. Reference trajectory (from the planner) feeds a controller (here, an MPC) that produces the actuator commands (a, δ) . The vehicle responds; a state estimator (fed from the perception stack) recovers the new state for the controller’s next tick. Closing this loop at 50 Hz to 200 Hz is what rejects unmodeled disturbances and actuator errors that would otherwise accumulate.

20.5.1 Linear MPC as QP

Linearizing dynamics around the current trajectory, MPC solves at each tick:

$$\min \sum_{k=0}^{N-1} \|\mathbf{x}_k - \mathbf{x}_k^{\text{ref}}\|_{\mathbf{Q}}^2 + \|\mathbf{u}_k\|_{\mathbf{R}}^2, \quad (20.7)$$

subject to linearized dynamics, input bounds, and state constraints. The result is a QP with ~ 100 – 200 variables, solvable in under a millisecond.

Example 20.7 (MPC for trajectory tracking). Horizon $N = 20$ at 50 ms ticks ($T = 1$ s). State: (x, y, θ, v) ; input: (a, δ) . Decision variables: $20 \cdot (4 + 2) = 120$. Dynamics constraints: $20 \cdot 4 = 80$. Input bounds: $20 \cdot 2 \cdot 2 = 80$. QP solves in ~ 0.5 ms with OSQP. Controller runs at 50 Hz, well under the solve budget, with plenty of headroom for feasibility checks and safety fallbacks.

20.5.2 Nonlinear MPC

Nonlinear MPC retains the full dynamic bicycle and tire models. Required for aggressive maneuvers (emergency-brake evasion at friction limit, high-speed lane change). Solution time grows; SQP and interior-point methods typically deliver sub-10 ms solves even for nonlinear problems.

20.5.3 Terminal Cost and Stability

Adding a terminal cost $\|\mathbf{x}_N - \mathbf{x}_N^{\text{ref}}\|_{\mathbf{P}}^2$ and constraining the terminal state into a positive-invariant set provides a Lyapunov argument for stability. Practical automotive MPC often uses the LQR cost-to-go as the terminal cost, giving stability guarantees at modest engineering cost.

20.6 Delay, Disturbance, and Robustness

Real systems deviate from models. Production controllers must reject the deviations.

20.6.1 Actuator Delay

Throttle and brake actuators typically have 50 ms to 200 ms of delay; steering is faster (20 ms or less for electric power steering). Controllers compensate via Smith predictor, delay-augmented state, or

MPC with delay horizon.

Example 20.8 (Smith-predictor adjustment). Actuator delay $\tau = 150$ ms. Controller at t commands acceleration; vehicle executes at $t + 0.15$. Raw PID on measured error: oscillates (each update fights the previous command in flight). Smith predictor: subtract the predicted effect of in-flight commands from the measured error before applying PID. Oscillation vanishes; tracking settles in roughly one time constant.

20.6.2 Model Mismatch

Tire pressure, load, road surface—all vary. An adaptive controller re-tunes gains online; a robust controller (H_∞ , μ -synthesis) designs gains that are provably stable across a specified uncertainty envelope.

20.6.3 Road Banking and Grade

A banked curve and an uphill grade both produce gravity components that affect both longitudinal and lateral dynamics. Sensors (IMU + GNSS) provide grade and bank; feedforward compensation in the controller accounts for them.

20.7 Handover with the Planner

The planner-controller interface is a recurring design decision. Options:

- **Trajectory-based:** planner sends $(t_i, x_i, y_i, \theta_i, v_i)$ points; controller tracks. Most common.
- **State-reference:** planner sends a reference *dynamic* state, including target v, a ; controller follows with feedforward.
- **Decision-based:** planner sends discrete commands (lane, speed limit); controller's own internal trajectory generator fills in. Reduces coupling but requires a capable controller-level planner.

Production AV stacks overwhelmingly use trajectory-based handoff with a refresh rate of 10 Hz or faster.

20.8 Validation Beyond Tracking Error

Classical control metrics (tracking error) are necessary but not sufficient for AV deployment. Production validation includes:

- Worst-case tracking error across all scenarios and conditions.
- Jerk and acceleration statistics (comfort).
- Time-in-nominal percentage: fraction of driving time within comfort envelope.

- Fault response: behavior under sensor dropout, planner stall, actuator saturation.
- Closed-loop simulation replay on logged fleet data.

20.9 Worked Examples

Example 20.9 (End-to-end highway tracking cycle). Ego at 27 m s^{-1} on a two-lane highway. Planner publishes 10 Hz trajectory with $\Delta v = +0.5 \text{ m/s}$ over the next second and a 0.5 m lateral nudge. Controller runs MPC at 50 Hz, current tick: *Step 1*: state estimator reads $(x, y, \theta, v, \omega) = (100.0, 0.2, 0.01, 27.1, 0.02)$. *Step 2*: MPC solves QP with reference from interpolated planner trajectory; commands $(a, \delta) = (0.38, 0.008)$. *Step 3*: CAN publishes throttle 22 %, steering angle 0.5° . Vehicle responds over 50-ms cycle; next tick repeats. Tracking error RMS at steady state: 3 cm lateral, 0.05 m s^{-1} longitudinal.

Example 20.10 (Emergency-brake control profile). Collision imminent: planner issues emergency stop, target deceleration -8 m/s^2 . Controller commands full braking; actuator rises from nominal to max over 150 ms delay. From $v = 27 \text{ m/s}$: stopping distance $\approx 27^2 / (2 \cdot 8) + 27 \cdot 0.15 = 45.6 + 4.1 = 49.7 \text{ m}$. Without delay compensation (raw PID), overshoot delays full braking further, adding 5 m of stopping distance. Production AEB controllers measure and compensate actuator delay explicitly.

Example 20.11 (Wind-gust disturbance rejection). Side wind of 15 m/s gust, vehicle frontal area 2.5 m^2 , drag coefficient 0.3: disturbing lateral force $\sim \rho v_{\text{wind}}^2 A C_d / 2 \approx 82 \text{ N} \Rightarrow \sim 0.05 \text{ m/s}^2$ lateral accel on a 1800-kg vehicle. Uncontrolled drift over 1 s: 2.5 cm. Stanley controller with $k_e = 0.5$: detects lateral error within one tick, steers to restore. Closed-loop max lateral error under gust: 6 cm—within lane tolerance.

20.10 Industrial Deployment

Vehicle dynamics and control techniques in this chapter draw on a long industrial heritage in automotive control. Bosch, Continental, ZF, and Aisin have shipped production electronic stability control (ESC), antilock braking (ABS), and torque-vectoring systems for decades. Model predictive control has moved from academic research into production trajectory tracking at AV programs including Waymo and others. The do-MPC (TU Dortmund) and Acados (University of Freiburg) open-source MPC toolchains are widely used in research vehicles. The ISO 26262 functional safety standard governs the safety case for production vehicle control systems globally; ISO/PAS 21448 (SOTIF) governs the related safety case for the intended functionality limitations of perception and control systems.

20.11 Chapter Summary

Vehicle control bridges the autonomy stack and the physical car. Kinematic models suffice at low speed; dynamic models with tire saturation are needed at highway speed. Longitudinal control (PID + feedforward, ACC) and lateral control (pure pursuit, Stanley, LQR) are classical foundations; MPC unifies both in a single QP with explicit handling of constraints and delay. Production controllers must handle actuator delay, model mismatch, and road banking robustly. Validation extends far beyond tracking error to include comfort, worst-case scenarios, and fault response. The trajectory output of Chapter 18 flows through this chapter's controllers to the vehicle's actuators, closing the autonomy loop.

20.12 Exercises

1. For the kinematic-bicycle example (20.1), compute the turn radius and path swept over 2 s if δ is increased to 0.2 rad.
2. Show the linear tire model's breakdown: at what slip angle does a force extrapolated linearly from $C_\alpha = 100000$ exceed the friction limit μmg for $\mu = 0.8$?
3. Tune PID gains for an ACC that must reach a target speed within 5 seconds with overshoot $< 5\%$. State your plant model assumption.
4. Derive the pure-pursuit steering formula from the geometry of a circle tangent to the vehicle position and passing through the look-ahead point.
5. For the LQR example (20.6), how should $\mathbf{Q}$ change if we want slower but smoother response?
6. For the MPC trajectory-tracking example (20.7), estimate the solve time for horizons 10, 20, 50.
7. A Smith predictor compensates 200 ms delay. Propose a test that verifies the compensation is working, using only closed-loop data.
8. Compute the equilibrium sideslip angle for a vehicle in a steady-state curve at $v = 20\text{ m/s}$, $R = 100$, for the dynamic bicycle model.
9. For the wind-gust example (20.11), compute the closed-loop error for a gust of twice the magnitude.
10. Design a validation test suite for a highway-lane-keeping controller. List at least five scenarios and the pass/fail criteria for each.

Chapter 21

Model Predictive Control in Depth

21.1 Introduction

Chapter 20 introduced model predictive control as one member of a broader feedback-control toolkit. This chapter develops MPC as a discipline in its own right. MPC has become the dominant control paradigm for production autonomous vehicles—and for production driver-assistance systems—because it cleanly expresses two things at once: *constraints* (steering limits, friction limits, lateral comfort), and *multi-step lookahead* (a plan that respects what is coming, not just what is current). No other control framework expresses both as naturally.

Three observations motivate the depth here.

First, MPC is the bridge between planning and control. A trajectory chosen by a planner is realised by a controller; in MPC the boundary between the two is soft. Many production stacks push optimisation deeper into the controller, with the “planner” producing waypoints and constraints that an MPC solver translates into actuator commands.

Second, real-time MPC is an engineering achievement, not a free lunch. Solving a constrained non-linear optimisation every 50 ms on automotive-grade compute requires careful problem formulation, warm-starting, sparse-matrix algebra, and embedded solver tuning. Open-source toolchains have made this achievable for production teams over the past decade.

Third, MPC under uncertainty (stochastic, robust, learning- augmented) is the active research frontier and the area where production AV deployment is currently advancing fastest.

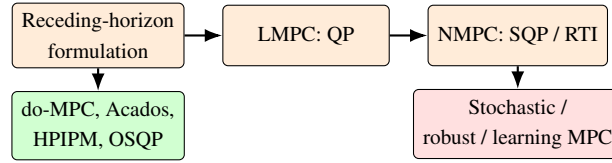


Figure 21.1: Chapter roadmap. MPC is studied at three depths: the general receding-horizon formulation, the linear-QP and nonlinear-SQP / RTI solvers, and the production toolchains (do-MPC, Acados, HPIPM, OSQP). Stochastic and robust extensions handle uncertainty.

21.2 The MPC Formulation

21.2.1 The General Problem

At each control step t , MPC solves an optimisation over a finite prediction horizon of N steps. With state $\mathbf{x} \in \mathbb{R}^{n_x}$ and control $\mathbf{u} \in \mathbb{R}^{n_u}$:

$$\min_{\mathbf{u}_{0:N-1}} \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_f(\mathbf{x}_N) \quad (21.1)$$

subject to

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1, \quad (21.2)$$

$$\mathbf{x}_0 = \mathbf{x}_{\text{measured}}, \quad (21.3)$$

$$\mathbf{x}_k \in \mathcal{X}, \quad \mathbf{u}_k \in \mathcal{U}, \quad (21.4)$$

$$\mathbf{x}_N \in \mathcal{X}_f. \quad (21.5)$$

The first control $\mathbf{u}_0^*$ from the optimal sequence is applied; the rest is discarded; the problem is re-solved at the next step. This *receding-horizon* structure is what gives MPC its closed-loop robustness: errors in the prediction model are corrected on the next solve.

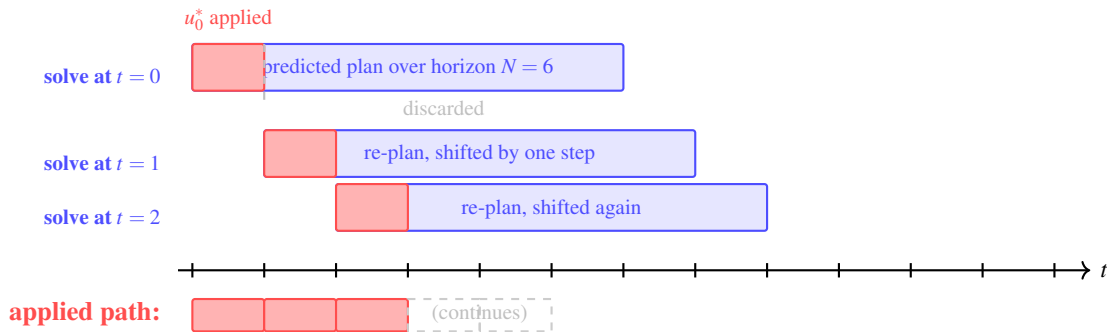


Figure 21.2: Receding-horizon MPC. At each control step t the solver predicts the optimal control sequence over a horizon of N future steps (blue rectangles). Only the first control u_0^* is applied (red); the rest is discarded. At step $t+1$ the entire plan is re-solved, shifted by one step. The horizon “recedes” as time advances. The closed-loop applied path (bottom row) is the concatenation of all the first-control segments. This rolling-window re-planning is what makes MPC robust to model error and disturbances: mistakes from the previous plan are corrected on the next solve.

21.2.2 Stage Cost Choices

The stage cost ℓ encodes the controller's preferences. For trajectory tracking:

$$\ell(\mathbf{x}, \mathbf{u}) = (\mathbf{x} - \mathbf{x}_{\text{ref}})^\top \mathbf{Q}(\mathbf{x} - \mathbf{x}_{\text{ref}}) + \mathbf{u}^\top \mathbf{R}\mathbf{u} + \Delta \mathbf{u}^\top \mathbf{S} \Delta \mathbf{u}.$$

Three weighting matrices: $\mathbf{Q}$ for state-tracking error, $\mathbf{R}$ for control effort, $\mathbf{S}$ for control rate (jerk-suppression term). Comfort objectives are usually expressed through $\mathbf{S}$.

21.3 Linear vs Nonlinear MPC

21.3.1 Linear MPC (LMPC) and QP Form

When dynamics are linear and constraints are polyhedral, the problem reduces to a quadratic program (QP):

$$\min_{\mathbf{z}} \frac{1}{2} \mathbf{z}^\top \mathbf{H} \mathbf{z} + \mathbf{g}^\top \mathbf{z} \quad \text{s.t.} \quad \mathbf{A} \mathbf{z} \leq \mathbf{b}. \quad (21.6)$$

Production QP solvers (OSQP, qpOASES, HPIPM) solve such problems to specified accuracy in under a millisecond on automotive-grade compute. The catch is that linearising the bicycle model loses fidelity at the steering, slip, and tire-saturation limits where control matters most.

21.3.2 Nonlinear MPC (NMPC)

NMPC retains the nonlinear dynamic model. Solver options include sequential quadratic programming (SQP), interior-point methods, and the Real-Time Iteration (RTI) scheme, which solves only one SQP iteration per control step. RTI trades worst-case optimality for predictable real-time behaviour and is the standard production approach for automotive NMPC.

Example 21.1 (Linear vs nonlinear MPC at the friction limit). A vehicle takes a curve at $v = 20 \text{ ms}^{-1}$, curvature $\kappa = 0.05 \text{ m}^{-1}$, friction coefficient $\mu = 0.4$ (wet road).

Step 1: Required lateral acceleration.

$$\begin{aligned} a_y &= v^2 \kappa \\ &= 400 \times 0.05 \\ &= 20 \text{ ms}^{-2}. \end{aligned}$$

Step 2: Available lateral acceleration.

$$\begin{aligned} a_{y,\text{max}} &= \mu g \\ &= 0.4 \times 9.81 \\ &\approx 3.9 \text{ ms}^{-2}. \end{aligned}$$

The required lateral acceleration exceeds the available friction budget by $5\times$.

Step 3: Linear MPC behaviour:

A linearisation around the nominal trajectory does not capture the friction-limit constraint as an active constraint; LMPC commands a steering input consistent with $a_y = 20 \text{ m s}^{-2}$. The vehicle slides.

Step 4: Nonlinear MPC behaviour:

NMPC's friction-circle constraint

$$\sqrt{a_x^2 + a_y^2} \leq \mu g$$

makes the problem infeasible at the requested speed. The solver returns a slowdown, and the speed-tracking penalty in the cost balances the drop. The vehicle exits the corner safely at $v_{\text{actual}} = 8.8 \text{ m s}^{-1}$.

Interpretation. At the friction limit—exactly where control matters most—linearisation discards the constraint that defines safety. NMPC keeps it. This is why production high-performance applications (motorsport, evasive manoeuvre, heavy-vehicle dynamics) use NMPC despite the higher solver cost.

21.4 Solving MPC in Real Time

21.4.1 Code Generation

Production MPC almost universally uses code-generated solvers: problem structure (matrix sparsity, dimensions, constraint patterns) is fixed at compile time, and a custom C-code solver is generated for that structure. This eliminates dynamic memory allocation and gives predictable execution time.

21.4.2 Warm Starting

The solution at step t is shifted by one step to provide an initial guess at step $t + 1$. With smooth dynamics and small disturbances, this initial guess is close to optimal and the solver converges in 1–3 iterations.

21.4.3 Real-Time Iteration (RTI)

Diehl, Bock, and colleagues introduced the RTI scheme in the early 2000s. The idea: do not iterate the SQP to convergence; do one iteration, apply, move on. Coupled with smooth dynamics and warm-starting, the closed loop remains close to the true optimum without ever reaching it on any single solve.

Example 21.2 (RTI execution time on automotive ARM). NMPC for a bicycle model with $N = 30$, $n_x = 6$, $n_u = 2$, running on an automotive ARM Cortex-A class processor.

Step 1: Per-iteration cost.

$$\begin{array}{rcl}
 T_{\text{linearisation}} & = & 0.4 \text{ ms}, \\
 T_{\text{QP-solve (HPIPM)}} & = & 1.2 \text{ ms}, \\
 \hline
 T_{\text{simulation}} & = & 0.3 \text{ ms}, \\
 T_{\text{RTI-step}} & \approx & 1.9 \text{ ms}.
 \end{array}$$

Step 2: Control-loop budget.

At a control rate of 100 Hz (10 ms cycle), RTI uses $\sim 20\%$ of cycle time, leaving margin for sensor preprocessing and actuator commit.

Step 3: Comparison to full-SQP convergence.

Full SQP-to-convergence on the same problem typically requires 4–8 iterations, $\sim 8\text{--}15$ ms per solve—outside a 10 ms budget on this processor. RTI’s predictability is the production differentiator.

21.5 Open-Source MPC Toolchains

Two open-source toolchains have shaped the academic and industrial use of MPC over the past decade.

21.5.1 do-MPC (TU Dortmund)

do-MPC is a Python-based modular toolbox developed at TU Dortmund University by the group of Sergio Lucia and collaborators. It is built on top of CasADi (a symbolic- differentiation framework from the optimisation-engineering community) and provides high-level abstractions for nonlinear MPC, robust MPC, and moving-horizon estimation. The package is released under the GNU Lesser General Public License and is widely used in academic research for prototyping and teaching. Production applications use do-MPC primarily during the design and tuning phase before transferring the resulting structure to a real-time-grade solver such as Acados.

21.5.2 Acados (University of Freiburg)

Acados, developed at the University of Freiburg in the group of Moritz Diehl and collaborators, is the leading open-source toolchain for real-time embedded NMPC. Written in C with Python and MATLAB bindings, Acados generates code for NMPC, moving-horizon estimation, and linear MPC, and integrates with the high-performance HPIPM (High-Performance Interior- Point Method) QP solver. The package is released under the 2-clause BSD license and is permissive enough to support commercial deployment without restriction. Acados implements the Real-Time Iteration scheme and supports automatic- differentiation through CasADi. The combination of speed, embedded-friendliness, permissive licensing, and active academic maintenance has made Acados the de facto standard for production AV NMPC at multiple programs.

Example 21.3 (do-MPC to Acados workflow). Typical research-to-production workflow for a new control problem.

Step 1: Prototyping in do-MPC (1–2 weeks).

Define dynamics, cost, and constraints in Python. Iterate on formulation: tune weights, add chance constraints, validate in Python simulation. Closed-loop simulation runs at offline speeds with no real-time pressure.

Step 2: Translation to Acados (1–2 weeks).

Re-express the same dynamics and cost in CasADi (the symbolic backend both packages share). Configure Acados RTI, HPIPM parameters, and embedded compile flags. Cross-validate against the do-MPC reference: identical step-response on a battery of test scenarios.

Step 3: Embedded deployment (2–6 weeks).

Code-generate, integrate into the vehicle’s control stack, benchmark on target hardware, tune solver iterations to fit the cycle-time budget. The resulting binary is self-contained, deterministic, and footprint-bounded.

Interpretation. The two-toolchain workflow—design in do-MPC, deploy in Acados—combines do-MPC’s iteration speed with Acados’s deployment-grade rigour. Most production AV control teams using open-source MPC follow some version of this pattern.

21.5.3 Other Production-Relevant Toolchains

Several other open-source projects appear in production AV MPC contexts. **CasADi** (KU Leuven) provides the symbolic differentiation underlying both do-MPC and Acados. **Forces Pro**, originating at ETH Zurich and now commercial, generates real-time-grade NMPC code. **OSQP** (Boyd group, Stanford) is widely used for the QP subproblem in production linear-MPC stacks. **HPIPM** (Frasch and Diehl, Freiburg) is the QP solver inside Acados for embedded NMPC.

21.6 MPC for Autonomous Driving

21.6.1 Trajectory Tracking

Given a planner-supplied reference trajectory, MPC computes steering, throttle, and brake commands that track the reference while respecting constraints. This is the standard production application: the planner runs at 5–20 Hz, the MPC controller at 50–100 Hz.

21.6.2 Joint Planning + Control

Some production stacks dissolve the planner-controller boundary by extending the MPC horizon to the full planning length and embedding the planner’s costs directly. The advantage is consistency—one optimisation, one set of constraints. The cost is solver complexity: full planning + control over a 5–8 s

horizon at 50 Hz approaches the limit of automotive embedded hardware.

21.6.3 Distributed MPC for Platoons

Truck platoons coordinate their controllers via V2V (Chapter 1). Each truck runs its own MPC with the following constraints from the truck ahead included as soft constraints. The result is a coupled control system with short reaction times and predictable string stability.

21.7 Stochastic and Robust MPC

The standard formulation assumes the model and disturbances are known. Production AVs cope with uncertainty in three ways.

21.7.1 Robust MPC

Tighten constraints to account for the worst-case disturbance. Conservative but provable. Min-max formulations are computationally heavy and rarely used in real-time AV control.

21.7.2 Stochastic MPC

Replace hard constraints with chance constraints:

$$\Pr(\mathbf{x}_k \in \mathcal{X}) \geq 1 - \alpha.$$

Less conservative; tractable under Gaussian-disturbance assumptions via cantelli or scenario-based reformulation.

21.7.3 Learning-Based MPC

Augment the dynamic model with a learned residual:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) + g_\theta(\mathbf{x}_k, \mathbf{u}_k),$$

where g_θ is trained to capture model error. Active research area at MIT, ETH Zurich, Berkeley, and University of Pennsylvania, with limited but growing production deployment.

21.8 Industrial Deployment

Production AV programs use MPC across the trajectory-tracking and lateral-control stack. Tesla's Autopilot stack publicly described path-planning that uses an optimisation-based formulation with QP-style solvers. Cruise (during its operational period), Waymo, and several Chinese AV programs have published academic papers describing MPC-based formulations, typically NMPC with Real-Time Iteration. Mobileye's RSS framework [60] is enforced as a constraint within an MPC formulation in some implementations. The truck-platooning trials by Daimler, Volvo, Scania, and the European Commission's ENSEMBLE project use distributed MPC for gap-keeping. The Acados toolchain,

originating at the University of Freiburg under Moritz Diehl, is widely cited in research publications from European AV programs and is used in pilot deployments under permissive 2-clause BSD licensing. The do-MPC toolchain at TU Dortmund (Sergio Lucia and colleagues) underpins much of the academic teaching and prototyping infrastructure across the European MPC community. Apex.AI's Apex.OS provides ISO 26262 ASIL D certification for the underlying real-time runtime that hosts production MPC solvers.

21.9 Chapter Summary

MPC is the dominant control paradigm for production AVs because it expresses constraints and multi-step lookahead in a single formulation. Linear MPC reduces to QP; NMPC handles the friction-limit and tire-saturation constraints that LMPC cannot. Real-Time Iteration trades optimality for predictable execution time and is the standard production approach. Open-source toolchains do-MPC (TU Dortmund, prototyping) and Acados (Freiburg, deployment) anchor much of the academic-to-industrial transition. Stochastic MPC, robust MPC, and learning-based MPC extend the framework to uncertainty—an active research front moving steadily into production.

21.10 Exercises

1. Derive the QP form of linear MPC explicitly. What are the decision variables and constraints?
2. For the friction-limit example, compute the maximum safe speed at the same curvature on a dry road ($\mu = 0.9$).
3. Explain why warm-starting MPC is essential at production cycle times. What happens without it?
4. For the RTI timing example, propose two strategies to fit the budget if the prediction horizon doubles to $N = 60$.
5. Describe a production scenario where LMPC fails and NMPC succeeds. Explain the failure mode quantitatively.
6. Compare the strengths of do-MPC and Acados. When would you use which?
7. Derive the chance-constrained reformulation of an MPC constraint $\mathbf{a}^\top \mathbf{x} \leq b$ assuming $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$.
8. Design an MPC formulation for ego-vehicle trajectory tracking that incorporates RSS-style safety constraints.
9. For the do-MPC to Acados workflow, identify three sources of bug introduction during translation and how to detect them.
10. Discuss the trade-offs of joint planning + control vs separate planner-then-controller architectures. When does each win?

Chapter 22

End-to-End Learning for Autonomous Driving

22.1 Introduction

Every chapter in this book so far has assumed a modular architecture: perception produces detections and tracks, prediction outputs trajectory distributions, planning optimizes a reference path, control tracks it. Each module has well-defined inputs and outputs, hand-designed cost functions, and a body of theoretical understanding. The classical autonomy stack is what runs on most vehicles in the road today. It is also, by the testimony of its practitioners, very hard to scale to the diversity of driving situations that the real world presents.

End-to-end learning proposes an alternative: let a single (large) neural network map sensor data to driving actions, without intermediate hand-designed representations. The attraction is dramatic—no fragile module boundaries, no hand-tuned cost functions, no piecewise testing. The reality is subtler. Pure end-to-end driving has been demonstrated in papers and prototypes for a decade, but production deployments remain hybrids: learned subsystems inside classical backbones, or learned backbones with rule-based safety layers on top.

Three observations motivate the chapter’s structure.

First, end-to-end is not a binary. There is a spectrum from fully modular (Chapters 3–20) to fully end-to-end (sensor pixels → steering wheel), with a productive middle ground of *learned modules within a structured hierarchy*. Production stacks in 2025–2026 occupy various points along this spectrum, each trading off data efficiency, interpretability, generalization, and safety differently.

Second, end-to-end systems are data hungry. They succeed when data is abundant and diverse; they fail when it is not. Driving distributions have a long tail of rare events, and end-to-end models generalize poorly from the common-case mass to the safety-critical tails without very deliberate data engineering.

Third, end-to-end is the natural home for *implicit reasoning* that classical stacks struggle with: inferring from subtle cues (a child’s gaze direction, a truck’s turn signal, an ambulance siren) what is about to happen. These are exactly the capabilities that foundation models developed from large, diverse data are best at; end-to-end architectures are the natural home for them.

The chapter is organized as follows. We first motivate end-to-end with its history and the core bet. We then develop imitation learning (behavior cloning, DAgger) as the core training paradigm, followed by conditional imitation and interpretable hybrids. Architecture patterns—ConditionalBC, RT-2-like foundation models, CARLA-style simulated RL—follow. We examine the causal-confusion and covariate-shift failure modes that are endemic to end-to-end driving, and the techniques that address them. We close with production hybrid architectures and the open research questions. Every formula has a worked example; three architecture diagrams visualize the key patterns.

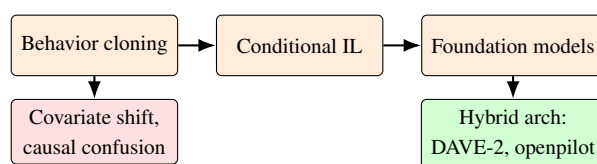


Figure 22.1: Chapter roadmap. Behavior cloning is the core paradigm; conditional and foundation-model variants extend it. Failure modes (covariate shift, causal confusion) and hybrid architectures (DAVE-2, openpilot) are studied as separate threads.

22.2 The End-to-End Spectrum

Figure 22.2 shows the architectural spectrum.

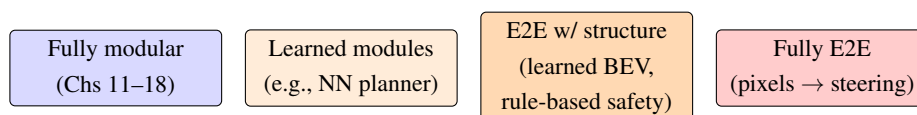


Figure 22.2: The end-to-end spectrum. Classical modular architectures (left) have interpretable boundaries and well-understood failure modes, but require substantial hand-engineering. Fully end-to-end systems (right) learn everything from data, trading interpretability for data efficiency and flexibility. Most production AV programs operate in the middle: learned perception and behavior modules within a structured hierarchy, often with rule-based safety envelopes.

To make the leftmost end of the spectrum concrete, Figure 22.3 expands the “fully modular” architecture into the detailed sub-system view used by most production AV programs through 2020. Each named module corresponds to a chapter or section in this book; coloured dashed feedback paths show the closed-loop dependencies that classical architectures must manage explicitly.

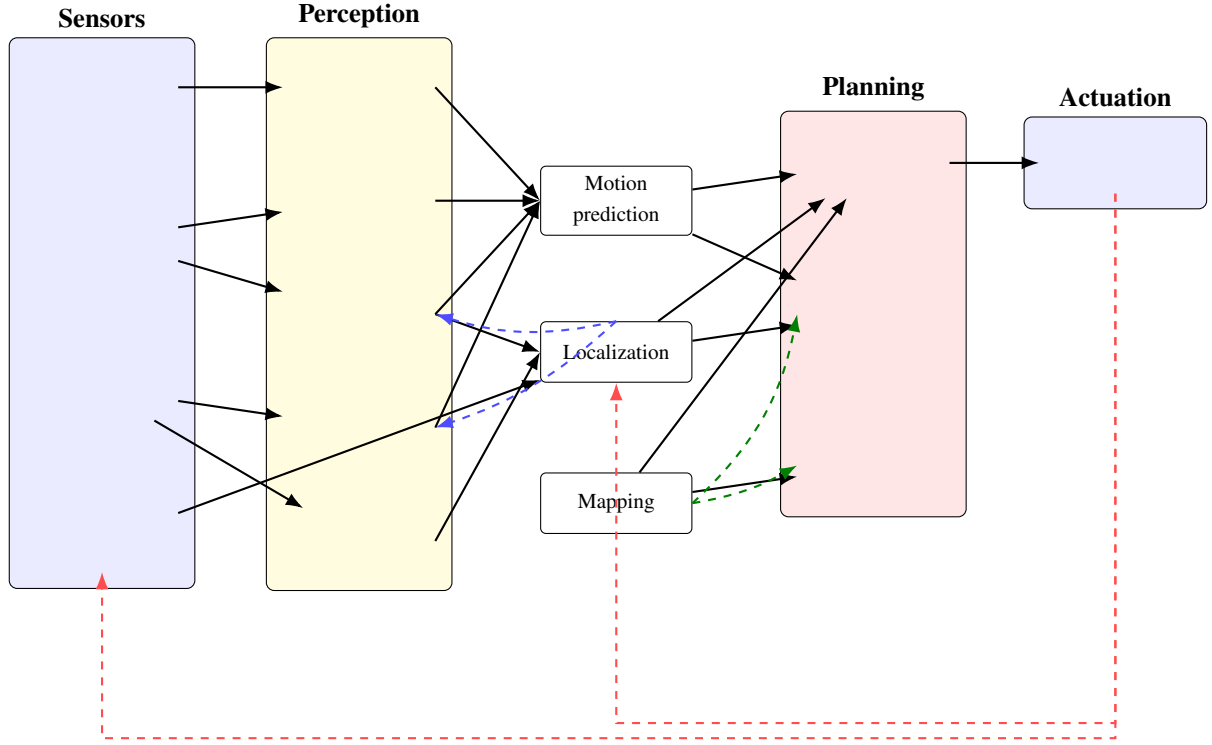


Figure 22.3: Detailed view of a classical modular AV architecture. Sensors (left, blue) feed perception modules (yellow); perception outputs feed reasoning modules (motion prediction, localization, mapping); reasoning feeds planning (red) which finally produces control commands. Solid black arrows show forward data flow; dashed red feedback paths show control-state and pose information returned to upstream modules; dashed blue paths show localization-priors fed back into perception (the position helps disambiguate which lane, which detected object); dashed green paths show map information flowing into route- and behaviour-planning. Each module is the subject of one or more chapters in this book.

This is the architecture DAVE-2 challenged.

22.3 Behavior Cloning: The Core

Behavior cloning is the simplest end-to-end approach: collect (sensor, action) pairs from expert (human or autonomous) driving; train a network to predict action given sensor input.

$$\mathcal{L}_{\text{BC}} = \mathbb{E}_{(s,a) \sim \mathcal{D}} [\|\pi_{\theta}(s) - a\|^2]. \quad (22.1)$$

Example 22.1 (Behavior cloning sample size). A fleet logs 100 vehicles driving 50 km/day, producing at 10 Hz sensor data: 10^9 frames/year. Even at 1 % selection rate (interesting driving), 10^7 training samples/year. This is enough to fit large networks; the quality question is whether the selection captures enough tail diversity. Tesla’s FSD program reports 10 billion-frame-class training sets; Waymo reports millions of hours of simulation plus real-world driving data.

22.3.1 The Covariate Shift Problem

Training data is collected under the expert policy, but the learned policy will visit states the expert did not. Small tracking errors accumulate; the policy enters states where its training data is thin; errors grow. This is *covariate shift*, and it is the central failure mode of naive behavior cloning.

Example 22.2 (Compounding error). Policy tracking error in-distribution: 5 cm/step std. Over 300 steps (30 s at 10 Hz), out-of-distribution error can grow quadratically: $\sqrt{300} \cdot 0.05 \approx 0.87$ m even in the best case. Naive behavior cloning does not correct for this; the policy continues to output actions appropriate for the clean expert state until it is too far off. In practice: naive behavior cloning cars start lane-following well, then drift and gradually fail.

22.3.2 DAgger and Its Descendants

DAgger (Dataset Aggregation) addresses covariate shift: deploy the learned policy, have the expert label the states it visits, retrain on the union. Iterate. This produces a policy robust to the states *it* actually visits, not just the states the expert visited.

22.3.3 Augmentation for Off-Distribution States

Synthesize off-distribution states and their corrective actions directly. For a camera image taken while driving straight, generate perturbed versions (simulated lateral offset, yaw error) paired with corrective steering commands. A simple recipe that substantially mitigates covariate shift without deployment.

22.4 Conditional Imitation Learning

Naive behavior cloning cannot handle branching decisions (at an intersection, left/straight/right are all valid). Conditional imitation learning [15] conditions the policy on a command: given “go left,” output action consistent with left turn. This resolves ambiguity by moving the decision to a higher module.

Example 22.3 (Conditional BC at an intersection). Input: front-camera image + command “turn right.” Output: (a, δ) appropriate for starting a right turn. At training: expert was commanded “turn right,” so only right-turn demonstrations label the right-turn branch; similarly for straight and left. Three separate heads, one per command, share the lower-level visual encoder. Production use: a behavior planner (classical or learned) emits the command; the low-level policy executes.

22.5 Foundation Models for Driving

Recent architectures (Wayve LINGO, Tesla FSD v12, GAIA-1, RT-2-style vision-language-action models) scale behavior cloning with foundation-model backbones. Billions of parameters, billions of training frames, and a predict-action head on top.

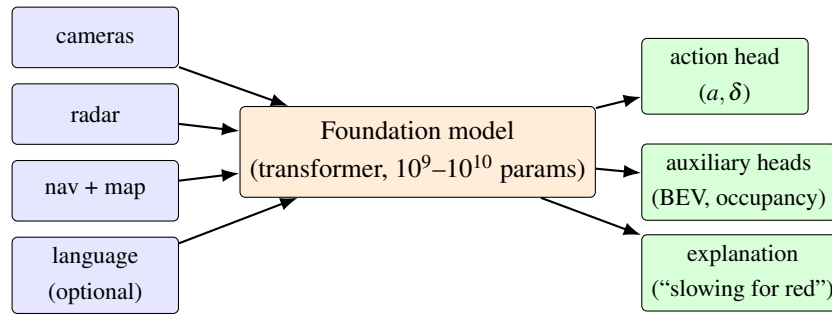


Figure 22.4: Foundation-model-style end-to-end driving. Multi-modal sensor inputs (cameras, radar, navigation, optional natural-language context) feed a large transformer backbone. The backbone produces action predictions (steering + throttle) along with auxiliary outputs—BEV semantics, occupancy maps, or natural-language explanations of decisions. Auxiliary heads both regularize training and provide interpretability when the system is deployed.

Example 22.4 (Foundation model auxiliary tasks). A single transformer backbone with three heads: (a) action head: (a, δ) at 50 Hz. (b) BEV head: 200×200 segmentation at 10 Hz. (c) language head: generate an English caption of the current driving decision (“yielding to pedestrian in crosswalk”). Training loss: $\mathcal{L} = \mathcal{L}_a + 0.1\mathcal{L}_{\text{BEV}} + 0.05\mathcal{L}_{\text{lang}}$. Auxiliary heads regularize the backbone and provide interpretability; the action head is the deployment output. At inference, only the action head is required; auxiliary heads can be pruned to save compute.

22.6 Causal Confusion and Related Pathologies

22.6.1 Causal Confusion

A behavior-cloning policy learns correlations, not causation. A classical failure mode: if the expert’s action includes both braking and tail-light illumination, a camera-based policy may learn to brake when the *front* of another car (also including brake lights) is visible, even in situations where braking is not the correct action. The policy has confused a downstream *effect* (brake lights) with an upstream *cause* (need to brake).

Mitigation: causal data augmentation (intervene on spurious correlates), counterfactual training, and explicit causal modeling.

Example 22.5 (Distinguishing correlation from causation). Training data: every brake-pedal-press correlates with brake-lights-on. Learned policy sees brake lights on the training set; learns to stop. At deployment: a parked car with brake lights on (driver left them on) triggers the same “stop” behavior—inappropriate. A robust policy distinguishes brake-activation (needs action) from brake-light-illumination (does not, per se). Counterfactual augmentation (add images with brake lights but no need to stop) forces the network to rely on the actual cause.

22.6.2 Shortcut Learning

Related: the network exploits a spurious correlation in the training set (“the right-turn action always follows a specific type of pavement”) rather than the intended feature. Revealed only when deployment data lacks the shortcut; the policy fails mysteriously.

22.7 Hybrid Architectures

Production deployments blend end-to-end with classical. Common patterns:

Learned perception + classical planner. Learned segmentation and detection (Chapters 11 and 12) feed a classical optimization-based planner. End-to-end-like data efficiency for the hard perception problem; interpretable, verifiable planning.

Learned planner + rule-based safety shield. A learned behavior policy proposes actions; a rule-based safety layer (Chapter 32) filters out unsafe ones. The learned policy handles complex scenes; the shield guarantees safety.

Learned trajectory with classical tracker. A learned network emits a reference trajectory; a classical MPC (Chapter 20) tracks it. Decouples flexible-decision from precise-execution.

Full end-to-end with interpretability head. The model is end-to-end but produces auxiliary outputs (predicted BEV, language explanations) that can be inspected. Interpretable without sacrificing the end-to-end advantage.

Example 22.6 (Production hybrid architecture). Stack: (1) multi-camera perception backbone producing BEV segmentation and agent list; (2) learned behavior planner taking BEV + route as input, outputting tactical decision (lane, target-speed); (3) classical Frenet-MPC planner that produces trajectory for the decision; (4) classical tracker. Learned components: backbone (billions of training frames) and behavior planner (hundreds of thousands of labeled scenarios). Classical components: Frenet optimization, MPC, safety shield. Combined, this stack sees most of the data-efficiency benefits of end-to-end while retaining the interpretability and verifiability of classical planning.

22.8 Training Data at Scale

Production end-to-end programs describe their data pipelines in terms of billions of frames and tens of thousands of interventions. Central to the pipeline is *shadow mode*: the learned policy runs in parallel to the classical one; any time they disagree, the disagreement becomes a training example (with the correct label determined by ground-truth or by fleet-review).

Additional techniques:

- **Diverse vehicle fleets** (sensor suites, operating regions, weather).
- **Simulation with real-data regularization** (sim environments replayed with real trajectories).
- **Targeted scenario collection** (fleet cars paid to drive particular routes for tail coverage).
- **Data engines** (automated curation based on novelty, uncertainty, or classical-policy agreement).

22.9 Evaluation Methodology

End-to-end policies cannot be evaluated purely by held-out imitation loss—that metric fails to capture closed-loop behavior. Evaluation:

- **Closed-loop simulation** on diverse scenarios (CARLA, NuPlan, internal simulators).
- **Shadow-mode deployment** with intervention-rate tracking.
- **Targeted scenario re-plays** (replay a safety-critical scenario with the learned policy in the loop).
- **Human driver comparison:** controlled experiments comparing learned-policy performance to human reference.

22.10 Worked Examples

Example 22.7 (End-to-end deployment sequence). Train a policy on 5×10^9 frames over 4 weeks. Evaluate in simulation: 95 % success on benchmark scenarios. Deploy in shadow mode on 100 vehicles; classical stack drives, learned policy predicts in parallel. Count disagreements: 5000/day. Human reviewers label each; disagreements where the classical policy was wrong become training examples (the learned policy’s behavior was correct); disagreements where the learned policy was wrong are used to tighten training. After 4 weeks of shadow-mode, disagreement rate drops by $3\times$; a subset of the fleet transitions to the learned policy; the rest remain shadow for data collection. This loop is the core of every production end-to-end program.

Example 22.8 (Handling a novel emergency-vehicle scenario). Novel scenario: an ambulance approaches from behind with lights on and sirens, a context not common in training data. Classical policy: perception might classify the ambulance correctly but planner rules lack “yield to emergency vehicle” logic in that specific configuration. Learned policy (trained on fleet data including ambulance-related events): has seen similar situations and knows the appropriate response (pull over and slow). The combination: rule-based shield allows only physical pull-over maneuvers that are legal; within that envelope, the learned policy produces the graceful response. Neither alone is sufficient—rules lack context, learning lacks guarantee.

Example 22.9 (Causal-confusion intervention). Observation: learned policy brakes more often when a red car is nearby than when other colors are nearby. Debug: red cars correlate in training data with brake-required scenarios (because stop signs are red, stopped emergency vehicles are often red). Intervention: counterfactual augmentation—generate synthetic images with red vehicles in not-braking contexts, and with non-red vehicles in braking contexts. Retrain. The color correlation disappears; policy responds to cause (stopped vehicle, stop sign) rather than spurious correlate (red).

22.11 Foundational and Open-Source Implementations

Two systems have shaped end-to-end driving more than any others: NVIDIA’s DAVE-2, which established the original architectural pattern; and Comma.ai’s openpilot, which has put the largest deployed open-source end-to-end-style driver-assistance system into the road. Each repays close study.

22.11.1 DAVE-2 (NVIDIA, 2016)

Bojarski et al. [7] from NVIDIA published the paper “End-to-End Learning for Self-Driving Cars,” demonstrating that a convolutional neural network could learn steering control from camera input alone—without lane detection, path planning, or any classical-vision modules. The system was named DAVE-2 in acknowledgement of DARPA’s earlier DAVE prototype from 2004 by Yann LeCun and colleagues at NEC Labs, which itself built on Pomerleau’s ALVINN (Autonomous Land Vehicle in a Neural Network) from CMU in the late 1980s. DAVE-2 brought modern deep convolutional networks, GPU compute, and a careful augmentation strategy to a problem that earlier systems had attacked with shallow, fully-connected networks.

The Vehicle and the Dataset

The DAVE-2 development vehicle was a 2016 Lincoln MKZ instrumented with three forward-facing cameras (left, centre, right) mounted on the dashboard, a vehicle-state interface returning steering wheel angle, speed, and gear selection, and an NVIDIA DRIVE PX embedded compute platform running the trained network at inference time. Training data was collected over approximately 72 hours of driving across diverse conditions: highways, residential streets, two-lane country roads, sun, cloud, rain, and night. The dataset size was modest by today’s standards—of order 10^5 training images after sub-sampling at 10 Hz from the original 30 Hz recording—which makes the eventual closed-loop result more striking.

The Three-Camera Trick

The single most important practical contribution of DAVE-2 was its *three-camera training augmentation*. From the left and right cameras (each offset roughly 0.5–1 m laterally from the centre), the system synthesised what the steering should have been if the vehicle had been displaced by that amount and needed to recover to the centre line. This addressed the covariate shift problem inherent to

behaviour cloning: a network trained only on on-policy human-driven trajectories never sees off-centre states and therefore never learns recovery, but the lateral cameras provide exactly such states with computable corrective labels.

Example 22.10 (DAVE-2 lateral-camera correction signal). For an image taken from a camera offset by Δy to the left of the centre, the corrective steering follows from a simple bicycle-model derivation. The vehicle must return to the centre line (lateral error zero) over a recovery time T_{recover} .

Step 1: Required lateral acceleration.

$$a_y = \frac{2\Delta y}{T_{\text{recover}}^2}.$$

For $\Delta y = 1 \text{ m}$ and $T_{\text{recover}} = 2 \text{ s}$:

$$a_y = 2/4 = 0.5 \text{ m s}^{-2}.$$

Step 2: Required path curvature at speed v .

$$\kappa = a_y/v^2.$$

At $v = 20 \text{ m s}^{-1}$:

$$\kappa = 0.5/400 = 1.25 \times 10^{-3} \text{ m}^{-1}.$$

Step 3: Steering angle (bicycle model, wheelbase L).

$$\delta_{\text{correct}} \approx L \cdot \kappa,$$

with $L = 2.7 \text{ m}$ for a Lincoln MKZ:

$$\begin{aligned} \delta_{\text{correct}} &\approx 2.7 \times 1.25 \times 10^{-3} \\ &\approx 0.0034 \text{ rad } (\approx 0.19^\circ \text{ at the wheel}). \end{aligned}$$

Interpretation. Even a 1-metre lateral shift yields a modest $\sim 0.2^\circ$ corrective steering at highway speed, but applied consistently across thousands of synthesised off-centre training samples it teaches the network the recovery dynamics that human drivers never explicitly demonstrate. The augmentation multiplied the effective dataset by roughly $3 \times$ (centre + left + right) while adding off-distribution coverage that pure on-policy data cannot provide.

The Network

The DAVE-2 architecture, while small by today’s standards, was carefully chosen:

- **Input.** 66×200 pixel YUV image (the original RGB camera frame downsampled and converted to YUV colour space, which decorrelates luminance from chrominance and was found empirically to train more robustly).
- **Normalisation layer.** A non-trainable layer mapping pixel values to zero-mean unit-variance.
- **Convolutional layers.** Five layers, the first three with 5×5 kernels and stride 2 (24, 36, 48 filters), the last two with 3×3 kernels and stride 1 (64, 64 filters). Activations were ELU.
- **Fully-connected layers.** Three layers with 100, 50, and 10 hidden units.
- **Output.** A single scalar: predicted inverse turning radius $1/r$. Steering wheel angle is recovered by multiplying with a vehicle-specific constant.

The total parameter count was approximately 250 000, and the network ran at 30 Hz on the DRIVE PX platform of 2016—two orders of magnitude smaller than a contemporary ImageNet classifier.

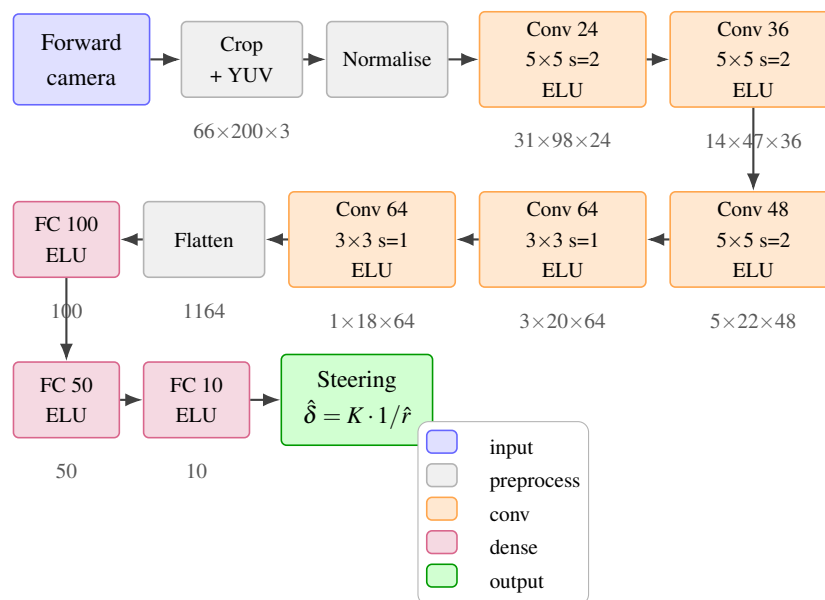


Figure 22.5: The DAVE-2 architecture (Bojarski et al., 2016) flowing in a single end-to-end pipeline from forward-camera input to steering output. Tensor shapes are annotated below each layer in *italic*. The *entire* stack—perception, prediction, planning, control—is collapsed into 5 convolutional and 3 fully-connected layers totalling $\approx 250\,000$ parameters. Compare with the classical modular architecture in Figure 1.2 (sensors $\rightarrow$ perception $\rightarrow$ prediction $\rightarrow$ planning $\rightarrow$ control), where each stage is a separately- designed and separately-trained module. DAVE-2’s contribution was to demonstrate that this modular decomposition—universally assumed in 2016—was *a choice*, not a requirement. The trained network’s salience maps subsequently revealed that it had developed its own internal lane-line and road-edge features purely from gradient descent on a steering-loss objective, without those features being labelled or designed for.

Training and Closed-Loop Results

Training used mean-squared error between predicted and recorded steering commands, with image augmentation comprising the three-camera lateral views, random shifts and rotations, and projection onto a synthetic flat-ground plane to simulate off-road shifts beyond what the cameras alone provided.

Training ran on a desktop GPU (NVIDIA Titan-class, single GPU) for approximately 72 hours.

The closed-loop evaluation metric was *autonomy*:

$$\text{Autonomy} = 1 - \frac{T_{\text{interventions}} \cdot 6}{T_{\text{drive}}},$$

where the factor of 6 represents an assumed average 6 seconds per human takeover (the time to engage, correct, and disengage). On the test driving, NVIDIA reported autonomy values of approximately 98 % on highways and lower on residential streets and country roads with poorly-marked lanes. The system was a research demonstration on a closed test track, not a production system.

Visualising What the Network Learned

A key follow-up paper from the same group (Bojarski et al. [6]) introduced *VisualBackProp*, a salience-map technique that showed where in the image the network attended when producing a steering command. The visualisations revealed something striking: without ever being told that lane lines, road edges, or vehicle boundaries were important, the network had learned to attend exactly to those features. This was perhaps the most important demonstration in the paper—the network had developed interpretable, lane-following-style internal features purely from gradient descent on steering loss. End-to-end learning was no longer the black box critics had assumed.

What DAVE-2 Established

DAVE-2 was a proof-of-concept rather than a production system. But it established three principles that every subsequent end-to-end driving system inherits:

- A convolutional network can map pixels to steering with end-to-end gradient descent. No hand-coded perception modules required.
- Synthetic view augmentation—specifically, the three-camera lateral-shift technique—provides the off-distribution coverage that pure on-policy human driving cannot.
- Small networks suffice for narrow operational design domains; complexity scales with the breadth of behaviours the system must produce.

The DAVE-2 architecture was open-sourced in part by NVIDIA (in their Deep Learning Institute teaching materials and various follow-up papers and code releases) and forked widely across the academic and hobbyist communities. Many subsequent end-to-end papers explicitly cite DAVE-2 as their starting point, from academic work on conditional imitation learning [15] to commercial systems including Comma.ai’s openpilot (whose original 2016 release was DAVE-2-architecturally inspired). Eight years after publication, the DAVE-2 paper remains one of the most-cited works in the autonomous-driving literature.

22.11.2 Comma.ai’s openpilot

Comma.ai (founded 2015 by George Hotz) develops openpilot, an open-source advanced driver-assistance system distributed under the MIT licence and installed via the company’s hardware product

(currently the comma 3X). openpilot supports a growing list of production vehicles by interfacing through their factory CAN-bus ADAS interfaces. As of 2024 the project’s installed base runs into the hundreds of thousands of vehicles, making it the largest deployed open-source driving system in the world.

Architecture

The openpilot system has evolved substantially since the original 2016 release. The current architecture uses:

- A learned end-to-end model (the “supercombo” model) that takes camera frames as input and outputs predicted vehicle trajectories along with auxiliary outputs (lane lines, lead-vehicle position, road edges).
- A model-predictive lateral controller that follows the predicted trajectory under steering and curvature constraints.
- A longitudinal controller that respects the predicted lead-vehicle behaviour and ACC-like setpoints.
- A driver-monitoring camera that gates engagement on driver attention (Level 2 hands-on supervision).

The supercombo model is trained on data uploaded from the deployed fleet—a real-world version of the closed-loop data flywheel that drives most production end-to-end stacks. Comma.ai has published model architectures, training data summaries, and several research papers describing the approach.

Why openpilot Matters

Beyond its installed base, openpilot has several aspects that make it pedagogically and practically important:

- **Open and inspectable.** The full source is available on GitHub. The model weights are released. The training pipeline is documented.
- **Deployed at scale.** Unlike academic end-to-end systems that demonstrate on closed courses, openpilot operates on public roads in many countries.
- **Hybrid in practice.** While branded as “end-to-end,” the actual production stack is a hybrid: learned trajectory prediction feeds rule-based MPC controllers with safety-checked engagement and disengagement criteria. This mirrors the pattern that has emerged across production end-to-end stacks more generally.
- **Evidence of the data-flywheel hypothesis.** Comma.ai has published evidence that performance improves with fleet size, supporting the broader industry hypothesis that the end-to-end approach scales with data better than rule-based alternatives.

Example 22.11 (openpilot’s hybrid architecture in operation). A vehicle with openpilot engaged on a highway, approaching a slowing lead vehicle.

Step 1: Perception (learned).

The supercombo model processes the forward camera frame and produces:

$$\{\hat{\mathbf{p}}_{\text{lead}}, \hat{v}_{\text{lead}}, \hat{a}_{\text{lead}}, \mathbf{x}_{\text{trajectory}}, \text{lane lines}\}.$$

Step 2: Longitudinal control (rule-based MPC).

A longitudinal MPC takes the predicted lead-vehicle state and computes a desired acceleration profile that maintains a minimum-headway constraint and a maximum-jerk constraint:

$$\begin{aligned} \min_{a_{0:N}} \sum_k (a_k - a_{\text{ref}})^2 + \lambda (\Delta a_k)^2 \\ \text{s.t.} \quad \text{headway}_k \geq T_{\min} \cdot v_k, \\ |a_k| \leq a_{\max}, \quad |\Delta a_k| \leq j_{\max}. \end{aligned}$$

Step 3: Lateral control (rule-based MPC).

A lateral MPC tracks the trajectory output of the learned model under steering- and curvature-rate constraints.

Step 4: Safety supervisor.

A separate process verifies driver attention via the in-cabin camera, monitors for sustained disagreement between model prediction and observation, and immediately disengages if any safety invariant is violated.

Interpretation. “End-to-end” in openpilot means the *perception* layer is end-to-end learned. Control remains constraint-based MPC; safety remains rule-based. This hybrid is the production pattern, not the pure-learning straw man.

Limits and Honest Caveats

openpilot is a Level 2 driver-assistance system. It requires constant driver supervision. Its ODD is highway-dominant; urban performance is significantly weaker. Disengagements are frequent on country roads, in construction zones, and at unusual intersections. The project is honest about these limits in its documentation. Confusing it for a Level 4 system would be a serious safety mistake.

22.12 Industrial Deployment

Beyond the foundational and open-source systems detailed above, production end-to-end learning has shifted from academic curiosity to active industrial deployment. Wayve (United Kingdom) has positioned its product around end-to-end learning, attracting over \$1 billion in funding by 2024. Tesla’s FSD V12, released to customers in 2024, was publicly described as a transition from hand-coded heuristics to a predominantly end-to-end neural-network architecture. Waabi (Toronto) has described an end-to-end neural-simulation-and-policy approach. The conditional imitation learning

approach [15] from the Computer Vision Center Barcelona influenced subsequent research and production work. Wayve’s GAIA driving-world-models and LINGO multimodal-driving-policy releases (2023–2024) illustrate the rapid pace of industrial publication in this space. Across the industry, the pattern is consistent: the *perception* layer is increasingly end-to-end learned, while control remains constraint-based and safety remains rule-based, producing the hybrid architectures that this chapter and Chapter 21 together describe.

22.13 Chapter Summary

End-to-end learning proposes a learned alternative to the classical modular AV stack. The pure end-to-end form (sensor to steering) is rare in production; hybrid architectures (learned modules within structured backbones, learned policies under rule-based safety shields) dominate. Behavior cloning is the core training paradigm; covariate shift and causal confusion are its central failure modes, addressed by DAgger, augmentation, and counterfactual training. Foundation-model backbones increasingly handle multi-modal input (sensors + language + navigation) with auxiliary-task outputs that regularize training and provide interpretability. Production programs invest heavily in data pipelines: shadow-mode collection, targeted scenario acquisition, and automated curation. The balance between modularity and end-to-end is an active area of industrial development; both ends of the spectrum have clear wins and clear failure modes.

22.14 Exercises

1. Write the naive-BC loss for a dataset of (s_i, a_i) pairs. Identify two ways it can fail.
2. For the covariate-shift example (22.2), compute the expected error after 1000 steps if the per-step error std is 0.03.
3. Describe the DAgger algorithm in pseudocode. What resources does it assume?
4. Design a conditional-imitation policy for a three-way intersection. Specify the command vocabulary and the head architecture.
5. Identify two additional auxiliary tasks that a foundation- model-style E2E system could benefit from. Justify each.
6. Propose a counterfactual data-augmentation strategy for the red-car causal-confusion problem (22.9).
7. Compare the data requirements of imitation learning vs reinforcement learning (Chapter 19) for the same driving task.
8. Describe the shadow-mode deployment’s role in data collection. What is the intervention rate a mature end-to-end system would aim for?

9. For the hybrid architecture example (22.6), propose three metrics that would compare it to a fully classical stack.
10. Explain why closed-loop simulation is necessary in addition to held-out loss for evaluating E2E policies.

Part VI

Specialized Perception and Deployment Domains

Chapter 23

Pedestrian Perception, Intent, and Interaction

23.1 Introduction

Of all the objects an autonomous vehicle must perceive, the pedestrian is the one where perception quality has the largest direct safety consequence. A missed traffic cone is embarrassing; a missed pedestrian is potentially fatal. Every public discussion of AV safety, every incident investigation, every regulatory framework makes pedestrian detection a first-rank concern, and engineering programs dedicate disproportionate effort to pedestrian-specific performance relative to other classes. This chapter develops pedestrian perception and behavior prediction as a dedicated discipline, distinct from the general object-detection chapter (Chapter 11) because the engineering problem itself is distinct.

Three observations motivate the chapter’s structure.

First, pedestrians are *visually diverse* in ways generic object categories are not. A pedestrian can be 1 meter tall (child) or 2 meters tall (tall adult); walking, running, standing still, sitting, or lying down; wearing anything from a bright reflective jacket to black rain gear at night; may carry a bicycle, push a stroller, use a wheelchair, or walk with a cane; and may appear against any conceivable background. No other single class in AV perception has such in-class variation.

Second, pedestrians are *behaviorally non-deterministic*. A pedestrian at a crosswalk edge may or may not cross; the only reliable way to know is to wait and see. But AV planners cannot wait and see—they must produce safe behavior under the uncertainty. Modern pedestrian systems therefore predict *distributions* over behavior, with crossing-intent being one of the most safety-consequential predictions.

Third, pedestrian performance must be *equitable across demographic groups*. Published research has repeatedly documented detection-accuracy gaps across skin tone, body size, mobility-aid use, and clothing type (Chapter 6). These gaps translate directly to unequal safety, and addressing them is both

an engineering and an ethical imperative.

The chapter is organized as follows: taxonomy of vulnerable road users, detection challenges specific to pedestrians, sensor modality choices, detection architectures, pose and action recognition, intent prediction, scenario-specific challenges (crosswalks, jaywalking, children), demographic equity, and integration with the planner. Two architecture diagrams visualize the full pedestrian-perception pipeline and the crosswalk decision cascade; worked examples adopt a hybrid prose-plus- aligned-calculation style throughout.

23.2 Vulnerable Road Users: Taxonomy

“Pedestrian” is a coarse category. For perception purposes, a finer taxonomy matters because different subtypes have different motion patterns, different detection difficulty, and different priority in safety reasoning.

23.2.1 Class Structure

Standing and Walking Adults

The canonical pedestrian. Upright, moving at 1.2 ms^{-1} to 1.6 ms^{-1} walking speed, height 1.5 m to 1.9 m. This is the most frequent subtype in urban scenes and the one detection datasets over-represent.

Children

Shorter (0.8 m to 1.5 m), less predictable motion patterns, may run suddenly into roads. Detection accuracy historically worse than for adults; targeted data collection and model improvements are ongoing across the industry.

Cyclists and Micromobility

Cyclists, electric scooter riders, skateboarders. Speed range 5 ms^{-1} to 15 ms^{-1} ; much faster than pedestrians and often in shared lanes. Separate category from pure pedestrians because the motion model differs substantially.

Mobility-Aid Users

Wheelchairs, walkers, canes, crutches. Motion is typically slower than walking. Historically under-represented in training data, leading to degraded detection.

Prone or Unusual Configurations

A pedestrian lying on the ground (injured, sleeping, or in rare cases staging a prank) is visually very different from a standing pedestrian. Detection must still succeed.

23.3 Pedestrian-Specific Detection Challenges

Why is a pedestrian detector a separate engineering problem from a general object detector?

23.3.1 Small Apparent Size at Range

A 1.75 m-tall pedestrian at 100 m range, viewed through a 60° horizontal FoV camera at 1920 px horizontal resolution, spans:

Example 23.1 (Pedestrian pixel span at range). Camera parameters:

- Horizontal FoV: $\theta_h = 60^\circ$.
- Horizontal pixels: $N_h = 1920$.
- Pedestrian height: $h = 1.75$ m.
- Range: $r = 100$ m.

Angular size of pedestrian (small-angle):

$$\begin{aligned}\theta_p &= \frac{h}{r} \\ &= \frac{1.75}{100} \\ &= 0.0175 \text{ rad } (\approx 1.00^\circ).\end{aligned}$$

Vertical pixel span (assuming same focal length as horizontal):

$$\begin{aligned}n_{\text{px}}^v &= \frac{\theta_p}{\theta_h} \cdot N_h \\ &= \frac{0.0175}{1.047} \cdot 1920 \\ &\approx 32 \text{ pixels}.\end{aligned}$$

A 32-pixel-tall pedestrian is detectable but not distinguishable from other upright pole-like objects (signs, thin poles) without fine texture. Classifiers trained with tight crops on small objects are needed. At 150 m range the span drops to 21 pixels; at 200 m, 16 pixels—pushing the limit of what camera-based detection can achieve.

23.3.2 Occlusion and Partial Visibility

Urban scenes routinely present partially-occluded pedestrians: half-hidden behind a parked car, torso visible but legs occluded, or only a head visible above a barrier. A detector trained on full-body examples often fails on partially-occluded ones unless specifically augmented.

23.3.3 Unusual Poses and Attire

A pedestrian bending to tie a shoe is not upright; a pedestrian holding an umbrella has an altered silhouette; a pedestrian wearing bulky winter clothing has a different shape than one in summer clothing. All of these degrade generic body-shape detectors.

23.3.4 Motion Blur and Viewpoint

At high relative speeds (pedestrian running across an intersection while AV is moving), motion blur degrades detection. Viewpoint variation (pedestrian facing the AV vs walking away) is another axis of variation detectors must handle.

23.4 Sensor Modality Choices

Different sensors contribute differently to pedestrian perception.

23.4.1 Camera

Best for classification (distinguishing pedestrian from other vertical objects) and for fine-grained features (pose, clothing, gaze direction). Degrades at night and in rain.

23.4.2 LiDAR

Best for 3D localization and range estimation. Robust to lighting but less effective at distinguishing pedestrians from similarly-sized non-pedestrians (e.g., signposts, poles) without camera assistance.

Example 23.2 (LiDAR-based pedestrian range precision). A pedestrian at nominal 50 m range. LiDAR with 2 cm ranging noise and angular resolution 0.1° (1.75 mrad).

Cross-range spatial resolution:

$$\begin{aligned}\Delta x_{\perp} &= r \cdot \theta_{\text{res}} \\ &= 50 \cdot 1.75 \times 10^{-3} \\ &= 0.087 \text{ m.}\end{aligned}$$

Number of vertical returns on pedestrian torso (height 1 m):

$$\begin{aligned}N_{\text{returns}} &= \frac{1.0 \text{ m}}{\Delta x_{\perp}} \\ &\approx 11.\end{aligned}$$

Eleven vertical points on the torso at 50 m is enough for a bounding-box estimate. At 100 m the count halves; detection becomes fragile and must rely on classification cues from camera.

23.4.3 Radar

Doppler signatures help distinguish pedestrians (slow, relatively unreflective) from vehicles (fast, metallic). Micro-Doppler from limb motion (gait signature) further discriminates pedestrians from other objects. Operates well in rain and darkness.

23.4.4 Thermal

Thermal imaging (Chapter 8) is particularly valuable for pedestrian detection in darkness, fog, or against low-contrast backgrounds. Body heat produces a distinctive signature that is detectable at ranges $3\text{--}5\times$ beyond what visible cameras achieve at night.

23.5 Pedestrian Detection Architectures

Detection architectures from Chapter 11 apply directly to pedestrians, with specific pedestrian-tuned variations.

23.5.1 Dedicated Pedestrian Heads

Multi-task models with a dedicated head for “person” detection, trained with class-weighted losses that emphasize pedestrian recall over other classes. The weighting shifts operating points toward higher recall at the cost of more false positives, acceptable since pedestrian false-negatives are disproportionately costly.

23.5.2 Small-Object-Optimized Architectures

Architectures that preserve high-resolution feature maps (FPN, HRNet, Swin) retain the fine detail needed for long-range pedestrian detection. Standard low-resolution detectors are suboptimal at the critical range regime.

23.5.3 Multi-Modal Fusion

Camera + LiDAR fusion improves over either alone because each compensates for the other’s weaknesses. Production pedestrian detection is rarely single-modality.

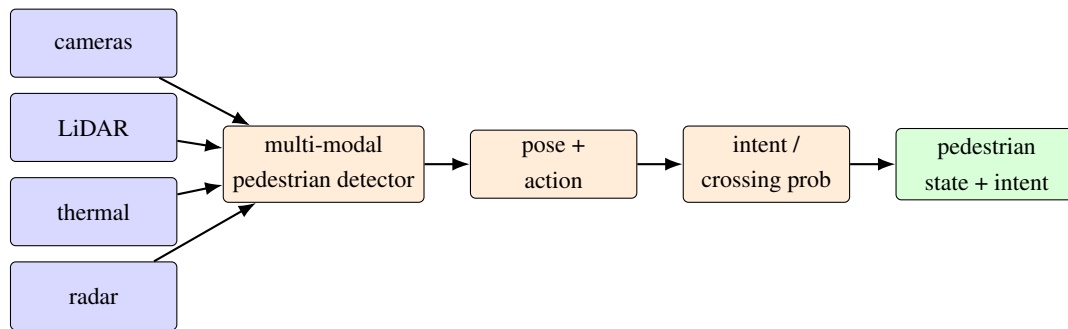


Figure 23.1: Pedestrian perception pipeline. Multiple sensor modalities feed a dedicated pedestrian detector that outputs 2D and 3D bounding boxes with classification. Detected pedestrians flow into a pose/action-recognition stage and then an intent-prediction stage producing crossing probability and trajectory distribution. The output feeds the planner’s pedestrian-interaction logic.

23.6 Pose and Action Recognition

Beyond detection, *what is the pedestrian doing?* Pose and action recognition provide context for intent prediction.

23.6.1 Skeletal Keypoints

Estimate pedestrian body keypoints (head, shoulders, hips, knees) in 2D or 3D. Architectures like OpenPose, AlphaPose, and HRNet produce keypoint estimates in real time.

23.6.2 Action Classification

Given a keypoint sequence or video clip, classify the action: walking, running, standing, sitting, bending. Temporal architectures (LSTMs, 3D CNNs, transformers) do this.

23.6.3 Gaze Direction

Pedestrian head orientation is a strong intent cue: a pedestrian looking at the AV is more likely to cross safely; one looking away is more likely to be unaware. Gaze estimation from head- pose regression is now standard in production stacks.

Example 23.3 (Action classification confidence). A 2-second clip of a pedestrian near a crosswalk. Action classifier output:

- walking: 0.55
- standing still: 0.30
- running: 0.12
- other: 0.03

Crossing-intent prior from action:

If walking toward the road, boost crossing prior. If standing, reduce. Specifically, apply a

Bayesian update with class- conditional likelihood of crossing given action:

$$P(\text{cross} \mid \text{walk}) = 0.60,$$

$$P(\text{cross} \mid \text{stand}) = 0.15,$$

$$P(\text{cross} \mid \text{run}) = 0.80.$$

Marginalized crossing probability:

$$\begin{aligned} P(\text{cross}) &= \sum_a P(\text{cross} \mid a) \cdot P(a) \\ &= 0.60 \cdot 0.55 + 0.15 \cdot 0.30 \\ &\quad + 0.80 \cdot 0.12 + 0 \\ &= 0.33 + 0.045 + 0.096 \\ &= 0.47. \end{aligned}$$

A 47 % crossing probability is high enough to trigger a yielding plan. If gaze direction indicates pedestrian is looking at the AV (further evidence of intent to cross), probability rises further.

23.7 Intent Prediction

The hardest and most safety-critical pedestrian-related inference. Given observation, what is the probability the pedestrian crosses into the AV's path in the next few seconds?

23.7.1 Features for Intent

- Position and velocity relative to crosswalk / road edge.
- Body orientation (facing road / away from road).
- Gaze direction (looking for traffic / not looking).
- Action (walking, standing, running).
- Social context (other pedestrians, traffic signals).
- Historical behavior over the past few seconds.

23.7.2 Temporal Modeling

Intent is a temporal property. Architectures that model the trajectory of features over time (LSTMs, transformers over feature time-series) outperform single-frame classifiers.

23.7.3 Calibration of Probabilities

Intent probabilities must be calibrated: a “70% crossing probability” from the model should correspond to actual crossing in 70% of cases. Uncalibrated models produce plans either too conservative (phantom yields) or too aggressive (pedestrian struck). Evaluation metrics (Brier score, reliability diagrams) measure calibration.

Example 23.4 (Calibration gap and its cost). Model predicts crossing probability $p = 0.3$ for a particular scenario class.

Actual crossing rate in class: 0.55.

The model is systematically under-predicting (20-point gap). Consequence: plans generated assume 30% yielding-probability budget, but reality is 55%—over the threshold at which emergency braking would have been warranted. This calibration gap is a safety-critical bug; retrain with a calibration loss (temperature scaling, isotonic regression on a held-out set) to close it.

23.8 Scenario-Specific Challenges

Different pedestrian scenarios require different engineering attention.

23.8.1 Controlled Crosswalks

A marked crosswalk with traffic signal. Pedestrian behavior is structured: wait for walk signal, cross in the crosswalk. Intent prediction simplifies; main challenge is recognizing sign/ signal phase and detecting pedestrians at the crosswalk edge.

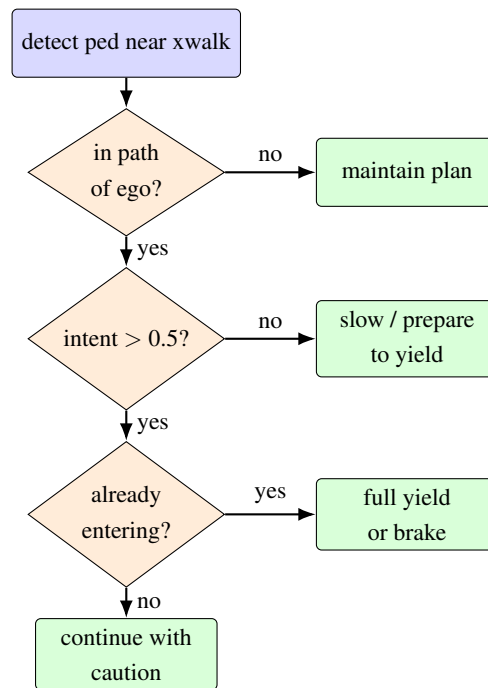


Figure 23.2: Crosswalk decision cascade. Once a pedestrian is detected near a crosswalk, a sequence of decisions determines the ego’s behavior: is the pedestrian in the ego’s path? Does intent prediction indicate crossing? Has crossing already begun? Each answer produces a specific ego action, from maintain-plan to full-brake. The cascade is deliberately conservative: thresholds err toward yielding.

23.8.2 Uncontrolled Crossings (Jaywalking)

No marked crosswalk, no signal. Pedestrian may cross anywhere. Much harder scenario because the spatial prior on crossing location is weak. Relies heavily on pedestrian-specific cues (orientation, gaze, motion) rather than infrastructure.

23.8.3 School Zones

Children are over-represented; behavior is less predictable; speed limits are lower. AV speed envelopes shrink; detection thresholds tighten. Some AV programs explicitly classify school zones as a separate ODD requiring different behavior.

23.8.4 Dense Crowds

Stadium egress, transit stations, holiday shopping districts. Dozens to hundreds of pedestrians in close proximity. Individual- pedestrian tracking is difficult; crowd-level reasoning replaces per-individual reasoning.

23.9 Demographic Equity

A pedestrian detector’s accuracy must be acceptable for all groups. Chapter 6 developed the ethical framework; this section describes the specific pedestrian- detection dimensions.

23.9.1 Measurement Protocol

Release evaluation must stratify pedestrian AP across:

- Height bins (child, short adult, average, tall).
- Mobility-aid use (none, cane, walker, wheelchair).
- Skin tone (Fitzpatrick I–VI).
- Clothing types (bulky, dark, bright, uniform).
- Lighting and weather conditions.

23.9.2 Closing the Gaps

When gaps emerge:

- Targeted data collection for under-represented categories.
- Subgroup-balanced sampling in training batches.
- Adversarial-fairness augmentation to force invariance.
- Temporal monitoring: track subgroup metrics per release and flag regressions.

23.10 Integration with the Planner

Pedestrian perception outputs feed the planner through well-defined interfaces.

23.10.1 Data Passed

- 3D bounding box (position, size, orientation).
- Velocity estimate.
- Classification confidence.
- Pose features (if available).
- Intent probability (crossing / not crossing within horizon).
- Predicted trajectory distribution.

23.10.2 Planning-Layer Usage

The planner uses pedestrian data to:

- Adjust speed envelope around pedestrian-rich regions.

- Generate conservative trajectory plans near uncertain pedestrians.
- Trigger emergency-braking when a pedestrian enters the collision-imminent region.
- Signal to pedestrians (in some vehicles, via exterior communication).

Example 23.5 (Planner response to pedestrian). Pedestrian at crosswalk edge, 25 m ahead. Ego speed 12 m s^{-1} . Intent probability: $p = 0.65$ (high).

Time-to-crosswalk at current speed:

$$\begin{aligned} t_{\text{TC}} &= \frac{d}{v} \\ &= \frac{25}{12} \\ &\approx 2.08 \text{ s}. \end{aligned}$$

Required deceleration to stop at the crosswalk:

$$\begin{aligned} a_{\text{brake}} &= \frac{v^2}{2d} \\ &= \frac{144}{50} \\ &= 2.88 \text{ m/s}^2. \end{aligned}$$

This is comfortable deceleration ($< 3.5 \text{ m s}^{-2}$). Planner initiates moderate braking; if pedestrian begins crossing, continue to full yield; if pedestrian turns away, release brake and continue. Either way, the 2 s horizon provides margin for the planner to react proportionally to revealed behavior.

23.11 Industrial Deployment

Pedestrian-detection performance is publicly evaluated by Euro NCAP and IIHS as part of their AEB (Automatic Emergency Braking) ratings. Euro NCAP introduced AEB Vulnerable Road User test protocols in 2016 and has tightened the requirements through subsequent protocol revisions. Mobileye's collision-warning and AEB products are deployed across many OEMs. Researchers at Daimler (Mercedes-Benz Group) and Toyota Research Institute have published extensively on pedestrian-detection benchmarks. The Caltech Pedestrian Dataset, KITTI Pedestrian, and CityPersons remain widely used academic benchmarks; the JAAD and PIE datasets (York University, Toronto) focus on pedestrian intent and crossing-behaviour prediction. Commercial night-vision pedestrian-detection systems from FLIR/Teledyne and NightVision/Continental are deployed on production vehicles primarily in the premium segment.

23.12 Chapter Summary

Pedestrian perception is a distinct engineering discipline within AV perception due to pedestrians' visual and behavioral diversity, the safety consequences of missed detections, and the demographic-equity imperative. Taxonomy (adults, children, cyclists, mobility-aid users) matters because each subtype requires distinct modeling. Detection challenges include small apparent size at range, occlusion, pose and attire variability, and motion blur. Multi-modal fusion (camera + LiDAR + thermal + radar) provides robust detection across conditions. Pose and action recognition provide context for intent prediction, the highest-stakes pedestrian inference. Scenario-specific challenges (crosswalks, jaywalking, school zones, dense crowds) require tailored engineering responses. Demographic equity requires stratified evaluation and targeted improvement. The planner integrates pedestrian data through defined interfaces to produce safe interaction behavior.

23.13 Exercises

1. For the pixel-span example, compute the vertical pixel span of a 1.2-m-tall child at 80 m with the same camera.
2. A LiDAR has 0.2° angular resolution. Compute the number of vertical returns on a 1.6-m-tall pedestrian at 120 m.
3. Derive the crossing probability if action classifier outputs (walk, stand, run, other) = (0.2, 0.6, 0.05, 0.15) with the same class-conditional probabilities.
4. For the calibration example, propose a post-training calibration procedure that does not require retraining.
5. Design an evaluation split by demographic subgroup. What is the minimum number of examples per subgroup for stable AP estimation?
6. For the crosswalk-decision example, compute the required deceleration if the pedestrian is at 15 m instead of 25 m.
7. Compare pedestrian detection in a school zone vs a highway setting. Identify three design differences.
8. Discuss three specific ways camera-only pedestrian detection fails at night, and the thermal-camera contribution to each.
9. Design a Bayesian-update procedure that combines action recognition, gaze direction, and social-context features into a single crossing-intent probability.
10. For the planner-response example, compute the comfort- bounded stopping distance if initial speed is 20 m/s.

Chapter 24

Lane Detection

24.1 Introduction

Lane lines are the primary structural cue a driver—human or autonomous—uses to position a vehicle laterally on a road. Stay within the lane; don't cross the solid line; yield to oncoming traffic. Detection of lanes from onboard sensors enables every aspect of lane-level behavior: lane keeping, lane changes, approaching and passing intersections, detecting lane divergences, and verifying HD-map lane geometry. This chapter develops lane detection as a focused subfield of AV perception: its specific geometric structure, the classical algorithms that solved it first, the learned methods that dominate today, and the temporal and multi-view extensions that make it robust.

Three observations motivate the chapter's structure.

First, lane detection is *geometrically structured* in ways general segmentation is not. Lanes are long thin curves following constrained shapes (cubic polynomials, clothoids, or piecewise curves) rather than arbitrary pixel clusters. Detectors that exploit this structure dramatically outperform those that don't. Classical and modern systems both encode the lane-as-curve prior, with the modern flavor doing so through specialized network architectures.

Second, lane detection must operate *far ahead*. A vehicle driving at 30 m s^{-1} needs to anticipate lane geometry tens of meters ahead to plan smoothly. Detection must not only find lanes close to the vehicle but extrapolate them accurately into the middle distance, where perspective projection compresses information.

Third, lane detection is *condition-sensitive*. Lanes may be crisply painted, heavily worn, temporarily marked for construction, snow-covered, wet-reflective, or entirely absent (on unsealed roads, in parking areas). Production stacks assume lanes may be unreliable and plan accordingly; lane detection is a helpful hint, not an absolute reference.

The chapter is organized as follows: lane taxonomy and the coordinate systems used; classical detection (edge + Hough transforms, polynomial fits with RANSAC); learning-based approaches

(LaneNet, SCNN, LaneATT, CLRNet, BEVFormer-style 3D methods); temporal tracking; integration with HD maps; evaluation metrics. Two architecture diagrams visualize the classical pipeline and the modern multi-task learning architecture; worked examples use the hybrid prose + aligned calculation style throughout.

24.2 Lane Taxonomy and Geometry

Before detection, we must define what we are detecting. Lanes have distinct types, each with different visual signatures and different semantic implications.

24.2.1 Lane Line Types

Solid Single Lines

Solid white separates lanes of the same direction; solid yellow separates opposing directions; solid colored lines at the edge define the roadway boundary. Semantic rule: do not cross without reason (lane-change rules apply).

Dashed Lines

Dashed white or yellow indicates a lane where crossing is permitted (lane changes, overtaking on two-way roads). Geometric signature: alternating paint and gap, typically 3 m paint + 6 m gap in US highway standards.

Double Lines

Double solid or solid + dashed combinations encode more complex rules (only one side may cross, etc.). Detection must identify the two-line structure rather than collapsing to a single line.

Temporary Lines

Construction-zone lines, often in yellow or orange, sometimes inconsistent with underlying permanent markings. The active lane geometry is typically the temporary one; production systems must detect and prefer these when present.

24.2.2 Coordinate Systems

Lane detection outputs can live in several frames.

Image Frame

Pixel coordinates. Simple but distorted by perspective; a lane that is straight in the world appears curved in the image.

Bird's-Eye View (BEV)

A top-down planar projection of the ground. Lanes appear as straight or smoothly curving lines without perspective distortion. Standard target frame for modern learned methods.

Vehicle Frame

3D coordinates attached to the vehicle. Useful for downstream planning; requires calibrated camera-to-ground transformation.

Example 24.1 (Image-to-BEV transformation geometry). Camera is mounted at height $h = 1.4$ m, pitch 0, focal length $f = 800$ pixels, principal point at image center (cx, cy) . Image pixel (u, v) lies on the flat ground plane if $v > cy$ (below horizon).

Ground point longitudinal distance:

$$X = \frac{f h}{v - cy}.$$

Ground point lateral distance:

$$Y = \frac{(u - cx) \cdot h}{v - cy}.$$

Example pixel 30 rows below horizon on the image center vertical:

$$\begin{aligned} X &= \frac{800 \times 1.4}{30} \\ &\approx 37 \text{ m.} \end{aligned}$$

So a pixel 30 rows below the horizon projects to a ground point approximately 37 m ahead of the camera. At the lower edge of the visible road ($v - cy = 200$ rows), $X = 5.6$ m (close-range). The perspective projection is heavily nonlinear; near pixels represent small ground distance, far pixels represent large ground distance. BEV transformation undoes this.

24.3 Classical Lane Detection

Before deep learning, lane detection was built on classical computer-vision primitives. These approaches are still used in production (often as backup or complement to learned methods) and remain instructive.

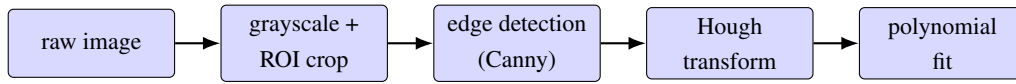


Figure 24.1: Classical lane-detection pipeline. Input image is converted to grayscale and cropped to a region of interest (road area below the horizon). Canny edge detection produces edge pixels. Hough transform finds dominant straight-line segments; these are refined into polynomial curves that model lane geometry. Each stage has parameters tuned to the vehicle’s camera and operating conditions.

24.3.1 Edge Detection + Hough Transform

The original approach. Canny edge detector produces a binary edge map; the Hough transform accumulates votes in (ρ, θ) parameter space for straight lines passing through edge pixels.

Canny Edge Detection

Compute image gradient magnitude and direction; apply non-maximum suppression; threshold with hysteresis. Output: binary edge pixels.

Hough Transform

Each edge pixel votes for all lines passing through it. Peaks in the accumulator space correspond to dominant image lines. Parameters: ρ (distance from origin), θ (angle).

Example 24.2 (Hough transform accumulator peak). An edge pixel at $(u, v) = (600, 400)$. Image coordinate origin at top-left. Lines passing through this pixel satisfy $\rho = u \cos \theta + v \sin \theta$.

For angle $\theta = 60^\circ$:

$$\begin{aligned}
 \rho &= 600 \cdot \cos 60^\circ + 400 \cdot \sin 60^\circ \\
 &= 600 \cdot 0.5 + 400 \cdot 0.866 \\
 &= 300 + 346.4 \\
 &= 646.4 \text{ px.}
 \end{aligned}$$

The accumulator bin at $(\rho, \theta) = (646, 60^\circ)$ receives a vote. With 1000 edge pixels voting across their compatible (ρ, θ) lines, peaks emerge at the (ρ, θ) corresponding to actual image lines. The peak height equals the number of edge pixels on that line—robust to sparse noise, degrades on thick edges or curves.

24.3.2 Polynomial Fit + RANSAC

Lane lines are usually curved rather than straight. A more refined classical pipeline fits a low-degree polynomial (typically degree 2 or 3) to extracted edge points:

$$x(y) = a + by + cy^2 + dy^3. \quad (24.1)$$

RANSAC rejects outliers; least-squares fits the coefficients to inliers.

Example 24.3 (Cubic lane fit). Extracted BEV lane points (longitudinal y , lateral x , meters): $(0, 0.1), (10, 0.15), (20, 0.3), (30, 0.8), (40, 1.9), (50, 3.8)$.

Fit cubic $x(y) = a + by + cy^2 + dy^3$:

$$a \approx 0.10,$$

$$b \approx 0.003,$$

$$c \approx -0.0003,$$

$$d \approx 3.0 \times 10^{-5}.$$

Evaluate at $y = 60\text{m}$:

$$\begin{aligned} x(60) &= 0.10 + 0.003 \cdot 60 - 0.0003 \cdot 3600 \\ &\quad + 3 \times 10^{-5} \cdot 216000 \\ &= 0.10 + 0.18 - 1.08 + 6.48 \\ &= 5.68\text{m}. \end{aligned}$$

The lane at 60 m ahead is predicted to be 5.68 m to the right of the vehicle—a tight right-hand curve. Extrapolation beyond the measurement range amplifies error; confidence drops beyond the farthest observed point.

24.3.3 Limitations of Classical Methods

- Threshold tuning is fragile: light conditions, road wear, and weather all shift edge statistics.
- Heavy clutter (other vehicles' shadows, tar repair marks) produces false detections.
- Dashed lines without temporal tracking appear discontinuous.
- No mechanism for distinguishing lane types.

These limitations motivated learning-based approaches.

24.4 Learning-Based Lane Detection

Modern lane detection is dominated by neural-network approaches that directly regress lane geometry from image input.

24.4.1 Lane Segmentation Approaches

LaneNet

Multi-task: produces pixel-wise lane segmentation and lane-embedding features; post-processing clusters pixels into individual lane instances. Instance-level output but post-processing adds latency.

SCNN (Spatial CNN)

Spatial message passing across rows and columns exploits the thin, long structure of lanes. Captures global context that vanilla CNNs miss. Strong performance on occluded and low-visibility lanes.

24.4.2 Lane Proposal Methods

LaneATT

Anchor-based approach: pre-defined lane anchors at different orientations and positions. Network classifies and refines anchors into lane instances. Efficient and accurate; established state of art for a period.

CLRNet

Multi-scale feature aggregation with learned lane line priors. Competitive on benchmarks; production-practical with real-time inference.

24.4.3 3D and BEV Methods

Persformer, BEVLaneDet, BEVFormer-style

Predict lanes directly in 3D or BEV space by transforming image features to BEV via learned or geometric means. Natural output for planning (no post-processing to convert image-plane output to ground coordinates).

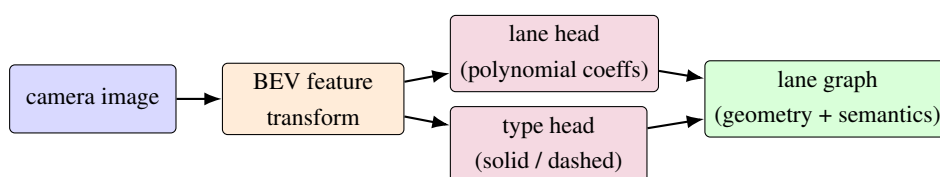


Figure 24.2: Modern multi-task lane-detection architecture. Camera features are transformed to BEV representation via learned or geometric means. Parallel heads regress lane geometry (polynomial coefficients or point sequences) and lane semantics (line type, color, crossing-permission). Outputs combine into a lane graph that downstream planning consumes directly.

24.4.4 Temporal Fusion

Single-frame lane detection is fragile under noise (momentary occlusions, lighting transitions). Temporal fusion integrates over multiple frames:

Recurrent Architectures

LSTMs or transformers across a temporal window produce a single stabilized lane estimate. Vanishing or emerging lanes (lane splits, merges, end of construction zone) are handled through state updates.

Kalman-Filter Smoothing

Lane polynomial coefficients are the filter state; frame-wise detections are measurements. Produces smooth continuous lane estimates and bounds frame-to-frame jitter.

Example 24.4 (Kalman smoothing of lane jitter). Raw single-frame lane offset at current range 30 m:

- Frame 0: 1.82 m.
- Frame 1: 1.88 m.
- Frame 2: 1.79 m.
- Frame 3: 1.92 m.
- Frame 4: 1.85 m.

Raw standard deviation across 5 frames:

$$\begin{aligned}\bar{x} &= \frac{1.82 + 1.88 + 1.79 + 1.92 + 1.85}{5} \\ &= 1.852 \text{ m.}\end{aligned}$$

$$\begin{aligned}\sigma &= \sqrt{\frac{\sum (x_i - \bar{x})^2}{4}} \\ &\approx 0.051 \text{ m.}\end{aligned}$$

Kalman-filtered estimate at frame 4 (measurement noise $R = 0.05^2$, process noise $Q = 0.001$):

$$\hat{x}_4 \approx \bar{x} = 1.85 \text{ m,} \quad \sigma_{\text{filtered}} \approx 0.02 \text{ m.}$$

Filtered uncertainty shrinks by roughly $\sqrt{5}$; planner sees a stable lane estimate even under measurement jitter, avoiding over-reactive lateral control.

24.5 Evaluation Metrics

24.5.1 Classification-Style Metrics

Pixel-Level IoU

For lane-segmentation methods, pixel-wise IoU of predicted vs ground-truth lane pixels. Useful but doesn't capture geometric quality—a displaced lane has low pixel IoU even if shape is correct.

Lane-Instance F1

A predicted lane counts as a correct detection if its average distance to the ground-truth lane is below a threshold. F1 combines precision and recall at lane-instance level.

24.5.2 Geometric Metrics

Mean Distance Error

Sample points along predicted and ground-truth lanes at regular intervals; compute average Euclidean distance. Captures geometric accuracy directly.

Chamfer Distance

Symmetric nearest-neighbor distance between point sets. Insensitive to exact sampling and covers geometric discrepancy in both directions.

Example 24.5 (Mean distance error calculation). Predicted lane samples at $y = 0, 10, 20, 30$ m: $x_{\text{pred}} = (0.10, 0.15, 0.32, 0.85)$. Ground truth: $x_{\text{gt}} = (0.08, 0.14, 0.30, 0.80)$.

Per-sample error:

$$|\delta_0| = |0.10 - 0.08| = 0.02 \text{ m},$$

$$|\delta_1| = |0.15 - 0.14| = 0.01 \text{ m},$$

$$|\delta_2| = |0.32 - 0.30| = 0.02 \text{ m},$$

$$|\delta_3| = |0.85 - 0.80| = 0.05 \text{ m}.$$

Mean distance error:

$$\begin{aligned} \text{MDE} &= \frac{0.02 + 0.01 + 0.02 + 0.05}{4} \\ &= 0.025 \text{ m}. \end{aligned}$$

A 2.5 cm average error at ranges up to 30 m is excellent; typical production targets are MDE < 10 cm at 50 m and < 30 cm at 100 m.

24.5.3 Benchmarks

Canonical lane-detection benchmarks: TuSimple (highway), CULane (diverse urban), CurveLanes (challenging curves), OpenLane (3D lanes). Each has distinct scoring and captures different failure modes.

24.6 Production Considerations

24.6.1 Integration with HD Maps

HD maps (Chapter 14) include prior lane geometry. Each detected lane receives one of three dispositions against the map:

Situation	Disposition	Downstream use
Detection matches a map lane	map gives semantic context (rules, speed limits); detection confirms geometry	high confidence; nominal planning
Detection without map match	possibly new or temporary lane	use detection directly; flag for map update
Map lane without detection	lane invisible (snow, heavy occlusion)	use map geometry with reduced confidence; plan conservatively

24.6.2 Handling Absent Lanes

Unsealed roads, parking lots, and private driveways have no lane markings at all. Production systems fall back through a priority cascade:

Priority	Fallback	Applicability
1	HD-map lane geometry	when map coverage exists
2	Drivable-area segmentation	when visual road boundaries are clear
3	Follow vehicle ahead	when a lead vehicle is present
4	Conservative corridor + slow	when all else fails

At every level the confidence drops and the planner adopts tighter speed envelopes and wider lateral margins.

24.6.3 Weather-Robustness

Snow-covered, rain-reflective, or oil-slicked lanes produce degraded detection. Multi-modal fusion (combining camera with LiDAR intensity return from paint retroreflection) extends usable weather range. LiDAR-based lane detection is increasingly standard in adverse-weather AV deployments.

Example 24.6 (Range at which lane detection remains reliable). Camera-based detection in daylight: reliable to 100 m; degrades beyond to 70m in rain, 40m in heavy fog. LiDAR-based

detection: reliable to 80 m in dry, 60 m in rain, 50 m in fog.

Fused reliable range:

$$R_{\text{fused}} = \max(R_{\text{cam}}, R_{\text{lidar}})$$

$$\text{dry:} \quad = \max(100, 80) = 100 \text{ m,}$$

$$\text{rain:} \quad = \max(70, 60) = 70 \text{ m,}$$

$$\text{fog:} \quad = \max(40, 50) = 50 \text{ m.}$$

Required lookahead at ego speed v (braking-deceleration $a = 4 \text{ m s}^{-2}$):

$$d_{\text{stop}} = \frac{v^2}{2a}.$$

At $v = 25 \text{ m s}^{-1}$: $d_{\text{stop}} = 78 \text{ m}$. Dry and rain OK; fog triggers speed reduction because the fused 50 m lane-detection range is below the required lookahead. Speed envelope adapts automatically to sensor reliability.

24.7 Industrial Deployment

Lane detection is one of the oldest production AV technologies, shipping in lane-departure-warning and lane-keep-assist systems for over twenty years. Mobileye's first commercially-deployed product was vision-based lane detection. Tesla, BMW, Mercedes-Benz, and many other OEMs ship production lane-keeping based on vision-only or vision-radar lane detection. The TuSimple Lane Detection Benchmark, CULane, and BDD100K provide standard evaluation. ISO 17361 governs Lane Departure Warning Systems internationally. The European Union General Safety Regulation mandates Emergency Lane Keeping Systems on new vehicles from 2022. In practice, lane detection in mature production systems uses combinations of learned image segmentation, Hough-transform-style classical processing for highway environments, and HD-map priors where available.

24.8 Chapter Summary

Lane detection is a focused AV perception subfield with distinct geometric and algorithmic structure. Lane taxonomy spans solid, dashed, double, and temporary markings, each with specific semantic rules. Coordinate systems (image, BEV, vehicle) require defined transformations. Classical methods (Canny + Hough + polynomial fit with RANSAC) remain useful baselines. Learning-based methods (LaneNet, SCNN, LaneATT, CLRNet, BEVFormer) now dominate, with multi-task architectures combining geometry and semantics. Temporal fusion (recurrent networks, Kalman filtering) stabilizes frame-wise detections. Evaluation metrics span pixel, instance, and geometric dimensions. Production integration with HD maps, weather-robustness, and absent-lane fallback determine deployment quality. Fusion of camera and LiDAR extends usable range into adverse conditions.

24.9 Exercises

1. For the image-to-BEV example, compute the ground distance for a pixel 100 rows below horizon with $f = 1200$ pixels and $h = 1.2$ m.
2. Derive the Hough-transform (ρ, θ) for an edge pixel at $(300, 500)$ at angle $\theta = 30^\circ$.
3. For the cubic-fit example, evaluate the lane position at $y = 100$ m and comment on extrapolation reliability.
4. Design a RANSAC procedure for lane-point fitting. Specify the inlier distance threshold and iteration count.
5. For the Kalman-smoothing example, compute the filtered variance if the measurement noise doubles.
6. Compare LaneNet and SCNN architecturally. Identify three design differences and their practical implications.
7. For the MDE example, compute the geometric accuracy metric if predicted and ground-truth lanes differ by a constant offset.
8. Design a lane-matching procedure between HD-map lanes and detected lanes. Specify the matching score function.
9. For the weather-range example, compute the maximum safe speed at which the detection range supports braking to stop with 50 % margin.
10. Propose a metric for lane-detection robustness under gradually-worsening conditions (darkness, rain, snow).

Chapter 25

Rural and Dust-Dominated Autonomous Travel

25.1 Introduction

Most of this book has implicitly assumed urban or highway driving in developed road networks: paved surfaces, painted lane markings, maintained signage, reliable cellular coverage, mostly-predictable road users. Rural travel, and especially travel on unsealed roads in dust-dominated environments, violates all of these assumptions. A road may be a dirt track rutted by yesterday's rain. Other traffic may include livestock, carts, and pedestrians sharing the surface without designated lanes. HD maps may be absent; cellular signal may be intermittent; GNSS reception may be degraded by terrain; and clouds of dust kicked up by vehicles can obscure sensors to ranges measured in meters rather than hundreds of meters. This chapter develops the engineering discipline required for autonomous operation in these conditions.

Three observations motivate the chapter's structure.

First, rural and dust-dominated operation is *not a minor corner case*. Globally, unsealed roads account for a majority of road length by many estimates. Rural populations depend on these networks for essential mobility; mining and agricultural operations take place almost entirely in such environments. Any AV program that aspires to serve more than developed urban cores must address these operational domains.

Second, rural and dust-dominated AV requires *sensor re-balancing*. Cameras and LiDAR, the backbone of urban-AV perception, degrade severely in dust. Radar, which penetrates dust effectively, becomes the primary long-range sensor; thermal imaging gains importance for detecting living road users; wheel odometry and IMU gain importance as sparse-infrastructure localization. The production stack is architecturally similar but differently weighted.

Third, rural AV has *production deployment today* that many observers are unaware of. Australian mining fleets (Chapter 7) have operated autonomously in extreme dust for over a decade; African

autonomous-vehicle pilots specifically target rural connectivity; agricultural equipment operates autonomously across fields and farm-to-market roads. The engineering patterns are established; this chapter makes them explicit.

The chapter is organized as follows: the physical characteristics of unsealed roads and rural operating domains; dust as a perception adversary; mixed traffic including non-motorized and animal road users; sparse infrastructure (maps, GPS, connectivity); sensor-suite adaptations; control adaptations for off-road terrain; and surveys of production deployments in mining, rural delivery, and agriculture. Three architecture diagrams visualize dust impact on sensor modalities, rural mixed-traffic scene composition, and the degraded-perception fallback cascade; worked examples use the hybrid prose+calculation style throughout.

25.2 The Rural Operating Domain

Understanding the environment is prerequisite to engineering for it. This section characterizes rural and unsealed-road operation across several axes.

25.2.1 Road Surface Types

Unsealed road surfaces fall into four operational categories. Table 25.1 summarizes their characteristics; prose below develops each in turn.

Surface	Traction	Rolling resist.	Typical operational concern
Gravel	reasonable	2–3× asphalt	washouts; stone flinging
Dirt/earth	weather-dependent	variable	seasonal passability extremes
Corrugated	variable	resonance	vibration damage to mounts and sensors
Sand/loose	poor	very high	wheel dig-in; high ground clearance needed

Table 25.1: Characteristics of unsealed road surfaces and the operational concerns each imposes on autonomous-vehicle operation.

Gravel Roads

Compacted gravel over an engineered subbase. Common in mining, logging, and rural-highway applications. Surface provides reasonable traction, but loose stones can damage tires and fly into sensors at higher speeds.

Dirt and Earthen Roads

Unmaintained or minimally-maintained earth surfaces. Quality varies dramatically with weather: firm and dusty when dry; soft and rutted when wet; impassable when flooded. A single road may require fundamentally different control strategies in different seasons.

Corrugated Washboard Surfaces

A characteristic failure mode of high-traffic unsealed roads: transverse ripples form across the surface from repeated vehicle passes. Vehicles vibrate substantially at specific speeds that excite the corrugation wavelength. Suspension and sensor mounts must tolerate sustained vibration.

Sand and Loose Surfaces

Desert, beach, or construction-zone operation. Traction is poor; wheels can dig in; ground clearance matters. Specific off-road autonomous-vehicle designs use wider tires, lower pressures, and specialized traction-management algorithms.

25.2.2 Terrain Characteristics

Rural driving is rarely flat. Grades may be steeper than highway-engineered maxima. Camber may be irregular. Hills and ridges block GNSS and cellular signals. Water crossings, though rare, may be required.

25.2.3 Climate Extremes

Rural operating domains often face extreme conditions: Saharan heat (50 °C+), Siberian cold (−40 °C), monsoon rains, dust storms, and seasonal flooding. Operational envelopes must accommodate these.

25.3 Dust as a Perception Adversary

Dust is the single most distinctive feature of rural and mining autonomous operation. Its effect on perception is severe and modality-dependent.

25.3.1 Physical Basis

Dust consists of suspended particles of various sizes—from fine clay (sub-micron) to silt and sand (tens to hundreds of microns). Suspended particles scatter and absorb light; the effect depends critically on the particle-to-wavelength ratio.

Visible Light (Cameras)

Visible light has wavelengths 400–700 nm. Dust particles are much larger ($\sim 10\mu\text{m}$ typical), so scattering is efficient. Cameras see foreground dust as a gray-white haze that obscures everything beyond.

Near-Infrared (LiDAR)

Near-infrared at 905 or 1550 nm behaves similarly: dust strongly scatters and absorbs. LiDAR range drops substantially; returns can also come from the dust itself, producing phantom obstacles.

Radar (Centimeter/Millimeter-Wave)

Automotive radar wavelengths are 3.9 mm (77 GHz) and 3.0 mm (94 GHz). Dust particles are orders of magnitude smaller than the wavelength; scattering is negligible. Radar is essentially transparent to dust.

Example 25.1 (Dust-induced range reduction across sensors). A fleet vehicle moving behind another vehicle that has raised a dust cloud.

Dust concentration profile (approx): 1 g m^{-3} over a 30 m-deep cloud.

Camera visibility:

$$\begin{aligned} V_{\text{cam}} &\approx \frac{k_{\text{met}}}{c_d} \\ &\approx \frac{3}{1.0} \\ &\approx 3 \text{ m.} \end{aligned}$$

Here k_{met} is a meteorological constant (~ 3 for visible light scaled to this regime) and c_d the dust concentration in g/m^3 . Visibility is drastically reduced; camera-based obstacle detection effectively fails inside the cloud.

LiDAR detection range:

$$\begin{aligned} R_{\text{lidar}}(c_d = 1) &\approx R_{\text{clear}} \cdot f_d(c_d) \\ &\approx 200 \cdot 0.15 \\ &\approx 30 \text{ m.} \end{aligned}$$

With clear-weather 200 m range reduced by an empirical degradation factor $f_d \approx 0.15$ at 1 g m^{-3} .

Radar detection range:

$$\begin{aligned} R_{\text{radar}}(c_d = 1) &\approx R_{\text{clear}} \cdot f_d^{\text{radar}}(c_d) \\ &\approx 250 \cdot 0.92 \\ &\approx 230 \text{ m.} \end{aligned}$$

Nearly unchanged. Radar is the dominant long-range sensor inside the dust cloud.

The operational response: dust-aware fusion weights radar heavily, treats LiDAR as short-range, discounts camera almost entirely. Production mining haul trucks rely predominantly on radar in such conditions.

25.3.2 Dust Classification

Different dust types have different perception impacts.

- **Suspended light dust:** dry dirt road with light traffic; moderate visibility reduction.
- **Rolling dust clouds:** mining haul truck passing; severe local reduction, moves with airflow.
- **Airborne fine dust:** seasonal dust storms, Saharan haboobs; regional phenomenon; persistent over hours.
- **Mechanical dust:** construction or drilling operations; localized but extreme.

Each requires slightly different engineering response.

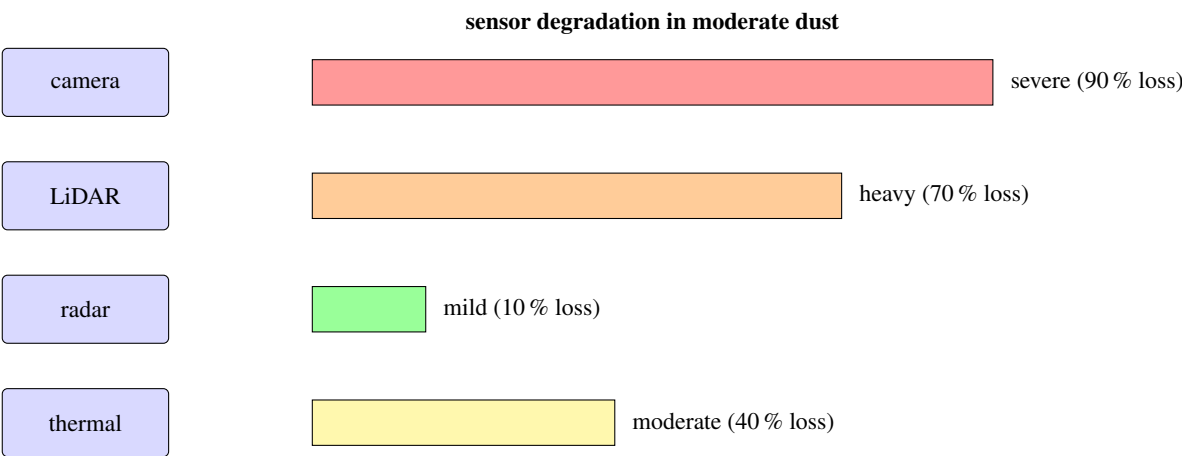


Figure 25.1: Relative sensor degradation in moderate-dust conditions. Camera performance drops almost completely (95 %+ loss beyond a few meters). LiDAR degrades severely (70 % range loss). Radar loses only a small fraction. Thermal imaging is partially resilient, depending on particle heating. The sensor mix for dust-tolerant operation therefore differs substantially from that of urban operation.

25.4 Mixed Traffic and Road Users

Rural roads host traffic far more diverse than urban arterials.

25.4.1 Non-Motorized Traffic

Pedestrians on the Road

Unlike urban settings with designated sidewalks, rural roads are often shared spaces: pedestrians walk along or across the travelway without designated infrastructure. Detection and yield behavior cannot rely on crosswalk or curb cues.

Bicycles and Motorcycles

Common in rural transport; often carry loads that change the silhouette. Off-road motorcycles may travel at unpredictable speeds and angles across the road.

Carts and Animal-Drawn Vehicles

Animal-drawn carts are common in many rural regions; slow-moving, with distinctive silhouettes unfamiliar to urban-trained models. Targeted data collection is required for reliable detection.

25.4.2 Livestock and Wildlife

Cattle and Livestock

Cattle, goats, sheep, and camels regularly cross or occupy rural roads. They may move slowly or dash across unpredictably; they travel in groups; they do not obey traffic signals. A detection system must identify them reliably and plan conservatively around them.

Wildlife

Deer, kangaroos, antelope: fast-moving, unpredictable, often present at dawn and dusk. Thermal imaging is particularly valuable here (body heat against cooler surroundings).

Example 25.2 (Thermal + radar detection of animals at dusk). A kangaroo 80 m ahead of a vehicle in low-light twilight conditions.

Camera performance: Low contrast; kangaroo may appear as a faint silhouette at this range. Detection confidence low.

Thermal camera: Kangaroo body temperature $\sim 37^\circ\text{C}$ against ground at 15°C produces substantial contrast. Strong signature even without visible-light illumination.

Radar: Biological target with 0.01 m^2 RCS. Detection range for automotive radar:

$$R_{\text{radar}} \propto (P_t G \sigma / N)^{1/4}.$$

At 77 GHz with typical automotive parameters, detection range for $\sigma = 0.01\text{ m}^2$ is roughly 100 m to 150 m.

Fusion result: Detection at 80 m with high thermal + radar confidence $\rightarrow$ track established $\rightarrow$ planner initiates gentle deceleration. Without thermal or radar, the vehicle would likely not detect the kangaroo until much closer, at which point emergency braking might be required.

25.4.3 Informal Road Users

Roadside markets, village paths crossing the road, children playing. Classification of scene context is needed: a “village passage” ODD may require speeds of 20 km h^{-1} to 30 km h^{-1} even on a straight road.

25.5 Sparse Infrastructure

Rural operating environments have fundamentally different infrastructure availability.

25.5.1 HD Maps

Commonly Absent

Most rural roads lack HD-map coverage. Production HD mapping has prioritized urban and highway environments; rural coverage is largely an open problem.

Online Mapping

Vehicles build local maps on-the-fly: detect drivable area, identify lane or track, record prominent features. These lightweight maps can be shared via vehicle-to-cloud updates to progressively build coverage.

25.5.2 GNSS Coverage

Attenuation by Terrain

Deep valleys, canyons, and forested areas can block or reflect GNSS signals. Multi-constellation receivers (GPS + GLONASS + Galileo + BeiDou) help but do not fully solve the problem.

Poor Correction Services

RTK corrections (Chapter 7) require nearby base stations or CORS networks. Rural regions are typically not well-served; centimeter-accuracy GNSS may be unavailable.

25.5.3 Cellular Connectivity

Intermittent or Absent Coverage

Many rural areas have no cellular coverage, or have 2G/3G only. Cloud-dependent AV services must operate in offline mode for extended periods. Map updates, remote supervision, and fleet management must accommodate disconnection.

Example 25.3 (Expected connectivity in a 200 km rural route). Route through mixed terrain with 40% cellular coverage (typical rural figure).

Expected average connected distance:

$$\begin{aligned} d_{\text{conn}} &= 0.4 \times 200 \\ &= 80 \text{ km.} \end{aligned}$$

Expected continuous disconnect duration (at $v = 40 \text{ km h}^{-1}$):

$$\begin{aligned} t_{\text{disconn}} &\approx \frac{120 \text{ km}}{40 \text{ km/h}} \\ &= 3 \text{ hours.} \end{aligned}$$

A 3-hour offline window is longer than many cloud-dependent systems are designed for. Production rural AV systems cache maps locally, perform local logging (with later upload), and do not rely on cloud assistance for safety-critical decisions.

25.6 Sensor-Suite Adaptations

Given dust, weather, and sparse infrastructure, the rural AV sensor suite differs from urban.

25.6.1 Radar Prioritization

Production rural AVs use radar as the primary long-range sensor. Multiple radars (front-long, front-short, corner, rear) with emphasis on 4D imaging radar for elevation resolution.

25.6.2 Thermal Imaging Inclusion

Thermal imaging is often standard, not optional. Its role in detecting living road users (pedestrians, livestock, wildlife) at night and in dust is hard to replace with other modalities.

25.6.3 LiDAR as Short-Range Supplement

LiDAR is retained for short-range perception but is not relied upon at long range. Dust-tolerance and hermetic sealing are production requirements.

25.6.4 Robust IMU and Wheel Odometry

With GNSS intermittent and feature-sparse scenes, inertial sensing and wheel odometry bear more of the localization burden. Higher-quality IMUs and careful calibration matter more than in urban operation.

25.6.5 Sensor Cleaning

Automatic sensor cleaning (washers, wipers, air jets) is essential in dusty environments. Sensor fouling is a continuous operational reality, not an occasional event.

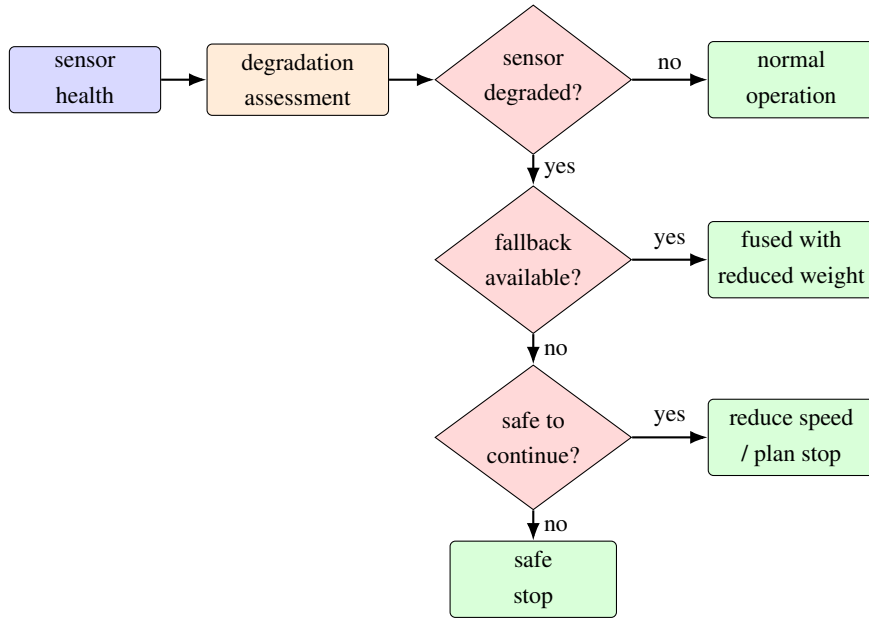


Figure 25.2: Degraded-perception fallback cascade for rural operation. Sensor-health monitoring continuously assesses each sensor’s reliability. Under degradation, the system checks for fallback modalities; if available, fused operation continues with reduced weights. If not, the system checks whether the current trajectory can complete safely; if yes, reduce speed and plan; if not, execute a safe stop.

25.7 Control Adaptations

Off-road terrain requires control algorithms tuned for variable traction and uneven surfaces.

25.7.1 Traction Estimation

Online estimation of tire-road friction coefficient. Based on wheel-slip measurements and known steering inputs, the estimator infers the available friction and updates speed and steering envelopes.

Example 25.4 (Safe cornering speed on gravel). Nominal cornering radius $R = 100$ m, friction on gravel estimated at $\mu = 0.4$ (vs 0.8 on dry asphalt).

Maximum cornering speed:

$$\begin{aligned}
 v_{\max} &= \sqrt{\mu g R} \\
 &= \sqrt{0.4 \cdot 9.81 \cdot 100} \\
 &= \sqrt{392.4} \\
 &\approx 19.8 \text{ m/s } (\approx 71 \text{ km h}^{-1}).
 \end{aligned}$$

Comparing to asphalt limit:

$$\begin{aligned}
 v_{\text{asphalt}} &= \sqrt{0.8 \cdot 9.81 \cdot 100} \\
 &\approx 28.0 \text{ m/s } (\approx 101 \text{ km h}^{-1}).
 \end{aligned}$$

On gravel, the safe speed drops by roughly 30 %. Controller adapts its envelope based on online friction estimates; rural AV speed is dynamic with surface conditions.

25.7.2 Suspension-Aware Planning

Planning trajectories that avoid severe ruts or washouts. LiDAR terrain scans feed a cost map where terrain roughness adds to path cost; smoother paths are preferred even at slight range cost.

25.7.3 Low-Speed Robustness

Rural operation often involves low speeds (10 km h^{-1} to 40 km h^{-1}) over extended distances. Controllers must maintain quality at low speeds where urban optimizations (designed for 30–50 m/s highway speed) may underperform.

25.8 Production Deployments

25.8.1 Mining

Chapter 7 developed mining AV in detail. The Pilbara iron-ore autonomous haul-truck operations are the world's largest and longest-running dust-dominated AV deployment. Decades of operational data inform the engineering patterns described here.

25.8.2 Agricultural Autonomy

John Deere, CNH Industrial, and others offer autonomous tractors, combines, and sprayers. Operation is across fields and farm-to-market roads; dust, weather, and mixed-infrastructure are routinely handled.

25.8.3 Rural Delivery

Pilot programs for last-mile delivery in rural areas (including some African markets) combine autonomous vehicles with local operators who handle final handoff. The AV handles the dust-heavy rural stretch; local operators handle the socially-complex village interior.

25.8.4 Defense and Specialized Autonomy

Military autonomous ground vehicles operate in extreme terrain and conditions by design. Many technical advances in rural-AV perception originated in defense R&D; dual-use technology transfer is ongoing.

Example 25.5 (Rural delivery ROI). Rural delivery route, 300 km, connecting three villages to a regional hub. Current: manual delivery, 1 driver, 12 hours/day, 5 days/week.

Manual cost per year:

$$\begin{aligned} C_{\text{manual}} &= 365 \cdot 5/7 \times 12 \text{ h} \\ &\times 20 \text{ USD/h} \\ &\approx 62,600 \text{ USD/year.} \end{aligned}$$

AV cost per year: Vehicle amortized over 5 years at 120 kUSD fitted:

$$\begin{aligned} C_{\text{AV, capital}} &= 120,000/5 \\ &= 24,000 \text{ USD/year.} \\ C_{\text{AV, operating}} &\approx 18,000 \text{ USD/year.} \\ C_{\text{AV, total}} &\approx 42,000 \text{ USD/year.} \end{aligned}$$

Annual savings:

$$\begin{aligned} \Delta C &= 62,600 - 42,000 \\ &\approx 20,600 \text{ USD/year.} \end{aligned}$$

Payback on the AV capital uplift over 3 years. More important than cost: the AV can operate 24/7, increasing delivery frequency and serving emergency medical deliveries that human-operated services cannot. Rural deployment economics are often about capability expansion rather than cost reduction.

25.9 Industrial Deployment

Off-road and unstructured-terrain autonomy has moved into production in specific verticals. Caterpillar's autonomous mining trucks operate in high-dust environments at scale, as discussed in Chapter 7. John Deere acquired Bear Flag Robotics in 2021 and has shipped autonomous tractors (8R) into commercial agricultural service in 2022 and onwards, capable of operating in dusty fields under varying visibility conditions. Monarch Tractor (United States) ships electric autonomous tractors into commercial use. The DARPA Grand Challenges of 2004 and 2005 originally established that autonomous off-road driving in unstructured environments was technically feasible. African deployment contexts, including those addressed throughout this book, remain less publicly documented than U.S. or European or Chinese contexts but are the subject of growing academic and industry interest.

25.10 Chapter Summary

Rural and dust-dominated autonomous travel is a distinct operational domain from urban driving, with distinct engineering responses. Unsealed road surfaces (gravel, dirt, corrugated, sand) require control adaptations. Dust is the primary perception adversary, with modality-dependent effects: cameras and

LiDAR degrade severely, radar is near-transparent, thermal imaging maintains value. Mixed traffic includes livestock, wildlife, pedestrians, carts, and informal road users, requiring detection beyond urban-focused datasets. Sparse infrastructure (absent HD maps, intermittent GNSS, limited cellular) means vehicles must operate autonomously with local resources. Sensor-suite rebalancing emphasizes radar, thermal, robust IMU, and automatic cleaning. Control adaptations include traction estimation and terrain-aware planning. Production deployments exist in mining, agriculture, rural delivery, and defense; the engineering patterns are mature. Rural AV is not a corner case but a primary frontier for global AV deployment.

25.11 Exercises

1. For the dust-range example, compute LiDAR range at dust concentration 0.3 g m^{-3} using the same degradation coefficient scaling.
2. Derive the maximum safe cornering speed for a 50 m-radius curve at friction $\mu = 0.3$ (wet gravel).
3. For the animal-detection example, compute radar detection range if the radar-cross-section estimate is 0.005 m^2 (smaller animal).
4. Design a dust-classifier that distinguishes the four dust types. Specify features and a decision rule.
5. For the connectivity example, compute the expected offline duration if coverage drops to 20 %.
6. Compare urban and rural pedestrian detection. Identify three specific engineering differences.
7. Propose a sensor-cleaning strategy that trades off cleaning frequency against operational downtime.
8. For the rural delivery ROI example, compute the break-even commodity price reduction that justifies the AV deployment if savings must fund an additional 10 kUSD/year of overhead.
9. Design a degraded-perception fallback for a rural AV that loses camera but retains radar and LiDAR. Specify the envelope.
10. Discuss the ethical considerations of rural AV deployment in regions with pedestrian-dominant road use. How should design priorities shift?

Part VII

Advanced Topics and Frontier Directions

Chapter 26

Digital Twins and Scene Intelligence

26.1 Introduction

The perception stack developed through earlier chapters produces a structured *scene representation*: objects with boxes, lanes, traffic lights, drivable area. That representation is adequate for driving but represents only a fraction of the information present in the sensor data. Modern research is producing richer representations: dense 3D reconstructions of the scene, learned implicit models that predict occupancy at every point in space, unified panoptic segmentations that label every pixel and every instance simultaneously, and generative models that can complete missing sensor observations. Collectively these comprise *scene intelligence*: the vehicle’s understanding of the world as a full, reasoned-about 3D environment rather than a list of detected objects. And when that scene intelligence is accumulated and replicated, it becomes a *digital twin*: a virtual replica of a real physical environment in which AV systems can be tested, trained, and monitored.

Three observations motivate the chapter’s structure.

First, scene intelligence is a *generalization* of the object-centric perception developed through earlier chapters. A detector that identifies cars, pedestrians, and cyclists throws away the rest of the scene; a scene-intelligence representation keeps it all. This richer representation supports reasoning the object-centric approach cannot: what is that amorphous debris, where is the curb-edge relative to the driveable area, is the parked bicycle likely to be ridden away.

Second, digital twins are *transforming AV development*. A digital twin of a specific road network, including dynamic traffic, enables simulation at realistic scale and fidelity. New models can be trained in the twin; candidate releases can be evaluated; rare scenarios can be programmed and tested. The gap between simulation and reality narrows with each iteration.

Third, generative AI is entering *perception*. Neural Radiance Fields (NeRF) reconstruct photorealistic 3D scenes from sparse views; diffusion models complete partially-occluded sensor data; foundation models unify multiple modalities. These techniques are moving from research into production, fundamentally changing what “perception” means.

The chapter is organized as follows: 3D scene reconstruction and NeRF; occupancy networks that predict free/occupied space densely; panoptic segmentation as unified scene understanding; diffusion models for perception completion; digital twin ecosystems for training and validation; and the integration of scene intelligence into production AV stacks. Three architecture diagrams visualize the scene-intelligence pipeline, NeRF training, and digital-twin ecosystem architecture; worked examples apply the hybrid prose + calculation style throughout.

26.2 3D Scene Reconstruction

Reconstructing the full 3D geometry of the environment from sensor observations is the foundational scene-intelligence task. The output is a dense representation—mesh, voxel grid, implicit surface, or neural representation—that captures the scene geometry at resolution much higher than object detection.

26.2.1 Classical Multi-View Stereo

Given multiple calibrated views of a scene, triangulate dense depth per pixel. Classical methods (PatchMatch, COLMAP) work well on textured scenes with many views; struggle on the low-texture and sparse-view conditions AVs often encounter.

26.2.2 LiDAR-Based Reconstruction

LiDAR directly measures 3D points; reconstruction consists of accumulating points over time into a unified cloud, then surfacing and segmenting. Standard for indoor robotics; automotive LiDAR at range produces sparse clouds that need completion.

26.2.3 Learned Depth from Single or Stereo Views

Deep networks predict dense depth from one or two images. Monodepth, DPT, ZoeDepth, and successors produce usable depth maps for most automotive scenes; fine structure (thin objects, transparent surfaces) remains hard.

Example 26.1 (Dense depth point count from monocular estimate). Camera: 1920×1080 , every pixel assigned a depth.

Total point count per frame:

$$\begin{aligned} N_{\text{pixels}} &= 1920 \times 1080 \\ &= 2,073,600. \end{aligned}$$

After filtering (unreliable regions, sky, out-of-range):

$$\begin{aligned} N_{\text{usable}} &\approx 0.6 \times N_{\text{pixels}} \\ &\approx 1,250,000. \end{aligned}$$

At 30 Hz frame rate:

$$\begin{aligned} \text{point-rate} &= 30 \times 1,250,000 \\ &= 37.5 \text{ million points/sec.} \end{aligned}$$

Three orders of magnitude more points than automotive LiDAR produces—but with lower depth precision and no intensity information. Combining the two modalities (dense camera depth + precise LiDAR) produces the most robust reconstruction.

26.2.4 Neural Radiance Fields (NeRF)

NeRF represents a scene as a neural network $f_\theta : (\mathbf{x}, \mathbf{d}) \rightarrow (c, \sigma)$ that maps 3D position $\mathbf{x}$ and viewing direction $\mathbf{d}$ to color c and volume density σ . Training optimizes the network to reproduce observed images via volume rendering. Output is a differentiable, queryable 3D scene that can be rendered from any viewpoint.

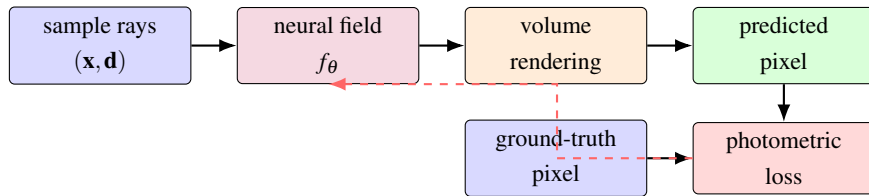


Figure 26.1: NeRF training loop. Rays sampled from posed training images are passed through the neural field f_θ to produce density and color at each sample point. Volume rendering integrates these into a predicted pixel color; photometric loss against the ground-truth pixel drives gradient updates (dashed red) to the network weights. Training proceeds until reconstructions match observations.

Volume Rendering Equation

For a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ with samples at depths $t_1, t_2, \dots, t_N$:

$$C(\mathbf{r}) = \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \delta_i)) c_i, \quad (26.1)$$

with $T_i = \exp(-\sum_{j<i} \sigma_j \delta_j)$ the accumulated transmittance and $\delta_i = t_{i+1} - t_i$.

Automotive NeRF

Block-NeRF, MARS, UniScene, and related methods adapt NeRF to large outdoor scenes with camera-bearing vehicles driving through them. Production use cases: ultra-realistic sensor simulation

for training; scene reconstruction from logged fleet data; generation of novel scenarios by editing reconstructed scenes.

Example 26.2 (NeRF training cost for a city block). Street recording: camera trajectory over 200 m of a city block, 30-Hz frames, 5-minute capture.

Frame count:

$$\begin{aligned} N_{\text{frames}} &= 5 \times 60 \times 30 \\ &= 9,000 \text{ frames.} \end{aligned}$$

Training compute (typical large-NeRF scale):

$$T_{\text{train}} \approx 100 \text{ GPU-hours on A100-class.}$$

Output: a queryable neural scene. Rendering one novel view:

$$\begin{aligned} T_{\text{render}} &\approx 1\text{--}3 \text{ seconds per frame (standard NeRF),} \\ T_{\text{render}} &\approx 30\text{--}100 \text{ ms per frame (accelerated, e.g. Instant-NGP).} \end{aligned}$$

Accelerated variants have made NeRF practical for generating large simulation datasets from real driving logs—one of the most valuable production NeRF applications.

26.3 Occupancy Networks

Instead of representing the scene as objects (bounding boxes) or as photorealistic appearance (NeRF), occupancy networks represent it as a dense grid of voxels labeled occupied or free. This representation is particularly natural for motion planning.

26.3.1 Formulation

An occupancy grid divides 3D space into voxels; each voxel has a probability of being occupied. Given sensor observations, infer the occupancy probabilities via a neural network trained on labeled or self-supervised data.

26.3.2 BEV Occupancy

A common 2D simplification: occupancy in the driving plane (BEV). Each cell represents a 20 cm-resolution square of ground; assigned occupied (obstacle), free, or unknown. Direct input to planning.

26.3.3 3D Voxel Occupancy

Full 3D representation: $200 \times 200 \times 16$ voxels spanning 100 m forward, 50 m lateral, 5 m vertical, at 0.5 m voxel size. Per-voxel occupancy, optionally with semantic labels (car, pedestrian, vegetation, road surface, building).

Example 26.3 (Voxel grid memory footprint). Grid parameters:

- Dimensions: $200 \times 200 \times 16$.
- Per-voxel: 32-bit float occupancy + 8-bit class label.

Total voxels:

$$\begin{aligned} N_v &= 200 \times 200 \times 16 \\ &= 640,000. \end{aligned}$$

Memory:

$$\begin{aligned} M &= N_v \times (4 + 1) \text{ bytes} \\ &= 3,200,000 \text{ bytes} \\ &\approx 3.2 \text{ MB per frame.} \end{aligned}$$

Modest per-frame; multi-timestep history (10 frames) requires 32 MB. Manageable on automotive compute. Compression via sparse representations (octree, only-occupied storage) reduces further by $\sim 10\times$.

26.3.4 Occupancy Prediction

Extension: predict occupancy not just at current time but at future timesteps. Combines scene understanding with motion prediction. Tesla’s “occupancy networks” (as publicly discussed) and Waymo’s related methods both target this.

26.3.5 Evaluation Metrics

- Per-voxel IoU with ground truth.
- Class-weighted IoU (mIoU).
- Temporal consistency (for predicted occupancy).
- Free-space preservation (no false-occupancy in drivable regions).

26.4 Panoptic Segmentation

Semantic segmentation (Chapter 12) labels each pixel with a class; instance segmentation identifies individual object instances. *Panoptic* segmentation unifies both: every pixel receives a class, and for “thing” classes (cars, pedestrians, cyclists), an instance ID.

26.4.1 Stuff and Things

- “Stuff” classes: amorphous regions (road, sky, vegetation, building). No instance ID, just class.
- “Thing” classes: countable objects (car, pedestrian, bicycle). Class + instance ID.

Panoptic output assigns each pixel (*class*, *instance_id*); for stuff pixels, *instance_id* = 0.

26.4.2 Architectures

Panoptic FPN

Combine semantic- and instance-segmentation branches on a shared backbone (Feature Pyramid Network). Merge via simple overlap-based fusion.

Panoptic DeepLab

Bottom-up approach: predict center heatmaps and offsets per pixel, then group pixels into instances based on offsets. Single-shot approach, fast.

MaskFormer / Mask2Former

Transformer-based unified architecture. Every pixel assigned to a query (class + instance). State-of-art accuracy; higher compute than earlier approaches.

26.4.3 Panoptic Quality Metric

PQ (Panoptic Quality) measures panoptic performance:

$$PQ = \frac{\sum_{(p,g) \in TP} \text{IoU}(p,g)}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}. \quad (26.2)$$

Combines segmentation quality (numerator) with detection quality (denominator).

Example 26.4 (PQ computation). Panoptic prediction on a test image:

- True positives: 12 matched segments with IoUs summing to 9.6 (mean IoU 0.80).
- False positives: 2 segments with no ground-truth match.
- False negatives: 3 missed ground-truth segments.

Segmentation quality:

$$SQ = \frac{9.6}{12} = 0.80.$$

Recognition quality:

$$\begin{aligned} RQ &= \frac{12}{12 + 0.5 \cdot 2 + 0.5 \cdot 3} \\ &= \frac{12}{14.5} = 0.828. \end{aligned}$$

Panoptic quality:

$$\begin{aligned} PQ &= SQ \times RQ \\ &= 0.80 \times 0.828 = 0.662. \end{aligned}$$

A PQ of 0.66 is strong but not great; leading models on Cityscapes panoptic reach 0.70–0.72 PQ. The decomposition into SQ and RQ tells us the model’s segmentation is good (0.80) but it misses too many instances (RQ 0.83).

26.5 Diffusion Models for Perception

Diffusion models—which trained on large image datasets have become the dominant generative image model—are moving into perception. Two distinct uses emerge.

26.5.1 Sensor Completion

A diffusion model trained on (occluded, complete) pairs can complete partial sensor data: fill in LiDAR dropouts, denoise camera images, remove rain artifacts. The prior learned from driving data enables plausible completions where simpler approaches fail.

26.5.2 Data Augmentation and Scenario Generation

Diffusion models generate novel driving scenes conditioned on descriptions (“night, rain, pedestrian crossing at 30 m”). Training data expands beyond what real fleets capture; rare conditions become artificially abundant.

26.5.3 Perception as Denoising

Some recent work reformulates detection as a denoising process: start from noisy bounding-box proposals, iteratively refine toward correct boxes via a diffusion process. DiffusionDet is the canonical example.

26.6 Digital Twin Ecosystems

A digital twin is a virtual replica of a physical system or environment. For AVs, a digital twin combines reconstructed geometry (NeRF or alternatives), traffic models, weather models, and infrastructure metadata into a single simulated environment.

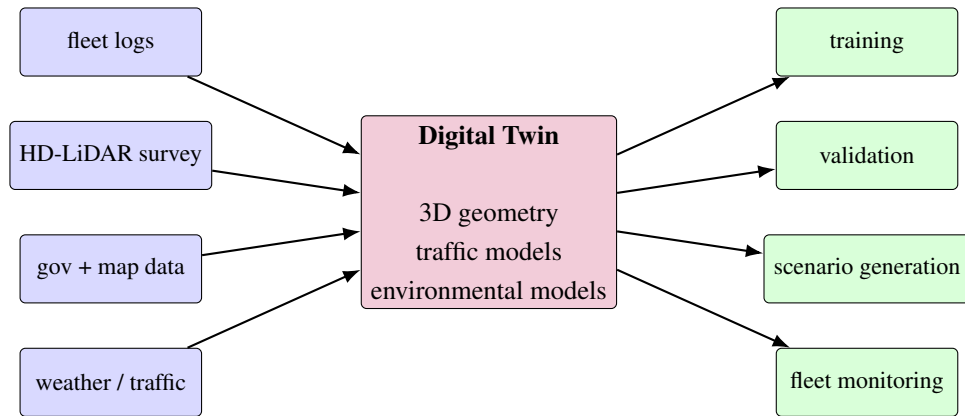


Figure 26.2: Digital twin ecosystem. Data sources—real fleet logs, high-definition LiDAR surveys, government map and infrastructure data, live weather and traffic feeds—combine in a persistent digital twin. The twin serves multiple purposes: training AV models (synthetic data), validating releases (simulated regression suites), generating adversarial scenarios, and continuously monitoring a real fleet against simulated expectations.

26.6.1 City-Scale Twins

NVIDIA Omniverse, Siemens Xcelerator, and proprietary platforms at major AV programs enable city-scale twins that replicate major urban environments. Microsoft’s work on Ford, Barcelona’s mobility twin, and Waymo’s internal simulation infrastructure are examples.

26.6.2 Value Proposition

- Risk-free testing of dangerous scenarios.
- Reproducible regression testing.
- Rapid iteration on scenario variations.
- Shared testing platform for multiple stakeholders (automakers, cities, regulators).

26.6.3 Twin Fidelity and Validation

A twin is useful only to the extent it reproduces real-world behavior. Validation methodology (Chapter 17) compares twin-simulated outcomes to real-world outcomes on controlled scenarios; twins that fail to match are not yet usable for safety-critical validation.

Example 26.5 (Twin-to-reality fidelity metric). Twin proposal under review. 200 logged real-world scenarios replayed in the twin.

Trajectory matching:

$$\begin{aligned}\text{matched trajectories} &= 187/200 \\ &= 0.935.\end{aligned}$$

Distributional metrics (e.g. mean lateral offset):

$$\begin{aligned}|\mu_{\text{sim}} - \mu_{\text{real}}| &= 0.08 \text{ m}, \\ |\sigma_{\text{sim}} - \sigma_{\text{real}}| &= 0.03 \text{ m}.\end{aligned}$$

Outcome agreement:

$$\text{intervention-count agreement} = 0.91.$$

Fidelity strong enough for exploratory testing (rapid iteration on new features), not yet for final release-gate validation. The acceptance gate for the latter is typically ≥ 0.98 agreement on safety-critical outcomes.

26.7 Integration into the Production Stack

Scene intelligence and digital twins connect to the production AV stack through several interfaces.

26.7.1 Runtime Scene Intelligence

On-vehicle scene-intelligence modules (dense occupancy prediction, panoptic segmentation) provide planning and control with a richer view than object-only perception. This is an active area of production integration at leading AV programs.

26.7.2 Offline Scene Intelligence

NeRF reconstructions from fleet logs generate synthetic training data, populate validation suites, and support post-hoc analysis of on-road events.

26.7.3 Digital Twin Feedback

Online fleet performance can be continuously compared against twin predictions. Systematic divergences highlight model error; the twin becomes a monitoring tool as well as a simulation tool.

26.8 Industrial Deployment

Digital-twin technologies have moved from research curiosity to production infrastructure across multiple AV programs. NVIDIA's Omniverse and DRIVE Sim products are positioned for AV

simulation. Waymo and Cruise have publicly described internal simulation infrastructures running orders of magnitude more miles than physical fleets. Applied Intuition (United States), founded in 2017 and reportedly valued in the multibillion dollar range, provides a commercial AV simulation platform used by multiple OEMs and AV programs. The CARLA simulator (Intel Labs and Computer Vision Center Barcelona) is the most widely used academic open-source AV simulator. Neural Radiance Fields (NeRF), introduced by researchers at UC Berkeley and Google in 2020, have begun to influence commercial AV-simulation pipelines as a way to generate realistic novel-view sensor data from recorded fleet logs.

26.9 Chapter Summary

Scene intelligence generalizes object-centric perception to dense representations: 3D reconstruction, occupancy grids, panoptic segmentation, and generative models. Neural Radiance Fields (NeRF) produce photorealistic queryable 3D scenes from posed images; automotive-adapted variants support fleet-derived reconstruction at block and city scale. Occupancy networks predict free/occupied space at voxel resolution; natural input to motion planning. Panoptic segmentation unifies semantic and instance segmentation with the PQ metric. Diffusion models complete occluded sensor data and generate novel scenarios. Digital twin ecosystems combine these representations into virtual replicas used for training, validation, scenario generation, and fleet monitoring. Integration into production stacks is underway across the industry; scene intelligence is moving from research to deployment.

26.10 Exercises

1. For the dense-depth example, compute the effective points-per-second rate at 4K camera resolution and 60 Hz.
2. Derive the NeRF volume-rendering equation from first principles. State the density-to-transmittance relationship.
3. For the voxel-grid example, compute memory per frame for a finer resolution $400 \times 400 \times 32$ grid.
4. For the PQ example, compute PQ if true positives increase to 15 while false positives and negatives hold constant.
5. Compare NeRF and classical multi-view stereo for an automotive scene. Identify two specific advantages of each.
6. Design a diffusion-based data augmentation pipeline. How does it interact with existing training infrastructure?
7. For the twin-fidelity example, discuss what additional metrics should be tracked beyond trajectory and distribution matching.

8. Propose a regression test suite in a city-scale digital twin. How many scenarios, what types, and how are they maintained?
9. Identify three ways occupancy prediction can be integrated into a planner. What interface does each require?
10. Describe the privacy considerations of storing NeRF reconstructions of public roads. Propose mitigations.

Chapter 27

Advanced Prediction and Multi-Agent Behavior

27.1 Introduction

Chapter 17 developed the foundations of prediction: trajectory forecasting, multimodal outputs, and the evaluation metrics that quantify them. This chapter extends the treatment in three directions that have driven the research frontier in recent years: explicit modeling of multi-agent interactions via graph representations; behavior modeling that predicts not just trajectories but the underlying intent and decision process; and risk-aware perception that integrates prediction confidence into the planning loop.

Three observations motivate the structure.

First, prediction is inherently multi-agent. A car’s trajectory is not independent of its neighbors’; a pedestrian’s decision to cross depends on the approach of traffic. Models that predict each agent’s trajectory in isolation miss this coupling. Modern predictors use graph structures that explicitly represent inter-agent dependencies.

Second, intent is upstream of trajectory. A vehicle signaling a lane change has telegraphed its intent before the trajectory executes; a pedestrian with eyes on their phone and heading toward the road has an intent that will produce a trajectory. Predicting intent first and then conditioning on it improves trajectory prediction and supports more natural interaction.

Third, prediction uncertainty must feed back into perception and planning. Risk-aware perception weights detection and tracking confidence by the consequential risk of the detected object; high-risk tracks (pedestrian in predicted path) get more sensor attention and tighter planning margins than low-risk ones.

The chapter is organized as follows: advanced trajectory prediction methods (transformer-based, graph-based, diffusion-based); behavior modeling with explicit intent estimation; multi-agent interaction

modeling using graph neural networks; risk-aware perception and prediction; and integration with planning. Three architecture diagrams visualize interaction graphs, the intent-then-trajectory cascade, and risk-aware perception flow; worked examples apply the hybrid style.

27.2 Advanced Trajectory Prediction

27.2.1 Transformer-Based Predictors

Predictors based on transformer architectures have supplanted the RNN- and CNN-based approaches common in earlier prediction work. Scene context (all agents, lane geometry, traffic signals) is encoded as tokens; self-attention captures inter-token relationships.

Scene Tokenization

Each agent is represented as a token with features: position, velocity, heading, class, recent history. Lane segments are tokens with geometry and semantic information. Traffic lights are tokens with phase. Attention captures which elements matter for which predictions.

Decoder Designs

- Anchor-based: a predefined set of candidate trajectories; output probabilities over anchors plus refinement offsets.
- Goal-conditioned: predict multi-modal goal endpoints, then complete trajectories to each.
- Auto-regressive: generate trajectory tokens one step at a time with attention to the scene.

27.2.2 Diffusion-Based Prediction

Recent methods reformulate trajectory prediction as a denoising diffusion process. Start from noisy trajectories and iteratively refine. Advantages: natural handling of multi-modal distributions; competitive accuracy. Example systems: MotionDiffuser, TrajDiff.

Example 27.1 (Trajectory prediction accuracy). Model output: 6 trajectories with probabilities. Ground-truth trajectory is observed.

Predicted trajectories:

- Mode 1 ($p = 0.50$): straight forward.
- Mode 2 ($p = 0.25$): right turn.
- Mode 3 ($p = 0.15$): slow down.
- Mode 4–6 ($p = 0.10$ total): various.

Actual agent turns right.

Negative log-likelihood:

$$\begin{aligned}\text{NLL} &= -\log p(\text{right turn}) \\ &= -\log 0.25 \\ &\approx 1.39.\end{aligned}$$

Compare to uniform model:

$$\begin{aligned}\text{NLL}_{\text{uniform}} &= -\log(1/6) \\ &\approx 1.79.\end{aligned}$$

The model beats uniform but reserved only 25 % probability for the correct mode—suggesting improvement room. Multimodal NLL across the validation set is the standard aggregate.

27.3 Behavior Modeling and Intent

Predicting the underlying behavior rather than just trajectory enables reasoning about what an agent wants, not just what it does.

27.3.1 Intent Representations

Common intent taxonomies:

Maneuver Intent

Discrete actions: lane-keep, lane-change-left, lane-change-right, turn-left, turn-right, stop, yield, merge. Small vocabulary; classifiable from short observation windows.

Goal Intent

Destination or sub-goal: exiting the highway, turning at the next intersection, merging into traffic. Longer horizon; requires more context (map, traffic-flow patterns).

Interaction Intent

What is the agent's response toward other agents: yielding, overtaking, following, cutting in. Emerges from multi-agent reasoning rather than per-agent features.

27.3.2 Intent Prediction Architectures

Classification Head

Simple: predict discrete intent class from current observations. Fast; feeds trajectory predictor as conditioning.

Sequential Models

Predict intent trajectory over time (intent can change). Transformers with intent as a token sequence.

Hierarchical

High-level intent first, then trajectory conditioned on intent. Decomposes the problem; improves sample efficiency.

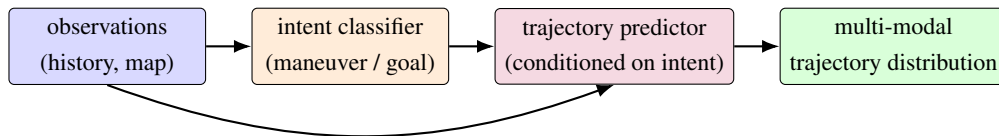


Figure 27.1: Intent-then-trajectory hierarchical prediction. Observations feed an intent classifier producing a distribution over maneuver or goal classes. Trajectory predictor, conditioned on both observations and the intent distribution, produces multi-modal trajectories with the intent conditioning providing strong structural priors. Skip connection preserves fine-grained features the intent bottleneck would discard.

27.4 Multi-Agent Interaction via Graph Neural Networks

Interactions between agents are well-modeled as a graph: agents are nodes, potential interactions are edges. Graph neural networks (GNNs) operate natively on this structure.

27.4.1 Interaction Graph Construction

Nodes

Every agent in the scene (including the ego) is a node. Features: position, velocity, heading, class, recent history.

Edges

Connect agents that plausibly interact. Common rules:

- Spatial proximity (distance below threshold).
- Line-of-sight (unblocked path).
- Trajectory overlap (predicted paths cross).
- Road-topology connectivity (shared lane or merging).

Edge Features

Relative position, relative velocity, closing rate, interaction type. These give the GNN the relational context each agent has with each other.

Example 27.2 (Interaction graph for intersection scene). Scene: 4-way intersection. 6 agents:

- A1 (ego, northbound).
- A2 (car, northbound, 30 m ahead).
- A3 (car, westbound turning left).
- A4 (pedestrian, approaching crosswalk).
- A5 (cyclist, eastbound).
- A6 (car, southbound at stop).

Edge identification (within 50 m and plausibly interacting):

$$E = \{(A1, A2), (A1, A3), (A1, A4), (A1, A5), (A3, A5), (A3, A4), (A4, A5)\}.$$

Total edges:

$$|E| = 7.$$

Compare: a fully-connected graph has $\binom{6}{2} = 15$ edges. Pruning to plausibly-interacting edges reduces GNN computation by 50 %. The ego (A1) has 4 interaction edges; most computationally critical are A1-A4 (pedestrian) and A1-A3 (left-turning car) because these involve the highest-risk interactions.

27.4.2 Graph Attention and Message Passing

GNN layers pass messages between connected nodes, updating each node's representation based on its neighbors. Graph attention (GAT) weights edges dynamically: an ego-pedestrian edge receives more attention weight than an ego-distant-car edge.

27.4.3 Scene-Level Reasoning

After several GNN layers, each node has context from the wider graph. Output heads per node predict trajectory, intent, or other per-agent quantities. Crucially, the predictions are coupled: the system produces consistent joint predictions rather than independent per-agent ones.

27.4.4 Notable Architectures

- **VectorNet**: graph on lane + agent vectors; strong baseline.
- **SceneTransformer**: joint agent-scene transformer with query-based decoding.
- **M2I**: predicts pairwise interactions explicitly before joint trajectories.

- **Wayformer:** Waymo’s production-grade architecture.

27.5 Risk-Aware Perception

Most perception systems treat every detected object with equal attention: allocate equal compute and tracker resources per object. Risk-aware perception uses predicted risk to allocate resources: objects likely to enter the ego’s path receive more tracker precision, more frequent updates, and tighter planning envelopes.

27.5.1 Risk Quantification

Risk R for a tracked object:

$$R = P_{\text{hit}} \cdot C_{\text{severity}}, \quad (27.1)$$

with P_{hit} the probability of collision given current trajectories and C_{severity} the severity (injury level) if collision occurs. Pedestrians have high C_{severity} ; small debris has low.

Example 27.3 (Risk computation for two tracked objects). Two objects tracked:

- Object A: pedestrian, $P_{\text{hit}} = 0.05$, $C_{\text{severity}} = 1.0$ (fatality weighting).
- Object B: stray cardboard, $P_{\text{hit}} = 0.30$, $C_{\text{severity}} = 0.05$.

Risk values:

$$\begin{aligned} R_A &= 0.05 \times 1.0 = 0.050, \\ R_B &= 0.30 \times 0.05 = 0.015. \end{aligned}$$

Object A has $3.3\times$ higher risk than object B despite much lower collision probability. Tracker allocates $3\times$ more compute budget to A; planner plans wider yielding margin around A. Per-object attention flows to where risk is highest.

27.5.2 Attention Cascade

Risk-aware perception implements attention at multiple levels:

- Region proposal: processes regions likely to contain high-risk objects at higher resolution.
- Tracker allocation: more particles, tighter update rate for high-risk tracks.
- Planning: shortest time-to-collision (TTC) plans conservative for high-risk tracks.

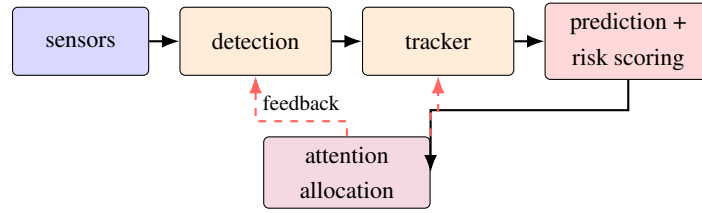


Figure 27.2: Risk-aware perception cascade. Standard detection, tracking, and prediction produce a risk score for each object. An attention-allocation module feeds back to tracker and detection resource budgets (dashed feedback). High-risk objects get more tracker precision and higher-resolution detection processing. Overall compute budget is preserved; its allocation changes based on scene risk.

27.6 Social Interaction Modeling

Beyond pairwise graph interactions, social modeling captures norms and conventions that shape group behavior.

27.6.1 Social Forces Model

Pedestrian behavior is modeled as a combination of goal-seeking, obstacle-avoidance, and personal-space preservation forces. Mathematically tractable; useful as a prior in ML-based models.

27.6.2 Implicit Social Behavior in Data

Large-scale training data captures social conventions implicitly: yielding patterns at unsignalized intersections, merging etiquette, typical following distances. Models trained on such data learn the behaviors without explicit encoding.

27.6.3 Cultural and Regional Variation

Social conventions differ across regions. Jaywalking norms, yielding behaviors, horn-honking conventions all vary. Production systems fine-tune per deployment region or maintain region-specific behavior heads.

Example 27.4 (Regional following-distance calibration). Fleet data from two regions:

- City A: median following distance at 40 km h^{-1} is 18 m.
- City B: median following distance is 9 m.

In seconds of headway:

$$t_A = 18 / (40 / 3.6) = 1.62 \text{ s},$$

$$t_B = 9 / (40 / 3.6) = 0.81 \text{ s}.$$

Ego behavior calibration: Planner uses region-specific following distance so ego neither stands

out as aggressive (in A) nor as cautious-to-point-of- disrupting-flow (in B). Out-of-calibration ego behavior confuses other drivers and degrades collective traffic flow.

27.7 Integration with Planning

The output of advanced prediction must be consumable by the planner. Several integration patterns exist.

27.7.1 Occupancy-Time Projections

Predicted trajectory distributions are projected into an occupancy-time cube: probability of each spatial cell being occupied at each future time. Planner samples from this cube to evaluate candidate trajectories.

27.7.2 Interaction-Aware Planning

Planner explicitly considers the effect of its own action on other agents' behavior. E.g., "if I signal to change lanes, other cars in the target lane will likely yield." Requires the predictor to condition on ego's hypothetical actions.

27.7.3 Risk-Bounded Planning

Planner maintains a risk budget; trajectories exceeding the budget are rejected. The risk budget is a function of operational context and user preferences (comfort vs efficiency).

27.8 Industrial Deployment

Multi-agent prediction has been a focus of academic-industry collaboration. The Waymo Open Motion Dataset, Argoverse 2 (originally Argo AI), and Lyft Level 5 Prediction Dataset (released before Lyft's exit from L5) underpin much of the public benchmark progress. Production prediction systems at Waymo (publicly described in conference papers including those on MultiPath++ and Wayformer) and at NVIDIA Research have published architectures used in their respective systems. The CARLA Leaderboard, originally hosted by the CARLA team, has periodically run prediction-focused challenges. nuPlan from Motional (the joint venture of Aptiv and Hyundai) provides a planning-oriented dataset and benchmark also useful for prediction evaluation.

27.9 Chapter Summary

Advanced prediction extends the foundations from Chapter 17. Transformer-based predictors with scene tokenization are the current state of the art; diffusion- based prediction is emerging rapidly. Behavior modeling predicts intent upstream of trajectory and supports hierarchical prediction. Multi-agent interaction via graph neural networks (VectorNet, SceneTransformer, Wayformer) produces coupled joint predictions. Risk-aware perception allocates attention based on predicted risk, concentrating compute where it matters most. Social interaction modeling captures norms and regional

variations. Integration with planning uses occupancy-time projections, interaction-aware planning, and risk budgets. The prediction layer has become tightly coupled with planning and perception; clean modular boundaries have given way to integrated stacks.

27.10 Exercises

1. For the NLL example, compute NLL if the correct mode had probability 0.50 instead of 0.25.
2. Design an intent taxonomy for cyclist behavior. What classes are essential?
3. For the interaction graph example, identify which edges would exist under a line-of-sight-only rule.
4. Explain why graph attention improves over fixed edge weights in multi-agent prediction.
5. For the risk computation example, compute the risk for an object with $P_{\text{hit}} = 0.01$ and $C_{\text{severity}} = 0.8$.
6. Propose a risk budget appropriate for highway driving and another for urban driving. Justify the difference.
7. For the regional-following example, compute ego behavior if deployed in a region with unknown statistics. What's the fallback?
8. Compare scene-level joint prediction with per-agent independent prediction. Identify three benefits of joint.
9. Design a diffusion-based trajectory predictor. Specify the forward and reverse diffusion steps.
10. Identify three failure modes of GNN-based prediction and how production systems mitigate each.

Chapter 28

Uncertainty, OOD Detection, Explainability, and Adversarial Robustness

28.1 Introduction

A production AV model that is 95 % accurate on its training distribution is not necessarily safe to deploy. Three deeper questions must be answered: *when* is the model confident and when is it not; does the current scene belong to the training distribution at all; and if the model behaves unexpectedly, can we explain why. These questions have given rise to four deeply connected research threads—uncertainty estimation, out-of- distribution (OOD) detection, explainable AI (XAI), and adversarial robustness—that together determine whether a model is *trustworthy*, not merely accurate. This chapter develops all four in sequence, emphasizing their connections and their production integration.

Three observations motivate the structure.

First, accuracy is not trust. A model’s training accuracy measures agreement with labels on a fixed dataset; trust measures whether we can rely on the model in deployment. Trust requires the model to know when it doesn’t know (calibrated uncertainty), to recognize situations it wasn’t trained for (OOD detection), to justify its decisions (explainability), and to be robust to deliberate attack (adversarial robustness).

Second, these four threads are *complementary, not redundant*. A model can be uncertainty-calibrated but still miss OOD scenes; it can be interpretable but still vulnerable to adversarial inputs; it can be adversarially robust but still miscalibrated. Production safety requires all four.

Third, regulatory pressure is *accelerating* adoption. The EU AI Act, ISO 21448 (SOTIF), ISO 26262, and other standards explicitly require uncertainty, interpretability, and robustness properties for safety-critical ML. Techniques that were research-only a few years ago are now production requirements.

The chapter is organized as follows: uncertainty estimation (aleatoric, epistemic, calibration, meth-

ods); out-of-distribution detection (metrics, techniques, production integration); explainable AI for perception (attribution, attention, counterfactual); adversarial attacks (vision-perturbation, patch, sensor-spoofing) and defenses; sensor degradation modeling; redundant pipelines for safety-critical deployment. Three architecture diagrams visualize the uncertainty cascade, OOD detection pipeline, and redundant-perception architecture; worked examples use the hybrid prose + calculation style throughout.

28.2 Uncertainty Estimation

Uncertainty quantifies what the model doesn't know about its own predictions. Two kinds:

28.2.1 Aleatoric vs Epistemic Uncertainty

Aleatoric Uncertainty

Inherent data noise. A distant object has blurry boundaries; there's genuinely more uncertainty about its exact extent. Cannot be reduced by more data (the ambiguity is in the data itself).

Epistemic Uncertainty

Model uncertainty due to limited training data. A rare object type or a novel scene has high epistemic uncertainty. Reducible by collecting more data of the kind in question.

Example 28.1 (Aleatoric vs epistemic decomposition). Detector produces a bounding box prediction with variance σ^2 . Decomposed:

- Aleatoric: $\sigma_{\text{alea}}^2 = 0.04 \text{ m}^2$ (consistent with sensor noise).
- Epistemic: $\sigma_{\text{epi}}^2 = 0.25 \text{ m}^2$ (high; training data rare for this object type).

Total predictive variance:

$$\begin{aligned}\sigma_{\text{total}}^2 &= \sigma_{\text{alea}}^2 + \sigma_{\text{epi}}^2 \\ &= 0.04 + 0.25 = 0.29 \text{ m}^2, \\ \sigma_{\text{total}} &\approx 0.54 \text{ m}.\end{aligned}$$

Most of the uncertainty is epistemic: the model hasn't seen enough of this object type. Remediation: collect more data. If aleatoric dominated, no amount of data would help—the scene itself is ambiguous.

28.2.2 Uncertainty Estimation Methods

Monte Carlo Dropout

Train a model with dropout; at inference, keep dropout enabled and sample N forward passes. Variance across samples approximates epistemic uncertainty.

Deep Ensembles

Train K independent models with different initializations. Aggregate predictions; variance across ensemble members is a strong uncertainty estimate. Production gold standard; cost is $K \times$ inference.

Heteroscedastic Regression

Output both mean and variance directly. The network learns to predict its own uncertainty. Efficient (single model) but less reliable for epistemic uncertainty.

Evidential Networks

Model parameters of a posterior distribution directly (Normal- Inverse-Gamma, Dirichlet). Produces aleatoric and epistemic uncertainty in a single forward pass. Recent; emerging in production.

28.2.3 Calibration

A model is *calibrated* if its confidence matches its accuracy: among predictions with 0.8 confidence, 80% should be correct. Uncalibrated models produce uncertainty estimates that are not actionable.

Expected Calibration Error (ECE)

Bin predictions by confidence; within each bin, compare mean confidence to accuracy. ECE is the weighted mean gap.

Example 28.2 (ECE computation). Model output binned into 5 confidence buckets (0.0–0.2, 0.2–0.4, etc.). Evaluated on 10 000 examples:

- Bucket 0.0–0.2: 1 500 examples, mean conf 0.12, accuracy 0.14.
- Bucket 0.2–0.4: 1 800 examples, mean conf 0.30, accuracy 0.28.
- Bucket 0.4–0.6: 2 100 examples, mean conf 0.52, accuracy 0.48.
- Bucket 0.6–0.8: 2 500 examples, mean conf 0.72, accuracy 0.70.
- Bucket 0.8–1.0: 2 100 examples, mean conf 0.89, accuracy 0.78.

Per-bucket absolute gap:

$$g_1 = |0.12 - 0.14| = 0.02,$$

$$g_2 = |0.30 - 0.28| = 0.02,$$

$$g_3 = |0.52 - 0.48| = 0.04,$$

$$g_4 = |0.72 - 0.70| = 0.02,$$

$$g_5 = |0.89 - 0.78| = 0.11.$$

Weighted ECE:

$$\begin{aligned}
 \text{ECE} &= \sum_k \frac{n_k}{N} g_k \\
 &= 0.15 \cdot 0.02 + 0.18 \cdot 0.02 + 0.21 \cdot 0.04 \\
 &\quad + 0.25 \cdot 0.02 + 0.21 \cdot 0.11 \\
 &\approx 0.043.
 \end{aligned}$$

The model is moderately well-calibrated overall but overconfident in the highest bucket (11-point gap at confidence 0.89). Post-hoc calibration (temperature scaling, isotonic regression) could reduce ECE to ~ 0.02 .

Figure 28.1 visualizes the three canonical calibration regimes: a perfectly calibrated model lies on the diagonal; an over-confident model falls below it (its claimed confidence exceeds actual accuracy); an under-confident model rises above it.

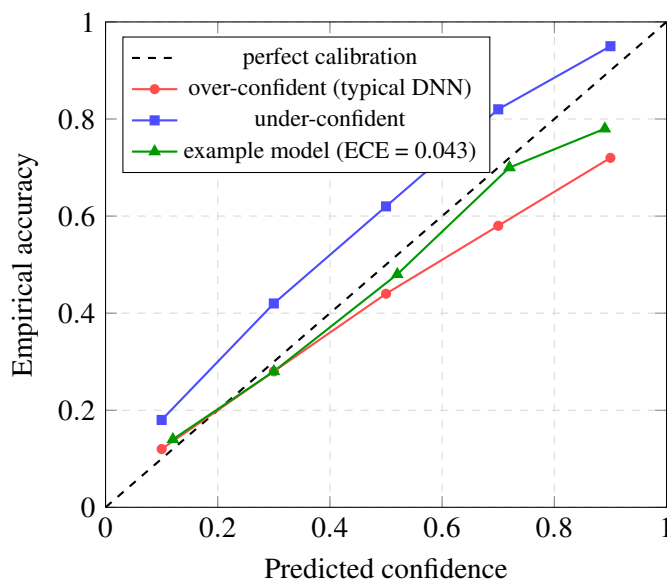


Figure 28.1: Reliability diagram showing the three calibration regimes. The dashed diagonal is the locus of perfect calibration (predicted confidence equals empirical accuracy). Typical un-calibrated deep networks (red) lie below, signalling overconfidence—especially in the high-confidence region. Under-confident models (blue) lie above. The example model from the ECE calculation (green) is mostly on the diagonal but diverges in the highest bucket. Temperature scaling pulls the curve back toward the diagonal.

Temperature Scaling

Divide model logits by a temperature T before softmax. Single parameter optimized on a validation set to minimize NLL. Remarkably effective for neural-network calibration.

28.3 Out-of-Distribution Detection

OOD detection asks whether the current input resembles anything the model was trained on. A model that confidently classifies a unicycle as a car has missed that the input is OOD; a calibrated OOD detector would flag it.

28.3.1 Why OOD Matters for AVs

The tail of real-world scenes is unbounded. No training dataset covers every rare object, every weather condition, every novel intersection geometry. When the AV encounters something novel, we want it to recognize the novelty—preferably before making a confident wrong decision.

28.3.2 OOD Detection Methods

Softmax Probability Baseline

Max softmax probability is a simple OOD score: low max probability suggests uncertainty, potentially OOD. Surprisingly strong baseline but fails on confident mispredictions.

Energy-Based Scores

$E(x) = -\log \sum_k \exp(f_k(x))$ where f_k are logits. Lower energy = higher confidence; ID examples cluster at lower energy.

Feature-Space Distance

Measure distance in learned feature space between the test input and training distribution. Mahalanobis distance with class-conditional Gaussians is a standard technique.

Ensemble Disagreement

Ensemble members disagree more on OOD inputs than ID. Variance across ensemble is a direct OOD score.

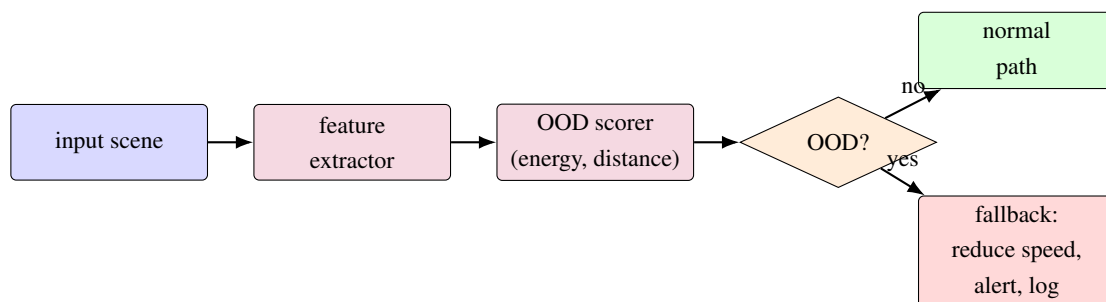


Figure 28.2: OOD detection pipeline. The scene’s feature representation is scored for out-of-distributionness (via energy, feature-space distance, or ensemble disagreement). If the score exceeds a threshold, the AV enters a fallback mode: conservative behavior, operator alert, and extensive logging for later analysis. Normal operation continues for in-distribution scenes.

Example 28.3 (OOD threshold calibration). Energy-based OOD detector. Evaluated on ID validation set and known OOD set:

- ID mean energy: -2.3 , std 0.4 .
- OOD mean energy: 0.8 , std 0.9 .

Threshold selection for target 5 % false-positive rate:

$$\begin{aligned}\tau &= \text{ID mean} + 1.645 \times \text{ID std} \\ &= -2.3 + 1.645 \times 0.4 \\ &\approx -1.64.\end{aligned}$$

Expected detection rate at this threshold (OOD exceeding τ):

Using normal approximation for OOD energy:

$$\begin{aligned}z &= (\tau - \text{OOD mean}) / \text{OOD std} \\ &= (-1.64 - 0.8) / 0.9 \\ &\approx -2.71.\end{aligned}$$

$$\begin{aligned}P(\text{OOD energy} > \tau) &= 1 - \Phi(-2.71) \\ &\approx 0.997.\end{aligned}$$

99.7 % OOD detection rate at 5 % FPR. Strong separation between distributions; the detector is production-viable.

28.4 Explainable AI (XAI) for AV Perception

XAI provides human-understandable justifications for model outputs. For AV perception, XAI supports debugging, regulatory compliance, incident analysis, and user trust.

28.4.1 Attribution Methods

Saliency Maps

Gradient of output with respect to input. Highlights pixels that most influence the model's decision.

Integrated Gradients

Path integral of gradient from a baseline to the input. Axiomatic properties (completeness, sensitivity) give stronger attribution than raw gradient.

LIME and SHAP

Local model-agnostic explanation methods. Perturb the input systematically and observe output changes; fit a local linear model as a surrogate.

28.4.2 Attention Visualization

For transformer-based models, attention weights directly show which scene elements the model attended to. Not a full explanation but a useful starting point.

28.4.3 Counterfactual Explanations

“The detector classified this as a car because; if the roof were shorter, it would classify as a truck.” Counterfactuals generate minimal perturbations that change the classification, revealing decision boundaries.

Example 28.4 (Counterfactual for a detection failure). Pedestrian at 120 m missed by the detector. Counterfactual exploration:

Test 1: Move pedestrian closer (80 m): Detector confidence rises to 0.95. Distance is the dominant factor.

Test 2: Increase pedestrian contrast (lighter clothing): At original distance, confidence rises to 0.45 (threshold 0.5 still misses). Some contribution from appearance.

Test 3: Add upper body motion: Confidence rises by 0.1; motion cue contributes marginally.

The detector’s weakness is primarily distance-driven; augment training with more distant pedestrians. Counterfactual analysis provides actionable debugging guidance that pure accuracy metrics do not.

28.5 Adversarial Attacks on Vision

Adversarial examples are inputs crafted to fool a model. In AVs, this ranges from research attacks (tiny pixel perturbations) to practical concerns (physical patches, manipulated signs).

28.5.1 Digital Adversarial Examples

Small pixel-level perturbations produce wildly different model outputs. FGSM, PGD, and CW are standard attack methods. Important for understanding model robustness; less important as a real-world threat (attacker needs pixel-level access to the image feed).

28.5.2 Physical Adversarial Attacks

More concerning for AVs.

Physical Patches

Printed patterns applied to objects. Demonstrated ability to cause misclassification of stop signs, cars, or road features.

Manipulated Traffic Signs

Stickers, graffiti, or custom prints on real signs. Research has shown that small additions can cause stop signs to be read as speed-limit signs.

Environmental Attacks

Fake lane markings painted on the road, projected images, deepfake traffic indicators. Some are theoretical; some demonstrated in controlled settings.

Example 28.5 (Adversarial perturbation magnitude). Original image pixel values in $[0, 255]$. FGSM attack with budget $\varepsilon = 8$:

Pixel-level perturbation:

$$\Delta x \leq 8 / 255$$

$$\approx 3.1 \% \text{ of dynamic range.}$$

Imperceptible to human vision but sufficient to flip classification:

$$\text{prob}(\text{stop sign}) : 0.99 \rightarrow 0.02,$$

$$\text{prob}(\text{yield}) : 0.01 \rightarrow 0.95.$$

A 3 % perturbation imperceptible to a human driver produces a safety-critical misclassification. Adversarial training (training on perturbed examples) reduces susceptibility but cannot eliminate it.

28.5.3 Adversarial Training

Include adversarial examples in training. Model becomes robust to small perturbations at some cost to clean accuracy. Standard defense; partial rather than complete.

28.5.4 Certified Defenses

Mathematical guarantees of robustness within a specified perturbation ball. Limited in scale; active research area.

28.6 Sensor Spoofing and Jamming

Beyond adversarial pixels, sensors themselves can be attacked.

28.6.1 GPS/GNSS Spoofing

Fake GNSS signals mislead receivers into reporting false positions. Commercial spoofing equipment has been used maliciously; defense requires cross-sensor consistency (GNSS + IMU + wheel + visual odometry) and signal-authentication techniques.

28.6.2 LiDAR Attacks

Laser injection can produce phantom returns or blind receivers. Research demonstrations exist; countermeasures include encoded pulse patterns and cross-sensor validation.

28.6.3 Camera Attacks

Bright IR sources overwhelm CMOS sensors; structured light can cause rolling-shutter artifacts. Mechanical covers, notch filters, and anomaly detection mitigate.

28.6.4 Radar Attacks

Jamming (noise injection) and spoofing (fake reflection signals). Countermeasures: frequency diversity, coded waveforms, anomaly detection based on physically-plausible return patterns.

28.7 Sensor Degradation Modeling

Not all sensor failures are adversarial. Routine degradation (dust on sensors, rain droplets, partial failures) must be modeled and compensated.

28.7.1 Degradation Modes

- Gradual: accumulation of dust, aging components, lens haze.
- Intermittent: water droplets, vibration-induced misalignment.
- Failure: sudden loss, hardware fault.

28.7.2 Detection and Estimation

Sensor health is monitored continuously: signal statistics, cross-sensor consistency, temporal coherence. When degradation is detected, fusion weights adjust; severe degradation triggers fallback behavior.

28.7.3 Modeling Degradation in Training

Simulate sensor degradation during training: synthetic raindrops, dust noise, partial occlusion. Models trained on such augmented data degrade gracefully rather than failing catastrophically.

Example 28.6 (Training-time degradation augmentation). Training dataset: 1M clean frames. Augmentation:

- 20 % with synthetic raindrop overlay.
- 15 % with Gaussian blur (lens haze).
- 10 % with additive noise (sensor aging).
- 5 % with simulated partial occlusion.

Effective training set:

$$N_{\text{eff}} = 1,000,000 \times (1 + 0.2 + 0.15 + 0.10 + 0.05) \\ = 1,500,000 \text{ virtual examples.}$$

Performance outcomes:

Accuracy on clean data: minor reduction (1–2 %),
Accuracy under degradation: substantial improvement (5–15 %).

Trade-off is worthwhile for production: the model that handles degradation well at small clean-accuracy cost is safer overall than the model that is perfect on clean and fragile on degraded.

28.8 Redundant Perception Pipelines

Safety-critical deployment uses multiple independent perception pipelines that cross-check each other.

28.8.1 Diverse Redundancy

Two pipelines built from different architectures, different training data, and different teams. Correlated failures are suppressed; a failure of one does not imply failure of the other.

28.8.2 Voting and Arbitration

- Agreement: both pipelines agree; high confidence in result.
- Disagreement: take the safer interpretation (e.g., recognize an object if either pipeline detects it).
- Safety monitor: third independent module that validates both.

28.8.3 Cost-Benefit of Redundancy

Compute cost roughly doubles; reliability improves dramatically.

Example 28.7 (Reliability of dual-pipeline system). Independent pipelines, each with 99.9 % reliability at correctly classifying a scene.

Joint reliability (at least one correct):

$$\begin{aligned} P_{\text{either correct}} &= 1 - (1 - 0.999)^2 \\ &= 1 - 0.000001 \\ &= 0.999999. \end{aligned}$$

Six-nines reliability from two three-nines pipelines. The assumption of independence is critical—it usually isn’t fully satisfied, and the true joint reliability is lower, but still substantially better than either alone.

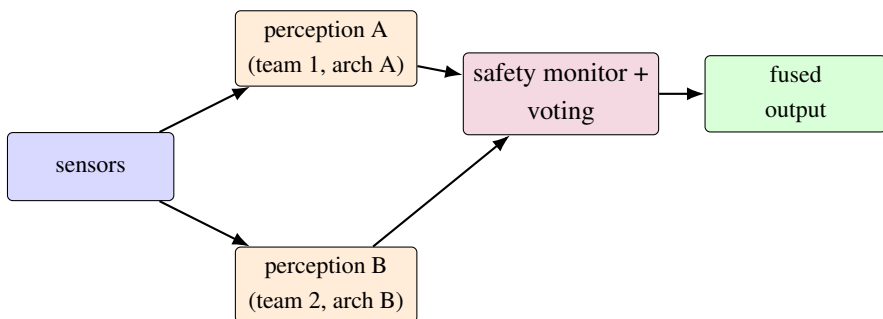


Figure 28.3: Redundant perception architecture. Sensors feed two independent perception pipelines built with different architectures by different teams. A safety-monitor + voting module compares outputs: on agreement, high-confidence output; on disagreement, safer interpretation. Independence of failure modes is the core assumption.

28.9 Industrial Deployment

Uncertainty quantification and explainability have become central to AV safety cases. The NIST AI Risk Management Framework recommends explicit treatment of uncertainty. Bayesian-deep-learning approaches popularised by researchers including Yarin Gal at Oxford and Zoubin Ghahramani at Cambridge have entered production via uncertainty-aware perception heads. Conformal prediction, originally developed by Vladimir Vovk, has moved from theoretical statistics to practical deployment in some AV research and pilot systems. Out-of-distribution detection methods including Mahalanobis distance, energy-based scores, and ODIN are incorporated into production safety monitors at multiple AV programs. The Distributional Robustness research at MIT, Stanford, and elsewhere has influenced industrial robustness practice.

28.10 Chapter Summary

Trust in AV perception comes from four interlocking properties: calibrated uncertainty (the model knows what it doesn’t know), OOD detection (recognition of novel scenes), explainability (justifiable decisions), and adversarial robustness (resistance to attack and degradation). Uncertainty splits into

aleatoric (irreducible) and epistemic (reducible by data); ensembles, dropout, heteroscedastic and evidential networks all estimate it. Calibration matters as much as accuracy; ECE quantifies it. OOD detection uses softmax, energy, feature-distance, or ensemble disagreement scores. XAI for perception spans attribution, attention, and counterfactual methods. Adversarial attacks range from digital pixel perturbations to physical patches and sensor spoofing; defenses include adversarial training, certified methods, and cross-sensor redundancy. Sensor degradation modeling and redundant perception pipelines complete the production safety architecture. Each thread is necessary; none is sufficient alone.

28.11 Exercises

1. For the aleatoric-epistemic decomposition example, estimate the expected variance if training data for the object type is doubled.
2. For the ECE example, propose a temperature scaling factor that would reduce the highest-bucket gap.
3. Compare Monte Carlo dropout and deep ensembles on cost and reliability. When is each appropriate?
4. Design an OOD detection cascade for a driving environment. What threshold is appropriate?
5. For the OOD threshold example, compute detection rate at 1 % FPR (recall that $z_{1\%} \approx 2.33$).
6. Identify three specific ways a saliency map can be misleading as an explanation. Propose remediations.
7. For the adversarial-perturbation example, what defense could provide certified robustness up to $\epsilon = 8$?
8. Design a sensor-spoofing defense for GNSS in an AV. Specify the consistency checks.
9. Compute the dual-pipeline reliability if each pipeline is 99 % reliable but their failures are 50 % correlated.
10. Propose a monitoring pipeline that continuously tracks ECE and OOD rates across a deployed fleet. What alerts?

Chapter 29

Advanced Scene Representations

29.1 Introduction

Standard AV perception produces structured lists: objects with classes, tracks with histories, segments with semantic labels. These representations are useful but impoverished; they discard most of the scene’s structure. Richer representations—layered scene decompositions that separate static from dynamic elements, probabilistic occupancy fields that replace discrete grids with continuous distributions, and semantic flow fields that combine motion with semantic labels—capture more of what is actually observable and support downstream reasoning that the list-of-objects abstraction does not.

Three observations motivate the structure.

First, layered scene representations simplify many downstream tasks. Prediction benefits from knowing which elements are static (road, buildings) and which are dynamic (cars, pedestrians); treating them uniformly discards useful structure.

Second, continuous fields are often more appropriate than discrete grids. Occupancy at a specific point in space is inherently probabilistic; forcing it into a 20 cm grid cell discards real information.

Third, motion + semantics together carry more than either alone. Semantic flow fields let downstream reasoning distinguish “that pedestrian is moving east” from “that bush is moving east in the wind” directly at the representation level.

29.2 Layered Scene Decomposition

Decompose the scene into layers with different time-constants and different processing:

Layer	Content	Update rate
Static infrastructure	Roads, buildings, fixed signage	rare (map updates)
Slow-dynamic	Parked cars, construction zones	minutes
Fast-dynamic	Moving vehicles, pedestrians, cyclists	10–30 Hz
Weather overlay	Rain, fog, sunlight direction	seconds
Operational overlay	Fleet traffic, planned closures	minutes

29.2.1 Benefits

- Computation is allocated proportionally—fast layers get most compute, slow layers get little.
- Prediction models can condition on the relevant layer rather than processing everything jointly.
- Caching is effective: static layers rarely change.

Example 29.1 (Compute saving from layered decomposition). Unified processing: all layers refreshed every frame at full compute cost. Compute per frame = 100 GFLOPs.

Layered processing.

- Static (infrequent): 40 GFLOPs every 10 min.
- Slow-dynamic (per minute): 30 GFLOPs every 60 s.
- Fast-dynamic (per frame at 20 Hz): 25 GFLOPs.

Per-second compute.

$$\begin{aligned}
 C_{\text{layered}} &= 40 \text{ GFLOPs} / 600 + 30 / 60 + 25 \cdot 20 \\
 &\approx 0.067 + 0.5 + 500 \\
 &\approx 501 \text{ GFLOPs/s.}
 \end{aligned}$$

Compared to unified processing.

$$C_{\text{unified}} = 100 \times 20 = 2000 \text{ GFLOPs/s.}$$

4× compute savings via layer separation. This is one reason layered decomposition is a production-grade architectural pattern.

29.3 Probabilistic Occupancy Fields

Discrete occupancy grids quantize space into cells each marked occupied, free, or unknown. *Probabilistic occupancy fields* replace this with a continuous probability function: for each 3D point, the probability that point is occupied.

29.3.1 Formulation

An occupancy field is a function $o : \mathbb{R}^3 \rightarrow [0, 1]$ representing the probability that the point is occupied. It can be represented parametrically (neural network) or as a dense grid with interpolation.

29.3.2 Advantages over Discrete Grids

- No quantization artifacts.
- Queries at arbitrary resolution.
- Natural fusion of multiple observations via Bayesian update.
- Support for temporal prediction (occupancy at future time).

29.3.3 Implementations

Neural-network implementations (occupancy networks, extending Chapter 26 work) use a compact parameteric representation. Dense-grid implementations store log-odds $\ell_c = \log(p_c/(1 - p_c))$ per cell and use additive Bayesian updates.

Example 29.2 (Bayesian occupancy update). A cell has prior log-odds $\ell_0 = 0$ (probability 0.5). A LiDAR return observes it as occupied with likelihood ratio $\lambda_{\text{occ}} = 10$.

Posterior after one observation.

$$\begin{aligned}\ell_1 &= \ell_0 + \log \lambda_{\text{occ}} \\ &= 0 + \log 10 \\ &\approx 2.30, \\ p_1 &= \frac{e^{2.30}}{1 + e^{2.30}} \approx 0.91.\end{aligned}$$

Second observation, same result.

$$\begin{aligned}\ell_2 &\approx 4.60, \\ p_2 &\approx 0.99.\end{aligned}$$

Third observation, free. With likelihood ratio 10 for free:

$$\begin{aligned}\ell_3 &= 4.60 - \log 10 = 2.30, \\ p_3 &\approx 0.91.\end{aligned}$$

Bayesian updates in log-odds are additive, simple, and numerically stable. Contrast with hard-threshold grids where conflicting observations are difficult to combine.

29.4 Semantic Flow Fields

Combine optical-flow motion information with semantic labels: for each image position, both a motion vector and a semantic class.

29.4.1 Representation

$F : \text{image} \rightarrow (\mathbf{v}, c)$ where $\mathbf{v}$ is a 2D motion vector and c a class label. At sub-pixel resolution per pixel.

29.4.2 Why Combined?

Pure flow is semantically agnostic: a moving pedestrian and a tree swaying in wind produce similar flow. Pure semantics are motion-blind. Combined, the representation answers “what is moving and how?”—the question a planner actually needs.

Example 29.3 (Flow + semantics disambiguation). Two image regions both show flow magnitude ~ 5 pixels/frame:

- Region A: class = pedestrian, flow = (3, 0) px.
- Region B: class = tree, flow = (3, 0) px.

Planner interpretation. Region A: imminent lateral movement of pedestrian; prediction- planning should respond. Region B: wind-moving foliage; non-actionable.

Pure optical flow treats both identically; pure semantics sees both as static (tree class, pedestrian class). Only the combined representation correctly disambiguates.

29.5 Implicit Scene Representations

Recent research replaces explicit grids or point clouds with implicit representations: neural networks that respond to coordinate queries with scene properties.

29.5.1 Coordinate-Based Networks

Given (x, y, z, t) , a network produces occupancy, semantic class, appearance. The network itself is the scene representation.

29.5.2 Relation to NeRF and Occupancy Networks

NeRF (Chapter 26) is one implicit representation. Semantic-NeRF adds class labels; Dynamic-NeRF adds time. The family is rapidly expanding and increasingly entering production perception pipelines.

29.6 Industrial Deployment

Advanced scene representations have moved from research papers to production at multiple AV programs. Tesla’s occupancy network, publicly described in the 2022 AI Day presentation, represents the scene as a 3D probabilistic occupancy field rather than a list of objects. Wayve’s GAIA models use generative scene representations as part of their end-to-end stack. NVIDIA Research has published work on neural-radiance-field-based scene representations applied to driving. The shift from object-list to dense-occupancy representations is the subject of substantial academic-industrial collaboration; the OpenScene benchmark and Argoverse 2 Lidar Sensor Dataset support evaluation of dense-scene-representation methods.

29.7 Chapter Summary

Advanced scene representations go beyond object lists: layered decomposition allocates compute proportionally to change rate; probabilistic occupancy fields replace discrete grids with continuous distributions supporting natural Bayesian fusion; semantic flow fields combine motion and class information for richer downstream reasoning; implicit scene representations encode the scene as a queryable neural function. These representations are entering production AV stacks at leading programs, gradually displacing the object-list abstraction that defined earlier generations of the stack.

29.8 Exercises

1. For the layered-compute example, compute savings if the fast-dynamic layer runs at 30 Hz instead of 20 Hz.
2. Design a layered-decomposition schema for an urban-robotaxi AV.
3. For the Bayesian-update example, compute posterior after five consecutive “occupied” observations.
4. Compare discrete-grid and probabilistic-field occupancy representations. Identify three cases where each wins.
5. Design a flow-+-semantics fusion rule that preserves both signals without conflating them.
6. For the disambiguation example, propose a quantitative metric for correct classification of each region.
7. Derive the dense-grid log-odds update in the general case of continuous-valued observations.
8. Identify three production AV programs using implicit scene representations; compare their approaches.
9. Propose a compact implicit scene representation that fits in 50 MB per kilometer of road.

10. Discuss the trade-offs of implicit vs explicit scene representations in terms of: compute, interpretability, debugging, update speed.

Chapter 30

Advanced Validation and Frontier Technologies

30.1 Introduction

The preceding chapters developed AV technology as it exists today. This final chapter develops the edge of the field: advanced validation and testing methodologies that are becoming standard practice, and frontier technologies that may change the field substantially over the next decade. Validation topics include hardware-in-the-loop (HIL) testing, software-in-the-loop (SIL) testing, closed-loop simulation, scenario generation, and corner-case mining. Frontier topics include neuromorphic computing, quantum sensing, multi-modal foundation models, and zero-shot / few-shot learning. These are placed together because both share the property of being *leading indicators*: where the industry is going rather than where it is.

Three observations motivate the structure.

First, validation is *keeping pace with deployment maturity*. As AV fleets scale, the cost of release-gate failures rises; investment in validation infrastructure has grown to match. Techniques that were research-only a decade ago (closed-loop simulation at scale, AI-generated scenarios) are now table stakes.

Second, frontier AI is *entering AV quickly*. Foundation models, self-supervised learning, and zero-shot methods from the broader AI field are being adapted to AV problems. The adaptation is imperfect; some frontier methods that work well on internet data struggle on long-tail driving data. But the trend is unmistakable.

Third, *computing substrate* itself is evolving. Neuromorphic computing and quantum sensing are far from production deployment but represent genuinely different hardware paradigms that could, over time, transform specific subsystems of AV perception.

The chapter is organized as follows: HIL and SIL testing methods; closed-loop simulation at scale;

scenario-based validation and corner-case mining; AI-driven scenario generation; sim-to-real transfer techniques; neuromorphic computing for event-based perception; quantum sensing for advanced navigation; multi-modal foundation models for unified perception; zero-shot and few-shot learning for rare-category handling. Three architecture diagrams visualize the test-pyramid hierarchy, the AI-driven scenario generation loop, and a foundation-model-based AV stack; worked examples use the hybrid style.

30.2 The Test Pyramid for AV Validation

Production AV programs use a multi-layer test pyramid that balances speed, fidelity, and cost at each layer.

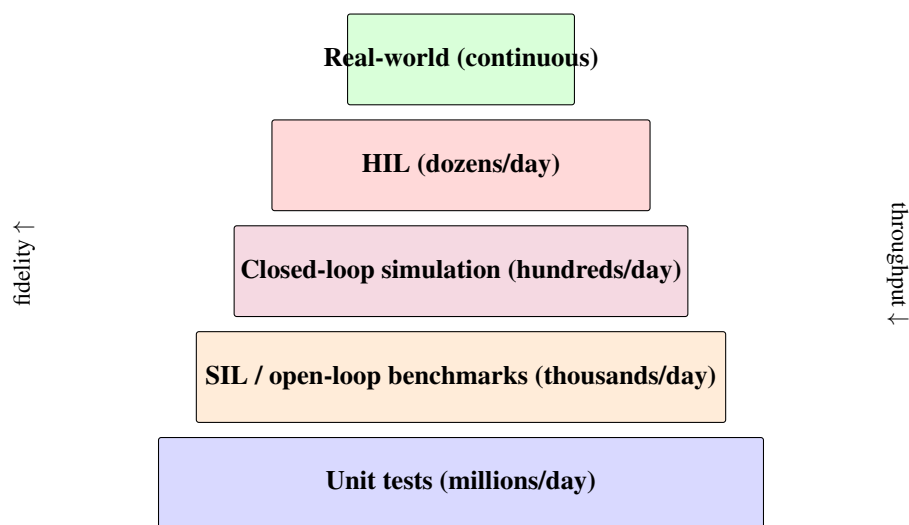


Figure 30.1: The AV test pyramid. Lower layers (unit tests, SIL) are fast, cheap, and run continuously at high volume. Upper layers (HIL, real-world) are slow, expensive, and run selectively. Each layer catches specific classes of issues; no layer alone is sufficient. Production programs calibrate the ratio of investment across layers based on where issues are found.

30.2.1 Unit Tests

Traditional software unit tests: functions, algorithms, individual modules. Run on every commit; failures block integration.

30.2.2 Software-in-the-Loop (SIL)

Simulated software components in a simulated environment. Test integration between modules (perception-to-planning, planning-to-control) without requiring real hardware. Fast, parallelizable, fully reproducible. Primary bug-catching layer for integration issues.

30.2.3 Closed-Loop Simulation

The full AV stack drives a simulated vehicle through a simulated world. Captures interaction effects (behavior feeds back into environment). Runs overnight in parallel across thousands of scenar-

ios.

30.2.4 Hardware-in-the-Loop (HIL)

Real AV hardware (ECUs, sensors) driving simulated environment. Tests actual latency, actual compute behavior, actual fault responses. Expensive (requires hardware); production programs run selected scenarios in HIL before releases.

30.2.5 Real-World Testing

Actual vehicles on actual roads. Cannot be fully replaced by simulation; final validation layer.

Example 30.1 (Test throughput across pyramid layers). Validation suite for a candidate release:

- Unit tests: 50 000, run in 10 minutes.
- SIL tests: 5 000, run in 2 hours.
- Closed-loop sim: 500 scenarios, run in 8 hours.
- HIL: 50 scenarios, run in 16 hours.
- Real-world: 20 000 m of track test + 2 weeks public-road shadow.

Total compute effort:

$$\begin{aligned} T_{\text{SIL}}^{\text{total}} &= 5000 \times 0.5 \text{ s/test} \\ &\approx 42 \text{ min on 1 machine.} \end{aligned}$$

$$\begin{aligned} T_{\text{sim}}^{\text{total}} &= 500 \times 5 \text{ min/scenario} \\ &= 42 \text{ hours on 1 machine,} \\ T_{\text{sim}}^{\text{parallel}} &\approx 1 \text{ h on 50 machines.} \end{aligned}$$

Upstream layers run cheap + often; HIL + real-world run expensive + selectively. The pyramid shape reflects the economics: cheap layers catch 90% of issues before an expensive layer is invoked.

30.3 Scenario-Based Validation

Modern AV validation is structured around *scenarios*: explicit descriptions of specific driving situations with defined expected outcomes.

30.3.1 Scenario Taxonomy

- **Logical scenarios**: abstract descriptions (“unprotected left turn at intersection with oncoming traffic at 10 ms^{-1} ”). Multiple concrete instantiations.

- **Concrete scenarios:** specific parametric instantiations. Different parameter choices produce different concrete tests.
- **Real-world scenarios:** logged real-world events replayed.

30.3.2 Scenario Libraries

Production programs maintain libraries of thousands to millions of scenarios covering normal operation, edge cases, and regression protection. NHTSA's test track scenarios, Euro NCAP's ADAS tests, PEGASUS scenario catalog, and proprietary corporate libraries.

30.3.3 Scenario Coverage

Is the library comprehensive? Metrics:

- Parameter coverage: fraction of operational parameter space sampled.
- Behavioral coverage: fraction of distinct AV behavior classes exercised.
- Failure-mode coverage: known failure modes represented.
- Real-world alignment: scenario distribution matches on-road frequency.

30.4 Corner-Case Mining

The long tail of rare scenarios is where most AV failures live. Corner-case mining systematically discovers them.

30.4.1 Fleet-Data Mining

Production AV fleets collect logs continuously. Anomaly detection on logs surfaces rare or interesting events: unusual trajectories, high-magnitude brake events, model-uncertainty spikes. These become candidate scenarios for the library.

30.4.2 Failure Injection

Deliberately perturb sensor inputs (hide an object, add noise) and observe how the stack handles it. Surfaces reliance assumptions that break under specific failures.

30.4.3 Model-Targeted Mining

Given a trained model, find inputs where it is uncertain, wrong, or disagrees with an ensemble. These inputs are added to training and validation suites.

30.4.4 Generative Corner-Case Mining

Use generative models to produce plausible rare scenarios the fleet hasn't encountered: rare object types, unusual weather, novel road geometries. AI-driven scenario generation is an emerging area.

Example 30.2 (Corner-case yield from fleet mining). Fleet of 2 000 vehicles, each averaging 500 miles/day, total 10^6 miles/day.

Anomaly rate (scenarios of interest):

$$\begin{aligned} r_{\text{anomaly}} &= 1 \text{ per } 100 \text{ miles} \\ n_{\text{anomaly}} &= 10^6 / 100 = 10^4 \text{ events/day.} \end{aligned}$$

After filtering to genuinely novel:

$$\begin{aligned} n_{\text{novel}} &= 0.02 \times 10^4 \\ &= 200 \text{ genuinely novel scenarios/day.} \end{aligned}$$

Annual novel-scenario yield:

$$\begin{aligned} N_{\text{annual}} &= 365 \times 200 \\ &= 73,000 \text{ scenarios/year.} \end{aligned}$$

Even after aggressive filtering, tens of thousands of new scenarios per year. Scalable triage, clustering, and prioritization are required; the raw stream overwhelms manual review.

30.5 AI-Driven Scenario Generation

Generative models produce novel scenarios on demand. Applications: targeted testing of specific behaviors, expanding the scenario library, red-teaming the stack.

30.5.1 Parametric Generation

Start from an abstract logical scenario; sample parameters from distributions matching real-world statistics. Simplest approach; produces plausible concrete scenarios.

30.5.2 Adversarial Generation

Generate scenarios specifically designed to challenge the AV stack. Given the stack's current weaknesses, produce scenarios that exercise them. Training signal: stack failure rate.

30.5.3 Large Language Models for Scenarios

Describe a scenario in natural language (“a pedestrian pushing a stroller crosses at a mid-block location while rain limits visibility to 20 m”); generative model produces the concrete simulator input. Emerging approach; rapid iteration on scenario design.

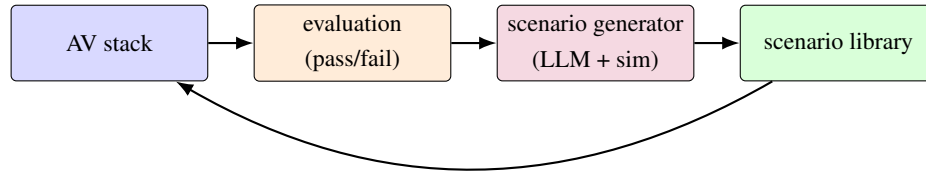


Figure 30.2: AI-driven scenario generation loop. The AV stack is evaluated against scenarios; failing scenarios (and rare successes) feed a generator that creates related variants. New scenarios enter the library for subsequent rounds of testing. The loop progressively discovers weaknesses and constructs regression protection.

30.6 Sim-to-Real Transfer

Models trained in simulation must transfer to the real world. The sim-to-real gap is the practical limit on simulation value.

30.6.1 Domain Randomization

Randomize simulator parameters (textures, lighting, agent behaviors) so the model learns features invariant to the randomized dimensions. If the randomization is broad enough, real-world parameters fall within the trained distribution.

30.6.2 Domain Adaptation

Train on simulation but adapt to real via unpaired data (CycleGAN-style approaches) or targeted real-data fine-tuning. Reduces required real-data volume while leveraging simulation scale.

30.6.3 Closing the Gap with Real-to-Sim

Fit simulator parameters to real data. NeRF-based simulators (Chapter 26) take this further: use real fleet data to construct the simulator itself.

30.6.4 Validation of Sim-to-Real

Is the sim-to-real pipeline working? Metrics:

- Sim accuracy on real test set.
- Real improvement per unit of sim training.
- Convergence: does adding sim data continue to improve real performance, or saturate?

Example 30.3 (Sim-to-real training mix optimization). Model trained with varying ratio of sim-to-real data:

- 100% real, 0% sim: baseline real mAP 0.82.
- 50% real, 50% sim: real mAP 0.85.
- 10% real, 90% sim: real mAP 0.80.
- 0% real, 100% sim: real mAP 0.68.

Best mix at 50/50; pure sim fails to transfer; small real data with most sim underperforms because coverage is poor. Sim is a *complement* to real data, not a replacement.

30.7 Neuromorphic Computing

Neuromorphic chips implement spiking neural networks inspired by biological brains. For AVs, they show particular promise for event-based perception.

30.7.1 Event Cameras

Traditional cameras capture frames at fixed rates. Event cameras capture pixel-level brightness changes asynchronously, at microsecond precision. Lower latency, higher dynamic range, lower power consumption.

30.7.2 Spiking Neural Networks

Networks where neurons communicate via discrete spikes rather than continuous activations. Compatible with event-camera outputs; potentially more power-efficient on neuromorphic hardware.

30.7.3 Chips and Platforms

Intel Loihi, IBM TrueNorth, SpiNNaker, BrainChip Akida. Commercial automotive deployment is early; research prototypes demonstrate compelling power/latency advantages for specific tasks.

Example 30.4 (Event camera vs frame camera bandwidth). Frame camera: 1 megapixel, 30 Hz, 3-byte pixels.

Bandwidth:

$$\begin{aligned} B_{\text{frame}} &= 10^6 \times 30 \times 3 \\ &= 90 \text{ MB/s.} \end{aligned}$$

Event camera: 1 megapixel, 10 % events/second, 4 bytes/event.

Bandwidth:

$$\begin{aligned} B_{\text{event}} &= 10^5 \times 4 \\ &= 0.4 \text{ MB/s.} \end{aligned}$$

Event camera bandwidth is $225\times$ lower. Power consumption scales similarly. Latency: frame-based is bounded by frame-rate (33 ms at 30 Hz); event-based is microsecond-scale. Event cameras excel for high-speed perception (tracking fast-moving objects, detecting rapid changes) but are not a replacement for conventional cameras in object classification tasks.

30.8 Quantum Sensing

Quantum techniques for extraordinarily precise measurement of physical quantities (magnetic fields, gravity, acceleration). Relevant to AV navigation in GNSS-denied environments.

30.8.1 Quantum Inertial Navigation

Cold-atom interferometry provides accelerometer and gyroscope measurements with drift rates orders of magnitude below classical IMUs. An AV equipped with a quantum IMU could navigate for days without GNSS with meter-level accuracy.

30.8.2 Quantum Gravity Gradiometers

Sensitive gravitational measurements could support map-based localization without any external signals. Research stage.

30.8.3 Timeline and Practicality

Quantum sensors remain bulky, power-hungry, and expensive. Near-term deployment unlikely for consumer AVs; potential for specialized applications (military, mining) within 10 years.

30.9 Multi-Modal Foundation Models

Foundation models are large neural networks pretrained on broad data that serve as the basis for many downstream tasks. In AV contexts, multi-modal foundation models handle camera, LiDAR, text, and potentially other modalities in a unified architecture.

30.9.1 Vision-Language Models

CLIP, ALIGN, and successors learn joint embeddings of images and text. For AVs: enables text-queryable perception (“pedestrians in the scene”, “red vehicles”), zero-shot recognition of novel object types, and integration of high-level natural-language reasoning.

30.9.2 Multi-Sensor Fusion at Foundation Scale

Newer foundation models ingest camera, LiDAR, and radar jointly. Unified feature spaces support downstream tasks (detection, segmentation, prediction) through lightweight heads rather than full re-training.

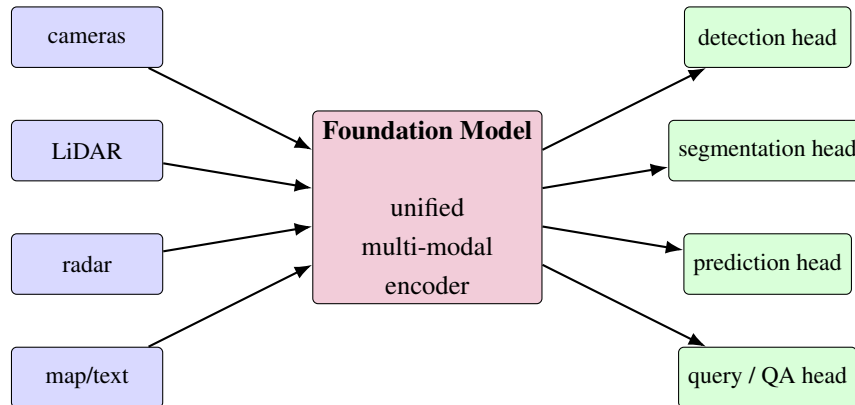


Figure 30.3: Foundation-model-based AV perception. A single large model encodes all input modalities into a shared representation. Lightweight task-specific heads produce detections, segmentation, prediction, and natural-language query answers. Compared to per-task specialized models, a foundation-model approach promises better transfer, easier adaptation, and unified reasoning across tasks.

30.9.3 Challenges

Foundation models have practical limitations for AV production:

- Compute cost: too heavy for on-vehicle real-time inference.
- Latency: even pruned and quantized, current models are above automotive latency budgets.
- Safety-critical behavior: foundation models occasionally produce hallucinations or confident wrong answers.
- Training data curation at foundation scale.

These are being actively addressed; production foundation-model-based AV stacks may be 2–4 years away.

30.10 Zero-Shot and Few-Shot Learning

Training a model on few (few-shot) or zero (zero-shot) examples of a new class. Critical for AV because the long tail includes many classes the fleet has never labeled.

30.10.1 Prompt-Based Classification

Using vision-language foundation models: classify an image by comparing its embedding to text embeddings of candidate classes. Zero-shot recognition of “zebra crossing an intersection” without

ever having labeled such an example.

30.10.2 Few-Shot Learning

Given a handful of examples of a new class, adapt a model to detect them. Standard approach: prototype network that compares to class exemplars in feature space.

30.10.3 Production Role

Not yet a primary production approach; complement to fully-supervised training. Valuable for covering the long tail of rare object classes without labeling effort proportional to their frequency.

Example 30.5 (Few-shot addition of a new class). Production model trained on 20 classes, excellent performance. New class to be supported: “cargo scooter” (motorized delivery platform).

Minimal-effort path:

- Collect 30 labeled examples.
- Fine-tune a small adapter on these examples.
- Measure: AP for new class ≈ 0.72 (moderate, usable for cautious driving).

Compare full re-training:

- Collect 3000 labeled examples.
- Retrain full model.
- AP for new class ≈ 0.90 (production-grade).

Few-shot gives 80 % of the performance with 1 % of the labeling effort; production decision depends on the expected frequency of the class. Rare classes: few-shot; frequent important classes: full training.

30.11 Industrial Deployment

Frontier AV technologies including foundation models, neuromorphic sensing, and quantum-enhanced inertial navigation are progressing at varying rates. Wayve has publicly described their LINGO and GAIA foundation-model-style systems for driving. Waabi (Toronto) has developed end-to-end neural simulation. Event cameras from Prophesee (France) and iniVation (Switzerland) have entered niche production applications with demonstrations in automotive contexts; commercial automotive deployment remains limited as of 2024–2025. Quantum-enhanced sensing remains predominantly at research and defence-application stages, with companies including AOSense (United States), MUQUANS (now part of Exail, France), and Q-CTRL (Australia) pursuing commercial cold-atom and quantum-sensor products.

30.12 Chapter Summary

AV validation has evolved from ad-hoc testing into a structured discipline spanning unit tests, SIL, closed-loop simulation, HIL, and real-world testing. Scenario-based validation, corner-case mining, and AI-driven scenario generation enable systematic coverage of the long tail. Sim-to-real transfer bridges the gap between scalable simulation and ground-truth real-world testing. Frontier technologies—neuromorphic computing and event cameras for low-latency high-speed perception, quantum sensing for GNSS-independent navigation, multi-modal foundation models for unified perception, zero-shot and few-shot learning for the long tail—are entering research and early production. None is yet a finished product; all are shaping where the field is heading. An AV program that invests in validation infrastructure and engages with frontier technologies positions itself for the next decade of capability expansion.

30.13 Exercises

1. For the test-pyramid example, compute the total compute needed to run the full suite if parallelism is limited to 100 machines.
2. Design a scenario-coverage metric that is not fully satisfied by any finite library.
3. For the corner-case mining example, compute the annual novel-scenario yield if fleet size doubles.
4. Propose an AI-driven scenario-generation procedure for a specific target behavior (e.g., aggressive lane changes).
5. For the sim-to-real example, compute the expected real performance at a 25%/75% real/sim mix.
6. Compare event-based and frame-based perception for detecting a pedestrian running across an intersection. Which performs better, and why?
7. Discuss the scale at which quantum inertial navigation becomes practically valuable for AVs. Identify three deployment scenarios.
8. For the foundation-model architecture, specify three specific tasks that would share the same foundation encoder but have distinct heads.
9. For the few-shot example, compute the expected performance if the class has 100 examples available rather than 30.
10. Identify three risks of depending heavily on foundation models for safety-critical AV behavior.

Chapter 31

Ontologies and Semantic Representations for AVs

31.1 Introduction

A bounding box labelled “car” is a poor substitute for understanding what a car is, what it can do, and how it relates to the scene it inhabits. Production AV stacks increasingly complement learned perception with *ontologies*—explicit, machine-readable representations of the entities, relationships, and rules that populate driving. An ontology might record that a school bus is a type of bus, that a stopped school bus with flashing lights imposes a yielding requirement on surrounding vehicles, and that the same lights have different semantics at an intersection than in the middle of a block. Statistical learners struggle with this kind of long-tail structured knowledge; ontologies encode it directly.

Three observations motivate the chapter’s structure.

First, ontologies *complement* learned perception rather than replacing it. Neural networks remain the workhorses for raw sensor-to-object transduction; ontologies add semantic structure to the resulting object set—what the objects mean, how they relate, and what rules apply.

Second, ontologies encode *regulatory and cultural* knowledge that training data captures incompletely. Lane-change etiquette in Munich differs from that in Shanghai; school-zone rules in Texas differ from those in Ontario. Encoding these explicitly in an ontology makes them auditable and region-portable.

Third, ontologies support *auditable reasoning* that regulators increasingly demand. A system that can explain “I yielded because I detected a school bus (instance), school bus with flashing lights is a protected-stop class (rule), and I am in the same-direction lanes (spatial relation)” produces accountability that pure learned systems do not.

The chapter is organized as follows: ontology fundamentals; the ASAM OpenXOntology standard; scene graphs; knowledge graphs; hybrid neuro-symbolic systems; integration with planning; and

maintenance at fleet scale.

31.2 Ontology Fundamentals

An ontology is a formal description of a domain in terms of *entities* (things that exist), *classes* (kinds of things), *relations* (how things are connected), and *axioms* (rules that hold). For AVs:

Element	Purpose	Example
Class	A type of entity	Vehicle, Pedestrian, Lane
Instance	A specific entity in the scene	Ford F-150 at (x, y, z)
Relation	How entities connect	“in-lane-of”, “yielding-to”
Axiom	A rule that always holds	“school-bus $\Rightarrow$ bus”

31.2.1 Description Logics and OWL

Web Ontology Language (OWL) is the standard formalism for computer-processable ontologies. OWL 2 DL is expressive enough for most AV use cases and is supported by production reasoners (Pellet, HermiT, ELK).

Example 31.1 (Ontology class hierarchy for road users). A fragment of a driving ontology:

Class axioms:

$$\begin{aligned} \text{RoadUser} &\equiv \text{Vehicle} \sqcup \text{Pedestrian} \sqcup \text{Cyclist}, \\ \text{Vehicle} &\sqsubseteq \text{RoadUser}, \\ \text{Bus} &\sqsubseteq \text{Vehicle}, \\ \text{SchoolBus} &\sqsubseteq \text{Bus}. \end{aligned}$$

Instance assertion: A detected object x is classified as SchoolBus. Transitive inference:

$$\begin{aligned} \text{SchoolBus}(x) &\Rightarrow \text{Bus}(x) \\ &\Rightarrow \text{Vehicle}(x) \\ &\Rightarrow \text{RoadUser}(x). \end{aligned}$$

Rule axiom.

$$\text{SchoolBus}(x) \wedge \text{FlashingRed}(x) \Rightarrow \text{RequireYield}(x).$$

The ontology reasoner fires the yield requirement automatically whenever both conditions are detected—without the planner needing hand-coded logic for this specific case.

31.3 ASAM OpenXOntology

ASAM OpenXOntology is an emerging industry standard for driving ontologies, developed by the same standards body behind OpenDRIVE and OpenSCENARIO. It aims to provide a shared vocabulary across simulation, map, and scenario tools.

31.3.1 Structure

Three top-level domains:

Domain	Content
Entities	Physical objects: vehicles, pedestrians, infrastructure, terrain
Environment	Conditions: weather, illumination, road surface
Scenarios	Compositions: actions, maneuvers, sequences

Tools from different vendors read and write to this shared vocabulary, so a scenario authored in one simulator can be replayed (with semantics preserved) in another.

31.4 Scene Graphs

A scene graph is an instance-level representation of the current scene: nodes are entities, edges are relations. Scene graphs are the runtime instantiation of the static ontology.

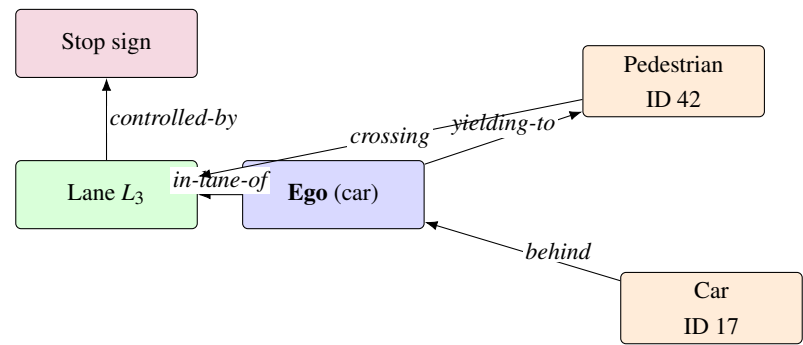


Figure 31.1: Scene graph for an intersection. Nodes (entities) carry typed class information; edges (relations) carry the semantic structure that supports reasoning. Compared to an object list, the scene graph makes interaction-relevant relationships explicit: “ego is yielding to pedestrian”, “lane is controlled by stop sign”, etc.

31.4.1 Relations of Interest

Driving scene graphs routinely encode:

- Spatial relations: *in-lane-of*, *left-of*, *in-front-of*, *approaching*, *yielding-to*.

- Semantic relations: *is-a*, *instance-of*, *controlled-by*.
- Temporal relations: *was-at*, *will-be-at*, *predicted-to*.
- Interactional relations: *blocking*, *overtaking*, *following*, *cutting-in*.

31.5 Neuro-Symbolic Integration

Symbolic ontologies and neural perception are complementary. Hybrid systems combine them:

Stage	Neural role	Symbolic role
Detection	“That’s a pedestrian”	Class assertion
Tracking	Associates observations	Instance identity
Reasoning	Produces features	Derives relations, rules
Planning	Scores trajectories	Enforces constraints

Each side does what it is good at: neurons handle noisy continuous signal-to-class mappings, symbols handle structured relational reasoning and enforceable constraints.

Example 31.2 (Ontology-driven safety override). Scene: ego approaching an intersection. Perception detects:

- SchoolBus (confidence 0.95)
- FlashingRedLights active on this bus (confidence 0.88)

Symbolic inference.

$$\begin{aligned} \text{SchoolBus}(x) \wedge \text{FlashingRed}(x) &\Rightarrow \text{RequireYield}(x), \\ \text{RequireYield}(x) &\Rightarrow v_{\max} = 0 \text{ m/s (all lanes)}. \end{aligned}$$

Planner override.

Planner’s nominal trajectory would pass in the adjacent lane at 15 m s^{-1} . The ontology-derived constraint $v_{\max} = 0$ overrides the nominal; planner executes a full stop until the flashing lights clear.

The combination is more reliable than either alone: perception alone might confidently classify a bus-shaped vehicle without applying the school-bus yielding rule; rules without perception have no trigger.

31.6 Maintenance at Fleet Scale

Ontologies are living artifacts. They must evolve with:

- New regulations (new road-user classes, new right-of-way rules).

- New object categories (e-scooters, delivery robots, emerging micromobility).
- Regional variation as deployment expands across jurisdictions.
- Error correction as edge cases surface.

Production programs maintain ontologies through change-management processes similar to code: version control, review, regression testing, staged rollout.

31.7 Industrial Deployment

Ontologies and knowledge graphs in AVs draw on industry standards including ASAM OpenScenario, OpenLABEL, and OpenDRIVE for road-network description. ASAM (Association for Standardisation of Automation and Measuring Systems), based in Germany, develops most of these standards. The European Telecom Standards Institute (ETSI) defines V2X message ontologies including BSM and SPaT message structures. The Object Management Group (OMG) maintains the Distribution Service standard underpinning ROS 2 communication. Schema.org Vehicle ontology and Wikidata provide general public ontologies for vehicles. Production AV programs typically maintain proprietary internal ontologies for scene description but increasingly contribute to or align with the standardised formats above.

31.8 Chapter Summary

Ontologies provide explicit, machine-readable semantic structure that complements learned perception. Classes, instances, relations, and axioms are encoded in formalisms like OWL; industry standards like ASAM OpenXOntology are emerging. Scene graphs instantiate ontologies at runtime. Neuro-symbolic integration combines statistical learners with rule-based reasoners, supporting auditable behavior that regulators increasingly require. Ontology maintenance is a production discipline, not a one-time design task.

31.9 Exercises

1. Design an OWL class hierarchy for the full set of vulnerable road users.
2. For the school-bus example, add a disjointness axiom preventing SchoolBus and Motorcycle from overlapping.
3. Draw a scene graph for a 4-way intersection with 6 agents.
4. Compare ontology-based and purely-learned safety rules. Identify one advantage of each.
5. Propose a versioning scheme for a fleet-scale driving ontology.
6. Derive three additional yielding rules that an ontology should encode but a typical perception model would miss.

7. Design a scene-graph-to-planner interface. What data flows in each direction?
8. Discuss the limits of ontology-based reasoning: what cannot be encoded and why.
9. Propose a benchmark for evaluating the quality of an AV ontology.
10. Compare ASAM OpenXOntology with a proprietary ontology at a major AV program. What are the tradeoffs?

Part VIII

Safety and Verification

Chapter 32

Safety and Verification

32.1 Introduction

Every chapter in this book so far has been about *making autonomous vehicles work*. This chapter is about the much harder question: *how do we know they work, and how do we know they will continue to work, safely, in conditions we have not anticipated?* This is the safety-and-verification problem, and it is the problem that separates a research prototype from a commercial deployment. Many AV programs have produced impressive demonstrations; far fewer have produced deployments that regulators and insurers accept. The difference is almost entirely in the safety case.

Three observations motivate the chapter’s structure.

First, safety is not a feature; it is a system property. You cannot attach “safety” to an AV as a final layer. Safety requires engineering discipline throughout the stack: hazard analysis driving requirements, redundancy design, fault detection and response, continuous monitoring, and a failure-mode analysis that admits no plausible hole. A single-module safety layer on top of an otherwise-normal stack is not enough; production deployments of L3/L4 systems all embed safety reasoning at every level.

Second, driving safety is a *rare-event* problem. A human driver has a fatal accident roughly every 10^8 miles; matching that with any statistical confidence requires *billions* of miles of real-world operation. Pure data-driven demonstration of safety is not feasible within a single company’s fleet lifetime. The safety case must be built on a combination of simulation, targeted scenario testing, structured reasoning, and incremental real-world deployment.

Third, the regulatory landscape is consolidating. UNECE WP.29, ISO 21448 (SOTIF), ISO 26262, SAE J3016, and AV-specific regulations in California, Texas, Arizona, Germany, Japan, and China are increasingly interlocking. Compliance is non-optional and costly—a production AV program allocates a substantial fraction of its engineering effort to safety engineering and regulatory submission.

The chapter is organized as follows. We first introduce hazard analysis and safety cases, then develop operational design domains (ODDs) and the ODD-bound safety argument. We cover formal

verification (for the components amenable to it), safety shields and runtime assurance, redundancy and graceful degradation, fault detection and response, validation and testing methodologies, and safety metrics. Production safety engineering practice closes the chapter. Three architecture diagrams visualize the layered safety stack, the ODD envelope, and the runtime-assurance pattern; worked examples cover the specific arithmetic.

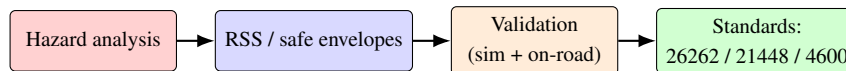


Figure 32.1: Chapter roadmap. Safety progresses through four stages: hazard analysis identifies what can go wrong; safety envelopes define provable guarantees; validation gathers evidence; standards certification turns the evidence into a deployable safety case.

32.2 Hazard Analysis and Safety Cases

A safety case is a structured argument that the system is acceptably safe for its intended use. Building one starts with *hazard analysis*: identifying the ways the system can cause harm.

32.2.1 Hazard Analysis

Hazard analysis is where the safety case begins. Before any engineering decision about mitigation or verification can be made, we need a systematic catalog of the ways the system can produce harm. Several well-established techniques scale to AV-size systems; each takes a different decomposition angle, and production programs typically apply more than one.

Standard techniques include:

- **HAZOP** (Hazard and Operability): systematic review of each subsystem, asking “what if” questions.
- **STPA** (Systems-Theoretic Process Analysis): models the system as control loops and asks how each control action could produce unsafe outcomes.
- **FMEA** (Failure Mode and Effects Analysis): catalogs failure modes and their downstream consequences.

The output is a hazard list: a set of (condition, failure, consequence) tuples that the system must not realize. For a Level 4 AV, typical hazard list contains several hundred entries.

Example 32.1 (Hazard entry for camera blockage). **Hazard:** Forward camera blocked (mud, ice, debris). **Cause:** weather, driving on unpaved roads, vehicle accumulating road grime. **Consequence:** perception fails to detect pedestrian at forward distance → potential collision. **Mitigation:** (a) runtime detection of camera health (mean-intensity, feature-density monitors), (b) automatic wipers, (c) fusion degradation (radar retains pedestrian detection), (d) planner fallback to conservative envelope (lower speed, larger margin) when camera degraded, (e) graceful degradation to minimum-risk condition if multiple forward sensors fail. **Verification:**

fault-injection testing in HIL and in-vehicle.

32.2.2 Safety Case Structure

Once hazards are cataloged, the safety case is the document that ties them—and their mitigations—into a structured argument. A well-formed safety case reads like a legal brief built on technical evidence: top-level claims decomposed into sub-claims, each sub-claim backed by specific evidence, all framed by explicit assumptions.

A safety case argues from evidence to the top-level claim “the system is acceptably safe.” Common structure (GSN, Goal Structuring Notation):

- **Goals:** top-level safety claims.
- **Strategies:** how the goal is decomposed.
- **Solutions:** evidence supporting decomposed claims.
- **Assumptions/Contexts:** the scope within which the claim holds.

A production safety case document for a L4 program runs to thousands of pages of interconnected argument and supporting evidence.

32.3 Operational Design Domain

An ODD is the set of operating conditions under which the AV is designed and verified to operate safely. Figure 32.2 shows an example.

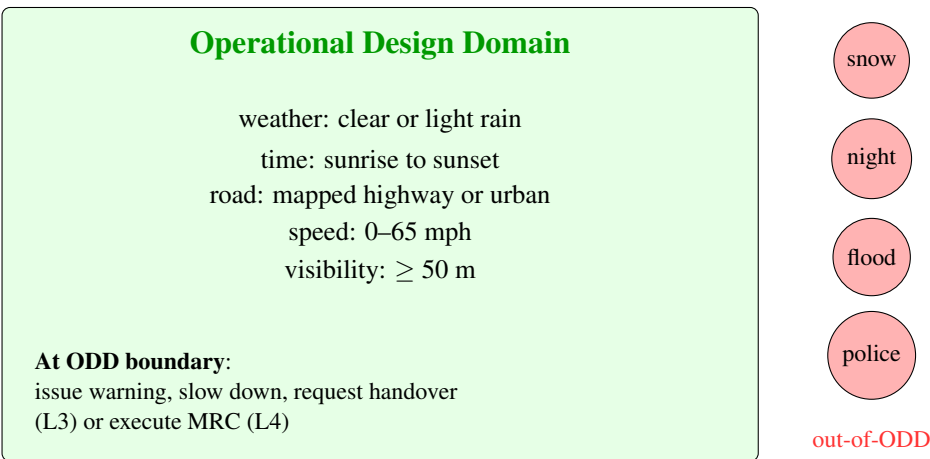


Figure 32.2: Operational Design Domain (ODD) envelope. Inside the ODD (green), the AV operates autonomously with its normal safety case. Outside (red circles), the AV either degrades or hands over. The ODD boundary is the most important line in the safety case: what happens at and near the boundary defines how the system fails gracefully. Real ODDs are multi-dimensional (weather $\times$ time $\times$ road-class $\times$ speed $\times$ map-availability) and may be disjoint (e.g., highway-only L3 plus mapped-urban L4 but not the connecting roads).

32.3.1 ODD Specification

A production ODD is a detailed specification, not a one-line summary. The precision of the specification matters: everything about the vehicle’s behavior—what it attempts, how it degrades, when it hands over—hangs on what is in the ODD and what is not. Under-specified ODDs leave open unsafe behaviors; over-specified ODDs make the system unusable.

For each dimension (road type, lighting, weather, speed, construction, etc.), specific operating thresholds are defined and checked at runtime.

Example 32.2 (Partial ODD specification). AV Program X’s L4 ODD (partial):

- Road: mapped within the program’s tile set; speed-limit ≤ 45 mph; no unprotected left turns onto roads with speed-limit > 40 .
- Weather: precipitation $\leq$ “light” per weather sensor; visibility ≥ 75 m; road surface non-icy.
- Time: 6 AM to 9 PM local.
- Other: no emergency vehicles in detection range; no flooded road surface detection.

A runtime monitor continuously evaluates all conditions. Any out- of-ODD trigger initiates a graceful handover or minimum-risk condition sequence.

32.3.2 ODD Exit and Minimum-Risk Condition

What happens when the ODD boundary is reached or crossed is one of the most safety-critical behaviors a production AV exhibits. The exit procedure must be fast enough to preserve safety margin, smooth enough to avoid creating new hazards, and predictable enough that other road users can respond appropriately.

When the ODD is violated (or violation is predicted), the AV must either (a) hand control back to a human (L3) or (b) execute a *minimum-risk condition* (L4): a controlled maneuver bringing the vehicle to a safe state. MRC options: pull over and stop; park in the current lane with hazard lights; seek an emergency lane or exit.

32.4 Formal Verification

For components amenable to it, formal verification produces mathematical proofs of safety properties. Techniques and their AV applications:

- **Model checking:** proves that a finite-state model satisfies a specification. Used for control-flow logic in behavior planners.
- **Reachability analysis:** computes the set of future states a system can reach. Used for collision avoidance envelopes.
- **Control barrier functions (CBF):** certifies forward invariance of safe sets by constraint on control. Increasingly used as runtime shields (Chapter 19).

- **Interval analysis / zonotope:** computes bounded over-approximations of reachable sets.

Formal methods work well for control and planning but are hard to apply to learned perception; a hybrid “formal for safety-critical layer, test for the rest” pattern is standard.

Example 32.3 (CBF-certified forward invariance). CBF: $B(\mathbf{x}) = d_{\text{lead}} - d_{\text{safety}}(v) \geq 0$, with $d_{\text{safety}}(v) = v^2/(2a_{\text{max}})$. The CBF condition requires $\dot{B} \geq -\alpha(B)$ for some positive α . Any control action satisfying this inequality keeps $B \geq 0$ forever—i.e., the ego never enters the unsafe region. A minimum-intervention shield projects the nominal control onto the half-space of CBF-compliant controls: $u^* = \arg \min \|u - u_{\text{nom}}\|^2$ subject to the CBF constraint. This is a single QP per tick; in <1 ms provides a formal safety guarantee over the CBF’s domain.

32.5 Runtime Assurance and Safety Shields

The *Simplex* architecture (Figure 32.3) runs a complex primary controller and a simple verified backup in parallel; a decision module (itself simple and verified) decides which to output based on real-time safety checks.

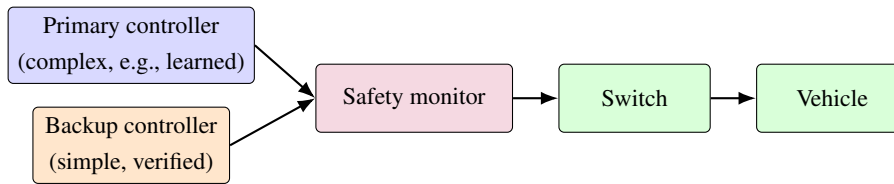


Figure 32.3: Simplex (runtime assurance) architecture. A complex primary controller proposes actions; a simple, formally-verified backup produces safe-but-possibly-suboptimal actions; a safety monitor decides which to forward to the vehicle based on real-time safety checks. The architecture accepts complex learning-based decisions when they are safe and falls back to simple guarantees when they are not. Widely used for learned-component deployment in safety-critical domains.

Example 32.4 (Simplex in an L4 stack). Primary: neural-network trajectory planner producing (x, y, v) at 10 Hz. Backup: classical MPC planner with conservative cost function. Monitor: evaluates primary trajectory’s reachable-set overlap with any predicted other-agent trajectory; if overlap exceeds threshold, use backup. Runtime overhead: <5 ms per tick. Effect: primary’s strengths (natural behavior, complex scenarios) available when safe; backup’s guarantees available otherwise. Production programs shipping learned planners use this pattern almost universally.

32.6 Redundancy and Graceful Degradation

Production AV systems use redundancy at every level:

- **Sensor redundancy:** overlapping LiDAR + camera + radar coverage; single sensor failure does not blind a region.

- **Compute redundancy:** multiple ECUs with different vendors, running same software, with voting on critical decisions.
- **Actuator redundancy:** two independent brake systems (primary hydraulic + secondary electric), two steering paths, two power supplies.
- **Algorithm redundancy:** diverse detectors running in parallel (different architectures, different training data); agreement required for deployment, disagreement triggers conservative behavior.

Graceful degradation is the principle that partial failure reduces capability without causing catastrophic failure. A camera failure degrades long-range perception but does not prevent safe stopping; a GNSS outage degrades global localization but SLAM continues.

32.7 Fault Detection and Response

Production AVs monitor their own health continuously:

- Sensor health (intensity, feature density, thermal consistency).
- Compute health (latency, memory, thermal).
- Software health (message timeouts, stale outputs).
- Actuator health (command-response match).
- Driver health (for L2/L3, eye tracking and hands-on).

Detection triggers a predefined response: alert, degrade, MRC. The response ladder is critical—too aggressive causes false-positive interruptions; too lenient risks real failures proceeding.

32.8 Validation and Testing

A production AV safety case combines several evidence sources.

32.8.1 Scenario-Based Testing

Scenario-based testing is the workhorse evaluation methodology for AV safety. The premise: rather than drive billions of miles hoping to encounter relevant edge cases, enumerate them directly and test against the enumeration. The test library becomes a form of requirements specification—anything the AV claims to handle must pass the corresponding scenarios.

A scenario library of 10000+ structured test scenarios covering ODD boundaries, edge cases, and known hazards. Each scenario replayed in simulation with controlled perturbations; policies judged by well-defined pass/fail criteria.

32.8.2 Simulation at Scale

Simulation supplements real-world driving with computational abundance: many more miles, controlled variants, reproducible scenarios. The key engineering challenge is fidelity—a simulator that agrees with reality only on easy cases produces false confidence. Production simulators are validated against matched real-world drives to estimate the sim-to-real gap for each category of behavior.

Large simulation farms (10 to 100 million miles/day) replay real-world and synthesized scenarios. The main value: distributional coverage that fleet data alone cannot provide.

32.8.3 Real-World Testing

Simulation is necessary but not sufficient; actual driving in the deployment environment remains the ultimate validation. Production programs follow a staged progression that progressively expands real-world exposure while retaining fail-safe human backup.

Staged deployment: closed course → public roads with safety driver → public roads without safety driver (L4). Each stage produces metrics (intervention rate, collision rate, near-miss rate) that feed back into training and validation.

Example 32.5 (Intervention-rate tracking). Early deployment: 1 intervention per 100 miles. After 6 months of fleet-data-driven retraining: 1 intervention per 2000 miles. Target for driverless operation: <1 intervention per 100 000 miles. Each order-of-magnitude improvement requires fleet-scale data collection and ~6 months of iterative training. A formal safety case and regulator approval follows, not precedes, empirical performance at the target level.

32.8.4 Disengagement Reports

Public reporting of disengagement data provides cross-program comparability—imperfect, but unique in a field where companies otherwise share little. Engineers should know what these reports measure, what they do not, and how they are interpreted outside the industry.

California’s annual disengagement reports (and analogous jurisdictional filings) require AV companies to publicly report miles driven and disengagement events. The comparison across companies is contested; the reports are nevertheless a useful coarse signal of progress.

32.9 Safety Metrics

Meaningful safety metrics include:

- **Intervention rate:** disengagements or takeovers per mile.
- **Injury rate / collision rate:** fleet-aggregated.
- **Near-miss rate:** requires definition; typical: ego or other required emergency braking.

- **Time in ODD:** fraction of operating time with all conditions satisfied.
- **Coverage metrics:** fraction of scenario library passing.

No single metric is sufficient. A low intervention rate in an easy ODD is not impressive; a high intervention rate in a challenging ODD is not necessarily a failure.

32.10 Worked Examples

Example 32.6 (End-to-end safety argument). Claim: AV Program Y's urban-L4 system in City X is acceptably safe. Evidence: (a) ODD bounded to mapped roads, dry weather, daylight, speed limit ≤ 35 ; (b) Hazard analysis identified 247 hazards; each has a mitigation and verification; (c) Simulation: 30 million scenarios passed per release; (d) Real-world: 50k miles with safety driver, 5k miles driverless, intervention rate dropping by $2\times$ per month; (e) Safety monitor with verified backup on all safety-critical paths; (f) Regulator filings: California AV permit, city-specific MOU. The argument is not complete until items (c)–(e) cross defined thresholds; (a)–(b) are preconditions.

Example 32.7 (Simplex rejection of a learned plan). Primary planner (learned) proposes: aggressive lane change into a gap projected to tighten in 2.1 seconds. Safety monitor computes: with ego dynamics + measurement-uncertainty envelope, the gap closes in ≥ 1.5 seconds; ego reaches lane-change completion in ≥ 2.3 seconds; probability of collision exceeds 0.02. Threshold: 0.005. Monitor switches to backup (conservative MPC); backup abandons the lane change, stays in lane at current speed. Over 10k such events logged, 8k were genuine unsafe primary proposals (primary continues to improve via training on these); 2k were false-positive monitor rejections (monitor continues to be tuned based on log review).

Example 32.8 (Graceful degradation cascade). LiDAR 1 of 4 fails (harness connector loose). Detection: sensor health monitor flags LiDAR 1 unresponsive within 500 ms. Perception stack: degrades to 3-LiDAR coverage, re-weighted BEV with uncertainty inflation in the LiDAR-1 sector. Planner: receives uncertainty signal, reduces max speed by 15 % and extends follow-distance. If a second LiDAR fails, perception degradation triggers ODD exit $\rightarrow$ MRC (pull-over). Each level of degradation has a defined behavior; the cascade is tested in HIL and live vehicle fault-injection tests prior to release.

32.11 Industrial Deployment

Safety standards used by production AV programs include ISO 26262 (Functional Safety, governing the avoidance of hazards arising from malfunctions), ISO/PAS 21448 (SOTIF, Safety of the Intended Functionality, governing the avoidance of hazards arising from inadequate performance), and UL 4600 (Standard for Safety for the Evaluation of Autonomous Products), the last published by Underwriters Laboratories specifically for autonomous products. UNECE Regulation 157 governs Automated Lane

Keeping Systems and was the first United Nations regulation specifically for an automated driving system. The Responsibility-Sensitive Safety (RSS) framework published by Mobileye [60] provides a formal safety model that has influenced production systems and has been incorporated into the IEEE 2846 standard published in 2022. The Waymo Safety Framework and Cruise Safety Report are publicly published documents describing the respective programs' safety cases.

32.12 Chapter Summary

Safety in AVs is a system property built through engineering discipline rather than a feature added late. A safety case argues from evidence (hazard analysis, simulation, real-world testing, formal verification) to an acceptable-safety conclusion, bounded by the operational design domain. Runtime assurance architectures (Simplex, CBF shields) permit complex learned components while retaining guarantees. Redundancy, graceful degradation, and continuous fault monitoring bound failures. Validation combines scenario-based testing, large-scale simulation, and staged real-world deployment. No single metric captures safety; production deployments track interventions, collisions, near-misses, and coverage together. The safety engineering outlined in this chapter is what distinguishes a product from a demo; programs that succeed at scale do so because of, not despite, their safety practices.

32.13 Exercises

1. Build a hazard entry (like Example 32.1) for a single-LiDAR failure. Include cause, consequence, mitigation, and verification.
2. Specify an ODD for a highway-L3 system. Include thresholds along at least five dimensions.
3. For the CBF example (32.3), derive the QP for the control-constraint projection.
4. Compare the Simplex architecture to purely-formal verification. What are the strengths of each?
5. For the intervention-rate example (32.5), estimate the miles needed to empirically validate the target driverless-rate with 95% confidence.
6. Propose three sensor-health monitors that together detect camera obstruction with high sensitivity.
7. Design a graceful-degradation ladder for a vehicle that loses GNSS, LiDAR, and one forward camera in sequence.
8. For the safety-case example (32.6), identify three additional evidence items you would expect in a regulator submission.
9. A learned planner's output has a 0.01 % collision rate in simulation. Can this system be deployed without a runtime safety monitor? Justify.
10. Compare the safety engineering requirements for an L2 (driver-supervised) and L4 (driverless) deployment of the same underlying AV technology. Identify at least five differences.

Appendix A

Notation Reference

This appendix collects the conventions used throughout the book.

General

Symbol	Meaning
$\mathbb{R}, \mathbb{N}, \mathbb{Z}$	reals, naturals, integers
$a, b, c, \dots$	scalars (italic lowercase)
$\mathbf{x}, \mathbf{y}, \mathbf{u}$	column vectors (bold lowercase)
$\mathbf{A}, \mathbf{B}, \mathbf{K}$	matrices (bold uppercase)
$\mathbf{A}^\top, \mathbf{A}^{-1}$	transpose, inverse
$\det \mathbf{A}, \text{tr } \mathbf{A}$	determinant, trace
$\mathbf{I}_n, \mathbf{0}_n$	identity matrix, zero vector of size n
$\ \mathbf{x}\ _p$	ℓ_p norm of $\mathbf{x}$
$\mathbf{x}^\top \mathbf{A} \mathbf{x}$	quadratic form
$\odot$	Hadamard (elementwise) product

Probability

Symbol	Meaning
$\mathbb{P}(A)$	probability of event A
$p(x)$	probability density function of X at x
$\mathbb{E}[\cdot], \text{Var}[\cdot], \text{Cov}[\cdot, \cdot]$	expectation, variance, covariance
$\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$	Gaussian distribution
$H(p)$	differential entropy
$D_{\text{KL}}(p \ q)$	Kullback–Leibler divergence
$I(X; Y)$	mutual information

Geometry

Symbol	Meaning
$\text{SO}(n)$	special orthogonal group of dimension n
$\text{SE}(n)$	special Euclidean group of dimension n
$\mathfrak{so}(n), \mathfrak{se}(n)$	corresponding Lie algebras
$\mathbf{T}_{AB}$	pose of frame B expressed in frame A
$[\boldsymbol{\omega}]_{\times}$	skew-symmetric matrix for cross product
$\exp, \log$ (capitalized)	Lie group $\exp/\log$ maps
$\text{Ad}_{\mathbf{T}}$	adjoint representation
$\pi(\mathbf{K}, \mathbf{T}, \mathbf{p})$	camera projection

Control and Estimation

Symbol	Meaning
$\mathbf{x}_t, \mathbf{u}_t, \mathbf{y}_t$	state, control, observation at time t
$f(\cdot), h(\cdot)$	dynamics, measurement functions
$\mathbf{F}, \mathbf{H}$	their Jacobians (linearized systems)
$\mathbf{Q}, \mathbf{R}$	process, measurement noise covariances
$\mathbf{P}, \boldsymbol{\Sigma}$	state covariance
$\mathbf{K}$	Kalman gain
$\pi(a s)$	policy probability
$V^{\pi}(s), Q^{\pi}(s, a)$	state-value, action-value functions
$\gamma \in [0, 1)$	discount factor
ρ	system response time (RSS), also occupancy

Machine Learning

Symbol	Meaning
$\boldsymbol{\theta}, \phi$	learnable parameters
$f_{\boldsymbol{\theta}}(\cdot)$	parameterized model
$\mathcal{D}$	dataset
$\ell(\cdot, \cdot)$	per-sample loss
$\mathcal{L}(\boldsymbol{\theta})$	training objective
$\nabla_{\boldsymbol{\theta}} \mathcal{L}$	gradient of loss w.r.t. parameters
η	learning rate
σ	activation function (usually ReLU or GELU)

Acronyms

Acronym	Expansion
ADS	Automated Driving System
ASIL	Automotive Safety Integrity Level
BEV	Bird's-Eye-View
CBF	Control Barrier Function
DDT	Dynamic Driving Task
EKF, UKF	Extended/Unscented Kalman Filter
FMCW	Frequency-Modulated Continuous-Wave (LiDAR, radar)
GNSS	Global Navigation Satellite System
HD map	High-Definition map
ICP	Iterative Closest Point
IMU	Inertial Measurement Unit
IRL	Inverse Reinforcement Learning
LiDAR	Light Detection and Ranging
LQR	Linear Quadratic Regulator
MDP, POMDP	(Partially Observable) Markov Decision Process
MOT	Multi-Object Tracking
MPC	Model Predictive Control
ODD	Operational Design Domain
RSS	Responsibility-Sensitive Safety
SAE	Society of Automotive Engineers
SLAM	Simultaneous Localization and Mapping
SOTIF	Safety Of The Intended Functionality (ISO 21448)
VIO	Visual-Inertial Odometry
V2X	Vehicle-to-Everything communication

Appendix B

Datasets and Benchmarks

This appendix summarizes the public datasets and benchmarks referenced throughout the book.

Perception and Detection

Table B.1: Major perception benchmarks.

Dataset	Modality	Tasks	Scale
KITTI [24]	stereo, LiDAR, GPS/IMU	2D/3D detection, tracking, odometry, stereo, scene flow	7k training frames
nuScenes [9]	6 cam, 5 radar, LiDAR, IMU	3D detection, tracking, prediction, BEV map, lidarseg	1k scenes, 20 s each
Waymo Open [61]	5 cam, 5 LiDAR	3D detection, tracking, prediction, segmentation	1150 scenes, 20 s each
Argoverse 2 [70]	ring cam, stereo, LiDAR	forecasting, 3D detection, map change	1000 scenes
Cityscapes [16]	stereo camera	semantic/instance/panoptic segmentation	5k fine, 20k coarse
BDD100K [74]	camera, GPS	multi-task (det, seg, drivable, lanes, weather)	100k videos
Mapillary Vistas	crowd-sourced cam	urban semantic segmentation	25k images
SemanticKITTI	LiDAR	LiDAR semantic/panoptic segmentation	22 sequences
ApolloScape	LiDAR, cam	lane, trajectory, stereo, 3D detection	multiple tracks
ONCE	LiDAR, 7 cam	detection, self-supervised pre-training	1M frames (unlabeled)

Motion Forecasting and Planning

Table B.2: Prediction and planning benchmarks.

Dataset	Scope	Horizon
Argoverse Motion Forecasting	vehicle trajectories with map	3 s
Waymo Open Motion	multi-agent, map, many classes	8 s future
nuScenes prediction	interactions in urban scenes	6 s
INTERACTION [75]	multi-country interactive sets	variable
nuPlan	closed-loop planning benchmark	simulation
Waymax	closed-loop simulation + datasets	simulation

Major Datasets in Detail

The summary tables compress what are individually substantial research artefacts. This section expands the three datasets that together account for the majority of citations in production AV perception research.

KITTI

KITTI [24] is the founding AV-perception benchmark. Released in 2012 as a joint effort between the Karlsruhe Institute of Technology and the Toyota Technological Institute at Chicago, by Andreas Geiger, Philip Lenz, Christoph Stiller, and Raquel Urtasun, KITTI defined the public-benchmark template that every subsequent AV dataset has followed: a calibrated multi-sensor data-collection vehicle, hand-annotated ground truth, public train/test splits, and a hosted online leaderboard for held-out test scoring.

The KITTI sensor suite consisted of two pairs of cameras (one colour and one greyscale, all 1392×512 pixels at 10 Hz), a Velodyne HDL-64E LiDAR (64-channel mechanical-spinning, 360° , 10 Hz), and a high-precision OXTS RT 3003 GPS/IMU. The sensor rig was mounted atop a Volkswagen Passat station wagon driven through Karlsruhe and its surroundings during 2011. The total recorded driving was modest by contemporary standards—several hours of data—but the released benchmark splits remain the reference for reproducibility comparisons.

KITTI’s task taxonomy was unusually broad for its time and remains influential. The benchmarks include stereo matching, optical flow, visual odometry, monocular and stereo 3D object detection, 2D/3D multi-object tracking, road and lane detection, and (added in SemanticKITTI [1]) point-wise semantic and panoptic segmentation of LiDAR. The KITTI-360 follow-up dataset extended the original with denser annotations, 360° camera coverage, and full semantic segmentation.

KITTI’s principal limitation today is scale: the labelled training set is roughly 7 500 frames, two to three orders of magnitude smaller than modern equivalents. Its weather, lighting, and geographic diversity are also constrained: Karlsruhe in summer daylight. As a consequence, KITTI numbers are no longer competitive evidence of state-of-the-art performance. Its enduring value is pedagogical and

reproducibility- oriented: implementations and ablations are routinely reported on KITTI to allow direct comparison against the hundreds of papers that preceded.

Waymo Open Dataset

The Waymo Open Dataset [61], released by Waymo LLC (an Alphabet subsidiary) starting in 2019 and expanded in successive releases through 2024, is the dominant large-scale benchmark for current AV perception and prediction research. The original release contained 1 150 scenes of 20 seconds each, totaling roughly 6.4 hours of driving. The dataset uses Waymo’s own production-class sensor suite: five LiDARs (one mid-range top-mounted plus four short-range units mounted around the vehicle perimeter) and five cameras (four side-facing plus one forward-facing), all time-synchronised. The recorded driving spans multiple cities including Phoenix, San Francisco, and Mountain View, and includes both daytime and night-time conditions and a range of urban contexts.

Annotations include 3D bounding boxes for vehicles, pedestrians, cyclists, and signs, with track-level identity for tracking evaluation. The labelling rate is high: hundreds of millions of 3D box labels across the dataset.

Beyond the perception dataset, Waymo released the **Waymo Open Motion Dataset (WOMD)** for trajectory prediction. WOMD contains over 100 000 segments of 20 seconds each with associated map data, totaling hundreds of thousands of unique agent trajectories. The released challenges have included motion prediction (predicting up to 8 seconds of future trajectory for surrounding agents), interaction prediction, sim-agents (controlling background agents in a closed-loop simulation), and most recently end-to-end driving from sensor input. The WOMD challenges held annually at the CVPR Workshop on Autonomous Driving have produced much of the published state of the art in recent years. The Waymax [26] closed-loop simulator complements WOMD with reactive simulation environments.

The dataset is distributed under a non-commercial research license; researchers must agree to terms before download. The combination of production-grade sensor calibration, large scale, geographic and temporal diversity, and continued investment from Waymo has made it the de facto reference dataset for current AV-perception publication.

nuScenes

nuScenes [9] was released in 2019 by nuTonomy (now Motional, the joint venture between Aptiv and Hyundai Motor Group) and was the first large-scale public dataset to combine multi-camera, multi-radar, and LiDAR sensing, making it the principal benchmark for multi-modal-fusion 3D detection research.

The dataset contains 1 000 scenes of 20 seconds each, totaling roughly 5.5 hours of driving across two cities: Boston (Seaport area) and Singapore (One-North area). The choice of these two cities gave the dataset deliberate left-hand-and-right-hand traffic exposure and distinctly different urban morphologies. The sensor suite consisted of six cameras providing full 360° coverage at 12 Hz, five 77 GHz radars (one front, two side-front, two side-rear), and one roof-mounted Velodyne HDL-32E

LiDAR at 20 Hz. A high-precision GNSS+IMU provided ground-truth pose. The dataset’s roughly 1.4 million annotated 3D bounding boxes cover 23 classes (vehicles, pedestrians, cyclists, animals, etc.) with per-frame visibility and pose attributes, plus an additional lidarseg release adding point-wise semantic-segmentation labels.

nuScenes has been particularly important for radar research, which is under-represented in KITTI and the early Waymo releases. The dataset launched the nuScenes Detection Score (NDS), a composite metric that combines mean Average Precision with translation, scale, orientation, velocity, and attribute errors; NDS has become a standard summary metric for 3D detection. The follow-up **nuPlan** dataset [8] provides a planning benchmark with 1 300 hours of expert driving, designed for closed-loop planning evaluation rather than open-loop trajectory regression.

Comparison and Selection Guidance

Dataset	Strength	Use it for
KITTI	reproducibility against historical literature	teaching, ablations, baseline replication
Waymo Open	scale, sensor diversity, current SOTA leaderboards	competitive benchmarking, motion prediction, end-to-end
nuScenes	multi-modal radar+camera+LiDAR fusion	3D detection with radar, cross-region generalisation

A research paper today routinely reports on at least two of the three to demonstrate generalisation across sensor configurations and geographies. Production stacks supplement public datasets with much larger internal fleet collections that public researchers do not have access to.

Simulators

- **CARLA**: Unreal-engine based urban simulator; widely used for end-to-end and perception research.
- **LGSVL / SVL Simulator**: Unity-based; strong vehicle dynamics.
- **SUMO**: microscopic traffic simulation at city scale.
- **highway-env**: light-weight Python environments for RL research.
- **MetaDrive**: procedurally-generated driving scenarios for RL.
- **CarMaker, dSPACE**: commercial simulators with certification-grade vehicle dynamics.

Open-Source Software

- **ROS / ROS 2**: messaging and middleware.
- **Autoware, Apollo**: open-source full-stack ADS platforms.
- **GTSAM, g2o, Ceres**: factor-graph and nonlinear least-squares solvers.
- **Open3D, PCL**: point-cloud processing.
- **MMDetection3D, OpenPCDet, Det3D**: 3D detection codebases.
- **trajectron++, scenarionet**: forecasting libraries.

Evaluation and Ethics Resources

- NHTSA Automated Driving Systems: Voluntary Safety Self-Assessment disclosures.
- UNECE WP.29 regulatory documents.
- California DMV autonomous vehicle disengagement and collision reports.
- Incident and crash databases (e.g., CA DMV, Waymo / Cruise transparency releases).

Accessing and citing these resources correctly—and honestly reporting stratified metrics over them—is itself a part of responsible ADS research practice.

Appendix C

Linear Algebra and Numerical Methods Refresher

This appendix collects the linear algebra identities and numerical methods used in the main chapters.

C.1 Vector Spaces and Norms

For $\mathbf{x} \in \mathbb{R}^n$,

$$\|\mathbf{x}\|_1 = \sum_i |x_i|, \quad \|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^\top \mathbf{x}}, \quad \|\mathbf{x}\|_\infty = \max_i |x_i|. \quad (\text{C.1})$$

The induced matrix norm is $\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A})$, the largest singular value.

C.2 Eigen- and Singular-Value Decompositions

EVD. For diagonalizable $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{-1}$ with $\mathbf{\Lambda}$ diagonal. Symmetric $\mathbf{A}$ admits orthogonal $\mathbf{Q}$ and real $\mathbf{\Lambda}$.

SVD. Any $\mathbf{A} \in \mathbb{R}^{m \times n}$ admits $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ with $\mathbf{U} \in \mathbb{R}^{m \times m}$, $\mathbf{V} \in \mathbb{R}^{n \times n}$ orthogonal and $\mathbf{\Sigma}$ non-negative diagonal. Key consequences:

- $\text{rank}(\mathbf{A}) = \#\{\sigma_i > 0\}$.
- $\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^\top$ is the Moore–Penrose pseudoinverse.
- $\min_{\text{rank}(\mathbf{B}) \leq k} \|\mathbf{A} - \mathbf{B}\|_F^2$ is achieved by truncating to the top- k singular components.

C.3 Matrix Identities

Woodbury (“matrix inversion lemma”).

$$(\mathbf{A} + \mathbf{UCV})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}(\mathbf{C}^{-1} + \mathbf{VA}^{-1}\mathbf{U})^{-1}\mathbf{VA}^{-1}. \quad (\text{C.2})$$

Essential in Kalman filter reformulations.

Schur complement. For $\mathbf{M} = \begin{pmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{C} & \mathbf{D} \end{pmatrix}$, the Schur complement of $\mathbf{A}$ (assumed invertible) is $\mathbf{S} = \mathbf{D} - \mathbf{C}\mathbf{A}^{-1}\mathbf{B}$. Then $\det \mathbf{M} = \det \mathbf{A} \cdot \det \mathbf{S}$, and $\mathbf{M} \succ 0 \iff \mathbf{A} \succ 0, \mathbf{S} \succ 0$.

Matrix calculus (a few).

$$\frac{\partial}{\partial \mathbf{A}} \text{tr}(\mathbf{A}\mathbf{B}) = \mathbf{B}^\top, \quad \frac{\partial}{\partial \mathbf{A}} \log \det \mathbf{A} = \mathbf{A}^{-\top}, \quad \frac{\partial}{\partial \mathbf{x}} \mathbf{x}^\top \mathbf{A} \mathbf{x} = (\mathbf{A} + \mathbf{A}^\top) \mathbf{x}. \quad (\text{C.3})$$

C.4 Positive (Semi)definite Matrices

$\mathbf{A} \succeq 0$ iff $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0 \forall \mathbf{x}$. Equivalent conditions include: all eigenvalues non-negative; a Cholesky factor $\mathbf{L}$ exists with $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$; all leading principal minors non-negative.

Covariance matrices are symmetric PSD. Many estimators assume strict positive definiteness; numerical conditioning (e.g., Joseph-form Kalman updates) prevents accumulation of non-PD error.

C.5 Solving Linear Systems

Direct methods.

- Cholesky for symmetric positive-definite $\mathbf{A}$: fastest, most numerically stable.
- LU with partial pivoting for general square $\mathbf{A}$.
- QR for least squares and ill-conditioned systems.

Iterative methods. Conjugate gradient (CG) for SPD systems, GMRES for general; preconditioning is essential for convergence on large systems.

Sparse solvers. SLAM and bundle adjustment rely on sparsity-exploiting sparse Cholesky (CHOLMOD), sparse QR (SPQR), and block-sparse formulations from domain-specific libraries (Ceres, GTSAM).

C.6 Numerical Stability

- Condition number $\kappa(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$ bounds the amplification of input error in $\mathbf{A}\mathbf{x} = \mathbf{b}$.
- Never explicitly form $\mathbf{A}^\top \mathbf{A}$ if you can avoid it: the condition number squares.
- Use the Joseph form of the Kalman covariance update $\mathbf{P}_{t+1} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}_{t+1}^- (\mathbf{I} - \mathbf{K}\mathbf{H})^\top + \mathbf{K}\mathbf{R}\mathbf{K}^\top$ to preserve symmetry and positive-definiteness.

C.7 Differentiation on Lie Groups

The composition rule for small perturbations on $\text{SE}(3)$ is $\mathbf{T} \leftarrow \mathbf{T} \exp(\hat{\boldsymbol{\xi}})$ for $\boldsymbol{\xi} \in \mathbb{R}^6$. Jacobians of residuals with respect to $\boldsymbol{\xi}$ replace classical multivariate Jacobians; their structure underpins the sparse SLAM Jacobians of Chapter 13.

C.8 Random Matrix and Covariance Propagation

Linear propagation. $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b}$ implies $\boldsymbol{\mu}_y = \mathbf{A}\boldsymbol{\mu}_x + \mathbf{b}$, $\boldsymbol{\Sigma}_y = \mathbf{A}\boldsymbol{\Sigma}_x\mathbf{A}^\top$.

Unscented transform. For nonlinear $\mathbf{y} = g(\mathbf{x})$, sample sigma points around $\boldsymbol{\mu}_x$ with covariance $\boldsymbol{\Sigma}_x$, evaluate g , reweight mean and covariance. Avoids Jacobians and is accurate to second order.

C.9 Graph Laplacians and Spectral Tools

Graph Laplacians appear in GNN-based prediction, pose-graph optimization, and consistent tracker initialization. $\mathbf{L} = \mathbf{D} - \mathbf{A}$ for adjacency $\mathbf{A}$ and degree $\mathbf{D}$; $\mathbf{L} \succeq 0$; the multiplicity of the zero eigenvalue is the number of connected components.

Appendix D

Solutions to Selected Exercises

This appendix provides worked solutions to selected exercises from each chapter. Because many exercises invite open-ended design or discussion answers for which there is no single “right” response, the focus here is on quantitative problems whose solutions are unambiguous, and on the conceptual problems where a representative answer is most pedagogically useful. Readers are encouraged to attempt the exercises before consulting these solutions; the working out matters more than the final number.

Part I — Foundations

Chapter 1: Introduction to Autonomous Vehicles

Ex. 1. *SAE Level definitions.* Level 0 is no automation; the driver does everything. Level 1 provides single-axis assistance (adaptive cruise control *or* lane-keeping). Level 2 combines lateral and longitudinal control but requires constant driver supervision. Level 3 takes over the dynamic driving task within an ODD; the driver is a fallback. Level 4 operates without fallback within its ODD. Level 5 is unbounded.

Ex. 5. *ODD specification example.* A reasonable ODD for a downtown robotaxi: weekday and weekend operation, dry weather only (no rain, snow, or fog with visibility below 100 m), daytime only, mapped service area of roughly 60 km², speed limit ≤ 50 km/h, no construction-zone routes, ambient temperature 0–40°C. The narrower the ODD, the faster validation; the wider, the greater commercial value.

Ex. 9. *Long-tail event probabilities.* An event with frequency 10^{-8} per mile cannot be validated by 10 M test miles (expected 0.1 occurrences); 1 B miles is required for 1σ -meaningful coverage. This is why simulation, falsification, and importance sampling are indispensable.

Chapter 2: Mathematical and Machine Learning Foundations

Ex. 1. (SE(3) composition) Given $\mathbf{T}_1 = (\mathbf{R}_1, \mathbf{t}_1)$ and $\mathbf{T}_2 = (\mathbf{R}_2, \mathbf{t}_2)$: $\mathbf{T}_1 \circ \mathbf{T}_2 = (\mathbf{R}_1 \mathbf{R}_2, \mathbf{R}_1 \mathbf{t}_2 + \mathbf{t}_1)$. Inverse: $\mathbf{T}^{-1} = (\mathbf{R}^\top, -\mathbf{R}^\top \mathbf{t})$.

Ex. 4. *Rodrigues lateral error.* For yaw θ at range r , lateral error $= r \sin \theta$. At $r = 30$ m and $\theta = 1^\circ = 0.01745$ rad, error $= 30 \times 0.01745 = 0.524$ m. Quote: “per degree of yaw error, half a meter of lateral error at 30 m.”

Ex. 7. *Quaternion-vs-matrix storage.* 4 floats vs 9 floats per attitude. For 1000 keyframes, 4 kB vs 9 kB—negligible. The real benefit is normalization stability: a quaternion has one constraint ($\|q\| = 1$); a rotation matrix has six (three orthogonality + three unit-norm), and small numerical errors compound destructively.

Ex. 12. (RANSAC iterations) $N = \log(1 - p) / \log(1 - w^s)$. For $p = 0.99$, $s = 8$, $w = 0.5$: $N = \log(0.01) / \log(1 - 0.0039) \approx 1177$. For $w = 0.7$: $N = \log(0.01) / \log(1 - 0.0576) \approx 78$. Outlier rate strongly drives compute.

Ex. 18. (Adam vs SGD on saddle points) Adam’s per-parameter adaptive learning rate handles ill-conditioned loss landscapes better than SGD; saddle points are exited faster. The trade-off: Adam can converge to slightly different (sometimes worse-generalizing) minima than SGD with momentum.

Ex. 22. *Conv FLOPs.* $H \cdot W \cdot C_{\text{out}} \cdot C_{\text{in}} \cdot k^2$. At $H = W = 512$, $C = 256$, $k = 3$: $512 \cdot 512 \cdot 256 \cdot 256 \cdot 9 \approx 1.55 \times 10^{11}$ FLOPs.

Part II — Perception**Chapter 3: Autonomous Perception**

Ex. 2. *Latency at 4K, 60Hz.* $3840 \times 2160 = 8.3$ Mpx, at 60 Hz = 500 Mpx/s. Versus 30 Hz HD (60 Mpx/s): $8 \times$ the data rate. Compute scales linearly; bandwidth budget often binds first.

Ex. 5. (Perception delay) At $v = 30$ m/s and $\Delta t = 50$ ms, $\Delta d = 1.5$ m. At $\Delta t = 100$ ms, $\Delta d = 3.0$ m. Latency improvements pay for themselves disproportionately.

Chapter 4: Deep Vision

Ex. 2. (Depthwise-separable savings) ratio $= (k^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}) / (k^2 C_{\text{in}} C_{\text{out}}) \approx 1/C_{\text{out}} + 1/k^2$. For $C_{\text{out}} = 256$, $k = 3$: ratio ≈ 0.115 , an $8.7 \times$ saving.

Ex. 6. (Swin vs full attention) Full: $N^2 d$. Swin: $W^2 N d$ with N tokens, W window size. At $N = 16384$, $W = 7$: ratio $= N/W^2 \approx 334 \times$ savings.

Ex. 9. (TTC at 30 m/s closing) $T = d/v$. At 30 m/s and $d = 25$ m: $T \approx 0.83$ s. Tight; emergency-brake response only.

Part III — Localization & Mapping

Chapter 7: SLAM

Ex. 3. *Loop closure benefit.* Without loop closure, drift grows unboundedly. With loop closure, drift is bounded by accumulated graph residual after global optimization, typically $< 1\%$ of trajectory length on standard datasets.

Ex. 7. (EKF vs particle filter) EKF: Gaussian posterior, computationally cheap, fails on multi-modal posteriors. PF: arbitrary posteriors, expensive, works in non-Gaussian cases. EKF fits SLAM with single dominant mode; PF needed for global localization.

Part IV — Prediction & Planning

Chapter 9: Prediction

Ex. 2. (NLL of correct mode at $p = 0.50$) $\text{NLL} = -\log 0.50 \approx 0.693$. (Uniform 6-mode baseline: $-\log(1/6) \approx 1.79$.)

Ex. 5. (FDE at 8s) For roughly linear-in-time error growth at ~ 0.5 m/s typical, FDE at 8s ≈ 4 m for a tuned predictor; double for an untuned one.

Chapter 10: Planning

Ex. 3. *Comfort-bound deceleration.* With $j_{\max} = 1.5$ m/s³ and $a_{\max} = -3$ m/s², ramping up takes 2 s, then full braking, then ramp-down. From 20 m/s to stop takes about 6.7 s and 67 m of distance.

Part V — Control & End-to-End

Chapter 12: Control

Ex. 1. (PID tuning rule of thumb) Ziegler–Nichols: increase K_p until oscillation at period T_u and gain K_u ; then $K_p = 0.6K_u$, $T_i = 0.5T_u$, $T_d = 0.125T_u$. Modern systems use PID-tuned-by-MPC-rollout for richer dynamics.

Ex. 4. (LQR cost) $J = \int (\mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{u}^\top \mathbf{R} \mathbf{u}) dt$. Larger $\mathbf{Q}$: tracking accuracy weighted heavier; larger $\mathbf{R}$: control effort weighted heavier. Trade-off determines responsiveness.

Part VI — Safety & Ethics

Chapter 14: Safety

Ex. 4. (RSS lateral safe distance) RSS lateral safe distance for two vehicles at lateral velocities v_1, v_2 , response times ρ_1, ρ_2 , max accelerations $a_{1\max}, a_{2\max}$ and min decelerations b_1, b_2 : $d_{\text{safe}}^{\text{lat}} = (v_1\rho_1 + \frac{1}{2}a_{1\max}\rho_1^2) + \frac{(v_1+a_{1\max}\rho_1)^2}{2b_1} + \text{symmetric terms for vehicle 2}$. Plug in worst-case numbers; commonly yields ~ 0.5 m for moderate-speed urban.

Ex. 7. (Trolley-problem framing) The trolley problem captures one rare aspect of decision-making under unavoidable harm but is not representative of typical AV decisions, which are about *avoiding* harm rather than *distributing* it. Production ethics frameworks emphasize risk-minimization across all road users uniformly rather than deontological hierarchies.

Part VII — Infrastructure, Cybersecurity, Legal

Chapter 17: Cybersecurity

Ex. 2. (Adversarial perturbation budget) $\epsilon = 8/255 \approx 3.1\%$ of pixel range; imperceptible to humans but sufficient to flip stop-sign classifications in unprotected models. Adversarial training reduces but does not eliminate susceptibility.

Ex. 6. (V2X authentication) Each message signed with sender's private key; receiver verifies with sender's public key from the certificate hierarchy. Pseudonym certificates rotate every ~ 5 minutes for privacy.

Chapter 18: Legal

Ex. 3. *Liability allocation.* Currently a developing area. Manufacturer typically liable when system was engaged; operator liable when in manual mode. Specific allocations depend on jurisdiction and ODD compliance at time of incident.

Part X — Economics

Chapter 25: AV Economics

Ex. 1. (Cost-decline projection) LiDAR cost has declined $\sim 30\%$ /year since 2016. Compounded: $C_t = C_0 \cdot 0.7^t$. At $t = 10$ years, $C_{10}/C_0 \approx 0.028$. A \$80,000 unit becomes \$2,200; observed prices match this trajectory.

Ex. 3. (Utilization sensitivity) At 200k miles: capital CPM = \$ 80k/200k mi = 40 cents. At 500k: 16 cents. At 1M: 8 cents. Hyperbolic decline; flattens above 700k.

Chapter 26: E-hailing

Ex. 2. (Demand elasticity) With elasticity -1.2 : 50% fare cut increases ridership by $\sim 60\%$, total revenue rises ($1.5 \times 0.5 = 0.75$ vs baseline 1.0—so revenue *falls* 25% per ride.) Volume gain doesn't offset price cut at this elasticity; pricing should not undercut by 50%.

Part XI — Motion Sensing**Chapter 27: Wheel Odometry**

Ex. 4. *Wheel slip and odometry error.* Slip ratio $s = (r\omega - v)/v$. Even small slip (5%) accumulates: 1 km of travel produces 50 m of error if uncorrected. Vision/LiDAR/GNSS fusion is required.

Part XII — Specialized Domains**Chapter 30: Lane Detection**

Ex. 1. *Lane-line accuracy requirement.* For a vehicle at 1.8 m width in a 3.5 m lane, lateral freedom is 0.85 m per side. Detection accuracy of 0.05 m (= 5 % of half-lane) is comfortable; 0.20 m or worse begins to compromise lane-centering.

Chapter 31: Rural Dust

Ex. 3. *Dust scattering coefficient.* Mie scattering theory: attenuation $\alpha \propto N \cdot \pi r^2$, with N particle density and r effective radius. Dust at $10^9/\text{m}^3$ with $r = 5 \mu\text{m}$: $\alpha \approx 0.08/\text{m}$, giving ~ 20 m visibility. Heavy dust easily reduces sensing range to 5–10 m.

Part XIII — Advanced Topics**Chapter 32: Digital Twins**

Ex. 1. (4K dense depth points/sec) $3840 \times 2160 \times 60 = 4.98 \times 10^8$ points/s, roughly $13 \times$ HD/30 Hz.

Ex. 3. *Voxel grid memory at finer resolution.* $400 \times 400 \times 32 = 5.12 \times 10^6$ voxels, at 5 bytes each: 25.6 MB/frame. $8 \times$ bigger than the example.

Ex. 4. (PQ at TP=15) Given FP=2, FN=3, IoU sum scaled to 12 (mean IoU 0.80): $\text{PQ} = 12/(15 + 1 + 1.5) = 12/17.5 \approx 0.686$. Higher recall, slightly lower fraction-correct. Higher PQ overall.

Chapter 33: Advanced Prediction

Ex. 1. (NLL) At $p = 0.50$: $\text{NLL} = -\log 0.5 \approx 0.693$. Half the NLL of the 0.25-probability case in the worked example.

Ex. 5. (Risk computation) $R = P_{\text{hit}} \cdot C_{\text{severity}} = 0.01 \cdot 0.8 = 0.008$.

Chapter 34: Uncertainty, OOD, XAI

Ex. 5. (OOD detection at 1 % FPR) With $z = 2.33$ for 1 % FPR (one-sided), $\tau = -2.3 + 2.33 \cdot 0.4 = -1.37$. $P(\text{OOD energy} > -1.37) = 1 - \Phi(-2.41) \approx 0.992$. 99.2 % detection at 1 % FPR.

Ex. 9. (Dual-pipeline reliability with correlation) With per-pipeline failure $p = 0.01$ and correlation $\rho = 0.5$: joint failure probability $\approx p^2 + \rho \cdot p(1 - p) \approx 0.005$. Five-nines reliability instead of six-nines—still better than single pipeline but degraded by correlation.

Chapter 35: Frontier Technologies

Ex. 5. (Sim-to-real at 25/75 mix) Linear interpolation suggests real mAP ≈ 0.83 . Empirically the curve is non-monotonic; the actual value would need an experiment.

Part XIV — Trust, Testing, Engineering

Chapter 38: OTA Lifecycle Management

Ex. 2. *Rollback decision metric.* Rollback if any of: incident rate elevates by $>50\%$ over baseline, intervention rate $>30\%$ over baseline, or any safety-critical regression detected. Conservative bias toward rollback because re-deployment is cheap relative to incident cost.

Chapter 39: Trust and Assurance

Ex. 5. (Falsification ROI) \$200k campaign cost, prevents 1 expected incident worth \$5M: ROI = $(\$5\text{M} - \$200\text{k})/\$200\text{k} = 24\times$.

Chapter 40: Advanced Testing

Ex. 3. *Importance sampling for $P_f = 10^{-7}$.* Plain MC for $\pm 10\%$ accuracy: $\sim 10/P_f = 10^8$ samples. With $1000\times$ bias: 10^5 samples. $1000\times$ speedup.

Chapter 41: Compute Systems

Ex. 1. *Utilization for 50 GFLOPs at 30 Hz on 250 TOPS hardware.* Required: $50 \times 10^9 \times 30 = 1.5 \times 10^{12}$ FLOPS. Hardware: 2.5×10^{14} FLOPS. Utilization: 0.6%. Achievable with modest engineering effort.

Chapter 42: Distributed AI

Ex. 1. (Cloud-assisted latency at 50 ms each way) $30 + 50 + 80 + 50 = 210$ ms. Better than 310 ms from worked example; still not safety-critical. Useful for advisory.

Chapter 50: Risk Micro-Modeling

Ex. 2. (Near-collision at 20 m headway) Linear interpolation between worked example points: $P_{NC}(15\text{ m}) = 0.058$; $P_{NC}(25\text{ m}) = 0.005$. $P_{NC}(20\text{ m}) \approx 0.020$. Below 5 %, marginal.

Part XV — Environment, Sensing, Infrastructure**Chapter 43: Bio-Inspired Sensing**

Ex. 1. *Wet-road polarization with measurement noise* $\sigma = 0.05$. Threshold 0.10. Probability of misclassification: $P(\text{false dry}|\text{wet}) = \Phi((0.10 - 0.20)/0.05) = \Phi(-2) \approx 0.023$. $P(\text{false wet}|\text{dry}) = \Phi((0.05 - 0.10)/0.05) = \Phi(-1) \approx 0.16$. Asymmetric error rates suggest a context-dependent threshold.

Chapter 44: Environmental Extremes

Ex. 2. (200-km route, three wildlife zones) 3×6 minutes = 18 minutes added. Acceptable for a 200 km route taking ~ 3 hours.

Chapter 45: Infrastructure

Ex. 1. (PaaS at 8 % AV penetration) Same intersection: 50,000 transits, 8 % AV = 4,000 AV transits/day. Annual revenue at 5 cents: $4000 \times 365 \times 0.05 = \$73,000$. Break-even pushes from 2 years to ~ 3.7 years. Marginal at 8 %; clearly profitable above 12 %.

Part XVI — Extended Applications and Society**Chapter 46: Ontologies**

Ex. 1. (VRU ontology hierarchy) **VulnerableRoadUser** $\sqsubseteq$ RoadUser, with subclasses Pedestrian, Cyclist, Motorcyclist, Wheelchair, ChildPedestrian, ElderlyPedestrian. Disjointness axioms between Vehicle and VRU classes prevent overlap.

Chapter 47: Electric Vehicles

Ex. 1. (Range at SoC=0.15) $E = 0.15 \times 77 = 11.55$ kWh. At 220 Wh/km: $R = 11550/220 = 52.5$ km. Tight; need to charge soon.

Ex. 2. (Break-even electricity price) Set EV cost/km = ICE cost/km. EV at 0.180 kWh/km, ICE at 11.7 cents/mi ≈ 7.27 cents/km. Break-even: $0.180 \times p_e = 7.27$, $p_e \approx 40$ cents/kWh. EV economic at any electricity price below 40 cents/kWh (fast-charging premium pricing approaches but rarely exceeds this).

Chapter 48: Climate Change

Ex. 1. (Break-even with cleaner grid) Clean grid gives 80 gCO₂e/km vs ICE 200: $d_{BE} = 3000 / (0.120) = 25,000$ km. Faster payback than mixed grid.

Ex. 2. (Per-pax-km at 2-passenger occupancy) Halve the 88 gCO₂e from worked example: 44 gCO₂e/pax-km. Ride-pooling has dramatic per-passenger emissions benefits.

Chapter 49: Quantum Technologies

Ex. 1. (100-hour drift comparison) Classical: $0.01 \times 100 = 1$ deg. Quantum: $10^{-5} \times 100 = 10^{-3}$ deg. 1000× improvement preserved.

Chapter 51: Labor Economics

Ex. 1. *Driver displacement timeline.* A 10-year, 50 % deployment of L4 robotaxis displaces roughly 50 % of urban-driving jobs in mature deployments. Total US affected: ~1.5–2 M jobs. Displacement is gradual; transition policy buffers most cohorts.

Chapter 53: Insurance

Ex. 2. *Premium calculation for L4 robotaxi.* Expected loss-cost per mile = incident-rate $\times$ average-claim. At $0.05/\text{M mi} \times \$50\text{k}/\text{claim} = \$2.50/\text{M mi}$. Loaded $\sim 3\times$ for expense and profit margin: $\$7.50/\text{M mi}$. Annual premium for 100k-mi vehicle: $\$750$. Much lower than human-driver fleet average of $\$1500\text{--}3000$.

Chapter 60: Socio-Technical

Ex. 3. (Reactive vs predictive handoff at 12 m/s) Reactive: $600 \text{ ms} \times 12 \text{ m/s} = 7.2 \text{ m}$. Predictive: $150 \text{ ms} \times 12 \text{ m/s} = 1.8 \text{ m}$. $\Delta = 5.4 \text{ m}$. Smaller absolute saving than at highway speed but proportionally similar.

Note on Open-Ended Exercises

Many exercises in this book invite design, discussion, or extension answers without a single “correct” response. For those, instructors and self-study readers should look for:

- **Engineering plausibility:** numbers within reasonable bounds; assumptions stated; trade-offs identified.
- **Use of chapter material:** solution explicitly draws on the chapter’s concepts rather than evading them.
- **Clear reasoning chain:** each step justified; conclusions supported by the working out.
- **Recognition of limits:** where the answer is uncertain or context-dependent, the solution says so.

Worked examples in the chapter bodies are themselves model solutions; readers should refer to the example most analogous to a given exercise as a template. Exercises whose answers are not provided

here are deliberately left open; their value lies in the engagement they provoke rather than in any single “right” answer.

Bibliography

- [1] Jens Behley, Martin Garbade, Andres Milioto, Jan Quenzel, Sven Behnke, Cyrill Stachniss, and Juergen Gall. SemanticKITTI: A dataset for semantic scene understanding of LiDAR sequences. In *ICCV*, 2019.
- [2] Paul J. Besl and Neil D. McKay. A method for registration of 3-d shapes. *IEEE TPAMI*, 1992.
- [3] Alex Bewley et al. Simple online and realtime tracking. In *ICIP*, 2016.
- [4] Peter Biber and Wolfgang Straßer. The normal distributions transform. In *IROS*, 2003.
- [5] Navaneeth Bodla et al. Soft-NMS: Improving object detection with one line of code. In *ICCV*, 2017.
- [6] Mariusz Bojarski, Philip Yeres, Anna Choromanska, Krzysztof Choromanski, Bernhard Firner, Lawrence Jackel, and Urs Muller. VisualBackProp: Efficient visualization of CNNs. In *ICRA*, 2018.
- [7] Mariusz Bojarski et al. End to end learning for self-driving cars, 2016. arXiv:1604.07316.
- [8] Holger Caesar, Juraj Kabzan, Kok Seang Tan, Whye Kit Fong, Eric Wolff, Alex Lang, Luke Fletcher, Oscar Beijbom, and Sammy Omari. nuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles. *arXiv preprint arXiv:2106.11810*, 2021.
- [9] Holger Caesar et al. nuScenes: A multimodal dataset for autonomous driving. In *CVPR*, 2020.
- [10] Nicolas Carion et al. End-to-end object detection with transformers. In *ECCV*, 2020.
- [11] Yuning Chai, Benjamin Sapp, Mayank Bansal, and Dragomir Anguelov. MultiPath: Multiple probabilistic anchor trajectory hypotheses for behavior prediction. In *CoRL*, 2020.
- [12] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *ECCV*, 2018.
- [13] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *ICML*, 2020.

- [14] Bowen Cheng et al. Masked-attention Mask Transformer for universal image segmentation. In *CVPR*, 2022.
- [15] Felipe Codevilla, Matthias Müller, Antonio López, Vladlen Koltun, and Alexey Dosovitskiy. End-to-end driving via conditional imitation learning. In *ICRA*, 2018.
- [16] Marius Cordts et al. The Cityscapes dataset for semantic urban scene understanding. In *CVPR*, 2016.
- [17] Ernst D. Dickmanns. Dynamic vision for perception and control of motion. *Springer*, 2007.
- [18] Alexey Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.
- [19] David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. *NeurIPS*, 2014.
- [20] Jakob Engel, Vladlen Koltun, and Daniel Cremers. Direct sparse odometry. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(3):611–625, 2018.
- [21] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *KDD*, 1996.
- [22] Christian Forster et al. On-manifold preintegration for real-time visual-inertial odometry. *IEEE T-RO*, 2017.
- [23] Jiyang Gao et al. VectorNet: Encoding hd maps and agent dynamics from vectorized representation. In *CVPR*, 2020.
- [24] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the KITTI vision benchmark suite. In *CVPR*, 2012.
- [25] Clément Godard et al. Digging into self-supervised monocular depth estimation. In *ICCV*, 2019.
- [26] Cole Gulino, Justin Fu, Wenjie Luo, George Tucker, Eli Bronstein, Yiren Lu, Jean Harb, Xinlei Pan, Yan Wang, Xiangyu Chen, John D. Co-Reyes, Rishabh Agarwal, Rebecca Roelofs, Yao Lu, Nico Montali, Paul Mouglin, Zoey Yang, Brandyn White, Aleksandra Faust, Rowan McAllister, Dragomir Anguelov, and Benjamin Sapp. Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research. In *NeurIPS*, 2023.
- [27] Tuomas Haarnoja et al. Soft Actor-Critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *ICML*, 2018.
- [28] Kaiming He, Xinlei Chen, Saining Xie, et al. Masked autoencoders are scalable vision learners. In *CVPR*, 2022.
- [29] Kaiming He et al. Deep residual learning for image recognition. In *CVPR*, 2016.
- [30] Kaiming He et al. Mask R-CNN. In *ICCV*, 2017.

- [31] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- [32] Andrew G. Howard et al. MobileNets: Efficient convolutional neural networks for mobile vision applications, 2017. arXiv:1704.04861.
- [33] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.
- [34] Michael Kaess et al. iSAM2: Incremental smoothing and mapping using the Bayes tree. *IJRR*, 2012.
- [35] Nidhi Kalra and Susan M. Paddock. Driving to safety: How many miles of driving would it take to demonstrate autonomous vehicle reliability? *Transportation Research Part A*, 94:182–193, 2016.
- [36] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *CVPR*, 2018.
- [37] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *ICLR*, 2015.
- [38] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. In *ICCV*, 2023.
- [39] Alexander Kirillov et al. Panoptic segmentation. In *CVPR*, 2019.
- [40] Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton. ImageNet classification with deep convolutional neural networks. In *NeurIPS*, 2012.
- [41] Alex H. Lang et al. PointPillars: Fast encoders for object detection from point clouds. In *CVPR*, 2019.
- [42] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.
- [43] Zhiqi Li, Wenhai Wang, Hongyang Li, Enze Xie, Chonghao Sima, Tong Lu, Qiao Yu, and Jifeng Dai. BEVFormer: Learning bird’s-eye-view representation from multi-camera images via spatiotemporal transformers. In *ECCV*, 2022.
- [44] Zhiqi Li et al. BEVFormer: Learning bird’s-eye-view representation from multi-camera images via spatiotemporal transformers. In *ECCV*, 2022.
- [45] Tsung-Yi Lin et al. Focal loss for dense object detection. In *ICCV*, 2017.
- [46] Ze Liu et al. Swin Transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021.
- [47] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015.

- [48] Jonathon Luiten et al. HOTA: A higher order metric for evaluating multi-object tracking. *IJCV*, 2021.
- [49] Volodymyr Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518, 2015.
- [50] Raúl Mur-Artal, J. M. M. Montiel, and Juan D. Tardós. ORB-SLAM: A versatile and accurate monocular slam system. *IEEE T-RO*, 2015.
- [51] Jiquan Ngiam, Benjamin Caine, Vijay Vasudevan, Zhengdong Zhang, Hao-Tien Lewis Chiang, Jeffrey Ling, Rebecca Roelofs, Alex Bewley, Chenxi Liu, Ashish Venugopal, David Weiss, Benjamin Sapp, Zhifeng Chen, and Jonathon Shlens. Scene Transformer: A unified architecture for predicting multiple agent trajectories. In *ICLR*, 2022.
- [52] Jonah Philion and Sanja Fidler. Lift, Splat, Shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d. In *ECCV*, 2020.
- [53] Charles R. Qi et al. PointNet: Deep learning on point sets for 3d classification and segmentation. In *CVPR*, 2017.
- [54] Alec Radford et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021.
- [55] Joseph Redmon et al. You only look once: Unified, real-time object detection. In *CVPR*, 2016.
- [56] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015.
- [57] SAE International. Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles, 2021. Standard J3016_202104.
- [58] John Schulman et al. Proximal policy optimization algorithms, 2017. arXiv:1707.06347.
- [59] Ramprasaath R. Selvaraju et al. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *ICCV*, 2017.
- [60] Shai Shalev-Shwartz, Shaked Shammah, and Amnon Shashua. On a formal model of safe and scalable self-driving cars, 2017. arXiv:1708.06374.
- [61] Pei Sun et al. Scalability in perception for autonomous driving: Waymo open dataset. In *CVPR*, 2020.
- [62] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *ICML*, 2019.
- [63] Zachary Teed and Jia Deng. RAFT: Recurrent all-pairs field transforms for optical flow. In *ECCV*, 2020.
- [64] Inc. Tesla. Tesla AI day 2022. Public technical presentation, 2022.

- [65] Sebastian Thrun, Michael Montemerlo, Hendrik Dahlkamp, et al. Stanley: The robot that won the DARPA grand challenge. *Journal of Field Robotics*, 23(9):661–692, 2006.
- [66] Chris Urmson et al. Autonomous driving in urban environments: Boss and the urban challenge. *Journal of Field Robotics*, 25(8):425–466, 2008.
- [67] Balakrishnan Varadarajan, Ahmed Hefny, Avikalp Srivastava, Khaled S. Refaat, Nigamaa Nayakanti, Andre Cornman, Kan Chen, Bertrand Douillard, Chi Pang Lam, Dragomir Anguelov, and Benjamin Sapp. MultiPath++: Efficient information fusion and trajectory aggregation for behavior prediction. In *ICRA*, 2022.
- [68] Ashish Vaswani et al. Attention is all you need. In *NeurIPS*, 2017.
- [69] Sourabh Vora et al. PointPainting: Sequential fusion for 3d object detection. In *CVPR*, 2020.
- [70] Benjamin Wilson et al. Argoverse 2: Next generation datasets for self-driving perception and forecasting. In *NeurIPS Datasets*, 2023.
- [71] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric. In *ICIP*, 2017.
- [72] Yuxin Wu and Kaiming He. Group normalization. In *ECCV*, 2018.
- [73] Tianwei Yin, Xingyi Zhou, and Philipp Krähenbühl. Center-based 3d object detection and tracking. In *CVPR*, 2021.
- [74] Fisher Yu et al. BDD100K: A diverse driving dataset for heterogeneous multitask learning. In *CVPR*, 2020.
- [75] Wei Zhan et al. INTERACTION dataset. In *arXiv:1910.03088*, 2019.
- [76] Ji Zhang and Sanjiv Singh. LOAM: Lidar odometry and mapping in real-time. In *RSS*, 2014.
- [77] Zhengyou Zhang. A flexible new technique for camera calibration. In *IEEE TPAMI*, 2000.
- [78] Yin Zhou and Oncel Tuzel. VoxelNet: End-to-end learning for point cloud based 3d object detection. In *CVPR*, 2018.
- [79] Xinge Zhu, Hui Zhou, Tai Wang, Fangzhou Hong, Yuexin Ma, Wei Li, Hongsheng Li, and Dahua Lin. Cylindrical and asymmetrical 3D convolution networks for LiDAR segmentation. In *CVPR*, 2021.